C++ Academy — Um Curso Completo de C++ Moderno

27 de agosto de 2026

Prepared by:
C++ Academy

Prepared for:
ASA Team

Sumário

Lista de figuras **xiii**

Lista de tabelas **xv**

I Introdução **2**
I.1 O que é o `cpp_academy` . . . 2
I.2 O panorama do repositório . . . 3
I.2.1 A anatomia de um subprojeto . . . 3
I.2.2 Os subprojetos, na ordem em que se apoiam . . . 4
I.2.3 Os subprojetos que este relatório passou a cobrir . . . 4
I.3 Objetivo e público-alvo deste relatório . . . 9
I.4 Como o relatório está organizado . . . 9
I.4.1 As dependências entre os capítulos . . . 10
I.5 Convenções deste relatório . . . 10
I.5.1 Os blocos de código . . . 11
I.5.2 As caixas de destaque . . . 12
I.5.3 Índice, glossário e referências cruzadas . . . 12
I.6 O ambiente descrito . . . 12
I.6.1 Compilando e rodando . . . 12
I.6.2 Todos os alvos são construídos . . . 13
I.6.3 Os limites do toolchain . . . 13
I.7 Os testes do repositório . . . 14
I.8 O que habilitar todos os alvos revelou . . . 16
I.8.1 Os quatro cabeçalhos que faltavam . . . 16
I.8.2 O `subq` que quebrava o alinhamento da pilha . . . 17
I.8.3 A promessa que o `rep movsb` não cumpre . . . 18
I.8.4 Dois bugs silenciosos numa função de imprimir traço . . . 19
I.8.5 Uma API que mudou de forma . . . 19
I.8.6 O demo que só funcionava na primeira execução . . . 20
I.8.7 A lição, enunciada . . . 21
I.9 Sanitizers . . . 21

II Fundamentos da linguagem **23**
II.1 O mapa da sintaxe . . . 23
II.1.1 Declarações e inicialização . . . 23
II.1.2 Controle de fluxo . . . 24
II.1.3 Funções . . . 25
II.1.4 Tipos definidos pelo usuário . . . 25
II.1.5 Classes: a lista de inicialização não é açúcar sintático . . . 26
II.1.6 Organização: *namespaces*, ADL e o pré-processador . . . 26
II.1.7 Expressões: precedência não é ordem de avaliação . . . 26
II.1.8 As armadilhas silenciosas, reunidas . . . 27
II.2 O sistema de tipos e as conversões explícitas . . . 27
II.2.1 Os quatro, em ordem de segurança . . . 27
II.2.2 Por que o *cast* à moda C é o pior dos cinco . . . 29
II.3 Semântica de valor: cópia, *move* e elisão . . . 29
II.3.1 Categorias de valor . . . 30
II.3.2 Rule of 0 / 3 / 5 . . . 30

II.3.3 Elisão de cópia . . . 32
II.3.4 *Perfect forwarding* . . . 32
II.3.5 *Move-only*, *ref-qualifiers* e *sink parameters* . . . 33
II.4 Sobrecarga de operadores . . . 33
II.4.1 Membro ou função livre . . . 34
II.4.2 `operator<=>`: seis operadores viram um . . . 35
II.4.3 Conversões e `explicit` . . . 36
II.4.4 `operator()`, `operator[]` e o *drill-down* de `operator->` . . . 36
II.4.5 Literais definidos pelo usuário . . . 37
II.5 Ponteiros inteligentes e RAII . . . 38
II.5.1 O problema, enunciado com precisão . . . 38
II.5.2 Os três ponteiros . . . 39
II.5.3 O custo, medido . . . 41
II.5.4 A medição que contrariou o conselho: `make_shared` e o `weak_ptr` . . . 42
II.5.5 A contagem é atômica — e não dá mais para desligá-la . . . 43
II.5.6 `enable_shared_from_this` e o construtor de *aliasing* . . . 43
II.5.7 Como passar um ponteiro inteligente para uma função . . . 44
II.5.8 Três padrões que dependem de `unique_ptr` . . . 45
II.6 Arquivos e *streams*: RAII com estado . . . 45
II.6.1 O *stream* como objeto RAII . . . 46
II.6.2 Os quatro bits de estado . . . 46
II.6.3 Bytes não são caracteres . . . 47
II.6.4 Texto, binário e o cursor . . . 48
II.6.5 O que os *streams* custam, medido . . . 48
II.6.6 Redirecionamento: `rdbuf` e o que ele não captura . . . 49
II.6.7 O *Logger*, e a lição de idempotência . . . 51
II.6.8 `std::filesystem`: a API dupla e a corrida . . . 51
II.7 O modelo de objeto . . . 52
II.7.1 Herança . . . 52
II.7.2 Funções virtuais e a *vtable* . . . 53
II.7.3 *Object slicing* . . . 54
II.7.4 Herança múltipla e o diamante . . . 55
II.7.5 RTTI e o seu custo . . . 55
II.8 Tratamento de erro . . . 56
II.8.1 O mecanismo, e por que ele é o mesmo assunto que RAII . . . 56
II.8.2 As quatro garantias . . . 57
II.8.3 A hierarquia padrão . . . 58
II.8.4 Erros como valores . . . 58
II.8.5 Escolhendo a forma . . . 59
II.9 Síntese: os erros que o compilador não denuncia . . . 60

III A biblioteca padrão 62
III.1 Containers, iteradores e algoritmos . . . 62
III.1.1 Os containers e o que custa cada um . . . 62
III.1.2 Invalidação de iteradores . . . 63
III.1.3 `emplace` contra `push`, medido . . . 63
III.1.4 Iteradores: categorias e sentinelas . . . 64
III.1.5 `<algorithm>`: peça a menor garantia que resolve . . . 65
III.1.6 Um container que satisfaz o contrato . . . 66
III.1.7 Quatro armadilhas que compilam . . . 66

III.2 Ranges: a mesma biblioteca, repensada . . . 67
III.2.1 As duas propriedades que definem uma *view* . . . 67
III.2.2 O `operator|` não é mágica . . . 68
III.2.3 Tempo de vida: `ref_view`, `owning_view` e o *dangling* . . . 68
III.2.4 Algoritmos com projeção . . . 69
III.2.5 Escrevendo os seus . . . 69
III.3 Entrada, saída e sistema de arquivos . . . 70
III.3.1 A hierarquia, e a diferença que importa . . . 70
III.3.2 *String streams* e `rdbuf` . . . 70
III.3.3 `std::filesystem` . . . 71
III.3.4 Um alvo que não é construído é um alvo que apodrece . . . 71
III.4 Tipos de vocabulário . . . 72
III.4.1 `std::optional` . . . 72
III.4.2 `std::variant` e o *overload idiom* . . . 73
III.4.3 `std::any` e a pergunta que quase sempre o elimina . . . 74
III.4.4 `tuple`, `pair` e os *structured bindings* . . . 74
III.4.5 `std::span` . . . 75
III.5 De onde vem a memória . . . 76
III.5.1 `new` e `delete`, por dentro . . . 76
III.5.2 *Pool*: blocos de tamanho fixo . . . 76
III.5.3 *Arena*: aloca tudo, libera tudo . . . 77
III.5.4 `std::pmr`: trocar o alocador sem trocar o tipo . . . 77
III.6 Co-rotinas: funções que sabem parar no meio . . . 78
III.6.1 A definição, e os três operadores . . . 79
III.6.2 O que o `<coroutine>` contém — e o que não contém . . . 79
III.6.3 O `promise_type`: o contrato que você escreve . . . 80
III.6.4 Um gerador completo . . . 81
III.6.5 Quanto custa o *frame*, medido . . . 81
III.6.6 O `coroutine_handle`: um ponteiro cru com dono . . . 82
III.6.7 O protocolo *Awaitable* . . . 83
III.6.8 `Task<T>`: esperar sem bloquear . . . 84
III.6.9 Duas armadilhas de tempo de vida que ninguém avisa . . . 85
III.6.10 Asio: onde a co-rotina paga o próprio custo . . . 86
III.7 Síntese . . . 87

IV Programação genérica **88**
IV.1 Templates . . . 88
IV.1.1 As formas básicas . . . 88
IV.1.2 Variádicos e *fold expressions* . . . 89
IV.1.3 SFINAE, e por que ele existiu . . . 89
IV.1.4 *Type traits* e `if constexpr` . . . 90
IV.1.5 CRTP: polimorfismo sem *vtable* . . . 90
IV.2 Concepts . . . 91
IV.2.1 As quatro formas de escrever a mesma restrição . . . 92
IV.2.2 As quatro espécies de *requirement* . . . 92
IV.2.3 *Subsumption* . . . 93
IV.2.4 Onde *concepts* rendem mais . . . 94
IV.3 Computação em tempo de compilação . . . 94
IV.3.1 `constexpr` é permissão; `consteval` é obrigação . . . 94
IV.3.2 `static_assert`: onde não há substituto . . . 96

IV.3.3 NTTP: o valor entra no tipo . . . 96
IV.3.4 Gerando dados na compilação . . . 97
IV.4 Lambdas e objetos chamáveis . . . 97
IV.4.1 Capturas . . . 98
IV.4.2 Genéricas . . . 98
IV.4.3 `std::function`: *type erasure* e o seu preço . . . 99
IV.4.4 `std::invoke` e ponteiros para membro . . . 100
IV.4.5 Cinco padrões que só existem por causa das *lambdas* . . . 100
IV.5 Síntese . . . 101

V Design e arquitetura 102
V.1 Os cinco princípios SOLID . . . 102
V.1.1 SRP — responsabilidade única . . . 102
V.1.2 OCP — aberto para extensão, fechado para modificação . . . 103
V.1.3 LSP — substituição de Liskov . . . 104
V.1.4 ISP — segregação de interfaces . . . 106
V.1.5 DIP — inversão de dependência . . . 107
V.2 Padrões de projeto . . . 108
V.2.1 O eixo que decide quase tudo: o *expression problem* . . . 108
V.2.2 Os cinco criacionais . . . 110
V.2.3 Os sete estruturais . . . 112
V.2.4 Os onze comportamentais . . . 115
V.2.5 As confusões que sobram . . . 118
V.2.6 A versão moderna, padrão a padrão . . . 118
V.2.7 *Interpreter*: onde ele ganha e onde perde . . . 120
V.3 Entity Component System . . . 120
V.3.1 O problema . . . 120
V.3.2 Os três pilares . . . 121
V.3.3 *Type erasure* para guardar tipos heterogêneos . . . 121
V.3.4 A *query*: onde as peças se encaixam . . . 122
V.3.5 O que se ganha e o que se perde . . . 122
V.4 Plugins: DIP até o limite do sistema operacional . . . 123
V.4.1 O que é, exatamente, carregar código em tempo de execução . . . 123
V.4.2 Os símbolos exportados . . . 123
V.4.3 `dlopen` e a ordem certa de checar erro . . . 124
V.4.4 O *deleter* customizado, e por que ele é crítico . . . 125
V.4.5 O registro, e a ordem em que tudo morre . . . 125
V.4.6 Onde estão os *plugins*? A resolução de caminho . . . 126
V.4.7 O ciclo completo . . . 127
V.4.8 A lição que só apareceu quando os alvos foram construídos . . . 127
V.5 Síntese . . . 128

VI Concorrência 129
VI.1 *Threads* e primitivos de sincronização . . . 129
VI.1.1 *Threads*: a obrigação que ninguém lembra . . . 129
VI.1.2 *Mutex*: por que `contador++` não basta . . . 130
VI.1.3 `condition_variable`: esperar sem girar . . . 130
VI.1.4 `future`, `promise` e `async` . . . 131
VI.1.5 Atômicos e ordem de memória . . . 131
VI.1.6 *Thread pool*: onde tudo se junta . . . 132

VI.2 Concorrência moderna: o que o C++20 acrescentou . . . 133
VI.2.1 `jthread`: o destrutor que não mata o programa . . . 133
VI.2.2 `latch`, `barrier` e `counting_semaphore` . . . 135
VI.2.3 O modelo de memória . . . 137
VI.2.4 CAS: a primitiva que constrói todas as outras . . . 139
VI.2.5 *Lock-free* na prática . . . 141
VI.3 Co-rotinas . . . 143
VI.3.1 *Generator*: preguiça com estado . . . 144
VI.3.2 O `promise_type`: o contrato que a linguagem exige . . . 144
VI.3.3 *Task* e o protocolo *Awaitable* . . . 145
VI.3.4 O que o compilador gera . . . 145
VI.3.5 Onde as co-rotinas realmente pagam: I/O . . . 145
VI.4 Paralelismo aplicado: dois Mandelbrots . . . 146
VI.4.1 O laço interno, e por que ele não usa `std::complex` . . . 146
VI.4.2 Com `std::thread`: paralelismo por *tiles* . . . 147
VI.4.3 Com OpenMP: uma linha . . . 148
VI.5 Síntese . . . 149

VII Ecossistema: rede, dados e mensageria 151
VII.1 Antes de tudo: estes alvos são construídos . . . 151
VII.2 Sockets com Asio . . . 152
VII.2.1 O vocabulário mínimo, definido antes de ser usado . . . 152
VII.2.2 O `io_context` é o *event loop* . . . 153
VII.2.3 TCP síncrono: um servidor de eco em trinta linhas . . . 153
VII.2.4 As duas APIs de erro . . . 154
VII.2.5 TCP assíncrono e o problema do tempo de vida . . . 155
VII.2.6 `post` e `strand`: concorrência sem *mutex* . . . 156
VII.2.7 TCP e UDP . . . 157
VII.2.8 Temporizadores . . . 158
VII.2.9 HTTP cru: o protocolo é texto . . . 158
VII.2.10 A armadilha do macro que o Conan já definiu . . . 159
VII.2.11 As co-rotinas, que é onde tudo isso converge . . . 160
VII.3 Cliente HTTP e JSON . . . 160
VII.3.1 A interface, e o que ela decide . . . 160
VII.3.2 Envolver uma API C, com RAII em vez de disciplina . . . 161
VII.3.3 Dois bugs num separador de tela . . . 163
VII.3.4 Mapeamento automático entre JSON e *structs* . . . 163
VII.4 Banco de dados e o padrão *Repository* . . . 164
VII.4.1 O que o SOCI é, e o vocabulário que ele traz . . . 164
VII.4.2 A pilha de abstração . . . 165
VII.4.3 Por que o backend PostgreSQL é opcional . . . 166
VII.4.4 *Prepared statements* não são otimização . . . 166
VII.4.5 Transações como RAII de negócio . . . 167
VII.4.6 Um exemplo que só funciona na primeira execução é um exemplo quebrado . 168
VII.5 Mensageria com AMQP . . . 169
VII.5.1 O vocabulário do AMQP . . . 170
VII.5.2 Exchange, *binding* e fila . . . 170
VII.5.3 Duas famílias de *callback*, e o erro de compilação que as separa . . . 171
VII.5.4 O ciclo de vida em seis estágios . . . 171
VII.5.5 Tudo é *callback* . . . 172

VII.5.6 ACK: a diferença entre entregue e processado . . . 173
VII.5.7 A porta, que quase nunca é a que se espera . . . 173
VII.6 Nuvem: AWS SDK e o modelo `Outcome` . . . 174
VII.6.1 O bug que provou o argumento do capítulo . . . 174
VII.6.2 O ciclo de vida global, e o bloco que parece decorativo . . . 174
VII.6.3 `Outcome<R, E>`: erro como valor, de novo . . . 175
VII.6.4 A medição que contrariou a expectativa . . . 175
VII.7 Síntese: cinco bibliotecas, cinco convenções de erro . . . 176

VIII Gráficos, física e simulação 178
VIII.1 Física sem renderização . . . 178
VIII.1.1 Vocabulário: o que é cada peça . . . 179
VIII.1.2 O *pipeline* de um quadro . . . 179
VIII.1.3 Estático e dinâmico: a massa é a chave . . . 180
VIII.1.4 Por que o passo de tempo é fixo por dentro . . . 180
VIII.1.5 O que a execução realmente imprime . . . 180
VIII.1.6 Duas leituras diferentes da mesma posição . . . 181
VIII.2 Renderizando a física . . . 182
VIII.2.1 Problema 1: um motor 3D fazendo física 2D . . . 183
VIII.2.2 Problema 2: dois sistemas de coordenadas incompatíveis . . . 183
VIII.2.3 Problema 3: o quadro lento . . . 184
VIII.2.4 Problema 4: a física e o desenho discordam em 392 pixels . . . 184
VIII.2.5 RAII sobre uma API que não o oferece — e o que ele não conserta . . . 185
VIII.2.6 Da orientação do Bullet ao ângulo do SFML . . . 187
VIII.3 Constraints, *ragdoll* e corpos deformáveis . . . 187
VIII.3.1 Grau de liberdade: o conceito que organiza tudo . . . 187
VIII.3.2 O *ragdoll* medido, e o README que mente . . . 188
VIII.3.3 Duas linhas fáceis de esquecer . . . 189
VIII.3.4 A corda é um *soft body*, e o mundo muda por causa dela . . . 189
VIII.4 Qt e SFML na mesma janela . . . 190
VIII.4.1 O truque: entregar o *handle* nativo ao SFML . . . 191
VIII.4.2 `QTimer` como *game loop* . . . 191
VIII.4.3 Uma medição incômoda: 60 quadros por segundo não são 60 . . . 192
VIII.4.4 `qt_sfml`: a física escrita à mão, e o seu defeito clássico . . . 193
VIII.4.5 `qt_sfml2`: um renderizador 3D escrito à mão sobre um canvas 2D . . . 193
VIII.5 Álgebra linear numérica . . . 195
VIII.5.1 LU: fatore uma vez, resolva muitas . . . 196
VIII.5.2 QR: regressão linear é mínimos quadrados . . . 197
VIII.5.3 SVD: a mais cara e a mais informativa . . . 197
VIII.5.4 Autovalores, PCA e a escolha do *solver* . . . 199
VIII.6 Simulação de voo com autopiloto . . . 200
VIII.6.1 A arquitetura, e o padrão que ela repete . . . 200
VIII.6.2 PID: três termos, três papéis . . . 200
VIII.6.3 O laço de missão . . . 201
VIII.7 Síntese . . . 201

IX Baixo nível e hardware 203
IX.1 Assembly inline . . . 203
IX.1.1 A forma de um `asm` estendido . . . 203
IX.1.2 `volatile`: quando o compilador apagaria o bloco . . . 204
IX.1.3 Instruções sem equivalente em C++ . . . 204

IX.1.4 Legibilidade: operandos nomeados . . . 205
IX.2 Assembly em arquivo separado . . . 206
IX.2.1 A fronteira: `extern "C"` e a ABI . . . 206
IX.2.2 Duas instruções que valem a leitura . . . 207
IX.2.3 O *build* não precisa de nada especial . . . 208
IX.2.4 O que a execução revela — e que a compilação não revelaria . . . 208
IX.3 O curso de assembly . . . 208
IX.3.1 Três ideias que atravessam o curso . . . 209
IX.3.2 Módulo 1 — o que é, afinal, um registrador . . . 210
IX.3.3 Módulo 2 — endereçar memória . . . 211
IX.3.4 Módulo 3 — aritmética e o registrador RFLAGS . . . 212
IX.3.5 Módulo 4 — operações bit a bit . . . 213
IX.3.6 Módulo 5 — controle de fluxo . . . 213
IX.3.7 Módulo 6 — a pilha, e onde moram as variáveis locais . . . 214
IX.3.8 Módulo 7 — a ABI, e o bug que explica por que ela existe . . . 215
IX.3.9 Módulos 8 e 9 — SIMD: XMM, YMM e o custo de misturá-los . . . 216
IX.3.10 Módulo 10 — operações de *string*, e o segundo bug de ABI . . . 217
IX.3.11 Módulo 11 — *syscalls*: a fronteira com o *kernel* . . . 219
IX.3.12 Módulo 12 — a FPU x87, e por que ela não deve ser usada . . . 219
IX.3.13 Módulo 13 — otimização, e a medição que contrariou a expectativa . . . 220
IX.3.14 Módulo 14 — ler o que o compilador escreveu . . . 221
IX.3.15 Módulo 15 — o *capstone*, e o defeito que só aparece rodando . . . 222
IX.4 Um alvo que não é construído é um alvo que apodrece . . . 223
IX.5 Síntese . . . 224

X Interoperação e visão computacional 225
X.1 Python importando C++ . . . 225
X.1.1 pybind11: a lógica separada do *binding* . . . 225
X.1.2 A API C, e o que ela obriga a escrever . . . 225
X.1.3 O GIL, e por que ele obriga a pensar . . . 226
X.2 C++ hospedando Python . . . 227
X.2.1 O ciclo de vida, e o *guard* que o garante . . . 227
X.2.2 A contagem de referências, e as duas espécies de ponteiro . . . 227
X.2.3 As três formas de conversar . . . 228
X.3 Análise léxica e sintática . . . 228
X.3.1 Duas fases, duas ferramentas . . . 228
X.3.2 Precedência declarada, não codificada . . . 228
X.3.3 A fronteira com o C++ . . . 229
X.4 Inferência de rede neural em C++ . . . 229
X.4.1 O trabalho está nas pontas . . . 229
X.4.2 Medir latência com percentis, não com média . . . 230
X.4.3 A interface, e a decisão de projeto que ela revela . . . 230
X.5 Síntese . . . 231

XI Ferramentas do repositório 232
XI.1 O estado do *build*, em números . . . 232
XI.2 Meson: o sistema de *build* . . . 232
XI.2.1 Três programas, e a divisão que explica tudo . . . 233
XI.2.2 Os dois tempos . . . 233
XI.2.3 Anatomia do `meson.build` da raiz . . . 233
XI.2.4 `declare_dependency()`: o recurso que separa um *build* limpo . . . 234

XI.2.5 Dependências, e o par que torna uma delas opcional . . . 234
XI.2.6 Do *build* para dentro do C++ . . . 235
XI.2.7 Testes, e a dívida que foi quitada . . . 235
XI.2.8 As armadilhas, e o diagnóstico de cada uma . . . 236
XI.3 Um alvo que não é construído é um alvo que apodrece . . . 237
XI.3.1 Quatro `#include` que faltavam . . . 237
XI.3.2 Dois bugs de ABI, que só aparecem ao *executar* . . . 237
XI.3.3 Três bugs de biblioteca externa . . . 238
XI.4 CMake: alvos, escopos e o *build* que o ecossistema fala . . . 239
XI.4.1 Antes de tudo: o que o CMake é . . . 239
XI.4.2 Alvos: a única ideia do "CMake moderno" . . . 240
XI.4.3 Os três escopos, e a pergunta que decide entre eles . . . 240
XI.4.4 *Generator expressions*: o `if` que roda tarde demais . . . 241
XI.4.5 `find_package`: os dois modos . . . 242
XI.4.6 A detecção graciosa, e a máquina de quem não tem GPU . . . 242
XI.4.7 `FetchContent` contra `find_package` . . . 243
XI.4.8 Conan e vcpkg: eles não substituem o CMake, alimentam-no . . . 244
XI.4.9 Código gerado durante o *build* . . . 244
XI.4.10 OpenMP, MPI e CUDA: a mesma conta, quatro caminhos . . . 246
XI.4.11 CTest, e a diferença entre 1 teste e 11 . . . 248
XI.4.12 CMake ou Meson? A comparação honesta . . . 249
XI.5 Flex e Bison: quando o fonte de entrada não é C++ . . . 250
XI.5.1 Duas ferramentas, três saídas, um executável . . . 250
XI.6 Doxygen: documentação a partir do código . . . 251
XI.6.1 Os comandos que descrevem o contrato . . . 251
XI.6.2 A configuração que muda o resultado . . . 252
XI.7 Sandbox . . . 252
XI.8 Síntese: a receita completa para um subprojeto novo . . . 253

XII Tipos de vocabulário e computação em tempo de compilação 254
XII.1 O vocabulário compartilhado . . . 254
XII.1.1 `std::optional`: a ausência dentro do tipo . . . 255
XII.1.2 `std::variant`: a soma de tipos que faltava . . . 257
XII.1.3 `std::any` e o apagamento de tipo . . . 259
XII.1.4 `tuple`, `pair` e os *structured bindings* . . . 260
XII.1.5 `std::span`: a janela sobre memória alheia . . . 261
XII.2 Computação em tempo de compilação . . . 263
XII.2.1 `constexpr` é permissão, não garantia . . . 263
XII.2.2 `consteval` e `constinit`: as garantias . . . 264
XII.2.3 `static_assert` e `type_traits` . . . 266
XII.2.4 NTTP: quando o valor entra no tipo . . . 268
XII.2.5 Tabelas geradas na compilação . . . 269
XII.3 O inventário medido do C++23 . . . 271
XII.3.1 O inventário, e as seis entradas corrigidas . . . 271
XII.3.2 O que a linguagem ganhou . . . 272
XII.3.3 O que a biblioteca ganhou . . . 273
XII.3.4 O que falta, e o substituto de cada coisa . . . 274
XII.4 Módulos nomeados . . . 275
XII.4.1 Dois defeitos do GCC 13, encontrados escrevendo o demo . . . 276
XII.4.2 As armadilhas do *toolchain* . . . 277

XII.4.3 Por que este subprojeto fica fora do Meson . . . 278
XII.5 Síntese . . . 279

XIII Texto, tempo e bits **280**
XIII.1 Texto . . . 280
XIII.1.1 `std::string` por dentro: a SSO, medida . . . 281
XIII.1.2 `capacity`, crescimento e `reserve` . . . 281
XIII.1.3 Concatenação: por que `+` custa mais . . . 282
XIII.1.4 O que `std::string` não sabe . . . 282
XIII.1.5 `std::string_view`: a janela que não possui . . . 283
XIII.1.6 `std::format`: formatação verificada em compilação . . . 285
XIII.1.7 `<charconv>`: texto e número sem alocar e sem locale . . . 287
XIII.1.8 `std::regex`: o custo real . . . 288
XIII.2 Tempo . . . 290
XIII.2.1 `duration`: a unidade vive no tipo . . . 290
XIII.2.2 Conversões: implícitas quando são exatas . . . 291
XIII.2.3 Aritmética e unidades derivadas . . . 291
XIII.2.4 Os relógios, e por que escolher o errado é um bug . . . 292
XIII.2.5 `time_point`: instantes, e a álgebra que faz sentido . . . 292
XIII.2.6 O calendário do C++20 . . . 293
XIII.2.7 Como cronometrar sem se enganar . . . 294
XIII.3 Bits . . . 296
XIII.3.1 As quatro operações com máscara . . . 296
XIII.3.2 A promoção inteira, que morde . . . 297
XIII.3.3 *Shift* fora de faixa . . . 297
XIII.3.4 `»` em *signed* contra *unsigned* . . . 297
XIII.3.5 Extrair e inserir campos, de forma portável . . . 298
XIII.3.6 `<bit>`: as operações que o hardware já sabia fazer . . . 299
XIII.3.7 `std::bitset`: bits com nome . . . 300
XIII.3.8 Campos de bits nativos, e por que evitá-los . . . 301
XIII.3.9 *Endianness*, `bit_cast` e serialização . . . 302
XIII.4 O IO de arquivo com *streams* . . . 303
XIII.4.1 Os quatro objetos globais, e a diferença que importa . . . 304
XIII.4.2 Manipuladores: poderosos e pegajosos . . . 304
XIII.4.3 Arquivos: texto, binário e posicionamento . . . 304
XIII.4.4 *String streams*: a mesma interface, em memória . . . 305
XIII.4.5 `rdbuf`: o ponto de extensão que torna tudo componível . . . 305
XIII.4.6 Um *logger*, e o que ele reúne . . . 306
XIII.4.7 `std::filesystem`, em uma linha por conceito . . . 307
XIII.5 Síntese . . . 307

XIV O modelo de objeto por dentro **309**
XIV.1 O layout de um objeto . . . 309
XIV.1.1 Alinhamento: a exigência do hardware . . . 309
XIV.1.2 A ordem dos membros muda o tamanho . . . 310
XIV.1.3 Por que existe *padding* no fim . . . 310
XIV.1.4 `memcmp` e o lixo do *padding* . . . 311
XIV.1.5 `struct` vazia: por que 1 e não 0 . . . 311
XIV.1.6 `alignas`: mandando no alinhamento . . . 312
XIV.1.7 As quatro categorias de tipo . . . 313
XIV.1.8 EBO e `[[no_unique_address]]` . . . 314

XIV.1.9 O vptr, e o que `virtual` custa em bytes . . . 315
XIV.1.10 Não adivinhe: inspecione . . . 317
XIV.2 Apagamento de tipo . . . 317
XIV.2.1 O problema, e o que a herança cobra por resolvê-lo . . . 317
XIV.2.2 `std::function` do zero: Conceito, Modelo, Wrapper . . . 318
XIV.2.3 SBO: medido, não presumido . . . 319
XIV.2.4 Vtable manual, sem herança nenhuma . . . 320
XIV.2.5 O apagamento que já vem no padrão . . . 321
XIV.2.6 A medição que decide . . . 322
XIV.2.7 O problema da expressão . . . 322
XIV.2.8 Polimorfismo com semântica de valor . . . 323
XIV.3 Da unidade de tradução ao binário . . . 324
XIV.3.1 O que é uma unidade de tradução . . . 324
XIV.3.2 Quanto custa um `#include` . . . 325
XIV.3.3 A ODR, medida com duas TUs reais . . . 326
XIV.3.4 Os três tipos de *linkage* . . . 327
XIV.3.5 `static`: uma palavra, quatro significados . . . 328
XIV.3.6 *Name mangling* e `extern "C"` . . . 329
XIV.3.7 ABI: o contrato binário . . . 330
XIV.3.8 Pimpl: o *firewall* de ABI . . . 331
XIV.4 Síntese . . . 332

XV Corretude e desempenho **334**
XV.1 Testes automatizados . . . 334
XV.1.1 A pirâmide, e por que ela é uma pirâmide . . . 334
XV.1.2 F.I.R.S.T. e o que testar primeiro . . . 335
XV.1.3 O sistema sob teste . . . 336
XV.1.4 Anatomia de um teste . . . 336
XV.1.5 `EXPECT` contra `ASSERT`, e o catálogo . . . 337
XV.1.6 Ponto flutuante . . . 338
XV.1.7 Fixtures e o ciclo de vida . . . 338
XV.1.8 As quatro variantes, e as FRONTEIRAS . . . 339
XV.1.9 Os cinco dublês e a injeção de dependência . . . 340
XV.1.10 gmock: comportamento e interação . . . 341
XV.1.11 *Death tests*: verificar que o programa morre . . . 342
XV.1.12 Cobertura, e o que 100% não garante . . . 343
XV.2 Comportamento indefinido . . . 344
XV.2.1 As quatro categorias, e por que UB existe . . . 344
XV.2.2 Como o UB licencia o otimizador . . . 345
XV.2.3 Os *sanitizers*, medidos . . . 346
XV.2.4 *Strict aliasing* e `bit_cast` . . . 348
XV.2.5 As cinco camadas de defesa . . . 349
XV.3 Desempenho: medir, e não supor . . . 350
XV.3.1 A hierarquia de memória e a linha de cache . . . 350
XV.3.2 Linha contra coluna, `vector` contra `list`, e o prefetcher . . . 351
XV.3.3 AoS contra SoA . . . 352
XV.3.4 *False sharing*: 11,7× em código correto . . . 353
XV.3.5 Predição de desvio, e o experimento que não reproduz mais . . . 354
XV.3.6 Como medir sem se enganar . . . 355
XV.4 Síntese . . . 357

XVI Apêndice: trilhas, cobertura e o que falta **358**
XVI.1 Atalhos por objetivo . . . 358
XVI.2 Onde cada conceito aparece . . . 359
XVI.3 Estado de cobertura da linguagem . . . 359
XVI.3.1 Os padrões GoF: o catálogo fechou . . . 361
XVI.4 Os limites do *toolchain* . . . 361
XVI.4.1 Dívida estrutural: as quatro pendências foram quitadas . . . 361
XVI.5 Um alvo que não é construído é um alvo que apodrece . . . 362
XVI.5.1 Os quatro cabeçalhos que não se bastavam . . . 363
XVI.5.2 O alinhamento de 16 bytes, e o crash que aparece no lugar errado . . . 364
XVI.5.3 A promessa que o `rep movsb` não cumpre . . . 364
XVI.5.4 Três bytes que não cabem num `char` . . . 365
XVI.5.5 A API que mudou de forma embaixo do código . . . 366
XVI.5.6 O demo que só funcionava na primeira execução . . . 367
XVI.5.7 A lição, enunciada . . . 368
XVI.6 O que os subprojetos novos mediram . . . 368
XVI.6.1 O custo de um `#include`, medido . . . 369
XVI.6.2 O `static` em cabeçalho, medido com duas TUs . . . 369
XVI.6.3 Os sanitizers, medidos em vez de citados . . . 370
XVI.6.4 Quando a medição contrariou a expectativa . . . 371
XVI.6.5 Três números de texto que mudam código . . . 372
XVI.7 Encerramento . . . 373

Referências **374**

Glossário **375**

Índice remissivo **378**

Lista de figuras

1 As dependências conceituais que a trilha de estudo respeita. A seta cheia indica pré-requisito real (estudar fora de ordem custa caro); a tracejada, apoio conveniente. `value_semantics` é a raiz de quase tudo: é ele que define quando o compilador copia e quando move. 11

2 A árvore de decisão de `src/casts/`. Nenhum galho termina em *cast* à moda C, e apenas um termina em `dynamic_cast` — que, ao aparecer numa cadeia de `if`, quase sempre indica uma função virtual faltando. 29

3 As categorias de valor. O *xvalue* é o único que pertence aos dois ramos: tem identidade *e* pode ser movido — é exatamente o que `std::move` produz. 30

4 A hierarquia de *streams*. A distinção prática entre `cerr` e `clog` é o *buffer*: os dois escrevem em *stderr*, mas só o primeiro garante que a mensagem saia antes de um *crash*. 70

5 A inversão. A seta do MySQL passou a apontar *para dentro*, na direção da abstração que a própria política definiu — e é a abstração morar na camada de negócio, e não numa camada neutra, que caracteriza a inversão. 107

6 A seta de `SociConnection` aponta *para cima*: é a inversão de dependência da Seção V.1. Trocar de banco não muda uma linha de `ProductRepository` nem de `main.cpp` — muda só o DSN passado a `db::connect()`. 165

7 O *pipeline* do Bullet. A *broadphase* é barata e aproximada — descarta a maioria dos pares comparando caixas alinhadas aos eixos; a *narrowphase* é cara e exata, e só roda no que sobrou. A separação é a razão de uma cena com milhares de corpos ser viável. 179

8 O laço de controle. `FlightState` e `ControlInputs` são *aggregates* simples — os tipos de vocabulário desta aplicação — e é por eles, e não por chamadas à API do JSBSim, que as três peças conversam. 200

9 A separação clássica. O analisador léxico não conhece gramática — só reconhece padrões e emite *tokens*; o analisador sintático não conhece caracteres — só consome *tokens*. É a mesma divisão de responsabilidade que o SRP (Seção V.1) recomendaria. 228

10 O *pipeline*. A caixa do meio é uma chamada de biblioteca; as duas caixas em volta são onde estão os erros — e onde está boa parte do tempo de execução. 229

11 As três peças do apagamento de tipo. Só o **Modelo** conhece o `T`: ele herda do Conceito e some atrás do ponteiro. Quem escreveu o `T` nunca ouviu falar de nenhuma das três. 318

12 O *pipeline* de `src/linkage/`. O compilador nunca vê duas TUs ao mesmo tempo; quem descobre que faltou ou sobrou uma definição é o *linker* — e por isso os erros dele são de outra natureza. 325

13 A pirâmide de testes. A forma é consequência do custo por teste em três eixos: **tempo** (microssegundos contra minutos), **estabilidade** (mais peças, mais motivos de falhar sem haver bug) e **diagnóstico** (um unitário aponta a linha; um E2E aponta "algo, em algum lugar"). 335

14 A maior assimetria do hardware moderno. Em 200 ciclos o processador executaria centenas de instruções — por isso, em código que percorre dados, "quantas operações" quase nunca é a pergunta certa, e "quantos *cache miss*" é. 350

Lista de tabelas

1 Os subprojetos de `src/`: os 57 declarados em `src/meson.build`, na ordem em que aquele arquivo os declara, mais `modules` e `cmake`, que têm sistema de *build* próprio. Os marcados com ⋆ foram acrescentados depois da primeira edição deste relatório e são apresentados na Seção I.2.3. 4

2 O que se presumia sobre o GCC 13.3 e o que a medição por feature-test macro mostrou. Fonte: `src/cpp23/`. 6

3 Mapa entre as etapas da trilha de estudo do repositório, os capítulos deste relatório e os subprojetos de `src/` que cada capítulo desenvolve. 10

4 O ambiente de referência do repositório. 12

5 Recursos de C++23 e a disponibilidade medida no compilador de referência. A coluna da direita é a macro que `src/cpp23/` consulta para produzir esta tabela. 14

6 Onde os testes moram, o que cada conjunto exercita e como se roda. 15

7 As três combinações de `const` com ponteiro. 24

8 *Linkage* das construções que mais aparecem em cabeçalhos. 24

9 Como passar um parâmetro, e o que cada forma comunica. 25

10 `enum` contra `enum class`. 25

11 `union` contra `std::variant`. 26

12 O que usar no lugar de cada macro. 26

13 Construções que não significam o que parecem — e se o compilador avisa. Cada uma tem demonstração executável em `src/syntax/`. 27

14 Os quatro *casts* do C++: o que fazem e o que verificam. 28

15 Cinco erros de semântica de valor. Nenhum é apontado pelo compilador com os avisos padrão. 29

16 A tabela de assinaturas, agora completa. 34

17 Qual forma usar para cada família de operadores. 34

18 As três categorias de ordenação e o que cada uma promete. 35

19 As quatro formas de “algo chamável”. 37

20 Os quatro modos de errar a posse, e por que nenhum deles aparece no teste. 38

21 Qual ponteiro para qual relação de posse. 39

22 `release()` contra `reset()` — a diferença de uma chamada de destrutor. 39

23 `sizeof` medido com `g++ -std=c++20` e libstdc++ 13. 41

24 Alocações e bytes por criação, medidos com `operator new` instrumentado sobre um objeto de 64 bytes. 42

25 O que cada assinatura comunica (`demo_patterns.hpp`, "regras de ouro"). 45

26 Os bits de estado de um *stream*, e o que cada um significa. 46

27 200 000 linhas escritas, lidas ou concatenadas; média de duas execuções, `-O2`, disco local. 49

28 As três formas de herança. 52

29 As quatro garantias de segurança em face de exceções. 57

30 As quatro operações monádicas de `std::expected`. 59

31 As cinco formas de reportar erro, comparadas. 60

32 Os erros silenciosos do capítulo, e a correção de cada um. 61

33 Complexidade e disposição em memória dos containers principais. 62

34 O que cada operação invalida. 63

35 As quatro formas de pôr um par no `map`. 64

36 Os algoritmos de ordenação e o que cada um promete. 65

37 O que o *pipeline* faz com a origem. 68

38 Cada tipo de vocabulário responde a uma pergunta diferente. 72

39 Acessando um `optional`. 72

40 Acessando um `variant`. Só `visit` verifica exaustividade. 73

41 `variant` favorece operações novas; herança favorece tipos novos. 74

42 `span<const T>` contra `const span<T>` — o que cada um congela. 75

43 O que o *pool* troca pelo que perde em generalidade. 77

44 Escolhendo a estratégia de alocação. 78

45 Os três operadores que tornam uma função uma co-rotina. 79

46 Tudo o que o `<coroutine>` do C++20 fornece. 79

47 Os métodos do `promise_type` e quando o compilador chama cada um. 80

48 A interface do `std::coroutine_handle`. 82

49 Os três métodos do protocolo *Awaitable*. 83

50 Os três retornos possíveis de `await_suspend`. 84

51 Qual primitiva usar, e por quê. 87

52 As formas de *template* da linguagem. 88

53 As formas de *fold expression*. 89

54 CRTP e `virtual` resolvem problemas parecidos com trocas opostas. 91

55 SFINAE e *concepts*, ponto a ponto. 94

56 As três palavras, e o que cada uma exige. 95

57 Para poucas dezenas de entradas fixas, o array ordenado ganha nas três colunas. 97

58 As três formas de receber ou guardar um chamável. 99

59 Recursão em *lambda*: as três formas. 100

60 Sinais de violação de SRP, e o que cada um costuma indicar. 103

61 O que muda ao acrescentar um caso novo. Código que não é tocado não pode regredir. 103

62 Quatro formas de injetar variação. "Homogêneo" significa poder guardar variantes no mesmo `vector`. 104

63 O derivado pode ser mais *generoso*, nunca mais *exigente* — a mesma contravariância dos tipos de função. 105

64 O que fazer num `override` que não se sabe implementar — e por que nenhuma das opções serve. 106

65 A regra da dependência: cada círculo só inclui os de dentro. 108

66 Os 24 padrões implementados em `src/design-of-pattern/`, e o que o executável de cada um demonstra. A coluna da direita reproduz o que o `README.md` do subprojeto registra como objetivo de cada demo. 109

67 O *expression problem*, como o `README.md` de `design-of-pattern` o formula: qual eixo do seu problema cresce? 110

68 As duas opções que o próprio GoF apresenta para o *Composite*, e por que o subprojeto escolheu a segunda. 113

69 Os três tipos de passo do *Template Method*, como o demo `dist/bin/template_method` os classifica. 116

70 *Visitor* clássico contra `variant` + `visit`, como o demo `dist/bin/visitor` compara na seção 6. 117

71 Os oito pares que mais confundem, e a pergunta que separa cada um. 118

72 A versão clássica, a moderna e o que se perde ao trocar. As perdas foram transcritas das seções "versão moderna" dos executáveis. 118

73 *Interpreter* escrito à mão contra gerador de *parser*. 120

74 A separação radical entre identidade, dados e lógica. 121

75 ECS contra hierarquia de herança. 122

76 As duas formas de usar uma biblioteca compartilhada. Só a segunda permite que o *plugin* seja escrito depois do *host*. 123

77 As quatro *flags* de `dlopen` que importam, e quando a alternativa se justifica. 125

78 Bugs reais encontrados ao habilitar alvos que estavam desativados. A coluna da direita é o que cada um ensina — e nenhum deles teria aparecido em revisão de código. 128

79 Os invólucros RAII de *lock*. 130

80 As três formas de obter um `future`. 131

81 As ordens de memória, do mais barato ao mais caro. 131

82 As armadilhas clássicas do capítulo, reunidas. 133

83 Por que um não substitui o outro. 137

84 Os três operadores, e o análogo em linguagens que já os tinham. 144

85 Qual primitiva para qual problema. 146

86 As duas implementações do mesmo fractal, comparadas. 149

87 O que usar conforme o que está esperando. 149

88 As garantias que o TCP dá e o UDP não. 157

89 As opções que `http::get` define, e o que cada uma evita. Valores extraídos de `src/http-client/http_client.cpp`. 162

90 Os quatro tipos de *exchange* e como cada um roteia. 170

91 Os seis estágios do *handler*, na ordem. Publicar antes de `onReady` é a origem da maioria dos "publiquei e sumiu". 171

92 As três respostas possíveis a uma mensagem recebida. 173

93 O mesmo conceito, três nomes — e o que cada um oferece além do teste. 175

94 Como cada biblioteca deste capítulo reporta falha, e onde o erro é observado. 176

95 As três *constraints* do vocabulário do Bullet, e o que cada uma modela. 188

96 Os limites angulares de fato programados em `ragdoll.cpp`, convertidos de radianos para graus. 188

97 Os dois subprojetos de integração Qt + SFML, como estão hoje. 193

98 As quatro decomposições, e quando cada uma é a ferramenta certa. 196

99 Cinco perguntas respondidas pela mesma decomposição. 198

100 Os números medidos deste capítulo, e o que cada um ensina. 202

101 As duas sintaxes, lado a lado. 203

102 As *constraints* usadas no subprojeto. 204

103 Assembly embutido contra assembly em arquivo próprio. 206

104 O essencial da ABI System V AMD64 (Linux). 207

105 As quinze lições, e o assunto de cada uma. 209

106 Modos de endereçamento e a construção C++ correspondente (de `src/asm_course/02_memory.cpp`). 211

107 Os bits de RFLAGS que importam, do cabeçalho de `src/asm_course/03_arithmetic.cpp`. 212

108 As três direitas do x86, com os valores medidos em `dist/bin/asm_04_bitwise`. 213

109 O que cabe num registrador vetorial, e o sufixo que o diz (de `08_simd_sse.cpp` e `09_simd_avx.cpp`). 216

110 As quatro operações de *string* e seus registradores implícitos (de `src/asm_course/10_string_ops.cpp`). 218

111 Convenção de *syscall* contra convenção de função (de `src/asm_course/11_syscalls.cpp`). 219

112 As ferramentas, do cabeçalho de `src/asm_course/14_disassembly.cpp`. 221

113 Expressões idiomáticas que o compilador emite o tempo todo. 221

114 Quando descer de nível — e por qual porta. 224

115 As duas formas de escrever a mesma extensão. 226

116 Duas semânticas de posse com a mesma assinatura. 227

117 Gerador de *parser* contra *parser* escrito à mão. 229

118 O *build* completo, medido com `make configure && make` numa árvore limpa. 232

119 Quem faz o quê, e quando. 233

120 Configuração e *build* são universos distintos. 233

121 As duas formas de gerar arquivo. 235

122 Onde os testes moram hoje. Fonte: o comentário de `meson.build` da raiz e `src/testing/README.md`. 236

123 Sintoma, causa e correção — a tabela mais útil do subprojeto. 236

124 Os quatro cabeçalhos ausentes, todos em alvos que não eram compilados. 237

125 Defeitos revelados ao habilitar os alvos "pesados". 238

126 As três fases do CMake, e o que existe em cada uma. 239

127 A tradução, comando a comando. Fonte: `src/cmake/CMakeLists.txt`, seção 2. 240

128 Os três escopos de propagação. 240

129 As duas formas de obter uma dependência. Fonte: `src/cmake/README.md`, demo 2. 243

130 Os três modelos de paralelismo, lado a lado. Fonte: `src/cmake/README.md`, demo 4. 246

131 Registrar o binário contra registrar os casos. 248

132 Fonte: `src/cmake/README.md`, seção final. 250

133 As cinco perguntas do vocabulário compartilhado. 254

134 Extraindo o valor de um `optional`. 255

135 Acessando um `variant`. Só `visit` verifica exaustividade. 257

136 `variant` favorece operações novas; herança favorece tipos novos. 259

137 A escolha é 100% de legibilidade: o custo é o mesmo. 261

138 `span<const T>` contra `const span<T>`. 262

139 O que passou a caber numa função `constexpr`, por padrão. 264

140 As três palavras e o que cada uma exige. 265

141 `constinit` não implica `const`. 266

142 O catálogo de *traits* que aparece de verdade. 267

143 O momento da falha decide se há alternativa. 267

144 Para poucas dezenas de entradas fixas, o arranjo ordenado ganha em tudo. 270

145 O que a documentação sugeria contra o que a medição encontrou no GCC 13.3. 272

146 O que o GCC 13.3 não tem, e o que usar no lugar. 275

147 O que muda entre *header* e módulo. 276

148 Estado da adoção de módulos, por *toolchain*. 278

149 As três formas corretas de levar um *view* a uma API que espera C-string. 283

150 Escolhendo a assinatura de um parâmetro de texto. 284

151 As quatro portas de saída da biblioteca de formatação. 286

152 As cinco formas de converter texto em número, e o defeito de cada uma. 287

153 As três formas de aplicar uma expressão regular. 289

154 Quando a expressão regular é a ferramenta certa. 290

155 Os três relógios clássicos e para que serve cada um. 292

156 A álgebra de instantes e durações. 293

157 As três defesas contra a eliminação, em ordem de preferência. 295

158 Qual estatística responde a qual pergunta. 296

159 Os quatro idiomas com máscara. 296

160 As quatro formas de reinterpretar bits, e o veredito de cada uma. 299

161 Saída de `src/bits/`: as quatro funções sobre dois valores. 300

162 `bitset` contra as alternativas. 301

163 A ferramenta certa para cada objetivo com bits. 302

164 Os modos de abertura mais usados. 304

165 Redirecionamento no *shell* — a referência que acompanha o subprojeto. 307

166 `sizeof` e `alignof` medidos por `src/layout/` (GCC 13, x86-64 System V). 311

167 O que cada categoria de tipo autoriza. 313

168 As três formas de guardar coisas heterogêneas. 318

169 `virtual` contra vtable manual. 321

170 O problema da expressão: cada abordagem escolhe um eixo. 322

171 `unique_ptr<Base>` contra o tipo-valor polimórfico. 324

172 Cada mensagem, a etapa em que nasce e a causa típica. 326

173 *Linkage* por forma de declaração. 328

174 Os quatro `static`, que não têm nada em comum entre si. 328

175 Mudanças que quebram a ABI sem quebrar a API. 331

176 F.I.R.S.T.: o que cada letra exige, e o que a viola. 335

177 As famílias de asserções. Todas têm a variante `ASSERT_`. 337

178 O que cada macro de teste faz variar. 339

179 Os cinco dublês de teste, e o que cada um verifica. 340

180 Os quatro tipos de cobertura, para `if (a && b) faz_algo();`. 343

181 As quatro categorias. As três primeiras são seguras; a quarta não tem limite. 344

182 O catálogo de UB, agrupado por quem tem chance de pegá-lo. 345

183 Os quatro *sanitizers* e o custo de cada um. 347

184 Matriz de detecção medida. Três resultados contrariam o senso comum. 347

185 Substituições que eliminam categorias inteiras de UB. 349

186 A ordem certa de otimizar: do que mais rende para o que menos rende. 356

187 Sete trilhas mínimas, por objetivo. Entre parênteses, a seção correspondente deste relatório; os subprojetos marcados com † entraram no repositório depois dos capítulos e ainda não têm seção própria — o material deles está no `README.md` de cada um e, resumido, na Seção XVI.6. 358

188 Sete ideias que atravessam o relatório. 359

189 Tier 1 — fundamentos da linguagem. Todos concluídos. 359

190 Tier 2 — C++ moderno. Os cinco subprojetos que a edição anterior listava como "planejados" foram concluídos, e um sexto (`file_io`) migrou para cá. 360

191 Tiers 3 a 5 — concorrência, modelo de objeto e arquitetura. Nenhuma linha continua "planejada". 360

192 Os 24 padrões implementados em `src/design-of-pattern/`, um diretório e um binário cada. Os treze acrescentados na última fase estão em **negrito**. 361

193 As quatro dívidas estruturais e como cada uma foi encerrada. Duas delas mudaram de estado *depois* da primeira redação deste relatório. 362

194 Os nove defeitos que estavam latentes em alvos desabilitados. Seis apareceram na compilação; três só na execução — e são os três mais instrutivos. 363

195 Medições dos subprojetos novos, todas obtidas na máquina descrita na Tabela 4 (GCC 13.3). Os valores absolutos variam com o processador; a ordem de grandeza entre os casos comparados, não. 368

196 Linhas produzidas pelo pré-processador, GCC 13.3 com `-std=c++20`. Medido, não estimado: a primeira versão desta tabela era chutada e cinco dos seis números estavam errados — `<cstddef>` por um fator de 4. 369

197 Matriz de detecção, medida com `-fsanitize=address,undefined` no GCC 13.3. Os três casos em que o resultado surpreendeu estão em **negrito**. 370

198 As contradições registradas no `src/ROADMAP.md`. Em todas, a explicação do resultado inesperado era mais interessante que a expectativa. 371

Aviso

Este relatório foi construído com o auxílio de inteligência artificial generativa e revisado manualmente por um grupo de especialistas humanos.
O conteúdo técnico foi derivado dos `README.md` dos subprojetos de `src/` e conferido contra o código-fonte que eles descrevem (Seção I.5). A responsabilidade pelo texto final é da equipe revisora.

I Introdução

I.1 O que é o `cpp_academy`

O `cpp_academy` [1] é um repositório de *subprojetos didáticos* em C++ moderno. Cada subprojeto vive em um diretório próprio sob `src/`, compila sozinho, produz um executável que imprime o que acabou de demonstrar, e vem acompanhado de um `README.md` que explica a teoria por trás do que o binário mostra.

A decisão de projeto que governa o repositório inteiro — e, por consequência, este relatório — é que **nenhuma afirmação sobre a linguagem fica sem demonstração executável**. Quando o texto diz que um destrutor vazio suprime as operações de *move*, existe um programa que conta as construções e imprime o número; quando diz que `dynamic_cast` é mais caro que uma chamada virtual, existe um *benchmark* que mede os dois. O que se ganha com isso é a diferença entre *saber de ouvir falar* e *ter visto acontecer*.

O repositório nasceu forte em *ecossistema* — SFML, Bullet, Qt, ONNX, JSBSim, AWS, Asio — e fraco em *linguagem*. O `src/ROADMAP.md` existe para rastrear e fechar essa assimetria, e boa parte dos subprojetos de fundamentos descritos aqui é resultado desse esforço.

Em números, no estado que este relatório descreve: **57 subprojetos** declarados em `src/meson.build`, **98 executáveis** instalados em `dist/bin/` por um `make install`, e **zero *warnings*** na compilação completa. Os dois primeiros números não coincidem porque um subprojeto pode produzir mais de um binário: `design-of-pattern` gera um executável por padrão GoF, `asm_course` um por módulo do curso, `rabbitmq` um produtor e um consumidor, `testing` um executável didático mais cinco binários de teste que não se instalam.

O "zero *warnings*" pede a definição do que está sendo prometido, porque *warning* — *aviso do compilador* — é uma palavra que cada projeto usa com um rigor diferente. Um aviso não é um erro: o programa compila e o binário sai. Ele é o compilador dizendo "isto é legal, mas quase certamente não é o que você quis dizer". O `meson.build` da raiz declara `warning_level=3`, e é o Meson quem traduz esse número para as opções do compilador:

```
project('academy', 'cpp', 'c',
    version: '1.0.0',
    default_options: [
        'cpp_std=c++20',
        'warning_level=3',
    ],
)
```

Listagem 1: O nível de aviso, declarado uma vez para o repositório inteiro, em `meson.build` da raiz.

No GCC e no Clang, `warning_level=3` vira `-Wall -Wextra -Wpedantic`. Os três nomes enganam e vale desfazer o engano de uma vez:

- `-Wall` *não* liga todos os avisos, apesar do nome. Liga o conjunto que a experiência mostrou ser quase sempre um erro real — variável não usada, comparação entre sinal e sem sinal, chave de `switch` não tratada.
- `-Wextra` acrescenta o segundo escalão: os que às vezes são intencionais e por isso não cabiam em `-Wall` — parâmetro não usado, comparação sempre verdadeira, ordem de inicialização de membros diferente da ordem de declaração.

- `-Wpedantic` exige o padrão puro: reclama de extensões do GNU que o código usa sem perceber, como *arrays* de tamanho variável ou vírgula sobrando no fim de um `enum`.

Dois dos bugs descritos na Seção I.8 apareceram primeiro como avisos desse conjunto, num alvo que ninguém compilava — e é por isso que a promessa de compilar sem avisos só vale alguma coisa quando *tudo* é compilado.

Regra — o repositório constrói todos os alvos, sempre

Não há uma única linha `subdir()` comentada em `src/meson.build`. Os subprojetos de dependência pesada — Qt, SFML, Bullet, ONNX Runtime, JSBSim, AWS SDK, AMQP-CPP, SOCI — são construídos junto com os demais. A regra existe porque o repositório já pagou o preço do contrário quatro vezes, e a Seção I.8 conta as quatro.

Por que — por que um subprojeto por assunto, e não um livro com capítulos

Um capítulo de livro pode alegar que o `std::vector` copia em vez de mover quando o construtor de *move* não é `noexcept`; um subprojeto tem de *provar*. A granularidade escolhida — um diretório, um `meson.build`, um executável, um `README` — é o menor pacote que sustenta essa prova, e é também o menor pacote que alguém consegue estudar em uma sessão sem carregar o resto do repositório junto.

I.2 O panorama do repositório

Antes da trilha, o mapa. Esta seção diz o que existe hoje em `src/`, em que ordem os assuntos se apoiam, e — para os subprojetos acrescentados depois da primeira edição deste relatório — qual é o número que cada um mede.

I.2.1 A anatomia de um subprojeto

"Subprojeto" aqui tem uma forma fixa, e conhecê-la poupa tempo em todos os capítulos seguintes:

```
src/layout/
  meson.build          # como se constroi: fontes, dependencias, nome do alvo
  main.cpp             # o unico .cpp: chama run() de cada demo, em ordem
  demo_tamanho.hpp     # 1 . um assunto por cabecalho, ~200 linhas
  demo_alignas.hpp     # 2 . cada um com namespace proprio e uma funcao run()
  demo_categorias.hpp  # 3 .
  README.md            # a teoria, as medicoes e as armadilhas
```

Listagem 2: A forma que todo subprojeto de `src/` tem.

Três consequências dessa forma, e nenhuma é acidental:

- **Um `.cpp` só.** Os demos moram em cabeçalhos (`.hpp`) com funções `inline`, e não em unidades de tradução separadas. Isso mantém o `meson.build` de três linhas e faz o subprojeto compilar em um comando. A exceção é `linkage`, que precisa de duas unidades de tradução porque o assunto *é* o que acontece quando o mesmo cabeçalho é compilado duas vezes.
- **Um *namespace* por demo.** `namespace demo_alignas` isola os nomes, de modo que dois demos possam ter uma `struct Ponto` cada um sem colidir — e o leitor vê, pelo qualificador, de onde veio cada coisa.

- **O binário narra.** Rodar `./dist/bin/layout` imprime a mesma sequência de seções do `README`, com os números daquela máquina. O texto e o programa não podem divergir sem que alguém note.

Por que — por que a medição vem antes da afirmação

Boa parte do que "todo mundo sabe" sobre C++ é folclore com data de validade vencida: que `shared_ptr` é lento, que `virtual` custa caro, que `seq_cst` precisa ser evitado. Um subprojeto que mede transforma cada uma dessas frases em um número reproduzível — e, nas vezes em que o número contraria a expectativa, ensina algo muito mais valioso que a resposta: ensina que a pergunta estava mal feita. A Seção I.2.3 traz seis casos em que isso aconteceu neste repositório.

I.2.2 Os subprojetos, na ordem em que se apoiam

A Tabela 1 reproduz a ordem de `src/meson.build`, que não é alfabética: quem lê o arquivo de cima para baixo lê os assuntos na ordem em que um depende do outro.

Tabela 1: Os subprojetos de `src/`: os 57 declarados em `src/meson.build`, na ordem em que aquele arquivo os declara, mais `modules` e `cmake`, que têm sistema de *build* próprio. Os marcados com ⋆ foram acrescentados depois da primeira edição deste relatório e são apresentados na Seção I.2.3.

Bloco	Subprojetos
Fundamentos	`syntax`⋆, `casts`, `value_semantics`, `smart-pointers`, `memory`, `oop`, `errors`, `stl`, `ranges`
Genérica	`templates`, `concepts`, `lambdas`⋆
C++ moderno	`vocabulary_types`⋆, `compile_time`⋆, `operators`⋆, `strings`⋆, `chrono`⋆, `bits`⋆, `file_io`⋆, `cpp23`⋆, `coroutines`
Concorrência	`concurrency`, `concurrency2`⋆, `mandelbrot`, `mandelbrot_openmp`
Modelo de objeto	`type_erasure`⋆, `layout`⋆, `linkage`⋆, `testing`⋆, `undefined_behavior`⋆, `performance`⋆
Arquitetura	`solid`, `design-of-pattern`, `ecs`, `plugin_registry`
Baixo nível	`asm_course`, `inline_asm`, `outline_asm`
Ferramentas	`meson`, `cmake`⋆, `modules`⋆, `doxygen`, `parser`
Rede e dados	`network`, `http-client`, `database`, `rabbitmq`, `aws/s3`
Gráficos e física	`calculus`, `collision`, `sfml_bullet`, `sfml_bullet2`, `qt_sfml`, `qt_sfml2`, `flight-simulator`
Interoperabilidade	`python`, `embed-python`, `yolo`
Experimentação	`sandbox`

Dois dos subprojetos da tabela — `modules` e `cmake` — não aparecem como `subdir()` em `src/meson.build`, e isso também é decisão, não esquecimento: cada um tem sistema de *build* próprio justamente porque o sistema de *build* *é* o assunto. O `modules` usa um `Makefile` porque módulos do C++20 no GCC 13 exigem `-fmodules-ts` e uma ordem de compilação manual que o Meson ainda não deriva sozinho; o `cmake` usa um `CMakeLists.txt` porque ensinar CMake dentro do Meson seria ensinar de segunda mão.

I.2.3 Os subprojetos que este relatório passou a cobrir

Dezenove subprojetos entraram depois da primeira edição. Eles não são "assuntos avançados": são, quase todos, a parte da linguagem que o repositório afirmava sem demonstrar. O que segue é a apresentação de cada grupo, com o número que ele mede — porque é pelo número que se decide se vale a pena abrir o subprojeto.

Linguagem e biblioteca padrão. `syntax` é o mapa: toda construção sintática da linguagem — declaração, `if`, `switch`, laço, função, classe, *template*, exceção — num único executável comentado, apontando para o subprojeto que aprofunda cada uma. É o primeiro a ser lido e aquele ao qual se volta.

`strings` responde por que texto é o lugar em que o desempenho vaza sem ninguém perceber. O número mais útil dele é o limite da **SSO** (*small string optimization*) — a otimização que guarda uma *string* curta *dentro* do próprio objeto, sem ida ao *heap*:

```
1  sizeof(std::string) = 32 bytes
2   1 chars -> 0 alocacoes    <- cabe no SSO
3  15 chars -> 0 alocacoes    <- cabe no SSO
4  16 chars -> 1 alocacao
5
6  extrair  um campo de log, 1 milhao de linhas:
7    find + from_chars ....      0 ms
8    std::regex  .........   .  133 ms   <- com a regex construida FORA do laco
```

Listagem 3: Saída de `src/strings/`: o limite da SSO e o custo do `std::regex`, medidos nesta máquina.

Os dois números respondem a duas perguntas de projeto que aparecem toda semana: *vale a pena evitar `std::string` em nomes curtos?* (não: até 15 caracteres ela não aloca) e *posso usar `std::regex` no caminho quente?* (não).

`chrono` trata do sistema de unidades de tempo, e traz uma armadilha que sozinha justifica o subprojeto:

Armadilha — `high_resolution_clock` não serve para cronometrar

O nome promete precisão, e a promessa que falta é outra: *monotonicidade*. Um relógio monótono nunca anda para trás. Na libstdc++, `high_resolution_clock` é um apelido de `system_clock` — o relógio de parede, que o NTP ajusta. O subprojeto imprime `is_steady = false` e a consequência é literal: um intervalo medido com ele pode sair *negativo* se o ajuste cair no meio da medição. Para medir duração existe `std::chrono::steady_clock`, e só ele.

`bits` cobre os operadores que parecem os mais simples da linguagem e escondem três formas de UB (comportamento indefinido). A primeira é a **promoção inteira**: todo operando de tipo menor que `int` é convertido para `int` antes da operação, e o resultado deixa de ser do tipo que se escreveu.

Armadilha — `~x` de um `uint8_t` não é um `uint8_t`

Com `uint8_t x = 0x0F`, a expressão `~x` não vale `0xF0`: vale `-16`, um `int`. O bug que isso produz é `if (flags == ~MASCARA)` — comparação de um valor sem sinal com um `int` negativo, que nunca é verdadeira e nunca acusa erro. A defesa é declarar máscaras como `unsigned` ou `uint32_t`. Do mesmo grupo: `1 << 40` é UB (comportamento indefinido), porque o literal `1` é `int` de 32 bits — o certo é `1ULL << 40`.

`file_io` percorre a hierarquia de *streams* — e a distinção prática entre `cerr` (sem *buffer*, sobrevive a um *crash*) e `clog` (com *buffer*, adequado a log volumoso) — até `std::filesystem`. `vocabulary_types` apresenta os tipos que existem para serem falados entre bibliotecas: `std::optional`, `std::variant`, `std::any`, `std::tuple` e `std::span`, cada um substituindo uma convenção que o compilador não verificava. `operators` trata da sobrecarga canônica e do `operator<=>`; `compile_time` das quatro palavras que movem trabalho para a compilação; `lambdas` do que o compilador realmente gera quando se escreve uma lambda — uma classe com as capturas como membros, o que explica de uma vez `mutable`, o tipo único de cada lambda e o perigo de `[&]`.

O inventário que corrigiu o próprio relatório. `cpp23` é diferente de todos: além de ensinar as adições do C++23, ele *mede* quais existem neste *toolchain*, usando **feature-test macros** — macros que o próprio compilador e a própria biblioteca definem quando implementam uma proposta, com o ano e o mês da adoção como valor.

```
echo '#include <version>' > /tmp/v.cpp
g++ -std=c++23 -dM -E /tmp/v.cpp | grep __cpp_lib | sort
```

Listagem 4: O comando que despeja o inventário completo da biblioteca, citado em `src/cpp23/`.

O resultado contrariou a expectativa em seis linhas — e todas as seis na mesma direção, a de *subestimar* o que já estava disponível:

Tabela 2: O que se presumia sobre o GCC 13.3 e o que a medição por feature-test macro mostrou. Fonte: `src/cpp23/`.

Recurso	Presumido	Medido
`std::stacktrace`	ausente	presente (com biblioteca extra)
`views::zip`	ausente	presente
`ranges::fold_left`	ausente	presente
`views::chunk` / `slide`	ausente	presente
`std::move_only_function`	ausente	presente
`std::spanstream`	ausente	presente

Armadilha — a macro diz que a API existe, não que ela linka

O caso do `std::stacktrace` é o mais instrutivo dos seis. `__cpp_lib_stacktrace` está definida, o cabeçalho existe, o código compila — e o *link* falha com `undefined reference to '__glibcxx_backtrace_simple'`, porque a implementação mora numa biblioteca separada que exige `-lstdc++_libbacktrace` na linha de ligação. Uma feature-test macro é uma afirmação sobre a *interface*, nunca sobre as opções de *build*.

Modelo de objeto e engenharia. Seis subprojetos que respondem, com número, perguntas que costumam ser respondidas com opinião.

`layout` explica como os bytes de um objeto são realmente arrumados. A regra inteira cabe em três linhas — todo tipo tem um `alignof(T)` e seu endereço precisa ser múltiplo dele; o alinhamento de uma `struct` é o maior entre os de seus membros; `sizeof` é sempre múltiplo de `alignof`, para que *arrays* funcionem — e todo o resto decorre disso, inclusive o fato de `sizeof` quase nunca ser a soma dos membros.

```
struct  com 4 membros, na ordem "natural" ....... 24 bytes
os mesmos 4 membros, do maior para o menor ..... 16 bytes   (−33%)
membro vazio, sem atributo       ..............      16 bytes
o mesmo com [[no_unique_address]]  ..............       8 bytes
um virtual ~T() que ninguem chama  ..............      +8 bytes por objeto
    em 10 milhoes de objetos       ..............       76 MB
```

Listagem 5: Saída de `src/layout/`: o que se ganha reordenando membros e o que custa um destrutor virtual.

`linkage` é o único subprojeto com *duas* unidades de tradução, e por necessidade: o assunto é a **ODR** (*One Definition Rule*) — a regra que exige que cada entidade tenha uma única definição no programa inteiro. Uma **unidade de tradução** é um `.cpp` mais tudo o que os seus `#include` trouxeram, recursivamente; o compilador processa uma por vez, em completa ignorância das outras, e o *linker* cola tudo no fim. Praticamente todo erro confuso de *build* — `undefined reference`, `multiple definition` — é consequência direta desse modelo. O experimento das duas unidades:

```
inline int contador   -> 20 / 20   <- UM objeto, compartilhado
static int contador   -> 10 / 10   <- DOIS objetos, um por unidade
```

Listagem 6: Saída de `src/linkage/`: duas unidades de tradução incrementam a mesma variável dez vezes cada uma.

Armadilha — o `static` em cabeçalho que "funciona" até o segundo `.cpp`

Uma variável `static` num cabeçalho tem *linkage* interna: cada unidade de tradução que o inclui ganha a *sua própria cópia*. Enquanto o cabeçalho for incluído por um `.cpp` só, tudo funciona. No dia em que alguém o inclui num segundo, o programa passa a ter dois contadores — e o sintoma que aparece ("o cache não persiste", "o registro perdeu entradas") não aponta para o cabeçalho em lugar nenhum. Compila, linka, roda, e faz outra coisa. Para uma variável única e visível a todos, o instrumento correto é `inline` (C++17).

`type_erasure` abre a caixa do `std::function`: o que é *apagamento de tipo* — guardar um objeto de tipo `T` num lugar que não menciona `T`, preservando as operações que interessam — como se implementa, e quando *não* vale a pena, esta última respondida com medição.

```
SBO da std::function:  captura de 16 B cabe inline; 24 B ALOCA

despacho sobre 200 000 elementos, base = CRTP (1.0x):
  std::variant + visit  .....  1.0x
  virtual  .....  3.9x
  type erasure  .....  4.0x
o que domina esse 4x: LOCALIDADE, nao o despacho
```

Listagem 7: Saída de `src/type_erasure/`: o buffer interno da `std::function` e o custo relativo de quatro formas de despacho.

A última linha é a lição, e ela contraria a explicação que quase todo mundo dá: o custo de `virtual` não está na indireção pela *vtable*, está em que os objetos passam a viver espalhados pelo *heap* e o processador perde o *cache*. `performance` desenvolve exatamente esse ponto, e é o subprojeto mais explicitamente medido do repositório.

```
matriz 2048x2048, percorrida por linha vs por coluna  .....  35x
soma sequencial vs aleatoria (8 M ints)  .....  26x
false sharing: mesma linha de cache vs alignas(64)  .....  11.7x
std::vector vs std::list (2 M ints)  .....  6.6x
AoS vs SoA, laco que usa 1 campo de 10  .....  1.5x
laco com resultado descartado vs consumido  .....  eliminado
```

Listagem 8: Saída de `src/performance/`: a assimetria entre processador e memória, em razões medidas nesta máquina.

E as duas medições que *não* confirmaram a expectativa, que valem mais que as seis acima porque ensinam método:

```
vetor ordenado vs embaralhado, -O2   esperado ~3x   medido 1.0x
   -> o GCC gerou 'cmov': nao ha desvio para prever
AoS vs SoA usando TODOS os campos   esperado AoS   medido SoA 1.6x
```

Listagem 9: Saída de `src/performance/`: os dois experimentos em que a expectativa estava errada.

O primeiro é o exemplo clássico de predição de desvio, reproduzido em milhares de textos — e que, com `-O2` num compilador atual, simplesmente não acontece mais, porque o compilador troca o `if` por uma instrução de movimentação condicional, sem desvio. Um texto que apenas repetisse o folclore estaria ensinando algo falso; o subprojeto que mede descobre a mudança.

`undefined_behavior` trata do assunto que este relatório persegue desde a Seção I.1: o erro que compila, roda e mente. O ponto que quase todo mundo perde é que o efeito do UB (comportamento indefinido) *não fica restrito ao trecho errado* — ele pode aparecer antes da linha do erro, porque o compilador reorganizou o código supondo que o UB (comportamento indefinido) não aconteceria. O exemplo canônico não é acadêmico: é o CVE-2009-1897 do kernel Linux, em que o compilador removeu uma verificação de ponteiro nulo escrita *depois* de uma desreferência — e a remoção virou escalação de privilégio.

O subprojeto tem dois alvos, e a separação é deliberada: o executável instalado *narra* o assunto e não executa UB (comportamento indefinido) nenhum; um segundo binário, não instalado, contém doze casos de UB (comportamento indefinido) real, um por argumento de linha de comando, para ser rodado sob *sanitizer*. A matriz de detecção resultante é a parte que interessa:

```
caso            −O0                              −O1
heap            ASan: buffer−overflow            idem
uaf             ASan: heap−use−after−free        NADA   <− some com −O1
overflow        UBSan: signed integer overflow   idem
divisao         UBSan: division by zero          idem
vazamento       LSan: detected memory leaks      idem
aliasing        NADA                             NADA
nao inicializado NADA                            NADA
alinhamento     NADA (GCC)                       NADA (GCC)
```

Listagem 10: Saída de `src/undefined_behavior/`: o que ASan e UBSan pegaram, e o que passou.

Armadilha — *sanitizer* limpo não é prova de ausência de UB

Quatro das doze categorias não foram detectadas por nenhum dos dois *sanitizers*, e uma delas — *use-after-free* — deixou de ser detectada ao subir a otimização de `-O0` para `-O1`, porque o otimizador dissolveu a alocação que o ASan instrumentaria. Rodar sob *sanitizer* aumenta muito a chance de encontrar; não autoriza a conclusão "não há UB (comportamento indefinido) aqui".

`testing` fecha o grupo e tem estrutura própria, descrita na Seção I.7.

Concorrência moderna. `concurrency2` continua o `concurrency` onde o C++11 termina: `jthread` e `stop_token`, `latch`, `barrier`, `counting_semaphore`, o modelo de memória e programação *lock-free*. Três dos seus resultados contrariam o senso comum:

```
fetch_add sem contencao, x86−64:
    memory_order_seq_cst vs relaxed ..... 1.01x
reimplementar fetch_add com CAS, 4 threads:
    retrabalho (CAS que falhou e repetiu) .... 43−60%
pilha lock−free vs std::vector sob mutex .... 2.7x MAIS LENTA
```

Listagem 11: Saída de `src/concurrency2/`: três medições contra o folclore de concorrência.

A primeira linha desmonta a otimização que "todo mundo faz": no x86-64, um `fetch_add` com a ordem mais forte custou 1% a mais que a mais fraca, porque a arquitetura já garante quase tudo em hardware. Num ARM o mesmo experimento daria outro número — e é essa dependência de arquitetura, não a regra decorada, que o subprojeto ensina. A terceira desmonta a palavra *lock-free* como sinônimo de "rápido": ela é uma garantia de *progresso* (nenhuma *thread* pode ser bloqueada indefinidamente por outra que dormiu), não uma promessa de desempenho.

Ferramentas. `cmake` é o par de `meson`: ali o `meson.build` é o material didático, aqui o `CMakeLists.txt` é — alvos e o modelo de escopo `PRIVATE`/`PUBLIC`/`INTERFACE`, `find_package`, Protobuf gerado durante o *build*, OpenMP, MPI, CUDA, `install()` e CTest. `modules` apresenta os módulos do C++20, a substituição

do `#include`, e documenta com honestidade o estado do suporte no GCC 13: os cinco problemas que o `#include` tem — reprocessamento do mesmo cabeçalho em cada `.cpp`, macros vazando nos dois sentidos, ordem de inclusão importando, *include guards*, ausência de encapsulamento — todos resolvidos de uma vez, ao custo de uma ordem de compilação obrigatória que o *build* precisa conhecer.

I.3 Objetivo e público-alvo deste relatório

Este documento é um **relatório de tópicos de C++**: ele reúne, num só texto e numa ordem única, a teoria que os subprojetos de `src/` desenvolvem em separado. Onde o repositório oferece quarenta portas de entrada independentes, o relatório oferece uma trilha.

O público é o desenvolvedor que já programa — em C++ ou em outra linguagem — e quer entender *o que o compilador faz com o seu código*, não apenas qual sintaxe compila. Não é um tutorial de primeiro contato com programação, e não substitui o padrão [2] nem a consulta diária ao `cppreference` [3].

O relatório privilegia as *decisões* — por que a linguagem é assim, que erro cada mecanismo previne, quanto custa — em vez da enumeração exaustiva de sintaxe. Em particular, ele insiste num tipo de conteúdo que a documentação de referência costuma omitir: o **erro silencioso**, aquele que compila sem *warning*, roda sem *crash* e só aparece como lentidão inexplicável ou como UB (comportamento indefinido) meses depois. Cada capítulo tem uma seção desses.

Regra — o relatório não substitui o repositório

Todo capítulo aponta para os diretórios de `src/` que ele descreve. O texto explica *por que*; o código mostra *que sim*. Ler um capítulo sem rodar ao menos uma vez o binário correspondente desperdiça metade do material — boa parte das tabelas deste relatório é saída de programa, não alegação de autor.

I.4 Como o relatório está organizado

A ordem dos capítulos é a **trilha didática** declarada no `README.md` da raiz do repositório, que vai do concreto ao abstrato e do núcleo da linguagem ao ecossistema. A Tabela 3 dá a correspondência exata entre as etapas daquela trilha, os capítulos deste relatório e os subprojetos que cada um cobre.

As quatro últimas entradas com número de capítulo — Seção XII, Seção XIII, Seção XIV e Seção XV — não correspondem a nenhuma etapa da trilha do `README.md`, e o motivo é histórico: elas cobrem os subprojetos que nasceram do `src/ROADMAP.md`, o documento que rastreia o que falta de cobertura da linguagem, depois que a trilha foi escrita. Este relatório os agrupa por afinidade de assunto, e não por ordem de chegada:

- `operators` entra na Seção II, logo depois de `value_semantics`: sobrecarregar `+` corretamente é uma aplicação direta de saber quando um argumento é copiado e quando é movido. `syntax` abre o mesmo capítulo, por ser o mapa de tudo o que vem depois.
- `lambdas` entra na Seção IV, porque a discussão `std::function` *versus* parâmetro de *template* só faz sentido depois de *templates*.
- `vocabulary_types` e `compile_time` formam a Seção XII com `cpp23` e `modules`: os quatro tratam de *o que o C++ moderno acrescentou*, e os dois últimos respondem à pergunta prática que os dois primeiros levantam — o que deste padrão o meu compilador realmente tem.

Tabela 3: Mapa entre as etapas da trilha de estudo do repositório, os capítulos deste relatório e os subprojetos de `src/` que cada capítulo desenvolve.

Etapa	Capítulo	Subprojetos
1	Seção II	`syntax`, `casts`, `value_semantics`, `operators`, `smart-pointers`, `oop`, `errors`
2	Seção III	`stl`, `ranges`, `memory`
3	Seção IV	`templates`, `concepts`, `lambdas`
4	Seção V	`solid`, `design-of-pattern`, `ecs`, `plugin_registry`
5	Seção VI	`concurrency`, `concurrency2`, `coroutines`, `mandelbrot`, `mandelbrot_openmp`
6	Seção VII	`network`, `http-client`, `database`, `rabbitmq`, `aws`
7	Seção VIII	`collision`, `sfml_bullet`, `sfml_bullet2`, `qt_sfml`, `qt_sfml2`, `calculus`, `flight-simulator`
8	Seção IX	`inline_asm`, `outline_asm`, `asm_course`
9	Seção X	`python`, `embed-python`, `parser`, `yolo`
—	Seção XI	`meson`, `cmake`, `doxygen`, `sandbox`
—	Seção XII	`vocabulary_types`, `compile_time`, `cpp23`, `modules`
—	Seção XIII	`strings`, `chrono`, `bits`, `file_io`
—	Seção XIV	`layout`, `type_erasure`, `linkage`
—	Seção XV	`testing`, `undefined_behavior`, `performance`
—	Seção XVI	trilhas por objetivo e estado de cobertura

- `strings`, `chrono`, `bits` e `file_io` formam a Seção XIII. A afinidade não é temática, é de método: os quatro tratam de representações de dados em que a intuição falha — um caractere que ocupa três bytes, um relógio que anda para trás, um `uint8_t` que vira `int`, um *buffer* que só é descarregado no fim.
- `layout`, `type_erasure` e `linkage` formam a Seção XIV: como os bytes de um objeto são arrumados, como se guarda um objeto sem mencionar o tipo dele, e o que o *linker* faz com tudo isso.
- `testing`, `undefined_behavior` e `performance` formam a Seção XV, que é o capítulo sobre *método*: como se prova que o código está certo, o que o compilador tem licença para fazer quando não está, e como se mede em vez de supor.
- `concurrency2` entra na Seção VI, imediatamente depois de `concurrency`: é a continuação daquele subprojeto no ponto em que o C++11 termina e o C++20 começa.
- `cmake` entra na Seção XI, ao lado de `meson`: o mesmo assunto por dois sistemas de *build* diferentes, que é a única forma de separar o que é conceito do que é sintaxe de uma ferramenta.

I.4.1 As dependências entre os capítulos

A trilha é uma sugestão, não uma exigência: cada subprojeto compila sozinho. O que a ordem reflete são as dependências *conceituais*, que existem mesmo quando não existem dependências de compilação — é difícil entender `shared_ptr` sem entender destrutor e cópia, e é difícil entender *ranges* sem entender iteradores. A Figura 1 desenha as que mais custam caro quando ignoradas.

I.5 Convenções deste relatório

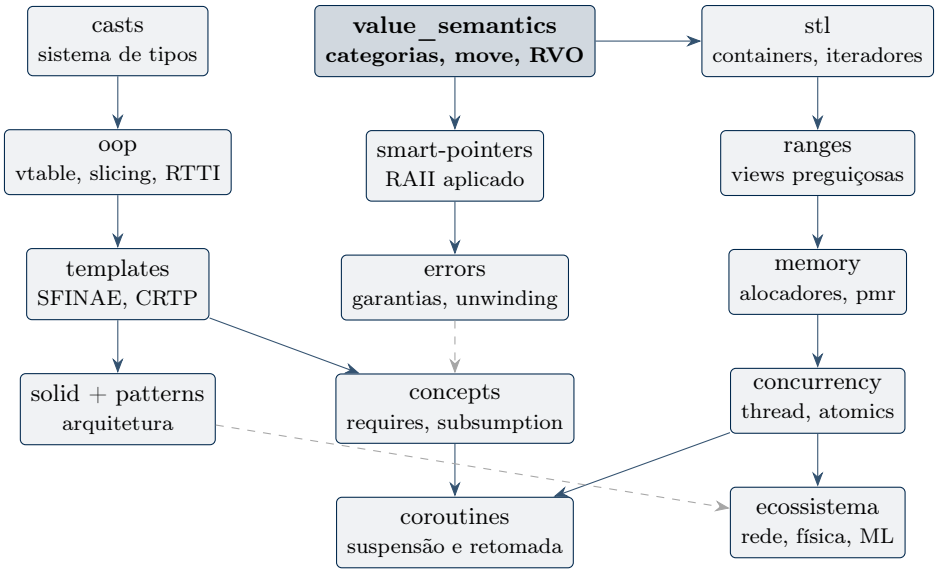


Figura 1: As dependências conceituais que a trilha de estudo respeita. A seta cheia indica pré-requisito real (estudar fora de ordem custa caro); a tracejada, apoio conveniente. `value_semantics` é a raiz de quase tudo: é ele que define quando o compilador copia e quando move.

I.5.1 Os blocos de código

O texto usa duas espécies de bloco de código, e a **legenda** é o que as distingue:

- **Transcrição.** Quando a legenda traz um caminho de arquivo — `src/oop/demo_virtual.hpp`, por exemplo — o conteúdo é transcrição do código-fonte do repositório, eventualmente condensada, com as omissões marcadas por `// ...`, mas nunca reescrita.
- **Ilustração.** Sem caminho na legenda, o bloco é código escrito para o texto: uma forma sintática mínima, um contraste entre duas versões, um antipadrão que não existe no repositório justamente porque é um antipadrão.
- **Saída de programa.** Quando o caminho da legenda termina em barra — `src/errors/` — ele nomeia o *subprojeto* que produziu aquela saída, e o bloco é o que o binário imprimiu. Boa parte das medições citadas neste relatório entra por aqui.

Essa convenção é conferida por programa. O `scripts/check-fontes.py` percorre os capítulos, localiza cada bloco cuja legenda cita um caminho `src/...`, reduz bloco e arquivo a sequências de *tokens* — descartando comentários, espaços e qualificadores de *namespace* — e mede que fração dos n-gramas do bloco ocorre no arquivo citado. Reprovar significa uma de duas coisas, e as duas pedem ação: ou a transcrição está infiel, ou o bloco não é transcrição e a legenda não deveria sugerir que fosse. Roda com:

```
make check                                  # instalação + conferência
python3 scripts/check-fontes.py -- listar   # bloco a bloco, com o percentual
```

Listagem 12: A conferência do texto contra o código-fonte.

Para um bloco que condensa uma interface reordenando membros — útil para ensinar, mas não uma cópia — a legenda diz *"resumo de"* antes do caminho, e o verificador não o cobra.

I.5.2 As caixas de destaque

Três caixas aparecem ao longo do texto, cada uma com um papel fixo:

- **Regra** — uma norma de escrita de C++ que o desenvolvedor deve seguir, quase sempre com contrapartida nas *Core Guidelines* [4].
- **Armadilha** — um comportamento real da linguagem ou da biblioteca que contraria a expectativa razoável e que o compilador não denuncia. É a categoria mais importante do relatório.
- **Por que** — a motivação de uma decisão de projeto, da linguagem ou do repositório.

Fora delas há apenas o **Aviso** que abre a página 1, de uso único.

I.5.3 Índice, glossário e referências cruzadas

Nomes de tipos e de classes marcados com `\classe` entram no **índice remissivo** ao final; cabeçalhos padrão e subprojetos entram em suas próprias famílias de verbetes. O **glossário** recolhe os termos que atravessam mais de um capítulo — RAII, UB (comportamento indefinido), categoria de valor, SFINAE — e imprime todos, citados ou não, porque quem consulta um glossário procura o termo, não a página em que ele foi introduzido.

I.6 O ambiente descrito

Tudo o que este relatório afirma sobre comportamento observado foi verificado no ambiente da Tabela 4. Onde o comportamento depende da versão do compilador ou do padrão de linguagem, o texto diz qual.

Tabela 4: O ambiente de referência do repositório.

Peça	Versão	Papel
GCC	13.3	compilador de referência
Padrão	C++20	fixado em `meson.build` da raiz
Meson + Ninja	1.0+	sistema de *build*
Conan	2.0+	dependências de terceiros
`fmt`	10.2.1	formatação — a única dependência comum a tudo
Google Test	1.14.0	testes

I.6.1 Compilando e rodando

A sequência mínima, a partir de um clone limpo:

```
make configure      # Conan baixa as dependencias e configura o Meson
make build          # compila os 57 subprojetos -- todos, sem excecao
make install        # instala os 98 binarios em dist/bin/
make test           # roda os 6 alvos registrados no 'meson test'
./dist/bin/value_semantics
```

Listagem 13: Do clone ao primeiro binário.

Para trabalhar em um subprojeto só, sem recompilar o resto:

```
source build/conanbuild.sh
meson compile -C build oop
./dist/bin/oop
```

Listagem 14: O ciclo de edição de um subprojeto.

Armadilha — um alvo novo exige `--reconfigure`

Acrescentar um `subdir()` em `src/meson.build` não basta para que o `meson compile` enxergue o alvo: o diretório de *build* já existente não sabe que ele passou a existir, e a resposta é `ERROR: Can't invoke target 'x': target not found`. É preciso `meson setup --reconfigure build .` uma vez. Depois disso, `make build` volta a bastar.

I.6.2 Todos os alvos são construídos

Não existe mais nenhuma linha `subdir()` comentada em `src/meson.build`: os 57 subprojetos são construídos, inclusive os de dependência pesada — Qt, SFML, Bullet, ONNX Runtime, JSBSim, AWS SDK, AMQP-CPP e SOCI. Um `make build` compila tudo; um `make install` deixa 98 executáveis em `dist/bin/`.

Isso muda a leitura deste relatório em um ponto prático: os capítulos VII a X descrevem programas que *também* rodam depois de um `make build`, sem edição nenhuma no sistema de *build*. O que eles exigem é que as bibliotecas estejam instaladas — o `make configure` chama o Conan para a maioria delas, e o Qt 6 vem do sistema (`sudo apt install qt6-base-dev`).

Habilitar os alvos que estavam comentados não foi uma mudança de uma linha: custou seis correções, quatro delas em defeitos que estavam latentes havia meses. A Seção I.8 conta cada uma, porque o conjunto ensina mais do que as partes.

I.6.3 Os limites do toolchain

O GCC 13.3 implementa quase todo o C++20 e boa parte do C++23, mas não tudo. A Tabela 5 registra o que o repositório usa, o que gostaria de usar e não pode — e, na última coluna, *como se sabe*: cada linha tem uma **feature-test macro**, uma constante que o compilador ou a biblioteca define quando implementa a proposta correspondente, com a data de adoção como valor. É a única fonte que não envelhece, porque vem do próprio *toolchain* em uso.

O "parcial" do `std::stacktrace` é o caso descrito na Seção I.2.3: a macro está definida, o cabeçalho existe, o código compila, e o *link* falha sem `-lstdc++_libbacktrace`. Seis linhas desta tabela estavam erradas antes de o `src/cpp23/` medi-las, e todas as seis diziam "não" sobre algo que existia — o efeito acumulado de consultar documentação de terceiros em vez do compilador que se tem à mão.

Regra — pergunte ao compilador, não à internet

Para saber se um recurso existe no *toolchain* em uso, a resposta definitiva sai em dois comandos, e não depende de nenhuma tabela publicada: `echo '#include <version>' > v.cpp` seguido de `g++ -std=c++23 -dM -E v.cpp | grep __cpp_lib | sort` despeja todas as macros de biblioteca definidas, com a data de adoção de cada uma. É o método que o `src/cpp23/` automatiza.

Tabela 5: Recursos de C++23 e a disponibilidade medida no compilador de referência. A coluna da direita é a macro que `src/cpp23/` consulta para produzir esta tabela.

Recurso	Padrão	GCC 13.3	Macro
`std::expected`	C++23	sim	`__cpp_lib_expected`
`if consteval`	C++23	sim	`__cpp_if_consteval`
subscript multidimensional	C++23	sim	`__cpp_multidimensional_subscript`
`static operator()`	C++23	sim	`__cpp_static_call_operator`
`views::zip`	C++23	sim	`__cpp_lib_ranges_zip`
`std::move_only_function`	C++23	sim	`__cpp_lib_move_only_function`
`std::byteswap`	C++23	sim	`__cpp_lib_byteswap`
`std::stacktrace`	C++23	parcial	`__cpp_lib_stacktrace`
deducing this	C++23	não	`__cpp_explicit_this_parameter`
`std::print`	C++23	não	`__cpp_lib_print`
`std::mdspan`	C++23	não	`__cpp_lib_mdspan`
`std::generator`	C++23	não	`__cpp_lib_generator`
`ranges::to<C>()`	C++23	não	`__cpp_lib_ranges_to_container`
`std::flat_map`	C++23	não	`__cpp_lib_flat_map`
`import std;`	C++23	não	—

O `cpp_std` da raiz continua sendo `c++20`. Treze alvos precisam de C++23 e declaram a exceção localmente, sem tocar na raiz: `errors` (`std::expected`), `vocabulary_types` (as monádicas de `optional`), `compile_time` (`if consteval`), `operators` (subscript multidimensional e `static operator()`), `lambdas`, `strings` (`string::contains`), `chrono`, `bits` (`std::byteswap`), `type_erasure`, `cpp23`, `meson` e dois dos padrões GoF de `design-of-pattern`. A forma é sempre a mesma:

```
executable('errors',
    ['main.cpp'],
    dependencies: [fmt_dep],
    override_options: ['cpp_std=c++23'],
     install : true,
    install_dir : get_option('prefix') / 'bin',
)
```

Listagem 15: O padrão elevado só onde é preciso, em `src/errors/meson.build`.

Regra — o padrão sobe por subprojeto, nunca na raiz

Elevar `cpp_std` no `meson.build` da raiz resolveria o problema de treze alvos e mudaria o significado de todos os outros — inclusive apagando a distinção, didática, entre o que já é C++20 e o que ainda é C++23. A exceção local é a forma de manter o repositório honesto sobre em que padrão cada coisa passou a existir.

I.7 Os testes do repositório

Durante muito tempo o repositório teve um diretório `tests/` na raiz cujo `meson.build` estava inteiramente comentado. Ele não construía nada, não rodava nada e não falhava nunca — e, por estar lá, dava a impressão contrária. Esse diretório foi **removido**. Os testes passaram a morar junto do código que testam, em três lugares:

Os seis primeiros — os cinco de `src/testing/` e o de `src/meson/` — são os que o `meson test` enxerga, porque são os que estão registrados com a função `test()` do Meson. Os do `src/cmake/` rodam pelo `ctest`, já que aquele subprojeto tem sistema de *build* próprio de propósito.

```
meson test -C build                    # os 6 alvos
```

Tabela 6: Onde os testes moram, o que cada conjunto exercita e como se roda.

Lugar	Alvos	O que é
`src/testing/`	5 alvos, 41 casos	GoogleTest de verdade sobre uma classe de carrinho de compras: básico, *fixture*, parametrizado, *death test* e *mock*
`src/meson/`	1 alvo	o `test()` do Meson demonstrado no próprio `meson.build` de aula
`src/cmake/`	CTest, 11 casos	o equivalente do outro lado: `enable_testing()`, `gtest_discover_tests`, `add_test`

```
meson test -C build testing_parametrizado -v    # um deles, com a saida
make test                                        # o mesmo, pelo Makefile

cd src/cmake && ctest --test-dir build --output-on-failure
```

Listagem 16: Rodando os testes.

Vale explicar a organização de `src/testing/`, porque ela é uma decisão de projeto e não um acidente. Cada arquivo `test_*.cpp` vira um binário independente e um `test()` separado:

```
foreach nome, fonte : testes_com_gtest
    exe = executable('testing_' + nome, [fonte],
        dependencies: [carrinho_dep, gtest_dep, gtest_main_dep],
        install : false,            # binarios de teste nao vao para dist/bin
    )
    test('testing_' + nome, exe, protocol: 'gtest', timeout: 60)
endforeach
```

Listagem 17: Um alvo de teste por arquivo, em `src/testing/meson.build`.

Duas coisas acontecem aí, e as duas valem para qualquer projeto:

- `install: false` — um binário de teste não é um entregável. Ele não vai para `dist/bin/`, e é por isso que o repositório tem 98 binários instalados e mais alguns que só existem dentro de `build/`.
- `protocol: 'gtest'` — sem essa opção, o Meson só sabe se o processo saiu com código zero ou não, e reporta "passou" ou "falhou" para o binário inteiro. Com ela, o Meson lê a saída estruturada do GoogleTest e reporta *caso a caso*, o que torna o `--gtest_filter` quase sempre desnecessário.

Por que — por que um binário de teste por assunto, e não um só com tudo

Um executável único com os 41 casos daria o mesmo resultado e uma informação a menos. Separado por assunto, a lista de falhas do `meson test` já diz *qual categoria* quebrou — `testing_fixture` falhando aponta para o estado compartilhado entre casos; `testing_death` falhando aponta para uma pré-condição que deixou de abortar — antes de qualquer um abrir a saída do teste. É a mesma lógica dos subprojetos: a granularidade certa é a que faz o nome do alvo carregar informação.

Armadilha — o alvo `testing_mock` pode desaparecer em silêncio

O *mock* exige o `gmock`, que vem junto do pacote `gtest` do Conan mas é uma dependência separada no `pkg-config`. O `meson.build` do subprojeto condiciona o alvo a `gmock_dep.found()` e emite um `warning()` de configuração quando não encontra. O efeito prático: numa máquina sem `gmock`, o `meson test` passa a rodar *cinco* alvos em vez de seis, todos verdes — e nada em vermelho avisa que um

conjunto inteiro de casos deixou de existir. Confira o número de alvos, não só a cor.

I.8 O que habilitar todos os alvos revelou

Durante boa parte da vida deste repositório, os subprojetos de dependência pesada — Qt, SFML, Bullet, ONNX Runtime, AWS SDK, AMQP-CPP, SOCI, JSBSim — tinham a linha `subdir()` comentada em `src/meson.build`. O argumento parecia razoável: quem só quer estudar `value_semantics` não deveria esperar o Qt compilar.

Hoje não há mais nenhuma linha comentada ali. Habilitar os alvos que faltavam custou uma tarde, e o que essa tarde revelou é material didático melhor do que qualquer exemplo inventado — porque nenhum dos problemas encontrados era hipotético. Todos estavam no repositório havia meses, e todos eram invisíveis pela mesma razão: **ninguém compilava aquele código**.

Por que — por que esta seção existe num capítulo de introdução

Um leitor que tropeça nesses seis bugs no capítulo 9, isolados, aprende seis fatos. Vendo-os juntos, e sabendo que os seis estavam exatamente nos alvos desabilitados, ele aprende uma coisa só, que vale mais: o alvo que não é construído é o alvo em que o defeito se instala. É uma afirmação sobre processo de engenharia, e este repositório tem a evidência para sustentá-la.

I.8.1 Os quatro cabeçalhos que faltavam

O primeiro grupo é o mais banal e o mais frequente na indústria: quatro `#include` ausentes.

```
1  src/solid/srp/demo_pipeline.hpp      faltava #include <algorithm>
2  src/concepts/demo_constraints.hpp    faltava #include <array>
3  src/file_io/demo_sstream.hpp         faltava #include <cmath>
4  src/aws/s3/main.cpp                  faltava #include <fmt/core.h>
```

Listagem 18: Os quatro cabeçalhos que faltavam, todos em alvos que não eram construídos. Fonte: o comentário no topo do índice de subprojetos, em `src/`.

Vale entender por que um cabeçalho ausente consegue passar despercebido, porque a explicação é uma propriedade real do modelo de compilação do C++ e não um descuido de quem escreveu. Um `#include` é uma *cópia de texto*: o pré-processador substitui a linha pelo conteúdo inteiro do arquivo, antes de o compilador ver qualquer coisa. Se `<vector>` inclui `<algorithm>` por conta própria naquela versão da biblioteca — e nada no padrão o proíbe —, então um arquivo que use `std::sort` sem incluir `<algorithm>` compila perfeitamente, *por acaso*. Ele quebra quando a biblioteca muda de versão, quando outro compilador organiza os cabeçalhos de outro jeito, ou quando alguém remove um `#include` aparentemente não usado três arquivos acima.

Regra — inclua o que você usa

A norma tem nome próprio na indústria — *Include What You Use* — e uma formulação de uma linha: todo nome que um arquivo menciona deve vir de um cabeçalho que *esse* arquivo inclui, nunca de um cabeçalho que ele obteve de carona. Ferramentas como o `include-what-you-use` automatizam a verificação, mas a verificação mais barata é a que este repositório passou a fazer: compilar tudo, sempre.

O caso do `src/aws/s3/main.cpp` é o mais eloquente dos quatro. O arquivo chamava `fmt::print` e não incluía `<fmt/core.h>`. Não era um erro sutil, era um erro grosseiro — e sobreviveu porque a linha `subdir('aws/s3')` estava comentada. Nenhuma quantidade de revisão de código substitui um compilador que roda.

I.8.2 O `subq` que quebrava o alinhamento da pilha

O segundo grupo só aparece ao *executar* o binário, não ao compilá-lo, e é onde a coisa fica interessante.

O módulo 7 do `asm_course` implementa, em *assembly* escrito à mão, as funções que a convenção de chamada exige. Uma delas, `asm_apply`, recebe um ponteiro de função e um inteiro e chama a função. A versão anterior fazia um `subq $8, %rsp` antes do `call`, com um comentário explicando que o registrador de pilha estava desalinhado por 8 bytes. A conta estava invertida, e o comentário era a prova de que quem escreveu tinha pensado no assunto — e errado o sinal.

Para entender o defeito é preciso a regra, e ela é curta. A ABI System V AMD64 — o contrato que todos os compiladores de Linux x86-64 respeitam — exige que, **no instante em que a instrução `call` é executada**, o registrador `RSP` (o topo da pilha) seja múltiplo de 16. A consequência imediata é que, ao *entrar* numa função, `RSP` vale um múltiplo de 16 mais 8, porque o `call` que trouxe o fluxo até ali empurrou 8 bytes de endereço de retorno.

```
1  entrada de asm_apply ......  resto 8     (o CALL empurrou o retorno)
2  pushq %rbp    ..............  resto 0     <- JA alinhado
3  subq  $8, %rsp  ............  resto 8     <- e aqui QUEBRAMOS
4  call  *%rax   ..............  o destino recebe resto 0    ERRADO
```

Listagem 19: A aritmética do alinhamento, passo a passo. O resto é sobre 16.

A função de destino esperava resto 8 na entrada e recebeu resto 0. Com isso ela calcula todos os seus deslocamentos internos oito bytes fora do lugar — e qualquer instrução `movaps` ou `movdqa`, que são cargas SSE de 16 bytes *alinhadas* e que o GCC emite à vontade em `-O2`, recebe um endereço que não é múltiplo de 16 e dispara `SIGSEGV`.

A correção é não fazer o `subq`: o `pushq %rbp` do prólogo já alinhou a pilha.

```
1  asm_apply:
2      pushq   %rbp
3      movq    %rsp, %rbp        # RSP = 0 (mod 16) — pronto para o CALL
4
5      movq    %rdi, %rax        # rax = endereço da função
6      movq    %rsi, %rdi        # rdi = x (1° arg da função destino)
7      call    *%rax             # chama fn(x) — resultado em rax
8
9      popq    %rbp
10     ret
```

Listagem 20: A versão corrigida, em `src/asm_course/07_calling_asm.s`.

Armadilha — o bug de alinhamento estoura em quem usa SSE, não em quem o quebrou

O sintoma era desconcertante. A pilha de chamadas mostrava o *crash dentro* da biblioteca `fmt`, quatro quadros acima do *assembly* — e a `fmt` estava perfeita:

```
1  #0  fmt::v10::detail::vformat_to<char>
2  #4  cpp_callback  (07_calling_conv.cpp:52)
3  #5  asm_apply     (07_calling_asm.s)        <- o culpado
```

Listagem 21: A pilha de chamadas do crash, como o depurador a mostrava.

Essa é a assinatura da classe inteira de bugs de alinhamento: o dano é causado por uma função e cobrado de outra, arbitrariamente distante, escolhida apenas por ser a primeira a executar uma instrução SSE alinhada. Quando um *crash* aponta para dentro de uma biblioteca madura e bem testada, a hipótese mais provável não é que a biblioteca esteja errada.

I.8.3 A promessa que o `rep movsb` não cumpre

O módulo 10 do mesmo curso demonstra as instruções de *string* do x86. Elas usam registradores *implícitos*: `rep movsb` copia `RCX` bytes do endereço em `RSI` para o endereço em `RDI`, incrementando os dois ponteiros a cada byte e zerando o contador ao fim.

O bloco de *assembly inline* declarava os três registradores como *entrada pura*:

```
// check-fontes: esquematico
asm volatile (
    "std\n\t"
    "rep movsb\n\t"
    "cld"
    :                                          // sem operandos de saida
    : "D"(&rev[7]), "S"(&arr[7]), "c"((size_t)8)    // ENTRADAS
    : "memory"
);
```

Listagem 22: A forma anterior — e errada. Ilustração; a versão corrigida está logo abaixo.

Declarar um registrador como entrada é uma **promessa ao compilador** de que o bloco não o altera. E o GCC confia na promessa: ele mantém outros valores vivos nesses registradores e os reutiliza depois do bloco, sem recarregá-los. O `rep movsb` modifica os três. O resultado era um ponteiro corrompido, e — de novo — o *crash* não aparecia no *assembly*, mas na linha seguinte a ele, no laço de verificação.

A correção é declarar os três como *read-write* com o modificador `+`, o que na sintaxe do GCC exige que sejam operandos de *saída* ligados a variáveis. Os valores finais não interessam a ninguém; o que interessa é o compilador *saber* que eles mudaram:

```
void*       d = &rev[7];
const void* o = &arr[7];
std::size_t n = 8;

asm volatile (
    "std\n\t"                 // DF=1: decrementar apos cada byte
    "rep movsb\n\t"           // copia 8 bytes de tras para frente
    "cld"                     // OBRIGATORIO: restaura DF=0 antes de retornar
    : "+D"(d), "+S"(o), "+c"(n)     // read-write: o bloco os ALTERA
    :
    : "memory"
);
```

Listagem 23: A versão corrigida, em `src/asm_course/10_string_ops.cpp`.

Armadilha — não se pode listar no *clobber* um registrador que já é operando

A reação natural de quem descobre o problema é acrescentar `"rdi"`, `"rsi"` e `"rcx"` à lista de *clobbers* — a terceira seção do bloco, que declara o que o *assembly* destrói. O GCC recusa: um registrador não pode ser operando e *clobber* ao mesmo tempo, porque as duas declarações se contradizem. O modificador

`+` existe exatamente para esse caso. A alternativa, quando os ponteiros finais não são necessários, é amarrar entrada e saída pelo índice: `"=D"(d)`, `"=S"(s)`, `"=c"(c)` nas saídas e `"0"(...)`, `"1"(...)`, `"2"(...)` nas entradas.

I.8.4 Dois bugs silenciosos numa função de imprimir traço

O `http-client` tem uma função de dez linhas que imprime um separador antes de cada seção. Ela abrigava dois defeitos independentes, e os dois são exemplares do tipo de erro que este relatório persegue.

O primeiro: `std::string(n, '–')`. O caractere – é o U+2500 da tabela Unicode, que em UTF-8 ocupa **três bytes**. Entre aspas *simples* ele não é um `char` — um `char` guarda um byte só. Ele vira uma *multi-character constant*: um `int` cujo valor a implementação escolhe, truncado depois para `char`. O GCC avisa, com `-Wmultichar` e `-Woverflow` (*"changes value from 14849152 to -128"*), e o separador saía como uma sequência de bytes inválidos no terminal.

O segundo: `40 - title.size()`. O método `size()` devolve `std::size_t`, que é um inteiro **sem sinal**. Numa expressão que mistura um literal `int` com um valor sem sinal, o C++ converte os dois para o tipo sem sinal — e a subtração passa a ser feita em aritmética modular. Com um título de 41 caracteres o resultado não é −1: é 18 446 744 073 709 551 615, e a construção do `std::string` tentaria alocar dezesseis exabytes.

```
1  const std::size_t largura = title.size() < 40 ? 40 - title.size() : 0;
2
3  std::string linha;
4  linha.reserve(largura * 3);                 // 3 bytes por caractere
5  for (std::size_t i = 0; i < largura; ++i) linha += "–";   // string, nao char
```

Listagem 24: A versão corrigida, em `src/http-client/main.cpp`.

Armadilha — subtração entre inteiros sem sinal não produz negativos

A regra vale para toda a linguagem, não só para `size()`: se qualquer operando de uma expressão aritmética for sem sinal e do mesmo tamanho ou maior, o outro é convertido para sem sinal, e um resultado que "deveria" ser negativo dá a volta pelo maior valor representável. O laço `for (size_t i = v.size() - 1; i >= 0; --i)` nunca termina pela mesma razão — `i >= 0` é sempre verdadeiro para um tipo sem sinal, e o compilador pode até avisar disso, mas só se os avisos estiverem ligados e o alvo for compilado.

I.8.5 Uma API que mudou de forma

O `rabbitmq` não compilava por um motivo diferente dos anteriores: a biblioteca AMQP-CPP mudou, e o código do subprojeto ainda falava a forma antiga. A tentativa era registrar os eventos de ciclo de vida como *lambdas* no objeto de conexão:

```
1  // check–fontes: esquematico
2  AMQP::TcpConnection connection(&handler, address);
3  connection.onConnected([]{ /* ... */ });            // nao existe
4  connection.onError([](const char* m){ /* ... */ });
5
6  // error: cannot convert 'main()::<lambda()>' to 'AMQP::TcpState*'
```

Listagem 25: A forma que não existe — e a mensagem de erro que ela produz.

A mensagem confunde porque `TcpConnection` *tem* métodos com esses nomes — só que eles pertencem à máquina de estados interna e recebem outra coisa. Os eventos de ciclo de vida da conexão pertencem ao **handler**, e a forma de reagir a eles é herdar e sobrescrever métodos virtuais:

```
class Handler : public AMQP::LibBoostAsioHandler {
public:
    Handler(boost::asio::io_context& io, std::string rotulo)
        : AMQP::LibBoostAsioHandler{io}, rotulo_{std::move(rotulo)} {}

    void onConnected(AMQP::TcpConnection* c) override {
        AMQP::LibBoostAsioHandler::onConnected(c);   // a base faz trabalho real
        fmt::print("[{}] onConnected -- TCP estabelecido (AMQP ainda NAO)\n",
                   rotulo_);
    }

    void onReady(AMQP::TcpConnection* c) override {
        AMQP::LibBoostAsioHandler::onReady(c);
        pronto_ = true;
    }
};
```

Listagem 26: A forma correta (resumo de `src/rabbitmq/handler.hpp`).

O episódio ensina duas coisas além do conserto. A primeira é a distinção entre os dois estilos de *callback*, e quando cada um cabe: *lambda por evento* serve quando cada chamada tem o seu próprio retorno independente — é assim que as operações de canal do AMQP-CPP funcionam (`declareQueue`, `consume`); *herança com virtual* serve quando há um conjunto fixo e conhecido de eventos que pertencem ao mesmo objeto e compartilham estado. A segunda é o ciclo de vida em si:

```
onAttached   objeto construido -- ainda nao ha socket
onConnected  TCP estabelecido -- ainda NAO ha login AMQP
onReady      handshake AMQP concluido   <- so AQUI da para publicar
onError      erro fatal; a conexao vai fechar
onClosed     fechamento limpo
onDetached   pode destruir o objeto
```

Listagem 27: Os seis estágios da conexão, transcritos de `src/rabbitmq/`.

Armadilha — publicar em `onConnected` é cedo demais

"Publiquei e a mensagem sumiu" é o relato mais comum de quem começa com AMQP, e a causa quase sempre é confundir `onConnected` com `onReady`. O primeiro significa apenas que o *socket* TCP abriu; o *handshake* do protocolo AMQP, com autenticação e negociação de canal, ainda não aconteceu, e o canal em que se publicaria ainda não existe.

I.8.6 O demo que só funcionava na primeira execução

O último caso não é um erro de linguagem: é um erro de projeto de exemplo, e merece figurar porque a lição atravessa qualquer tecnologia.

O `database` cria uma tabela com `CREATE TABLE IF NOT EXISTS` e insere alguns produtos. A coluna `name` tem restrição `UNIQUE`. Na primeira execução tudo funciona; na segunda, o `IF NOT EXISTS` preserva as linhas anteriores e o primeiro `INSERT` bate na restrição:

```
DbError: UNIQUE constraint failed: products.name
```

Listagem 28: Saída de `src/database/` antes da correção, na segunda execução.

O programa saía com código 1 e um *demo* aparentemente quebrado. A correção é uma linha, e o conceito que ela encarna tem nome: **idempotência** — a propriedade de uma operação que, executada uma vez ou dez, deixa o sistema no mesmo estado.

```
conn_.execute("DROP TABLE IF EXISTS products");

conn_.execute(R"(
    CREATE TABLE IF NOT EXISTS products (
        id      INTEGER PRIMARY KEY AUTOINCREMENT,
        name  TEXT    NOT NULL UNIQUE,
        price REAL    NOT NULL CHECK(price >= 0),
        stock INTEGER NOT NULL DEFAULT 0
    )
)");
```

Listagem 29: A linha que torna o *demo* idempotente, em `src/database/main.cpp`.

Regra — em produção o instrumento é migração, nunca `DROP`

Apagar a tabela é aceitável num programa didático cujo banco vive em `tmp/sqlite/` e existe para ser recriado. Em produção nunca se faz isso: apagar a tabela apaga os dados. Lá o instrumento é a **migração** — uma sequência versionada de scripts `ALTER TABLE`, cada um aplicado uma única vez e registrado numa tabela de controle, de modo que o esquema evolua sem perder o conteúdo. Flyway, Liquibase e Alembic são as ferramentas usuais.

I.8.7 A lição, enunciada

Seis defeitos, quatro deles latentes havia meses, todos no mesmo lugar: nos alvos que o *build* não construía. Nenhum foi encontrado por revisão de código, e nenhum teria sido encontrado por mais revisão de código.

Regra — um alvo que não é construído é um alvo que apodrece

Código que não é compilado não é código: é texto com aparência de código. Ele não sente a atualização da biblioteca, não sente a troca de compilador, não sente o `#include` removido três arquivos acima — e acumula defeitos em silêncio, num ritmo proporcional ao tempo em que ficou fora. Se um subprojeto não pode ser construído, o problema é dele: conserte-o ou apague-o. Não o esconda atrás de um `#`.

O custo dessa decisão é real e vale declará-lo: o *build* completo passou a exigir Qt 6, SFML, Bullet, OpenCV, ONNX Runtime, AWS SDK, AMQP-CPP, SOCI e JSBSim instalados, e demora mais. O que se compra com isso é a garantia de que os 98 binários que `dist/bin/` contém foram, todos, compilados e ligados com os avisos ligados, nesta máquina, nesta tarde. É a diferença entre um repositório que *tem* 57 subprojetos e um repositório que *sabe* que tem 57 subprojetos.

I.9 Sanitizers

Vários subprojetos mexem com tempo de vida, *layout* de objetos e concorrência — justamente as áreas em que o compilador cala e o programa mente. Para eles, o repositório recomenda uma passada sob *sanitizer*, que não exige mudança nenhuma no código:

```
meson setup build-asan -Db_sanitize=address,undefined .
meson compile -C build-asan oop
./build-asan/src/oop/oop
```

Listagem 30: Compilando um subprojeto com ASan e UBSan.

O ThreadSanitizer (`-Db_sanitize=thread`) é o instrumento equivalente para o Seção VI: ele é o único detector prático de *data race*, e detecta corridas que *não* se manifestaram naquela execução — que são as que interessam.

II Fundamentos da linguagem

Este é o capítulo mais longo do relatório, e o mais rentável: quase tudo o que vem depois — containers, *templates*, padrões de projeto, concorrência, integração com bibliotecas — é aplicação do que está aqui. O assunto comum às oito seções é um só: **o que o compilador faz com o seu código**, e onde ele faz algo diferente do que a leitura ingênua sugere.

A ordem interna reproduz a Etapa 1 da trilha do repositório, com duas inserções. `src/operators/` entra logo depois de `src/value_semantics/`, porque sobrecarregar um operador corretamente depende de saber quando um argumento é copiado e quando é movido. E `src/file_io/` entra logo depois de `src/smart-pointers/`, porque um *stream* é o primeiro recurso RAII que se encontra na prática — e o único da biblioteca padrão que sinaliza erro por *bit de estado* em vez de exceção, o que faz dele a ponte natural entre a Seção II.5 e a Seção II.8.

Por que — por que começar pelo que o compilador cala

As tabelas de armadilhas que abrem quase todas as seções deste capítulo não são folclore. Elas têm em comum a propriedade que torna o C++ difícil de aprender sozinho: **o programa compila, roda e dá a resposta certa** — até o dia em que não dá. Um destrutor vazio não gera *warning*; um `std::vector<Base>` não gera *warning*; um `const_cast` seguido de escrita não gera *warning*. Quem só lê a referência da linguagem nunca encontra esses casos, porque a referência descreve o que é válido, não o que é ilusório.

II.1 O mapa da sintaxe

O subprojeto `src/syntax/` é o mapa: toda construção da linguagem — declaração, seleção, iteração, função, tipo, classe, *namespace*, *template*, exceção — num só executável, comentada. É o subprojeto que se lê primeiro e ao qual se volta como referência; onde um tópico tem um subprojeto dedicado, ele mostra a *forma* e aponta para lá.

II.1.1 Declarações e inicialização

Toda declaração em C++ tem a mesma forma — especificadores, declarador, inicializador — e o inicializador é onde mora a primeira armadilha. Existem seis maneiras de inicializar uma variável, e elas não são equivalentes:

```
1  int a = 42;      // copy-init
2  int b(42);       // direct-init
3  int c{42};       // list-init       <- a preferida
4  int d = {42};    // copy-list-init
5  int e{};         // value-init      -> ZERO garantido
6  int f;           // default-init    -> LIXO indeterminado (ler é UB)
```

Listagem 31: As seis formas de inicializar, e o que cada uma garante.

A forma com chaves venceu porque **proíbe *narrowing***: a conversão que perde informação deixa de ser silenciosa e passa a ser erro de compilação.

```
1  int t = 3.14159;   // compila, trunca em silêncio
2  int s{3.14159};    // erro de compilação: narrowing conversion
```

Listagem 32: A diferença de uma chave.

Armadilha — com containers, {} muda o *significado*

A preferência por chaves tem uma exceção que dói. Para os containers da biblioteca padrão existe um construtor que recebe `std::initializer_list`, e ele *vence* os demais na resolução de sobrecarga: `std::vector<int> v(3,0)` constrói três zeros; `std::vector<int> v{3,0}` constrói dois elementos, 3 e 0. As duas linhas compilam e nenhuma gera *warning*.

`const` e ponteiros. A regra prática é ler a declaração da direita para a esquerda; o que muda é qual dos dois — o alvo ou o ponteiro — está congelado (Tabela 7).

Tabela 7: As três combinações de `const` com ponteiro.

Declaração	`*p = x`	`p = &y`
`const int* p`	não	sim
`int* const p`	sim	não
`const int* const p`	não	não

Duração e visibilidade. A *storage class* decide quanto tempo o objeto vive; o *linkage* decide quem enxerga o nome. As duas noções são independentes e costumam ser confundidas.

Tabela 8: *Linkage* das construções que mais aparecem em cabeçalhos.

Construção	*Linkage*
`int x;` global	externa
`static int x;` / `namespace { int x; }`	interna
`inline int x = 1;` (C++17)	externa, uma instância
`const int x = 1;` global	**interna** — a surpresa comum

A última linha é a que pega: uma constante global em um cabeçalho não viola a ODR porque cada unidade de tradução ganha a sua própria cópia — o que também significa que o endereço dela é diferente em cada `.cpp`. O `inline` do C++17 é o que permite ter constante em cabeçalho com *uma* instância só.

II.1.2 Controle de fluxo

São seis construções — `if`, `switch`, `while`, `do-while`, `for`, `range-for` — e três saltos. O que o C++ moderno acrescentou não foi construção nova: foi **escopo menor**.

```
1 if (auto it = m.find(k); it != m.end()) { /* usa it */ }      // C++17
2 for (auto f = fn(); const auto& x : v) { /* usa f e x */ }    // C++20
```

Listagem 33: O *init-statement*: a variável morre onde deixa de fazer sentido.

`if constexpr` é diferente de tudo o mais nessa lista: ele não escolhe um caminho em tempo de execução, ele **descarta um galho na compilação**. O galho descartado nem precisa ser válido para aquele tipo — é o que torna possível escrever uma função genérica que chama `.size()` só quando o tipo tem `.size()`.

Regra — `switch` sobre `enum class` não leva `default`

Com um `enum` escopado e sem `default`, acrescentar um enumerador novo faz o compilador apontar todos os `switch` que deixaram de ser exaustivos. Escrever `default:` desliga esse alarme para sempre. A ausência do `default` é uma *feature*, não um descuido — e é a forma mais barata que a linguagem oferece de verificação de exaustividade.

II.1.3 Funções

A tabela de passagem de parâmetros (Tabela 9) resolve a maioria das decisões de assinatura. Ela reaparece, refinada, na Seção II.3, quando a discussão incluir *move*.

Tabela 9: Como passar um parâmetro, e o que cada forma comunica.

Forma	Copia?	Modifica?	Aceita nulo?	Quando usar
`T`	sim	não	não	tipos baratos, *sink*
`T&`	não	sim	não	saída, entrada-e-saída
`const T&`	não	não	não	**leitura — o padrão**
`T*`	não	sim	sim	opcional, API C
`T&&`	não	consome	não	*move*, *forwarding*

A resolução de sobrecarga tenta, nesta ordem: correspondência exata, promoção, conversão padrão, conversão definida pelo usuário, reticências. E o **tipo de retorno não entra na assinatura** — `int f(int)` e `bool f(int)` colidem.

As variádicas têm duas formas, e a distância entre elas mede o que o C++ ganhou sobre o C:

```
1  int soma(int n, ...) ;                          // sem tipos, contagem manual, UB fácil
2
3  template <typename... A>
4  auto soma(A... a) { return (a + ...); }    // fold expression: tipado, compile-time
```

Listagem 34: Variádica à moda C e à moda C++.

II.1.4 Tipos definidos pelo usuário

`struct` e `class` são a mesma construção: muda só o acesso padrão. As distinções que importam de verdade estão em outro lugar (Tabelas 10 and 11).

Tabela 10: `enum` contra `enum class`.

	`enum`	`enum class`
escopo	vaza para fora	exige `Cor::Verde`
converte para `int`	sozinho	só com `static_cast`
tipo subjacente	escolha do compilador	sua (`: uint8_t`)
colisão de nomes	acontece	impossível

Do C++20 vieram os *designated initializers*, a resposta tardia à falta de argumentos nomeados: `Config c{.host = "api", .porta = 443, .tls = true}`. A restrição a conhecer: a ordem dos campos escritos tem de seguir a ordem de *declaração* dos membros — o C é mais permissivo aqui, e código portado de C pode não compilar.

Tabela 11: `union` contra `std::variant`.

	`union`	`std::variant`
quem guarda a etiqueta	você	ele
ler o membro errado	UB (comportamento indefinido) silencioso	`std::bad_variant_access`
tipos não triviais	não	sim

II.1.5 Classes: a lista de inicialização não é açúcar sintático

```
1 Classe(const T& r) : membro(r)  { }      // 1 construção
2 Classe(const T& r) { membro = r; }       // 1 construção padrão + 1 atribuição
```

Listagem 35: Inicializar e atribuir não são a mesma coisa.

A segunda forma exige que o membro seja construível por padrão, e faz o dobro do trabalho. Para membros `const` ou de tipo referência ela nem é uma opção: eles só podem ser *inicializados*.

Armadilha — a ordem da lista escrita é ignorada

Os membros são construídos na ordem de **declaração** na classe, não na ordem em que aparecem na lista de inicialização. Escrever a lista fora de ordem compila, roda na ordem de declaração mesmo assim, e — se um membro inicializar a partir de outro — lê um membro ainda não construído. O GCC avisa com `-Wreorder`, que não vem em `-Wall`.

II.1.6 Organização: *namespaces*, ADL e o pré-processador

A busca de nome do C++ considera também os *namespaces* dos *argumentos* da chamada — é a *Argument-Dependent Lookup*, e é o que faz `std::cout << x` e o idioma `using std::swap; swap(a,b)` funcionarem sem qualificação explícita. É também o mecanismo que torna os *hidden friends* da Seção II.4 eficientes.

Sobre macros, a regra é curta: quase toda uma tem substituto melhor (Tabela 12). Sobram os casos que *exigem* manipulação de texto — guardas de inclusão, `__LINE__`, portabilidade condicional.

Tabela 12: O que usar no lugar de cada macro.

Macro	Substituto
`#define PI 3.14`	`constexpr double pi = 3.14;`
`#define MAX(a,b)`	`template <class T> T max(T, T)`
`#define DEBUG_ON`	`constexpr bool` + `if constexpr`

Regra — nunca `using namespace` no escopo global de um cabeçalho

A diretiva impõe a decisão a *todo* arquivo que incluir aquele cabeçalho, direta ou indiretamente, e a consequência típica — uma ambiguidade nova que aparece em um `.cpp` que ninguém tocou — é praticamente impossível de diagnosticar a partir da mensagem de erro.

II.1.7 Expressões: precedência não é ordem de avaliação

Duas armadilhas de expressão aparecem em produção com frequência desproporcional ao quanto são discutidas.

A primeira é a comparação com sinais misturados. Numa comparação entre `int` e `unsigned`, o `int` é convertido — e -1 vira $4\,294\,967\,295$:

```
int a = −1;  unsigned b = 1;
a < b        // FALSE
```

Listagem 36: A comparação que inverte de resultado.

É daí que vem o `-Wsign-compare` ao escrever `int i; i < v.size()`, e é por isso que o índice de um laço sobre container deve ser `std::size_t` — ou, melhor, não existir, substituído por um *range-for*.

A segunda é confundir precedência com ordem de avaliação. `1 « 2 + 3` é `1 « 5`, porque `+` tem precedência maior que `«`. E, independentemente disso, a ordem em que os operandos de uma expressão são avaliados é, em geral, não especificada.

II.1.8 As armadilhas silenciosas, reunidas

A Tabela 13 recolhe o núcleo do argumento do subprojeto `src/syntax/`: cada linha é uma construção que compila, e a última coluna diz se você fica sabendo.

Tabela 13: Construções que não significam o que parecem — e se o compilador avisa. Cada uma tem demonstração executável em `src/syntax/`.

Escrito assim	O que acontece	Avisa?
`Widget w();`	declara uma *função*	não
`int x;` local	lixo indeterminado; ler é UB (comportamento indefinido)	não
`if (a) f(); g();`	`g()` roda sempre	às vezes
`052`	é octal, vale 42	não
`#define Q(x) x*x`, `Q(1+2)`	vira `1+2*1+2` $= 5$	não
`a < b`, `int` e `unsigned`	-1 vira $4\,294\,967\,295$	`-Wsign-compare`
`int t = 3.99;`	trunca em silêncio	não
`delete p` para `new T[n]`	UB (comportamento indefinido)	não
`[=]` num método	captura `this` por ponteiro	não
`case` sem `break`	cai para o seguinte	`-Wimplicit-fallthrough`

II.2 O sistema de tipos e as conversões explícitas

O subprojeto `src/casts/` é o menor pré-requisito da trilha, e ensina uma coisa que a sintaxe esconde: os quatro *casts* do C++ **não são quatro sinônimos com nomes diferentes**. Cada um é uma operação distinta, com verificações distintas e riscos distintos — e o *cast* à moda C é o apagador dessa distinção.

II.2.1 Os quatro, em ordem de segurança

`static_cast` — **o padrão.** Pede ao compilador que confira, *em compilação*, se a conversão faz sentido. Se não fizer, o programa não compila — que é o comportamento desejável. Serve para conversões numéricas explícitas, para subir na hierarquia (sempre seguro) e para descer quando você *sabe* o tipo real. Não serve entre tipos não relacionados, e é justamente por recusá-los que ele é o *cast* padrão.

Tabela 14: Os quatro *casts* do C++: o que fazem e o que verificam.

Cast	Faz	Verifica
`static_cast`	conversões relacionadas	compilação
`dynamic_cast`	desce na hierarquia	**execução** (exige `virtual`)
`const_cast`	remove/adiciona `const`	nada — escrever pode ser UB (comportamento indefinido)
`reinterpret_cast`	reinterpreta os bits	nada

`dynamic_cast` — **descer com verificação.** Usa RTTI para conferir em tempo de execução se o objeto realmente é do tipo pedido. Exige que a base tenha ao menos uma função virtual, e falha de duas maneiras diferentes conforme a forma usada: com ponteiro devolve `nullptr`; com referência lança `std::bad_cast`.

```
Animal* animal = new Cachorro();

if (Cachorro* c = dynamic_cast<Cachorro*>(animal)) {   // nullptr se falhar
    c->buscar();
}

try {
    Cachorro& cc = dynamic_cast<Cachorro&>(*animal);   // lança se falhar
    cc.buscar();
} catch (const std::bad_cast& e) {
    fmt::print("não é um Cachorro: {}\n", e.what());
}
```

Listagem 37: As duas formas de falha do `dynamic_cast`.

O custo não é desprezível: `dynamic_cast` percorre a árvore de herança em tempo de execução e não é $O(1)$. A Seção II.7 traz a medição, e a Seção II.7.5 traz as alternativas — que, quase sempre, se resumem a uma função virtual que não foi escrita.

`const_cast` — **a única aplicação legítima.** Remover `const` é aceitável em exatamente um cenário: atravessar uma interface antiga que esqueceu de declarar o parâmetro como `const`, sabendo que ela não modifica o objeto.

Armadilha — modificar um objeto genuinamente `const` é sempre UB (comportamento indefinido)

```
const int x = 42;                 // pode estar em memória somente-leitura
int* p = const_cast<int*>(&x);
*p = 100;                         // UB: pode falhar, pode ser ignorado,
                                  // pode "funcionar" e mudar em -O2
```

A regra é sobre a *origem* do objeto, não sobre o tipo do ponteiro: se ele foi declarado `const`, escrever nele é UB (comportamento indefinido) independentemente de quantos *casts* houver no caminho.

`reinterpret_cast` — **e por que `std::bit_cast` o substitui.** Tratar os bytes de um tipo como se fossem outro é legítimo em interoperação com C, em aritmética de endereços e ao inspecionar a representação. O que não é legítimo é *ler pelo tipo errado*: acessar um `float` através de um `int*` viola *strict aliasing*, e o compilador tem permissão para supor que os dois ponteiros nunca apontam para a mesma memória.

```
float f = 1.0f;

int* i = reinterpret_cast<int*>(&f);
*i = 42;                                  // UB: viola strict aliasing

int bits;
```

```cpp
std::memcpy(&bits, &f, sizeof(bits));      // correto, e o compilador otimiza
auto bits2 = std::bit_cast<std::uint32_t>(f);   // C++20: correto e constexpr
```

Listagem 38: A forma errada e as duas formas corretas de ver os bits de um `float`.

II.2.2 Por que o *cast* à moda C é o pior dos cinco

`(Tipo)valor` tenta, em ordem, cinco combinações — `static_cast`, `const_cast`, `reinterpret_cast` e duas composições. Quem escolhe é o compilador, e o programador não fica sabendo qual foi.

```cpp
const int x = 42;
int* p1 = (int*)&x;                  // faz const_cast em silêncio
int* p2 = const_cast<int*>(&x);      // a intenção fica escrita
```

Listagem 39: O mesmo UB (comportamento indefinido), uma vez escondido e outra vez visível.

Há um segundo argumento, mais prático que teórico: `static_cast` é *pesquisável*. Auditar um projeto atrás de todas as conversões perigosas é um `grep` quando os *casts* têm nome, e é impossível quando eles são um par de parênteses. O compilador ajuda a manter a disciplina com `-Wold-style-cast`.

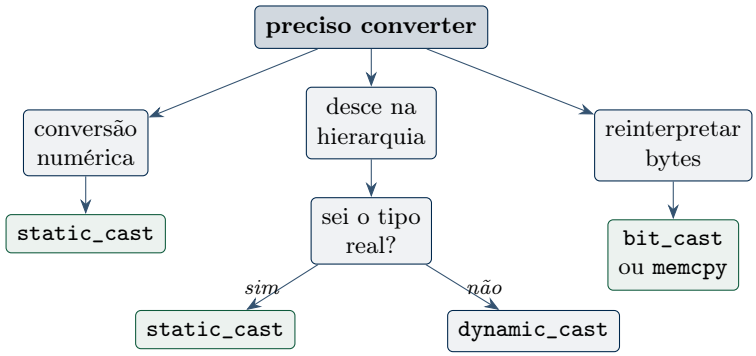


Figura 2: A árvore de decisão de `src/casts/`. Nenhum galho termina em *cast* à moda C, e apenas um termina em `dynamic_cast` — que, ao aparecer numa cadeia de `if`, quase sempre indica uma função virtual faltando.

II.3 Semântica de valor: cópia, *move* e elisão

O subprojeto `src/value_semantics/` sustenta todos os outros. `std::move` aparece em dezenas de arquivos do repositório; é aqui que se explica o que ele realmente faz — um `static_cast`, e nada mais.

O que torna esta seção urgente é a natureza dos erros que ela previne: nenhum deles gera *warning*, e todos custam desempenho ou correção (Tabela 15). O programa do subprojeto roda com contadores instrumentados, de modo que cada afirmação da tabela é *medida*, não alegada.

Tabela 15: Cinco erros de semântica de valor. Nenhum é apontado pelo compilador com os avisos padrão.

Erro	Consequência
declarar `~T() {}` vazio	suprime os *moves* — todo "*move*" vira cópia
esquecer `noexcept` no *move ctor*	`std::vector` passa a copiar ao realocar
`return std::move(x)`	desliga o NRVO: de zero operações para uma
`std::move` onde cabia `std::forward`	rouba o objeto do chamador sem avisar
repassar `T&&` sem `forward`	vira *lvalue* no caminho; o *move* se perde

II.3.1 Categorias de valor

Toda expressão tem um tipo **e** uma categoria de valor. A categoria responde a duas perguntas independentes: *tem identidade?* e *pode ser movida?* (Figura 3).

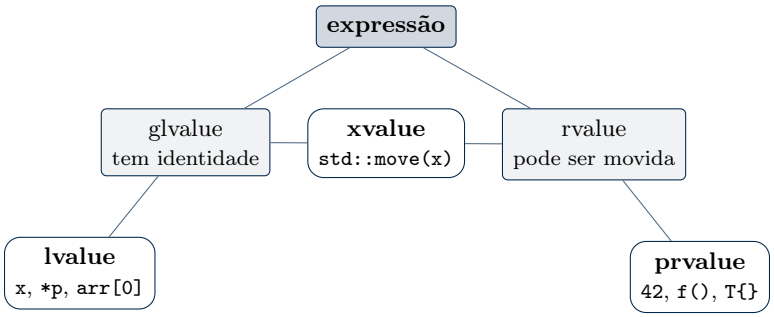


Figura 3: As categorias de valor. O *xvalue* é o único que pertence aos dois ramos: tem identidade *e* pode ser movido — é exatamente o que `std::move` produz.

Armadilha — tudo o que tem nome é *lvalue*

```
void recebe(std::string&& s) {
    alvo(s);                 // → escolhe o overload de LVALUE
    alvo(std::move(s));      // → escolhe o overload de rvalue
}
```

O parâmetro `s` foi *declarado* como referência a *rvalue*, mas a *expressão* `s` tem nome — e tudo o que tem nome é *lvalue*. É por isso que repassar um `T&&` sem `std::move`/`std::forward` produz uma cópia. A regra existe por segurança: sem ela, usar o parâmetro duas vezes esvaziaria o objeto na primeira.

`std::move` é um *cast*. A implementação cabe em três linhas, e nenhum byte é copiado ao chamá-la:

```
template <typename T>
constexpr std::remove_reference_t<T>&& meu_move(T&& t) noexcept {
    return static_cast<std::remove_reference_t<T>&&>(t);
}
```

Listagem 40: O que `std::move` faz, escrito por extenso.

O move de fato só acontece quando alguém *constrói ou atribui* a partir do *rvalue* resultante. `std::move` apenas autoriza.

`decltype` preserva a categoria. O detalhe dos parênteses surpreende, mas é consistente: `decltype(x)` pergunta pelo tipo *declarado* de `x`, enquanto `decltype((x))` pergunta pelo tipo da *expressão* `(x)`, que é um *lvalue* e portanto `int&`.

II.3.2 Rule of 0 / 3 / 5

Um tipo tem seis operações que o compilador pode gerar sozinho: construtor padrão, destrutor, construtor e atribuição de cópia, construtor e atribuição de *move*. As regras de geração são interligadas, e a consequência prática é a armadilha mais cara do capítulo.

Armadilha — um destrutor vazio custa todos os *moves*

```
struct ComDestrutor {
    std::vector<int> dados;
    ~ComDestrutor() {}      // isto sozinho suprime os DOIS moves
};
```

Declarar o destrutor — ainda que vazio — impede a geração implícita do construtor e da atribuição de *move*. O código continua compilando e rodando; só fica mais lento, porque todo "*move*" cai silenciosamente na cópia. Saída real do subprojeto:

```
ComDestrutor  — move real disponível? NÃO — cai silenciosamente na cópia
SemDestrutor  — move real disponível? sim
```

Note que `std::is_move_constructible_v` responderia `true` nos dois casos, porque `const T&` também aceita *rvalue*. O discriminador honesto é `std::is_nothrow_move_constructible_v`.

`noexcept` decide se o `vector` move ou copia. Ao realocar, `std::vector` precisa oferecer a garantia forte: se algo falhar no meio da transferência, o vetor original tem de continuar íntegro. Um construtor de *move* que pode lançar não permite essa promessa — depois de mover metade dos elementos, não há como voltar atrás. Por isso o container consulta `std::move_if_noexcept` e, na dúvida, copia:

```
MoveNoexcept    → 7 moves, 0 cópias
MovePodeLancar  → 0 moves, 7 cópias
```

Listagem 41: Saída de `src/value_semantics/`: a única diferença entre os dois tipos é a palavra `noexcept`.

Copy-and-swap. Uma única atribuição, recebida **por valor**, cobre cópia e *move* de uma vez — e ganha de graça a garantia forte e o tratamento correto da autoatribuição:

```
class Cofre {
public:
    explicit Cofre(std::size_t n) : tamanho_(n), dados_(new int[n]{}) {}
    ~Cofre() { delete[] dados_; }

    Cofre(const Cofre& o) : tamanho_(o.tamanho_), dados_(new int[o.tamanho_]) {
        for (std::size_t i = 0; i < tamanho_; ++i) dados_[i] = o.dados_[i];
    }

    Cofre(Cofre&& o) noexcept : tamanho_(o.tamanho_), dados_(o.dados_) {
        o.dados_ = nullptr;
        o.tamanho_ = 0;
    }

    // A única atribuição declarada — cobre os dois casos.
    Cofre& operator=(Cofre outro) noexcept {
        trocar(*this, outro);
        return *this;
    }

    friend void trocar(Cofre& a, Cofre& b) noexcept {
        std::swap(a.tamanho_, b.tamanho_);
        std::swap(a.dados_,   b.dados_);
    }

private:
    std::size_t tamanho_ = 0;
    int*        dados_   = nullptr;
};
```

Listagem 42: *Copy-and-swap* em `src/value_semantics/demo_rule_of_five.hpp` (condensado).

Chamada com *lvalue*, o parâmetro é construído por cópia; com *rvalue*, por *move*. Um corpo para os dois casos.

Regra — declare apenas a versão por valor

Ter `operator=(T)` e `operator=(T&&)` ao mesmo tempo é ambíguo para argumentos *rvalue*, e o programa não compila. O *copy-and-swap* é uma escolha exclusiva, não uma adição.

II.3.3 Elisão de cópia

Desde o C++17, retornar um *prvalue* não cria temporário nenhum: o objeto é construído diretamente no destino. Isso deixou de ser otimização e passou a ser *regra da linguagem* — e a prova é um tipo que não pode ser copiado nem movido:

```
1  struct Imovel {
2      int v;
3      explicit Imovel(int x) : v(x) {}
4      Imovel(const Imovel&)            = delete;
5      Imovel(Imovel&&)                 = delete;
6      Imovel& operator=(const Imovel&) = delete;
7      Imovel& operator=(Imovel&&)      = delete;
8  };
9  Imovel faz_imovel() {
10     return Imovel(42);          // compila: nenhuma cópia/move é sequer considerada
11 }
```

Listagem 43: A elisão obrigatória, em `src/value_semantics/demo_copy_elision.hpp`.

Armadilha — `return std::move(local)` é uma pessimização

A NRVO — elisão para uma variável local nomeada — continua sendo opcional e depende de o compilador enxergar o *nome* da variável no `return`. Envolvê-lo em `std::move` produz um *xvalue*, que já não é um nome: a elisão deixa de se aplicar e o que era zero operação vira um *move*.

Versão	Cópias	*Moves*
`return local;` (NRVO)	0	0
`return std::move(local);`	0	**1**

O GCC avisa com `-Wpessimizing-move`. Retornar uma variável local já trata a expressão como *rvalue* quando a elisão não se aplica — por exemplo quando há dois candidatos em ramos diferentes — de modo que o *fallback* natural já é o *move*, sem ajuda.

II.3.4 *Perfect forwarding*

Repassar um argumento preservando tipo, constância e categoria exige duas peças que só funcionam *juntas*: o parâmetro `T&&` deduzido e o `std::forward<T>` no repasse.

```
1  template <typename T>
2  void repassa(T&& arg) {          // 1. referência universal (T é DEDUZIDO)
3      alvo(std::forward<T>(arg)); // 2. forward condicional
4  }
```

Listagem 44: As duas peças do *perfect forwarding*.

O mecanismo é o *colapso de referências*: quando `T` é deduzido em `T&&`, um argumento *lvalue* de tipo `X` deduz `T = X&` e o parâmetro colapsa para `X&`; um *rvalue* deduz `T = X` e o parâmetro fica `X&&`. Das quatro combinações possíveis, apenas `&& &&` sobrevive como *rvalue*.

Armadilha — nem todo `T&&` é referência universal

```
1  void f(std::string&&);                              // rvalue ref pura
2  template<class T> void f(std::vector<T>&&);         // rvalue ref pura: T não está
3                                                      // na posição do parâmetro
4  template<class T> void f(T&&);                      // referência universal
```

A forma só é universal quando o parâmetro é exatamente `T&&` com `T` deduzido naquela posição. Nos outros dois casos, aplicar `std::forward` não faz sentido — e aplicar `std::move` está correto.

Regra — `move` em tipo concreto, `forward` em tipo deduzido

`std::move` é incondicional: rouba sempre. `std::forward<T>` é condicional: rouba só se o chamador ofereceu um *rvalue*. Trocar um pelo outro num *template* esvazia silenciosamente o objeto de quem chamou — o programa continua correto para o primeiro uso e falha no segundo.

II.3.5 *Move-only, ref-qualifiers e sink parameters*

Alguns recursos não podem ser duplicados: descritor de arquivo, *lock*, *socket*, *thread*. Para eles, copiar não é caro — é *errado*, e a forma de dizer isso é `= delete`, que torna o erro incompilável em vez de detectável em produção.

Os *ref-qualifiers* permitem que um método pergunte se `*this` é descartável, e portanto se pode roubar o próprio conteúdo:

```
1  const std::string& texto() const&  { return texto_; }              // lvalue: empresta
2  std::string        texto() &&      { return std::move(texto_); }  // rvalue: rouba
```

Listagem 45: O mesmo método, duas implementações conforme a categoria de `*this`.

Servem também para *proibir*: declarar `T& operator[](int) && = delete;` impede devolver uma referência para dentro de um temporário que morre na mesma expressão — um *dangling* que, sem isso, compila.

Quando a função vai **guardar** uma cópia, a assinatura correta é por valor seguida de `std::move` — o *sink parameter*:

```
1  void adicionar(std::string nome) {          // por valor
2      nomes_.push_back(std::move(nome));      // o move é obrigatório aqui
3  }
```

Listagem 46: Um *overload* que atende *lvalue* e *rvalue* de forma ótima.

Com *lvalue* há uma cópia — inevitável, porque o chamador quer manter o seu objeto; com *rvalue* há um *move*, que é barato. A alternativa de escrever `const T&` e `T&&` explodiria em 2^n *overloads* para n parâmetros.

II.4 Sobrecarga de operadores

O subprojeto `src/operators/` responde à pergunta de quando fazer um tipo se comportar como um tipo embutido — e, mais importante, quando não fazer. A regra que resolve a maioria das decisões é uma só:

Tabela 16: A tabela de assinaturas, agora completa.

Intenção	Assinatura
só ler o argumento	`const T&`
ler e modificar o do chamador	`T&`
guardar uma cópia	`T` por valor + `std::move` dentro
repassar em *template*	`T&&` + `std::forward<T>`
tipo pequeno e trivial	`T` por valor

Regra — não surpreenda

Se `+` não for comutativo, associativo e barato, escreva um método com nome. `a.concatenar(b)` nunca confundiu ninguém; `a + b` com semântica inesperada confunde todo mundo, inclusive o autor, seis meses depois.

II.4.1 Membro ou função livre

A diferença prática está na **conversão implícita do operando da esquerda**. Um operador membro não converte o próprio `this`:

```
m + 5.0   // funciona com operador membro (o double vira Dinheiro)
5.0 + m   // NÃO COMPILA: o 'this' não converte
```

Listagem 47: A assimetria que decide a forma.

Daí a divisão da Tabela 17. O caso dos operadores de *stream* não é escolha: `operator«` tem de ser livre porque o operando da esquerda é o `std::ostream`, que não é seu.

Tabela 17: Qual forma usar para cada família de operadores.

Operador	Forma
simétrico (`+ - * / ==`)	função **livre**
modifica o objeto (`+=`)	**membro**
`= [] () -> ->*`	**membro** (exigido pela linguagem)
`« »` de *stream*	**livre** (exigido)

O idioma que reconcilia as duas primeiras linhas é a **forma canônica**: a lógica vive no membro que modifica, e o operador simétrico é um *hidden friend* que a reaproveita.

```
Tipo& operator+=(const Tipo& o);              // membro: aqui mora a lógica
friend Tipo operator+(Tipo a, const Tipo& b)    // livre: 'a' por VALOR
    { return a += b; }
```

Listagem 48: A forma canônica — e o motivo de `a` vir por valor.

O parâmetro `a` por valor é a cópia de trabalho — e, se o chamador passou um temporário, ele é *movido* para lá em vez de copiado (Seção II.3). Definir o `friend` *dentro* do corpo da classe tem três ganhos: o *namespace* não é poluído, a resolução de sobrecarga considera menos candidatos — o que se nota no tempo de compilação de projetos grandes — e os dois operandos continuam podendo converter.

Armadilha — quatro operadores que não devem ser sobrecarregados

`&&` e `||` sobrecarregados **perdem o curto-circuito**: ambos os lados passam a ser avaliados, sempre. `,` perde a ordem de avaliação garantida. `&` unário quebra código genérico que espera obter o endereço. Os três casos compilam sem reclamação e mudam o significado de expressões escritas por terceiros.

II.4.2 `operator<=>`: seis operadores viram um

```
1  struct Versao {
2      int maior, menor, patch;
3
4      // UMA linha. O compilador compara os membros na ordem de DECLARAÇÃO e
5      // gera <, <=, >, >=, == e != a partir daqui.
6      auto operator<=>(const Versao&) const = default;
7  };
```

Listagem 49: Comparação total em uma linha, em `src/operators/demo_comparacao.hpp`.

A comparação gerada é lexicográfica na ordem de *declaração* dos membros — trocar a ordem dos campos na classe muda a semântica da comparação sem que nenhuma linha do `operator<=>` mude.

O valor devolvido não é um `int`: é uma **categoria de ordenação**, e a escolha entre as três (Tabela 18) é uma afirmação sobre o tipo.

Tabela 18: As três categorias de ordenação e o que cada uma promete.

Categoria	Ordem total	Equivalente = igual
`strong_ordering`	sim	**sim**
`weak_ordering`	sim	não
`partial_ordering`	**não**	não

Os dois casos-limite medidos pelo subprojeto explicam a tabela. Numa comparação de *strings* sem distinguir maiúsculas, `"ABC"` e `"abc"` são *equivalentes* mas não *iguais* — isso é `weak_ordering`. E com ponto flutuante, `1.0 < NaN`, `1.0 > NaN` e `1.0 == NaN` são **os três falsos** — é o que `partial_ordering` significa, e é por isso que ordenar um `vector<double>` com `NaN` dentro é UB (comportamento indefinido). Prefira a categoria mais forte que for verdadeira: quanto mais forte, mais os containers podem otimizar.

Regra — escreveu `<=>` à mão, escreva o `==` também

A ordenação precisa comparar campo a campo; a igualdade quase sempre tem um atalho barato — comparar o tamanho primeiro, por exemplo. Por isso o padrão deriva `==` de um `<=>` apenas quando ele é `= default`. Um `<=>` escrito à mão sem `==` deixa o tipo comparável por ordem e não comparável por igualdade.

Candidatos reescritos e revertidos. O C++20 deriva as comparações que faltam invertendo os operandos. Declarando apenas duas funções, o compilador gera oito operações:

```
1  struct Metros {
2      double valor;
3
4      // Comparação HOMOGÊNEA
5      auto operator<=>(const Metros&) const = default;
6
7      // Comparação HETEROGÊNEA: só declaramos Metros <=> double.
```

```
    // O compilador deriva 'double < Metros' invertendo (candidato revertido).
    std::partial_ordering operator<=>(double d) const { return valor <=> d; }
    bool operator==(double d) const { return valor == d; }
};
```

Listagem 50: Comparação heterogênea, em `src/operators/demo_comparacao.hpp`.

`m < 7.0` é *reescrito* como `(m <=> 7.0) < 0`; `7.0 > m` é *revertido* para `0 < (m <=> 7.0)`. Antes do C++20, o mesmo resultado exigia doze funções escritas à mão.

II.4.3 Conversões e `explicit`

Um construtor de um único argumento *já é* uma conversão implícita, esteja isso na intenção ou não:

```
Metros(double v);           // implícito: qualquer double vira Metros
explicit Metros(double v);  // explícito: só se alguém pedir
```

Listagem 51: Uma palavra separa o tipo forte do tipo decorativo.

Sem `explicit`, uma função que espera metros aceita alegremente um valor em segundos, e compila. É a mesma razão pela qual `std::vector<int> v(10)` cria dez zeros e `std::vector<int> v = 10` não compila.

A linguagem limita o encadeamento a **uma** conversão definida pelo usuário: dado `struct A { A(int); };` e `struct B { B(const A&); };`, a chamada `recebe_b(A{1})` compila e `recebe_b(1)` não, porque exigiria duas. Conversões *padrão* não contam para esse limite — `int` para `double`, derivado para base, `T*` para `const T*`, *enum* para `int`.

`explicit operator bool`. É a exceção que confirma a regra: o único operador de conversão que se marca `explicit` e ainda funciona implicitamente, nos contextos ditos *contextualmente convertíveis* — `if`, `while`, `for`, `!`, `&&`, `||`, `?:`, `static_assert`.

```
if (c)      // funciona
int n = c;  // NÃO COMPILA
c + 1;      // NÃO COMPILA
```

Listagem 52: O que `explicit` impede sem impedir o uso pretendido.

Por que — por que os *streams* adotaram `explicit operator bool`

Até o C++03, os *streams* convertiam para `void*` para permitir `if (arquivo)`. A consequência era que `std::cout << std::cin` compilava — imprimia um endereço. Sem `explicit`, um tipo `Conexao` com `operator bool` faria `conexao + 41` valer 42, e `conexaoA == conexaoB` compararia dois `bool`: duas conexões fechadas seriam "iguais".

II.4.4 `operator()`, `operator[]` e o *drill-down* de `operator->`

Um *functor* — objeto com `operator()` — ganha do ponteiro para função em tudo o que importa, e a razão é uma só: ele tem **tipo próprio**, então o compilador sabe exatamente qual código será chamado e pode fazer *inline*. Um *lambda é* um *functor*: o compilador gera uma classe anônima cujo `operator()` é `const` por padrão, e as capturas viram membros (Seção IV.4).

Tabela 19: As quatro formas de "algo chamável".

Forma	Tem estado	Inlinável	Tipo próprio
ponteiro para função	não	**não**	não
functor	sim	**sim**	sim
lambda	sim	**sim**	sim (anônimo)
`std::function`	sim	não	apagado

O `operator[]` multidimensional do C++23 fecha um buraco antigo: até então, uma matriz tinha três saídas e todas com defeito — `m(i,j)` parece chamada de função, `m[i][j]` exige uma classe *proxy* por dimensão, e `m[{i,j}]` constrói um par a cada acesso. Agora `vol[i, j, k]` é legítimo.

Armadilha — `a[i, j]` mudou de significado entre C++20 e C++23

Em C++20, `a[i, j]` era o operador *vírgula* dentro dos colchetes: avaliava `i`, descartava, e indexava por `j`. O C++20 deprecou a forma e o C++23 reciclou a sintaxe — de modo que código antigo muda de comportamento em silêncio ao ser recompilado. O aviso é `-Wcomma-subscript`.

O `operator->` tem uma regra que não existe em nenhum outro ponto da linguagem: se o que ele devolve **não** é um ponteiro cru, o compilador chama `operator->` de novo no resultado, recursivamente, até chegar a um ponteiro. É o mecanismo por trás de `unique_ptr`, `optional`, dos iteradores e de qualquer *proxy* de acesso.

II.4.5 Literais definidos pelo usuário

Os sufixos de literal permitem que uma grandeza chegue ao código com a unidade embutida no tipo. O caso mais interessante do subprojeto usa parâmetros de *template* inteiros para representar as dimensões físicas:

```
1  template <int Comprimento, int Massa, int Tempo> class Grandeza;
2  using Velocidade = Grandeza<1, 0, −1>;    // comprimento / tempo
```

Listagem 53: Dimensões como expoentes no tipo.

```
1  100_km / 2_h      = 13.89 m/s   ← tipo Velocidade, DERIVADO
2  70_kg * 9.81 m/s² = 686.70 N    ← tipo Forca, derivado
3  5_km + 300_m      = 5300 m      ✓
4  5_km + 2_s        → NÃO COMPILA ✗ dimensões incompatíveis
```

Listagem 54: Saída de `src/operators/`: o tipo do resultado é derivado pelo compilador, e o erro dimensional é de compilação.

E o custo é zero: `sizeof(Comprimento) == sizeof(double)`. É assim que funcionam as bibliotecas de unidades — e o próprio `std::chrono`, que é um sistema de unidades de tempo com verificação dimensional.

Regra — todo sufixo de usuário começa com _

Sufixos sem *underscore* inicial são reservados para a biblioteca padrão. `operator""km` compila hoje e pode colidir com uma adição futura do padrão amanhã — e a colisão aparece como um erro incompreensível num arquivo que ninguém alterou.

II.5 Ponteiros inteligentes e RAII

O subprojeto `src/smart-pointers/` é RAII aplicado, e só faz sentido depois de destrutores e *move* (Seção II.3). Antes de mostrar as ferramentas, convém enunciar com precisão o problema que elas resolvem — porque quem só viu a solução tende a usá-la como sinônimo de "ponteiro moderno", e a errar por excesso.

II.5.1 O problema, enunciado com precisão

Chame de **recurso** qualquer coisa que o programa *adquire* e depois precisa *devolver*: um bloco de memória, um arquivo aberto, um *socket*, um *mutex* travado, uma conexão de banco, uma textura na placa de vídeo. E chame de **posse** (*ownership*) a resposta à pergunta "quem é o responsável por devolver isto, e quando?".

Por que — por que a posse precisa estar no tipo, e não no comentário

Em C, a posse é uma convenção documentada: a página de manual do `strdup` diz que quem chama deve dar `free`. A convenção não é verificável — o compilador não a lê, o *linker* não a checa e o teste não a exercita, porque um vazamento não muda a resposta do programa. É por isso que os quatro erros da Tabela 20 atravessaram trinta anos de código: todos eles *compilam*, e três dos quatro passam pelo teste.

A memória do programa se divide, para este assunto, em duas regiões. A **pilha** (*stack*) guarda as variáveis locais; o compilador sabe onde cada uma nasce e morre, e emite a destruição sozinho. O **monte** (*heap*) guarda o que é criado com `new` e cujo tempo de vida o compilador *não* consegue deduzir — e é exatamente aí que a responsabilidade volta para o programador, com os resultados da Tabela 20.

Tabela 20: Os quatro modos de errar a posse, e por que nenhum deles aparece no teste.

Erro	O que acontece	Sintoma
vazamento (*leak*)	ninguém devolve	nenhum — só a memória cresce
double free	dois devolvem	aborto, às vezes
use-after-free	usa depois de devolver	lixo, às vezes
dangling pointer	aponta para o nada	UB (comportamento indefinido) silencioso

O argumento inteiro cabe num par de funções:

```
1  void ruim() {
2      Widget* w = new Widget();
3      if (condicao) return;          // LEAK
4      throw SomeException();         // LEAK
5      delete w;                      // só chega aqui no caso feliz
6  }
7
8  void bom() {
9      auto w = std::make_unique<Widget>();
10     if (condicao) return;          // destrutor chamado
11     throw SomeException();         // destrutor chamado
12 }                                  // destrutor chamado
```

Listagem 55: O mesmo recurso, com e sem RAII (`src/smart-pointers/README.md`).

O que muda não é a quantidade de código: é que na segunda versão **não existe caminho de saída que esqueça a liberação**, porque a liberação deixou de ser um comando e passou a ser uma consequência do escopo. Como o destrutor roda também durante o desenrolamento da pilha (Seção II.8), o mesmo mecanismo cobre o caminho da exceção sem uma linha a mais.

Regra — a definição operacional de RAII

Resource Acquisition Is Initialization: o construtor adquire o recurso e o destrutor o devolve, de modo que **um objeto vivo é um recurso válido**. A consequência prática é a que interessa: como o compilador garante que o destrutor de todo objeto automático roda — em `return`, em `break`, em `throw` — ele passa a garantir também a devolução. O nome é ruim (o que importa é a *destruição*), mas o mecanismo é o alicerce de metade deste livro.

II.5.2 Os três ponteiros

Tabela 21: Qual ponteiro para qual relação de posse.

Situação	Use	Custo
único dono do objeto	`std::unique_ptr`	zero sobre o ponteiro cru
posse compartilhada	`std::shared_ptr`	bloco de controle + contagem atômica
observar sem possuir	`std::weak_ptr`	idem, sem manter vivo
acesso sem posse	`T*` ou `T&`	zero

`unique_ptr`. Não é copiável — a tentativa é erro de compilação, não bug em produção — e a transferência de posse é explícita, via `std::move`. Depois do *move*, o ponteiro de origem vale `nullptr`; usá-lo é UB (comportamento indefinido). O *deleter* customizado é o que o torna um invólucro genérico de recurso, e não apenas de memória:

```
1 auto fechar = [](FILE* f){ if (f) std::fclose(f); };
2 std::unique_ptr<FILE, decltype(fechar)> arq(std::fopen("/dev/null","w"), fechar);
```

Listagem 56: `unique_ptr` como guarda de qualquer recurso (`demo_unique.hpp`, seção 1.5).

Duas operações do `unique_ptr` parecem sinônimas e não são (Tabela 22). A distinção é a fronteira entre "continuo protegido" e "voltei ao C".

Tabela 22: `release()` contra `reset()` — a diferença de uma chamada de destrutor.

Chamada	Destrói?	Fica nulo?	Depois disso
`p.release()`	**não**	sim	*você* deve dar `delete`
`p.reset()`	sim	sim	nada a fazer
`p.reset(q)`	sim (o antigo)	não	passa a possuir `q`
`p.get()`	não	não	empréstimo; não guarde

Armadilha — `release()` é o botão que desliga a proteção

A linha `Recurso* raw = p.release();` do `demo_unique.hpp` é seguida, no subprojeto, de um `delete raw;` explícito e de um comentário dizendo "obrigatório — senão vaza". Isso não é estilo: `release()` existe para entregar o ponteiro a uma API C que vai assumir a posse, e em qualquer outro uso ele apenas reintroduz os quatro erros da Tabela 20. Quando a intenção é só *emprestar*, o certo é `get()` — ou, melhor ainda, passar `*p` por referência.

Existe também `std::unique_ptr<T[]>`, que chama `delete[]` em vez de `delete` — e essa é a única razão para ele existir, porque trocar um pelo outro é UB (comportamento indefinido). Ainda assim, ele quase nunca é a resposta certa: não guarda o tamanho, não tem `begin()`/`end()` e não cresce. `std::vector` faz tudo isso pelo mesmo preço.

`shared_ptr.` Contagem de referências: o objeto morre quando o último `shared_ptr` morre. O que ele carrega além do ponteiro é o **bloco de controle**, um objeto separado com duas contagens — quantos *shared* e quantos *weak* apontam para o alvo — e o *deleter*. `make_shared` funde esse bloco com o objeto numa alocação só, contra as duas de `std::shared_ptr<T> p(new T)`, e é também seguro em face de exceções.

`weak_ptr.` Observa sem possuir, e existe por causa de um problema que a contagem de referências não resolve sozinha: o **ciclo**.

```
1  struct No { std::shared_ptr<No> proximo; };   // a → b → a: use_count nunca zera
2  struct No { std::weak_ptr<No>   proximo; };   // não incrementa a contagem
```

Listagem 57: O ciclo, e a mudança de uma palavra que o quebra (`demo_weak.hpp`, seções 3.2 e 3.3).

O subprojeto roda os dois casos em sequência, e a saída é a prova: no caso do ciclo os destrutores *não* aparecem.

```
1  — 3.2 Ciclo com shared_ptr → MEMORY LEAK —
2    [NodoComCiclo 'A—ciclo'] construído
3    [NodoComCiclo 'B—ciclo'] construído
4    use_count(a) = 2
5    use_count(b) = 2
6                                  ← nenhum destrutor: os dois vazaram
7
8  — 3.3 Ciclo com weak_ptr → SEM LEAK —
9    [NodoSemCiclo 'A—ok'] construído
10   [NodoSemCiclo 'B—ok'] construído
11   use_count(a) = 1
12   use_count(b) = 1
13   a->proximo = 'B—ok'
14   [NodoSemCiclo 'B—ok'] destruído
15   [NodoSemCiclo 'A—ok'] destruído
```

Listagem 58: Saída de `dist/bin/smart_pointers`, seções 3.2 e 3.3. Note quais destrutores faltam.

Para usar o objeto é preciso `lock()`, que devolve um `shared_ptr` vazio se ele já morreu — e é essa obrigação de verificar que torna o `weak_ptr` seguro. `expired()` responde a mesma pergunta sem produzir o `shared_ptr`, e por isso **não** serve como teste antes do uso: entre o `expired()` e a desreferência, outra *thread* pode ter soltado a última referência. O idioma correto é um só, e já traz o resultado consigo:

```
1  if (auto p = w.lock()) { p->usar(); }   // p mantém vivo enquanto durar o if
```

Listagem 59: O único uso seguro de um `weak_ptr`.

Armadilha — `use_count()` não conta os `weak_ptr`

Criar um `weak_ptr` a partir de um `shared_ptr` não altera `use_count()` — ele continua respondendo 1, como imprime a seção 3.1 do subprojeto. Isso é intencional (o `weak_ptr` não mantém o objeto vivo), mas contraria a leitura ingênua do nome. O que os *weak* mantêm vivo é o *bloco de controle*, não o objeto — consequência que a Seção II.5.3 mede em bytes.

II.5.3 O custo, medido

Aviso

Todos os números desta seção foram medidos neste repositório, em `AMD Ryzen 9 5900X`, GCC 13.3.0, libstdc++ 13, glibc 2.39, `-O2 -std=c++20`. Números de desempenho não são propriedades da linguagem: são propriedades de uma combinação de compilador, biblioteca, máquina e carga. O que se transporta de uma medição para outra é a *ordem de grandeza* e, sobretudo, o *método*.

A afirmação "`unique_ptr` tem custo zero" é repetida em todo lugar e quase nunca demonstrada. Demonstrar é fácil: basta comparar o código de máquina das duas versões da mesma função.

```
1  struct W { int a; W(int x):a(x){} ~W(); };
2  int cru()   { W* p = new W(7); int v = p->a; delete p; return v; }
3  int smart() { auto p = std::make_unique<W>(7); return p->a; }
```

Listagem 60: As duas funções comparadas.

```
1  _Z3cruv:                    _Z5smartv:
2    endbr64                     endbr64
3    pushq  %rbx                 pushq  %rbx
4    movl   $4, %edi             movl   $4, %edi
5    call   _Znwm@PLT            call   _Znwm@PLT      ← operator new
6    movl   $7, (%rax)           movl   $7, (%rax)
7    movq   %rax, %rbx           movq   %rax, %rbx
8    movq   %rax, %rdi           movq   %rax, %rdi
9    call   _ZN1WD1Ev@PLT        call   _ZN1WD1Ev@PLT  ← ~W()
10   movq   %rbx, %rdi           movq   %rbx, %rdi
11   movl   $4, %esi             movl   $4, %esi
12   call   _ZdlPvm@PLT          call   _ZdlPvm@PLT    ← operator delete
13   movl   $7, %eax             movl   $7, %eax
14   popq   %rbx                 popq   %rbx
15   ret                         ret
```

Listagem 61: `g++ -O2 -S`: a sequência de instruções das duas funções é idêntica, instrução por instrução — 14 cada.

Não é "quase igual": é a mesma sequência. O `unique_ptr` desaparece por completo na compilação, e o que sobra é o `new`/`delete` que você teria escrito à mão — com a diferença de que o `delete` agora é emitido pelo compilador em *todos* os caminhos de saída, inclusive nos que você esqueceria.

O tamanho segue a mesma lógica: o que não é usado não é pago (Tabela 23).

Tabela 23: `sizeof` medido com `g++ -std=c++20` e libstdc++ 13.

Tipo	bytes	Por quê
`int*`	8	referência
`std::unique_ptr<W>`	8	*deleter* sem estado, elidido
`std::unique_ptr<W[]>`	8	idem
`std::unique_ptr<W, lambda>`	8	*lambda* sem captura é vazia
`std::unique_ptr<W, void(*)(W*)>`	**16**	o ponteiro de função *tem* estado
`std::shared_ptr<W>`	16	alvo + bloco de controle
`std::weak_ptr<W>`	16	idem

Armadilha — o *deleter* pode dobrar o tamanho do ponteiro

As duas formas de guardar um `FILE*` parecem equivalentes e não são. Com uma *lambda* sem captura, o *deleter* é um tipo vazio e a *empty base optimization* o faz sumir: 8 bytes. Com `decltype(&std::fclose)`, o *deleter* é um ponteiro de função — um dado real, guardado em cada

instância: 16 bytes, o dobro, e um campo a mais para copiar em cada *move*. Num `std::vector` de um milhão desses guardas, a diferença é 8 MB de nada.

O `shared_ptr`, esse, custa. A Tabela 24 conta as chamadas a `operator new` substituindo-o globalmente por uma versão que incrementa um contador — técnica que reaparece no Seção III.5.

Tabela 24: Alocações e bytes por criação, medidos com `operator new` instrumentado sobre um objeto de 64 bytes.

Expressão	Alocações	Bytes	Sobrecusto
`std::make_unique<Big>()`	1	64	nenhum
`std::make_shared<Big>()`	1	80	+16 (bloco embutido)
`std::shared_ptr<Big>(new Big())`	**2**	88	+24 e uma segunda `malloc`

E a contagem custa tempo. Vinte milhões de iterações de cada laço, tempo médio por operação:

```
1  copia de shared_ptr (incr + decr atomico) : 4,43 ns/op
2  weak_ptr::lock()                          : 5,27 ns/op
3  copia de ponteiro cru                     : 0,00 ns/op   ← ver a armadilha abaixo
```

Listagem 62: Custo por operação, 20 milhões de iterações, `-O2`.

Armadilha — o "0,00 ns" não é velocidade: é ausência

O laço do ponteiro cru mediu zero porque o compilador o *apagou*: copiar um ponteiro e somar um campo que ninguém observa não tem efeito colateral, e `-O2` remove o laço inteiro. O laço do `shared_ptr` sobreviveu justamente porque a contagem atômica **é** um efeito observável, que o compilador não tem licença para eliminar. Toda medição de microbenchmark tem esse risco, e o sintoma é sempre o mesmo: um número absurdamente bom. O hábito que evita a conclusão errada é conferir o assembly antes de acreditar — ou usar `benchmark::DoNotOptimize`. A conclusão honesta desta medição é "copiar `shared_ptr` custa cerca de 4 ns e copiar ponteiro cru custa menos que a resolução do experimento", não "é infinitamente mais rápido".

II.5.4 A medição que contrariou o conselho: `make_shared` e o `weak_ptr`

Todo material sobre `shared_ptr` — inclusive o `README.md` deste subprojeto — diz para preferir `make_shared`, e a Tabela 24 confirma a razão: uma alocação em vez de duas. Mas a fusão do bloco de controle com o objeto tem uma consequência que o conselho não menciona, e que só aparece quando se *mede a memória liberada* em vez de contar as alocações.

O experimento: um objeto de 1 KB, um `weak_ptr` que sobrevive ao último `shared_ptr`, e `operator delete` instrumentado para contar as liberações.

```
1  [make_shared]      dentro do escopo:              alocacoes=1  liberacoes=0
2  [make_shared]      ~Grande() rodou
3  [make_shared]      apos o ultimo shared morrer: liberacoes=0   ← 1 KB retido!
4  [make_shared]      apos o weak_ptr morrer:      liberacoes=1
5
6  [shared_ptr(new)]  dentro do escopo:              alocacoes=2
7  [shared_ptr(new)]  ~Grande() rodou
8  [shared_ptr(new)]  apos o ultimo shared morrer: liberacoes=1   ← 1 KB devolvido
9  [shared_ptr(new)]  apos o weak_ptr morrer:      liberacoes=2
```

Listagem 63: Liberações contadas com `operator delete` instrumentado, objeto de 1024 bytes, um `weak_ptr` vivo.

Os dois destroem o objeto na hora certa — `~Grande()` roda nos dois casos assim que o último `shared_ptr` morre. O que muda é a *memória*: com `make_shared` o bloco é um só, e enquanto houver um `weak_ptr` vivo o alocador não pode devolver nada, nem o quilobyte do objeto já destruído. Com `shared_ptr(new)`, o objeto tem alocação própria e volta imediatamente; só os 24 bytes do bloco de controle esperam pelo *weak*.

Regra — `make_shared` por padrão, `shared_ptr(new)` quando o *weak* sobrevive

Prefira `make_shared`: uma alocação em vez de duas, melhor localidade e segurança em face de exceções. **Exceto** quando o objeto for grande e houver `weak_ptr` com expectativa de vida maior que a dele — caches, tabelas de observadores, grafos com arestas fracas. Nesse caso a alocação extra compra a devolução imediata do objeto, e é o negócio certo. Este é o formato típico de um conselho bom: verdadeiro no caso comum, falso num caso nomeável, e decidível por medição.

II.5.5 A contagem é atômica — e não dá mais para desligá-la

Durante anos valeu o seguinte folclore: a libstdc++ detecta em tempo de execução se o programa é *single-threaded* e, nesse caso, usa incrementos comuns em vez de instruções atômicas; logo, quem não liga `-pthread` não paga pela atomicidade. Isso era verdade, e deixou de ser. A verificação era `__gthread_active_p()`, que perguntava se o símbolo `pthread_create` havia sido resolvido; desde a glibc 2.34 a `libpthread` foi fundida na `libc`, e o símbolo está sempre lá.

```
$ g++ -O2 -S copia.cpp          && grep -c 'lock ' copia.s
3
$ g++ -O2 -pthread -S copia.cpp && grep -c 'lock ' copia.s
3
```

Listagem 64: Contagem de prefixos `lock` numa cópia de `shared_ptr`, glibc 2.39.

Os mesmos três `lock xaddl` nos dois casos. A consequência prática: os 4,43 ns por cópia da seção anterior são o preço real, mesmo em programa de uma *thread* só. E a consequência didática é mais importante que a técnica — **um conselho de desempenho tem prazo de validade**, e o único modo de saber se o dele venceu é remedir.

Regra — passe `shared_ptr` por `const&` quando não for compartilhar

Cada passagem por valor é um par de operações atômicas: incremento na entrada, decremento na saída. Numa função chamada em laço, isso é o custo dominante. Passe por valor *só* quando a função for de fato guardar uma referência nova; para apenas usar o objeto, o parâmetro certo é `const T&` — ou `T&`, que nem menciona posse.

II.5.6 `enable_shared_from_this` e o construtor de *aliasing*

Duas facilidades do `shared_ptr` que o subprojeto cobre e que costumam faltar em material introdutório — ambas resolvem problemas reais, e ambas têm uma armadilha de mão dupla.

Um objeto que precisa de um `shared_ptr` de si mesmo. O caso aparece sempre que um método precisa se registrar em algum lugar (*callback* assíncrono, lista de observadores). A tentação é `return std::shared_ptr<T>(this)` — e isso cria um **segundo bloco de controle** para o mesmo endereço. Dois blocos, duas contagens, duas chegadas a zero: *double free*. A solução é herdar de

`std::enable_shared_from_this<T>`, que guarda um `weak_ptr` para o bloco existente e o promove sob demanda:

```
1  struct Observavel : std::enable_shared_from_this<Observavel> {
2      std::shared_ptr<Observavel> get_self() { return shared_from_this(); }
3  };
4  // medido: use_count 1 → 2 dentro do escopo de get_self() → 1 de volta
```

Listagem 65: `shared_from_this` devolve uma referência ao *mesmo* bloco (`demo_shared.hpp`, seção 2.5).

Armadilha — `shared_from_this` num objeto que não é de um `shared_ptr`

O mecanismo só funciona se o objeto *já* for gerenciado por um `shared_ptr` — é dele que vem o bloco de controle. Chamar `shared_from_this()` numa variável de pilha, ou dentro do próprio construtor (quando o `shared_ptr` externo ainda não existe), lança `std::bad_weak_ptr`. É por isso que classes assim costumam ter construtor privado e uma *factory* estática que devolve `make_shared`: a fábrica torna a pré-condição impossível de violar.

Um ponteiro para dentro do objeto. O *alias constructor* `std::shared_ptr<U>(dono, &dono->membro)` produz um `shared_ptr` que *aponta* para o membro mas *conta* no bloco do objeto inteiro. Serve para entregar uma parte sem entregar o todo:

```
1  [Carro 'Fusca'] construído
2  carro.use_count() = 2 (carro + motor_ptr)
3  motor_ptr->rpm = 3000
4  carro.use_count() após reset: 1
5  motor_ptr->rpm ainda acessível: 3000     ← o Carro continua vivo
6  [Carro 'Fusca'] destruído                ← só agora, com motor_ptr saindo
```

Listagem 66: Saída medida de `dist/bin/smart_pointers`, seção 2.6.

Armadilha — um *alias* pequeno segura um objeto grande

Exatamente a propriedade que torna o *alias constructor* seguro — o membro nunca sobrevive ao dono — é a que causa retenção acidental: um `shared_ptr<int>` de 16 bytes apontando para um campo de uma imagem de 40 MB mantém os 40 MB vivos. É o mesmo fenômeno do `make_shared` com `weak_ptr` da seção anterior, e o mesmo remédio: saber que existe, e copiar o valor quando o que se quer é só o valor.

II.5.7 Como passar um ponteiro inteligente para uma função

A assinatura de uma função é onde a posse fica *documentada pelo compilador*. A Tabela 25 traduz cada forma; ela é a versão para recursos da Tabela 9.

Regra — a maioria das funções não deveria ver um ponteiro inteligente

Se a função apenas *usa* o objeto, ela não tem opinião sobre posse e o parâmetro certo é `const Widget&`. Receber `const shared_ptr<Widget>&` onde bastava uma referência acopla a função a uma política de posse que ela não precisa conhecer — e impede chamá-la com um objeto de pilha, ou com um membro de outro objeto. Ponteiro inteligente em assinatura é para quem *vai mexer na posse*.

Tabela 25: O que cada assinatura comunica (`demo_patterns.hpp`, "regras de ouro").

Assinatura	Significado
`f(Widget&)`	só usa; posse não é assunto da função
`f(const Widget&)`	só lê — **o padrão**
`f(Widget*)`	só usa, e pode não haver objeto
`f(unique_ptr<W>)`	**toma** a posse exclusiva
`f(unique_ptr<W>&)`	pode *trocar* o ponteiro de quem chamou
`f(shared_ptr<W>)`	passa a compartilhar (custa duas atômicas)
`f(const shared_ptr<W>&)`	pode vir a compartilhar; não paga se não vier

II.5.8 Três padrões que dependem de `unique_ptr`

Guarda genérico. Qualquer recurso com função de fechamento vira RAII com um *deleter*: `FILE*`, *socket*, *mutex*, *handle* do sistema operacional. É o mesmo mecanismo que o Seção V.4 usa para `dlclose`.

Pimpl. Um `unique_ptr` para uma `struct Impl` apenas declarada no cabeçalho quebra a dependência de compilação entre a interface e o que ela usa: mudar os campos de `Impl` deixa de obrigar a recompilar quem inclui o cabeçalho. O detalhe que faz o idioma funcionar — e que é a causa de metade dos erros de quem o tenta pela primeira vez — é que o destrutor precisa ser *declarado* no cabeçalho e *definido* no `.cpp`, onde `Impl` já é um tipo completo. A razão é mecânica: o destrutor implícito instancia `default_delete<Impl>`, que faz `sizeof(Impl)` para verificar que o tipo não é incompleto.

```
1 /usr/include/c++/13/bits/unique_ptr.h:97:23: error:
2     invalid application of 'sizeof' to incomplete type 'Widget::Impl'
```

Listagem 67: Erro real do GCC 13.3 ao esquecer o destrutor fora de linha — note que ele aponta para dentro do `<memory>`, não para o seu código.

Armadilha — o Pimpl que "compila em um `.cpp` e quebra no outro"

Enquanto o único `.cpp` que destrói o objeto for aquele em que `Impl` está definido, tudo funciona. O erro aparece no primeiro *outro* arquivo que deixe um `Widget` sair de escopo — muitas vezes semanas depois, num arquivo que ninguém tocou. A profilaxia é declarar os cinco (Seção II.3) no cabeçalho e defini-los, mesmo que com `= default`, no `.cpp`.

Factory. Devolver `std::unique_ptr<Base>` torna a posse inequívoca no tipo de retorno: quem chamou é dono, e o compilador não deixa esquecer. É a assinatura que aparece em Seção V.2 e Seção V.4.

Regra — `new` e `delete` crus não aparecem em código de aplicação

Eles pertencem ao interior de um tipo que gerencia recurso — e, na prática, ao interior da biblioteca padrão. Código de aplicação usa `make_unique`, `make_shared` ou um container. O Seção III.5 mostra o que acontece dentro de `new`, e é a exceção que justifica a regra.

II.6 Arquivos e *streams*: RAII com estado

O subprojeto `src/file_io/` entra aqui, e não no capítulo da biblioteca padrão, por um motivo de assunto: um *stream* é o primeiro recurso RAII que o leitor encontra na prática, e é o único da biblioteca padrão

que **carrega estado de erro em vez de lançar**. Quem entendeu `unique_ptr` tem metade do modelo; a outra metade — por que `ifs » x` não lança quando falha, e o que exatamente acontece então — é o que esta seção fecha. O catálogo das classes, com a hierarquia desenhada, está na Seção III.3; aqui interessa o mecanismo e o que ele custa.

Por que — por que um arquivo é mais difícil que um bloco de memória

Memória tem um dono e uma operação de devolução. Um arquivo tem quatro coisas a mais, e cada uma é uma fonte de bug distinta. **(1)** Um *descritor* — um número inteiro que o núcleo do sistema entrega no `open` e cobra de volta no `close`; o processo tem uma cota deles, e vazá-los derruba o programa muito antes de a memória acabar. **(2)** Um *buffer*: o que você escreveu ainda não chegou ao disco, e só chega no *flush*. **(3)** Um *cursor*, a posição corrente de leitura e a de escrita, que são duas e independentes. **(4)** Um *estado*: a operação anterior pode ter falhado, e o *stream* guarda isso em vez de avisar. Ignorar qualquer um dos quatro produz um programa que funciona no teste e perde dados em produção.

II.6.1 O *stream* como objeto RAII

`std::ofstream` adquire no construtor e devolve no destrutor, exatamente como o `unique_ptr`: abre o arquivo, e no fim do escopo faz *flush* do *buffer* e fecha o descritor. É a razão pela qual o `demo_fstream.hpp` abre cada arquivo dentro de um bloco próprio — o fechamento acontece na chave, não numa chamada que se possa esquecer.

```
1 {
2     std::ofstream ofs(path);                  // abre
3     ofs << "Linha 1: Hello, C++!\n";
4 }                                             // flush + close, inclusive se lançar
```

Listagem 68: O bloco *é* o `close` (`demo_fstream.hpp`, seção 1).

Armadilha — o destrutor fecha, mas não conta se deu errado

O destrutor do `ofstream` faz *flush* e engole qualquer falha — disco cheio, cota estourada, sistema de arquivos remoto que caiu. Como o destrutor não pode lançar (Seção II.8), o erro simplesmente não é reportado, e o programa termina com código de saída zero tendo escrito um arquivo truncado. Quando a gravação *importa*, chame `close()` explicitamente e teste o *stream* depois — é a única forma de saber.

II.6.2 Os quatro bits de estado

Um *stream* não lança por padrão. Ele acende bits, e continua aceitando operações que, a partir dali, não fazem nada. Os bits são quatro (Tabela 26), e a confusão entre `fail` e `eof` é a origem do erro mais comum de todo o assunto.

Tabela 26: Os bits de estado de um *stream*, e o que cada um significa.

Consulta	Aceso quando	Recuperável?
`good()`	nenhum dos outros está aceso	—
`eof()`	a última leitura tocou o fim do arquivo	sim
`fail()`	a conversão falhou, ou o arquivo não abriu	sim, com `clear()`
`bad()`	erro de integridade do *stream*	**não**

O subprojeto imprime os quatro para um arquivo que não existe, e o resultado esclarece a diferença entre "acabou" e "deu errado":

```
— abrindo /tmp/nao_existe.txt —
  arquivo não abriu (is_open() = false)
  good()=false fail()=true bad()=false eof()=false
```

Listagem 69: Saída medida de `dist/bin/file_io`, demo 2, seção 5.

`fail()` está aceso, `eof()` não: não se chegou ao fim de coisa alguma, a abertura é que falhou. O mesmo par aparece ao ler um `int` de onde há a palavra `abc` — `fail()=true`, `eof()=false`, e o `int` fica **inalterado**, não zerado. Só depois de `clear()` o *stream* volta a funcionar; note, na saída do demo, que o texto `abc` continua no *buffer* e é lido em seguida como `std::string`.

Armadilha — `while (!ifs.eof())` lê a última linha duas vezes

`eof()` só acende *depois* de uma leitura que tentou passar do fim. No laço `while (!ifs.eof()) { std::getline(ifs, l); usa(l); }`, a última iteração lê nada, falha, e ainda assim entra no corpo com o valor antigo de `l`. A forma correta põe a leitura na condição — `while (std::getline(ifs, l))` — porque o *stream* converte para `bool` testando `!fail()`, que é a pergunta certa: "a leitura que eu acabei de pedir funcionou?".

Quem prefere exceções pode pedi-las com `exceptions(std::ios::failbit | std::ios::badbit)`, e aí a abertura malsucedida lança `std::ios_base::failure`. O subprojeto faz as duas coisas para comparar — e a mensagem que sai, `basic_ios::clear: iostream error`, ilustra bem por que a via das exceções é pouco usada aqui: ela não diz qual arquivo, nem por quê. Para saber isso é preciso o `errno`, ou o `std::error_code` do `<filesystem>`.

II.6.3 Bytes não são caracteres

O demo escreve quatro linhas de texto e pergunta o tamanho ao sistema de arquivos. O número que volta não é o número de caracteres:

```
  tamanho final: 89 bytes            ← fs:: file_size ()
$ wc -m -c /tmp/cpp_academy_texto.txt
      87      89                     ← 87 caracteres, 89 bytes
```

Listagem 70: `dist/bin/file_io`, demo 2, seção 1, confrontado com `wc`.

A diferença de dois bytes está numa linha só: o travessão de `"Linha 2: 42 -- 3.14"`. U+2014 ocupa **três** bytes em UTF-8, e todas as APIs de arquivo contam bytes.

Armadilha — o caractere de três bytes entre aspas simples

Esta não é uma preocupação acadêmica: foi um bug real deste repositório, em `src/http-client/main.cpp`. A linha escrevia `std::string(n, '—')` para desenhar uma régua — e U+2500, como U+2014, tem três bytes. Entre *aspas simples* isso não é um `char`: é uma *multi-character constant*, cujo valor é definido pela implementação e que o construtor trunca para um byte, produzindo lixo. A mesma função continha o segundo bug clássico do assunto: `40 - title.size()`, com `size()` devolvendo `std::size_t` — aritmética **sem sinal**, que para um título de 41 caracteres não dá −1, dá `18446744073709551615`. Os dois compilavam sem um *warning* sequer. Para repetir uma sequência multibyte, o certo é `std::string` concatenada em laço, ou uma `std::string_view` com o literal.

II.6.4 Texto, binário e o cursor

Em modo texto o *stream* formata: `ofs « 42` escreve dois caracteres. Em modo binário ele copia bytes: `ofs.write(ptr, n)` despeja a representação de memória tal como está. O demo grava três registros e mede:

```
3 registros escritos (24 bytes cada, total=72 bytes)
[ 1] id=  1 valor=3.14 nome='alpha'
[ 2] id=  2 valor=2.71 nome='beta'
[ 3] id=  3 valor=1.41 nome='gamma'
gcount()=0 (bytes lidos na última operação)
acesso aleatório → 2º registro: id=2 nome='beta'
```

Listagem 71: `dist/bin/file_io`, demo 2, seção 4.

Os 24 bytes por registro são `uint32_t` + `float` + `char[16]` — 4 + 4 + 16, sem preenchimento, porque o alinhamento mais restritivo da estrutura é 4 e o tamanho já é múltiplo dele. Não conte com isso: é uma coincidência aritmética desta estrutura, não uma regra (Seção XIV.1 trata do assunto).

Armadilha — `gcount()` depois do laço vale zero

A saída acima diz `gcount()=0` logo depois de ter lido três registros de 24 bytes. Não é bug: `gcount()` devolve os bytes da *última* operação de entrada não formatada, e a última foi a que encerrou o laço — a leitura que encontrou o fim do arquivo e leu zero bytes. Para saber quanto veio de um `read` parcial, é preciso consultar `gcount()` **dentro** do laço, imediatamente após cada leitura. É um caso exemplar da classe de erro deste capítulo: o programa está certo, a saída está certa, e a leitura ingênua está errada.

Gravar uma `struct` com `reinterpret_cast` congela o preenchimento, o tamanho dos tipos e a ordem dos bytes (*endianness*) da máquina que gravou: o arquivo de 72 bytes acima não é legível por um programa compilado com outro alinhamento, nem por uma máquina *big-endian*. A Seção XIII.4 volta ao ponto; dentro de uma máquina, para *cache* de curta duração, a técnica é legítima.

Um `fstream` aberto com `ios::in | ios::out` tem dois cursores independentes — o de leitura (`tellg`/`seekg`) e o de escrita (`tellp`/`seekp`) — e uma propriedade que costuma surpreender: escrever em um *offset* **substitui** os bytes que estavam lá. No demo, cinco caracteres gravados no início transformam a linha `ITEM_0` em `MODIF0`: "MODIF" cobriu "ITEM_" e o `0` sobreviveu. Não existe "inserir no meio de um arquivo"; inserir é reescrever o resto — e é por isso que editores de texto carregam o arquivo inteiro na memória.

II.6.5 O que os *streams* custam, medido

Três conselhos circulam sobre desempenho de *streams*. Medidos nesta máquina (mesmas condições da Seção II.5.3), um se confirma com folga, um se inverte e um encolhe.

O conselho que se confirmou. `std::endl` não é "quebra de linha": é quebra de linha **mais *flush***, e cada *flush* é uma chamada de sistema. Trinta e duas vezes mais lento é a diferença entre escrever um *log* de 200 mil linhas em 9 ms e em 0,28 s.

Tabela 27: 200 000 linhas escritas, lidas ou concatenadas; média de duas execuções, `-O2`, disco local.

Operação	Forma	Tempo	Veredito
escrever	`ofs « ... « '\n'`	8,8 ms	—
escrever	`ofs « ... « std::endl`	280 ms	**32× mais lento**
ler	`getline` linha a linha	2,9 ms	—
ler	`oss « ifs.rdbuf()`	2,4 ms	1,2× mais rápido
montar	`std::string` com `+=`	3,6 ms	**mais rápido**
montar	`std::ostringstream`	6,6 ms	1,8× mais lento

Regra — `'\n'` sempre; `std::endl` só quando o *flush* for o objetivo

Use `std::endl` apenas quando a mensagem precisa estar no disco *agora* — a última linha antes de um aborto previsto, por exemplo. Para qualquer outra coisa, `'\n'`. O `Logger` do subprojeto faz a escolha oposta de propósito, e a Seção II.6.7 explica por quê.

O conselho que se inverteu. O `README.md` do subprojeto apresenta o `ostringstream` como "concatenação eficiente (evita N alocações do `+` com `string`)". A medição diz o contrário: o `std::string` com `+=` foi 1,8 vez *mais rápido*. A explicação é que o conselho envelheceu junto com a implementação. Ele era verdadeiro contra `a + b + c`, que constrói temporários; contra `+=`, não é — `std::string` cresce com capacidade geométrica, e o *amortizado* de `+=` é uma cópia de bytes, enquanto cada `«` do *stream* atravessa a maquinaria de formatação, *locale* e *sentry*. O valor do `ostringstream` é a *formatação* (`setw`, `setprecision`, tabelas alinhadas), não a velocidade de emenda.

O conselho que encolheu. O terceiro é `std::ios_base::sync_with_stdio(false)`, que a folclore das maratonas de programação vende como ganho de ordem de grandeza. Medido aqui, com 300 mil linhas em `std::cout` redirecionado para `/dev/null`:

```
sync=true  : 12,3 ms
sync=false : 10,1 ms      ← 1,2×, não 10×
```

Listagem 72: A mesma carga com e sem sincronização com o `stdio` do C.

Não é que a folclore esteja errada; é que ela mede outra coisa. O ganho grande aparece em *leitura* caractere a caractere com `std::cin`, em que a sincronização custa uma ida ao *buffer* do C por caractere. Na escrita em lote, o tempo é dominado pela formatação, e a sincronização some no ruído.

Regra — um número de *benchmark* pertence a uma carga, não a uma API

As três medições acima ensinam o mesmo método. Antes de repetir um conselho de desempenho, pergunte três coisas: *qual carga* foi medida, *em que versão* da biblioteca, e *o que o compilador pode ter apagado*. Um conselho que não responde às três é folclore — inclusive quando é verdadeiro.

II.6.6 Redirecionamento: `rdbuf` e o que ele não captura

Todo *stream* é uma fachada de formatação sobre um `std::streambuf`, que é quem de fato move os bytes. Trocar o *buffer* redireciona a saída sem tocar em nenhuma das chamadas que escrevem nela — mecanismo que a Seção III.3 apresenta e que a Seção XIII.4 detalha, com o `RedirectGuard` e o `TeeBuf` do subprojeto

transcritos. O que cabe acrescentar aqui são duas armadilhas de tempo de vida, ambas da natureza deste capítulo: erros que compilam.

A primeira explica por que o `RedirectGuard` declara *deletados* o construtor de cópia e a atribuição. Copiá-lo faria dois objetos restaurarem o mesmo *buffer*, e o segundo restauraria sobre um estado já restaurado — exatamente a razão pela qual `unique_ptr` também não é copiável (Seção II.3). Um guarda de recurso é, por construção, *move-only*.

Armadilha — o *buffer* redirecionado não pode morrer antes do *stream*

`std::cout.rdbuf(arquivo.rdbuf())` guarda em `std::cout` um ponteiro para um `streambuf` que pertence ao `ofstream`. Se o `ofstream` sair de escopo antes da restauração, `std::cout` fica com um ponteiro pendurado — e o próximo `«` é UB (comportamento indefinido), possivelmente milhares de linhas depois. O comentário em maiúsculas no `demo_logging.hpp` ("IMPORTANTE: restaurar antes de qualquer `fmt::print` ou destruição") existe por isso. Com o `RedirectGuard`, a ordem correta é consequência da ordem de declaração: o arquivo é declarado antes do guarda, e destruído depois dele.

Há um segundo detalhe, e este é o que mais surpreende num repositório em que quase toda saída passa por `fmt::print`. O experimento: redirecionar `std::cout` para um arquivo e escrever por três vias diferentes.

```
1 std::ofstream f("/tmp/redir_test.txt");
2 std::streambuf* orig = std::cout.rdbuf(f.rdbuf());
3 std::cout    << "via std::cout\n";
4 fmt::print("via fmt::print\n");
5 std::printf("via printf\n");
6 std::cout.rdbuf(orig);
```

Listagem 73: O experimento — três escritas, um redirecionamento.

```
1 -- conteudo do arquivo --
2   via std::cout
3
4 -- o que continuou saindo no terminal --
5 via fmt::print
6 via printf
```

Listagem 74: Resultado medido: só uma das três foi capturada.

Armadilha — trocar o `rdbuf` de `cout` não redireciona `fmt::print`

`std::cout.rdbuf()` mexe em um objeto C++; `fmt::print` e `printf` escrevem no *descritor* 1, sem passar por ele. Quem captura saída para teste trocando o *buffer* de `std::cout` capturará exatamente nada num programa que usa `fmt`. Para redirecionar de verdade é preciso descer um nível: `freopen` (nível C, global e difícil de desfazer) ou `dup2` (nível POSIX, reversível guardando o *fd* antigo com `dup`). É por isso que o `Logger` do subprojeto recebe o *stream* de destino como parâmetro em vez de confiar em redirecionamento: injetar a dependência é mais simples do que interceptá-la.

Na direção oposta está o `TeeBuf` do subprojeto: em vez de trocar um destino por outro, escreve nos dois, implementando em C++ o que a *shell* faria com `./prog 2>&1 | tee log.txt`. Ele é um `std::streambuf` próprio, e a Seção XIII.4 o transcreve — vale antecipar apenas que o contrato mínimo de um `streambuf` de saída são dois métodos, `overflow` e `xsputn`, e que ele herda a mesma armadilha de tempo de vida: o `TeeBuf` aponta para dois *buffers* que precisam sobreviver a ele.

II.6.7 O *Logger*, e a lição de idempotência

O `Logger` do subprojeto reúne o capítulo inteiro num objeto de sessenta linhas: níveis (`TRACE` a `FATAL`), *timestamp* montado com `ostringstream` e `put_time`, cor ANSI no terminal e texto limpo no arquivo, `std::mutex` para ser utilizável de várias *threads*, e `ERROR`/`FATAL` desviados para `std::cerr`. Esse desvio é verificável na hora, e vale fazê-lo: rodando com `1> /dev/null`, restam na tela apenas as duas últimas linhas; com `2> /dev/null`, restam todas as outras.

Há, porém, uma observação que só aparece rodando o programa duas vezes. O arquivo é aberto com `std::ios::app`, e o demo imprime o conteúdo *inteiro* do arquivo ao final:

```
1  conteúdo:
2    [11:33:57.270]  [INFO ] Logger iniciado → /tmp/cpp_academy_logger.log
3    ...
4    [12:44:22.471]  [INFO ] Logger iniciado → /tmp/cpp_academy_logger.log
5    ...
6    [14:24:39.785]  [INFO ] Logger iniciado → /tmp/cpp_academy_logger.log
7    ...                                     ← oito sessões acumuladas
```

Listagem 75: Sétima execução consecutiva de `dist/bin/file_io`: a saída do demo cresce a cada rodada.

Para um *log* de verdade, `ios::app` é a escolha certa — perder o histórico a cada reinício seria pior. Para um *demo*, é um defeito: a saída do programa depende de quantas vezes ele já rodou, o que impede compará-la com uma saída de referência e, portanto, testá-la.

Regra — um programa de demonstração deve ser idempotente

Rodar duas vezes tem de produzir a mesma saída. Onde o assunto exige estado persistente, o programa deve começar limpando o que ele mesmo criou. Este repositório aprendeu a regra do jeito difícil: ao habilitar os alvos que antes não eram compilados, o demo de `src/database/` passou na primeira execução e falhou na segunda com violação de restrição `UNIQUE`, porque inseria as mesmas linhas de novo. A correção — apagar a tabela antes de popular — é de uma linha; achar o problema exigiu *rodar duas vezes*, que é justamente o que ninguém faz com um alvo que acabou de compilar.

II.6.8 `std::filesystem`: a API dupla e a corrida

O `<filesystem>` do C++17 fecha a seção. A Seção III.3 já apresenta o `fs::path` e os iteradores de diretório; o que cabe aqui, enquanto o assunto ainda é tratamento de erro, são duas propriedades da API.

A primeira é que **toda função tem duas assinaturas**: uma que lança `fs::filesystem_error` e outra que recebe um `std::error_code&` e não lança. O demo executa as duas sobre o mesmo arquivo inexistente, e a diferença de qualidade da mensagem é visível:

```
1  exceção: filesystem error: cannot get file size:
2           No such file or directory [/tmp/nao_existe_absolutamente.xyz]
3  path1:   /tmp/nao_existe_absolutamente.xyz
4  code:    No such file or directory
5  error_code: No such file or directory (valor=2)
```

Listagem 76: `dist/bin/file_io`, demo 5, seção 5.

A exceção carrega o *caminho* — `path1()` e, em operações de dois argumentos, `path2()` — que o `error_code` sozinho não tem. O critério de escolha é a frequência esperada do erro: um arquivo de configuração que deveria existir e não existe é excepcional; percorrer um diretório em que arquivos podem sumir no meio do caminho, não.

A segunda propriedade é uma armadilha de concorrência que a API não tem como evitar.

Armadilha — `exists()` responde sobre o passado

`if (fs::exists(p)) abrir(p);` é uma corrida: entre a pergunta e a abertura, outro processo pode apagar, criar ou substituir `p` por um *link* simbólico. O padrão tem nome — TOCTOU, *time of check to time of use* — e é uma classe inteira de vulnerabilidades de segurança. O remédio é não perguntar: tente a operação e trate a falha, que é o que o par `ifstream` + `is_open()` já faz naturalmente. Use `exists()` para relatar, nunca para decidir.

Vale registrar ainda que `fs::permissions` manipula os bits POSIX com `perm_options::add` e `::remove`, e que o demo os usa para remover a permissão de escrita do dono — e depois restaurá-la, sem o que o `fs::remove` final falharia. O detalhe é pequeno e ensina algo geral: um demo que altera permissões precisa desfazer o que fez, pela mesma razão da regra de idempotência acima.

Aviso

O `README.md` do `src/file_io/` ainda instrui a "ativar o subprojeto em `src/meson.build`" descomentando um `subdir`. Essa instrução está obsoleta: **não há mais nenhum `subdir()` comentado** — os 57 subprojetos são construídos, e a compilação instala 98 binários em `dist/bin`, sem um *warning* sob `-Wall -Wextra -Wpedantic`. A mudança não foi cosmética: ao habilitar os alvos que estavam desligados, apareceram quatro `#include` faltando — entre eles o `<fmt/core.h>` de `src/aws/s3/main.cpp` — e dois bugs de ABI em `src/asm_course/` que estavam ali havia meses. **Um alvo que não é construído é um alvo que apodrece**: o compilador é o único leitor que nunca se distrai, e desligá-lo de um diretório equivale a parar de revisar aquele diretório.

II.7 O modelo de objeto

Herança e `virtual` aparecem em quase todo subprojeto do repositório — `src/design-of-pattern/`, `src/plugin_registry/`, `src/ecs/` — mas sempre *usados*, nunca *explicados*. O subprojeto `src/oop/` fecha essa lacuna: o que o compilador gera por baixo do `virtual`, e os quatro modos clássicos de o polimorfismo desaparecer sem que ninguém perceba.

II.7.1 Herança

Das três formas de herdar, apenas a pública cria substituibilidade (Tabela 28). As outras duas são composição com sintaxe diferente — e composição costuma ser a escolha melhor, com a vantagem de não convidar ninguém a fazer *upcast*.

Tabela 28: As três formas de herança.

Forma	Significado	Converte para a base?
`public`	"é um"	sim
`protected`	"é implementado em termos de"	não
`private`	"é implementado em termos de"	não

A ordem de construção é base, depois membros na ordem de *declaração*, depois o corpo do construtor; a destruição é o inverso exato.

Armadilha — declarar um membro esconde *todos* os *overloads* do mesmo nome

```
struct Base  { void f(int); void f(const std::string&); };
struct Ruim : Base { void f(double); };          // escondeu os dois

Ruim r;
r.f(42);          // chama f(double) — 42 vira 42.0, em silêncio
r.f("texto");     // nem compila
```

A busca de nome para assim que encontra o nome, *antes* da resolução de sobrecarga: os *overloads* da base nem chegam a ser candidatos. A correção é uma linha — `using Base::f;`.

II.7.2 Funções virtuais e a *vtable*

O mecanismo é conhecido, mas convém dizê-lo com precisão: cada classe polimórfica ganha uma vtable / vptr — um vetor de ponteiros de função — e cada *objeto* ganha um *vptr* apontando para a tabela da sua classe real. A chamada `p->f()` vira, aproximadamente, `(*p->vptr[índice_de_f])(p)`. O subprojeto mede as consequências:

```
sizeof(SemVirtual) = 8 bytes (2 ints)
sizeof(ComVirtual) = 16 bytes (2 ints + vptr + padding)
vptr de dois Círculos são iguais? true     ← vtable é POR CLASSE
vptr de Círculo == vptr de Forma? false  ← vptr é POR OBJETO
```

Listagem 77: Saída de `src/oop/`: a *vtable* é por classe, o *vptr* é por objeto.

O custo em memória é um ponteiro por objeto. O custo maior, quase sempre, é outro: a chamada virtual **não pode sofrer *inline***, e com isso se perdem as otimizações que dependiam de ver o corpo da função.

`override` e o bug silencioso. Uma assinatura ligeiramente diferente não sobrescreve nada — cria um *overload* novo, e as chamadas pela base continuam indo para a base:

```
struct Sensor {
    virtual double ler(int /*canal*/) const { fmt::print("       Sensor::ler(int)\n"); return 0.0; }
    virtual ~Sensor() = default;
};

struct SensorQuebrado : Sensor {
    // Repare no 'double' em vez de 'int': assinatura diferente.
    // Isto NÃO sobrescreve nada — cria um novo overload.
    double ler(double canal) const { fmt::print("       SensorQuebrado::ler(double)\n");
                                     return canal; }
};

struct SensorCerto : Sensor {
    // 'override' faz o compilador VERIFICAR que existe um virtual com esta
    // assinatura na base. Se não existir, é erro de compilação.
    double ler(int canal) const override { fmt::print("       SensorCerto::ler(int)\n");
                                           return canal; }
};
```

Listagem 78: O bug que `override` previne, em `src/oop/demo_virtual.hpp`.

Regra — `override` sempre; `final` quando a classe não é para herdar

`override` custa zero em tempo de execução e transforma um bug silencioso em erro de compilação. `final` vai além: impede herdar a classe ou sobrescrever o método, e permite ao compilador *devirtualizar* chamadas — o que devolve a possibilidade de *inline*.

Destrutor virtual. `delete ptr_base` com destrutor não virtual é UB (comportamento indefinido): o destrutor do derivado simplesmente não roda, e o que ele liberaria vaza. O GCC recusa com `-Wdelete-non-virtual-dtor` e o AddressSanitizer aborta o programa — razão pela qual a linha correspondente está comentada no subprojeto, de propósito.

Virtual dentro do construtor. Durante o construtor da base, o *vptr* ainda é o da base: a chamada não desce para o derivado. Faz sentido — os membros do derivado ainda não existem — e é exatamente por isso que a construção em duas fases ("construtor chama `inicializar()` virtual") não funciona em C++.

Armadilha — argumento padrão em função virtual

```
struct BaseArg {
    virtual void f(int x = 10) const { fmt::print("       BaseArg::f(x={})\n", x); }
    virtual ~BaseArg() = default;
};
struct DerivadaArg : BaseArg {
    void f(int x = 20) const override { fmt::print("       DerivadaArg::f(x={})\n", x); }
};
```

O *corpo* é escolhido dinamicamente, mas o *argumento padrão* é escolhido estaticamente, pelo tipo do ponteiro. Chamado por `Derivada d; d.f()` imprime `x=20`; chamado por `Base& b = d; b.f()` imprime `x=10` — corpo do derivado, valor padrão da base. A solução é o idioma NVI: um método público não virtual aplica o padrão e delega a um virtual privado.

II.7.3 *Object slicing*

Copiar um derivado para uma variável do tipo base não é conversão: é **amputação**. O que não cabe em `sizeof(Base)` é descartado, e com ele o *vptr* — ou seja, o polimorfismo.

```
struct Animal {
    std::string nome;

    explicit Animal(std::string n) : nome(std::move(n)) {}
    virtual ~Animal() = default;

    virtual std::string falar() const { return "..."; }
    virtual std::string descrever() const { return fmt::format("Animal '{}'", nome); }
};

struct Cachorro : Animal {
    int numero_de_truques;          // ← estado extra, some no corte

    Cachorro(std::string n, int truques)
        : Animal(std::move(n)), numero_de_truques(truques) {}

    std::string falar() const override { return "Au!"; }
    std::string descrever() const override {
        return fmt::format("Cachorro '{}' com {} truques", nome, numero_de_truques);
    }
};
```

Listagem 79: Os tipos de `src/oop/demo_slicing.hpp`: o estado extra é o que some no corte.

O caso que aparece em produção não é a atribuição explícita: é o **parâmetro**. A diferença entre `void f(Animal a)` e `void f(const Animal& a)` é um caractere, e nenhum *warning* separa as duas.

Armadilha — `std::vector<Base>` é sempre errado

O container reserva `sizeof(Base)` por posição. Não existe espaço físico para um derivado maior — não é uma questão de política, é de aritmética:

```
sizeof(Animal)   = 40 bytes
sizeof(Cachorro) = 48 bytes
→ dois Cachorros entraram no vector<Animal>, dois Animals saíram
```

A correção é guardar indireção: `std::vector<std::unique_ptr<Animal»`.

O corte pode ser tornado incompilável, e vale escolher a receita conforme a intenção: base abstrata (um `= 0`) é a mais forte, porque nem instancia; cópia `= delete` na base tira a cópia também dos derivados; cópia `protected` deixa derivados copiarem entre si e impede o mundo externo de cortar.

II.7.4 Herança múltipla e o diamante

Herdar de várias *interfaces* puras — só virtuais puros, sem estado — não tem o problema do diamante, porque não há dados a duplicar. Mas há um efeito de *layout* que surpreende: os subobjetos ficam em *offsets* diferentes, e converter o ponteiro **ajusta o endereço**.

```
&b              = 0x7ffc2c9a4700
(Desenhavel*) = 0x7ffc2c9a4700
(Salvavel*)   = 0x7ffc2c9a4708   ← mesmo objeto, endereço diferente
```

Listagem 80: Saída de `src/oop/`: o mesmo objeto, três endereços.

Comparar ponteiros de bases diferentes, portanto, não responde se são o mesmo objeto. Quem responde é `dynamic_cast<void*>`, que devolve o endereço do objeto mais derivado.

Quando as bases têm estado, o diamante aparece: sem `virtual`, a classe final contém *duas* cópias da base comum, com estados independentes, e a conversão para ela é ambígua — nem compila. Com herança virtual, contém uma, e o `sizeof` cai (96 para 64 bytes, no exemplo do subprojeto).

Armadilha — quem inicializa a base virtual é a classe *mais derivada*

Numa hierarquia em diamante com herança virtual, as chamadas ao construtor da base virtual escritas nas classes intermediárias são simplesmente **ignoradas**. Quem escolhe com que argumentos a base virtual é construída é a classe mais derivada — e isso significa que toda classe concreta da hierarquia precisa saber inicializar uma base da qual talvez nem se considere responsável.

Quando a variação é conhecida em compilação, nada disso é necessário: o CRTP dá polimorfismo estático sem *vtable* e sem *vptr*, e *mixins* vazios desaparecem por otimização de base vazia (Seção IV.1).

II.7.5 RTTI e o seu custo

`typeid` aplicado a um objeto **polimórfico** é avaliado em tempo de execução e devolve o tipo *dinâmico*; aplicado a um não polimórfico, é resolvido em compilação e devolve o tipo *estático*. A diferença entre as duas situações é uma função virtual na base.

```
typeid(c).name() cru       = N9demo_rtti7CirculoE
demanglado                 = demo_rtti::Circulo
typeid(f) via Forma&       = demo_rtti::Circulo          ← dinâmico
não-polimórfica via base&  = demo_rtti::NaoPolimorfica   ← estático
```

Listagem 81: Saída de `src/oop/`: o nome cru vem no formato do Itanium ABI [5] e precisa de `abi::__cxa_demangle`.

`typeid` testa igualdade *exata* de tipo; `dynamic_cast` aceita derivados. São perguntas diferentes: um quadrado que herda de retângulo falha no primeiro teste e passa no segundo.

A medição do subprojeto, sobre 200 000 iterações de cada operação, ordena os custos: chamada virtual é a mais barata, comparação de `typeid` vem depois, e `dynamic_cast` é a mais cara por boa margem — porque percorre a árvore de herança e não é $O(1)$.

Regra — uma cadeia de `if (dynamic_cast<X>)` é uma função virtual que não foi escrita

As três alternativas, em ordem de preferência: (a) um método virtual, e o objeto responde por si; (b) o padrão *Visitor*, que faz despacho duplo sem RTTI (Seção V.2); (c) `std::variant` com `std::visit`, quando a hierarquia é fechada — sem herança nenhuma, e com verificação de exaustividade pelo compilador (Seção III.4).

II.8 Tratamento de erro

O subprojeto `src/errors/` trata do mecanismo das exceções, dos quatro níveis de garantia que toda função oferece — escolhendo-os ou não — e das duas alternativas baseadas em valor. Ele compila com C++23 por causa de `std::expected` (Seção I.6).

A decisão entre exceção e valor de retorno não é questão de gosto. O subprojeto mede:

```
sem exceção          :      12 µs
lançando+capturando:  12716 µs     ← ~1000×
```

Listagem 82: Saída de `src/errors/`: 20 000 iterações.

Exceções são **zero-cost no caminho feliz** e caríssimas no caminho de erro. É isso, e não a estética, que decide quando usá-las: para o que é excepcional e raro, exceção; para o que é esperado e frequente, valor.

II.8.1 O mecanismo, e por que ele é o mesmo assunto que RAII

Um `throw` faz quatro coisas: copia ou move o objeto lançado para uma área controlada pelo *runtime*; desenrola a pilha até o primeiro *handler* compatível; **roda todo destrutor de objeto local** durante o desenrolamento; executa o *handler*. O terceiro passo é o que torna RAII e exceções indissociáveis:

```
+ nível−1 construído
+ nível−2 construído
+ nível−3 construído
nível−3 vai lançar ...
− nível−3 DESTRUÍDO (durante o unwinding)
− nível−2 DESTRUÍDO (durante o unwinding)
− nível−1 DESTRUÍDO (durante o unwinding)
handler alcançado: erro originado no nível 3
```

Listagem 83: Saída de `src/errors/`: três destrutores rodaram sem uma linha de limpeza escrita à mão.

Os *handlers* são testados **em ordem**, de cima para baixo, e o primeiro compatível vence — não há "melhor correspondência" como na resolução de sobrecarga. Por isso o *handler* mais derivado vem primeiro.

Armadilha — capturar por valor *fatia* a exceção

`catch (std::runtime_error e)` sofre *object slicing* exatamente como qualquer outra cópia para o tipo base (Seção II.7): o objeto derivado chega degradado, e o `what()` vira genérico. O GCC pega com `-Wcatch-value`. Lançar, por outro lado, é sempre por valor: `throw T(...)`, nunca `throw new T(...)` — este último vaza.

A mesma armadilha reaparece no repasse. `throw;` relança *o mesmo* objeto, com tipo dinâmico intacto; `throw e;` lança uma **cópia** cortada para o tipo estático da variável do `catch`:

```
com 'throw;'     → chegou como ErroDeRede: [HTTP 404] não encontrado
com 'throw e;'   → chegou degradado como std::exception: 'std::exception'
```

Listagem 84: Saída de `src/errors/`: a diferença de um caractere.

II.8.2 As quatro garantias

Toda função oferece uma das garantias da Tabela 29, tenha o autor pensado nisso ou não. A escolha razoável é mirar na básica sempre, e na forte quando o custo permitir.

Tabela 29: As quatro garantias de segurança em face de exceções.

Garantia	Promessa
nenhuma	nada — pode vazar ou corromper
básica	não vaza, invariantes valem, valores não especificados
forte	ou completou, ou nada mudou (*commit-or-rollback*)
nothrow	não lança, ponto

Subir de nível costuma custar menos do que parece — às vezes apenas **reordenar as operações**, deixando o que pode lançar antes do que muda estado:

```
void ruim(const std::string& k) {
    ++contador_;                                  // estado já mudou
    itens_.push_back(std::make_unique<Item>(k));  // e isto pode lançar
}

void bom(const std::string& k) {
    auto novo = std::make_unique<Item>(k);        // o arriscado primeiro
    itens_.push_back(std::move(novo));
    ++contador_;                                  // o nothrow por último
}
```

Listagem 85: A mesma função, com garantia diferente.

Para a garantia forte, o instrumento é o mesmo *copy-and-swap* da Seção II.3: faça o trabalho arriscado numa cópia e, só quando tudo tiver dado certo, troque — porque `swap` é *nothrow*.

Armadilha — destrutor que lança durante o *unwinding* chama `std::terminate`

Se um destrutor lançar enquanto outra exceção está em voo, há duas exceções simultâneas e o padrão responde encerrando o programa. Por isso todo destrutor é implicitamente `noexcept`. O padrão correto é engolir e registrar o erro no destrutor, e oferecer um `fechar()` público para quem quiser tratá-lo de verdade — é assim que `std::ofstream` se comporta.

Regra — `noexcept` é promessa, não impedimento

`noexcept` não impede a função de lançar: promete que ela não lança. Quebrar a promessa não produz *unwinding* nem *handler* — `std::terminate` direto. Em troca, a biblioteca otimiza: é o que faz `std::vector` mover em vez de copiar ao realocar. Marque `noexcept` em construtores e atribuições de *move*, em `swap`, em destrutores e em *getters* triviais.

II.8.3 A hierarquia padrão

A pergunta que separa os dois grandes ramos é uma só: *isto poderia ter sido evitado por quem chamou, verificando antes?*

- **Sim** → `std::logic_error` e derivados (`invalid_argument`, `domain_error`, `length_error`, `out_of_range`). É bug. Não capture em produção; corrija o código.
- **Não** → `std::runtime_error` e derivados (`range_error`, `overflow_error`, `system_error`). É o mundo. Trate, tente de novo, faça *fallback*.

Uma exceção customizada deve derivar de `std::runtime_error`, que já gerencia a *string* do `what()` — e essa *string* precisa sobreviver ao objeto lançado, o que é a razão de não se poder montá-la na hora dentro do `what()` e devolver `.c_str()` de um temporário.

Vale também dar ao seu domínio uma **raiz própria**:

```
1 struct ErroDaAcademia : std::runtime_error { using std::runtime_error::runtime_error; };
2 struct ErroDeParse    : ErroDaAcademia     { using ErroDaAcademia::ErroDaAcademia; };
3 struct ErroDeRede     : ErroDaAcademia     { int codigo; /* ... */ };
```

Listagem 86: Uma raiz por domínio permite capturar tudo o que é seu sem pegar o que não é.

Para acrescentar contexto sem apagar a causa, `std::throw_with_nested` na subida e `std::rethrow_if_nested` na leitura produzem o rastro encadeado:

```
1 falha ao iniciar o servidor na porta 443
2   não foi possível carregar o certificado TLS
3     arquivo '/etc/ssl/cert.pem' não encontrado
```

Listagem 87: Saída de `src/errors/`: cada camada acrescenta contexto.

Regra — `catch(...)` é legítimo em exatamente três lugares

No `main`, na função de entrada de uma *thread* — onde uma exceção que escapa chama `std::terminate`, e o remédio é `std::exception_ptr` — e numa fronteira com código C, que não sabe desenrolar pilha de C++. Em qualquer outro ponto, `catch(...)` está escondendo um bug.

II.8.4 Erros como valores

`std::error_code` é um par (valor inteiro, categoria) que identifica um erro de forma extensível, sem alocar e sem lançar. A distinção que o torna interoperável é entre `error_code` — o que *aconteceu*, específico da plataforma ou da biblioteca — e `error_condition` — o que aquilo *significa*, portável. Uma categoria

própria que implemente `default_error_condition` passa a ser comparável com `std::errc`, e é assim que Asio, `<filesystem>` e o código do usuário coexistem sem conhecer uns aos outros.

Armadilha — a polaridade do `bool` é invertida entre as duas formas

Com `std::error_code`, `if (ec)` significa **houve erro**. Com `std::expected`, `if (r)` significa **sucesso**. As duas convenções coexistem no mesmo programa, e a confusão entre elas produz código que inverte o tratamento de erro sem que nada deixe de compilar.

`std::expected<T, E>` (C++23) resolve o que faltava às outras formas: `optional<T>` diz que não há valor mas não diz por quê; `error_code` diz o porquê mas não carrega o valor.

```
std::expected<int, Falha> para_inteiro(std::string_view s) noexcept {
    if (s.empty()) return std::unexpected{Falha::vazio};

    int valor = 0;
    const auto [ptr, ec] = std::from_chars(s.data(), s.data() + s.size(), valor);

    if (ec == std::errc::result_out_of_range) return std::unexpected{Falha::fora_do_intervalo};
    if (ec != std::errc{} || ptr != s.data() + s.size())
        return std::unexpected{Falha::caractere_invalido};

    return valor;                          // sucesso: conversão implícita para expected
}
```

Listagem 88: `std::expected` em `src/errors/demo_expected.hpp`.

O ganho real, porém, é o encadeamento monádico: a primeira falha curto-circuita todas as etapas seguintes, e nenhum `if` aparece no meio do *pipeline*.

```
    auto pipeline = [](std::string_view entrada) {
        return para_inteiro(entrada)              // expected<int, Falha>
            .and_then(exigir_positivo)            // só roda se deu certo
            .transform([](int v) { return v * 2; })    // mapeia só o sucesso
            .transform([](int v) { return fmt::format("resultado={}", v); });
    };
```

Listagem 89: O *pipeline* de `src/errors/demo_expected.hpp`.

Tabela 30: As quatro operações monádicas de `std::expected`.

Operação	Recebe	Devolve	Papel
`and_then(f)`	`T`	`expected<U,E>`	encadeia
`transform(f)`	`T`	`U`	mapeia o sucesso
`or_else(f)`	`E`	`expected<T,G>`	recupera
`transform_error(f)`	`E`	`G`	traduz o erro

Armadilha — `std::expected` não é `[[nodiscard]]` no padrão

Ignorar o retorno de uma função que devolve `expected` compila sem um pio — e o erro desaparece por completo, o que é pior do que a exceção que se queria evitar. Marque você as suas funções: `[[nodiscard]] std::expected<int,Falha> para_inteiro(...);`

II.8.5 Escolhendo a forma

E os dois critérios que decidem, na prática:

Tabela 31: As cinco formas de reportar erro, comparadas.

Forma	Motivo do erro	Custo	Ignorável?
`optional<T>`	não	zero	sim
exceção	sim, rico	alto	não
out-param + `bool`	via bool/enum	zero	sim, e em silêncio
`error_code`	sim, extensível	zero	sim
`expected<T,E>`	sim, **tipado**	zero	sim, se não marcar

- **Frequência.** Esperado e frequente → `expected` ou `error_code`. Excepcional e raro → exceção. Bug do chamador → `assert` ou `logic_error`.
- **Distância.** Se o erro precisa atravessar muitas camadas que nada têm a ver com ele, exceção — propagar `expected` muda a assinatura de todas elas. E em construtor ou operador, exceção é a única opção: não há valor de retorno disponível.

II.9 Síntese: os erros que o compilador não denuncia

O capítulo inteiro se resume na Tabela 32, que reúne as armadilhas das oito seções e — a coluna que importa — o que fazer a respeito.

Quatro delas — *slicing*, destrutor não virtual, *move* suprimido e uso após *move* — envolvem tempo de vida ou *layout* de objeto, e para essas o instrumento certo não é a leitura atenta, é o *sanitizer* (Seção I.9). Os subprojetos `src/value_semantics/`, `src/oop/` e `src/errors/` recomendam explicitamente uma passada sob `-Db_sanitize=address,undefined` justamente por isso.

Tabela 32: Os erros silenciosos do capítulo, e a correção de cada um.

Erro	Seção	Correção
`int t = 3.99;` trunca	II.1	inicialize com `{}`
lista de inicialização fora de ordem	II.1	`-Wreorder`; siga a declaração
cast à moda C esconde qual conversão	II.2	`-Wold-style-cast`
escrever em objeto `const`	II.2	não use `const_cast` para isso
ler `float` por `int*`	II.2	`std::bit_cast` ou `memcpy`
destrutor vazio suprime os *moves*	II.3	Rule of 0; ou `= default` nos cinco
move ctor sem `noexcept`	II.3	marque `noexcept`
`return std::move(local)`	II.3	`return local;`
`move` onde cabia `forward`	II.3	`forward<T>` em tipo deduzido
sobrecarregar `&&` ou `||`	II.4	não sobrecarregue
construtor de um argumento sem `explicit`	II.4	`explicit` por padrão
`a[i, j]` recompilado em C++23	II.4	`-Wcomma-subscript`
ciclo de `shared_ptr`	II.5	`weak_ptr` num dos lados
`shared_ptr<T>(this)` num método	II.5	`enable_shared_from_this`
weak vivo retém o objeto inteiro	II.5	`shared_ptr(new)` se for grande
`release()` usado como empréstimo	II.5	`get()`, ou passe `*p`
`while (!ifs.eof())`	II.6	ponha a leitura na condição
`gcount()` lido fora do laço	II.6	consulte logo após cada `read`
`'–'` entre aspas simples	II.6	literal de `string`; UTF-8 tem 3 bytes
`40 - s.size()` dá a volta	II.6	`-Wsign-conversion`; converta antes
`cout.rdbuf()` não pega `fmt::print`	II.6	`dup2`, ou injete o *stream*
`std::endl` em laço	II.6	`'\n'`; 32× medido
`fs::exists()` antes de abrir	II.6	tente e trate a falha (TOCTOU)
método derivado esconde *overloads*	II.7	`using Base::f;`
assinatura errada ao "sobrescrever"	II.7	`override` sempre
`delete` por base sem dtor virtual	II.7	destrutor virtual, ou `final`
slicing em parâmetro por valor	II.7	`const T&`; base abstrata
argumento padrão em virtual	II.7	idioma NVI
capturar exceção por valor	II.8	`catch (const T&)`; `-Wcatch-value`
`throw e;` para repassar	II.8	`throw;`
retorno `expected` ignorado	II.8	`[[nodiscard]]`

III A biblioteca padrão

A biblioteca padrão do C++ não é um repertório de utilidades avulsas: é um *sistema*, com um princípio de projeto explícito. Containers de um lado, algoritmos do outro, e iterador como a cola que evita $N \times M$ implementações — N containers vezes M algoritmos.

Este capítulo percorre esse sistema em seis camadas: os containers e os algoritmos que os manipulam (Seção III.1); a releitura que o C++20 fez de tudo isso com *ranges* (Seção III.2); a entrada e a saída (Seção III.3); os tipos de vocabulário que atravessam fronteiras de biblioteca (Seção III.4); por baixo de tudo, de onde vem a memória que os containers usam (Seção III.5); e, por último, o menor cabeçalho da biblioteca inteira — o `<coroutine>`, com que o compilador constrói funções capazes de parar no meio (Seção III.6).

O fio condutor é o mesmo do Seção II: a escolha errada aqui não produz erro de compilação nem resposta errada. Produz **lentidão silenciosa** — ou, pior, UB (comportamento indefinido) num programa que passa nos testes.

III.1 Containers, iteradores e algoritmos

O subprojeto `src/stl/` abre com uma medição que resume o capítulo inteiro:

```
vector  (memmove contíguo) :     7 ms
list    (salto de ponteiro):   976 ms      ← 140×
```

Listagem 90: Saída de `src/stl/`: 20 000 inserções mantendo a ordem.

A `std::list` tem a **melhor** complexidade assintótica para inserção no meio — $O(1)$ dado o iterador, contra $O(n)$ do `vector` — e o **pior** desempenho real, por duas ordens de grandeza.

Por que — por que a complexidade assintótica mente aqui

A notação $O(\cdot)$ conta operações e supõe que todas custam igual. Numa máquina com cache, isso é falso por um fator de aproximadamente cem: percorrer elementos contíguos aproveita a linha de cache inteira e o *prefetcher* do processador, enquanto seguir ponteiros para nós espalhados na *heap* paga uma falta de cache por elemento. O `memmove` do `vector` move muitos bytes, mas os move na velocidade da memória; a `list` move poucos bytes, na velocidade das faltas de cache.

III.1.1 Os containers e o que custa cada um

Tabela 33: Complexidade e disposição em memória dos containers principais.

Container	Acesso	Inserção	Busca	Memória
`vector`	$O(1)$	$O(1)$ no fim*	$O(n)$	contígua
`deque`	$O(1)$	$O(1)$ nas duas pontas	$O(n)$	blocos
`list`	$O(n)$	$O(1)$ com iterador	$O(n)$	nós
`map` / `set`	—	$O(\log n)$	$O(\log n)$	nós
`unordered_map` / `set`	—	$O(1)$ médio	$O(1)$ médio	baldes

* amortizado: o *buffer* dobra a cada estouro.

O crescimento amortizado do `vector` é o motivo de `reserve()` existir — e o subprojeto mostra o efeito:

```
inserindo 20 elementos: 0→1 1→2 2→4 4→8 8→16 16→32
com reserve(20):        0 realocações
```

Listagem 91: Saída de `src/stl/`: o mesmo laço, com e sem `reserve`.

Armadilha — `clear()` não devolve memória, e `shrink_to_fit()` é um pedido

`clear()` zera o `size` e mantém a `capacity` intacta — comportamento correto, porque o container supõe que você vai reencher. `shrink_to_fit()` é uma *requisição não vinculante*: a implementação tem o direito de ignorá-la. A forma que funciona sempre é o idioma do vetor vazio: `std::vector<int>{}.swap(v);`.

III.1.2 Invalidação de iteradores

A invalidação de iteradores é a origem mais comum de UB (comportamento indefinido) em código que "funcionava". As regras variam por container (Tabela 34) e não são intuitivas — em particular, a distinção entre invalidar *iteradores* e invalidar *referências* pega quase todo mundo.

Tabela 34: O que cada operação invalida.

Operação	Invalida
`vector`: realocação	**tudo**
`vector`: `erase(it)`	de `it` até o fim
`deque`: *insert*/*erase* no meio	tudo
`deque`: `push_front/back`	iteradores sim, referências não
`list`: qualquer	só o elemento removido
`map` / `set`: `insert`	nada
`unordered_*`: *rehash*	iteradores sim, referências não

O caso prático mais frequente é apagar durante a iteração:

```cpp
// errado: `it` é invalidado por erase, e o ++it opera sobre lixo
for (auto it = v.begin(); it != v.end(); ++it)
    if (pred(*it)) v.erase(it);

// certo: erase DEVOLVE o iterador do próximo
for (auto it = v.begin(); it != v.end(); )
    it = pred(*it) ? v.erase(it) : std::next(it);

// melhor (C++20)
std::erase_if(v, pred);
```

Listagem 92: A forma errada, a correta e a moderna.

Regra — guarde índice, não iterador nem referência

Se uma posição em um `vector` precisa sobreviver a uma inserção, guarde o *índice*. O índice continua correto depois da realocação; o iterador e a referência, não. Vale também para `std::span` e para pares de iteradores guardados como membro (Seção III.4).

III.1.3 `emplace` contra `push`, medido

A crença comum é que `emplace_back` é sempre melhor. A medição diz o que ele realmente muda:

```
push_back(Caro(...))        ctor=1 cópias=0 moves=1
emplace_back("a","b")       ctor=1 cópias=0 moves=0   ← constrói no lugar
push_back(lvalue)           ctor=0 cópias=1 moves=0
push_back(std::move(c))     ctor=0 cópias=0 moves=1   ← idêntico a emplace
```

Listagem 93: Saída de `src/stl/`: contadores instrumentados.

`emplace` ganha quando você tem os **argumentos** do construtor. Com um objeto já pronto, `push_back(std::move(x))` faz exatamente o mesmo.

Armadilha — `emplace_back` aceita conversões `explicit` em silêncio

Como `emplace` chama o construtor diretamente, ele passa por cima do `explicit` que você pôs justamente para impedir a conversão acidental (Seção II.4). O que `push_back` recusaria, `emplace_back` constrói sem comentar.

Para `std::map`, a diferença entre as quatro formas de inserir é semântica, não de desempenho (Tabela 35) — e escolher errado sobrescreve ou deixa de sobrescrever dados.

Tabela 35: As quatro formas de pôr um par no `map`.

Forma	Comportamento
`m[k] = v`	insere ou sobrescreve; exige construtor padrão
`insert({k,v})`	**não** sobrescreve
`insert_or_assign(k,v)`	sobrescreve
`try_emplace(k, args...)`	só constrói o valor se a chave faltar

III.1.4 Iteradores: categorias e sentinelas

A categoria de um iterador não é rótulo documental — ela decide **o que compila**. `*(v.begin() + 3)` funciona sobre um `vector` e não compila sobre uma `list`, porque `list` não tem `operator+`. A saída genérica é `std::advance`, que é $O(1)$ ou $O(n)$ conforme a categoria — e essa especialização, que antes do C++17 exigia *tag dispatch* com *overloads*, hoje é um `if constexpr`.

Escrever um iterador do zero exige cinco *typedefs* — `iterator_category`, `value_type`, `difference_type`, `pointer`, `reference` — e os operadores da categoria escolhida. O `Intervalo` do subprojeto gera os números sob demanda, sem armazenar nada, e ainda assim `accumulate`, `count_if` e `copy` funcionam sobre ele.

A novidade do C++20 é o **sentinela**: até então, `begin()` e `end()` tinham de ser do mesmo tipo, o que obrigava o "iterador de fim" a carregar estado inútil. Agora o fim pode ser um tipo diferente, que só sabe responder "acabou?":

```cpp
struct SentinelaDeNulo {};                // não guarda nada

class IteradorDeCString {
public:
    using iterator_category = std::forward_iterator_tag;
    using value_type        = char;
    using difference_type   = std::ptrdiff_t;
    // ...
    bool operator==(const IteradorDeCString& o) const { return p_ == o.p_; }
    // A comparação com o SENTINELA é o que fecha o laço:
    bool operator==(SentinelaDeNulo) const { return *p_ == '\0'; }

private:
    const char* p_;
```

```
15 };
```

Listagem 94: Um sentinela em `src/stl/demo_iterators.hpp` (condensado).

Isso percorre uma *C-string* **sem chamar `strlen` antes** — antes do C++20, o `end()` teria de ser um ponteiro para o `'
0'`, o que exigiria percorrer a string para poder percorrê-la. É o mesmo mecanismo que permite a `views::take_while` ser preguiçosa de verdade (Seção III.2).

III.1.5 `<algorithm>`: peça a menor garantia que resolve

Tabela 36: Os algoritmos de ordenação e o que cada um promete.

Algoritmo	Custo	Garante
`sort`	$O(n \log n)$	tudo ordenado
`stable_sort`	$O(n \log n)$ + memória	… e a ordem relativa dos iguais
`partial_sort`	$O(n \log k)$	os k primeiros
`nth_element`	$O(n)$ médio	só a posição n

Peça a **menor** garantia que resolve o problema: é a mais barata. E não confie em observação casual para decidir se precisa de estabilidade — o subprojeto registra a razão:

```
1 sort == stable_sort ? NÃO — sort embaralhou os empates
2 primeiros 6 do sort        : #116 #032 #152 #148 #036 #144
3 primeiros 6 do stable_sort: #000 #004 #008 #012 #016 #020
```

Listagem 95: Saída de `src/stl/`: com 6 elementos a instabilidade não aparece; com 200 e 4 chaves distintas, salta.

O motivo é que o *introsort* do libstdc++ cai no *insertion sort* em intervalos pequenos, e esse é estável *por acidente*. Testar com poucos elementos leva à conclusão errada.

Armadilha — o comparador precisa ser *strict weak ordering*

O comparador responde "`a` vem *antes* de `b`?" e tem de ser irreflexivo, assimétrico e transitivo. Escrever `<=` em vez de `<` torna `comp(a,a)` verdadeiro, e `std::sort` pode então **ler fora do intervalo** — é UB (comportamento indefinido) de verdade, não pedantismo. Detectável com `-D_GLIBCXX_DEBUG`. Para múltiplos critérios, `std::tie` dá o desempate lexicográfico correto de graça: `return std::tie(a.idade, a.nome) < std::tie(b.idade, b.nome);`

O mesmo vale para a busca binária: `binary_search`, `lower_bound` e `upper_bound` **exigem** sequência ordenada, e aplicá-los a uma sequência qualquer é UB (comportamento indefinido) — sem erro, sem *warning*, só resposta errada. O uso mais útil de `lower_bound`, aliás, não é buscar: é descobrir *onde inserir* mantendo a ordem.

Armadilha — o tipo do valor inicial decide como `accumulate` acumula

```
1 std::accumulate(reais.begin(), reais.end(), 0);     // truncou para int!
2 std::accumulate(reais.begin(), reais.end(), 0.0);   // correto
```

O acumulador tem o tipo do terceiro argumento, não o dos elementos. Um `0` onde cabia `0.0` soma *doubles* em uma variável `int`, truncando a cada passo.

Vale conhecer também a distinção entre `accumulate` e `reduce` (C++17): o primeiro é uma dobra à esquerda, sequencial e determinística; o segundo não garante ordem — e é exatamente por isso que pode ser paralelizado.

III.1.6 Um container que satisfaz o contrato

O subprojeto termina construindo um `RingBuffer<T, N>` — fila circular de capacidade fixa, sem alocação, com os dados num `std::array`. O contrato pago são os *typedefs* e os quatro métodos de iteração; o iterador é um *template* sobre a constância (`iterador_base<bool EhConst>`), que é o truque padrão para não escrever `iterator` e `const_iterator` duas vezes.

O que se ganha sem escrever mais nada é desproporcional: *range-for*, `accumulate`, `max_element`, `count_if`, `find`, `sort`, `make_reverse_iterator`, `ranges::sort`, `ranges::max`. E o contrato *antigo* — *typedefs* mais operadores — já satisfaz os *concepts novos*:

```
1 static_assert(std::random_access_iterator<RB::iterator>);
2 static_assert(std::ranges::random_access_range<RB>);
```

Listagem 96: O contrato clássico satisfazendo os *concepts* do C++20.

Regra — só escreva um container quando nenhuma combinação da STL resolver

O que ainda falta ao `RingBuffer` para ser um container de biblioteca dá a medida do custo real: `allocator_type`, `insert`/`erase` com iterador, `swap` achável por ADL, `operator<=>`, *deduction guides* e garantias de exceção documentadas. Na dúvida, componha.

III.1.7 Quatro armadilhas que compilam

`std::remove` não remove. Um algoritmo só enxerga iteradores: ele não tem acesso ao container e não pode mudar o tamanho. O que ele faz é *particionar*, empurrando os elementos a manter para a frente e devolvendo o novo fim lógico.

```
1 original:                1 2 3 2 4 2 5   (size=7)
2 após std::remove(2): 1 3 4 5 4 2 5   (size=7)   ← size NÃO mudou
3 elementos válidos:   1 3 4 5
```

Listagem 97: Saída de `src/stl/`: o `size` não muda.

O idioma completo é `v.erase(std::remove(v.begin(), v.end(), 2), v.end());` em C++20, `std::erase(v, 2)`. Vale para `unique`, `remove_if` e todo algoritmo que "remove".

`vector<bool>` não é um container de `bool`. É uma especialização que empacota oito valores por byte, e cujo `operator[]` devolve um *proxy*:

```
1 auto x = vb[0];      // o tipo é std::_Bit_reference, não bool
2 bool& r = vb[0];     // não compila
3 x = false;           // ← muda o vector! o "valor" era uma referência
```

Listagem 98: O *proxy* que se disfarça de valor.

As alternativas são `vector<char>`, `deque<bool>` e `bitset<N>`. Se for mesmo necessário usá-lo, escreva `bool x = vb[i];` — nunca `auto`.

`map::operator[]` insere. A linha `if (estoque["banana"] > 0)` *cria* a entrada `{"banana", 0}` como efeito colateral de uma consulta. É por isso que `operator[]` não existe em `map` constante. Para consultar: `find`, `at`, `count`, `contains` (C++20).

`string_view` de um temporário. Um *view* é um par (ponteiro, tamanho) sem posse: guardar um como membro ou devolvê-lo apontando para um temporário produz *dangling*. A regra prática é que `string_view` é *parâmetro*, não membro nem retorno — e que `.data()` dele **não é terminado em nulo**, portanto não pode ser passado a uma API C.

III.2 Ranges: a mesma biblioteca, repensada

O `<ranges>` do C++20 é a maior adição do padrão à biblioteca, e o subprojeto `src/ranges/` justifica a existência dele com uma comparação direta. Encadear algoritmos clássicos custa N passadas e $N-1$ containers temporários:

```
std::vector<int> passo1; std::copy_if(v.begin(), v.end(), back_inserter(passo1), pred);
std::vector<int> passo2; std::transform(passo1.begin(), passo1.end(), back_inserter(passo2), f);
std::vector<int> passo3(passo2.begin(), passo2.begin() + 4);
// → 2 temporários, 3 passadas, e processou os 20 elementos antes de descartar 16

auto r = v | vw::filter(pred) | vw::transform(f) | vw::take(4);
// → 0 temporários, 1 passada, e PARA assim que junta 4
```

Listagem 99: O mesmo problema, com algoritmos clássicos e com *views*.

III.2.1 As duas propriedades que definem uma *view*

Uma *view* (C++20) é **preguiçosa** — o trabalho acontece na iteração, não na construção — e **não possui** os dados, de modo que copiá-la é $O(1)$. A preguiça não é alegação: os *lambdas* do subprojeto imprimem quando são chamados, e a saída mostra que *nada* acontece até alguém iterar, e que cada elemento atravessa o *pipeline* inteiro antes do próximo:

```
construindo a view (nada deve ser impresso):
 ... view construída. Nenhum elemento foi tocado.
agora iterando:
    filter (1)
    filter (2)
   transform(2)
   → 20
    filter (3)
    ...
```

Listagem 100: Saída de `src/ranges/`: não há vetor intermediário em lugar nenhum.

A distinção entre `take_while` e `filter` decorre disso: o primeiro **para** no primeiro elemento que falha, o segundo testa todos. Sobre `1 2 9 3 4`, `take_while(< 5)` devolve `1 2` e `filter(< 5)` devolve `1 2 3 4`.

E é a preguiça que torna possível operar sobre sequências **infinitas**:

```
vw::iota(1)                                  // 1, 2, 3, ... sem fim
    | vw::transform([](int n) { return n*n; })
    | vw::filter([](int n) { return n % 2; })
    | vw::take(8);                           // → 1 9 25 49 81 121 169 225
```

Listagem 101: Uma *view* infinita, tornada finita pelo `take`.

III.2.2 O `operator|` não é mágica

As três formas abaixo são a mesma coisa, e a primeira existe só para inverter a ordem de leitura:

```
v | vw:: filter (p)          // pipe
vw:: filter (p)(v)           // chamada
rg :: filter _view (v, p)    // construtor
```

Listagem 102: `a | b | c` é literalmente `c(b(a))`.

O ganho estrutural é que adaptadores são *objetos*: podem ser nomeados, guardados e **compostos antes de ver um range**. Não existe equivalente clássico — não há como guardar "um `copy_if` seguido de um `transform`" numa variável.

```
auto so_pares       = vw:: filter ([]( int x) { return x % 2 == 0; });
auto dobrar         = vw::transform([](int x) { return x * 2; });
auto pares_dobrados = so_pares | dobrar;      // ainda sem nenhum range

a | pares_dobrados;   b | pares_dobrados;     // reutiliz ável
```

Listagem 103: Um adaptador composto, reutilizável.

III.2.3 Tempo de vida: `ref_view`, `owning_view` e o *dangling*

O que acontece com a origem do *pipeline* depende da categoria de valor dela (Seção II.3), e a consequência é contraintuitiva.

Tabela 37: O que o *pipeline* faz com a origem.

Origem	O que acontece
lvalue (`v`)	`ref_view` — 8 bytes, referencia
prvalue (`produz()`)	`owning_view` — **assume a posse**, move para dentro

Armadilha — o temporário é seguro; a variável local não é

```
auto r = produz_vetor() | vw::filter(par);     // SEGURO: owning_ view assumiu a posse

auto f() {
    std :: vector<int> local{1,2,3};
    return local | vw:: filter (par);          // ref_ view para ‘local ‘
}                                               // ‘local ‘ morre aqui → dangling
```

A intuição de que "temporário é perigoso, variável local é segura" se inverte por completo. O conceito que responde à pergunta certa — "os iteradores sobrevivem ao objeto?" — é `borrowed_range`: falso para `std::vector<int>`, verdadeiro para `std::vector<int>&`, `std::string_view` e `std::span<int>`. É por isso que devolver um `span` é seguro e devolver uma *view* de um `vector` local não é.

O exemplo completo do subprojeto processa um CSV sem construir uma única `std::string`:

```
auto para_sv = [](auto&& r) {
    return std::string_view(std::to_address(r.begin()),
                            static_cast<std::size_t>(rg::distance(r)));
};

auto registros = csv
    | vw:: split ('\n')
    | vw::transform(para_sv)
    | vw:: filter  ([]( std :: string_view l) { return !l.empty() && l.front() != '#'; });
```

Listagem 104: O *pipeline* de `src/ranges/demo_pipelines.hpp`.

Armadilha — `&*r.begin()` sobre um *subrange* vazio é UB (comportamento indefinido)

A última linha de um CSV terminado em `\n` produz um *subrange* vazio, cujo `begin()` é igual ao `end()` da string — e desreferenciar o `end()` é UB (comportamento indefinido). `std::to_address` obtém o ponteiro **sem desreferenciar**, e é por isso que ele aparece no código acima.

III.2.4 Algoritmos com projeção

A assinatura dos algoritmos de `std::ranges` é sempre `algoritmo(range, comparador = {}, projeção = {})`. A projeção diz *o que* comparar; o comparador diz *como*:

```
1  std::sort(v.begin(), v.end(),
2            [](const P& a, const P& b) { return a.idade < b.idade; });
3
4  rg::sort(v, {}, &P::idade);        // a mesma coisa
```

Listagem 105: A mesma ordenação, escrita duas vezes.

Os retornos também melhoraram: os algoritmos clássicos jogam informação fora, e os de *ranges* devolvem *structs* nomeadas — `copy_n` devolve os *dois* iteradores, `for_each` devolve o *functor* com o estado que ele acumulou.

E há uma proteção que transforma um bug de execução em erro de compilação:

```
1  auto it = rg::find(produz(), 3);     // o tipo do retorno é std::ranges::dangling
2  *it;                                 // NÃO COMPILA
```

Listagem 106: O *dangling* detectado pelo sistema de tipos.

O equivalente clássico — `std::find(produz().begin(), produz().end(), 3)` — compila, roda, e é UB (comportamento indefinido) *duplo*, porque as duas chamadas a `produz()` devolvem vetores diferentes.

Armadilha — a ordem no *pipeline* é o sistema de tipos

Cada adaptador preserva ou **degrada** a categoria do *range*: `transform` e `take` preservam; `filter` degrada para bidirecional, porque não há como saltar N elementos sem testar o predicado em cada um. A consequência é que um adaptador que exija `random_access_range` **não compila** depois de um `filter`, e a mensagem do GCC é literalmente `'std::random_access_iterator_tag' is not a base of 'std::bidirectional_iterator_tag'`. Ponha o que exige acesso aleatório *antes* de qualquer `filter`.

III.2.5 Escrevendo os seus

São três níveis de esforço crescente. **Nível 1**: um *range* é `begin()` e `end()` — dois métodos, e você ganha *range-for*, `rg::sort` com projeção e o direito de iniciar um *pipeline*. **Nível 2**: herdar de `std::ranges::view_interface<Derivada>` — CRTP, o mesmo do Seção IV.1 — deriva de graça `empty()`, `front()`, `back()`, `operator[]`, `operator bool` e `size()`. **Nível 3**: um adaptador com `operator|` exige

um objeto que, chamado com `(range, args)`, devolva a *view*, e chamado só com `(args)`, devolva um adaptador parcial. São cerca de quinze linhas em C++20; em C++23, `std::ranges::range_adaptor_closure` dispensa o `operator|` escrito à mão.

Armadilha — não meça *views* em `-O0`

Em `-O2`, um *pipeline* compila para o mesmo código de um laço escrito à mão. Em `-O0` — o padrão do repositório — são várias camadas de chamada que ninguém faz *inline*, e a medição diz o contrário da verdade. Onde as *views* ganham mesmo sem otimização é em eliminar containers temporários e em curto-circuitar com `take`. Onde podem perder: `filter_view::begin()` é $O(n)$, porque precisa achar o primeiro elemento válido.

O que ainda falta no GCC 13 é considerável, e é bom saber antes de planejar código: não há versão em *ranges* dos algoritmos numéricos (`accumulate`, `reduce`, `inner_product`); não há versões paralelas; e `ranges::to<C>()`, `views::zip`, `views::enumerate`, `views::chunk` e `ranges::fold_left` só chegaram no GCC 14. Para acumular sobre uma *view* hoje, materializa-se a parte numérica e usa-se o `accumulate` clássico.

III.3 Entrada, saída e sistema de arquivos

O subprojeto `src/file_io/` cobre a família de *streams*, os *string streams*, o redirecionamento e o `std::filesystem` do C++17. É independente do resto do capítulo, mas usa os idiomas da STL o tempo todo.

III.3.1 A hierarquia, e a diferença que importa

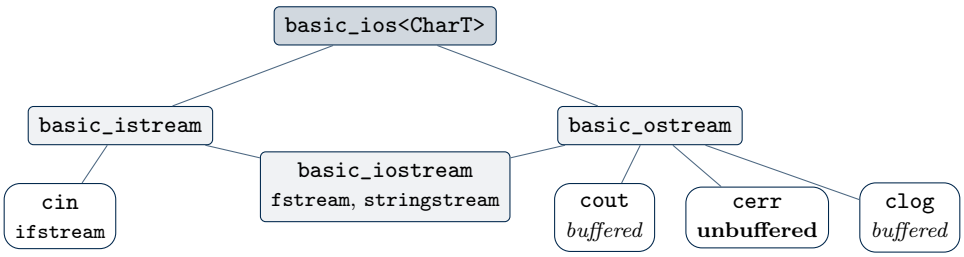


Figura 4: A hierarquia de *streams*. A distinção prática entre `cerr` e `clog` é o *buffer*: os dois escrevem em *stderr*, mas só o primeiro garante que a mensagem saia antes de um *crash*.

Regra — `cerr` para o que precisa sobreviver ao *crash*, `clog` para volume

`std::cerr` não é *buffered*: cada escrita chega ao destino imediatamente, o que custa caro e é exatamente o que se quer para a última mensagem antes de um aborto. `std::clog` escreve no mesmo lugar com *buffer*, e é a escolha correta para *log* volumoso — desde que se aceite perder as últimas linhas num encerramento anormal.

III.3.2 *String streams* e `rdbuf`

Os *string streams* dão um destino em memória com a mesma interface de um arquivo: `ostringstream` para montar texto formatado, `istringstream` para *parsear* `chave=valor` ou CSV, `stringstream` como *buffer*

bidirecional.

O que torna a família toda componível é o `rdbuf()`: o *buffer* de qualquer *stream* pode ser trocado por outro, e é assim que se redireciona `std::cout` para um arquivo sem tocar em nenhuma das chamadas que escrevem nele.

```
std::ofstream arquivo("log.txt");
std::streambuf* original = std::cout.rdbuf(arquivo.rdbuf());   // redireciona
std::cout << "vai para o arquivo";
std::cout.rdbuf(original);                                      // restaura
```

Listagem 107: Redirecionamento por troca de *buffer*.

E é claro que restaurar à mão é exatamente o que RAII existe para evitar: o `RedirectGuard` do subprojeto guarda o *buffer* original no construtor e o repõe no destrutor, de modo que o redirecionamento sobrevive inclusive a uma exceção (Seção II.8). Na mesma linha, o `TeeStream` implementa em C++ o que o `tee` do Unix faz na *shell*: um `streambuf` que sobrescreve `overflow` e `xsputn` para escrever em dois destinos ao mesmo tempo.

III.3.3 `std::filesystem`

O `<filesystem>` do C++17 substitui o par `stat`/`opendir` por uma API portável: `fs::path` compõe caminhos com `operator/` e decompõe com `stem()`, `extension()`, `parent_path()`; `directory_iterator` e `recursive_directory_iterator` percorrem diretórios como *ranges*.

A característica que mais importa para o Seção II.8 é que **cada função tem duas assinaturas** — uma que lança `fs::filesystem_error` e outra que recebe um `std::error_code&` e não lança:

```
auto t = fs::file_size(p, ec);   // não lança; teste 'ec'
auto t = fs::file_size(p);       // lança fs::filesystem_error
```

Listagem 108: A API dupla, e o critério de escolha.

Escolha pela *frequência esperada* do erro: um arquivo que deveria existir e não existe é excepcional; percorrer um diretório em que alguns arquivos podem ter sumido no meio do caminho, não.

III.3.4 Um alvo que não é construído é um alvo que apodrece

Este subprojeto rendeu, sem querer, a lição de engenharia mais barata do repositório. O `demo_sstream.hpp` usa `std::round` para formatar números antes de despejá-los num `ostringstream`, e durante muito tempo **não incluía** o `<cmath>`: compilava assim mesmo, porque algum outro cabeçalho da cadeia arrastava a declaração por acidente. O erro só apareceu no dia em que o alvo passou a ser efetivamente compilado.

Armadilha — o `#include` que "não faz falta" faz, e o compilador não avisa até mudar de versão

Um cabeçalho da biblioteca padrão pode incluir outros, e o conjunto exato é uma decisão da implementação — não do padrão. Um arquivo que usa `std::round` sem incluir `<cmath>` compila enquanto algum vizinho o arrastar, e para de compilar quando o vizinho mudar, quando o GCC for atualizado, ou quando o arquivo for compilado numa ordem diferente. A regra é *include what you use*: cada arquivo declara os cabeçalhos dos nomes que ele mesmo usa, mesmo que "já esteja funcionando".
Este repositório pagou o preço quatro vezes, e o `src/meson.build` registra as quatro no topo do arquivo: `demo_sstream.hpp` sem `<cmath>`, `solid/srp/demo_pipeline.hpp` sem `<algorithm>`,

`concepts/demo_constraints.hpp` sem `<array>` e `aws/s3/main.cpp` sem `<fmt/core.h>`. Os quatro estavam **latentes exatamente nos alvos que não eram construídos**, e os quatro apareceram no minuto seguinte a habilitá-los.

Regra — não esconda um alvo quebrado atrás de um #

Comentar o `subdir()` de um subprojeto que não compila não conserta nada: congela o defeito e some com o sinal. Hoje o `src/meson.build` não tem nenhum `subdir()` comentado — os 57 subprojetos são construídos, produzem 98 binários instalados em `dist/bin` e compilam sem um único aviso sob `-Wall -Wextra -Wpedantic`. A regra que isso encarna vale para qualquer repositório: se um alvo não pode ser construído, o problema é dele — conserte-o ou apague-o.

III.4 Tipos de vocabulário

O subprojeto `src/vocabulary_types/` trata dos tipos que existem para *serem falados entre bibliotecas* — não para resolver um domínio, mas para que duas bibliotecas que não se conhecem possam trocar "pode não haver valor" sem inventar cada uma a sua convenção.

Tabela 38: Cada tipo de vocabulário responde a uma pergunta diferente.

Tipo	Responde	Tipos conhecidos em
`optional<T>`	*pode não haver valor*	compilação, 1 tipo
`variant<Ts...>`	*é um destes*	compilação, N tipos
`any`	*é qualquer coisa*	**execução**
`tuple<Ts...>`	*são todos estes*	compilação, N tipos
`span<T>`	*é uma janela sobre aqueles*	compilação, 1 tipo

O subprojeto compila com C++23 (Seção I.6), porque as operações monádicas de `optional` — `and_then`, `transform`, `or_else` — só chegaram nesse padrão.

O argumento a favor desses tipos é que tudo o que eles substituem são **convenções não verificáveis**: o ponteiro nulo para "sem valor" não obriga ninguém a testar; a `union` com *tag* não impede ler o membro errado; o par (ponteiro, tamanho) são dois parâmetros que precisam concordar e nada garante que concordem.

III.4.1 `std::optional`

As três formas de acessar diferem no que fazem quando não há valor (Tabela 39) — e a primeira é a perigosa, porque não falha: falha em silêncio.

Tabela 39: Acessando um `optional`.

Forma	Se vazio	Quando usar
`*opt`	**UB (comportamento indefinido)**	já testou com `if`
`opt.value()`	lança `bad_optional_access`	quer rede de segurança
`opt.value_or(x)`	devolve `x`	tem um padrão óbvio

Vale insistir num ponto que o nome esconde: `optional` **não é um ponteiro**, e não aloca. É o valor mais um `bool`, e o `T` vive dentro do próprio `optional`:

```
sizeof(int)                = 4
sizeof(optional<int>)      = 8      ← int + flag + padding
&optional  = 0x7ffe9c642e00
&*optional = 0x7ffe9c642e00         ← o T vive DENTRO do optional
```

Listagem 109: Saída de `src/vocabulary_types/`.

Armadilha — `optional<bool>` tem três estados

`if (o)` responde "*tem valor?*", não "*o valor é verdadeiro?*". Um `optional<bool>` contendo `false` responde `true` ao `if` — e `value_or(false)` responde `false` tanto para o vazio quanto para o `false` armazenado, o que apaga a distinção que motivou usar `optional`. Prefira um `enum class Resposta { Sim, Nao, NaoInformado }`.

Por que — por que `optional<T&>` não existe

`opt = outro` seria ambíguo: *religar* a referência para outro objeto, ou *atribuir* ao objeto referido? Como não há resposta óbvia, o padrão proibiu. Na prática, `T*` já é o *optional reference* do C++ — com a desvantagem de não dizer isso no nome.

III.4.2 `std::variant` e o *overload idiom*

O que `variant` acrescenta à `union` é a etiqueta gerenciada e a recusa em mentir: ler o tipo errado dá `bad_variant_access` em vez de UB (comportamento indefinido). Das quatro formas de acessar, uma se destaca:

Tabela 40: Acessando um `variant`. Só `visit` verifica exaustividade.

Forma	Se o tipo estiver errado	Exaustividade
`get_if<T>`	`nullptr`	não verifica
`get<T>`	`bad_variant_access`	não verifica
`visit`	não acontece	**garantida**

`visit` é o único em que **esquecer um caso é erro de compilação** — a mesma propriedade que faz o `switch` sem `default` sobre `enum class` valer a pena (Seção II.1). O que o torna legível são duas linhas de biblioteca:

```
// Herda de todos os lambdas e traz os operator() de cada um para o escopo.
template <typename... Ts>
struct Overload : Ts... { using Ts::operator()...; };

// Deduction guide — em C++20 é opcional, mas explícito não custa.
template <typename... Ts> Overload(Ts...) -> Overload<Ts...>;
```

Listagem 110: O *overload idiom*, em `src/vocabulary_types/demo_variant.hpp`.

Herdar de cada *lambda* e expor o `operator()` de todos deixa a resolução de sobrecarga normal fazer o despacho. E como `visit` aceita *vários variants* de uma vez, o compilador gera a matriz de combinações: é o padrão *Visitor* do GoF sem hierarquia, sem `accept()` e sem vtable / vptr (Seção V.2).

Armadilha — `valueless_by_exception`

Se a construção do novo valor lançar *durante* uma atribuição, o antigo já foi destruído e o novo não existe. O `variant` entra num estado sem valor, com `index()` igual a -1:

```
antes:  index=0 valueless=false
exceção durante a atribuição: falhei na cópia
depois: index=-1 valueless=true
```

Evita-se pondo um tipo trivial (ou `std::monostate`) como primeira alternativa, preferindo `emplace` à atribuição, e marcando o *move* como `noexcept` — a Rule of 5 do Seção II.3 outra vez.

O *expression problem*. A comparação entre `variant` e hierarquia virtual não tem vencedor absoluto: cada um facilita um eixo de extensão e dificulta o outro (Tabela 41). A escolha se faz perguntando qual eixo vai variar mais.

Tabela 41: `variant` favorece operações novas; herança favorece tipos novos.

Critério	`variant`	herança virtual
tipos novos *externos*	não	**sim**
operações novas	**sim**	recompila tudo
exaustividade verificada	**sim**	não
alocação por nó	opcional	sempre

III.4.3 `std::any` e a pergunta que quase sempre o elimina

O subprojeto substitui o `operator new` global por uma versão que conta, e o resultado dispensa argumentação:

```
any<Pequeno> ( 4B) → 0 alocação(ões)
any<Medio>   (16B) → 1 alocação(ões), 16 bytes
any<Grande>  (64B) → 1 alocação(ões), 64 bytes
variant<...Grande>  → 0 alocação(ões)      ← nunca aloca
```

Listagem 111: Saída de `src/vocabulary_types/`: alocações contadas.

A otimização de objeto pequeno da libstdc++ acaba em `sizeof(void*)`; acima disso, *cada* `any` é um `malloc` mais um `free`.

Regra — "eu consigo listar os tipos possíveis?"

Se **sim**, use `variant` — ganha exaustividade e zero alocação. Se **não, mas todos fazem a mesma coisa**, use uma interface virtual. Só se **não, e cada um faz coisa diferente** é que `any` é a resposta: quando o conjunto de tipos é aberto *por projeto*, definido por quem *usa* a biblioteca. O sintoma de `any` mal usado é uma cascata de `any_cast` — que *é* um `variant`, escrito à mão e sem exaustividade.

III.4.4 `tuple`, `pair` e os *structured bindings*

Os *structured bindings* do C++17 salvaram a `tuple` de ser ilegível: `auto [nome, idade, salario] = obter_registro();` dá nomes no ponto de uso, coisa que `std::get<0>(r)` nunca deu.

Armadilha — os nomes do *binding* são novos, não *aliases*

`auto [a,b]` **copia**; `auto& [a,b]` referencia; `const auto& [a,b]` é o padrão seguro. Escrever `auto` por reflexo num *binding* sobre um objeto grande copia o objeto inteiro sem avisar. E `std::tie` guarda *referências*: nunca devolva o resultado de um `tie` — para devolver, `make_tuple`.

Percorrer uma `tuple` não é possível com *range-for*, porque cada elemento tem um **tipo** diferente. Exige desdobrar em compilação, o que uma *fold expression* resolve em uma linha: `std::apply([&f](const auto&... e) { (f(e), ...); }, t);`. O mesmo `std::apply` — e o `std::make_from_tuple` para construtores — é a base de `std::bind`, de *thread pools* e de sistemas de tarefas (Seção VI.1).

Quanto a escolher entre `tuple` e `struct`: o `sizeof` é **idêntico**, então a decisão é inteiramente de legibilidade. Use `tuple` quando os campos não têm nome natural, quando o agregado é interno e efêmero, ou quando o código é genérico sobre N tipos; use `struct` quando o agregado é do domínio, atravessa uma API pública, tem invariantes ou tem três ou mais campos. Desde o C++20, a inicialização designada (Seção II.1) tirou o último argumento a favor da `tuple` em código de aplicação.

III.4.5 `std::span`

`span` substitui o par (ponteiro, tamanho) por um objeto em que os dois andam juntos — e, sendo um *range*, traz *range-for* e `<ranges>` junto:

```
1 void soma_antiga(const int* dados, size_t n);   // soma_antiga(bruto, 500) compila
2 int  soma_span(std::span<const int> dados);     // o tamanho vem do objeto
```

Listagem 112: O parâmetro que não pode discordar de si mesmo.

O *extent* pode ser dinâmico ou estático, e a diferença aparece no `sizeof`: `span<int>` ocupa 16 bytes (ponteiro e tamanho); `span<int,3>` ocupa 8, porque o 3 está no *tipo*. A conversão de estático para dinâmico é implícita; a inversa exige `.first<3>()`, e quem escreve assume o risco.

Tabela 42: `span<const T>` contra `const span<T>` — o que cada um congela.

Quer proteger	Escreva
os **elementos**	`span<const T>` — 95% dos casos
a **janela**	`const span<T>`
ambos	`const span<const T>`

Armadilha — um `span` é tão frágil quanto um par de iteradores

```
1 std::span<int> s{v};
2 v.push_back(4);      // realocou → s está pendurado
3 s[0] = 1;            // heap-use-after-free
```

Porque *é* isso: um ponteiro e um tamanho. As três regras de uso são (1) `span` é **parâmetro** — o uso dominante e sempre seguro; (2) como retorno, só se o *buffer* sobreviver à chamada; (3) como membro, exige documentar quem é o dono. E `subspan` fora dos limites é UB (comportamento indefinido), não exceção — as versões com *template* (`.first<4>()`) verificam em compilação quando o *extent* é estático.

III.5 De onde vem a memória

O subprojeto `src/memory/` leva o capítulo ao andar de baixo: o que `new`/`delete` realmente fazem, e o que se ganha ao substituí-los quando o padrão de uso é conhecido. Só rende depois de conhecer o que os containers alocam — daí a posição na trilha.

III.5.1 `new` e `delete`, por dentro

`new T(args)` são duas operações: `operator new(sizeof(T))`, que consegue os bytes e pode lançar `std::bad_alloc`, e o *placement new*, que constrói o objeto neles. `delete p` é o inverso, na ordem inversa: destrutor, depois liberação. Separar as duas metades é o que permite o *placement new* puro — construir num *buffer* já existente, sem alocar:

```
alignas(MyObject) std::byte buf[sizeof(MyObject)];

MyObject* obj = new(buf) MyObject(42);   // constrói, sem alocar
obj->~MyObject();                        // destrói, sem liberar
// buffer ainda válido — pode ser reusado
```

Listagem 113: Construção e destruição desacopladas da alocação.

Sobrescrever o `operator new` global é a técnica de diagnóstico usada em todo o repositório para *contar* alocações — é o que produziu a tabela de `std::any` da Seção III.4.

O custo que se quer evitar tem quatro componentes: a eventual chamada ao sistema operacional, a busca por um bloco livre, a fragmentação acumulada e — a mais incômoda em tempo real — a **falta de determinismo**: o custo varia com o estado da *heap*.

III.5.2 *Pool*: blocos de tamanho fixo

Quando muitos objetos do *mesmo tamanho* são criados e destruídos com alta frequência, a fragmentação chega ao ponto em que existe memória livre suficiente e nenhum bloco contíguo grande o bastante. O *pool* elimina o problema pela raiz: todos os blocos têm o mesmo tamanho, então qualquer bloco livre serve para qualquer pedido.

A estrutura é uma *free list* embutida na própria memória livre — os primeiros `sizeof(void*)` bytes de um bloco livre guardam o ponteiro para o próximo:

```
    // — alloc(): retira o primeiro bloco da free list — O(1) ————————
    T* alloc() {
        if (!livre_) return nullptr; // pool esgotado

        void* bloco = livre_;
        livre_ = *reinterpret_cast<void**>(livre_); // avança free list
        ++em_uso_;
        return static_cast<T*>(bloco); // memória crua — construtor não chamado ainda
    }

    // — free(): devolve bloco à free list — O(1) ——————————————————
    void free(T* ptr) {
        // Devolve à frente da free list
        *reinterpret_cast<void**>(ptr) = livre_;
        livre_ = ptr;
        --em_uso_;
    }
```

Listagem 114: As duas operações centrais, em `src/memory/demo_pool_allocator.hpp`.

Tabela 43: O que o *pool* troca pelo que perde em generalidade.

Característica	`new`/`delete`	Pool
tempo de alocação	amortizado, variável	$O(1)$ constante
fragmentação	pode ocorrer	zero (blocos fixos)
overhead por objeto	8–16 bytes de metadados	zero
localidade de cache	ruim (disperso)	excelente (contíguo)

III.5.3 *Arena*: aloca tudo, libera tudo

A *arena* vai além: alocar é incrementar um deslocamento, e liberar individualmente **não existe**. O que existe é `reset()`, que zera o contador — $O(1)$, independentemente de quantos objetos havia.

```
ArenaAllocator frame_arena(1024 * 1024);   // 1 MB por frame

while (true) {
    frame_arena.reset();    // "libera" tudo — O(1)
    // ... alocações do frame ...
}
```

Listagem 115: O uso clássico da arena: memória com escopo de quadro.

Armadilha — a arena não chama destrutores

`reset()` devolve a memória sem destruir o que estava nela. Para tipos triviais isso é exatamente o que se quer; para um tipo que possui recurso — um `std::string`, um descritor de arquivo — é vazamento garantido. A arena é a estrutura certa para dados de quadro, e a errada para qualquer coisa que precise de RAII.

III.5.4 `std::pmr`: trocar o alocador sem trocar o tipo

O parâmetro `Allocator` clássico dos containers tem um defeito estrutural: ele entra no *tipo*. `vector<int,A>` e `vector<int,B>` são tipos distintos, não intercambiáveis, não passáveis para a mesma função. O `std::pmr` do C++17 resolve movendo o polimorfismo para tempo de execução: o container guarda um ponteiro para um `memory_resource` e chama através dele.

```
std::byte buf[4096];
std::pmr::monotonic_buffer_resource local_arena(buf, sizeof(buf));

std::pmr::vector<int> v(&local_arena);
v.push_back(1);
v.push_back(2);
// nenhuma alocação na heap
```

Listagem 116: Um `vector` que aloca na pilha.

Os recursos padrão cobrem os três casos deste capítulo: `monotonic_buffer_resource` é a arena; `unsynchronized_pool_resource` é o *pool*; `synchronized_pool_resource` é o mesmo com sincronização. E eles se encadeiam: um recurso pode receber outro como *upstream*, que atende quando o primeiro esgota.

Tabela 44: Escolhendo a estratégia de alocação.

Cenário	Estratégia
uso geral	`new`/`delete`
muitos objetos iguais, alta frequência	*pool*
objetos temporários de quadro (jogos)	arena / `monotonic_buffer`
protótipo rápido de alocador	`std::pmr`
vários containers com o mesmo alocador	`std::pmr`

Regra — meça antes de trocar o alocador

Um alocador customizado é uma aposta sobre o padrão de uso, e a aposta errada piora tudo: a arena vaza se os objetos tiverem destrutor; o *pool* desperdiça memória se os tamanhos variarem; ambos abrem mão da simplicidade. O ganho prometido — entre uma e duas ordens de grandeza nas alocações — só se realiza quando o programa *de fato* aloca em massa, e a única forma de saber é medir.

III.6 Co-rotinas: funções que sabem parar no meio

O último cabeçalho deste capítulo é também o menor da biblioteca padrão inteira: o `<coroutine>` do C++20 tem cinco tipos e uma função livre, e não oferece nenhum container, nenhum algoritmo e nenhuma classe pronta para usar. O que ele oferece é o *vocabulário* com que o compilador conversa com o código do usuário — e é por isso que o assunto fecha um capítulo sobre a biblioteca padrão em vez de abrir um sobre a linguagem.

O subprojeto `src/coroutines/` ensina o mecanismo em quatro degraus: `demo_generator.hpp` (produzir uma sequência), `demo_task.hpp` (esperar um resultado), `demo_internals.hpp` (o que o compilador gera) e `demo_asio.hpp` (I/O de verdade). A Seção VI.3 volta ao mesmo subprojeto pelo ângulo da concorrência — concorrência sem *thread*; aqui o assunto é o outro: quais tipos a biblioteca fornece, o que exatamente o compilador chama neles e quanto isso custa em memória.

Por que — o problema: um estado que precisa sobreviver ao `return`

Uma função comum tem um contrato rígido: ela é chamada, roda até o fim e devolve o controle uma única vez. As variáveis locais dela vivem num *frame* na pilha, e esse *frame* deixa de existir no instante do `return`. Isso resolve quase tudo — e falha em dois problemas muito comuns:

(1) produzir uma sequência longa ou infinita. Para devolver os números de Fibonacci até um limite, uma função comum precisa montar o `vector` inteiro antes de devolver qualquer coisa: paga memória proporcional ao tamanho da sequência e não consegue representar sequências sem fim.

(2) esperar por algo lento sem parar a *thread*. Ao ler de um socket, uma função comum bloqueia: a *thread* fica parada, ocupando pilha (tipicamente 8 MB de espaço de endereçamento) e uma vaga no escalonador, sem fazer trabalho nenhum.

As saídas anteriores ao C++20 eram todas ruins de maneiras diferentes: escrever a máquina de estados à mão (correto e ilegível); usar uma *thread* por espera (simples e caro); inverter o fluxo de controle em *callbacks* (barato e impossível de ler, o *callback hell*). A co-rotina resolve o problema na raiz: ela é uma função cujo *frame* **não morre** no `return`.

III.6.1 A definição, e os três operadores

Uma co-rotina (C++20) é uma função que pode **suspender** — devolver o controle a quem a chamou sem terminar — e depois ser **retomada**, continuando exatamente da linha seguinte, com todas as variáveis locais intactas. Para que isso seja possível, o *frame* dela não fica na pilha: fica num bloco separado (normalmente na *heap*, Seção III.5) que sobrevive à suspensão.

Uma função é uma co-rotina se o corpo dela usar qualquer um dos três operadores da Tabela 45.

Tabela 45: Os três operadores que tornam uma função uma co-rotina.

Operador	O que faz	Quem o consome
`co_yield v`	suspende e entrega `v` ao chamador	`promise.yield_value(v)`
`co_await e`	suspende até `e` completar	o *awaitable* de `e`
`co_return v`	termina a co-rotina com valor	`promise.return_value(v)`
`co_return;`	termina a co-rotina sem valor	`promise.return_void()`

Armadilha — a assinatura não diz que a função é uma co-rotina

Não existe palavra-chave no cabeçalho da função. `Generator<int> contar(int)` e `Generator<int> vazio(int)` têm assinaturas idênticas; se o corpo da primeira contiver `co_yield` e o da segunda não, apenas a primeira é co-rotina — e a segunda nem compila, porque uma função que devolve um tipo com `promise_type` mas nunca suspende continua sendo uma função comum, obrigada a fazer `return` de um `Generator` que ela não tem como construir. A consequência prática é que **trocar `co_return x` por `return x` muda a natureza da função**, e o erro de compilação aponta para o tipo de retorno, não para a linha trocada.

III.6.2 O que o `<coroutine>` contém — e o que não contém

A biblioteca padrão participa disso com muito pouco. A Tabela 46 é o conteúdo inteiro do cabeçalho.

Tabela 46: Tudo o que o `<coroutine>` do C++20 fornece.

Nome	Para que serve
`coroutine_handle<P>`	ponteiro para o *frame*; retoma, destrói, consulta
`coroutine_handle<>`	a mesma coisa, sem o tipo da *promise*
`coroutine_traits<...>`	como o compilador acha o `promise_type`
`suspend_always`	*awaitable* trivial que sempre suspende
`suspend_never`	*awaitable* trivial que nunca suspende
`noop_coroutine()`	*handle* sentinela cuja retomada não faz nada

Não há `generator`, não há `task`, não há *event loop*. Quem quiser qualquer um dos três escreve — ou usa uma biblioteca que escreveu. Isso é medível com as macros de teste de recurso, que é como o subprojeto `src/cpp23/` inventaria o padrão em vez de acreditar na documentação:

```
__cpp_impl_coroutine = 201902    <- a linguagem tem co-rotinas
__cpp_lib_coroutine  = 201902    <- o cabecalho <coroutine> existe
__cpp_lib_generator  = ausente   <- std::generator (C++23) NAO existe aqui
```

Listagem 117: Medição no GCC 13.3 (o compilador deste livro), com `-std=c++23`: o mecanismo está lá, o tipo pronto não.

Regra — o C++20 entregou o mecanismo, não os tipos

Essa é a diferença mais importante entre as co-rotinas do C++ e as de Python, JavaScript ou C#. Nessas linguagens, `yield` e `await` vêm com uma semântica fixa embutida. No C++, os operadores apenas *chamam métodos de um tipo que você fornece* — e é isso que permite que a mesma palavra `co_await` funcione tanto num `Task<T>` de trinta linhas quanto no Asio, sem que o padrão soubesse que Asio existe. O preço é que a primeira co-rotina de qualquer projeto custa escrever um `promise_type`.

III.6.3 O `promise_type`: o contrato que você escreve

Quando o compilador vê uma co-rotina cujo tipo de retorno é `G`, ele procura `G::promise_type` e passa a chamar métodos dele em pontos definidos do ciclo de vida (Tabela 47). Nenhum desses métodos é virtual e nenhum é herdado de nada: o acoplamento é por *nome*, verificado em compilação.

Tabela 47: Os métodos do `promise_type` e quando o compilador chama cada um.

Método	Quando é chamado
`get_return_object()`	na criação; o valor devolvido é o que o *chamador* recebe
`initial_suspend()`	logo depois, antes da primeira linha do corpo
`yield_value(v)`	em cada `co_yield v`
`return_value(v)`	em `co_return v`
`return_void()`	em `co_return;` ou ao cair no fim do corpo
`unhandled_exception()`	se uma exceção escapar do corpo
`final_suspend()`	depois de `return_*`, sempre

`return_value` e `return_void` são mutuamente exclusivos: declarar os dois é erro de compilação.

Ou seja, a criação de uma co-rotina é sempre esta sequência fixa:

```
// 1. aloca o frame (operator new, salvo elisao)
// 2. copia os parametros (1000) para dentro do frame
// 3. constroi a promise_type dentro do frame
// 4. auto ret = promise.get_return_object();   // o Generator do chamador
// 5. co_await promise.initial_suspend();       // suspend_always -> para aqui
// 6. return ret;                               // o corpo AINDA nao rodou
```

Listagem 118: O que `auto g = fibonacci(1000);` realmente executa.

O passo 5 é a decisão de projeto mais consequente do tipo, e o subprojeto a documenta no próprio código:

```
// initial_suspend -> suspend_always: a coroutine nao executa nada
// ao ser criada. So comeca quando chamamos .next() pela primeira vez.
// (Alternativa: suspend_never = executa ate o primeiro co_yield logo)
std::suspend_always initial_suspend() noexcept { return {}; }

// final_suspend -> suspend_always: ao terminar, a coroutine suspende
// em vez de se destruir automaticamente. Isso e necessario para que
// o Generator (que guarda o handle) possa destrui-la de forma segura.
std::suspend_always final_suspend() noexcept { return {}; }
```

Listagem 119: As duas suspensões, em `src/coroutines/demo_generator.hpp`.

Regra — `initial_suspend` escolhe entre preguiçoso e ansioso; `final_suspend` escolhe quem destrói

`suspend_always` em `initial_suspend` produz uma co-rotina *lazy*: nada roda até alguém retomar. `suspend_never` produz uma co-rotina *eager*: o corpo roda até a primeira suspensão já na chamada. Em `final_suspend`, `suspend_always` mantém o *frame* vivo depois do fim — e obriga alguém a destruí-lo; `suspend_never` faz o *frame* se autodestruir — e invalida todo *handle* que aponte para ele. Um

gerador precisa de `suspend_always` nos dois lugares, porque precisa ler o último valor *depois* que a co-rotina parou.

III.6.4 Um gerador completo

Com o `promise_type` pronto, a função geradora é indistinguível de um laço comum — e é esse o ponto:

```
1  Generator<long long> fibonacci(long long limite) {
2      long long a = 0, b = 1;
3      while (a <= limite) {
4          co_yield a;                 // suspende aqui, entrega 'a'
5          auto prox = a + b;          // ao retomar, continua desta linha
6          a = b;
7          b = prox;
8      }
9      // ao sair do while, a coroutine termina -> return_void() chamado
10 }
```

Listagem 120: `src/coroutines/demo_generator.hpp`: sequência finita, sem container nenhum.

O `Generator<T>` do subprojeto implementa `begin()`/`end()` com `std::default_sentinel_t` — exatamente a técnica de sentinela da Seção III.2 — e por isso funciona direto no `for` de intervalo:

```
1  -- 1.2 Fibonacci ate 1000 -- range-for sobre Generator --
2    Fibonacci <= 1000: 0 1 1 2 3 5 8 13 21 34 55 89 144 233 377 610 987
3
4  -- 1.3 Pipeline: contar -> filtrar pares -> dobrar --
5    Resultados: 0 4 8 12 16 20
6    (pipeline: contar(0) | filtrar (par) | mapear(x2))
```

Listagem 121: Saída de `dist/bin/coroutines`, demos 1.2 e 1.3.

O terceiro demo merece atenção: `contar(0)` é uma sequência **infinita** (`while (true) co_yield n++;`), e mesmo assim o programa termina. Ele termina porque cada valor só é calculado quando pedido, e o consumidor pede seis. É a mesma preguiça das *views* da Seção III.2, com uma diferença decisiva: aqui o estado do produtor vive no *frame* da co-rotina, e não num *lambda* `mutable` que pode ser percorrido duas vezes e não reinicia.

Exceções atravessam a fronteira sem tratamento especial, porque `unhandled_exception()` guarda o `std::exception_ptr` e o invólucro o relança no chamador:

```
1    valor: 0
2    valor: 1
3    valor: 2
4    Excecao capturada no chamador: valor proibido: 3
```

Listagem 122: Saída de `dist/bin/coroutines`, demo 1.4: a exceção sai do *frame* e chega ao `catch` do chamador.

III.6.5 Quanto custa o *frame*, medido

Como o *frame* não cabe na pilha, ele é alocado — e este capítulo já mostrou (Seção III.5) que alocação é onde o desempenho se perde. Vale então medir, substituindo o `operator new` global por uma versão que conta, como faz o subprojeto `src/vocabulary_types/`:

```
frame de contar(int)             :   64 bytes
frame de fibonacci(long long)    :   96 bytes
frame de nomes(std::string)      :  160 bytes

os 17 valores de Fibonacci ate 1000:
  co—rotina : 1 alocacao,   96 bytes no total
  vector    : 6 alocacoes,  504 bytes somados (crescimento 1,2,4,8,16,32)
```

Listagem 123: Saída de `src/coroutines/`: medição com `operator new` instrumentado, GCC 13.3 em `-O2`, sobre o `Generator<T>` daquele subprojeto.

Três leituras deste quadro. Primeira: o *frame* é alocado **uma vez**, na criação — as dezessete retomadas não alocam nada. Segunda: o tamanho não depende do comprimento da sequência, e sim de quantas variáveis locais precisam **sobreviver a uma suspensão**; por isso `contar` (um `int`) custa menos que `fibonacci` (dois `long long`), e a versão que mantém uma `std::string` viva através do `co_yield` custa mais que as duas. Terceira: a comparação com o `vector` inverte com o tamanho — para dezessete elementos os 504 bytes do `vector` perdem, mas para uma sequência de dezessete milhões a co-rotina continua em 96 bytes e o `vector` não.

Regra — a co-rotina troca memória proporcional ao dado por memória constante

O gerador é a estrutura certa quando a sequência é grande, infinita, ou quando o consumidor pode parar no meio — os três casos em que materializar é desperdício. Quando a sequência é curta e vai ser percorrida várias vezes, o `vector` ganha: uma alocação também, mas sem indireção, sem máquina de estados e com acesso aleatório. Como sempre neste capítulo, a decisão é sobre o padrão de uso, não sobre elegância.

Aviso

A alocação do *frame* é a única do mecanismo, mas ela é obrigatória apenas *no papel*: o padrão autoriza o compilador a eliminá-la (*HALO*, *heap allocation elision*) quando consegue provar que o *frame* não sobrevive ao chamador — tipicamente quando a co-rotina é criada, consumida e destruída na mesma função, e tudo é *inline*. É uma otimização real, mas não é garantida por nada: não conte com ela num orçamento de latência sem medir com `operator new` instrumentado, exatamente como acima.

III.6.6 O `coroutine_handle`: um ponteiro cru com dono

O *handle* é o único acesso ao *frame*. Ele é do tamanho de um ponteiro, é copiável trivialmente e **não tem semântica de dono nenhuma** — é um `void*` com métodos (Tabela 48).

Tabela 48: A interface do `std::coroutine_handle`.

Operação	Efeito
`resume()` / `operator()`	retoma a co-rotina no ponto de suspensão
`done()`	terminou? (só válido se estiver *suspensa*)
`destroy()`	destrói o *frame*; ninguém faz isso por você
`promise()`	referência à *promise* dentro do *frame*
`from_promise(p)`	o caminho inverso: da *promise* ao *handle*
`address()` / `from_address(p)`	converte para `void*` e de volta

É por isso que todo tipo de co-rotina útil é um invólucro RAII em volta do *handle*: o `Generator<T>` do subprojeto proíbe cópia, permite movimento e chama `destroy()` no destrutor — exatamente o desenho de um `unique_ptr` (Seção II.5).

Armadilha — `done()` sobre um *frame* já destruído é *use-after-free*, e ele responde

O demo 3.2 do subprojeto contrasta `suspend_never` com `suspend_always` criando uma `EagerTask` que usa `suspend_never` nas *duas* suspensões. A expectativa natural é que, como a co-rotina roda inteira na criação, `handle.done()` devolva `true`. A medição diz outra coisa:

```
  Apos a criacao -- a coroutine ja terminou
  handle.done() = false            <- nao era o esperado

ASAN: heap-use-after-free on address 0x504000000010
  READ of size 8 in coroutine_handle<promise_type>::done() const
  0x504000000010 is located 0 bytes inside of 40-byte region
  freed by thread T0 here:  <dentro de coroutine_eager()>
```

Listagem 124: Saída de `dist/bin/coroutines`, demo 3.2 — e o motivo, sob `-fsanitize=address`.

`final_suspend()` devolvendo `suspend_never` significa "não suspenda no fim", e uma co-rotina que não suspende no fim **destrói o próprio *frame*** e devolve os 40 bytes. O *handle* guardado na `EagerTask` ficou pendurado no instante seguinte, e `done()` leu memória liberada. O `false` impresso não é uma resposta: é lixo. A regra que sai daí é dura e simples — **depois de `final_suspend()` não suspender, o *handle* é inválido**, e a única forma de descobrir isso sem sanitizer é um *crash* intermitente em produção.

III.6.7 O protocolo *Awaitable*

`co_await` é o operador aberto do mecanismo: ele funciona sobre *qualquer* tipo que implemente três métodos (Tabela 49). Nenhuma herança, nenhum *concept* obrigatório — o compilador simplesmente chama os nomes.

Tabela 49: Os três métodos do protocolo *Awaitable*.

Método	Papel
`await_ready()`	devolve `true` se o resultado já existe; nesse caso não há suspensão nenhuma
`await_suspend(h)`	chamado *já suspenso*: registra `h` em algum lugar que vá retomá-lo depois
`await_resume()`	chamado na retomada; o que ele devolver é o valor da expressão `co_await`

A expressão `auto x = co_await a;` se expande, em essência, em:

```
if (!a.await_ready()) {
    // o frame ja esta suspenso quando await_suspend roda
    a.await_suspend(handle_desta_corotina);
    return_ao_chamador();          // <- a thread vai fazer outra coisa
}
retomada:
auto x = a.await_resume();
```

Listagem 125: O que o compilador gera para um `co_await`.

O detalhe que decide o desenho está no tipo de retorno de `await_suspend`, que pode ser três coisas diferentes (Tabela 50).

O terceiro caso é o *symmetric transfer*, e existe por um motivo concreto:

Armadilha — cadeias de co-rotinas estouram a pilha se `await_suspend` chamar `resume()`

`handle.resume()` é uma chamada de função comum: empilha um *frame* de chamada. Numa cadeia em que A retoma B, que retoma C, que retoma D, a pilha cresce com o comprimento da cadeia — e

Tabela 50: Os três retornos possíveis de `await_suspend`.

Retorno	Significado
`void`	suspende e devolve o controle ao chamador
`bool`	`false` cancela a suspensão e retoma já
`coroutine_handle<>`	suspende **e** retoma o *handle* devolvido

um *ping-pong* entre duas co-rotinas que se retomam mutuamente estoura a pilha em poucos milhares de iterações. Devolver o *handle* em vez de retomá-lo converte a retomada numa *tail call* garantida pelo padrão: o compilador emite um salto, e a pilha fica constante.

```
1 void await_suspend(handle h) { outro.resume(); }   // pilha cresce
2 auto await_suspend(handle h) { return outro;   }   // pilha constante
```

Listagem 126: A diferença inteira.

Quando não há ninguém a retomar, devolva `std::noop_coroutine()` — o *handle* sentinela cuja retomada não faz nada e apenas devolve o controle. Devolver um *handle* nulo é UB (comportamento indefinido).

III.6.8 `Task<T>`: esperar sem bloquear

Se `co_yield` produz vários valores, `co_await` espera por um. A diferença em relação ao `std::future` é exatamente o que justifica todo o mecanismo:

```
1 int r = future.get();      // BLOQUEIA: a thread para aqui, sem fazer nada
2 int r = co_await task;     // SUSPENDE: a thread sai e faz outro trabalho
```

Listagem 127: Bloquear a *thread* contra suspender a co-rotina.

O `Task<T>` do subprojeto encaixa as duas metades do protocolo ao mesmo tempo: ele *tem* um `promise_type` (é retorno de co-rotina) e *é* um *awaitable* (pode aparecer depois de `co_await`). A peça que costura as duas é a **continuação**: ao ser aguardada, a `Task` guarda o *handle* de quem a aguarda, e no `final_suspend` devolve esse *handle* — *symmetric transfer* em uso real.

```
1  struct FinalAwaiter {
2      bool await_ready() noexcept { return false; }
3
4      std::coroutine_handle<> await_suspend(
5          std::coroutine_handle<promise_type> h) noexcept {
6          auto cont = h.promise().continuacao_;
7          if (cont) return cont;                  // retoma o chamador
8          return std::noop_coroutine();           // ninguem esperando, para
9      }
10
11     void await_resume() noexcept {}
12 };
```

Listagem 128: O `FinalAwaiter` de `src/coroutines/demo_task.hpp`: "ao terminar, retome quem me esperava".

O resultado é código assíncrono que se lê de cima para baixo:

```
1 Task<int> buscar_dado(int id) {
2     co_await SuspendFor{50ms};
3     co_return id * 10;
4 }
5
```

```
6  Task<int> processar() {
7      int a = co_await buscar_dado(1);
8      int b = co_await buscar_dado(2);
9      co_return a + b;
10 }
```

Listagem 129: `src/coroutines/demo_task.hpp`: duas esperas encadeadas, sem *callback* e sem *thread* parada.

```
1  [processar] iniciando
2  [Task] buscando dado 1... (aguardando 50ms)
3  [Task] dado 1 pronto!
4  [Task] buscando dado 2... (aguardando 50ms)
5  [Task] dado 2 pronto!
6  [processar] a=10, b=20, soma=30
7  Soma final recebida no main: 30
```

Listagem 130: Saída de `dist/bin/coroutines`, demo 2.2.

Armadilha — o `SuspendFor` do demo é uma muleta didática, e o código diz isso

O `SuspendFor{50ms}` implementa o protocolo *awaitable* da forma mais simples possível: em `await_suspend` ele cria uma `std::thread` destacada que dorme e depois chama `handle.resume()`. Isso ensina o protocolo, mas é o *oposto* do ganho prometido — uma *thread* por espera é exatamente o custo que as co-rotinas existem para eliminar. O próprio `demo_asio.hpp` abre com a ressalva: *"até aqui usamos `SuspendFor{}` que dorme em thread separada — isso não é verdadeiro async"*. Pelo mesmo motivo, o `get_sync()` que permite chamar uma `Task` do `main()` (que não é co-rotina) faz espera ocupada com `sleep_for(1ms)`. Em código de produção, o lugar do `await_suspend` é registrar o *handle* num *event loop*, e é o que a próxima seção faz.

III.6.9 Duas armadilhas de tempo de vida que ninguém avisa

Armadilha — parâmetros por referência morrem no primeiro `co_await`

Os parâmetros de uma co-rotina são **copiados para dentro do *frame*** — mas uma referência é copiada *como referência*, e o objeto ao qual ela se refere continua na pilha do chamador. Se a co-rotina for lazy, o chamador já pode ter devolvido antes do corpo rodar:

```
1  Task<void> log(const std::string& msg);   // referencia vive no chamador
2  co_spawn(io, log(std::string("ola")), detached);   // temporario morre aqui
3  // ... o corpo de log() so roda depois -> le uma string destruida
```

Listagem 131: O erro clássico.

A regra é receber **por valor** tudo o que a co-rotina precisar usar depois de uma suspensão. O mesmo vale para `std::string_view` e `std::span` (Seção III.4): eles são referências disfarçadas, e o *frame* não estende a vida de nada. E o *lambda* genérico é o pior caso, porque parece cópia e não é: capturar por `[&]` numa co-rotina é quase sempre bug.

Armadilha — o *frame* vira um tipo sem *linkage*, e o GCC avisa

O subprojeto compila limpo sob `-Wall -Wextra -Wpedantic`, com uma única exceção declarada no `meson.build`: `-Wno-subobject-linkage`. O motivo é que `filtrar()` e `mapear()` são *templates* que recebem *lambdas*; o *lambda* é um tipo local, sem *linkage*, e o compilador o guarda como campo do *frame* — que é uma `struct` gerada. Removendo a supressão, o GCC 13.3 emite exatamente dois avisos:

```
1  demo_generator.hpp:189: warning: ... filtrar <...>::Frame has a field 'pred'
2          whose type has no linkage [-Wsubobject-linkage]
```

```
3 | demo_generator.hpp:197: warning: ...mapear<...>::Frame has a field 'func'
4 |         whose type has no linkage [-Wsubobject-linkage]
```

Listagem 132: Saída de `src/coroutines/` ao recompilar sem `-Wno-subobject-linkage`.

Não há problema real de *linkage*: o *frame* nunca cruza fronteira de unidade de tradução. O que o caso ensina é o critério — **suprimir um aviso é aceitável quando o motivo está escrito ao lado da supressão**, e o `meson.build` do subprojeto o escreve.

III.6.10 Asio: onde a co-rotina paga o próprio custo

O ganho só aparece com um *event loop* de verdade por baixo, e é isso que o Asio fornece: `asio::awaitable<T>` é o `Task<T>` pronto, `asio::use_awaitable` é o *token* que converte qualquer operação assíncrona de *callback* em algo aguardável, e `asio::co_spawn` lança a co-rotina dentro do `io_context`.

```
1  asio::awaitable<void> esperar_e_imprimir(std::string nome, ms delay) {
2      auto executor = co_await asio::this_coro::executor;
3      asio::steady_timer timer(executor, delay);
4      co_await timer.async_wait(asio::use_awaitable);   // suspende, nao bloqueia
5      fmt::print("   [{}] timer disparado apos {}ms!\n", nome, delay.count());
6  }
7
8  asio::co_spawn(io, esperar_e_imprimir("A", 300ms), asio::detached);
9  asio::co_spawn(io, esperar_e_imprimir("B", 100ms), asio::detached);
10 asio::co_spawn(io, esperar_e_imprimir("C", 200ms), asio::detached);
11 io.run();
```

Listagem 133: `src/coroutines/demo_asio.hpp`: três co-rotinas, uma *thread*.

```
1  [A] iniciando, esperara 300ms
2  [B] iniciando, esperara 100ms
3  [C] iniciando, esperara 200ms
4  [B] timer disparado apos 100ms!
5  [C] timer disparado apos 200ms!
6  [A] timer disparado apos 300ms!
7  1 thread, 0 bloqueios -- cada suspensao libera a thread
```

Listagem 134: Saída de `dist/bin/coroutines`, demo 4.1: a ordem de término é a das esperas, não a da criação.

Note o que a saída prova: as três iniciaram na ordem A, B, C e terminaram na ordem B, C, A, **numa única *thread***. Com *threads* isso exigiria três pilhas; aqui exigiu três *frames* de algumas dezenas de bytes.

O demo 4.2 leva a ideia ao caso que a justifica: um servidor de eco em que cada conexão é uma co-rotina.

```
1  asio::awaitable<void> echo_session(tcp::socket socket) {
2      char buffer[1024];
3      while (true) {
4          std::size_t n = co_await socket.async_read_some(
5              asio::buffer(buffer), asio::use_awaitable);
6          co_await asio::async_write(
7              socket, asio::buffer(buffer, n), asio::use_awaitable);
8      }
9  }
```

Listagem 135: Uma sessão TCP inteira, escrita como código sequencial.

```
1  [servidor] ouvindo na porta 12399
2  [servidor] cliente 1 conectado
3  [servidor] cliente 2 conectado
4  [cliente] enviou: 'ola mundo!' -> recebeu echo: 'ola mundo!'
5  [cliente] enviou: 'coroutines!' -> recebeu echo: 'coroutines!'
```

```
6 | Servidor, listener e 2 clientes -- tudo em 1 thread
```

Listagem 136: Saída de `dist/bin/coroutines`, demo 4.2: servidor, *listener* e dois clientes no mesmo `io_context`.

O contraste com o modelo *uma* thread *por conexão* é de ordem de grandeza: cada *thread* reserva pilha própria e um descritor de escalonamento, o que na prática limita servidores a alguns milhares de conexões; cada co-rotina custa um *frame* de dezenas ou centenas de bytes. E o desempenho por operação é o mesmo dos *callbacks* — é o *mesmo* `io_context` por baixo, com a diferença de que o fluxo de controle voltou a ser linear.

Tabela 51: Qual primitiva usar, e por quê.

Problema	Ferramenta
produzir sequência longa ou infinita	`Generator<T>` com `co_yield`
sequência curta, percorrida várias vezes	`std::vector` (Seção III.1)
transformar sequência sem estado próprio	*views* (Seção III.2)
esperar um resultado assíncrono	`Task<T>` com `co_await`
I/O de rede ou disco sem parar a *thread*	`asio::awaitable<T>`
trabalho de CPU em paralelo	*threads* (Seção VI)

Regra — co-rotina é concorrência, não paralelismo

Nada neste mecanismo cria *thread* nenhuma. Uma co-rotina suspensa não usa CPU, mas uma co-rotina que faz cálculo pesado ocupa a *thread* que a retomou pelo tempo inteiro do cálculo — e trava todas as outras co-rotinas do mesmo `io_context`. Co-rotinas resolvem **esperas**; *threads* resolvem **trabalho**. Misturar os dois papéis é o erro mais comum de quem chega das linguagens em que `async` implica um pool escondido.

III.7 Síntese

O capítulo tem um padrão recorrente que vale nomear: quase toda armadilha aqui é uma **abstração que vaza de propósito**. `std::remove` não remove porque algoritmos não conhecem containers — e essa ignorância é exatamente o que permite $N + M$ implementações em vez de $N \times M$. `vector<bool>` mente sobre ser um container porque trocou a semântica pela densidade. `string_view` e `span` são frágeis porque não possuem nada — e é por não possuírem que custam zero. As *views* do `<ranges>` penduram quando a origem é local porque são preguiçosas, e a preguiça é o que elimina os temporários. O *frame* de uma co-rotina sobrevive ao `return` e por isso precisa que alguém o destrua — e é por sobreviver que ele permite suspender e retomar.

Em todos os casos, o preço da propriedade que se quer é a armadilha que se recebe junto. Conhecer a troca é o que separa usar a biblioteca de brigar com ela.

Regra — as duas ferramentas de diagnóstico deste capítulo

Além dos *sanitizers* (Seção I.9), a biblioteca padrão do GCC tem um modo de depuração próprio: `-D_GLIBCXX_DEBUG` verifica em tempo de execução iteradores invalidados, comparadores que não são *strict weak ordering*, comparação de iteradores de containers diferentes e índices fora do intervalo. Nenhuma dessas verificações existe no modo normal, e todas cobrem armadilhas descritas aqui. Vale rodar os subprojetos deste capítulo sob as duas ferramentas ao menos uma vez.

IV Programação genérica

O C++ tem duas linguagens dentro de si. Uma roda quando o programa roda; a outra roda quando o programa *compila*, e é sobre ela que este capítulo trata.

A ordem interna importa mais aqui do que em qualquer outro capítulo. `src/templates/` vem antes de `src/concepts/` porque *concepts* é a resposta do C++20 a um problema que só se entende depois de ter sofrido com SFINAE — estudar na ordem inversa inverte a história e transforma uma solução elegante em sintaxe arbitrária. `src/compile_time/` vem depois porque `constexpr` e *templates* são as duas metades da mesma capacidade de calcular antes de rodar. E `src/lambdas/` fecha o capítulo porque a discussão central dele — `std::function` contra parâmetro de *template* — só faz sentido para quem já sabe o que um parâmetro de *template* custa.

IV.1 Templates

Um *template* não é uma função nem uma classe: é uma **receita** a partir da qual o compilador gera funções e classes. A consequência que governa tudo o mais é que cada instanciação gera código *separado*, especializado para aquele tipo — daí virem juntos o desempenho (nada é resolvido em tempo de execução) e os dois custos clássicos: binário maior e mensagens de erro proporcionais à profundidade da instanciação.

IV.1.1 As formas básicas

O subprojeto `src/templates/` percorre a progressão inteira. As duas formas de partida são o *template* de função, cujo parâmetro é deduzido do argumento, e o de classe, que até o C++17 exigia o argumento escrito à mão:

```
1  template<typename T>
2  T max(T a, T b) { return (a > b) ? a : b; }
3
4  max(3, 7);            // T = int, deduzido
5  max(3.14, 2.71);      // T = double, deduzido
6  max<std::string>("abc", "xyz");   // T explícito
```

Listagem 137: Dedução na função; CTAD, a partir do C++17, também na classe.

E há uma família inteira de formas que vale conhecer antes de precisar delas (Tabela 52), porque cada uma resolve um problema que as outras não resolvem.

Tabela 52: As formas de *template* da linguagem.

Forma	O que declara
`template <typename T> T maior(T, T);`	*template* de função
`template <typename T> class Pilha { };`	*template* de classe
`template <typename T, std::size_t N> class Buf;`	NTTP: um *valor* como parâmetro
`template <typename T = int> struct V { };`	argumento padrão
`template <> struct X<int> { };`	especialização **total**
`template <typename T> struct X<T*> { };`	especialização **parcial** (só classes)
`template <typename T> constexpr bool eh_v = ...;`	*template* de variável
`template <typename T> using Par = std::pair<T,T>;`	*template* de *alias*

Armadilha — nomes dependentes exigem `typename`

Dentro de um *template*, `T::value_type` é ambíguo para o compilador: pode ser um tipo ou um membro estático. Ele supõe *valor* por padrão, e a correção é escrever `typename T::value_type`. O sintoma é uma mensagem de erro que reclama de sintaxe numa linha que parece sintaticamente perfeita. A mesma história vale para `template` antes de um membro *template* dependente.

Armadilha — especialização parcial não existe para funções

Só *templates de classe* podem ser parcialmente especializados. Para funções, a ferramenta correta é a resolução de sobrecarga — ou, hoje, um *concept* (Seção IV.2). Tentar especializar parcialmente uma função leva a um erro obscuro ou, pior, a uma sobrecarga que compila e nunca é escolhida.

IV.1.2 Variádicos e *fold expressions*

Antes do C++11, aceitar um número variável de argumentos tipados exigia escrever uma sobrecarga para cada aridade. Os *parameter packs* substituem toda essa família por um *template*, e as *fold expressions* do C++17 substituem a recursão que os processava:

```
// C++03: uma sobrecarga por aridade — e nenhuma verificação de tipo na variádica C
void imprimir() {}                                  // C++11: caso base

template<typename First, typename... Rest>          // C++11: recursão
void imprimir(First first, Rest... rest) {
    fmt::print("{} ", first);
    imprimir(rest...);
}

template<typename... Ts>                            // C++17: fold expression
auto soma(Ts... vals) { return (vals + ...); }
```

Listagem 138: O mesmo problema, em três gerações da linguagem.

A *fold* tem quatro formas, e a diferença entre elas é a associatividade — que importa quando o operador não é associativo (Tabela 53).

Tabela 53: As formas de *fold expression*.

Escrito	Expande para
`(args + ...)`	`a1 + (a2 + (a3 + a4))`
`(... + args)`	`((a1 + a2) + a3) + a4`
`((f(args)), ...)`	`f(a1), f(a2), f(a3), ...`
`((vals > 0) && ...)`	conjunção de todos

IV.1.3 SFINAE, e por que ele existiu

SFINAE é uma regra de resolução de sobrecarga: se substituir os parâmetros de *template* produzir um tipo inválido *na assinatura*, o candidato é apenas descartado, sem erro. A regra existe para permitir sobrecargas que se excluem mutuamente conforme uma propriedade do tipo:

```
template<typename T>
typename std::enable_if<std::is_integral<T>::value, T>::type
dobro(T x) { return x * 2; }
```

```cpp
template<typename T>
typename std::enable_if<std::is_floating_point<T>::value, T>::type
dobro(T x) { return x * 2.0; }
```

Listagem 139: Duas sobrecargas mutuamente exclusivas, à moda pré-C++20.

E o *detection idiom* é a aplicação mais elaborada da mesma regra: descobrir se um tipo tem determinado membro, sem exigir que ele o tenha.

```cpp
template<typename T, typename = void>
struct tem_size : std::false_type {};

template<typename T>
struct tem_size<T, std::void_t<decltype(std::declval<T>().size())>>
    : std::true_type {};

static_assert(tem_size<std::vector<int>>::value);
static_assert(!tem_size<int>::value);
```

Listagem 140: O *detection idiom*: especialização parcial que só é válida se `.size()` existir.

Por que — por que ninguém escreve isso de bom grado

As três linhas acima expressam "T tem `size()`" — uma ideia que cabe em cinco palavras — por meio de uma especialização parcial cujo segundo parâmetro existe apenas para ser descartado. É ilegível de propósito nenhum: ninguém projetou o SFINAE para isso, ele apenas *acabou* servindo. Quando falha, a mensagem cita o interior da instanciação em vez do erro do chamador. É esse acúmulo que os *concepts* vieram desfazer.

IV.1.4 *Type traits* e `if constexpr`

Os *type traits* são a interface de introspecção: perguntas (`is_integral_v`, `is_polymorphic_v`, `is_nothrow_move_constructible_v`) e transformações (`remove_const_t`, `decay_t`, `add_pointer_t`). `decay_t` merece nota: é exatamente o que `auto` faz ao deduzir por valor — `decay_t<const int&>` é `int`, e `decay_t<int[5]>` é `int*`.

O `if constexpr` do C++17 substituiu boa parte do SFINAE em código novo, porque escolhe o galho *descartando* o outro — que nem precisa ser válido para aquele tipo:

```cpp
template<typename T>
void processar(T val) {
    if constexpr (std::is_integral_v<T>) {
        fmt::print("inteiro: {} (dobro: {})\n", val, val * 2);
    } else if constexpr (std::is_floating_point_v<T>) {
        fmt::print("float: {:.4f}\n", val);
    } else {
        fmt::print("outro tipo\n");
    }
}
```

Listagem 141: Um corpo, três comportamentos, nenhuma sobrecarga.

IV.1.5 CRTP: polimorfismo sem *vtable*

O CRTP é a classe derivada passando *a si mesma* como argumento de *template* da base. Isso dá à base o tipo exato do derivado em tempo de compilação, e portanto o direito de chamá-lo por `static_cast` em vez de por vtable / vptr:

```cpp
    // A base chama métodos da derivada via static_cast — zero overhead
    double area() const {
        return static_cast<const Derived*>(this)->area_impl();
    }

    double perimetro() const {
        return static_cast<const Derived*>(this)->perimetro_impl();
    }
```

Listagem 142: A base chamando a derivada, em `src/templates/demo_crtp.hpp`.

A aplicação que mais aparece, porém, não é substituir `virtual`: é o *mixin* que deriva operações a partir de poucas primitivas. O usuário escreve `==` e `<`; os outros quatro operadores chegam de graça:

```cpp
struct Comparavel {
    bool operator!=(const Derived& outro) const {
        return !(static_cast<const Derived&>(*this) == outro);
    }
    bool operator>(const Derived& outro) const {
        return outro < static_cast<const Derived&>(*this);
    }
    bool operator<=(const Derived& outro) const {
        return !(static_cast<const Derived&>(*this) > outro);
    }
    bool operator>=(const Derived& outro) const {
        return !(static_cast<const Derived&>(*this) < outro);
    }
};
```

Listagem 143: Um *mixin* de comparação, em `src/templates/demo_crtp.hpp`.

Regra — em C++20, esse *mixin* específico virou uma linha

O `operator<=>` (Seção II.4) deriva os seis operadores de comparação a partir de um `= default`. O CRTP continua valendo para *outras* famílias de operações derivadas — é o mecanismo de `std::ranges::view_interface` (Seção III.2) — mas comparação já não é motivo para escrevê-lo.

Tabela 54: CRTP e `virtual` resolvem problemas parecidos com trocas opostas.

Aspecto	`virtual`	CRTP
resolução	execução	compilação
custo	vtable + vptr	zero
container polimórfico	`vector<Base*>`	**não** (tipos diferentes)
código gerado	um por hierarquia	um por tipo
mensagens de erro	legíveis	verbosas

A terceira linha é a decisiva. CRTP não tem base comum em tempo de execução: `Circulo` e `Quadrado` derivados de `FormaBase<Circulo>` e `FormaBase<Quadrado>` não têm nenhum tipo em comum, e portanto não podem ir para o mesmo container. Se o programa precisa de uma coleção heterogênea, CRTP não é opção — e é aí que o Seção V.3 propõe uma terceira via.

IV.2 Concepts

O subprojeto `src/concepts/` começa pela comparação que justifica a existência da funcionalidade:

```cpp
template<typename T>
typename std::enable_if<std::is_integral<T>::value, T>::type
soma(T a, T b) { return a + b; }     // C++17
```

```
4
5  template<std::integral T>
6  T soma(T a, T b) { return a + b; }   // C++20
```

Listagem 144: A mesma restrição, antes e depois do C++20.

Um *concept* é um predicado booleano sobre tipos, avaliado em compilação. A diferença prática em relação ao `static_assert` é *quando* ele age, e essa diferença muda o resultado do programa.

IV.2.1 As quatro formas de escrever a mesma restrição

```
1  template <Somavel T>            T f(T, T);                 // (a) no lugar do typename
2  template <typename T> requires Somavel<T> T f(T, T);       // (b) cláusula requires
3  template <typename T> T f(T, T) requires Somavel<T>;       // (c) requires no fim
4  auto f(Somavel auto a, Somavel auto b);                    // (d) abreviada
```

Listagem 145: Quatro sintaxes equivalentes.

A forma (d) é a mais curta e a mais enganosa: cada `auto` restringido introduz um parâmetro de *template independente*, de modo que `f(1, 2.0)` compila — os dois `Somavel` podem ser tipos diferentes. Quando os tipos precisam coincidir, use a forma (a).

IV.2.2 As quatro espécies de *requirement*

Dentro de um `requires`, quatro coisas diferentes podem ser exigidas, e o subprojeto tem um exemplo de cada.

Simple requirement — a expressão precisa compilar; nada se diz sobre o tipo do resultado:

```
1  // Concept: tipo que pode ser somado com si mesmo
2  template <typename T>
3  concept Somavel = requires(T a, T b) {
4      a + b;          // simple requirement: a + b precisa compilar
5      a - b;          // a - b também precisa compilar
6      a += b;         // a += b também
7  };
```

Listagem 146: *Simple requirements*, em `src/concepts/demo_requires.hpp`.

Type requirement — um tipo aninhado precisa existir. É a forma de exigir o contrato de container discutido na Seção III.1:

```
1  template <typename T>
2  concept Container = requires {
3      typename T::value_type;       // type requirement: value_type existe
4      typename T::iterator;         // type requirement: iterator existe
5      typename T::const_iterator;   // type requirement: const_iterator existe
6  } && requires(T c) {
7      c.begin();                    // begin() existe
8      c.end();                      // end() existe
9      c.size();                     // size() existe
10 };
```

Listagem 147: *Type requirements* combinados com expressões, em `src/concepts/demo_requires.hpp`.

Compound requirement — a expressão precisa compilar *e* o resultado precisa satisfazer outro *concept*: `{ a + b } -> std::same_as<T>;`. É a forma que verifica tipo de retorno, e a que torna *concepts* estritamente mais expressivos que `enable_if` sobre *traits*.

Nested requirement — um `requires` dentro do `requires`, que aplica outro *concept* ou uma condição booleana:

```
1 // Concept: tipo numérico de precisão adequada (≥ 4 bytes)
2 template <typename T>
3 concept NumericoPreciso = requires {
4     requires std::floating_point<T>;          // nested: sub-concept aplicado
5     requires (sizeof(T) >= sizeof(float));    // nested: constraint booleana
6 };
```

Listagem 148: *Nested requirements*, em `src/concepts/demo_requires.hpp`.

Armadilha — um `requires` sem `requires` é sempre verdadeiro

A construção `requires { expr; }` verifica se `expr` *compila*; ela não verifica se `expr` é *verdadeira*. Escrever `requires { sizeof(T) >= 4; }` é um *simple requirement* que pergunta se a comparação compila — e ela sempre compila. A forma correta precisa do segundo `requires`, como acima. É o erro mais comum de quem começa, e ele não produz nenhum diagnóstico: o *concept* passa a aceitar tudo.

IV.2.3 *Subsumption*

Esta é a propriedade que nenhuma emulação com `enable_if` oferecia sem trabalho manual: quando dois *templates* restritos são candidatos, o compilador escolhe o **mais específico**, e sabe qual é o mais específico comparando os *concepts* termo a termo.

```
1 template<typename T> concept Numerico = std::is_arithmetic_v<T>;
2 template<typename T> concept NumericoInteiro = Numerico<T> && std::integral<T>;
3
4 template<Numerico T>         void processar(T x) { /* genérico */ }
5 template<NumericoInteiro T> void processar(T x) { /* inteiro  */ }
6
7 processar(3.14);    // genérico — double não é integral
8 processar(42);      // inteiro  — satisfaz os dois, e NumericoInteiro subsume
```

Listagem 149: A escolha automática da restrição mais forte.

Com `enable_if`, as duas versões seriam ambíguas para `int`, e a desambiguação exigiria negar explicitamente a condição na versão genérica.

Regra — *concept* para restringir, `static_assert` para afirmar

A diferença é *quando* cada um age:

Ferramenta	Quando falha	Afeta a sobrecarga?
`static_assert`	**depois** da escolha	não
concept	**durante** a escolha	**sim**

Um `static_assert` dentro de um *template* já escolhido não deixa outra sobrecarga assumir — ele apenas transforma a escolha ruim num erro. O *concept* remove o candidato da disputa, e a sobrecarga não restrita vence normalmente.

IV.2.4 Onde *concepts* rendem mais

Duas aplicações do subprojeto merecem destaque porque não são sobre restringir um argumento, e sim sobre **escrever um contrato**.

A primeira é o algoritmo genérico que declara exatamente o que exige do *range* e do predicado — uma assinatura que documenta a si mesma e que falha no ponto da chamada, não trinta níveis abaixo:

```
1 template<std::ranges::input_range Range, typename Pred>
2     requires std::predicate<Pred, std::ranges::range_value_t<Range>>
3 auto filtrar(const Range& range, Pred pred);
```

Listagem 150: O contrato inteiro na assinatura.

A segunda é o *policy-based design*: um *concept* descreve o que uma política precisa oferecer, e a classe que a recebe passa a recusar em compilação qualquer política incompleta — com uma mensagem que nomeia o método que falta, em vez de apontar para a linha em que ele seria chamado.

Tabela 55: SFINAE e *concepts*, ponto a ponto.

Aspecto	SFINAE	Concepts
legibilidade	ruim (aninhamento, `enable_if`)	boa
mensagens de erro	gigantescas	citam a restrição violada
subsumption	manual e verbosa	automática
verificação	implícita, por efeito colateral	explícita, via `requires`
reutilização	difícil	trivial (são nomeados)

IV.3 Computação em tempo de compilação

O subprojeto `src/compile_time/` trata das quatro palavras-chave que movem trabalho da execução para a compilação — `constexpr`, `consteval`, `constinit`, `static_assert` — e dos parâmetros de *template* que são valores. Compila com C++23, por causa de `if consteval` (Seção I.6).

O argumento a favor não é apenas desempenho. É este:

Por que — em contexto `constexpr`, UB (comportamento indefinido) vira erro de compilação

```
1 constexpr int x = 1 / 0;      // erro de compilação, não crash em produção
2 constexpr int y = arr[10];    // erro se fora dos limites
```

O avaliador de expressões constantes é obrigado a recusar qualquer operação cujo comportamento o padrão não define. Código que passa sob `static_assert` é código **provadamente** livre de UB (comportamento indefinido) naquele caminho — uma garantia que nenhuma bateria de testes oferece, porque testes cobrem entradas e isto cobre o caminho inteiro.

IV.3.1 `constexpr` é permissão; `consteval` é obrigação

A mesma função `constexpr` serve aos dois mundos: chamada num contexto que exige constante, é avaliada na compilação; chamada com um argumento de execução, é compilada normalmente.

Tabela 56: As três palavras, e o que cada uma exige.

Palavra	Avaliação em compilação	Argumento de execução
(nenhuma)	não	aceita
`constexpr`	**permitida**	aceita
`consteval`	**obrigatória**	**erro**

Forçar a avaliação em compilação, quando a função é apenas `constexpr`, se faz pelo contexto: `constexpr int a = f();` força naquela declaração; `static_assert(f() == x);` força e ainda verifica; e `std::array<int, f()>` força, porque um NTTP é o contexto mais exigente que existe.

A aplicação mais útil de `consteval` é a **validação de argumento literal**. Como um `throw` que escapa de uma avaliação constante é erro de compilação, o construtor abaixo recusa portas inválidas antes de o programa existir:

```
class Porta {
public:
    consteval explicit Porta(int valor) : valor_(valor) {
        if (valor < 1 || valor > 65535) {
            // Numa avaliação em compilação, um throw que escapa é ERRO DE
            // COMPILAÇÃO. A mensagem do compilador cita esta linha.
            throw "porta fora do intervalo 1..65535";
        }
    }
    constexpr int valor() const { return valor_; }
private:
    int valor_;
};
```

Listagem 151: Validação em compilação, em `src/compile_time/demo_consteval.hpp`.

É exatamente o mecanismo do `std::format` do C++20: `std::format("{:d}", "texto")` falha na **compilação**, e não com uma exceção em produção.

Armadilha — `std::is_constant_evaluated()` tem duas pegadinhas

Dentro de um `if` comum, ele responde `true` sempre que a função *é* `constexpr`, mesmo quando a chamada está sendo compilada para execução. E `if constexpr (std::is_constant_evaluated())` é **sempre** verdadeiro, porque a própria condição é avaliada como constante. O `if consteval` do C++23 existe para não ter nenhuma das duas.

Armadilha — `constexpr std::vector` não compila

Desde o C++20 é possível usar `std::vector` e `std::string` *durante* uma avaliação constante — mas vale a **regra da alocação transitória**: tudo o que for alocado precisa ser liberado antes de a avaliação terminar, porque a memória do compilador não sobrevive até a execução. Logo, um `vector` pode ser usado no meio do cálculo e nunca ser o resultado dele. Para devolver dados de uma função `constexpr`, o tipo é `std::array`.

O terceiro membro da família resolve um problema diferente. `constinit` exige inicialização em compilação, mas **permite** mutação depois — o que o torna a resposta direta ao *static initialization order fiasco*: se o compilador recusa qualquer inicialização que não caiba na compilação, o fiasco deixa de ser um bug de execução e vira um erro de *build*.

IV.3.2 `static_assert`: onde não há substituto

Regra — a mensagem é o produto

Uma mensagem de `static_assert` deve dizer **o que fazer**, não o que falhou — quem lê já sabe o que falhou, porque o compilador cita a condição.

```
static_assert(std::is_trivially_copyable_v<T>,
    "serializar() copia bytes crus e só aceita tipos trivialmente copiáveis. "
    "Tipos com ponteiros, std::string ou destrutor próprio precisam de uma "
    "sobrecarga específica.");
```

Custa uma linha e poupa a próxima pessoa de reconstruir a intenção a partir do código.

E há um domínio em que *concepts* não substituem `static_assert`, porque *concepts* restringem *templates* e não dizem nada sobre *layout*: a verificação de ABI.

```
static_assert(sizeof(CabecalhoDeRede) == 8,
    "O cabeçalho tem 8 bytes por especificação. Se este assert falhou, o "
    "padding mudou e os pacotes serão incompatíveis com a outra ponta.");
static_assert(offsetof(CabecalhoDeRede, comprimento) == 2);
```

Listagem 152: Afirmações sobre *layout* que nenhum *concept* expressa.

IV.3.3 NTTP: o valor entra no tipo

Um parâmetro de *template* que não é um tipo faz o *valor* participar da identidade do tipo. `BufferFixo<int,4>` e `BufferFixo<int,16>` são tipos **diferentes**, e `capacidade()` não lê memória nenhuma.

A aplicação que mais rende é verificação dimensional — a mesma ideia dos literais de unidade da Seção II.4, agora com dimensões de matriz:

```
Matriz<2,3> * Matriz<3,4> → Matriz<2,4>     // compila
Matriz<2,3> * Matriz<2,2> → NÃO COMPILA     // dimensões incompatíveis
```

Listagem 153: Erro de dimensão detectado em compilação.

Sem NTTP, as dimensões seriam campos, e o erro só apareceria em tempo de execução — provavelmente como leitura fora dos limites.

O C++20 permitiu NTTP de tipo classe, o que finalmente admite *strings* como parâmetro de *template*. A condição é o tipo ser *structural*: todos os membros públicos, sem `private`, sem `virtual`, e os membros também *structural* — razão pela qual `std::string` não serve, e um `StringFixa` escrito à mão serve.

Armadilha — cada valor distinto é uma instanciação nova

NTTP tem um custo que não aparece em nenhum *benchmark* de execução: binário maior e compilação mais lenta, proporcionalmente ao número de valores distintos usados. Use quando os valores forem poucos, fixos e participarem da *correção* do programa. Não use quando o valor vier de configuração ou de entrada — e, se a intenção era apenas evitar uma alocação, o instrumento certo é `std::span` (Seção III.4).

IV.3.4 Gerando dados na compilação

O demo final gera uma tabela de CRC-32 e um mapa de códigos HTTP em tempo de compilação, e depois *prova* onde eles foram parar:

```
$ objdump -t dist/bin/compile_time | grep -i TABELA
.rodata       0x400  TABELA_CRC32
.data.rel.ro  0x0d8  TABELA_HTTP
```

Listagem 154: Saída de `src/compile_time/`: as tabelas no binário.

As seções diferem, e a razão é instrutiva: `TABELA_CRC32` contém apenas `uint32_t` — valores puros, que vão para `.rodata`. `TABELA_HTTP` contém `string_view`, que são **ponteiros**: o endereço só é conhecido quando o *loader* mapeia o binário, e por isso ela vai para `.data.rel.ro` — somente-leitura *após* a relocação. Nenhuma das duas está em `.bss`, o que indicaria inicialização em tempo de execução.

Como `std::sort` é `constexpr` desde o C++20, a tabela pode ser escrita fora de ordem e ordenada pelo compilador — e a busca binária em tempo de execução passa a ser válida porque a ordenação foi *provada*, não testada:

```
static_assert(std::is_sorted(TABELA_HTTP.begin(), TABELA_HTTP.end()));
```

Listagem 155: A pré-condição da busca binária, verificada na compilação.

Tabela 57: Para poucas dezenas de entradas fixas, o array ordenado ganha nas três colunas.

Estrutura	`constexpr`	Aloca	Busca
`std::map`	não	sim	$O(\log n)$
`std::unordered_map`	não	sim	$O(1)$ médio
array ordenado	**sim**	**não**	$O(\log n)$

Armadilha — uma API `constexpr` robusta usa índices, não ponteiros

O subprojeto registra uma lição que só aparece na prática: `return p ? *p : padrao;` parece natural, mas converter um ponteiro-para-membro-de-objeto-`constexpr` em `bool` **não é expressão constante** em todos os modos do GCC — o mesmo código compila normalmente e falha sob `-fsanitize`. Aritmética de *índice* é sempre constante, e por isso é a base mais robusta.

Regra — o custo é pago pelo compilador, e ele é lento

O interpretador de `constexpr` do GCC é ordens de grandeza mais lento que código nativo, e tem teto (`-fconstexpr-ops-limit`, 33,5 milhões de passos). Não vale a pena quando a tabela é grande e o *build* fica lento, quando os dados mudam com frequência, ou quando o cálculo já era trivial na inicialização. Vale muito quando a tabela é pequena e imutável, quando se quer *provar* uma propriedade, em ambiente sem *heap* (embarcado, *kernel*), e para eliminar inicialização estática dinâmica.

IV.4 Lambdas e objetos chamáveis

O subprojeto `src/lambdas/` parte de uma única ideia que explica praticamente tudo o mais:

```
auto f = [x](int y) { return x + y; };

// o compilador gera, aproximadamente:
```

```
class __lambda {
    int x;                                  // a captura vira MEMBRO
public:
    auto operator()(int y) const { return x + y; }
};
```

Listagem 156: Uma *lambda* é uma classe.

Disso decorrem, sem nenhum acréscimo: por que `mutable` existe (o `operator()` gerado é `const`); por que capturar por referência é perigoso (o membro é uma referência, e ela pode pendurar); por que cada *lambda* tem tipo único (é uma classe anônima nova a cada expressão); e por que `[=]` dentro de um método é a armadilha número um.

IV.4.1 Capturas

```
sizeof ([]{})      = 1     ← classe VAZIA
sizeof ([a]{})     = 4     = sizeof(int)
sizeof ([&a]{})    = 8     = um ponteiro
sizeof ([s]{})     = 32    = sizeof(std::string)
```

Listagem 157: Saída de `src/lambdas/`: o tamanho da classe gerada é a soma das capturas.

Armadilha — `[=]` num método captura `this` por ponteiro, não os membros

```
[=] { return nome_; }
  // NÃO copia `nome_`
  // copia `this`, e `nome_` vira `this->nome_`
```

Captura	Copia	Seguro após o objeto morrer
`[this]`	o ponteiro	**não**
`[=]`	o ponteiro (!)	**não**
`[*this]`	o **objeto**	sim (C++17)
`[n=nome_]`	só o membro	sim, e é o mais barato

O C++20 **depreciou** a captura implícita de `this` via `[=]`. A regra: se a *lambda* pode sobreviver ao objeto — *callback*, *thread*, `std::function` guardada — nunca capture `this`.

A *init-capture* do C++14 resolve quatro problemas com uma sintaxe só: `[r = std::move(ptr)]` captura tipos *move-only*; `[total = v.size() * 10]` pré-calcula uma vez em vez de a cada chamada; `[h = cfg.host]` captura um *membro* sem copiar o objeto inteiro; e `[&n = contador]` dá nome curto a algo de nome longo.

Armadilha — ordem de avaliação de argumentos, medida num bug real

O próprio subprojeto encontrou o caso ao instrumentar uma *lambda* `mutable`: `print("{} {} {}", f(), f(), f())` tem ordem de avaliação **não especificada**, e o GCC avalia da direita para a esquerda — imprimindo `3 2 1`. Quando o efeito colateral importa, use variáveis nomeadas em vez de várias chamadas na mesma expressão.

IV.4.2 Genéricas

Uma *lambda* com parâmetro `auto` é um *template* de `operator()`. A forma com *template* explícito, do C++20, é preferível sempre que o *tipo* precisar de nome — para `std::forward`, para declarar variáveis

daquele tipo, ou para acessar `T::algo`:

```
[](auto&& x)          { return std::forward<decltype(x)>(x); }   // verboso
[]<typename T>(T&& x) { return std::forward<T>(x); }             // legível
```

Listagem 158: A mesma *lambda*, com e sem nome para o tipo.

E vale a mesma medição da Seção II.3, agora dentro da *lambda*:

```
sem forward, arg rvalue: 1 cópia(s), 0 move(s)     ← o parâmetro tem NOME, é lvalue
com forward, arg rvalue: 0 cópia(s), 1 move(s)
com forward, arg lvalue: 1 cópia(s), 0 move(s)     ← correto
```

Listagem 159: Saída de `src/lambdas/`: o parâmetro tem nome, logo é *lvalue*.

IV.4.3 `std::function`: *type erasure* e o seu preço

`std::function` resolve o problema criado pelo fato de cada *lambda* ter tipo próprio: guardar chamáveis heterogêneos com a mesma assinatura. O mecanismo é *type erasure*, e a estrutura é a mesma do `std::any` (Seção III.4) e do ECS (Seção V.3):

```
struct Conceito { virtual R chamar(Args...) const = 0; };
template <typename F> struct Modelo : Conceito { F f_; /* ... */ };
std::unique_ptr<Conceito> impl_;
```

Listagem 160: O esqueleto do *type erasure*.

O tipo concreto some da assinatura mas continua vivo dentro do `Modelo<F>` — e é exatamente daí que vem o custo: a chamada passa por uma vtable / vptr e o corpo não pode sofrer *inline*.

```
template<F>       :  12 ms   ← inlinável
std::function     :  43 ms   ← chamada virtual (3,5×)
ponteiro p/ função :  13 ms
```

Listagem 161: Saída de `src/lambdas/`: 2 milhões de elementos.

A medição foi feita em `-O0`; em `-O2` a diferença **aumenta**, porque o *template* costuma desaparecer por completo enquanto `std::function` mantém a indireção. Há ainda o custo de alocação: a otimização de objeto pequeno cobre até certo tamanho de captura, e acima disso cada `std::function` é um *malloc*.

Regra — não use `std::function` como parâmetro de quem só chama e descarta

É o erro mais comum da família. Uma função que recebe um comparador, o chama e o esquece deve recebê-lo como `template <typename Comp>`: assim ele é inlinado e não aloca. `std::function` é para **guardar** — membro, container, fila de tarefas — e ali o custo se paga.

Tabela 58: As três formas de receber ou guardar um chamável.

Ferramenta	Inlina	Aloca	Guardável	Assinatura
`template <typename F>`	**sim**	não	não	genérica
ponteiro para função	não	não	sim	fixa
`std::function`	não	talvez	**sim**	fixa

IV.4.4 `std::invoke` e ponteiros para membro

A linguagem trata **cinco** sintaxes de chamada de forma diferente — `f(args)`, `(obj.*pm)(args)`, `(ptr->*pm)(args)`, `obj.*pd`, `ptr->*pd` — e escrever código genérico sobre elas exigiria um `if constexpr` para cada caso. `std::invoke` unifica as cinco, e é por isso que ele aparece na implementação de `std::thread`, `std::async` e de qualquer *thread pool* (Seção VI.1).

O uso mais prático de um ponteiro para membro, no entanto, é como **projeção** nos algoritmos de *ranges*: `std::ranges::sort(equipe, {}, &Pessoa::idade)` (Seção III.2).

Regra — *lambda* em vez de `std::bind`

`std::bind` usa *placeholders* opacos, copia os argumentos ligados por padrão, gera erros ilegíveis quando aninhado e trata `bind` dentro de `bind` de forma surpreendente. As *Core Guidelines* [4] recomendam *lambda* (F.50). Quando a aplicação parcial é literalmente "fixar os primeiros argumentos", `std::bind_front` (C++20) é legível, inlinável e não aloca.

IV.4.5 Cinco padrões que só existem por causa das *lambdas*

IIFE. Inicializar um `const` que precisa de várias instruções, sem criar uma função auxiliar — e sem abrir mão do `const`:

```
const std::string prefixo = [nivel] {
    if (nivel >= 3) return "[DEBUG] ";
    if (nivel >= 2) return "[INFO] ";
    return "[ERRO] ";
}();
```

Listagem 162: *Immediately Invoked Function Expression.*

É praticamente obrigatório para inicializar um *membro* `const` com lógica na lista de inicialização (Seção II.1).

Overload idiom. Já apresentado na Seção III.4; o que o subprojeto acrescenta é a explicação de por que ele funciona — e por que a terceira linha é indispensável. `Overload : Ts...` herda de todas as *lambdas*, mas sem `using Ts::operator()...;` os `operator()` ficariam escondidos uns pelos outros: é o mesmo *name hiding* da Seção II.7.

Recursão. Uma *lambda* não tem nome dentro do próprio corpo. As três saídas diferem em custo e em disponibilidade (Tabela 59); enquanto o *deducing this* não chega ao GCC 13, o *y-combinator* é a opção certa em código quente.

Tabela 59: Recursão em *lambda*: as três formas.

Forma	Aloca	Inlina	Chamada	GCC 13
`std::function`	talvez	não	`f(n)`	sim
y-combinator	não	sim	`f(f, n)`	sim
deducing this	não	sim	`f(n)`	**não**

Scope guard. RAII para *qualquer* ação, escrito no ponto de uso, sem uma classe por recurso. É a generalização do que a Seção II.5 fazia com *deleters* customizados — e, com o `dispensar()`, cobre também o caso "desfaz, a menos que dê certo":

```
auto g = GuardaDeEscopo{[&]{ fclose(f); }};
auto desfazer = GuardaDeEscopo{[&]{ rollback(); }};
// ... deu certo:
desfazer.dispensar();                    // commit
```

Listagem 163: Limpeza e *rollback* com o mesmo mecanismo.

***Lambda* como comparador de container.** Desde o C++20, uma *lambda* sem captura é construtível por padrão, o que permite usá-la diretamente como argumento de *template*: `std::set<std::string, PorTamanho>`. A consequência a lembrar é que o comparador faz parte do **tipo**: dois *sets* com *lambdas* distintas são incompatíveis mesmo tendo a mesma lógica.

IV.5 Síntese

Os quatro subprojetos deste capítulo são quatro respostas à mesma pergunta: *quanto do programa dá para decidir antes de rodar?*

Templates decidem o *código*, gerando uma versão especializada por tipo. *Concepts* decidem a *admissibilidade*, movendo o erro para o ponto da chamada. `constexpr` decide os *valores*, e leva junto uma garantia que nenhum teste dá: naquele caminho não há UB (comportamento indefinido). E as *lambdas* decidem o *chamável*, com a ressalva de que `std::function` desfaz boa parte do ganho ao trocar o tipo estático por uma indireção.

Regra — a hierarquia de escolha em código genérico novo

Na ordem: (1) se dá para restringir com um *concept*, use *concept*; (2) se dá para escolher o galho com `if constexpr`, use `if constexpr`; (3) SFINAE apenas para manter código antigo ou para atingir compiladores anteriores ao C++20. E, para chamáveis: `template <typename F>` por padrão, `std::function` só quando for preciso *guardar*.

V Design e arquitetura

Os capítulos anteriores tratavam de *o que a linguagem faz*. Este trata de *como organizar o que se escreve com ela* — e a mudança de assunto vem com uma mudança de natureza: nada aqui é verificável pelo compilador. Um programa que viola os cinco princípios SOLID compila, roda e passa nos testes. O preço é cobrado depois, na primeira mudança de requisito.

A ordem interna é a da trilha, e ela tem uma lógica: `src/solid/` estabelece os princípios; `src/design-of-pattern/` mostra as soluções canônicas que os realizam; `src/ecs/` apresenta o contraponto radical à herança; e `src/plugin_registry/` leva o último princípio — inversão de dependência — até o limite do sistema operacional.

V.1 Os cinco princípios SOLID

O subprojeto `src/solid/` implementa cada princípio duas vezes: uma versão que o *viola* e uma que o *cumpre*, lado a lado, executáveis. É a única forma honesta de discutir o assunto, porque a versão violada quase sempre é a mais curta — e o custo dela não aparece até que alguém precise mudar algo.

V.1.1 SRP — responsabilidade única

A definição popular "uma classe, uma responsabilidade" é vaga a ponto de ser inútil. A definição precisa de Robert Martin [6] usa outra palavra:

> *"A module should be responsible to one, and only one, actor."*

Ator é um grupo de *stakeholders* que pode exigir a mesma mudança. Se duas equipes diferentes podem pedir alterações na mesma classe, ela tem duas responsabilidades — e a mudança pedida por uma vai quebrar o que a outra esperava.

O exemplo canônico do livro, reproduzido no subprojeto, é uma classe `Employee` que serve a três atores:

```
Employee serve a 3 atores:
    calculatePay()   → COO / RH
    reportHours()    → CFO (auditoria)
    save()           → CTO / DBA

    Risco: total_hours() compartilhado entre calculatePay() e reportHours()
    Uma mudança pedida pelo COO afeta silenciosamente o relatório do CFO!
```

Listagem 164: Saída de `src/solid/srp/`: o acoplamento oculto.

O perigo não é a classe ser grande. É o método privado `total_hours()` ser *compartilhado* entre dois atores que nunca conversaram: quando o COO pedir uma mudança na regra de horas, o relatório de auditoria do CFO muda junto, e ninguém será avisado.

Regra — a heurística do "e"

Se você precisa da conjunção "e" ou "ou" para descrever o que uma função faz, ela tem mais de uma responsabilidade. `process_and_print_student` é o nome delatando o problema. O complemento

natural para funções é a *Command-Query Separation*: um *command* muda estado e devolve `void`; uma *query* lê estado e não tem efeito colateral. Um `int increment_and_get()` é os dois ao mesmo tempo.

Tabela 60: Sinais de violação de SRP, e o que cada um costuma indicar.

Sinal	Problema provável
classe com mais de 200–300 linhas	múltiplas responsabilidades
nome genérico: `Manager`, `Helper`, `Utils`	*god class*
método com "e" ou "ou" na descrição	duas responsabilidades
teste que precisa de 5 ou mais *mocks*	a classe faz demais
mudar formato de e-mail quebra cálculo de salário	acoplamento oculto
classe inclui `<filesystem>`, `<regex>` *e* `<curl.h>`	mistura de camadas

Armadilha — SRP também pode ser aplicado demais

Classes de uma linha, um *getter* por classe, módulos tão pequenos que o custo de coordená-los supera o benefício — todos são sintomas de sobre-aplicação. A regra de ouro é **coesão acima de granularidade**: agrupe o que muda junto, separe o que muda por razões diferentes.

V.1.2 OCP — aberto para extensão, fechado para modificação

O sintoma de violação é sempre o mesmo: um `switch` ou uma cascata de `if/else if` sobre um *tipo*, repetida em várias funções.

```
1 double area(const Forma& f) {
2     if       (f.tipo == Circulo)   return 3.14159 * f.raio * f.raio;
3     else if  (f.tipo == Retangulo) return f.largura * f.altura;
4     // ← e um 'else if' por forma nova. Para sempre.
5 }
6 double perimetro(const Forma& f) { /* a MESMA cascata, repetida */ }
```

Listagem 165: A cascata, e o motivo de ela não escalar.

Acrescentar um triângulo exige editar **quatro** lugares já testados — e esquecer um deles não gera erro de compilação: gera resultado errado. Com polimorfismo, o triângulo é um *arquivo novo*, a função que consome nunca é tocada, e o `= 0` obriga a implementar tudo.

O subprojeto conta o custo, e a linha que decide é a terceira:

Tabela 61: O que muda ao acrescentar um caso novo. Código que não é tocado não pode regredir.

Ao acrescentar um caso	Com `switch`	Com OCP
arquivos criados	0	1
arquivos **editados**	3–5	**0**
testes a reescrever	3–5	**0**
risco de regressão	alto	nenhum

Quando o que varia é um *algoritmo* — e não uma família de entidades — a forma correta é injetá-lo, e há quatro mecanismos com trocas diferentes (Tabela 62).

O *concept* resolve o pior defeito da *policy* por *template*, que é a mensagem de erro. Declarando o contrato explicitamente, uma política incompleta falha *na linha da chamada*, dizendo qual expressão faltou — em vez de trezentas linhas de instanciação (Seção IV.2):

Tabela 62: Quatro formas de injetar variação. "Homogêneo" significa poder guardar variantes no mesmo `vector`.

Mecanismo	Extensão	Custo	Homogêneo?
herança	execução	vtable / vptr	sim
`std::function`	execução	indireção	sim
template (policy)	compilação	**zero**	não
concept	compilação	**zero**	não

```
template <typename P>
concept PoliticaDeDesconto = requires(double total) {
    { P::aplicar(total) } -> std::convertible_to<double>;
    { P::nome() }         -> std::convertible_to<std::string>;
};

template <PoliticaDeDesconto P> class Carrinho { /* ... */ };
```

Listagem 166: O contrato da política, escrito como *concept*.

O último demo do subprojeto vai além: um **registro auto-populado** elimina até a edição do ponto de composição. Uma macro no arquivo do exportador basta — `REGISTRAR_EXPORTADOR(ExportadorMarkdown, "markdown")` — e o laço que consome nunca foi editado, nem conhece os nomes das classes.

Armadilha — o registro auto-populado tem três armadilhas de ligação

Primeira: a ordem de inicialização estática entre unidades de tradução não é definida, então o registro precisa ser um *Meyers singleton* — função com `static` local — e nunca uma variável global. Segunda: o *linker* pode **descartar** o objeto estático se nada naquela unidade for referenciado (`--gc-sections`, bibliotecas estáticas), o que exige `--whole-archive` ou um símbolo âncora. Terceira: o fluxo fica indireto — não se descobre quem registrou lendo o `main`.

Regra — Rule of Three: OCP responde ao passado, não profetiza o futuro

OCP aplicado **antes** de existir um segundo caso é custo puro — e a abstração escolhida com uma amostra de tamanho um quase sempre é a errada. Na primeira vez, escreva direto. Na segunda, duplique e anote o incômodo. Na terceira, abstraia — porque agora você já viu o *eixo* da variação. E há o caso em que a cascata é a resposta certa: quando o conjunto é *fechado* (quatro naipes, três estados de semáforo), o `switch` é mais simples e mais rápido, ou então `std::variant` com `visit` (Seção III.4). A pergunta que decide: *casos novos vão chegar depois do* deploy*, vindos de quem não controla este arquivo?*

V.1.3 LSP — substituição de Liskov

É o terceiro princípio e o único que o compilador **não pode verificar**. LSP é sobre *comportamento*, não sobre assinatura: o compilador confere a assinatura, e ninguém confere o comportamento. Por isso violações de LSP compilam perfeitamente e aparecem em produção.

O exemplo canônico é canônico porque "quadrado é um retângulo" é verdade na matemática e mentira no código:

```
com Retangulo: largura=5, altura=4 → área esperada 20, obtida 20  ✓
com Quadrado:  largura=5, altura=4 → área esperada 20, obtida 16  ✗ LSP VIOLADO
```

Listagem 167: Saída de `src/solid/lsp/`: a mesma sequência de chamadas, resultados diferentes.

O erro não foi implementar mal o quadrado. Foi afirmar que ele *é um* retângulo **mutável** — e essa relação não sobrevive à mutabilidade. As duas saídas boas são a imutabilidade (sem *setters*, o conflito desaparece) e tratá-los como irmãos sob uma interface que ambos *podem* cumprir (`escala()` em vez de *setters* independentes).

Regra — a pergunta certa não é "X é um Y?"

É: *X consegue cumprir* ***todas*** *as promessas que Y faz aos seus clientes?* A primeira pergunta é sobre taxonomia; a segunda é sobre contrato — e só a segunda tem consequência para o código.

O contrato se decompõe em quatro regras, e todas seguem a mesma direção (Tabela 63).

Tabela 63: O derivado pode ser mais *generoso*, nunca mais *exigente* — a mesma contravariância dos tipos de função.

O subtipo pode...	Pré-condição	Pós-condição	Invariante
exigir **mais**	não	—	—
exigir **menos**	sim	—	—
entregar **mais**	—	sim	sim
entregar **menos**	—	não	não

As três primeiras regras são diretas: fortalecer uma pré-condição quebra quem chamava com argumentos que a base aceitava (um depósito "VIP" que exige mínimo de 100); enfraquecer uma pós-condição quebra quem contava com ela (um "ordenador" que também deduplica devolve menos elementos do que entraram); violar uma invariante sabota todo código que confiava nela (uma conta com cheque especial cujo saldo fica negativo).

Armadilha — a *history rule* não muda nenhuma assinatura

A quarta regra é a menos conhecida: um derivado não pode permitir mudanças de estado que a base proibia.

```
PontoImutavel  → cache válido ✓
PontoMutavel   → cache INVÁLIDO ✗ LSP
```

Nenhum método foi sobrescrito, nenhuma assinatura mudou. O que mudou foi o conjunto de **histórias possíveis** do objeto — e quem tinha um cache baseado na imutabilidade passou a ter um cache errado. É por isso que `final` e a imutabilidade são ferramentas de LSP, e não apenas de otimização.

Como o compilador não ajuda, o subprojeto oferece dois instrumentos práticos. O primeiro é o **teste de contrato**: uma suíte escrita contra a *base* e rodada contra *cada* derivado — em GoogleTest, é o que `TYPED_TEST_SUITE` faz.

```
PilhaVetor          5 passou, 0 falhou  ✓
PilhaLimitada(3)    3 passou, 2 falhou  ✗ LSP VIOLADO
  ✗ empilhar 10 → tamanho 10
  ✗ round−trip: 5 entraram, 5 devem sair
```

Listagem 168: Saída de `src/solid/lsp/`: o LSP virando código executável.

O segundo é o idioma **NVI**, já visto na Seção II.7 como solução para argumentos padrão em virtuais — e que aqui aparece no seu papel principal: colocar o contrato na base, onde o derivado não pode afrouxá-lo.

```cpp
std::string buscar(int id) {                  // público, NÃO virtual
    if (id <= 0) throw std::invalid_argument("id deve ser positivo");  // pré
    std::string r = buscar_impl(id);                                   // miolo
    if (r.empty()) throw std::runtime_error("devolveu vazio");         // pós
    return r;
}
```

```
7 private:
8     virtual std::string buscar_impl(int id) = 0;   // privado, virtual
```

Listagem 169: NVI: pré e pós-condições na fronteira pública, não virtual.

V.1.4 ISP — segregação de interfaces

ISP e LSP são o mesmo problema visto de dois lados: LSP pergunta *o derivado consegue cumprir tudo que a base promete?*; ISP pergunta *a base está prometendo coisas demais?*

A *fat interface* força um implementador a declarar métodos que ele não sabe fazer, e as três saídas disponíveis são todas ruins (Tabela 64).

Tabela 64: O que fazer num `override` que não se sabe implementar — e por que nenhuma das opções serve.

Saída	Problema
lançar exceção	viola o LSP — o cliente quebra
não fazer nada	pior: quebra em **silêncio**
devolver valor inventado	pior ainda: corrompe dados

A correção segrega cada capacidade em uma interface própria, e o ganho decisivo é de categoria: **o que era exceção em tempo de execução (ou silêncio) vira erro de compilação**.

```
1 void fluxo_de_impressao(Imprimivel& d, const std::string& doc);
2 void fluxo_de_arquivamento(Digitalizavel& d, const std::string& destino);
3
4 fluxo_de_impressao(scanner, "x");   // ERRO DE COMPILAÇÃO
```

Listagem 170: O cliente pede o mínimo de que precisa.

Por que — a interface pertence ao cliente, não ao fornecedor

A leitura ingênua do ISP é "interfaces pequenas". A leitura correta é mais forte: a interface é definida pelo que o **cliente** precisa, e por isso deve ser *declarada junto do cliente*. Um serviço de doze métodos pode ser consumido por dez clientes através de dez interfaces minúsculas, cada uma nomeada segundo o *papel* que cumpre, com adaptadores fininhos ligando o serviço legado a cada papel. O que isso compra: o enviador de e-mails **não consegue** tocar em dinheiro, garantido pelo tipo; e os dublês de teste têm dois métodos cada, em vez de doze — sem *framework* de *mock*, sem banco, sem rede.

Daí decorre uma regra de nomenclatura que muda o desenho: nomeie pelo **papel**, não pela classe concreta. `FonteDeContato` em vez de `IServicoDeUsuarios`; `LeitorDePedidos` em vez de `IRepositorioMySql`; qualquer coisa em vez de `IHelper`, `IManager` ou `IUtil`.

Armadilha — segregar demais vira sopa de interfaces

O limite é objetivo: cada interface deve ter *pelo menos um cliente que use todos os seus métodos*. Se nenhum cliente usa a interface inteira, ela ainda não foi segregada o bastante; se cada interface tem um método e nenhum cliente real, foi segregada demais.

V.1.5 DIP — inversão de dependência

O nome engana. **"Inversão" não é sobre injetar dependência** — é sobre a direção da seta no diagrama de módulos (Figura 5).

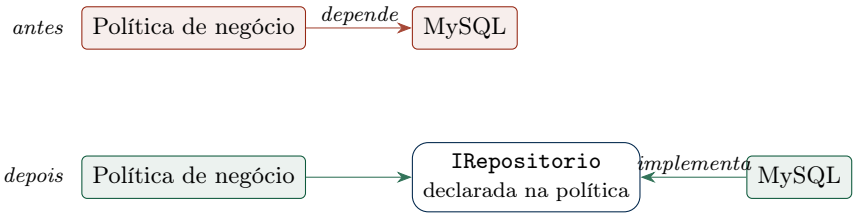


Figura 5: A inversão. A seta do MySQL passou a apontar *para dentro*, na direção da abstração que a própria política definiu — e é a abstração morar na camada de negócio, e não numa camada neutra, que caracteriza a inversão.

O detalhe que faz a diferença é onde a abstração é *declarada* e como é *nomeada*: `RepositorioDePedidos`, ao lado da política, e não `IBancoDeDados`, ao lado da infraestrutura. O mesmo serviço passa a rodar com MySQL, com arquivo e em memória — e nunca soube que MySQL existe.

Regra — o teste decisivo é a direção do `#include`

Em que direção o código-fonte aponta em tempo de compilação? Se o módulo de negócio faz `#include "mysql.hpp"`, não houve inversão nenhuma — por mais interfaces que existam no meio. A verificação é mecânica: `grep -r '#include' src/entidades/`. Se aparecer um cabeçalho de SQL, a arquitetura foi violada.

As formas de injeção diferem pelo momento em que a dependência é fixada: construtor para dependência obrigatória (o estado inválido fica inexprimível); parâmetro para o que varia a cada chamada; *template* quando a escolha é conhecida em compilação e o custo deve ser zero. Nesta última, o *concept* **é** a abstração:

```
1  template <RelogioEstatico R>
2  class AuditoriaEstatica {
3      R relogio_;                          // por VALOR: sem indireção, sem heap
4  };
5  // sizeof(AuditoriaEstatica<RelogioRapido>) = 1 byte
```

Listagem 171: Injeção estática: a dependência continua invertida, e não custa nada.

Se o serviço não pode construir o repositório, alguém precisa construir — e esse alguém é o **composition root**: um ponto único que pode conhecer tudo porque não tem regra de negócio nenhuma. O subprojeto o materializa em duas funções, `montar_producao()` e `montar_teste()`, e a consequência é que *todo `new` do programa está nessas duas funções.*

Armadilha — *service locator* não é injeção — é pior que o acoplamento direto

```
1  void fazer() {
2      auto* log = Locator::obter<Log>();   // ← o objeto vai BUSCAR
3  }
```

As dependências ficam **invisíveis na assinatura**, o teste vira estado global mutável, e a falha é em tempo de execução — ao passo que a injeção por construtor falha em compilação. A regra que separa os dois: na injeção, a dependência é **empurrada** para o objeto; no *locator*, o objeto vai **buscar**. Se a classe busca, ela conhece o buscador, e não houve inversão.

Em escala, DIP vira a *regra da dependência*: o código-fonte só pode apontar para dentro, na direção das políticas de mais alto nível (Tabela 65). E o ponto que dá nome ao princípio é que o fluxo de *controle* vai de dentro para fora — o caso de uso chama o banco — enquanto o fluxo de *código* vai de fora para dentro. São opostos.

Tabela 65: A regra da dependência: cada círculo só inclui os de dentro.

Círculo	Pode incluir	Volatilidade
entidades	**nada**	mínima
casos de uso	entidades	baixa
adaptadores	casos de uso + entidades	média
frameworks	tudo	máxima

Regra — inverta o volátil; não inverta o estável

DIP paga-se com uma interface, uma indireção e um arquivo por dependência. Não vale quando a dependência é *estável* — ninguém abstrai `std::string`, `std::vector` ou `fmt` — nem quando o programa é pequeno e sem testes automatizados, nem quando existe um implementador e não haverá outro. A pergunta que decide: *esta dependência é volátil?* Inverta banco, rede, relógio, sistema de arquivos e aleatoriedade. Não inverta o que é estável há trinta anos.

V.2 Padrões de projeto

O subprojeto `src/design-of-pattern/` implementa **24 padrões**: os 23 do catálogo GoF [7] e mais o *Null Object*, que não está no livro e aparece em quase todo código real. Cada padrão mora no seu diretório, tem `meson.build` próprio e produz um executável — são 24 alvos, todos construídos e todos executáveis a partir de `dist/bin/`. Nenhum está comentado, e os números citados adiante foram lidos da saída desses binários, não de livro.

Antes de tudo, uma definição, porque a palavra costuma ser usada de forma descuidada. Um **padrão de projeto** não é uma biblioteca, não é uma classe-base para herdar e não é uma receita a seguir: é o *nome* de uma solução que foi redescoberta de forma independente tantas vezes que valeu a pena catalogá-la. O valor está tanto na solução quanto no nome — dizer "isto é um *Strategy*" comunica em duas palavras o que levaria um parágrafo, e é por isso que o catálogo sobreviveu a três revisões da linguagem.

Por que — por que estudar 24 padrões se metade deles "sumiu" no C++ moderno?

Porque o que "sumiu" foi a *implementação*, não o *problema*. O *Strategy* continua existindo quando um `std::function` substitui a hierarquia — só que agora o padrão tem uma linha em vez de quatro classes. E saber que aquilo é um *Strategy* é o que permite reconhecer, seis meses depois, que a variação é do algoritmo e não do tipo. Um padrão é vocabulário antes de ser código: o dia em que o `std::function` não servir — porque o algoritmo precisa de estado, de nome, de ciclo de vida — é o padrão clássico que volta, e ele precisa estar aprendido.

V.2.1 O eixo que decide quase tudo: o *expression problem*

Vários padrões do catálogo são respostas à *mesma* pergunta, formulada de maneiras diferentes. Reconhecer a pergunta economiza metade do estudo.

Tabela 66: Os 24 padrões implementados em `src/design-of-pattern/`, e o que o executável de cada um demonstra. A coluna da direita reproduz o que o `README.md` do subprojeto registra como objetivo de cada demo.

Padrão	**Categoria**	**O que o demo mostra**
Factory Method	criacional	criação polimórfica por nome
Abstract Factory	criacional	famílias compatíveis; misturar temas fica inexprimível
Builder	criacional	`construir() &&` impede reusar o *builder*
Prototype	criacional	clone profundo medido; CRTP elimina o *boilerplate*
Singleton	criacional	*Meyers singleton* e o custo dele
Adapter	estrutural	API C legada → `expected` + RAII
Bridge	estrutural	$3 \times 3 = 9$ classes vira $3 + 3$; *pimpl* é um *Bridge* degenerado
Composite	estrutural	`adicionar` só no composto, por LSP/ISP
Decorator	estrutural	empilhamento de comportamento
Facade	estrutural	6 subsistemas → 1 chamada; o subsistema segue acessível
Flyweight	estrutural	**10,4× menos memória**, medido
Proxy	estrutural	virtual/proteção/*smart*; `std::shared_ptr` é um deles
Chain of Responsibility	comportamental	*pipeline* de *handlers*
Command	comportamental	*undo* por operação inversa
Interpreter	comportamental	AST com hierarquia virtual
Iterator	comportamental	iteradores próprios, direto e reverso
Mediator	comportamental	teia $N \times (N-1)$ vira estrela N
Memento	comportamental	opacidade real, via `friend`
Observer	comportamental	notificação $1 \to N$
State	comportamental	máquina de estados de TCP
Strategy	comportamental	algoritmos intercambiáveis
Template Method	comportamental	obrigatório *vs. hook vs.* fixo
Visitor	comportamental	despacho duplo, e o `variant` fazendo melhor
Null Object	fora do catálogo	eliminar verificações de `nullptr` espalhadas

Considere um programa que manipula uma coleção de **tipos** (nós de uma árvore sintática, formas geométricas, widgets) sobre os quais se executam **operações** (imprimir, avaliar, serializar, calcular área). Há duas direções em que esse programa pode crescer, e o *expression problem* é a observação de que **as duas ferramentas disponíveis são boas em direções opostas**.

Por que — o que exatamente é "aberto" e "fechado" aqui

Com uma **hierarquia virtual**, acrescentar um *tipo* é criar um arquivo: a classe nova herda da base, implementa os métodos, e nenhum código existente é tocado. Mas acrescentar uma *operação* é editar a base e *todas* as derivadas — e recompilar tudo.
Com um `std::variant`, é o inverso exato: acrescentar uma *operação* é escrever uma função livre nova, sem tocar em nada; mas acrescentar um *tipo* à lista do `variant` obriga a revisitar todos os `visit` do programa. A diferença decisiva é que, no segundo caso, **o compilador aponta cada lugar** — um `visit` que não cobre todas as alternativas não compila. Na hierarquia, esquecer de sobrescrever um método novo com corpo padrão compila e falha em execução.

Tabela 67: O *expression problem*, como o `README.md` de `design-of-pattern` o formula: qual eixo do seu problema cresce?

Cresce mais	Use	Padrões que realizam
os **tipos**	hierarquia virtual	*Visitor*, *Composite*, *Abstract Factory*
as **operações**	`variant` + `visit`	— (função livre, exaustividade verificada)

Regra — a pergunta que substitui metade da discussão sobre padrões

O que vai variar mais no meu problema: os tipos ou as operações? Se a resposta for "os tipos, e eles vêm de fora — de *plugins*, de bibliotecas que não controlo", a hierarquia virtual é a resposta certa e continua sendo. Se for "os tipos são poucos e conhecidos, mas toda semana alguém pede uma operação nova", o `std::variant` é a resposta certa e ainda dá exaustividade de graça. Nenhum dos dois é melhor em abstrato.

V.2.2 Os cinco criacionais

Os padrões criacionais respondem a uma pergunta só: *quem decide qual classe concreta será instanciada, e quando?* A resposta ingênua — o cliente, com `new` — acopla o cliente a todos os tipos concretos e é o que os cinco evitam, cada um por um motivo diferente.

***Factory Method*: criar sem nomear a classe.** O cliente pede `ShapeFactory::create("circle", {5.0})` e recebe um `unique_ptr<Shape>`. O `switch` sobre a *string* existe, mas existe em **um** lugar — e esse lugar é o único do programa que conhece os nomes concretos. É a mesma ideia do *composition root* da Seção V.1, aplicada a uma família só.

***Abstract Factory*: a consistência da família virando tipo.** A diferença em relação ao anterior costuma escapar, e o demo a torna visível numa linha:

```
// Com Factory simples, uma por produto:
auto b = criar_botao("escuro");
auto c = criar_campo("claro");    // COMPILA -- e a UI fica inconsistente

// Com Abstract Factory:
const FabricaDeWidgets& f = escolher_tema();
```

```
7  auto b = f.criar_botao("Salvar");
8  auto c = f.criar_campo();          // MESMA familia, garantido pelo TIPO
```

Listagem 172: Resumo de `src/design-of-pattern/abstract_factory/main.cpp`: o que a *Factory* simples não impede.

A família é escolhida uma vez; depois disso, misturar temas não é "desencorajado" — é **inexprimível**. E a função `renderizar_formulario` foi escrita uma vez, contra `FabricaDeWidgets`: acrescentar um terceiro tema não muda uma linha dela.

***Builder*: construção com validação no fim.** O problema aparece antes da solução. Sem *Builder*, há três saídas, e as três são ruins:

```
1  (a) construtor gigante:
2        Relatorio r{"Vendas","csv",true,false,true,100,",","utf-8",2};
3      -> trocar dois 'bool' de lugar COMPILA e muda o comportamento
4
5  (b) N sobrecargas:
6        Relatorio(t, f);  Relatorio(t, f, cabecalho);  ...
7      -> 2^n combinacoes para n campos opcionais
8
9  (c) setters publicos:
10       Relatorio r; r.set_titulo (...) ; r.set_formato(...);
11     -> entre as chamadas, o objeto esta INVALIDO e isso e visivel
```

Listagem 173: Saída de `dist/bin/builder`, seção 1: as três saídas e seus defeitos.

O *Builder* resolve as três: os campos obrigatórios estão no construtor do *builder* (não há como esquecê-los, porque o *builder* não existe sem eles), a ordem dos opcionais é livre, e cada passo tem *nome* — `.com_rodape()` é mais legível que um `true` na quarta posição. E a validação acontece uma vez, em `construir()`, quando todos os campos já foram definidos:

```
1  formato valido          -> ok: 'X' [html] cab=sim rod=nao sep=',' dec=2
2  formato invalido        -> recusado: formato deve ser csv, json ou html
3  casas_decimais = 99  -> recusado: casas_decimais fora de 0..10
4  csv sem separador       -> recusado: csv exige um separador
```

Listagem 174: Saída de `dist/bin/builder`, seção 3: regras que dependem de *dois* campos.

A última linha é a que justifica o padrão: um *setter* público jamais conseguiria validar "csv exige separador", porque no momento em que ele roda não se sabe se o outro campo já foi preenchido.

Armadilha — o `&&` nos métodos do *builder* não é enfeite

Todos os métodos do *builder* do subprojeto são qualificados com `&&` — inclusive `construir()`:

```
1  Construtor&& limite(int n) &&  { ...; return std::move(*this); }
2  Relatorio    construir()   &&;
```

O qualificador de referência (Seção II.3) faz com que os métodos só possam ser chamados sobre um *rvalue*. A consequência prática é que o *builder* **não pode ser reusado depois de construir**: o objeto foi movido para dentro do produto, e chamar outro método nele seria ler um objeto "movido de". O compilador passa a recusar exatamente o erro que a versão sem `&&` permitiria silenciosamente.

***Prototype*: copiar sem conhecer o tipo concreto.** A assinatura é `virtual std::unique_ptr<Forma> clonar() const`, e o detalhe que o demo mede é que a cópia precisa ser **profunda**: um nó composto que

copie apenas os ponteiros produz duas árvores compartilhando filhos. No caso do subprojeto, o código nem compilaria — `std::unique_ptr` não é copiável, e é o próprio sistema de tipos denunciando a cópia rasa.

Como todo `clonar()` é literalmente a mesma linha, uma camada CRTP (Seção IV.1) escreve por todas:

```
template <typename Derivada, typename Base = Forma>
class Clonavel : public Base {
    std::unique_ptr<Forma> clonar() const override {
        return std::make_unique<Derivada>(static_cast<const Derivada&>(*this));
    }
};
class Quadrado : public Clonavel<Quadrado> { /* ... */ };
```

Listagem 175: De `src/design-of-pattern/prototype/`: o CRTP eliminando o *boilerplate*. Nenhuma das classes concretas escreveu `clonar()`.

Singleton. Discutido adiante (Seção V.2.6), porque nele a "versão moderna" resolveu um problema e deixou o outro intacto.

Regra — o *Prototype* ganha da *Factory* num caso específico

Uma *factory* só sabe criar o que foi **compilado** nela. Um registro de protótipos guarda *instâncias-modelo* que podem ter sido configuradas em tempo de execução — carregadas de um arquivo, montadas pelo usuário — e clonadas sob demanda. É esse uso que dá nome ao padrão, e é a única situação em que ele vence claramente a *factory*.

V.2.3 Os sete estruturais

Os estruturais respondem a *como compor objetos em estruturas maiores sem que o cliente precise saber da composição*. Quatro deles — *Adapter*, *Decorator*, *Proxy* e *Facade* — têm praticamente o mesmo diagrama de classes, e a diferença entre eles é de *intenção*; a Tabela 71 adiante separa os quatro de uma vez.

***Adapter*: quando a interface é a errada.** O demo adapta uma biblioteca C legada, que não pode ser modificada, a uma interface moderna. São quatro incompatibilidades simultâneas, e o adaptador resolve as quatro:

```
codigo de erro  int      -> std::expected
int  magico de nivel     -> enum class Nivel
(char*, size_t)          -> std::string_view
create/destroy manual -> RAII
```

Listagem 176: Saída de `dist/bin/adapter`, seção 1: o que o adaptador traduz.

O ganho fica visível na terceira seção do demo: o `LEGACY_BUFFER_FULL` — um `int` de valor 2 — vira `std::unexpected{ErroDeLog::  BufferCheio}`, e o cliente trata com `if (r)` sem nunca ver o código numérico (Seção II.8). A função de negócio recebe `adapter::Log&` e nunca vai saber que existe uma biblioteca C por baixo.

***Bridge*: duas hierarquias que variam sozinhas.** É o padrão cujo benefício é puramente aritmético, e o demo faz a conta:

```
Sem Bridge, com heranca nos dois eixos:
    3 formas x 3 renderizadores = 9 classes
    acrescentar 1 renderizador  = +3 classes
    acrescentar 1 forma         = +3 classes

Com Bridge:
    3 formas + 3 renderizadores = 6 classes
    acrescentar 1 renderizador  = +1 classe
    acrescentar 1 forma         = +1 classe

A conta piora rapido: 5x5 seria 25 contra 10.
```

Listagem 177: Saída de `dist/bin/bridge`, seção 1: a explosão evitada.

O produto vira soma. E o demo mostra que os dois lados variam de verdade: `redimensionar()` é do lado da *abstração* e nenhum renderizador soube dele; a troca de SVG para ASCII é do lado da *implementação* e nenhuma forma soube dela.

Regra — *Bridge vs. Strategy*: a mesma forma, escalas diferentes

Os dois têm diagrama idêntico — um objeto guarda uma referência a uma interface e delega. A diferença: no *Strategy*, o outro lado é **um método** (um algoritmo trocável); no *Bridge*, o outro lado é **uma hierarquia inteira** (uma família de operações que forma um *backend*). E há um caso degenerado famoso: o idioma *pimpl* é um *Bridge* com exatamente uma implementação, usado não para variar mas para esconder — e para cortar tempo de compilação.

***Composite*: a árvore que o cliente trata como folha.** O demo monta uma árvore de diretórios e chama a mesma função com um arquivo e com a raiz inteira. A recursão está encapsulada em `Pasta::tamanho()`, e a remoção de uma subárvore não tem um único `delete`: o `std::unique_ptr` destrói tudo em cascata.

A decisão de design que o padrão força é onde colocar `adicionar()`, e o subprojeto escolhe o lado menos "uniforme" de propósito:

Tabela 68: As duas opções que o próprio GoF apresenta para o *Composite*, e por que o subprojeto escolheu a segunda.

`adicionar()` fica...	Uniformidade	Custo
na interface `No`, para todos	máxima	`Arquivo::adicionar()` viola LSP ou falha em silêncio
só no composto `Pasta`	parcial	o cliente precisa do tipo `Pasta` para compor

O argumento decisivo é prático: quase toda composição acontece na *construção* da árvore, onde os tipos concretos já são conhecidos. A uniformidade que importa é a das operações de *leitura* — e essa a segunda opção preserva inteira, sem obrigar `Arquivo` a prometer algo que não sabe cumprir.

Armadilha — um bug real encontrado escrevendo o *Composite*

A primeira versão do demo declarava `virtual void imprimir(int nivel = 0) const = 0;` — e `raiz->imprimir()` **não compilou**. O motivo: o `override` em `Pasta` não repetiu o argumento padrão, e argumento padrão é escolhido pelo tipo **estático**, não pelo dinâmico. A correção é o idioma NVI (Seção II.7):

```
1 void imprimir() const { imprimir_em(0); }   // NAO-virtual, com o default
2 virtual void imprimir_em(int) const = 0;    // virtual, sem default
```

Aqui o NVI não é questão de estilo: é o que compila. Vale registrar que este bug foi encontrado *porque*

o alvo foi construído — um padrão bonito num diretório que ninguém compila não ensina nada.

***Decorator*: composição em vez de herança.** O problema — acrescentar compressão, criptografia e *logging* a um *stream* em *qualquer combinação* — resolvido por herança exigiria uma classe por combinação. Com decoradores, cada um envolve o anterior:

```
auto stream = std::make_unique<LoggingDecorator>(
    std::make_unique<CompressionDecorator>(
        std::make_unique<EncryptionDecorator>(
            std::make_unique<FileStream>("arquivo.bin")
        )
    )
);
stream->write("dados confidenciais"); // log -> comprime -> criptografa -> arquivo
```

Listagem 178: Composição em tempo de execução: a ordem das camadas é a ordem das chamadas.

***Facade*: a ordem, escrita num lugar só.** O demo simula um sistema de *build* com seis subsistemas — resolvedor de dependências, compilador, cache, *linker*, testes e empacotador. Sem a fachada, o cliente escreve quinze linhas e precisa saber que `resolver` vem antes de `compilar`, que a saída de `compilar` alimenta `linkar`, e quais são os *defaults* razoáveis de cada um. Esse conhecimento fica duplicado em cada cliente. Com a fachada, é uma chamada.

O ponto que distingue a *Facade* de uma camada de abstração é que **o subsistema continua acessível**: quem precisa de um caso incomum chama os subsistemas diretamente. A fachada otimiza o *caso comum*, não todos os casos — e é por isso que ela não "esconde" nada.

Armadilha — a *Facade* que cresce tem nome: *God Object*

Uma fachada que vira a *única* porta de entrada, com quarenta métodos e estado próprio, deixou de simplificar e passou a acoplar. O sinal de alarme é objetivo: **ela acumular estado que os subsistemas não têm**. É a mesma patologia da *god class* do SRP (Seção V.1), e a mesma que ameaça o *Mediator*.

***Flyweight*: o único padrão do catálogo cujo ganho se mede.** A motivação é puramente de memória, e a divisão que define o padrão é entre estado **intrínseco** (compartilhável e imutável: o desenho da letra `'a'`, a fonte, o tamanho, o *bitmap* rasterizado) e **extrínseco** (único por instância: a posição e a cor). Um documento de cem mil caracteres precisa de cem mil *posições*, mas de cerca de cem glifos distintos.

```
documento de 12200 caracteres:

COM flyweight:
  glifos distintos      :          21
  memoria dos glifos    :        5019 B
  memoria do documento  :      292800 B  (12200 x 24 B)
  TOTAL                 :      297819 B

SEM flyweight:
  objetos completos     :       12200
  TOTAL                 :     3111000 B

economia: 10.4x menos memoria (3111000 B -> 297819 B)

sizeof(CaractereNaTela) =  24 B  (ponteiro + x,y + cor)
sizeof(CaractereGordo)  =  96 B  + as duas strings na heap
```

Listagem 179: Saída de `dist/bin/flyweight`, seção 3 — a economia, medida no próprio binário deste repositório.

A assinatura do glifo é o que torna o compartilhamento possível: `std::string desenhar(int x, int y, const std::string& cor) const` — `x`, `y` e `cor` são *parâmetros*, não membros. Um objeto que guardasse a posição não poderia ser reusado por ninguém.

Armadilha — cinco condições sem as quais o *Flyweight* não funciona

O demo lista as cinco, e a segunda é a menos óbvia. **(1)** O objeto compartilhado tem que ser **imutável** — por isso a fábrica devolve `const Glifo&` e o glifo não tem *setters*. **(2)** O contêiner da fábrica não pode **invalidar referências**: o demo usa `std::deque` e não `std::vector`, porque o `vector` realoca ao crescer e transforma em pendurado todo ponteiro já entregue (Seção III.1). **(3)** A fábrica precisa sobreviver a todos os clientes. **(4)** Em código concorrente, a *inserção* precisa de sincronização; a leitura dos glifos já criados é segura por serem imutáveis. **(5)** Não compensa se o estado intrínseco for pequeno — um ponteiro custa 8 bytes, e compartilhar um objeto de 4 bytes *gasta* memória. A conta é: compensa se `sizeof(intrínseco)` > `sizeof(ptr)` e houver repetição real.

O inverso interessante mora na biblioteca padrão: o *SSO* da `std::string` — medido em `src/strings/` — evita a indireção justamente para objetos *pequenos* — pelo motivo exato da quinta condição.

***Proxy*: mesma interface, acesso controlado.** O demo apresenta três variedades — *proxy* virtual (adia a criação cara, usando `mutable`), *proxy* de proteção (verifica permissão antes de repassar) e *smart proxy* (registra, mede, faz *cache*). E a pergunta que o separa do *Decorator* é única: *o objeto real é sempre chamado?* Se sim, é *Decorator*; se ele pode **não** repassar — negar, adiar, servir do *cache* — é *Proxy*. No demo, o *proxy* de proteção negou a chamada, e o *proxy* de instrumentação empilhado nem chegou a contá-la.

Vale notar que os *smart pointers* da biblioteca padrão são *proxies* no sentido estrito, e o `std::shared_ptr` é o exemplo mais visitado do catálogo sem que ninguém o chame assim.

V.2.4 Os onze comportamentais

Os comportamentais tratam de *como os objetos se comunicam*. Quatro deles já foram vistos de outro ângulo neste livro, e vale explicitar as ligações antes dos novos.

***Strategy* é OCP por injeção.** Cada algoritmo é um objeto trocável em tempo de execução, e o contexto que o usa nunca é editado. É literalmente o Demo 2 da Seção V.1 com outro nome — e a coincidência não é acidental: o princípio descreve a propriedade, o padrão descreve a realização.

***Command* transforma ação em objeto.** O par `execute()`/`undo()` é o que permite pilha de desfazer, fila de tarefas e *log* de operações. É a mesma reificação que a Seção VI.1 usa para enfileirar trabalho numa *thread pool* — com a diferença de que lá o objeto é um `std::function`.

***State* substitui o `switch` sobre estado.** Cada estado é um objeto, e a transição é o próprio estado trocando o ponteiro do contexto — o exemplo do subprojeto é uma conexão TCP passando de `Listen` para

`Established`. O ganho sobre a máquina de estados com `enum` é o mesmo do OCP: acrescentar um estado é acrescentar um arquivo.

***Chain of Responsibility* e *Observer*.** O primeiro encadeia *handlers* — autenticação, limite de taxa, validação, negócio — e cada elo decide se processa ou repassa; o segundo notifica N assinantes a partir de um evento. São os dois padrões mais presentes em código de rede, e o Seção VII.2 os reencontra sem os nomear.

***Template Method*: o que se protege é a *ordem*.** O esqueleto do algoritmo mora na base, num método **não virtual**, e os derivados preenchem os buracos. A distinção de três níveis é o que faz o padrão funcionar:

Tabela 69: Os três tipos de passo do *Template Method*, como o demo `dist/bin/template_method` os classifica.

Tipo	Declaração	O derivado...
obrigatório	`virtual ... = 0`	**tem que** implementar
hook	`virtual ... { padrão }`	**pode** sobrescrever
fixo	não virtual, `private`	não pode mudar

O demo quantifica o que cada variação precisou escrever: o `PipelineCsv` implementou **2** métodos (só os obrigatórios), o `PipelineJson` **3** (mais o *hook* de validação) e o `PipelineBanco` **6** (todos os *hooks*). O passo `ordenar()` é `private` e não virtual: ninguém o alcança. E `executar()` é não virtual pelo mesmo motivo — se um derivado pudesse sobrescrevê-lo, a garantia de ordem, que é o padrão inteiro, evaporaria. É NVI outra vez (Seção II.7).

***Mediator*: a teia virando estrela.** O demo é um formulário com seis componentes e regras cruzadas ("Enviar" só ativa com e-mail válido *e* termos aceitos; marcar "empresa" revela o CNPJ e o torna obrigatório). A conta que ele imprime é a razão de ser do padrão:

```
SEM mediador:
  email   -> botao, status
  termos  -> botao, status
  empresa -> cnpj, botao, status
  cnpj    -> botao, status
  6 componentes -> ate 30 referencias
  e acrescentar "aceito newsletter" toca varios deles

COM mediador:
  cada componente -> o mediador. So isso.
  6 referencias , e a regra num lugar so.
```

Listagem 180: Saída de `dist/bin/mediator`, seção 1: quem conhece quem.

O ganho real não é a contagem: é que os componentes ficam **reutilizáveis** — um campo de texto que não conhece nenhum outro campo pode ir para qualquer formulário.

Armadilha — o *Mediator* carrega o defeito na própria definição

Ao centralizar a interação, ele acumula toda a lógica que estava distribuída — e vira um *God Object*. Os sinais: `notificar` virando um `switch` de duzentas linhas; o mediador ganhando estado que não é de ninguém; acrescentar um componente exigindo entender o mediador inteiro. As três saídas que o demo registra: quebrar em mediadores menores por área; trocar o `switch` por um `std::map<std::string, std::function<void()>>`; ou extrair as *regras* para objetos próprios, deixando o mediador só roteando.

***Memento*: salvar estado sem furar o encapsulamento.** São três papéis — o **originador** (que cria e restaura), o **memento** (opaco) e o **zelador** (que guarda e nunca inspeciona). Em C++, a opacidade não é convenção: é o sistema de tipos.

```
1 class Editor {
2     class Estado {
3         std::string conteudo_;   // privado
4         std::size_t cursor_;     // privado
5         Estado(...);             // construtor PRIVADO
6         friend class Editor;     // so ele
7     };
8 };
```

Listagem 181: De `src/design-of-pattern/memento/`: a opacidade construída com classe aninhada privada e `friend`.

O que o zelador *não* consegue fazer é o valor do padrão: ler `conteudo_`, escrever em `cursor_` ou fabricar um `Estado` do nada — as três tentativas são erros de compilação. O que ele pode, e é tudo de que precisa: guardar num `vector`, devolver ao editor e ler os metadados públicos (rótulo, tamanho). O ganho concreto: o originador pode **trocar a representação interna** sem que o histórico sequer recompile.

Regra — *Memento vs. Command*: a operação tem inversa barata?

Se sim, use *Command* e guarde a *operação* — custo $O(\Delta)$. Inserir texto tem inversa óbvia (apagar). Se não — um filtro de imagem destrutivo —, use *Memento* e guarde o *estado* inteiro, custo $O(\text{estado})$. E os dois se combinam: um *Command* que não consegue se inverter guarda um *Memento* do estado anterior, que é exatamente como editores de imagem tratam filtros destrutivos. A distinção sutil é com o *Prototype*: o clone é um objeto **normal e usável**; o memento é **opaco** e só serve para restaurar.

***Visitor*: despacho duplo, e o preço dele.** O *Visitor* resolve o problema de acrescentar operações a uma hierarquia fechada sem editar as classes: cada nó tem `aceitar(Visitante&)`, que chama de volta o método específico do visitante — daí "duplo despacho". Ele funciona, e o demo o implementa por inteiro; mas o mesmo demo mede o que ele custa:

Tabela 70: *Visitor* clássico contra `variant` + `visit`, como o demo `dist/bin/visitor` compara na seção 6.

Critério	***Visitor***	`variant` + `visit`
operações novas	fácil	fácil
tipos novos	difícil	difícil
exaustividade verificada	não	**sim**
retorno com valor	não	**sim**
linhas de infraestrutura	≈ 40	≈ 3
alocação por nó	sempre	opcional
tipos de bibliotecas alheias	sim	não

Três defeitos do *Visitor* clássico desaparecem com `visit`: a operação passa a **devolver valor** (nada de acumular resultado num membro do visitante), os nós deixam de precisar do método `aceitar`, e o acoplamento circular entre nós e visitantes some. E há um ganho que o clássico nunca teve: `visit` faz despacho **múltiplo**, sobre N `variant` de uma vez — estender o *Visitor* para despacho triplo seria uma explosão de métodos.

V.2.5 As confusões que sobram

Quase todo mal-entendido sobre padrões cai em um de oito pares. O `README.md` do subprojeto os tabula, e cada demo dedica uma seção a separar o seu padrão dos vizinhos.

Tabela 71: Os oito pares que mais confundem, e a pergunta que separa cada um.

Confusão	O que separa
Adapter × *Decorator* × *Proxy*	*Adapter* **muda** a interface; *Decorator* **acrescenta** comportamento; *Proxy* **controla** o acesso (pode não repassar)
Bridge × *Strategy*	o outro lado é uma **hierarquia** (*Bridge*) ou **um método** (*Strategy*)
Facade × *Mediator*	a *Facade* **não acrescenta** lógica; o *Mediator* **coordena**
Template Method × *Strategy*	o que se protege é a **ordem** dos passos ou o **algoritmo** inteiro?
Mediator × *Observer*	existe uma **regra** que depende de vários participantes?
Memento × *Command*	a operação tem **inversa barata**?
Memento × *Prototype*	o clone é **usável**; o memento é **opaco**
Visitor × `variant`	a hierarquia é **aberta** (*Visitor*) ou **fechada** (`variant`)?

V.2.6 A versão moderna, padrão a padrão

Vários padrões do GoF descrevem soluções para limitações de linguagens dos anos 1990 — linguagens sem *closures*, sem uniões com tipo, sem inicialização por nome. Em C++20, parte deles tem realização mais simples. Por isso os treze padrões acrescentados na última fase do subprojeto trazem, além do padrão clássico, uma seção **"versão moderna"** no próprio executável — e é dela que sai a Tabela 72.

O ponto importante, e que a tabela sozinha não transmite: em **nenhum** caso a versão moderna substitui a clássica sem trocar algo. A última coluna é a que se deve ler primeiro.

Tabela 72: A versão clássica, a moderna e o que se perde ao trocar. As perdas foram transcritas das seções "versão moderna" dos executáveis.

Padrão	Clássica	Moderna	O que a moderna PERDE
Strategy	hierarquia virtual	`std::function` ou *concept*	nome, estado e ciclo de vida do algoritmo
Command	classe com `execute()`	*lambda* com *init-capture*	o `undo()` pareado, que a *lambda* não carrega
Visitor	duplo despacho com `accept()`	`std::visit` sobre `variant`	tipos vindos de bibliotecas alheias
Iterator	interface com `hasNext()`	iterador da STL, ou *view*	— (aqui a moderna ganha em tudo)
Singleton	ponteiro estático + *mutex*	*Meyers singleton*	nada; mas o problema real continua (adiante)
Builder	*builder* encadeado	*designated initializers*	validação, campos obrigatórios, ordem livre, etapas
Prototype	`clonar()` virtual	`variant` copiável, ou CRTP	a hierarquia deixa de ser aberta
Abstract Factory	fábrica virtual por família	`struct` de `std::function`	a garantia de consistência da família
Adapter	classe adaptadora	*lambda* adaptadora	RAII, múltiplos métodos, estado próprio
Bridge	duas hierarquias	`struct` de `std::function`	uma indireção por primitiva, em laço quente
Composite	`No` virtual + filhos	`variant` recursivo	tipos de nó novos vindos de fora
Template Method	*hooks* virtuais	`struct` de *callbacks*	a obrigatoriedade verificada em compilação
Flyweight	fábrica + `const T&`	`shared_ptr<const T>`	a **garantia de unicidade**

Builder vs. designated initializers. O C++20 permite `gerar({.titulo = "Vendas Q3", .formato = "json", .casas_decimais = 4})`, e isso dá três das quatro coisas que se queria do *Builder*: cada campo tem nome no ponto de uso, os omitidos usam o *default* do `struct`, e o código extra é zero. O que *não* dá — e é o que mantém o *Builder* vivo — são quatro coisas: não há **validação** (o agregado aceita qualquer

combinação), não há **campos obrigatórios**, a **ordem não é livre** (em C++, os designadores precisam seguir a ordem de declaração dos membros; em C, não), e não há **construção em etapas** com lógica entre elas. A regra que sai disso é simples: agregado sem invariantes, use *designated init*; produto com invariantes ou campos privados, use *Builder*.

***Template Method vs.* `struct` de *callbacks*.** A versão moderna troca a hierarquia por um agregado de `std::function`, e ganha o direito de montar os passos em tempo de execução, vindos de lugares diferentes. Perde uma garantia que era gratuita: nada obriga a preencher o *callback* obrigatório. Um `std::function` vazio **compila** e lança `std::bad_function_call` em execução — ao passo que o `virtual ... = 0` tornava o esquecimento um erro de compilação. Perde também o estado compartilhado entre os passos, que na versão clássica eram membros protegidos da base e agora precisam virar capturas.

***Bridge* e o custo do *type erasure*.** Substituir a hierarquia de renderizadores por um `struct` de `std::function` permite montar um *backend* em execução, misturando primitivas de origens diferentes. Mas cada `std::function` é um *type erasure*, e cada chamada custa uma indireção — que num laço de renderização é caminho quente. A medição de `src/lambdas/` registra **3,5**× de diferença entre a chamada direta e a passagem por `std::function` (Seção IV.4). Para um *Bridge* de renderização, a hierarquia virtual costuma ganhar: a implementação é escolhida *uma* vez na inicialização, e depois são milhões de chamadas — uma vtable / vptr é mais barata que N `std::function`. E se a escolha for conhecida em compilação, um *template* elimina os dois custos.

***Flyweight* e o `std::shared_ptr`.** Trocar a fábrica por `shared_ptr<const Glifo>` ganha contagem de referência automática — os glifos sem uso podem ser liberados. Perde a única coisa que o padrão promete: **nada impede criar dois `shared_ptr` para glifos idênticos**. É a fábrica que garante que "mesmo estado intrínseco" implica "mesmo objeto"; o ponteiro compartilhado, sozinho, não garante nada.

***Singleton*: a versão moderna resolveu o problema errado.** O *Meyers singleton* é *thread-safe* por garantia do padrão desde o C++11 — a inicialização de um `static` local é sincronizada pelo compilador:

```
1  class Logger {
2  public:
3      static Logger& instance() {
4          static Logger inst; // thread-safe por garantia do C++11 (6.7)
5          return inst;
6      }
7
8      Logger(const Logger&)            = delete;
9      Logger& operator=(const Logger&) = delete;
10
11 private:
12     Logger() = default;
13 };
```

Listagem 182: O *Meyers singleton*: *thread-safe* por garantia do padrão.

Armadilha — o problema do *Singleton* nunca foi a inicialização

Resolver a inicialização preguiçosa e a segurança entre *threads* não resolve o que realmente pesa: estado global que persiste entre testes, acoplamento oculto (qualquer código pode chamar `Logger::instance()`) e violação direta do DIP — a dependência não aparece na assinatura. É a mesma crítica que se faz ao *service locator* (Seção V.1). O uso legítimo existe — *logger* de processo,

configuração, registro de *plugins* (Seção V.4) — mas a pergunta certa continua sendo se a injeção por construtor não resolveria melhor.

E o *Null Object* é o complemento que torna o *Singleton* dispensável em teste: uma implementação que não faz nada, injetada no lugar da real. O ganho é eliminar as verificações de `nullptr` do cliente — o membro passa a ser uma *referência*, e referência nunca é nula. O demo mostra os dois lados da moeda: com um `NullLogger`, um erro de validação é *silenciado* — e ninguém percebe, porque não há verificação nenhuma.

Regra — `optional` para dados; *Null Object* para comportamento

`std::optional` obriga o chamador a verificar — e é isso que se quer para um *valor* que pode não existir (Seção III.4). O *Null Object* faz o contrário: garante que a chamada seja segura para que ninguém precise verificar — e é isso que se quer para um *serviço* opcional. Usar um no lugar do outro produz ou verificações desnecessárias ou ausência silenciosa de comportamento.

V.2.7 *Interpreter*: onde ele ganha e onde perde

O *Interpreter* do subprojeto implementa a mesma gramática de expressões aritméticas que o `src/parser/` do Seção X.3 resolve com Flex e Bison, o que permite a comparação direta (Tabela 73). A precedência dos operadores sai naturalmente da estrutura das chamadas: `expr` chama `term`, que chama `factor`.

Tabela 73: *Interpreter* escrito à mão contra gerador de *parser*.

	***Interpreter* GoF**	**Flex/Bison**
dependências	zero — C++ puro	`flex` + `bison` instalados
AST	objeto reutilizável	precisa ser implementada à parte
testabilidade	cada nó isoladamente	o *parser* inteiro
gramáticas grandes	trabalhoso	ideal
recuperação de erros	manual	robusta
quando usar	DSL embutida, < 20 regras	linguagem completa

Acrescentar um operador novo ao *Interpreter* são três passos — classe nova, função auxiliar, um caso no *parser* — e nenhum arquivo existente quebra. É OCP em ação, e é também exatamente o eixo em que uma AST baseada em `std::variant` *perderia*: acrescentar um *tipo* de nó a um `variant` recompila todos os `visit` (Seção III.4). É o *expression problem* da Seção V.2.1 aparecendo, pela última vez, com nome e sobrenome.

V.3 Entity Component System

O subprojeto `src/ecs/` implementa do zero o padrão arquitetural dominante nas *engines* de jogo — Unity, Unreal, Bevy. Ele entra na trilha logo depois dos padrões porque é o **contraponto direto** ao Seção II.7: composição em tempo de execução no lugar de herança.

V.3.1 O problema

Uma hierarquia de herança obriga a decidir, em tempo de compilação, quais combinações de capacidades existem — e o número de combinações cresce exponencialmente:

```
class EntidadeMovel   : public Entidade { Posicao pos; Velocidade vel; };
class EntidadeComSaude: public Entidade { Saude hp; };
class Jogador : public EntidadeMovel, public EntidadeComSaude { };  // e as outras?

Entity jogador = world.criar();
world.adicionar<Posicao>(jogador, 0.f, 0.f);
world.adicionar<Velocidade>(jogador, 5.f, 0.f);
world.adicionar<Saude>(jogador, 100, 100);
world.adicionar<Voando>(jogador);       // vira voadora em runtime, sem classe nova
```

Listagem 183: A explosão combinatória, e a alternativa.

V.3.2 Os três pilares

Tabela 74: A separação radical entre identidade, dados e lógica.

Pilar	Responsabilidade	Neste subprojeto
Entity	identificador único (só um número)	`uint32_t` com reciclagem
Component	dados puros, sem lógica	`Posicao`, `Velocidade`, `Saude`
System	lógica que opera sobre componentes	`movimento()`, `colisao()`, `dano()`

A entidade não é um objeto: é um número. Não tem métodos, não tem dados, não tem tipo. Os componentes são *aggregates* sem um único método — inclusive os de marcação, que não têm nem dados (`struct Jogador {};`) e existem só para que a presença deles seja consultável. E os sistemas são funções livres.

Por que — por que a reciclagem de IDs importa

Um jogo de longa duração cria e destrói projéteis a cada quadro. Sem reciclagem, o espaço de IDs cresce indefinidamente até estourar o `uint32_t`; com reciclagem, o `EntityManager` devolve o ID destruído a uma pilha e o reutiliza na próxima criação. O custo é que um ID reutilizado pode ser confundido com o antigo — razão pela qual ECS de produção usam *generational indices*, com um contador de geração no bit alto do identificador.

V.3.3 *Type erasure* para guardar tipos heterogêneos

O problema técnico central é armazenar `ComponentStore<Posicao>`, `ComponentStore<Saude>` e todas as outras no mesmo mapa — tipos diferentes, sem base comum natural. A solução é o mesmo *type erasure* de `std::function` e `std::any` (Seções III.4 and IV.4): uma interface não *template*, e um `std::type_index` como chave.

```
struct IComponentStore {                    // interface base, sem templates
    virtual ~IComponentStore() = default;
    virtual void remover(Entity e) = 0;
};

template<typename T>
class ComponentStore : public IComponentStore { /* ... */ };

std::unordered_map<std::type_index, std::unique_ptr<IComponentStore>> stores_;

template<typename T>
ComponentStore<T>& store_de() {
    auto key = std::type_index(typeid(T));
    return static_cast<ComponentStore<T>&>(*stores_[key]);
}
```

Listagem 184: A chave é o tipo, e o `static_cast` de volta é seguro porque a chave garante qual é.

V.3.4 A *query*: onde as peças se encaixam

A operação que define o ECS é a consulta "todas as entidades que têm *estes* componentes". E ela é uma aplicação direta de uma *fold expression* (Seção IV.1):

```
    template <typename... Ts, typename Func>
    void query(Func&& func) {
        // Itera sobre todas as entidades do primeiro tipo (menor iteração possível)
        // Depois verifica se têm os demais tipos
        auto& primeiro_store = store_de<first_t<Ts...>>();

        const auto& ents = primeiro_store.entidades();
        for (std::size_t i = 0; i < ents.size(); ++i) {
            Entity e = ents[i];
            if (tem_todos<Ts...>(e)) {
                func(e, get<Ts>(e)...);
            }
        }
    }
```

Listagem 185: A *query* genérica, em `src/ecs/world.hpp`.

Com ela, um sistema inteiro cabe em quatro linhas, e é uma função pura testável isoladamente:

```
void movimento(World& w, float dt) {
    w.query<Posicao, Velocidade>([&](Entity, Posicao& p, Velocidade& v) {
        p.x += v.dx * dt;
        p.y += v.dy * dt;
    });
}
```

Listagem 186: Um sistema completo.

V.3.5 O que se ganha e o que se perde

Tabela 75: ECS contra hierarquia de herança.

Critério	Herança	ECS
combinação de capacidades	explosão combinatória	composição em execução
localidade de cache	objetos dispersos	componentes do mesmo tipo juntos
acrescentar comportamento	classe nova	`adicionar<T>()` em execução
paralelismo	difícil	sistemas independentes
testabilidade	precisa do objeto	função pura sobre componentes
legibilidade do fluxo	direta	indireta (quem roda quando?)

Armadilha — o ECS deste subprojeto ainda não é *cache-friendly*

A promessa de localidade de cache do ECS depende de os componentes ficarem *contíguos* em memória — o que exige armazenamento em `vector` denso com um mapa esparso de índices. O `ComponentStore` aqui usa `std::unordered_map<Entity, T>`, que espalha os componentes pela *heap* exatamente como a herança fazia. A arquitetura está certa e o armazenamento é didático: trocá-lo por um *sparse set* não muda uma linha dos sistemas — que é, aliás, a melhor demonstração de que a separação vale a pena.

V.4 Plugins: DIP até o limite do sistema operacional

O subprojeto `src/plugin_registry/` carrega bibliotecas compartilhadas em tempo de execução com `dlopen`/`dlsym`, descobre os *plugins* disponíveis varrendo um diretório, e troca o algoritmo de controle sem recompilar nada. É o Seção V.1 levado ao extremo: a dependência não só está invertida como **nem existe no binário**.

O contrato é uma interface virtual pura — `IController`, com `name()`, `reset()` e `compute()` — e o que atravessa a fronteira são apenas *aggregates* simples (`State`, `Action`).

V.4.1 O que é, exatamente, carregar código em tempo de execução

Antes do código, os termos. Um **objeto compartilhado** (*shared object*, arquivo `.so` no Linux, `.dll` no Windows, `.dylib` no macOS) é um binário que contém código de máquina e uma **tabela de símbolos**: uma lista que diz, para cada função exportada, em que deslocamento dentro do arquivo ela começa. Nada nele é endereço absoluto — é código *relocável*, compilado com `-fPIC`, para poder ser carregado em qualquer posição da memória do processo.

Há duas maneiras de usar um `.so`, e a diferença entre elas é o assunto deste subprojeto (Tabela 76). Na **ligação dinâmica comum**, o *linker* anota no executável "preciso de `libfmt.so`", e quem carrega é o *loader* do sistema, antes de o `main` começar. Na **ligação dinâmica explícita**, ninguém anota nada: o programa chama `dlopen` em tempo de execução, com um caminho que pode ter vindo de um arquivo de configuração, e o arquivo pode não existir na hora da compilação.

Tabela 76: As duas formas de usar uma biblioteca compartilhada. Só a segunda permite que o *plugin* seja escrito depois do *host*.

	Ligação dinâmica comum	**`dlopen` explícito**
quem carrega	o *loader*, antes do `main`	o seu código, quando quiser
o nome do `.so`	fixo no executável	uma `std::string` qualquer
símbolo faltando	o programa nem inicia	exceção tratável
o arquivo precisa existir	na compilação	só na hora do `dlopen`
resolução de símbolo	pelo tipo, no *link*	**por nome literal**, em execução

A última linha é a que dita todo o desenho que vem a seguir: `dlsym` procura uma *string*. Não há verificação de tipos — o que atravessa a fronteira é um nome e um `void*`.

Por que — por que só *aggregates* simples cruzam a fronteira

`State` e `Action` são `struct` de `double`, sem vtable / vptr, sem alocação escondida, sem `std::string` dentro. Isso é deliberado, e o cabeçalho do subprojeto documenta o porquê: *host* e *plugin* são duas unidades compiladas separadamente, possivelmente por compiladores ou versões diferentes de biblioteca padrão. Um `std::string` passado por valor entre elas depende de os dois lados concordarem sobre o *layout* interno da `string` — e essa concordância é a ABI, que nem o padrão da linguagem garante. Um `struct` de `double` tem *layout* ditado pela ABI da plataforma (System V AMD64, no Linux), e essa, sim, é estável há décadas.

V.4.2 Os símbolos exportados

```cpp
extern "C" const char* plugin_name() { return "pid"; }

extern "C" plugin::IController* create_controller() {
    return new PIDController();
}

extern "C" void destroy_controller(plugin::IController* p) {
    delete p;
}
```

Listagem 187: Os três símbolos que todo *plugin* precisa exportar.

O `extern "C"` é obrigatório: `dlsym` busca por nome literal, e a decoração de nomes do C++ [5] transformaria `create_controller` em algo que ninguém digitaria. Note que o *plugin* exporta funções livres, não a classe — a classe permanece invisível, e o *host* só conhece a interface.

Como os três símbolos são sempre os mesmos, o subprojeto os empacota numa macro, e o cabeçalho `icontroller.hpp` explica, no próprio comentário, por que uma macro e não algo mais civilizado:

```cpp
#define REGISTER_CONTROLLER(ClassName, name_str)                    \
    extern "C" __attribute__((visibility("default")))               \
    const char* plugin_name() { return name_str; }                  \
    extern "C" __attribute__((visibility("default")))               \
    plugin::IController* create_controller() { return new ClassName(); }\
    extern "C" __attribute__((visibility("default")))               \
    void destroy_controller(plugin::IController* p) { delete p; }

// no plugin, uma linha:
REGISTER_CONTROLLER(PIDController, "pid")
```

Listagem 188: De `src/plugin_registry/icontroller.hpp`: um *plugin* inteiro se registra com uma linha.

As três alternativas foram descartadas por motivos concretos, e vale enunciá-los porque é raro ver uma macro justificada de forma defensável. Um *template* não serve: *templates* têm decoração de nomes, e `dlsym` não acharia nada. Uma função normal de auto-registro não serve: exigiria que o *plugin* se registrasse em algum ponto de inicialização estática — com todas as armadilhas de ordem de inicialização já vistas na Seção V.1. A macro resolve no pré-processador: zero custo em execução, zero infraestrutura, zero risco de esquecer um dos três símbolos.

O `__attribute__((visibility("default")))` é o complemento necessário: com `-fvisibility=hidden` — que é o padrão recomendado para bibliotecas compartilhadas, porque encolhe a tabela de símbolos e acelera o carregamento — os símbolos ficam *invisíveis* por omissão, e o `dlsym` não acharia justamente aqueles três. O atributo abre exceção para eles, e apenas para eles.

V.4.3 `dlopen` e a ordem certa de checar erro

```cpp
void* handle = dlopen(path.c_str(), RTLD_NOW | RTLD_LOCAL);
if (!handle) throw std::runtime_error("dlopen falhou: " + std::string(dlerror()));

dlerror();                                  // limpa erro anterior
auto fn = reinterpret_cast<FnPluginName>(dlsym(handle, "plugin_name"));
if (const char* err = dlerror()) {          // e só ENTÃO checa
    dlclose(handle);
    throw std::runtime_error("plugin_name não encontrado: " + std::string(err));
}
```

Listagem 189: As flags, e o motivo de `dlerror()` vir antes.

`RTLD_NOW` resolve todos os símbolos no carregamento, fazendo a falha acontecer cedo em vez de na primeira chamada; `RTLD_LOCAL` impede que os símbolos do *plugin* vazem para os outros. As alternativas existem e têm uso legítimo (Tabela 77) — mas as duas escolhas do subprojeto são as que se deve tomar por omissão.

Tabela 77: As quatro *flags* de `dlopen` que importam, e quando a alternativa se justifica.

Flag	O que faz	Quando escolher
`RTLD_NOW`	resolve todos os símbolos ao carregar	padrão: falha cedo, com mensagem
`RTLD_LAZY`	resolve cada símbolo na primeira chamada	`.so` enorme do qual se usa pouco
`RTLD_LOCAL`	os símbolos não vazam para outros `.so`	padrão: isola os *plugins*
`RTLD_GLOBAL`	os símbolos ficam globais	quando um *plugin* depende de outro

Armadilha — `RTLD_LAZY` adia o erro para o pior momento possível

Com `RTLD_LAZY`, um *plugin* a que falte um símbolo carrega sem reclamar e mata o processo horas depois, na primeira vez que aquele caminho de código for percorrido — tipicamente em produção, e nunca no teste. É a mesma categoria de erro dos alvos que não se constroem, discutida no fim desta seção: o problema não é a falha, é o *atraso* entre a causa e o sintoma.

Armadilha — `dlsym` pode devolver `nullptr` legitimamente

Testar o retorno de `dlsym` contra `nullptr` é errado: um símbolo pode existir e *valer* `nullptr`. A forma correta é chamar `dlerror()` antes, para limpar qualquer erro pendente, e conferir `dlerror()` depois. Como o repositório documenta no próprio código, é esta a ordem, e não a intuitiva.

V.4.4 O *deleter* customizado, e por que ele é crítico

Este é o ponto em que o subprojeto deixa de ser sobre *plugins* e passa a ser sobre ABI. O objeto foi alocado *dentro* do `.so`, com o alocador do `.so`. Se o *host* fizer `delete`, ele usará o alocador *dele* — e se os dois forem diferentes, o resultado é corrupção de *heap*.

```
1  std::unique_ptr<IController, FnDestroyController>
2  create(const std::string& name) {
3      // ...
4      // O segundo argumento do unique_ptr é o deleter:
5      // em vez de 'delete ptr', ele chama 'ph.destroy(ptr)'.
6      return { raw, ph.destroy };
7  }
```

Listagem 190: O *deleter* como parâmetro de *template*, em `src/plugin_registry/plugin_registry.hpp`.

Regra — quem aloca, destrói

Um objeto criado dentro de uma biblioteca dinâmica deve ser destruído por ela. O `unique_ptr` com *deleter* customizado (Seção II.5) é o instrumento exato para isso: a regra deixa de depender da disciplina de quem escreve o código de chamada e passa a estar no *tipo* do ponteiro. É a mesma técnica do `unique_ptr<FILE, decltype(closer)>`, aplicada a um problema muito mais caro de diagnosticar.

V.4.5 O registro, e a ordem em que tudo morre

O `ControllerRegistry` é um *Meyers singleton*: uma função com `static` local, exatamente como a Seção V.2.6 descreve. Aqui o uso é dos legítimos — há *handles* do sistema operacional abertos, e faz

sentido que exista um dono só —, e a classe declara isso no tipo: construtor de cópia e atribuição são `= delete`.

O destrutor fecha todos os *handles* com `dlclose`, e é aí que mora a armadilha mais cara deste subprojeto.

Armadilha — `dlclose` antes do último `unique_ptr` é código morto sendo executado

Quando `dlclose` descarrega um `.so`, o sistema operacional **desmapeia as páginas** em que aquele código estava. Todos os ponteiros para dentro dele — ponteiros de função, ponteiros para as vtable / vptr das classes do *plugin*, o próprio *deleter* guardado dentro do `unique_ptr` — passam a apontar para memória não mapeada. Destruir um `unique_ptr<IController, FnDestroyController>` depois disso não é "usar um ponteiro inválido": é saltar para um endereço que não existe mais, e o processo morre com `SIGSEGV` num *stack trace* que não menciona nenhuma linha do seu código.
A regra que evita isso é de tempo de vida, não de sintaxe: **todos os `unique_ptr` devem ser destruídos antes do registro**. O comentário no destrutor de `plugin_registry.hpp` registra exatamente essa dependência — e ela é a razão de o `main` criar os controladores dentro de funções de escopo curto (`run_episode`), em vez de guardá-los em variáveis globais.

V.4.6 Onde estão os *plugins*? A resolução de caminho

Um detalhe que parece trivial e não é. O *host* precisa saber em que diretório procurar os `.so` — e a resposta ingênua, um caminho relativo como `"plugins"`, resolve contra o **diretório de trabalho** do processo, não contra o local do executável. O mesmo binário passa a funcionar ou não dependendo de onde foi invocado: roda de dentro do `dist/`, falha se chamado por caminho absoluto de qualquer outro lugar.

A saída portável no Linux é `/proc/self/exe`: um *symlink* que o núcleo mantém apontando para o binário do próprio processo. Lê-lo devolve o caminho real, já com todos os *symlinks* resolvidos.

```
1 static std::filesystem::path diretorio_do_executavel() {
2     std::error_code ec;
3     auto exe = std::filesystem::read_symlink("/proc/self/exe", ec);
4     return ec ? std::filesystem::path{} : exe.parent_path();
5 }
```

Listagem 191: De `src/plugin_registry/main.cpp`: descobrir o próprio diretório, sem depender do `CWD`.

Com isso, a busca vira uma lista de candidatos com prioridade explícita, e cada um deles corresponde a um *layout* real do repositório:

```
1 if (argc > 1) return argv[1];                    // 1. explicito ganha sempre
2 candidatos.push_back(exe_dir / ".." / "plugins"); // 2. instalado: dist/bin -> dist/plugins
3 candidatos.push_back(exe_dir / "plugins");        // 3. build: ao lado do exe
4 candidatos.emplace_back("plugins");               // 4. ultimo recurso: o CWD
```

Listagem 192: A ordem dos candidatos — e o argumento de linha de comando vencendo todos.

E quando nenhum serve, o programa não estoura uma exceção não tratada: imprime os caminhos que *tentou*, ensina a sintaxe do argumento explícito e devolve `1`. É uma diferença pequena de código e enorme de diagnóstico — a mensagem contém a informação que o usuário precisa para consertar sozinho.

Regra — três formas de achar recursos, e a que se deve preferir

`argv[0]` não serve: ele contém o que o *shell* digitou, que pode ser um nome relativo ou nem existir. Variáveis de ambiente (`MEUAPP_PLUGIN_DIR`) servem como *override* e são o que se oferece ao

usuário avançado. Mas o *default* deve ser derivado do executável — `/proc/self/exe` no Linux, `GetModuleFileName` no Windows, `_NSGetExecutablePath` no macOS — porque é a única fonte que não depende de como o programa foi invocado.

V.4.7 O ciclo completo

A descoberta automática fecha o ciclo: o registro varre o diretório, tenta carregar cada `.so`, e — detalhe que importa em produção — captura a exceção por *plugin*, de modo que um *plugin* defeituoso não derruba os demais. O resultado é a troca de algoritmo em tempo de execução, sem recompilação:

```
=== Plugin Registry Demo ===

Varrendo '/home/lima/Projects/cpp_academy/dist/plugins'...
[registry] plugin 'pid' carregado de .../dist/plugins/pid.so
[registry] plugin 'lqr' carregado de .../dist/plugins/lqr.so
2 plugin(s) carregado(s).

=== Troca de controlador em runtime ===
(mesma interface, implementacao diferente, zero recompilacao)

  [pid]  alt= 2900.0 ft  spd= 88.0 kts  pitch=  0.0  | elev=+1.000  thr=0.750
  [lqr]  alt= 2900.3 ft  spd= 88.0 kts  pitch=  2.0  | elev=-0.011  thr=0.710
```

Listagem 193: Saída real de `dist/bin/plugin_registry`, executado sem argumento nenhum: o caminho foi deduzido de `/proc/self/exe`.

As duas últimas linhas são o subprojeto inteiro em duas medições: o **mesmo estado** entra nos dois controladores e sai uma ação diferente de cada um. O PID satura o profundor em `+1.000`; o LQR, com a mesma entrada, pede `-0.011`. O *host* não sabe qual é qual, nunca soube o nome das classes, e não foi recompilado entre as duas chamadas.

V.4.8 A lição que só apareceu quando os alvos foram construídos

Vale fechar este capítulo com um resultado de método, e não de projeto. Até uma revisão recente, boa parte dos alvos deste repositório estava com o `subdir()` comentado no `src/meson.build` — inclusive os "pesados" (Qt, SFML, Bullet, ONNX, JSBSim, AWS, RabbitMQ, SOCI). Hoje os **57** subprojetos são construídos, produzindo **98** binários instalados em `dist/bin`, com zero avisos sob `-Wall -Wextra -Wpedantic`.

Habilitar os alvos não foi uma operação administrativa: ela *encontrou bugs*, e todos estavam exatamente nos alvos que ninguém compilava.

Regra — um alvo que não é construído é um alvo que apodrece

Quatro `#include` faltando e dois bugs de ABI estavam *exatamente* nos alvos desabilitados — e não é coincidência. Um alvo comentado não é "código pausado": é código que continua envelhecendo enquanto as bibliotecas mudam de API, os cabeçalhos param de ser incluídos transitivamente e o compilador aprende avisos novos, sem que nada nunca o diga. O corolário é organizacional: o mesmo raciocínio que levou o diretório `tests/` da raiz a ser **removido** — ele estava 100% comentado e nunca construiu nada, dando a *impressão* de cobertura sem oferecer nenhuma. Os testes hoje moram junto do código que testam (`src/testing/` com 5 alvos e 41 casos, `src/meson/` com 1 alvo, `src/cmake/` com CTest e 11 casos), e `meson test` roda 6 alvos, todos passando. Um teste desabilitado é pior que teste nenhum, porque mente.

Tabela 78: Bugs reais encontrados ao habilitar alvos que estavam desativados. A coluna da direita é o que cada um ensina — e nenhum deles teria aparecido em revisão de código.

Onde	O que estava errado
`asm_course/07`	um `subq $8, %rsp` quebrava o alinhamento de 16 bytes que a ABI System V exige no `call`; o *crash* aparecia *dentro* do `fmt`, quatro quadros acima
`asm_course/10`	`RDI`/`RSI`/`RCX` declarados como entrada pura num *asm inline*, mas `rep movsb` os modifica — o GCC acreditou na promessa e reusou um registrador corrompido
`http-client`	`std::string(n, '–')`: U+2500 tem **três** bytes, e entre aspas simples vira *multi-character constant* truncada; e `40 - title.size()` é aritmética sem sinal, que dá a volta
`aws/s3`	faltava `#include <fmt/core.h>` — latente porque o alvo nunca era compilado
`rabbitmq`	a API do AMQP-CPP mudou: os *callbacks* de ciclo de vida são métodos virtuais do *handler*, não do `TcpConnection`
`database`	o demo não era idempotente: a segunda execução batia numa restrição `UNIQUE`

E o alinhamento entre esta lição e o assunto do capítulo é direto: as fronteiras de que SOLID, os padrões e o registro de *plugins* tratam só cumprem o que prometem se forem *exercitadas*. Uma interface que nada implementa, um padrão que nada instancia e um *plugin* que nada carrega são, os três, a mesma coisa — desenho não verificado.

V.5 Síntese

Os quatro subprojetos deste capítulo respondem à mesma pergunta em escalas crescentes: *onde colocar a fronteira, para que a mudança seguinte custe pouco?*

SOLID responde no nível da classe e do módulo. Os padrões respondem com soluções nomeadas para as fronteiras que reaparecem sempre. O ECS responde recusando a fronteira da herança e substituindo-a pela composição. E o registro de *plugins* responde levando a fronteira até o limite do processo, onde o outro lado é literalmente um arquivo que ainda não existia quando o programa foi compilado.

Armadilha — toda abstração deste capítulo é reversível — e cara antes da hora

Nenhum princípio aqui deve ser aplicado profilaticamente. OCP antes do segundo caso é custo puro; DIP sobre uma dependência estável é indireção sem ganho; ISP levado longe demais vira sopa de interfaces; um ECS num programa com três tipos de entidade é complexidade sem contrapartida. A pergunta que atravessa o capítulo inteiro — e que os quatro subprojetos formulam com palavras diferentes — é sempre: *esta variação já aconteceu duas vezes?*

VI Concorrência

Concorrência é o assunto em que os erros deixam de ser determinísticos. Tudo o que o Seção II classificou como "silencioso" ainda produzia o mesmo resultado a cada execução; aqui, o mesmo binário com a mesma entrada pode acertar mil vezes e falhar na milésima primeira — na máquina do cliente, sob carga, sem *stack trace* útil.

O capítulo percorre cinco subprojetos. `src/concurrency/` apresenta os primitivos da biblioteca padrão, da criação de uma *thread* até o *thread pool*. `src/concurrency2/` continua dali com o que o C++20 acrescentou — `jthread`, `stop_token`, `latch`, `barrier`, `counting_semaphore` e `atomic::wait` —, com o modelo de memória por baixo de todos eles e com a programação *lock-free* medida em vez de presumida. `src/coroutines/` apresenta a concorrência sem *threads* — suspensão e retomada dentro de uma única linha de execução. E `src/mandelbrot/` e `src/mandelbrot_openmp/` resolvem o *mesmo* problema de duas maneiras, o que permite medir o que custa cada abordagem em esforço e em ganho.

Regra — o ThreadSanitizer é obrigatório neste capítulo

Um *data race* é UB (comportamento indefinido), não "valor errado ocasional", e testar não o encontra: a ausência de falha numa execução não é evidência de ausência de corrida. O TSan (`-Db_sanitize=thread`) instrumenta os acessos e detecta corridas que *não* se manifestaram naquela execução — que são exatamente as que interessam. Rode os subprojetos deste capítulo sob ele ao menos uma vez.

VI.1 *Threads* e primitivos de sincronização

VI.1.1 *Threads*: a obrigação que ninguém lembra

Uma `std::thread` precisa ser `join`ada ou `detach`ada antes de ser destruída. Se não for, o destrutor chama `std::terminate` — o programa inteiro morre, sem *unwinding* e sem *handler*.

Por que — por que o destrutor encerra o programa em vez de fazer `join`

As duas alternativas eram piores. `join` implícito faria o destrutor bloquear por tempo indefinido num ponto do código que não parece bloqueante — inclusive durante o desenrolamento de uma exceção. `detach` implícito deixaria a *thread* rodando com referências para objetos que acabaram de ser destruídos. O comitê escolheu falhar ruidosamente em vez de escolher entre dois comportamentos surpreendentes.

A consequência prática é que o caminho de exceção precisa ser coberto, e a ferramenta é a de sempre — RAII:

```
1 class ThreadGuard {
2     std::thread& t_;
3 public:
4     explicit ThreadGuard(std::thread& t) : t_(t) {}
5     ~ThreadGuard() { if (t_.joinable()) t_.join(); }
6 };
```

Listagem 194: O *guard* que garante o `join` mesmo com exceção.

Regra — em C++20, `std::jthread` já faz isso

`std::jthread` faz `join` no destrutor e traz `stop_token` para cancelamento cooperativo. Ele não aparece *neste* subprojeto porque a intenção aqui é mostrar o mecanismo por baixo — mas em código novo, `jthread` é o padrão, e o `ThreadGuard` acima passa a ser matéria histórica. O subprojeto `src/concurrency2/` existe, é construído e o cobre por inteiro, junto com `latch`, `barrier` e `counting_semaphore` (Seção VI.2).

VI.1.2 *Mutex*: por que `contador++` não basta

`contador++` é leitura, modificação e escrita — três operações. Duas *threads* podem ler o mesmo valor, incrementar cada uma a sua cópia, e escrever o mesmo resultado: dois incrementos, um efeito. Com dois laços de mil iterações, o resultado final fica em qualquer ponto entre 1000 e 2000, e não é determinístico.

A família de *locks* da biblioteca padrão é escolhida pela flexibilidade necessária (Tabela 79) — e nenhum deles deve ser travado e destravado à mão.

Tabela 79: Os invólucros RAII de *lock*.

Invólucro	Permite	Use quando
`std::lock_guard`	travar e destravar no escopo	é o caso comum
`std::unique_lock`	destravar, retravar, `try_lock`	precisa de `condition_variable`
`std::shared_lock`	vários leitores simultâneos	leitura domina a escrita
`std::scoped_lock`	travar *vários* sem *deadlock*	precisa de dois *mutexes*

O `std::shared_mutex` implementa o *readers-writer lock*: muitos leitores em paralelo, um escritor exclusivo. Vale quando a leitura domina; com escrita frequente, o custo extra de bookkeeping torna o *mutex* simples mais rápido.

Armadilha — travar dois *mutexes* em ordens diferentes é *deadlock*

Se a *thread* A trava `m1` e depois `m2`, e a B trava `m2` e depois `m1`, mais cedo ou mais tarde as duas ficam paradas esperando uma pela outra — e não há *timeout*, não há erro, não há mensagem. As duas saídas: `std::scoped_lock(m1, m2)`, que usa um algoritmo livre de *deadlock*; ou uma **hierarquia de *locks*** documentada, em que todos travam sempre na mesma ordem.

VI.1.3 `condition_variable`: esperar sem girar

Um *mutex* protege dados; ele não sabe *esperar por um estado*. A `condition_variable` preenche essa lacuna: `wait` libera o *mutex*, dorme, e ao acordar readquire o *mutex* — tudo atomicamente, o que é a única razão de ela existir como primitivo em vez de ser escrita à mão.

```
1  // Produtor
2  {
3      std::lock_guard<std::mutex> lock(mtx);
4      fila.push(42);
5      pronto = true;
6  }
7  cv.notify_one();                    // notifica FORA do lock
8
9  // Consumidor
10 std::unique_lock<std::mutex> lock(mtx);
11 cv.wait(lock, [&]{ return pronto; });   // o predicado não é opcional
```

Listagem 195: O par produtor/consumidor mínimo.

Armadilha — `wait` sem predicado é sempre um bug

O POSIX admite *spurious wakeups*: o sistema operacional pode acordar uma *thread* sem que ninguém tenha notificado. E há o caso pior, que não depende do sistema: se a notificação acontecer *antes* de o consumidor entrar em `wait`, ela se perde, e o consumidor dorme para sempre. A sobrecarga com predicado resolve os dois, porque testa a condição antes de dormir e a cada despertar. Nunca escreva `cv.wait(lock)` sozinho.

VI.1.4 `future`, `promise` e `async`

A tríade cobre o caso "quero um valor que será produzido em outro lugar" (Tabela 80). O ganho sobre *thread* mais variável compartilhada é que a exceção também atravessa: se a função lançar, `future::get()` relança no chamador — ao passo que uma exceção que escapa da função de entrada de uma `std::thread` chama `std::terminate` (Seção II.8).

Tabela 80: As três formas de obter um `future`.

Ferramenta	Quando
`std::async`	a forma mais simples: uma função, um resultado
`std::promise`	o produtor decide *quando* entregar
`std::packaged_task`	embrulhar uma função para enfileirar depois

Armadilha — `std::async` sem política pode não criar *thread* nenhuma

A sobrecarga sem argumento de política permite ao compilador escolher entre `launch::async` e `launch::deferred` — e, se ele escolher *deferred*, a função só roda quando alguém chamar `get()`, na *thread* do chamador. Um programa que dispara dez `async` esperando paralelismo pode acabar executando tudo em sequência. Passe `std::launch::async` explicitamente quando quiser paralelismo de fato. E lembre: o destrutor de um `future` vindo de `std::async` **bloqueia** até a tarefa terminar.

VI.1.5 Atômicos e ordem de memória

Uma operação atômica resolve a corrida sobre *aquela* variável, sem *mutex* e sem bloqueio. O que ela **não** faz é estabelecer invariantes entre duas variáveis — para isso ainda é preciso um *lock*.

A parte difícil é a ordem de memória, que existe porque compilador e processador têm permissão para reordenar acessos. A ordem escolhida determina que reordenações continuam permitidas (Tabela 81).

Tabela 81: As ordens de memória, do mais barato ao mais caro.

Ordem	Garante	Custo
`relaxed`	só atomicidade; nenhuma ordenação	mínimo
`acquire`	leituras posteriores não sobem daqui	médio
`release`	escritas anteriores não descem daqui	médio
`seq_cst`	ordem total única para todas as *threads*	máximo (padrão)

O par *release*/*acquire* é o idioma de publicação, e é o que dá sentido à tabela:

```cpp
// Thread 1 (produtor)
dados = 42;                                  // escrita NÃO atômica
flag.store(true, memory_order_release);   // publica

// Thread 2 (consumidor)
while (!flag.load(memory_order_acquire)); // espera a publicação
fmt::print("{}\n", dados);                  // sempre 42
```

Listagem 196: Publicar dados por meio de uma *flag* atômica.

O *release* garante que a escrita em `dados` não seja movida para depois do `store`; o *acquire* garante que a leitura de `dados` não seja movida para antes do `load`. Sem o par, ler `dados` seria *data race* — mesmo com a *flag* sendo atômica.

Regra — comece com o padrão; só desça de ordem com medição

`seq_cst` é o padrão porque é o único que se comporta como a intuição espera. `relaxed` é correto apenas quando *nenhuma* outra variável depende daquela ordenação — um contador de estatísticas, por exemplo. Trocar por `relaxed` sem essa análise produz um programa que funciona em x86, onde o hardware já é fortemente ordenado, e falha em ARM.

VI.1.6 *Thread pool*: onde tudo se junta

O *thread pool* do subprojeto usa quase todos os primitivos anteriores: *threads* guardadas em um `vector`, uma fila protegida por *mutex*, uma `condition_variable` para acordar quem espera, uma *flag* de parada, e `join` no destrutor.

```cpp
    // — Destrutor: sinaliza parada e aguarda todas as threads ————————
    ~ThreadPool() {
        {
            std::unique_lock<std::mutex> lock(mtx_);
            parar_ = true;
        }
        cv_.notify_all(); // acorda todas as threads para que saiam do loop
        for (auto& t : workers_) {
            if (t.joinable()) t.join();
        }
    }
```

Listagem 197: O encerramento ordenado, em `src/concurrency/demo_patterns.hpp`.

E o laço do trabalhador tem dois detalhes que merecem atenção, ambos visíveis no código:

```cpp
                std::unique_lock<std::mutex> lock(mtx_);
                // Dorme até: haver tarefa OU sinal de parada
                cv_.wait(lock, [this]() {
                    return parar_ || !tarefas_.empty();
                });

                // Se parar_ e fila vazia → encerra a thread
                if (parar_ && tarefas_.empty()) return;

                tarefa = std::move(tarefas_.front());
                tarefas_.pop();
            }

            tarefa(); // executa FORA do lock — outras threads podem pegar tarefas
```

Listagem 198: O laço do trabalhador, em `src/concurrency/demo_patterns.hpp`.

O primeiro é o predicado composto: a *thread* acorda tanto para trabalhar quanto para morrer, e sem o segundo termo o encerramento travaria com todos os trabalhadores dormindo. O segundo é a chave do desempenho: a tarefa é **movida para fora** do escopo do *lock* e executada depois — se rodasse com o *lock* tomado, o *pool* inteiro seria serializado e não haveria paralelismo nenhum.

Tabela 82: As armadilhas clássicas do capítulo, reunidas.

Problema	Causa	Solução
data race	acesso não sincronizado	*mutex* ou atômico
deadlock	duas *threads* esperam uma pela outra	`scoped_lock` ou hierarquia
thread não `join`ada	destruída sem `join`/`detach`	*guard* RAII ou `jthread`
spurious wakeup	CV acordada sem notificação	predicado no `wait`
notificação perdida	`notify` antes do `wait`	predicado no `wait`
problema ABA	CAS vê valor antigo que voltou	CAS versionado, *hazard pointers*
parada que nunca chega	`stop_token` nunca consultado	checar entre unidades de trabalho
permissão vazada	`return` entre `acquire` e `release`	*guard* RAII caseiro
use-after-free	nó liberado com outro CAS em curso	*hazard pointers*, RCU, não liberar

VI.2 Concorrência moderna: o que o C++20 acrescentou

O subprojeto `src/concurrency2/` continua exatamente de onde a Seção VI.1 parou. Aquele cobre o núcleo do C++11; este acrescenta as três coisas que faltavam: os primitivos que o C++20 trouxe (`jthread`, `stop_token`, `latch`, `barrier`, `counting_semaphore` e `atomic::wait`), o **modelo de memória** que existe por baixo de todos eles, e a programação *lock-free* — com a fronteira *medida* em vez de presumida.

O motivo de o assunto merecer um subprojeto inteiro cabe numa frase do `README`: *a intuição sobre concorrência falha de um jeito específico e caro — o código errado funciona nos seus testes.* Um *data race* não estoura; ele fica latente por meses, e um dia o compilador atualiza, o `-O2` iça uma leitura para fora do laço e o serviço trava em produção.

Regra — um alvo que não é construído é um alvo que apodrece

Todos os 57 subprojetos do repositório são construídos hoje — 98 binários instalados em `dist/bin`, com zero avisos sob `-Wall -Wextra -Wpedantic`. Isso não é vaidade de *build*: ao reabilitar os alvos que antes ficavam comentados, apareceram quatro *includes* faltando e dois erros de ABI que ninguém tinha visto justamente porque aqueles alvos nunca eram compilados. Em concorrência o apodrecimento é pior, porque o defeito não é um erro de compilação que aparece na primeira tentativa: é uma corrida que só se manifesta sob carga. Por isso `src/concurrency2/` tem uma exigência declarada — compila sem avisos *e* roda limpo sob o ThreadSanitizer, com zero achados. Alvo que não é construído e executado não tem qualidade; tem apenas ausência de evidência.

VI.2.1 `jthread`: o destrutor que não mata o programa

Por que — por que `std::thread` precisava de um substituto

Ela tem dois defeitos. O primeiro já apareceu na Seção VI.1: o destrutor chama `std::terminate` se a *thread* ainda for *joinable* — esquecer um `join()`, ou sofrer uma exceção antes dele, mata o processo. O segundo é mais silencioso: **não havia forma padrão de pedir que a *thread* parasse**. Cada projeto reinventava o seu `std::atomic<bool> parar`, funcional mas incapaz de conversar com o resto

da biblioteca. `std::jthread` resolve os dois: RAII no destrutor (que faz `request_stop()` e depois `join()`) e um `stop_token` cooperativo embutido.

Escrever um bloco com uma `jthread` e sem `join` deixou de ser um erro: o destrutor cuida disso ao sair do escopo. E o caminho de exceção — que era justamente o que obrigava a escrever o `ThreadGuard` da página anterior — passa a ser coberto de graça, porque o desenrolamento destrói a `jthread` e o destrutor junta.

`stop_token`. É um objeto que responde a uma única pergunta: *alguém já pediu para parar?* Quem pede chama `request_stop()` na `jthread` (ou num `std::stop_source`); quem trabalha consulta `stop_requested()`. A ligação é automática, com uma regra sintática que surpreende: **se a *lambda* declarar um `std::stop_token` como primeiro parâmetro, a `jthread` o passa sozinha**.

```
1   // O 'std::stop_token st' no primeiro parametro e o que ativa o mecanismo
2   std::jthread trabalhador{[&iteracoes](std::stop_token st) {
3       while (!st.stop_requested()) {          // <- o ponto de checagem
4           ++iteracoes;
5           std::this_thread::sleep_for(5ms);
6       }
7       fmt::print("    [worker] parada pedida, saindo limpo\n");
8   }};
9
10  std::this_thread::sleep_for(40ms);
11  trabalhador.request_stop();
```

Listagem 199: O mecanismo inteiro, em `src/concurrency2/demo_jthread.hpp`.

Armadilha — `request_stop()` não interrompe nada: ele apenas pede

A parada é **cooperativa** — `request_stop()` liga uma *flag*, e só. Se a *thread* nunca consultar `stop_requested()`, ela não para, e o `join()` do destrutor espera para sempre: `std::jthread t{[](std::stop_token) { while (true) {} }};` trava o programa inteiro, sem mensagem. Não existe `thread::kill()` em C++, e a ausência é deliberada: matar uma *thread* no meio do caminho deixaria *mutexes* travados, destrutores sem rodar e invariantes quebrados — o `pthread_cancel` do POSIX existe e é considerado inseguro em C++ exatamente por isso. A contrapartida é que *onde* se checa vira decisão de projeto, e a resposta é sempre a mesma: **entre unidades de trabalho, nunca no meio de uma**, ou o estado fica pela metade.

Quem está bloqueado não checa *flag* nenhuma. Uma *thread* parada dentro de um `read()` de *socket* nunca chega ao `stop_requested()`, e o C++20 oferece duas saídas. A primeira é `std::stop_callback`: um objeto que registra uma função no `stop_token` e a dispara *no instante* do `request_stop()` — fechar o *socket* ali faz o `read()` retornar com erro. A segunda é a sobrecarga de `condition_variable_any` que aceita o *token* e devolve `false` quando a parada é pedida:

```
1   std::jthread consumidor{[&](std::stop_token st) {
2       while (true) {
3           std::unique_lock lk{m};
4           // Esta sobrecarga devolve 'false' se a parada foi pedida.
5           // Um 'cv.wait(lk, pred)' normal ficaria bloqueado para sempre.
6           if (!cv.wait(lk, st, [&fila] { return !fila.empty(); })) return;
7           fila.pop_back();
8       }
9   }};
```

Listagem 200: Sair de um `wait` bloqueado, em `src/concurrency2/demo_jthread.hpp`.

A distinção não tem contorno: `std::condition_variable` **não** tem essa sobrecarga; só `condition_variable_any` tem. Ela é mais lenta, porque aceita qualquer tipo *lockable*, e é a única que integra com `stop_token`.

Armadilha — o *callback* roda na *thread* de quem pediu a parada

Não na trabalhadora. Ele precisa ser curto e *thread-safe*, e não deve tomar nenhum *lock* que a trabalhadora possa estar segurando — sob pena de *deadlock* no ponto mais improvável do código.

Armadilha — parada pedida não é trabalho terminado — um bug real deste subprojeto

A primeira versão da fila SPSC de `demo_lockfree.hpp` (Seção VI.2.5) usava `stop_token` como condição de saída do consumidor. O destrutor da `jthread` chama `request_stop()` *antes* do `join()`; o consumidor via o pedido e saía com a fila ainda cheia. A saída do programa acusou `PERDEU DADOS` — a soma não fechava. A condição correta era o trabalho ter **terminado**, não a parada ter sido **pedida**, e o aviso ficou no fonte, sobre o laço consertado. A lição vale além do caso: `stop_token` é para *cancelamento*, não para *encerramento normal*. Quando o critério é "processei tudo", ele tem de ser escrito, não deduzido do ciclo de vida do objeto.

VI.2.2 `latch`, `barrier` e `counting_semaphore`

Por que — por que três primitivos novos, se *mutex* mais CV já dava conta

Dava — em três objetos e umas quinze linhas. Expressar "espere as N *threads* chegarem" exigia um *mutex*, uma `condition_variable` e um contador, com o predicado escrito à mão em dois lugares. Além de verboso, era caro: cada decremento travava o *mutex* e o `notify_all` acordava todo mundo. Os primitivos do C++20 dizem a intenção diretamente, e o *runtime* pode implementá-los com um *futex* sobre um contador atômico — sem *mutex* nenhum no caminho comum.

`latch`: a porteira de uso único. `count_down()` subtrai um; `wait()` bloqueia até o contador chegar a zero; chegou a zero, fica em zero para sempre — não há *reset*, e daí o "uso único". O primeiro uso é o óbvio: a *thread* principal espera as N trabalhadoras. O segundo é a **largada**, e quase ninguém lembra dele:

```
1   std::latch largada{1};                    // 1 so: o tiro de partida
2   std::latch prontos{kThreads};
3
4   for (int i = 0; i < kThreads; ++i) {
5       corredores.emplace_back([i, &largada, &prontos, &ordem] {
6           prontos.count_down();             // "estou na posicao"
7           largada.wait();                   // espera o tiro
8       });
9   }
10
11  prontos.wait();                           // todos posicionados
12  largada.count_down();                     // libera os 4 de uma vez
```

Listagem 201: A largada sincronizada, em `src/concurrency2/demo_sincronizadores.hpp`.

Sem esse padrão, qualquer medição de paralelismo perde o sentido: a *thread* 0 já processou metade do trabalho quando a *thread* 3 nasce, e o tempo medido inclui o custo de criar *threads* em vez do custo do trabalho. São *dois latches*: um para saber que todos chegaram à posição, outro para soltar todos juntos. Para quem participa *e* espera há `arrive_and_wait()`, que é `count_down()` mais `wait()` num passo.

Armadilha — `count_down()` além do zero é UB (comportamento indefinido)

O contador não satura: decrementá-lo depois de zerado é comportamento indefinido, e não há como desfazer. Um `latch` dimensionado com o número errado de participantes, ou uma trabalhadora que decremente duas vezes por engano, produzem um programa que passa em teste e é formalmente inválido.

`barrier`: a porteira que se rearma. É o mesmo contador, com uma diferença decisiva: ao chegar a zero ele **se rearma** para a fase seguinte. É o primitivo das simulações em passos — ninguém começa o passo $N+1$ antes de todos terminarem o passo N — e o análogo padrão do `#pragma omp barrier` da Seção VI.4. O segundo argumento do construtor é a **função de *completion***:

```
1  // O segundo argumento e a "completion function": roda UMA vez, em UMA das
2  // threads participantes, no instante em que a ultima chega — e antes de
3  // qualquer uma ser liberada. E o lugar seguro para trocar buffers.
4  std::barrier sincro{kThreads, [&fase]() noexcept {
5      fmt::print("        -- fim da fase {} (completion) --\n", fase.fetch_add(1));
6  }};
```

Listagem 202: A função de *completion*, em `src/concurrency2/demo_sincronizadores.hpp`.

"Roda uma vez, e antes de qualquer uma ser liberada" é a garantia que permite trocar os *buffers* de leitura e escrita entre passos de uma simulação sem *lock* nenhum: naquele instante, nenhuma outra *thread* está tocando nos dados. Repare no `noexcept` — a função é chamada num ponto em que uma exceção não teria para onde ir. E uma trabalhadora que vai encerrar antes das outras chama `arrive_and_drop()`, que reduz o número esperado nas fases seguintes; sem isso, as restantes esperariam para sempre por quem já morreu.

`counting_semaphore`: contar permissões. `acquire()` toma uma permissão (bloqueando se não houver nenhuma) e `release()` devolve uma, acordando um esperador. O parâmetro de *template* é o **máximo**; o do construtor é o valor **inicial** — os dois são frequentemente confundidos:

Em `std::counting_semaphore<8> vagas{2}`, o `8` é o teto e o `2` é quantas permissões existem no início — o subprojeto usa exatamente esse par para deixar no máximo duas de seis tarefas dentro da seção limitada, e mede o pico observado para provar que o limite valeu.

`std::binary_semaphore` é apelido de `counting_semaphore<1>`, e o seu uso característico não é exclusão mútua: é sinalizar que algo aconteceu, uma vez, com o semáforo começando em zero.

Armadilha — semáforo não é *mutex*, por mais que `counting_semaphore<1>` pareça

A diferença não é de contagem, é de **propriedade** (Tabela 83). Um *mutex* tem dono: quem travou é quem destrava, e destravar o *mutex* alheio é UB (comportamento indefinido). Um semáforo não tem dono — a *thread* A pode dar `acquire` e a B dar `release`, o que é *recurso* em semáforo e *bug* em *mutex*. E não existe `unique_lock` para semáforo: um `return` no meio da função ou uma exceção vazam a permissão para sempre, e o vazamento se manifesta como *deadlock* depois, longe da causa. Quando o RAII fizer falta, ele cabe em cinco linhas — um agregado que faz `acquire` no construtor, `release` no destrutor, e apaga a cópia.

Semáforo é para **contar**: "no máximo quatro conexões simultâneas", "o *buffer* tem dez vagas". O resumo de escolha do `README` resolve a maioria das dúvidas:

```
1  esperar N eventos, uma vez ..... latch
2  sincronizar rodadas repetidas ..... barrier
3  limitar acesso concorrente a K ..... counting_semaphore
```

Tabela 83: Por que um não substitui o outro.

Aspecto	*mutex*	semáforo
dono	sim, quem travou	não, ninguém
quem libera	só o dono	qualquer *thread*
contagem	0 ou 1	0 a N
RAII	`std::scoped_lock`	não há padrão
uso típico	exclusão mútua	recursos limitados

```
4 | sinalizar "aconteceu" uma vez ..... binary_semaphore
5 | exclusao mutua ..... mutex
6 | resultado de UMA operacao assincrona ..... future/promise
```

Listagem 203: Escolha rápida, de `src/concurrency2/`.

VI.2.3 O modelo de memória

Esta é a parte da linguagem em que a intuição mais falha, e a causa é simples de enunciar e difícil de aceitar: **não existe "a ordem em que o programa executou"**. O compilador reordena, porque nada no código de uma *thread* o impede. A CPU reordena, porque executa fora de ordem e adia escritas num *store buffer*. Os *caches* propagam escritas em ordens diferentes para núcleos diferentes. O que o C++11 definiu não foi uma ordem global, e sim uma relação **parcial** — *happens-before* — mais a regra de que ler algo que não *acontece-antes* da sua leitura é *data race*, isto é, UB (comportamento indefinido) total.

Happens-before, em termos concretos: dentro de uma mesma *thread*, tudo o que está escrito antes acontece-antes do que está escrito depois, e isso é gratuito. *Entre threads*, a relação só existe se alguma operação de sincronização a criar — destravar um *mutex* acontece-antes de travá-lo em seguida; criar uma *thread* acontece-antes da primeira instrução dela; um *store release* acontece-antes do *load acquire* que lê aquele valor. Fora dessas pontes não há ordem alguma, e os dois acessos são simultâneos por definição.

Armadilha — *data race* é UB (comportamento indefinido), não "valor errado ocasional"

A intuição errada é "no pior caso eu leio um valor velho". A realidade é que o compilador tem permissão para assumir que a corrida *não acontece* e otimizar em cima disso. O caso clássico é um `bool parar = false;` não atômico lido em `while (!parar) { trabalho(); }` enquanto outra *thread* escreve `true`: com `-O2`, o compilador vê que o laço não escreve em `parar`, iça a leitura para fora e gera `while (true)`. O programa trava para sempre — e o compilador **está certo**, porque a outra *thread* escrevendo ali já era UB (comportamento indefinido). Trocar por `std::atomic<bool>` resolve, porque aí a releitura passa a ser obrigatória.

Armadilha — `volatile` não serve para concorrência

Ele impede que a leitura seja otimizada, o que faz o laço acima funcionar por acidente — e é daí que vem a lenda. Mas `volatile` não dá garantia nenhuma de ordem entre variáveis e não garante atomicidade: um `volatile long long` pode ser lido pela metade numa máquina de 32 bits. Em C++, `volatile` é para registrador mapeado em memória (MMIO), não para *threads*.

***Acquire*/*release*, em uma frase.** Se a *thread* B faz um *load-acquire* que **lê o valor escrito** por um *store-release* da *thread* A, então tudo o que A escreveu antes daquele *store* é visível para B depois daquele *load*.

É assim que se publica uma estrutura inteira com um único atômico — e note que os campos publicados *não* são atômicos, de propósito:

```
    std::jthread leitor{[&] {
        while (!pronto.load(std::memory_order_acquire)) {   // <- ACQUIRE
            std::this_thread::yield();
        }
        // Aqui e SEGURO ler 'carga' sem ser atomica: o par release/acquire
        // criou happens-before entre as escritas do produtor e estas leituras.
        ok = (carga.a == 42 && carga.b == 3.5 && carga.c == "publicado");
    }};

    carga.a = 42;                                       // escritas comuns
    carga.b = 3.5;
    carga.c = "publicado";
    pronto.store(true, std::memory_order_release);      // <- RELEASE publica tudo
```

Listagem 204: Publicação por *flag*, em `src/concurrency2/demo_memory_model.hpp`.

A barreira é **direcional**, e é isso que a torna barata: *release* impede que qualquer coisa de antes desça para depois do *store*; *acquire* impede que qualquer coisa de depois suba para antes do *load*. Cada um bloqueia um sentido só; `seq_cst` bloqueia os dois *e* ainda impõe uma ordem total entre todas as operações `seq_cst` do programa.

Armadilha — um *release* sozinho não sincroniza nada

São três condições, e falhar em qualquer uma anula a garantia sem aviso: (a) tem de haver um *acquire* do outro lado; (b) esse *acquire* tem de **ler o valor que aquele *release* escreveu** — ler um valor mais antigo não sincroniza com coisa alguma; e (c) o par tem de ser **no mesmo átomo**. Fazer *release* em `a` e *acquire* em `b` é escrever duas barreiras que não se conversam.

`relaxed`: só atomicidade. Ele garante exatamente uma coisa: a operação é atômica — nenhum incremento se perde, nenhum valor "meio escrito" aparece. Não garante *nada* sobre a ordem em relação a outras variáveis. O subprojeto mede o caso legítimo (quatro *threads* somando 50 000 vezes cada uma com `fetch_add relaxed`) e o contador fecha em 200 000 sempre, porque ninguém depende da ordem dos incrementos. O uso errado é publicar dados: `pronto.store(true, relaxed)` deixa a outra *thread* enxergar `pronto == true` e `dados == 0`.

Regra — a contagem de referências é o exemplo canônico, e é assimétrica

No contador de um `std::shared_ptr`, o **incremento** pode ser `relaxed` — basta que nenhum se perca. O **decremento** precisa ser `acq_rel`: quem zera o contador vai destruir o objeto, e precisa enxergar tudo o que as outras *threads* fizeram com ele antes de soltarem as suas referências. Essa assimetria é a razão de o decremento custar mais que o incremento (Seção II.5).

`seq_cst`: o padrão, e quanto ele custou de fato. `x.store(1)` é `x.store(1, memory_order_seq_cst)` — a única ordem que corresponde à intuição, porque existe *uma* ordem total de todas as operações `seq_cst` com a qual todas as *threads* concordam. A crença difundida é que ela é cara e que trocar para `relaxed` é dinheiro no bolso. O subprojeto mede, com dois milhões de `fetch_add` sem contenção:

```
relaxed :    2.96 ms
seq_cst :    2.99 ms
razao   :    1.01x
```

Listagem 205: Saída de `src/concurrency2/`: dois milhões de `fetch_add` numa *thread* só, x86-64.

Um por cento — ou seja, ruído. A explicação é arquitetural: no x86-64, `fetch_add` vira `lock xadd` de qualquer maneira, a arquitetura é fortemente ordenada e *acquire*/*release* saem de graça. Num ARM ou POWER, onde `seq_cst` exige barreiras explícitas (`dmb ish`), a diferença é grande.

Regra — comece com `seq_cst`; só desça com o profiler e com a frase escrita

A medição acima é a razão de a regra existir. Descer de ordem é a otimização de menor retorno e maior risco do capítulo: no x86-64 costuma não pagar nada, e num ARM produz um programa que passa em todos os testes da sua máquina e falha na do cliente. Só desça quando (a) o *profiler* apontar aquela operação e (b) você conseguir escrever, em uma frase, qual *happens-before* está sendo preservado. Se não consegue escrever a frase, não desça.

Armadilha — atômico não é "*thread-safe*" automaticamente

`if (x > 0) x = x - 1;` são *duas* operações sobre um `std::atomic<int>`: uma leitura e uma escrita. Entre elas outra *thread* mexe, e o contador fica negativo. Cada operação é atômica; a *sequência* não é. Para torná-la atômica é preciso `compare_exchange` num laço, ou um *mutex*.

Armadilha — `atomic<T>` com `T` grande esconde um *lock*

O subprojeto mede: `atomic<int>` (4 bytes) e `atomic<double>` (8 bytes) são *lock-free*; uma *struct* de 32 bytes não é — o corte no x86-64 é 16 bytes, o alcance do `cmpxchg16b`. Quando não é *lock-free*, a implementação usa um *mutex* interno: continua correta, mas não pode mais ser usada em *signal handler* e perde a garantia de progresso. Consulte o `static constexpr is_always_lock_free`, que responde em compilação; o `is_lock_free()` de membro responde em execução e, para tipos grandes, vira chamada a `__atomic_is_lock_free` — o que obriga a linkar `-latomic`.

Falta uma quinta ordem na Tabela 81, e a omissão é deliberada: `memory_order_consume` está no padrão para não ser usado. Nenhum compilador o implementa de verdade — todos o promovem para `acquire` — e ele está em revisão desde o C++17. Escrevê-lo não produz erro, nem aviso, nem efeito: é documentação enganosa dentro do código. E nada nesta seção substitui rodar o TSan da regra de abertura do capítulo, que é o que transforma "não vi falhar" em evidência — `src/concurrency2/` é executado sob ele e reporta zero achados.

VI.2.4 CAS: a primitiva que constrói todas as outras

Por que — por que `fetch_add` não é suficiente

Ele resolve contadores, e só. "Incremente apenas se ainda for X", "insira na frente da lista", "atualize o máximo" — nada disso é uma soma. Todas essas operações precisam de uma primitiva capaz de dizer *"se o valor ainda for o que eu li, troque; senão, me diga o que ele é agora"*: o *compare-and-swap*. O hardware a expõe diretamente — `lock cmpxchg` no x86, o par `ldrex`/`strex` (LL/SC) no ARM — e tudo o mais (`fetch_add`, `fetch_or`, `exchange`) pode ser construído a partir dela.

A forma canônica é um laço de corpo vazio, e o detalhe que a faz funcionar está na assinatura: `esperado` é passado **por referência** e é reescrito pela própria função quando ela falha. Sem isso, cada volta precisaria de um `load` extra.

```
1  void atualiza_max(std::atomic<int>& m, int v) {
2      int atual = m.load();
3      while (v > atual &&
4             !m.compare_exchange_weak(atual, v)) {
5          // corpo VAZIO: o CAS já reescreveu 'atual' com o valor real
```

```
    }
}
```

Listagem 206: O laço de CAS, forma canônica.

A escolha entre as duas versões é mecânica. `_weak` pode falhar *espuriamente* — devolver `false` mesmo quando o valor era o esperado — porque mapeia direto no par LL/SC do ARM e do POWER, e por isso é a versão barata para usar *dentro* de um laço, onde a falha só custa outra volta. `_strong` só falha se o valor realmente mudou, e é a versão para quando não há laço externo. O CAS aceita ainda *duas* ordens de memória — `compare_exchange_weak(esp, novo, ORDEM_SUCESSO, ORDEM_FALHA)` — e a ordem de falha não pode ser mais forte que a de sucesso, nem ser `release`: uma falha não escreve nada, e não há o que publicar.

O preço da contenção, medido. O subprojeto reimplementa `fetch_add` em cima de CAS — o cenário mais contencioso possível, em que toda *thread* disputa o mesmo átomo em toda iteração — e instrumenta o laço com um segundo contador, que soma *tentativas* em vez de sucessos:

```
incrementos efetivados : 80000
tentativas de CAS      : 141160 – 201402   (varia por execucao)
falhas por contencao   : 43% – 60% de retrabalho
```

Listagem 207: Saída de `src/concurrency2/`: 4 *threads* × 20 000 incrementos por CAS.

Nenhum incremento se perde — a corretude está intacta. Mas entre 43% e 60% das voltas do laço foram trabalho jogado fora. É esse retrabalho que faz um algoritmo *lock-free* degradar sob contenção: ele nunca **bloqueia**, e ainda assim não é de graça. Com uma *thread* só, o retrabalho é zero. Na prática, use `fetch_add`: o hardware resolve num `lock xadd`, sem laço nenhum.

Por que — por que a primeira versão desta medição foi descartada

Ela media a atualização de um máximo, com cada *thread* usando uma faixa disjunta e crescente de valores. O resultado foi **0% de retrabalho**: cada valor já era um novo máximo, e o CAS acertava de primeira. A medição contradizia o texto — e o texto estava certo sobre o fenômeno, mas o experimento não reproduzia a condição de que ele falava, porque não havia disputa alguma pelo átomo. Trocou-se o experimento por um que força a disputa. A lição de método atravessa o livro: quando o número contraria a expectativa, a primeira pergunta é *o que este experimento está medindo de fato?* — e é frequente que a resposta seja "outra coisa".

`atomic_flag` e a espera atômica. `atomic_flag` é o único tipo que o padrão *exige* ser *lock-free* em toda implementação, e por isso o único utilizável sem ressalvas num *signal handler*. Tem três operações: `test_and_set()`, que liga a *flag* e devolve o valor *anterior*; `clear()`; e `test()`, que só existe desde o C++20 — versão em que a *flag* também passou a nascer limpa, aposentando o `ATOMIC_FLAG_INIT`.

Desde o C++20, todo `std::atomic<T>` também tem `wait`, `notify_one` e `notify_all`, implementados sobre *futex*. São a terceira maneira de esperar por uma condição: o laço de *spin* não precisa de *mutex* mas queima CPU; a `condition_variable` dorme mas exige *mutex*; `atomic::wait` dorme *e* dispensa o *mutex*.

Armadilha — `atomic_flag` não serve para "inicialize uma vez"

`if (!ja_inicializou.test_and_set()) inicializa();` elege corretamente uma única *thread* para inicializar — e deixa todas as outras seguirem em frente *sem esperar*, usando o objeto ainda não inicializado. Para inicialização única use `std::call_once` ou uma estática local, *thread-safe* desde o

C++11, que fazem as demais **esperarem**. A *flag* só diz quem chegou primeiro.

Armadilha — a semântica de `atomic::wait` é o inverso da intuição

`estado.wait(0)` significa "durma **enquanto** o valor ainda for 0", e não "espere até que ele seja 0" — o oposto de `cv.wait(lk, pred)`, que espera até o predicado ser verdadeiro. E, como a `condition_variable`, ela sofre despertar espúrio: em código real, o `wait` vai dentro de um laço que reconfere a condição.

***Spin lock*: o exercício que quase nunca se deve usar.** A implementação didática do subprojeto usa o idioma *test and test-and-set*, e o laço interno é o ponto:

```
1   class SpinLock {
2       std::atomic_flag travado_{};
3   public:
4       void lock() {
5           while (travado_.test_and_set(std::memory_order_acquire)) {
6               // `test()` puro nao escreve na linha de cache; ficar em
7               // `test_and_set` faria a linha pingar entre os nucleos a cada
8               // volta. Isto e o "test and test-and-set".
9               while (travado_.test(std::memory_order_relaxed))
10                  ;
11          }
12      }
13      void unlock() { travado_.clear(std::memory_order_release); }
14  };
```

Listagem 208: *Test and test-and-set*, em `src/concurrency2/demo_atomicos.hpp`.

Armadilha — o *spin lock* caseiro perde para o `std::mutex` do sistema

Ele funciona, e mesmo assim é quase sempre a escolha errada: quem espera queima CPU integral — num núcleo de que o dono do *lock* talvez precisasse para terminar e liberar; se o sistema operacional preempta o dono, os demais giram um quantum inteiro à toa; e não há RAII, reentrância nem detecção de *deadlock*. O detalhe decisivo é que o `std::mutex` do glibc **já faz um *spin* curto** antes de recorrer ao *futex*: o melhor dos dois mundos vem de graça. Só considere o *spin lock* quando a seção crítica tiver pouquíssimas instruções, nenhuma chamada de função, e você tiver medido.

VI.2.5 *Lock-free* na prática

Por que — o que "*lock-free*" quer dizer — e o que não quer

Não quer dizer "rápido". Quer dizer uma garantia de **progresso**. Um algoritmo *bloqueante* pode ser paralisado por uma única *thread* suspensa no lugar errado — o sistema operacional a tirou do processador com o *mutex* na mão, e todas as outras esperam. Um algoritmo *lock-free* garante que *alguma thread* sempre progride, ainda que uma específica possa ficar tentando indefinidamente. *Wait-free* é mais forte: *toda thread* progride num número limitado de passos. A garantia importa em duas situações concretas: código em *signal handler*, que não pode travar *mutex*, e sistemas de tempo real, em que o pior caso é o que conta.

A fila SPSC: a única que vale escrever à mão. SPSC é *single producer, single consumer*. Restringir o problema a **um** produtor e **um** consumidor o torna tratável, porque cada índice passa a ter um único

escritor: o produtor é o único que escreve `cabeca_`, o consumidor é o único que escreve `cauda_`. Não há CAS nenhum — só um par *release*/*acquire* de cada lado.

```
template <typename T, std::size_t N>
class FilaSPSC {
    static_assert((N & (N - 1)) == 0, "N precisa ser potencia de 2");

    // Cada indice na sua propria linha de cache: sem isso, o produtor
    // escrevendo 'cabeca_' invalidaria a linha do consumidor a cada push.
    alignas(kLinha) std::atomic<std::size_t> cabeca_{0};   // so o produtor escreve
    alignas(kLinha) std::atomic<std::size_t> cauda_{0};    // so o consumidor escreve
    alignas(kLinha) T                        buf_[N]{};

public:
    bool push(const T& v) {
        const std::size_t c = cabeca_.load(std::memory_order_relaxed);
        const std::size_t prox = (c + 1) & (N - 1);
        // acquire: precisa enxergar os pops ja feitos pelo consumidor
        if (prox == cauda_.load(std::memory_order_acquire)) return false;  // cheia
        buf_[c] = v;
        cabeca_.store(prox, std::memory_order_release);   // publica o dado junto
        return true;
    }

    std::optional<T> pop() {
        const std::size_t t = cauda_.load(std::memory_order_relaxed);
        if (t == cabeca_.load(std::memory_order_acquire)) return std::nullopt;  // vazia
        T v = buf_[t];
        cauda_.store((t + 1) & (N - 1), std::memory_order_release);
        return v;
    }
};
```

Listagem 209: A fila SPSC completa, em `src/concurrency2/demo_lockfree.hpp`.

Três decisões merecem leitura atenta. O `static_assert` de potência de dois permite trocar o `%` por um `&` com a máscara `N - 1`, e módulo é caro num caminho quente. O `alignas(kLinha)` — em que `kLinha` é `std::hardware_destructive_interference_size`, o tamanho mínimo que separa dois objetos para que não caiam na mesma linha de *cache* — evita que o produtor, ao escrever `cabeca_`, invalide a linha do consumidor a cada `push`: é o *false sharing* medido em `src/performance/`. E o par *release*/*acquire* publica *também* a escrita em `buf_[c]` feita na linha anterior — a mesma publicação por *flag* da Seção VI.2.3, aplicada a cada elemento. O resultado medido: 200 mil itens em cerca de 2,3 ms, ou aproximadamente 90 milhões de itens por segundo, com a soma íntegra.

Armadilha — a garantia da SPSC depende de ser exatamente 1 produtor e 1 consumidor

Com dois produtores, ambos podem ler a mesma `cabeca_`, escrever no mesmo *slot* e um sobrescrever o outro — não há CAS para arbitrar. Não há aviso de compilação nem verificação em execução: a classe simplesmente deixa de estar correta no momento em que alguém acrescenta uma segunda *thread* produtora, meses depois, sem ler o comentário.

A pilha *lock-free* e o problema ABA. A pilha já exige CAS em laço, porque vários produtores disputam o mesmo `topo_`. E é aqui que aparece a armadilha mais conhecida do assunto:

```
thread 1: le topo = A, vai chamar CAS(A -> A->prox = B)
thread 2: pop A, pop B, push A de volta
thread 1: CAS(A -> B) SUCEDE — o ponteiro e A de novo!
          Mas B ja saiu da pilha. O topo agora aponta para lixo.
```

Listagem 210: O problema ABA, como o subprojeto o descreve.

O CAS compara **endereços**, não identidades — e o alocador adora reciclar o endereço que acabou de ser liberado, o que torna o ABA muito mais provável do que a descrição sugere. As defesas conhecidas são todas caras: contador de versão junto do ponteiro (o que exige CAS de 128 bits, `cmpxchg16b`, e alinhamento a 16 bytes), *hazard pointers*, RCU ou *epoch-based reclamation*, ou simplesmente nunca liberar.

Armadilha — não há momento seguro para liberar um nó

No `pop()` da pilha, o CAS lê `t->prox` — e outra *thread* pode ter feito `delete t` um instante antes. É *use-after-free*, e o AddressSanitizer o acusa. A demonstração do subprojeto escapa porque **separa as fases**: primeiro só *pushes* concorrentes, depois só *pops* de uma *thread* — e o `delete` está marcado no fonte com uma advertência explícita de que é um bug em uso concorrente. Não há como saber se alguma *thread* ainda tem um ponteiro para o nó, e é essa impossibilidade — não o CAS — que torna estruturas *lock-free* encadeadas assunto de especialista.

`std::atomic<std::shared_ptr<T>>`, do C++20, resolve o ciclo de vida, e o GCC 13 já o tem — mas o seu `is_always_lock_free` é `false`: internamente há um *mutex*. Fica correto e deixa de ser *lock-free*, que era o ponto. Na prática, use as implementações prontas (`moodycamel::ConcurrentQueue`, `boost::lockfree`, `folly::MPMCQueue`) e reserve o esforço de escrever a sua para o caso SPSC, o único simples o bastante para ser revisado com calma.

A medição que fecha o assunto. Quatro *threads*, cem mil operações cada uma, dois experimentos:

```
contador:
  atomic fetch_add          :  3.87 ms
  mutex + ++                :  9.10 ms   (2.3x mais lento)

insercao:
  mutex + vector::push_back :  8.68 ms
  pilha lock-free (com new) : 23.61 ms   (2.7x mais LENTO)
```

Listagem 211: Saída de `src/concurrency2/`: 4 *threads* × 100 000 operações.

O primeiro resultado confirma a expectativa: quando a seção crítica é *uma instrução*, o atômico ganha com folga, porque o *mutex* acrescenta duas operações atômicas — travar e destravar — em volta dela. O segundo a contraria, e é o resultado que justifica a seção inteira: com trabalho um pouco maior dentro da seção crítica, **o *mutex* ganhou**, por um fator de 2,7. A pilha *lock-free* paga um `new` por nó — e o alocador tem o seu próprio *lock* interno, de modo que ela não é sequer tão *lock-free* quanto o nome sugere — e ainda sofre o retrabalho de CAS medido na Seção VI.2.4.

Regra — para ganhar desempenho, reduza a contenção; *lock-free* é para a garantia

Nesta ordem: primeiro tente eliminar o compartilhamento (dados por *thread*, *sharding*, ou simplesmente uma seção crítica menor); depois considere um atômico, se a operação couber em uma instrução; e só escolha uma estrutura *lock-free* quando precisar da *garantia de progresso* — *signal handler*, tempo real, ou um caminho em que suspender uma *thread* é inaceitável. Escolher *lock-free* esperando velocidade é, como a medição mostra, uma aposta que frequentemente se perde.

VI.3 Co-rotinas

O subprojeto `src/coroutines/` trata da adição do C++20 que separa *concorrência* de *paralelismo*. Uma co-rotina (C++20) não cria *thread* nenhuma: ela suspende a própria execução, devolve o controle a quem

chamou, e retoma depois exatamente de onde parou — com as variáveis locais preservadas num *frame* alocado à parte.

Uma função é uma co-rotina se usar qualquer um dos três operadores (Tabela 84). Não há palavra-chave no cabeçalho: a natureza da função é determinada pelo corpo, o que é uma decisão de projeto incomum e uma fonte frequente de confusão.

Tabela 84: Os três operadores, e o análogo em linguagens que já os tinham.

Operador	Significado	Análogo
`co_yield v`	suspende e entrega `v` ao chamador	`yield` do Python
`co_await expr`	suspende e aguarda `expr` completar	`await` do Python/JS
`co_return v`	termina a co-rotina com o valor `v`	`return`

VI.3.1 *Generator*: preguiça com estado

```
1  Generator<long long> fibonacci(long long limite) {
2      long long a = 0, b = 1;
3      while (a <= limite) {
4          co_yield a;
5          auto prox = a + b;
6          a = b;
7          b = prox;
8      }
9      // ao sair do while, a coroutine termina → return_void() chamado
10 }
```

Listagem 212: Um gerador infinito recortado por um limite, em `src/coroutines/demo_generator.hpp`.

Compare com o que a Seção III.2 dizia sobre *views*: elas também são preguiçosas, mas guardar estado numa *view* exige um *lambda* `mutable`, e isso é frágil — a *view* pode ser percorrida mais de uma vez, e o estado não reinicia. A co-rotina resolve porque o estado vive no *frame*, e o *frame* pertence ao gerador.

VI.3.2 O `promise_type`: o contrato que a linguagem exige

A linguagem fornece apenas o mecanismo de suspensão; *o que fazer* em cada ponto é decidido por um tipo que o usuário escreve. O compilador procura `T::promise_type` no tipo de retorno da co-rotina e chama seis métodos dele em momentos definidos. Os dois que mais confundem são as suspensões inicial e final:

```
1          // initial_suspend → suspend_always: a coroutine não executa nada
2          // ao ser criada. Só começa quando chamamos .next() pela primeira vez.
3          // (Alternativa: suspend_never = executa até o primeiro co_yield logo)
4          std::suspend_always initial_suspend() noexcept { return {}; }
5
6          // final_suspend → suspend_always: ao terminar, a coroutine suspende
7          // em vez de se destruir automaticamente. Isso é necessário para que
8          // o Generator (que guarda o handle) possa destruí-la de forma segura.
9          std::suspend_always final_suspend() noexcept { return {}; }
```

Listagem 213: Duas decisões de projeto do gerador, em `src/coroutines/demo_generator.hpp`.

Armadilha — o *frame* da co-rotina é alocado, e alguém tem de destruí-lo

O `coroutine_handle` é um ponteiro cru com semântica de dono: quem o guarda precisa chamar `destroy()`, e ninguém o faz por você. O `final_suspend` devolvendo `suspend_always` é justamente o

que evita a auto-destruição e transfere a responsabilidade para o invólucro (`Generator`, no caso), que a cumpre no destrutor. Um invólucro que esqueça isso vaza o *frame* a cada chamada — e o vazamento não aparece em teste curto.

VI.3.3 *Task* e o protocolo *Awaitable*

Enquanto `co_yield` produz uma sequência, `co_await` espera um resultado. E a diferença em relação a `std::future` é exatamente a que motiva o capítulo:

```
int resultado = future.get();      // BLOQUEIA: a thread fica parada aqui
int resultado = co_await task;     // SUSPENDE: a thread vai fazer outra coisa
```

Listagem 214: Bloquear a *thread* contra suspender a co-rotina.

Qualquer tipo participa de `co_await` se implementar três métodos: `await_ready()` (já tem resultado? então nem suspenda), `await_suspend(handle)` (o que fazer ao suspender) e `await_resume()` (o valor que a expressão devolve). É um protocolo aberto, e é por isso que Asio pôde integrar co-rotinas sem que o padrão soubesse de Asio.

VI.3.4 O que o compilador gera

A transformação é conhecida: a função vira uma **máquina de estados**, e as variáveis locais que sobrevivem a uma suspensão migram para o *frame*. O subprojeto reconstrói o esqueleto à mão — um `switch` sobre o estado, com `goto` para o ponto de retomada — o que torna imediatamente óbvio por que variáveis locais precisam ser realocadas: a pilha original já não existe quando a co-rotina é retomada.

Armadilha — cadeias longas de co-rotinas estouram a pilha sem *symmetric transfer*

Se `await_suspend` chamar `outro_handle.resume()`, cada retomada empilha um *frame* de chamada sobre o anterior, e uma cadeia longa termina em *stack overflow*. A correção é devolver o *handle* em vez de retomá-lo: `auto await_suspend(handle h) { return outro_handle; }`. O compilador transforma isso numa *tail call* — a pilha fica constante.

VI.3.5 Onde as co-rotinas realmente pagam: I/O

Com Asio, três co-rotinas concorrentes rodam em **uma** *thread*, e a ordem de término é a das esperas, não a da criação. Um servidor de eco dá uma co-rotina por cliente, e uma *thread* gerencia todas — escalável a milhares de conexões sem o custo de criar uma *thread* por conexão (Seção VII.2).

```
asio::awaitable<void> echo_session(tcp::socket socket) {
    char buffer[1024];
    while (true) {
        auto n = co_await socket.async_read_some(
            asio::buffer(buffer), asio::use_awaitable);
        co_await asio::async_write(
            socket, asio::buffer(buffer, n), asio::use_awaitable);
    }
}
```

Listagem 215: Uma sessão inteira, escrita como se fosse síncrona.

Tabela 85: Qual primitiva para qual problema.

Caso de uso	Use
gerar sequência de valores	`Generator<T>` com `co_yield`
operação assíncrona que devolve um valor	`Task<T>` com `co_await`
I/O de rede ou disco sem bloquear	`asio::awaitable<T>`
máquina de estados explícita	co-rotina com vários `co_yield`
concorrência leve em vez de *threads*	*pool* de co-rotinas num `io_context`

Regra — co-rotinas são para concorrência; *threads*, para paralelismo

Co-rotinas não usam mais de um núcleo: elas trocam de tarefa dentro da mesma *thread*, e o ganho é sobre *espera* — I/O, temporizadores, protocolos de rede. Trabalho que consome CPU não fica mais rápido por virar co-rotina, e o Mandelbrot da Seção VI.4 é o exemplo oposto: puro cálculo, que só melhora com núcleos de verdade.

VI.4 Paralelismo aplicado: dois Mandelbrots

O conjunto de Mandelbrot é o problema didático perfeito para paralelismo: **cada pixel é independente de todos os outros**. Não há dado compartilhado, não há ordem obrigatória, não há sincronização necessária — o que o torna *embaraçosamente paralelo*, e o que permite comparar duas implementações sem que a comparação seja poluída por diferenças de correção.

O algoritmo é o tempo de escape: para cada ponto c do plano complexo, itera-se $z_{n+1} = z_n^2 + c$ a partir de $z_0 = 0$, contando quantos passos a sequência leva para escapar do disco de raio 2. Se não escapar dentro do limite, o ponto é considerado pertencente ao conjunto.

VI.4.1 O laço interno, e por que ele não usa `std::complex`

Escrito com `std::complex<double>` e `std::abs(z) <= 2.0`, o algoritmo cabe em quatro linhas e é o que aparece em toda apresentação do assunto. Os dois renderizadores do repositório não o escrevem assim, e a razão está no caminho quente:

```
// iteracao de Mandelbrot: z_{n+1} = z_n^2 + c
double zr = 0.0, zi = 0.0;
int iter = 0;
while (iter < max_iter_ && zr*zr + zi*zi < 4.0) {
    double zr2 = zr*zr - zi*zi + cr;
    zi = 2.0 * zr * zi + ci;
    zr = zr2;
    ++iter;
}
```

Listagem 216: O laço de escape, em `src/mandelbrot/mandelbrot_renderer.cpp`.

Duas otimizações estão embutidas aí, e as duas são gerais o bastante para valer fora do fractal. A primeira: o teste de escape é $|z| > 2$, mas $|z| = \sqrt{z_r^2 + z_i^2}$ — e comparar $z_r^2 + z_i^2$ com 4 dá o *mesmo* resultado sem a raiz quadrada, que é uma das operações mais caras do processador. A segunda: `zr2` é uma variável temporária obrigatória, porque calcular `zi` depois de já ter sobrescrito `zr` usaria o valor errado — o tipo de erro que `std::complex` esconde e o código manual expõe. Escrever o laço à mão custa quatro linhas a mais e devolve o controle sobre exatamente estas duas decisões.

Regra — a comparação com o quadrado é o idioma, não uma esperteza local

Toda vez que um teste envolve distância — colisão de esferas, raio de busca, vizinho mais próximo —, compare os quadrados. A raiz quadrada é monótona, de modo que $a < b \Leftrightarrow a^2 < b^2$ para valores não negativos, e a comparação sem raiz é sempre válida. O mesmo idioma reaparece na detecção de colisões da Seção VIII.1.

VI.4.2 Com `std::thread`: paralelismo por *tiles*

O subprojeto `src/mandelbrot/` divide o quadro em faixas horizontais, uma por *thread*, e usa exatamente os primitivos da Seção VI.1 para saber quando todas terminaram: um contador protegido por *mutex* e uma `condition_variable` com predicado.

```
    int rows_per_tile = h_ / num_threads_;
    int y0 = 0;

    for (int t = 0; t < num_threads_; ++t) {
        int y1 = (t == num_threads_ - 1) ? h_ : y0 + rows_per_tile;
        // Captura vp por valor para que cada thread tenha sua copia
        workers.emplace_back([this, y0, y1, vp](){
            compute_tile(y0, y1, vp);
        });
        y0 = y1;
    }

    // Aguarda todos os tiles via condition_variable
    {
        std::unique_lock<std::mutex> lk(done_mutex_);
        done_cv_.wait(lk, [this]{ return done_count_ == num_threads_; });
    }

    for (auto& w : workers) w.join();
```

Listagem 217: A divisão em faixas e a espera pelo término, em `src/mandelbrot/mandelbrot_renderer.cpp`.

O `y1` da última faixa é `h_`, e não `y0 + rows_per_tile`: é assim que as linhas que sobram da divisão inteira (h raramente é múltiplo do número de *threads*) vão todas para a última faixa em vez de simplesmente não serem calculadas. O erro oposto — deixar de tratar o resto — produz uma tira preta no rodapé da imagem que só aparece em certas resoluções.

Armadilha — capturar por referência o que vai morrer antes da *thread*

A *lambda* captura `vp` **por valor**, e o comentário no fonte diz por quê. Capturar por referência é o reflexo natural — o objeto é pequeno, copiá-lo parece desperdício — e é exatamente o que cria um ponteiro pendente quando o chamador retorna antes de a *thread* terminar. Numa `std::thread` que se junta na mesma função o risco é contido; num *thread pool*, num `std::async` ou num `detach` ele é certeza. A regra prática é a inversa da que vale para parâmetros de função: **o que atravessa uma fronteira de *thread* vai por valor**, a menos que se possa provar que o dono sobrevive.

Cada *thread* escreve num intervalo disjunto de linhas: sem sobreposição, não há *data race* — e a ausência de sincronização no caminho quente é o que faz o paralelismo valer a pena. Ainda assim, o renderizador toma um *mutex* uma vez por faixa, e a estrutura merece atenção porque é o padrão correto para o caso geral:

```
    // buffer local da tile (sem precisar do mutex durante o calculo)
    std::vector<uint8_t> local((y1 - y0) * w_ * 4);
    // ... o laco de escape preenche 'local' ...
    {
```

```
5           std::lock_guard<std::mutex> lock(tile_mutex_);
6           int dst_off = y0 * w_ * 4;
7           std::copy(local.begin(), local.end(), pixels_.begin() + dst_off);
8       }
9
10      // Sinaliza conclusao desta tile
11      {
12          std::lock_guard<std::mutex> lk(done_mutex_);
13          ++done_count_;
14      }
15      done_cv_.notify_one();
```

Listagem 218: Calcular fora do *lock*, copiar dentro, em `src/mandelbrot/mandelbrot_renderer.cpp`.

Todo o cálculo — que é 99% do tempo — acontece num `vector` local, sem *lock* nenhum; a região crítica contém apenas um `std::copy`. É a mesma decisão do *thread pool* da Seção VI.1, que executa a tarefa fora do *lock*, e ela responde à única pergunta que importa ao dimensionar uma seção crítica: *o que aqui dentro precisa mesmo estar aqui dentro?* O comentário do fonte é honesto ao dizer que, com faixas disjuntas, o *lock* é "educativo" — ele não protege contra corrida nenhuma neste desenho, e existe para mostrar a forma que se usa quando as regiões *não* são disjuntas.

Repare também no segundo bloco: o contador `done_count_` é incrementado sob o `done_mutex_` e a notificação sai *fora* do escopo do *lock*. Notificar com o *mutex* na mão faz a *thread* acordada esbarrar imediatamente num *mutex* tomado e voltar a dormir — o *hurry up and wait* clássico.

Armadilha — a divisão estática desequilibra justamente neste problema

Faixas de mesmo tamanho não significam trabalho de mesmo tamanho: os pixels *dentro* do conjunto rodam o número máximo de iterações, enquanto os de fora escapam em poucas. As faixas centrais custam muito mais que as das bordas, e o tempo total é o da *thread* mais lenta — de modo que boa parte dos núcleos fica ociosa esperando. É precisamente o problema que o *scheduling* dinâmico da próxima seção resolve.

VI.4.3 Com OpenMP: uma linha

O subprojeto `src/mandelbrot_openmp/` resolve o mesmo problema com uma diretiva:

```
1  #pragma omp parallel for schedule(dynamic, 4) num_threads(num_threads_)
2  for (int y = 0; y < h_; ++y) {
3      for (int x = 0; x < w_; ++x) {
4          /* ... o mesmo cálculo ... */
5      }
6  }
```

Listagem 219: A diretiva que substitui a criação, a divisão e a espera.

O `schedule(dynamic, 4)` é o que corrige o desequilíbrio: em vez de distribuir as linhas de antemão, cada *thread* pega o próximo lote de quatro linhas quando termina o atual. Lotes menores equilibram melhor e custam mais sincronização; quatro é um compromisso razoável para este problema.

Por que — por que o repositório mantém as duas versões

A comparação é o conteúdo. A versão com `std::thread` ensina o que acontece — criação, divisão, sincronização, `join` — e é a única em que se pode intervir quando o padrão não cobre o caso. A versão OpenMP mostra que, para o laço paralelo simples, tudo aquilo já foi escrito por alguém, com balanceamento melhor do que o que se faria à mão. Saber as duas é saber quando o custo de escrever

Tabela 86: As duas implementações do mesmo fractal, comparadas.

Aspecto	`std::thread`	OpenMP
código necessário	dezenas de linhas	uma diretiva
controle	total	limitado ao padrão
balanceamento	manual	`schedule(dynamic)`
portabilidade	STL	precisa de `-fopenmp`
depurável	sim, é código comum	o *runtime* é opaco

a primeira se justifica.

Do vocabulário OpenMP, vale reter quatro construções além do laço paralelo: `reduction(+:soma)`, que dá a cada *thread* um acumulador privado e combina no fim — é o `accumulate` paralelo, e é o motivo de `std::reduce` não garantir ordem (Seção III.1); `critical`, que é um *mutex* com sintaxe de bloco; `barrier`, ponto em que todas esperam; e `omp_get_max_threads()`, para dimensionar em tempo de execução.

Regra — meça o *speedup*, e espere menos que o número de núcleos

A lei de Amdahl limita o ganho pela fração serial: com p a fração paralelizável e N núcleos, o *speedup* é $S = 1/\left((1-p) + p/N\right)$. Para o Mandelbrot, $p \approx 0{,}99$ e o ganho fica próximo do ideal — é por isso que ele é um bom exemplo, e também por que ele é um mau preditor: quase nenhum problema real tem p tão alto.

VI.5 Síntese

Os cinco subprojetos ocupam pontos diferentes de um mesmo espectro, e a escolha entre eles é quase sempre determinada pela natureza do gargalo (Tabela 87).

Tabela 87: O que usar conforme o que está esperando.

O gargalo é	Ferramenta	Onde
cálculo, muitos núcleos ociosos	*threads* ou OpenMP	VI.4
espera por I/O	co-rotinas	VI.3
muitas tarefas curtas	*thread pool*	VI.1
um resultado de outra *thread*	`future`/`promise`	VI.1
um contador compartilhado	`std::atomic`	VI.1
uma invariante entre dois dados	*mutex* (não atômico)	VI.1
o fim de N tarefas, uma vez	`std::latch`	VI.2
rodadas repetidas em fases	`std::barrier`	VI.2
limitar a K acessos simultâneos	`counting_semaphore`	VI.2
cancelar uma *thread* em execução	`jthread` e `stop_token`	VI.2
a garantia de progresso, não a velocidade	estrutura *lock-free*	VI.2.5

Vale reunir num só lugar os quatro números que `src/concurrency2/` mediu, porque três deles contrariam o que se costuma afirmar sobre o assunto: `seq_cst` custou **1,01×** o `relaxed` num `fetch_add` sem contenção no x86-64 (a otimização mais repetida do ramo não pagou nada); reimplementar `fetch_add` com CAS gerou **43% a 60%** de retrabalho com quatro *threads*; a fila SPSC entregou **cerca de 90 milhões de itens por segundo** sem um único CAS; e a pilha *lock-free* ficou **2,7× mais lenta** que um `vector::push_back` sob *mutex*. Nenhum desses números é transportável para outra máquina ou outra arquitetura — e é justamente por isso que eles estão aqui: para mostrar o que medir, e com que ceticismo ler o resultado.

E a regra que atravessa o capítulo inteiro é sobre o que *não* fazer:

Regra — o dado que não é compartilhado não precisa de sincronização

Todo primitivo deste capítulo existe para gerenciar compartilhamento, e todo primitivo custa. O Mandelbrot não precisa de nenhum *mutex* no caminho quente porque as *threads* escrevem em regiões disjuntas; o *thread pool* executa a tarefa fora do *lock* pelo mesmo motivo. Antes de escolher o primitivo certo, pergunte se dá para eliminar o compartilhamento — por partição, por cópia, ou por passar o dado em vez de referenciá-lo.

VII Ecossistema: rede, dados e mensageria

A partir daqui a ordem é por interesse: todos os subprojetos deste capítulo dependem apenas do núcleo da linguagem (capítulos II a VI) e nenhum depende dos outros. O que eles têm em comum é outra coisa — são o ponto em que o C++ deixa de conversar apenas consigo mesmo e passa a conversar com o mundo: com um *socket*, com um servidor HTTP, com um banco, com um *broker* de mensagens, com a nuvem.

E por isso trazem um assunto que os capítulos anteriores só tocaram de leve: a **falha externa**. Uma conversão de tipo errada é um bug do programa; uma conexão recusada não é. A Seção II.8 apresentou as formas de reportar erro; aqui elas aparecem em uso, e as cinco bibliotecas fizeram cinco escolhas diferentes — o que, por si só, já é uma lição.

VII.1 Antes de tudo: estes alvos são construídos

Edições anteriores deste relatório abriam o capítulo avisando que os subprojetos aqui descritos vinham com a linha `subdir()` comentada em `src/meson.build` e precisavam ser habilitados à mão. **Isso não vale mais, e a mudança é o assunto mais importante da seção.**

Regra — nenhum alvo deste repositório está comentado

Os 57 subprojetos de `src/` são construídos por padrão, inclusive os "pesados" — Qt, SFML, Bullet, ONNX, JSBSim, AWS, RabbitMQ, SOCI. O `make build` produz **98 binários** instalados em `dist/bin`, compilando com **zero avisos** sob `-Wall -Wextra -Wpedantic`. Vários programas deste capítulo ainda exigem um *recurso externo em execução* — conexão com a internet (VII.2, VII.3), um *broker* RabbitMQ (VII.5) ou credenciais AWS (VII.6) — mas isso é condição de *execução*, nunca mais de *compilação*.

Por que — um alvo que não é construído é um alvo que apodrece

Comentar um `subdir()` parece inofensivo: o alvo continua no disco, o código continua lá, e o *build* fica mais rápido. O que se perde é a única coisa que impedia o código de se degradar — a passagem do compilador. Ao religar os alvos deste repositório, seis defeitos reais apareceram nos primeiros minutos. Nenhum deles era novo: todos estavam ali havia meses, esperando exatamente o comando que nunca era dado.

O Seção V lista os seis na Tabela 78. Quatro são `#include` faltando — a classe de defeito mais banal que existe, e por isso mesmo a prova mais limpa do argumento: nem o mais trivial dos erros é pego por um alvo que ninguém compila. Dois são de ABI, e destes o de alinhamento merece um comentário, porque é a assinatura de uma família inteira de falhas. O System V ABI de x86-64 exige `RSP` alinhado a 16 bytes *no momento* do `CALL`. Um `subq $8, %rsp` a mais desalinha o ponteiro; a função chamada não percebe nada, e a seguinte também não — até que alguém, quatro quadros acima, execute uma instrução SSE que exige operando alinhado. O *crash* apareceu *dentro do fmt*.

Armadilha — o bug de alinhamento estoura em quem usa SSE, não em quem o quebrou

Essa é a marca registrada da falha de ABI: a distância entre a causa e o sintoma. Nenhuma pilha de chamadas aponta para o culpado, porque o culpado já retornou. Se um *crash* aparece dentro de uma biblioteca madura e bem testada, a hipótese mais provável não é um bug dela — é que alguém violou o contrato de chamada antes de chegar lá. O mesmo raciocínio vale para o outro: prometer ao compilador que um registrador é "só entrada" e então modificá-lo é mentir para o otimizador, e o

otimizador *acredita*.

Os testes seguiram o mesmo destino do resto. O diretório `tests/` da raiz — que estava 100% comentado e nunca construiu nada — foi **removido**, assim como o `sandbox/` da raiz. Os testes agora moram junto do que testam: `src/testing/` (5 alvos, 41 casos), `src/meson/` (1 alvo) e `src/cmake/` (CTest, 11 casos). O `meson test` executa 6 alvos, todos passando.

Regra — um alvo que não compila é um problema do alvo

Se um subprojeto não pode ser construído, conserte-o ou apague-o. Não o esconda atrás de um `#`. O único motivo legítimo para manter um alvo fora do *build* padrão é a dependência ser genuinamente indisponível na máquina — e mesmo aí a resposta certa é a detecção graciosa (`required: false` mais um `#if`), como faz o backend PostgreSQL do Seção VII.4, não o comentário.

VII.2 Sockets com Asio

O subprojeto `src/network/` usa Asio [8] — a biblioteca que deu origem à proposta de rede do padrão — para cobrir TCP síncrono, TCP assíncrono, UDP, temporizadores e HTTP construído à mão sobre TCP. São cinco demonstrações independentes, executadas em sequência pelo `main.cpp`, e é o melhor lugar para vir logo depois das co-rotinas (Seção VI.3), porque é onde elas pagam.

VII.2.1 O vocabulário mínimo, definido antes de ser usado

Rede tem um jargão pequeno e bem definido, e quase todo mal-entendido em código de *socket* vem de um destes termos ter sido lido por alto.

- **Socket** — a "tomada de rede". É o descritor pelo qual dois programas trocam bytes. Do lado do sistema operacional é um número inteiro; no Asio é um objeto com RAII, que fecha o descritor no destrutor.
- **Porta** — um número de 0 a 65535 que identifica *qual* serviço, dentro de uma máquina, deve receber os bytes. O endereço IP escolhe a máquina; a porta escolhe o programa.
- **Endpoint** — o par (endereço, porta). É o que se passa a `connect` ou a `bind`.
- **Resolver** — traduz um nome (`httpbin.org`) numa lista de *endpoints*. É o `getaddrinfo` do POSIX, embrulhado. A lista tem mais de um elemento porque um nome pode ter IPv4 e IPv6, e vários endereços em cada família.
- **Acceptor** — do lado do servidor, o objeto que fica ouvindo numa porta. Equivale à sequência `socket` + `bind` + `listen` do BSD, feita no construtor.
- **Handler** — em código assíncrono, a função chamada quando uma operação termina. Recebe sempre um `error_code` como primeiro argumento, porque não há para onde propagar uma exceção.

VII.2.2 O `io_context` é o *event loop*

Toda a arquitetura do Asio gira em torno de um objeto: o `io_context`. Ele é a fila de trabalho, o despachante e o dono das operações pendentes. `io.run()` processa os *handlers* das operações concluídas e **retorna quando não há mais trabalho pendente** — o que é a primeira surpresa de quem começa.

Em código síncrono o `io_context` é quase transparente: existe porque todo objeto de I/O precisa de um, mas nunca se chama `run()`. Em código assíncrono ele é o programa inteiro.

Armadilha — `io.run()` retorna imediatamente se nada foi enfileirado antes

O `io_context` não fica esperando: ele processa o que existe e sai. Um servidor que chame `run()` antes de registrar o primeiro *accept* simplesmente encerra. Para manter o laço vivo enquanto operações ainda serão enfileiradas de outra *thread*, existe o `work_guard` — e liberá-lo (`guard.reset()`) é o que permite ao `run()` terminar.

O subprojeto mede exatamente isso. A demonstração 4.5 monta um *pool* de três *threads* rodando o mesmo `io_context`, posta seis tarefas e só então libera o *guard*:

```
asio::io_context io;
auto guard = asio::make_work_guard(io);   // io.run() nao vai retornar

std::vector<std::thread> pool;
for (int i = 0; i < 3; ++i)
    pool.emplace_back([&io]() { io.run(); });

for (int i = 0; i < 6; ++i)
    asio::post(io, [i]() { fmt::print("  tarefa {} executada\n", i); });

std::this_thread::sleep_for(50ms);
guard.reset();                            // agora sim: run() pode retornar
for (auto& t : pool) t.join();
```

Listagem 220: `src/network/demo_timer.hpp`: o *guard* é o que separa "sem trabalho agora" de "sem trabalho nunca mais".

```
-- 4.5 io_context multithreaded com work_guard --
  tarefa 0 executada
  tarefa 1 executada
  tarefa 2 executada
  tarefa 3 executada
  tarefa 4 executada
  tarefa 5 executada
  6 tarefas executadas por 3 threads
```

Listagem 221: Saída medida de `./dist/bin/network`, demo 4.5. As seis tarefas foram distribuídas entre as três *threads* do *pool*.

VII.2.3 TCP síncrono: um servidor de eco em trinta linhas

A versão síncrona é a que se lê como um programa comum: aceitar, ler, escrever, fechar. Cada chamada bloqueia a *thread* até completar. O subprojeto sobe um servidor de eco numa *thread* e um cliente na principal, ambos na porta 12345.

```
asio::io_context io;
tcp::acceptor acceptor(io, tcp::endpoint(tcp::v4(), porta));

tcp::socket socket(io);
acceptor.accept(socket);                  // BLOQUEIA ate um cliente conectar
```

```
6
7  while (true) {
8      asio::streambuf buf;
9      std::size_t n = asio::read_until(socket, buf, '\n');    // BLOQUEIA
10
11     std::string linha(asio::buffers_begin(buf.data()),
12                       asio::buffers_begin(buf.data())
13                           + static_cast<std::ptrdiff_t>(n));
14     buf.consume(n);
15
16     if (linha == "FIM\n") { asio::write(socket, asio::buffer("BYE\n", 4)); break; }
17     asio::write(socket, asio::buffer(linha));                  // ecoa de volta
18 }
```

Listagem 222: `src/network/demo_tcp_sync.hpp`: o servidor inteiro. `acceptor` no construtor faz `bind` e `listen`; `accept` bloqueia.

Três detalhes carregam mais do que aparentam. O `streambuf` é um *buffer* de crescimento automático, e `read_until` pode ler *mais* do que a linha pedida — por isso o `n` devolvido delimita a linha e o `consume(n)` descarta só ela, deixando o resto para a próxima iteração. Quem trata o `streambuf` como se contivesse exatamente uma linha escreve um servidor que perde mensagens sob carga.

Armadilha — `read_until` não devolve o tamanho do que foi lido

Ele devolve a posição do delimitador. Os bytes que chegaram *depois* do `\n` continuam no `streambuf`, e é responsabilidade do chamador não perdê-los. TCP é um fluxo de bytes, não de mensagens: o protocolo não tem noção de "mensagem", e enquadrar o fluxo — por delimitador, por prefixo de tamanho, por formato fixo — é trabalho da aplicação.

VII.2.4 As duas APIs de erro

Asio oferece, para cada operação, uma versão que lança `asio::system_error` e outra que preenche um `asio::error_code&`. É exatamente a API dupla que a Seção III.3 descreveu para o `<filesystem>`, e pelo mesmo motivo: uma conexão recusada é rotina em um servidor e é excepcional em um cliente que roda uma vez.

```
1  asio::error_code ec;
2  asio::connect(socket, endpoints, ec);
3  if (ec) {
4      fmt::print("  connect falhou: {}\n", ec.message());
5      fmt::print("  categoria: '{}', valor: {}\n", ec.category().name(), ec.value());
6  }
```

Listagem 223: `src/network/demo_tcp_sync.hpp`, demo 1.3: a mesma chamada, sem exceção. O `error_code` carrega categoria e valor numérico, não só a mensagem.

```
1  -- 1.3 Error handling: error_code (sem excecao) --
2    connect falhou (esperado — porta 9999 fechada): Connection refused
3    categoria: 'asio.system', valor: 111
```

Listagem 224: Saída medida. O valor 111 é o `ECONNREFUSED` do Linux — o `error_code` preserva o código do sistema, não só o texto.

Regra — `error_code` nos *handlers*, exceção nas chamadas de topo

Nas operações assíncronas a escolha nem existe: o *handler* recebe um `error_code` porque não há para onde propagar uma exceção — ela escaparia do `io.run()` e derrubaria o laço de eventos. Reserve a versão que lança para o código síncrono de inicialização, onde falhar cedo e ruidoso é o comportamento

desejado.

Um caso especial merece nome próprio: `asio::error::eof` não é um erro, é o fim normal da conversa. O servidor fechou a conexão, e é assim que `Connection: close` do HTTP/1.0 sinaliza o fim do corpo. Tratar `eof` como falha produz um cliente que reporta erro em toda requisição bem-sucedida — e o código do subprojeto o filtra explicitamente em três lugares diferentes.

VII.2.5 TCP assíncrono e o problema do tempo de vida

A versão assíncrona tem exatamente a mesma lógica, escrita ao contrário: cada etapa vira um *callback* aninhado dentro do anterior, e o fluxo do programa some dentro da indentação.

```
resolver.async_resolve("httpbin.org", "80",
    [&](asio::error_code ec, asio::ip::tcp::resolver::results_type endpoints) {
        if (ec) return;
        asio::async_connect(socket, endpoints,
            [&](asio::error_code ec, asio::ip::tcp::endpoint) {
                if (ec) return;
                asio::async_write(socket, asio::buffer(*req),
                    [&, req](asio::error_code ec, std::size_t) {
                        if (ec) return;
                        // e a leitura entra mais um nivel...
                    });
            });
    });
```

Listagem 225: A mesma sequência — resolver, conectar, escrever — em três níveis de aninhamento.

Por que — por que trocar clareza por isso

Um servidor síncrono precisa de uma *thread* por conexão. Mil conexões são mil *threads*: mil pilhas de 8 MiB reservadas, mil entradas no escalonador, troca de contexto a cada I/O. Um servidor assíncrono atende as mesmas mil conexões com uma *thread* e mil operações registradas, porque a *thread* só é ocupada quando há bytes prontos. O custo é escrever o programa de dentro para fora.

O ponto difícil do modelo assíncrono não é o aninhamento — é o tempo de vida. Quando `async_read` retorna, a operação ainda não aconteceu; tudo o que ela referencia precisa continuar vivo até o *handler* rodar. O subprojeto resolve isso com o idioma canônico do Asio, que vale memorizar inteiro:

```
class Sessao : public std::enable_shared_from_this<Sessao> {
    void ler() {
        asio::async_read_until(socket_, buf_entrada_, '\n',
            // a captura por copia do shared_ptr e o que mantem *this vivo
            [self = shared_from_this()](asio::error_code ec, std::size_t bytes) {
                self->ao_ler(ec, bytes);
            });
        // async_read_until retorna IMEDIATAMENTE aqui
    }

    void ao_ler(asio::error_code ec, std::size_t bytes) {
        if (ec) return;          // nao re-registra: o ultimo shared_ptr morre
                                 // com o handler, e a Sessao e destruida
        // ... processa ...
        escrever();              // registra a proxima operacao: continua viva
    }
};
```

Listagem 226: `src/network/demo_tcp_async.hpp`: a sessão se mantém viva sozinha. Cada *handler* carrega um `shared_ptr` para o próprio objeto.

O mecanismo é elegante e não óbvio: enquanto houver uma operação pendente, existe um `shared_ptr` vivo dentro do *handler* registrado no `io_context`. Quando um *handler* decide não registrar mais nada, ele é o último dono — e o objeto morre exatamente ali, sem que ninguém precise chamar `delete`. O contador de referências (Seção II.5) está sendo usado como *máquina de estados*.

Armadilha — o *buffer* precisa sobreviver à operação assíncrona

Repare no `req` capturado por valor num `shared_ptr` na listagem anterior: a chamada a `async_write` **retorna imediatamente**, e o *buffer* passado a ela precisa continuar vivo até o *handler* rodar. Uma `std::string` local morreria antes disso, e o Asio escreveria memória liberada — um *use-after-free* que só se manifesta sob carga, quando a escrita não completa de imediato. É a mesma classe de erro que a Seção III.4 descreve para `span` e `string_view`, e é o motivo de `asio::buffer` *nunca* copiar: ele guarda ponteiro e tamanho, como um `std::span`.

O servidor assíncrono do subprojeto acrescenta o segundo idioma indispensável: o *accept* que se re-registra.

```
1  void aceitar() {
2      acceptor_.async_accept(
3          [this](asio::error_code ec, tcp::socket socket) {
4              if (!ec)
5                  std::make_shared<Sessao>(std::move(socket))->iniciar();
6              aceitar();        // re-registra para a proxima conexao
7          });
8  }
```

Listagem 227: `src/network/demo_tcp_async.hpp`: sem a última linha, o servidor atende exatamente um cliente e encerra.

Uma *thread*, três clientes simultâneos, e o registro de cada sessão intercalado na saída — que é a evidência de que nada está bloqueando:

```
1  -- 2.1 Servidor assincrono (multiplas conexoes, 1 thread) --
2    [server async] ouvindo na porta 12346
3    [sessao 1] cliente conectado de 127.0.0.1:32794
4    [sessao 1] recebido: 'hello do cliente 1'
5    [sessao 2] cliente conectado de 127.0.0.1:32810
6    [cli 1] 'hello do cliente 1' -> 'ECHO: hello do cliente 1'
7    [sessao 2] recebido: 'hello do cliente 2'
8    [sessao 1] recebido: 'FIM'
9    [cli 2] 'hello do cliente 2' -> 'ECHO: hello do cliente 2'
10   [cli 1] 'FIM' -> 'BYE'
11   [sessao 3] cliente conectado de 127.0.0.1:32816
```

Listagem 228: Saída medida, demo 2.1. As sessões 1, 2 e 3 progridem intercaladas numa única *thread*.

VII.2.6 `post` e `strand`: concorrência sem *mutex*

Duas peças menores do Asio resolvem problemas que a Seção VI.1 resolveu com *mutex*, e vale ver a alternativa.

`asio::post(io, f)` enfileira `f` para executar no `io_context`. É seguro chamá-lo de qualquer *thread*, e é a forma canônica de mandar trabalho para o laço de eventos sem compartilhar estado — o padrão de passagem de mensagem em vez de memória compartilhada.

`strand` vai além: é um executor que garante que *seus* handlers nunca rodam em paralelo, mesmo que várias *threads* chamem `io.run()` sobre o mesmo contexto. O subprojeto demonstra o resultado com um contador não atômico:

```
asio::io_context io;
asio::io_context::strand strand(io);
int contador = 0;                    // sem atomic, sem mutex

for (int i = 0; i < 5; ++i)
    asio::post(strand, [i, &contador]() { ++contador; /* ... */ });

std::thread t1([&io]() { io.run(); });    // duas threads no mesmo io_context
std::thread t2([&io]() { io.run(); });
t1.join(); t2.join();
// resultado final = 5 (esperado 5, sem race condition)
```

Listagem 229: `src/network/demo_tcp_async.hpp`, demo 2.3: `contador` é um `int` comum, e mesmo assim não há corrida.

Regra — *strand* substitui o *mutex*, não a disciplina

O *strand* serializa apenas os *handlers* postados *nele*. Uma variável tocada por um *handler* do *strand* e por código fora dele continua sujeita a *data race*. A vantagem sobre o *mutex* é que não há bloqueio: a *thread* nunca espera, ela simplesmente executa outro *handler* enquanto o *strand* está ocupado. É a diferença entre serializar por exclusão e serializar por agendamento.

VII.2.7 TCP e UDP

O subprojeto cobre os dois porque a escolha entre eles é de arquitetura, não de API (Tabela 88). UDP não tem *handshake*, não retransmite e não garante ordem — e é por isso que DNS, jogos e vídeo ao vivo o preferem: para esses, uma resposta atrasada é pior que uma resposta perdida.

Tabela 88: As garantias que o TCP dá e o UDP não.

Característica	TCP	UDP
conexão	sim (*handshake*)	não
confiabilidade	sim (retransmissão)	não
ordem	sim	não garantida
unidade	fluxo de bytes	datagrama
overhead	maior	menor
casos de uso	HTTP, banco de dados	DNS, jogos, vídeo ao vivo

A diferença de *unidade* é a que mais confunde. Em TCP não existe "mensagem": dois `write` de 10 bytes podem chegar como um `read` de 20, ou como três de 7, 7 e 6. Em UDP existe: um `send_to` de 10 bytes chega como um `receive_from` de 10 bytes — ou não chega. Isso simplifica o enquadramento e complica tudo o mais.

Há um terceiro modo, que o subprojeto demonstra e quase ninguém conhece: o *socket* UDP *conectado*.

```
udp::socket sock(io, udp::endpoint(udp::v4(), 0));    // porta 0 = SO escolhe

// Nao ha handshake: connect() so registra o endpoint remoto padrao
sock.connect(udp::endpoint(asio::ip::make_address("127.0.0.1"), PORTA));

sock.send(asio::buffer(std::string("ping")));    // send(), nao send_to()
std::array<char, 64> buf{};
std::size_t n = sock.receive(asio::buffer(buf));
```

Listagem 230: `src/network/demo_udp.hpp`, demo 3.3: `connect` em UDP não faz *handshake* nenhum.

Por que — para que serve conectar um socket sem conexão

Duas coisas concretas. Primeiro, o *kernel* passa a descartar datagramas vindos de qualquer outro *endpoint* — um filtro de graça, feito no sistema em vez de no seu `if`. Segundo, erros ICMP ("porta inalcançável") passam a ser reportados ao *socket*, o que num UDP não conectado é impossível: sem destino fixo, o sistema não sabe a qual envio o erro pertence. O nome "conectado" é infeliz; "endereço remoto padrão" seria melhor.

VII.2.8 Temporizadores

O `steady_timer` é a peça que substitui `sleep` em código assíncrono: em vez de bloquear a *thread*, ele agenda um *handler*. O padrão do temporizador periódico — reagendar dentro do próprio *handler* — é o esqueleto de qualquer *heartbeat*, *keep-alive* ou tentativa com *backoff*.

```
std::function<void(asio::error_code)> tick;
tick = [&](asio::error_code ec) {
    if (ec) return;                          // cancelado: nao reagenda
    fmt::print("   tick #{}\n", ++ticks);
    if (ticks < MAX) {
        timer.expires_after(15ms);           // reconfigura
        timer.async_wait(tick);              // re-registra
    }
    // quando ticks == MAX: nao re-registra -> io.run() retorna sozinho
};
timer.expires_after(15ms);
timer.async_wait(tick);
io.run();
```

Listagem 231: `src/network/demo_timer.hpp`, demo 4.4: o `std::function` é necessário porque o *lambda* precisa se referenciar.

Por que — por que `steady_clock` e não `system_clock`

O relógio do sistema pode ser ajustado — por NTP, pelo administrador, pelo horário de verão — e pode andar para trás. Um *timeout* de cinco segundos medido contra ele pode virar cinco horas ou disparar imediatamente. O `steady_clock` é monotônico: só anda para a frente, e a taxa não muda. É a escolha certa para *qualquer* medida de duração, dentro ou fora do Asio.

O cancelamento tem uma sutileza que aparece na demonstração 4.3: `timer.cancel()` não impede o *handler* de rodar — ele roda, com `ec == asio::error::operation_aborted`.

Armadilha — cancelar não é desregistrar

Todo *handler* registrado no Asio *será* chamado, exatamente uma vez. Cancelar apenas troca o `error_code` com que ele é chamado. Um *handler* que ignore o `ec` e execute a ação mesmo assim faz o programa se comportar como se o cancelamento não tivesse existido — e essa é a origem clássica do "fechei a conexão mas o *timeout* disparou de todo jeito". A primeira linha de todo *handler* é o teste do `ec`.

VII.2.9 HTTP cru: o protocolo é texto

A quinta demonstração constrói uma requisição HTTP/1.1 à mão sobre TCP, e o objetivo é desmistificar: HTTP/1.1 é uma conversa de *strings* com formato fixo, e nada mais.

```
std::string req =
    "GET /get HTTP/1.1\r\n"
    "Host: httpbin.org\r\n"
    "User-Agent: cpp_academy/1.0\r\n"
    "Accept: application/json\r\n"
    "Connection: close\r\n"
    "\r\n";
asio::write(socket, asio::buffer(req));
```

Listagem 232: `src/network/demo_http_raw.hpp`: a requisição inteira. A linha vazia final é obrigatória — é ela que marca o fim dos cabeçalhos.

Armadilha — o terminador de linha do HTTP é `\r\n`, não `\n`

E o cabeçalho `Host` não é opcional no HTTP/1.1: sem ele, um servidor que hospede vários domínios no mesmo IP não tem como saber qual site você quer, e responde `400 Bad Request`. Os dois erros produzem falhas que parecem de rede e são de formatação.

O `Connection: close` escolhe deliberadamente a estratégia mais simples de saber onde o corpo termina: o servidor fecha o *socket*. É por isso que o código lê com `asio::transfer_all()` e trata `eof` como sucesso. Um cliente de produção não pode fazer isso — perderia a reutilização de conexão — e precisa interpretar `Content-Length` ou o *chunked transfer encoding*.

```
-- 5.1 HTTP GET sincrono (httpbin.org) --
  status: 200 OK
  body (200 chars):
    {
  "args": {},
  "headers": {
    "Accept": "application/json",
    "Host": "httpbin.org",
    "User-Agent": "cpp_academy/1.0",

-- 5.2 HTTP GET assincrono --
  [async] 200 OK — body: { "uuid": "2c6523ea-c322-42ca-990a-17e1564e6102" }
```

Listagem 233: Saída medida das demos 5.1 e 5.2, com conectividade. Os dois caminhos — síncrono e assíncrono — produzem o mesmo resultado.

A demonstração termina com uma lista honesta do que *falta* para isso virar um cliente de verdade: TLS, redirecionamentos, *chunked encoding*, *keep-alive*, compressão, *cookies*, autenticação, HTTP/2, *timeouts* e *retry*. Todos os dez itens estão implementados na libcurl, e é exatamente essa lista que justifica a existência da próxima seção.

Regra — implemente o protocolo para entendê-lo, use a biblioteca para produção

Escrever HTTP cru uma vez ensina mais sobre depurar HTTP do que qualquer documentação. Manter HTTP cru em produção significa reimplementar, com menos testes, a lista acima. As duas afirmações são compatíveis, e o repositório mantém os dois subprojetos lado a lado justamente para não deixar dúvida sobre qual usar quando.

VII.2.10 A armadilha do macro que o Conan já definiu

Um detalhe de compilação, pequeno e instrutivo, aparece no topo dos cinco cabeçalhos do subprojeto:

```
#ifndef ASIO_STANDALONE
#  define ASIO_STANDALONE
```

```
3 #endif
4 #include <asio.hpp>
```

Listagem 234: `src/network/demo_tcp_sync.hpp`: o guarda não é zelo excessivo.

Armadilha — redefinir um macro com valor diferente é violação de ODR silenciosa

O pacote Asio do Conan 2 já define `ASIO_STANDALONE` na linha de compilação. Redefini-lo no cabeçalho *com o mesmo valor* ainda dispara `warning: "ASIO_STANDALONE" redefined` — e este repositório compila com zero avisos. Redefini-lo com valor *diferente* seria pior: duas unidades de tradução enxergariam declarações incompatíveis da mesma biblioteca, e o *linker* escolheria uma delas em silêncio. O guarda mantém o cabeçalho correto sozinho *e* sob o Conan.

VII.2.11 As co-rotinas, que é onde tudo isso converge

A resposta do C++20 ao aninhamento é escrever a mesma coisa como co-rotina (C++20): o fluxo volta a ser linear, e cada `co_await` é um ponto de suspensão em vez de um *callback*.

```
1 asio::co_spawn(io, [&]() -> asio::awaitable<void> {
2     auto [ec, endpoints] = co_await resolver.async_resolve("httpbin.org", "80",
3                                      asio::as_tuple(asio::use_awaitable));
4     co_await asio::async_connect(socket, endpoints, asio::use_awaitable);
5     co_await asio::async_write(socket, asio::buffer(request), asio::use_awaitable);
6     // ...
7 }, asio::detached);
```

Listagem 235: A versão com co-rotinas: a indentação some, a lógica fica. O tempo de vida dos *buffers* também deixa de ser problema — eles são variáveis locais do quadro da co-rotina.

Vale notar o ganho secundário, que quase nunca é mencionado: o problema de tempo de vida da listagem anterior *desaparece*. Variáveis locais de uma co-rotina vivem no quadro alocado dela, e o quadro sobrevive à suspensão. O `shared_ptr` em volta do *buffer* deixa de ser necessário porque a linguagem passou a gerenciar exatamente o que o programador gerenciava à mão.

VII.3 Cliente HTTP e JSON

O subprojeto `src/http-client/` é a camada de aplicação sobre o que o anterior mostrou em *sockets*: libcurl para o transporte, nlohmann/json para os dados. E o que ele ensina de mais transferível não é HTTP — é como se envolve uma biblioteca C em uma interface C++.

VII.3.1 A interface, e o que ela decide

```
1  namespace http {
2
3  using Headers = std::map<std::string, std::string>;
4
5  struct Response {
6      long        status_code;
7      std::string body;
8      Headers     headers;
9  };
10
11 struct HttpError : std::runtime_error {
```

```
12      using std::runtime_error::runtime_error;
13  };
14
15  // GET sincrono. Lanca HttpError em caso de falha de TRANSPORTE.
16  // Erros HTTP (4xx, 5xx) sao retornados normalmente no status_code.
17  Response get(const std::string& url, const Headers& request_headers = {});
18
19  } // namespace http
```

Listagem 236: `src/http-client/http_client.hpp`: a superfície pública inteira, com um valor padrão que dispensa a segunda sobrecarga.

Três decisões estão embutidas aí. A primeira é que `HttpError` deriva de `std::runtime_error` e herda o construtor com `using` — o idioma da Seção II.8 para dar ao domínio uma raiz própria sem escrever um construtor. A segunda é que `Headers` é um `std::map`, e não um `std::unordered_map`: cabeçalhos HTTP são poucos (dezenas), a ordenação torna a saída determinística, e a diferença de desempenho é irrelevante diante de uma ida à rede. A terceira é a mais importante:

Regra — status HTTP não é erro de transporte

`http::get` lança `HttpError` apenas em falhas de **transporte**: sem conexão, certificado inválido, *timeout*. Um `404` chega normalmente, com `status_code == 404` — porque o servidor *respondeu*, e a resposta é a informação. Confundir as duas categorias é o erro de projeto mais comum em invólucro de HTTP: transformar `404` em exceção obriga o chamador a usar `try/catch` para o controle de fluxo normal.

A demonstração 4 do subprojeto existe só para provar isso, e o resultado medido é o esperado — nenhuma exceção, corpo vazio, status 404:

```
1  -- GET /todos/9999 (404) --
2  status: 404
3  body  : {}
```

Listagem 237: Saída medida de `./dist/bin/http_client`. O `404` percorreu o caminho normal.

VII.3.2 Envolver uma API C, com RAII em vez de disciplina

A libcurl tem a forma típica de uma biblioteca C: um *handle* opaco criado por `curl_easy_init()`, configurado por dezenas de chamadas a `curl_easy_setopt`, executado por `curl_easy_perform` e liberado por `curl_easy_cleanup`. A lista de cabeçalhos é uma lista encadeada própria, `curl_slist`, com o seu próprio `curl_slist_free_all`.

A tradução ingênua — a que aparece no README do subprojeto, e em quase todo tutorial de libcurl — libera os dois recursos no fim da função. É código que vaza silenciosamente:

```
1  CURL* curl = curl_easy_init();
2  curl_slist* slist = nullptr;
3  // ... setopt ...
4  CURLcode res = curl_easy_perform(curl);
5  if (res != CURLE_OK)
6      throw HttpError(curl_easy_strerror(res));   // vaza curl E slist
7
8  curl_slist_free_all(slist);
9  curl_easy_cleanup(curl);
```

Listagem 238: A versão ingênua. Se `curl_easy_perform` falhar, o `throw` pula as duas liberações.

Armadilha — o vazamento está exatamente no caminho de erro

E o caminho de erro de um cliente HTTP não é raro: é a rede caindo. Um servidor que faça mil requisições por minuto e falhe em 1% delas vaza dez *handles* por minuto — cada um com *socket*, contexto TLS e *buffers*. O sintoma aparece horas depois, como esgotamento de descritores, longe da causa.

A implementação real do subprojeto não faz isso. Ela embrulha cada recurso num tipo com destrutor, e o problema deixa de existir:

```
1  struct CurlHandle {
2      CURL* h;
3      CurlHandle() : h(curl_easy_init()) { if (!h) throw HttpError("curl_easy_init failed"); }
4      ~CurlHandle() { curl_easy_cleanup(h); }
5
6      CurlHandle(const CurlHandle&)            = delete;   // nao copiavel:
7      CurlHandle& operator=(const CurlHandle&) = delete;   // dois cleanup = UB
8  };
9
10 struct CurlSlist {
11     curl_slist* list = nullptr;
12     void append(const std::string& h) { list = curl_slist_append(list, h.c_str()); }
13     ~CurlSlist() { if (list) curl_slist_free_all(list); }
14 };
```

Listagem 239: `src/http-client/http_client.cpp`: dois envelopes de quatro linhas resolvem a categoria inteira.

Regra — todo recurso de biblioteca C ganha um envelope antes de ser usado

A regra é mecânica: para cada par `criar`/`destruir` de uma API C, escreva um tipo cujo construtor cria e cujo destrutor destrói, e desabilite a cópia. São quatro linhas, e elas eliminam de uma vez todos os caminhos de saída — `return` antecipado, exceção, `break` — que a disciplina manual precisa cobrir um a um. Onde o recurso é um ponteiro simples, um `unique_ptr` com *deleter* customizado (Seção II.5) faz o mesmo sem escrever a classe.

Vale registrar o que o subprojeto configura, porque cada opção é uma decisão de segurança ou de robustez, e nenhuma delas é o padrão da libcurl por acaso:

Tabela 89: As opções que `http::get` define, e o que cada uma evita. Valores extraídos de `src/http-client/http_client.cpp`.

Opção	Valor	O que evita
`FOLLOWLOCATION`	1	tratar um `301` como resposta final
`MAXREDIRS`	5	laço infinito de redirecionamento
`CONNECTTIMEOUT`	10 s	travar num servidor que não responde ao SYN
`TIMEOUT`	30 s	travar num servidor que responde devagar para sempre
`SSL_VERIFYPEER`	1	aceitar certificado de qualquer um
`SSL_VERIFYHOST`	2	aceitar certificado válido *de outro domínio*

Armadilha — desligar a verificação de TLS é o atalho que vira brecha

`SSL_VERIFYPEER = 0` é a primeira resposta que qualquer busca oferece para "certificate verify failed". Ela transforma HTTPS em HTTP com passos extras: a conexão continua cifrada, mas contra ninguém — qualquer interceptador apresenta o próprio certificado e é aceito. A correção certa é apontar o *bundle* de certificados (`CURLOPT_CAINFO`), nunca desligar a verificação.

O *callback* de cabeçalhos é a outra metade do invólucro, e faz uma coisa que a especificação exige e quase todo cliente esquece: normaliza a chave para minúsculas, porque nomes de cabeçalho HTTP são *case-insensitive*.

Sem isso, `headers.at("content-type")` funciona com um servidor e falha com o próximo.

VII.3.3 Dois bugs num separador de tela

A função que imprime os separadores do `main.cpp` tinha *dois* defeitos, e os dois eram silenciosos. São material de primeira porque nenhum tem a ver com HTTP: são sobre o que um `char` é e sobre o que `size()` devolve.

```
1 void print_separator(const std::string& title) {
2     fmt::print(fmt::emphasis::bold, "\n-- {} ", title);
3     std::string linha(40 - title.size(), '-');    // (a) e (b)
4     fmt::print("{}\n", linha);
5 }
```

Listagem 240: A versão com os dois bugs. Uma linha, dois defeitos de categorias diferentes.

O defeito **(a)** está no literal `'-'`. Esse caractere é U+2500 BOX DRAWINGS LIGHT HORIZONTAL, e em UTF-8 ele ocupa **três bytes**. Entre aspas simples ele não é um `char`: é uma *multi-character constant*, um `int` de valor definido pela implementação, que ao ser convertido para `char` é truncado. O GCC diz exatamente isso, com `-Wmultichar` e `-Woverflow`:

```
1 warning: multi-character character constant [-Wmultichar]
2 warning: overflow in conversion from 'int' to 'char'
3           changes value from '14849152' to '-128' [-Woverflow]
```

Listagem 241: O aviso do GCC. Note o valor: 14849152 é o inteiro de três bytes que o literal produziu.

Regra — um `char` guarda um byte, não um caractere

Todo caractere fora do ASCII precisa de uma *string*, não de um literal de caractere. `"-"` é um `const char[4]` (três bytes mais o terminador) e funciona; `'-'` não. A correção do subprojeto é concatenar a *string* num laço, reservando três bytes por repetição — e a reserva explícita documenta, no próprio código, que um símbolo desenhado não custa um byte.

O defeito **(b)** é aritmético e mais perigoso. `title.size()` devolve `std::size_t`, que é *sem sinal*. A expressão `40 - title.size()` com um título de 41 caracteres não vale -1: vale $2^{64} - 1 = 18\,446\,744\,073\,709\,551\,615$. O construtor de `std::string` tentaria alocar dezesseis exabytes e lançaria `std::length_error` — se tivesse sorte.

Armadilha — subtrair de um `size_t` é a conversão implícita mais cara do C++

O `40` literal é `int`; `title.size()` é `std::size_t`. Nas conversões aritméticas usuais, o operando com sinal é convertido para *sem* sinal, não o contrário. O resultado nunca é negativo — ele dá a volta. E a expressão não gera aviso nenhum por padrão, porque a linguagem considera isso perfeitamente definido. A correção é testar antes de subtrair, que é o que o código faz hoje: `title.size() < 40 ? 40 - title.size() : 0`.

VII.3.4 Mapeamento automático entre JSON e *structs*

O que torna nlohmann/json produtivo é uma macro que gera as duas conversões a partir da lista de campos, sem tocar na *struct*:

```cpp
struct Todo {
    int          id;
    int          userId;
    std::string  title;
    bool         completed;
};

NLOHMANN_DEFINE_TYPE_NON_INTRUSIVE(Todo, id, userId, title, completed)
```

Listagem 242: `src/http-client/main.cpp`: uma linha substitui duas funções escritas à mão.

Feito isso, `json::parse(body).get<Todo>()` funciona, e `get<std::vector<Todo»()` também — a biblioteca compõe as conversões recursivamente. O mecanismo por baixo é a especialização de um *template* `adl_serializer` encontrada por ADL (Seção II.1), o que explica por que a macro precisa estar no mesmo *namespace* da *struct*.

Armadilha — a versão *non-intrusive* exige que todos os campos existam no JSON

`NLOHMANN_DEFINE_TYPE_NON_INTRUSIVE` lança `json::out_of_range` se um campo declarado faltar no objeto recebido. Para APIs em que campos são opcionais, existe a variante `_WITH_DEFAULT`, que usa o valor de um objeto construído por padrão. Escolher a variante errada produz um cliente que quebra quando o servidor acrescenta ou remove um campo — que é justamente o que servidores fazem.

A execução completa contra `jsonplaceholder.typicode.com` mede o caminho inteiro: transporte, *parse*, conversão para *struct* e filtragem em memória.

```
-- GET /todos/1 --
status       : 200
content-type : application/json; charset=utf-8
  [  1] delectus aut autem

-- GET /todos?userId=1 --
status: 200

Total     : 20
Concluidas: 11
```

Listagem 243: Saída medida de `./dist/bin/http_client`, demos 1 e 2. Vinte tarefas do usuário 1, onze concluídas.

VII.4 Banco de dados e o padrão *Repository*

O subprojeto `src/database/` usa SOCI para falar com SQLite e PostgreSQL, mas o assunto real é arquitetural: é o Seção V.1 aplicado a persistência.

VII.4.1 O que o SOCI é, e o vocabulário que ele traz

SOCI é uma camada C++ sobre os clientes nativos de vários bancos. Ela não é um ORM — não gera SQL, não mapeia classes para tabelas. Ela padroniza o *acesso*: a mesma sintaxe de sessão, *binding* e iteração para SQLite, PostgreSQL, MySQL, Oracle e ODBC.

- **Backend** — o adaptador para um banco concreto. É um objeto (`soci::sqlite3`, `soci::postgresql`) escolhido em tempo de execução, não de compilação.
- **Session** — a conexão. Construída com um *backend* e uma *string* de conexão.

- **DSN** (*Data Source Name*) — a *string* que descreve onde está o banco. Neste subprojeto ela leva um prefixo de esquema (`sqlite3://`, `postgresql://`) que decide o *backend*.
- **Statement** — uma consulta preparada, com espaços para parâmetros.
- **Rowset** — o resultado de um `SELECT`, iterável com `for` de intervalo.
- **Indicator** — o marcador que diz se um valor é nulo. Bancos têm `NULL`; C++ não, e o *indicator* é a ponte.

VII.4.2 A pilha de abstração

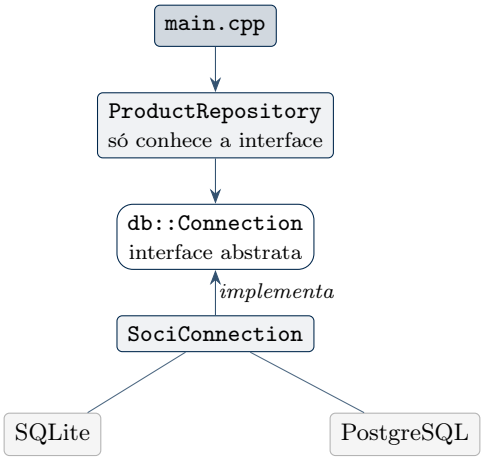


Figura 6: A seta de `SociConnection` aponta *para cima*: é a inversão de dependência da Seção V.1. Trocar de banco não muda uma linha de `ProductRepository` nem de `main.cpp` — muda só o DSN passado a `db::connect()`.

O `ProductRepository` concentra *todo* o SQL da aplicação, e é essa concentração que dá as duas propriedades desejadas: o resto do programa não conhece SQL, e trocar uma *query* não exige procurar por *strings* espalhadas.

A interface `db::Connection` tem exatamente quatro métodos, e a economia é deliberada: `execute` para DDL e DML sem retorno, `query` com *callback* por linha, `transaction` recebendo uma função, e `execute_prepared` com *binding* posicional. Nada mais atravessa a fronteira — nem `soci::session`, nem `soci::row`, nem o tipo de exceção da biblioteca.

```
static soci::backend_factory const& pick_backend(const std::string& dsn) {
    if (dsn.rfind("sqlite3://", 0) == 0) return soci::sqlite3;

#if defined(TEM_POSTGRESQL) && TEM_POSTGRESQL
    if (dsn.rfind("postgresql://", 0) == 0) return soci::postgresql;
    ...
#else
    if (dsn.rfind("postgresql://", 0) == 0)
        throw DbError("DSN PostgreSQL pedido, mas este binario foi compilado "
                      "SEM o backend PostgreSQL (falta libpq-dev).");
#endif
    throw DbError("DSN nao reconhecido. Use sqlite3://");
}
```

Listagem 244: `src/database/db.cpp`: o único lugar do arquivo que conhece *backends* concretos. Todo o resto fala com `soci::session`.

Regra — a mensagem de erro deve dizer o que fazer

Repare no texto lançado: ele não diz "*backend* indisponível". Diz qual *backend* foi pedido, por que não existe neste binário, e qual pacote instalar. Uma mensagem de erro é documentação entregue no pior momento possível — é quando ela precisa ser mais específica, não menos.

VII.4.3 Por que o backend PostgreSQL é opcional

Este é o exemplo concreto da detecção graciosa prometida na abertura do capítulo, e a razão é dupla e verificável.

Primeiro, o arquivo `.pc` que o Conan 2 gera para `soci-soci_postgresql` traz um `Requires: soci-soci_core none` malformado — o `none` é um defeito do gerador — e o `pkg-config` rejeita o pacote inteiro por causa dele. Segundo, mesmo quando o pacote é encontrado, o cabeçalho `soci-postgresql.h` inclui `libpq-fe.h`, o cabeçalho do cliente do PostgreSQL, que exige `libpq-dev` instalado no sistema. Sem ele: `fatal error: libpq-fe.h: No such file or directory`.

```
cc_db = meson.get_compiler('cpp')
tem_pg = soci_pg_dep.found() and cc_db.has_header('libpq-fe.h')

if tem_pg
    message('database: backend PostgreSQL HABILITADO')
else
    message('database: so SQLite (falta libpq-dev ou o .pc do backend)')
endif

executable('database',
    files('main.cpp', 'db.cpp'),
    dependencies: [fmt_dep, soci_dep, soci_sqlite_dep] +
                  (tem_pg ? [soci_pg_dep] : []),
    cpp_args: tem_pg ? ['-DTEM_POSTGRESQL=1'] : ['-DTEM_POSTGRESQL=0'],
    install : true)
```

Listagem 245: `src/database/meson.build`: perguntar ao compilador em vez de presumir.

Por que — por que isso não é o mesmo que comentar o alvo

A diferença é que o alvo *continua sendo construído*. O que varia é uma funcionalidade opcional, decidida por uma pergunta ao ambiente (`has_header`), registrada numa macro e refletida numa mensagem de erro útil em tempo de execução. O código do caminho SQLite — que é 100% do que o subprojeto ensina — passa pelo compilador em toda máquina, sempre. Um subprojeto *didático* não deve exigir um servidor de banco de dados para compilar: o SQLite é embutido, é um arquivo, sem processo e sem rede, e ensina exatamente as mesmas coisas — sessão, *prepared statement*, transação, *binding* e mapeamento de tipos.

VII.4.4 *Prepared statements* não são otimização

A razão principal de usar parâmetros em vez de concatenar *strings* é segurança, e o subprojeto mostra o ataque:

```
std::string name = get_user_input();                    // "' OR '1'='1"
conn_.execute("SELECT * FROM products WHERE name = '" + name + "'");
// Query resultante: SELECT * FROM products WHERE name = '' OR '1'='1'
// -> retorna TODOS os produtos
```

Listagem 246: A concatenação, e o que o atacante escreve no campo de texto.

Com `execute_prepared`, o valor nunca é interpretado como SQL: ele chega ao banco como *dado*, por um canal separado do texto da consulta. Não é questão de escapar caracteres corretamente — é questão de a consulta e os dados não compartilharem o mesmo canal.

Regra — nenhuma *string* vinda de fora entra no texto de uma consulta

A regra é sintática e verificável: se aparece `+` concatenando algo numa *string* de SQL, é bug de segurança. Vale inclusive para valores que "não podem" vir do usuário — eles acabam vindo. O benefício secundário é real mas menor: o banco reaproveita o plano de execução entre chamadas com o mesmo texto.

Do lado da implementação, o *binding* do SOCI é mais explícito do que o de outras bibliotecas, e vale ver a sequência inteira:

```
soci::statement st(session_);
st.alloc();
st.prepare(sql);

std::vector<soci::indicator> inds(params.size(), soci::i_ok);
for (std::size_t i = 0; i < params.size(); ++i)
    st.exchange(soci::use(params[i], inds[i]));    // liga POR REFERENCIA

st.define_and_bind();
st.execute(true);
return static_cast<int>(st.get_affected_rows());
```

Listagem 247: `src/database/db.cpp`: preparar, ligar, executar. Os cinco passos são obrigatórios e nesta ordem.

Armadilha — `soci::use` liga por referência, não por valor

O valor não é copiado no momento do `exchange`: o SOCI guarda o endereço e lê de lá no `execute`. Isso significa que `params` e `inds` precisam continuar vivos e *no mesmo endereço* até a execução terminar. Um `std::vector` que sofra `push_back` entre o *bind* e o `execute` realoca, invalida todos os endereços e leva o banco a ler memória liberada. O código acima é correto por construção: o vetor é dimensionado uma vez e nunca cresce. É a mesma disciplina de invalidação de iteradores da Seção III.1, aplicada a ponteiros que atravessaram uma fronteira de biblioteca.

Há um segundo detalhe instrutivo na adaptação de tipos. `SociRow::get_int` tenta `int` e, se falhar, tenta `long long`:

```
int get_int(const std::string& col) const override {
    try {
        return row_.get<int>(col);
    } catch (...) {
        return static_cast<int>(row_.get<long long>(col));
    }
}
```

Listagem 248: `src/database/db.cpp`: a mesma coluna tem tipo diferente conforme o backend.

O SQLite não tem tipos de coluna rígidos — ele tem *affinity* — e um `INTEGER` pode chegar como `int` ou como `long long` dependendo do valor e do *backend*. Absorver essa diferença é precisamente o trabalho da camada de adaptação: se ela vazasse para o `ProductRepository`, a promessa de portabilidade seria falsa.

VII.4.5 Transações como RAII de negócio

A interface expõe a transação como uma função que recebe outra função:

```
    conn_.transaction([&]() {
        for (auto& [id, qty] : items) {
            conn_.execute_prepared(
                "UPDATE products SET stock = stock - :1 "
                "WHERE id = :2 AND stock >= :1",
                {std::to_string(qty), std::to_string(id)}
            );
        }
    });
    // excecao -> rollback automatico; fim normal -> commit automatico
```

Listagem 249: `src/database/main.cpp`: atomicidade sem *commit* nem *rollback* escritos à mão pelo chamador.

É a garantia forte da Seção II.8 realizada pelo banco em vez de pelo programa: ou todas as operações valem, ou nenhuma vale. Do lado da implementação, o mecanismo é o *scope guard* explícito:

```
void transaction(std::function<void()> fn) override {
    soci::transaction tx(session_);
    try {
        fn();
        tx.commit();
    } catch (...) {
        tx.rollback();
        throw;              // o chamador PRECISA saber que falhou
    }
}
```

Listagem 250: `src/database/db.cpp`: o `catch(...)` relança depois de desfazer — desfazer não é tratar.

Regra — engolir a exceção depois do *rollback* é pior que não ter transação

Sem o `throw` final, o chamador veria a função retornar normalmente e concluiria que a operação teve sucesso — quando o banco desfez tudo. O programa passaria a operar sobre um estado que ele acha que existe. Desfazer é a metade fácil; comunicar é a que importa.

Armadilha — a condição de estoque está na consulta, não no C++

A cláusula `AND stock >= :1` não é decoração: verificar o estoque em C++ antes do `UPDATE` abriria uma janela entre a leitura e a escrita em que outra transação poderia consumir o mesmo estoque. Colocar a condição *na* consulta faz o banco decidir atomicamente. É o análogo, em nível de banco, da diferença entre `if (contador == 0) contador = 1` e um `compare_exchange` (Seção VI.1).

VII.4.6 Um exemplo que só funciona na primeira execução é um exemplo quebrado

O demo tinha um defeito que nada tem a ver com C++ e tudo a ver com escrever material didático. `create_table` usava `CREATE TABLE IF NOT EXISTS`, o que preserva as linhas da execução anterior. Na segunda vez que alguém rodava o programa, o primeiro `INSERT` batia na restrição de unicidade:

```
DbError: UNIQUE constraint failed: products.name
```

Listagem 251: O erro na segunda execução. O demo saía com código 1 e parecia quebrado.

A correção foi acrescentar um `DROP TABLE IF EXISTS` antes do `CREATE`, tornando o programa **idempotente** — executável quantas vezes se queira, sempre com o mesmo resultado.

Armadilha — o `DROP` é certo aqui e catastrófico em produção

Apagar a tabela apaga os dados. Num demo isso é a coisa certa; num sistema real é perda irreversível. O instrumento correto em produção é a **migração**: uma sequência versionada de *scripts* `ALTER TABLE`, cada um aplicado uma única vez e registrado numa tabela de controle, de modo que o banco saiba em que versão do esquema está. Ferramentas: Flyway, Liquibase, Alembic. A distinção entre "recriar" e "evoluir" o esquema é a primeira coisa que separa um exemplo de um sistema.

O caminho do arquivo de banco também é derivado, não escrito à mão — e o idioma vale registrar, porque resolve o problema de "o programa só funciona se eu o executar do diretório certo":

```
auto db_dir = fs::canonical(fs::read_symlink("/proc/self/exe"))
                  .parent_path()   // dist/bin/
                  .parent_path()   // dist/
                  .parent_path()   // raiz do projeto
              / "tmp" / "sqlite";
fs::create_directories(db_dir);     // idempotente: cria toda a arvore
const std::string dsn = "sqlite3://" + (db_dir / "academy.db").string();
```

Listagem 252: `src/database/main.cpp`: o caminho é relativo ao *executável*, não ao diretório de trabalho.

Armadilha — `/proc/self/exe` é específico do Linux

É o idioma mais direto para descobrir o caminho do próprio binário, e não é portável: no macOS a função é `_NSGetExecutablePath`, no Windows é `GetModuleFileNameW`. O `std::filesystem` padroniza a manipulação de caminhos, não a descoberta do executável — essa continua sendo território do sistema operacional.

A execução completa cobre o CRUD inteiro mais a transação, e o resultado medido mostra o débito atômico funcionando: o teclado saiu de 15 para 13, o mouse de 30 para 25, os dois na mesma transação.

```
-- CREATE TABLE --
  tabela 'products' criada
-- INSERT --
  5 produtos inseridos
-- TRANSACTION debit_stock --
  debitando: Teclado x2, Mouse x5
  ID   Nome                 Preco  Estoque
  1    Teclado Mecanico     R$ 349.90    13
  2    Mouse Gamer          R$ 189.99    25
  3    Monitor 27"          R$1699.00     8
  4    Headset USB          R$ 249.00     0
  5    Webcam 1080p         R$ 159.50    12
```

Listagem 253: Saída medida de `./dist/bin/database` (cores removidas). O banco resultante ocupa 16 KiB em `tmp/sqlite/academy.db`.

VII.5 Mensageria com AMQP

O subprojeto `src/rabbitmq/` combina o Asio da Seção VII.2 com o protocolo AMQP: dois executáveis — `rabbitMQ_producer` e `rabbitMQ_consumer` — que não se conhecem e nunca conversam diretamente.

Por que — o que um *broker* compra

Produtor e consumidor deixam de precisar estar vivos ao mesmo tempo, de conhecer o endereço um do outro, e de rodar na mesma velocidade. O *broker* absorve picos, distribui trabalho entre vários consumidores e reentrega o que não foi confirmado. O preço é uma peça de infraestrutura a mais para

operar — e uma classe nova de problemas: mensagem duplicada, mensagem fora de ordem, fila que cresce sem limite.

VII.5.1 O vocabulário do AMQP

- **Broker** — o servidor. Aqui, o RabbitMQ.
- **Connection** — uma conexão TCP com o *broker*. É cara: *handshake* TCP mais *handshake* AMQP mais autenticação.
- **Channel** — uma conexão *lógica* multiplexada dentro da conexão TCP. É barata, e é onde todas as operações acontecem. O padrão é uma conexão por processo e um canal por *thread*.
- **Exchange** — o ponto onde o produtor publica. Não armazena nada; apenas roteia.
- **Queue** — a fila que armazena mensagens até alguém consumir.
- **Binding** — a regra que liga uma *exchange* a uma fila.
- **Routing key** — a etiqueta que a mensagem carrega, e que a *exchange* compara com os *bindings*.
- **Delivery tag** — um identificador de entrega, por canal, que o consumidor usa para confirmar aquela mensagem específica.
- **Vhost** — um espaço de nomes isolado dentro do *broker*; o padrão é `/`.

Regra — canal é barato, conexão é cara

Abrir uma conexão TCP por operação é o erro de desempenho número um em código de AMQP. Abra a conexão uma vez, no início do processo, e crie canais. E não compartilhe um canal entre *threads*: o protocolo multiplexa quadros de canais diferentes na mesma conexão, e dois emissores no mesmo canal produzem quadros intercalados que o *broker* rejeita.

VII.5.2 Exchange, *binding* e fila

O produtor **não conhece a fila**: ele publica em uma *exchange*, que roteia segundo os *bindings* declarados. É essa indireção que permite acrescentar consumidores sem tocar no produtor — o mesmo desacoplamento do padrão *Observer* (Seção V.2), agora entre processos.

Tabela 90: Os quatro tipos de *exchange* e como cada um roteia.

Tipo	Roteia
`direct`	para a fila cuja *routing key* é igual à da mensagem
`fanout`	para **todas** as filas vinculadas
`topic`	por padrão, com curingas `*` e `#`
`headers`	pelos cabeçalhos da mensagem

No exemplo, `channel.publish("", nome, corpo)` usa a *exchange* padrão — *direct* com nome vazio — em que a *routing key* é o próprio nome da fila. É o atalho que faz o exemplo caber em uma linha, e também o que o torna enganoso: em um sistema real, publicar direto numa fila acopla o produtor à topologia, e é justamente o acoplamento que o *broker* deveria ter eliminado.

Armadilha — os três argumentos de `publish` são exchange, routing key, corpo

Não são fila, chave e corpo. A leitura errada funciona por acidente com a *exchange* padrão — porque ali a *routing key* coincide com o nome da fila — e falha silenciosamente no dia em que alguém declara uma *exchange* de verdade. A mensagem não dá erro: ela é publicada numa *exchange* sem *binding* correspondente e é descartada. "Publiquei e a mensagem sumiu" quase sempre é isto.

VII.5.3 Duas famílias de *callback*, e o erro de compilação que as separa

A primeira versão deste subprojeto não compilava, e o motivo é uma lição de projeto de API. O código escrito era o que todo tutorial mostra:

```
AMQP::TcpConnection connection(&handler, address);
connection.onConnected([]{ /* ... */ });                    // nao existe
connection.onError([](const char* m){ /* ... */ });
```

Listagem 254: A versão que não compila.

```
error: cannot convert 'main()::<lambda()>' to 'AMQP::TcpState*'
```

Listagem 255: A mensagem do compilador — confusa, porque o método *existe*, só que com outra assinatura e outro papel.

A API do AMQP-CPP mudou: os eventos de **ciclo de vida da conexão** pertencem ao *handler*, e a forma de reagir a eles é **herdar e sobrescrever métodos virtuais**. O `TcpConnection` tem métodos de nome parecido, mas eles são parte da máquina de estados interna — daí a mensagem falando de `TcpState`.

Regra — um conjunto fixo de eventos correlacionados pede herança; eventos avulsos pedem *lambda*

Esta biblioteca usa os dois estilos, e a escolha não é arbitrária. Os seis eventos de ciclo de vida da conexão são *fixos*, *conhecidos* e compartilham estado — viram métodos virtuais de um objeto. Os retornos de cada operação de canal (`declareQueue`, `consume`) são *independentes*: cada chamada tem o seu próprio sucesso e o seu próprio erro, e viram *lambdas* encadeadas no valor de retorno.

VII.5.4 O ciclo de vida em seis estágios

Tabela 91: Os seis estágios do *handler*, na ordem. Publicar antes de `onReady` é a origem da maioria dos "publiquei e sumiu".

Método	Significado
`onAttached`	objeto de conexão construído — ainda não há *socket*
`onConnected`	TCP estabelecido — ainda **não** há login AMQP
`onReady`	*handshake* AMQP concluído — só aqui dá para publicar
`onError`	erro fatal; a conexão vai fechar
`onClosed`	fechamento limpo
`onDetached`	seguro destruir o objeto

```
class Handler : public AMQP::LibBoostAsioHandler {
public:
    Handler(boost::asio::io_context& io, std::string rotulo)
        : AMQP::LibBoostAsioHandler{io}, rotulo_{std::move(rotulo)} {}

    void onReady(AMQP::TcpConnection* c) override {
        AMQP::LibBoostAsioHandler::onReady(c);   // a base faz trabalho REAL
```

```
        pronto_ = true;
        fmt::print("[{}] onReady — handshake AMQP concluido\n", rotulo_);
    }
    // ... onAttached, onConnected, onError, onClosed, onDetached ...
};
```

Listagem 256: `src/rabbitmq/handler.hpp`: herdar do *handler* do Boost.Asio e sobrescrever só o que interessa.

Armadilha — esquecer a chamada à implementação da base produz um programa que compila, conecta e nunca recebe nada

A `LibBoostAsioHandler` faz trabalho real nesses métodos: registra o *socket* no `io_context` e monitora leitura e escrita. Se o *override* não chamar `AMQP::LibBoostAsioHandler::onX(...)`, esse registro nunca acontece. E o compilador não avisa, porque tecnicamente o método *foi* implementado — é a armadilha clássica de sobrescrever um virtual que não é puro. Quando a base tem comportamento obrigatório, o padrão NVI (Seção V.1) é a defesa: a base expõe um método não virtual que faz o trabalho e chama um *hook* protegido.

A evidência do ciclo de vida vem de graça quando não há *broker*. Rodar o produtor sem servidor produz uma sequência que *prova* a tabela: o objeto é criado, o TCP falha, e o programa vai direto de `onAttached` para `onError` — `onConnected` e `onReady` nunca acontecem.

```
[PRODUCER] conectando em amqp://guest:guest@localhost:5672/
[PRODUCER] onAttached — objeto de conexao criado
[PRODUCER] ! erro de canal: Connection refused
[PRODUCER] ! onError — Connection refused
[PRODUCER] onDetached — seguro destruir
[PRODUCER] fim
```

Listagem 257: Saída medida de `./dist/bin/rabbitMQ_producer` sem *broker* rodando. Saída de processo: 1.

Regra — um cliente de rede sem servidor deve falhar rápido, dizer por quê e sair com código diferente de zero

O `main` termina com `return handler.erro() ? 1 : 0`, e o temporizador de 15 segundos garante que o processo não fica preso para sempre. Essas duas linhas transformam um demo em um teste de fumaça utilizável por um *script*: a saída diz o que houve, e o código de saída diz se funcionou. Compare com o Seção VII.6, que faz o contrário.

VII.5.5 Tudo é *callback*

A API do AMQP-CPP é inteiramente assíncrona, sobre o `io_context` do Boost.Asio. Nada acontece na ordem em que está escrito: `onReady` dispara quando o *handshake* AMQP termina, e é só ali que declarar a fila e publicar fazem sentido.

```
channel.onReady([&channel, &connection, quantas] {
    channel.declareQueue("hello")            // idempotente: cria se nao existir
        .onSuccess([&](const std::string& nome, uint32_t, uint32_t) {
            for (int i = 1; i <= quantas; ++i)
                channel.publish("", nome, fmt::format("mensagem {} de {}", i, quantas));
            connection.close();
        })
        .onError([](const char* m) { fmt::print("erro ao declarar: {}\n", m); });
});

io.run();    // so aqui algo de fato acontece
```

Listagem 258: `src/rabbitmq/producer.cpp`: declarar e publicar, cada um dentro do *callback* certo.

Que `declareQueue` seja **idempotente** é o que permite os dois lados declararem a mesma fila sem coordenação: nenhum dos dois precisa saber quem subiu primeiro. Se a fila já existe com os mesmos parâmetros, a chamada não faz nada.

Armadilha — `publish` não confirma nada

Por padrão, `publish` apenas escreve no *socket*. Se o *broker* rejeitar a mensagem — *exchange* inexistente, fila cheia, disco cheio — o produtor não fica sabendo. Para garantia de entrega existe o *publisher confirm* (`channel.confirmSelect()`), que faz cada publicação devolver um *deferred* confirmado pelo servidor. É o mesmo compromisso entre `send` e `send + ack` de qualquer protocolo, e custa uma ida e volta por mensagem.

VII.5.6 ACK: a diferença entre entregue e processado

O ponto conceitual do subprojeto é o reconhecimento. O *broker* só remove a mensagem da fila quando o consumidor confirma:

Tabela 92: As três respostas possíveis a uma mensagem recebida.

Chamada	Efeito no *broker*
`ack(tag)`	remove a mensagem: processada com sucesso
`nack(tag)`	reencaminha a outro consumidor ou manda à *dead-letter*
`nack(tag, false, true)`	devolve à frente da fila (*requeue*)

```
1  channel.consume(nome)
2      .onReceived([&channel](const AMQP::Message& msg,
3                             uint64_t etiqueta, bool redespachada) {
4          const std::string corpo{msg.body(), msg.bodySize()};
5          fmt::print("[CONSUMER] recebido: \"{}\"{}\n", corpo,
6                     redespachada ? "  (REDESPACHADA)" : "");
7          channel.ack(etiqueta);            // depois de processar, nunca antes
8      })
9      .onError([](const char* m) { fmt::print("erro no consume: {}\n", m); });
```

Listagem 259: `src/rabbitmq/consumer.cpp`: o `ack` vem **depois** de processar, e o *flag* de redespacho é informação útil.

Armadilha — confirmar antes de processar transforma falha em perda

Se o consumidor chamar `ack` logo ao receber e morrer no meio do processamento, a mensagem já saiu da fila e ninguém mais a verá. Confirmar *depois* do processamento inverte a garantia: se ele morrer antes do `ack`, o *broker* reentrega — e a mensagem pode ser processada duas vezes. Não há terceira opção; a escolha é entre *at-most-once* e *at-least-once*, e a segunda exige que o processamento seja **idempotente**.

O *flag* `redespachada` é a ferramenta que o protocolo dá para lidar com isso: ele avisa que esta mensagem já foi entregue antes. Um consumidor sério o usa para decidir entre reprocessar e mandar para a *dead-letter queue*, evitando o "veneno" — a mensagem que sempre falha e volta para a fila indefinidamente, consumindo o *broker* inteiro.

VII.5.7 A porta, que quase nunca é a que se espera

O subprojeto traz um `docker-compose.yml`, e ele contém uma pegadinha digna de registro: as portas publicadas não são as padrão.

```
ports:
  - "4672:5672"     # AMQP - a porta do HOST e 4672
  - "14672:15672"   # painel web - a porta do HOST e 14672
```

Listagem 260: Resumo de `src/rabbitmq/docker-compose.yml`: o mapeamento é `host:container`, e o host usa portas deslocadas.

Armadilha — o binário assume 5672 e o *compose* publica 4672

O padrão do AMQP é 5672, e é isso que o produtor e o consumidor usam quando nenhum argumento é passado. Com este `docker-compose.yml`, o servidor responde em 4672 no host, e a conexão é recusada com a mensagem mais genérica possível. Por isso os dois programas aceitam a URL como primeiro argumento: `./dist/bin/rabbitMQ_consumer amqp://guest:guest@localhost:4672/`. Endereço de servidor é configuração, e configuração não pertence ao código compilado.

VII.6 Nuvem: AWS SDK e o modelo `Outcome`

O subprojeto `src/aws/` é o menor do capítulo e faz uma coisa só — listar os *buckets* de um S3 — mas as vinte linhas contêm três lições que transferem para qualquer SDK grande. Um *bucket*, para quem nunca usou o S3, é o contêiner de nível mais alto do serviço: um espaço de nomes global onde objetos são guardados por chave.

VII.6.1 O bug que provou o argumento do capítulo

Este arquivo é um dos seis itens da Tabela 78, e o defeito era este:

```
// O arquivo usava `fmt::print` e nao incluia <fmt/core.h>. O alvo estava
// desabilitado no meson.build da raiz, entao o erro nunca apareceu.
#include <fmt/core.h>
```

Listagem 261: `src/aws/s3/main.cpp`: o comentário que hoje encabeça o arquivo.

Um arquivo de vinte linhas, com um `#include` faltando, dentro de um repositório que compila com zero avisos. A única razão de ele ter sobrevivido é que ninguém o compilava. Não há moral mais direta para a regra da abertura.

VII.6.2 O ciclo de vida global, e o bloco que parece decorativo

```
Aws::SDKOptions options;
Aws::InitAPI(options);    // (1) SEMPRE primeiro
{
    Aws::S3::S3Client client;                  // (2) credenciais automaticas
    auto outcome = client.ListBuckets();       // (3) chamada HTTP

    if (outcome.IsSuccess()) {                 // (4) testa ANTES de pegar
        for (auto& bucket : outcome.GetResult().GetBuckets())
            fmt::print("bucket: {}\n", bucket.GetName().c_str());
    } else {
        fmt::print("erro: {}\n", outcome.GetError().GetMessage().c_str());
    }
}                                              // (5) client destruido AQUI
Aws::ShutdownAPI(options);                     // (6) SEMPRE por ultimo
```

Listagem 262: A ordem que não pode ser trocada.

Armadilha — as chaves ao redor do `client` não são estilo

O SDK mantém recursos globais — *thread pool*, cliente HTTP, sistema de *log* — e `ShutdownAPI` os destrói. Se o `S3Client` for destruído *depois* disso, o destrutor dele acessa memória já liberada, e o programa termina com um *crash* no encerramento, longe da causa. O bloco força a ordem correta pela regra de escopo — e é um exemplo raro de RAII sendo usado para ordenar destruições *entre* objetos, não para liberar um recurso próprio.

VII.6.3 `Outcome<R, E>`: erro como valor, de novo

O SDK da AWS não usa exceções. Toda operação devolve um `Aws::Utils::Outcome<ResultType, ErrorType>`, e o chamador pergunta `IsSuccess()` antes de pegar o resultado.

Tabela 93: O mesmo conceito, três nomes — e o que cada um oferece além do teste.

Origem	Tipo	Teste de sucesso	Composição
AWS SDK for C++	`Outcome<R,E>`	`outcome.IsSuccess()`	nenhuma
C++23	`expected<T,E>`	`if (r)`	`and_then`, `transform`
Rust	`Result<T,E>`	`r.is_ok()`	`?`, `map`, `and_then`

A coluna da direita é a diferença prática, e não é pequena. Com `std::expected` escreve-se `buscar(id).and_then(validar).transform(formatar)`, e o primeiro erro curto-circuita a cadeia inteira sem um único `if`. Com `Outcome` não há nada disso: cada chamada exige o seu `IsSuccess()`, e uma sequência de cinco operações vira cinco blocos aninhados. O `Outcome` é anterior ao `std::expected` e resolve o mesmo problema com metade da ergonomia.

O contraste com a Seção II.8 é o ponto didático: o SDK escolheu erro como valor porque atravessa fronteiras de ABI, precisa funcionar com exceções desligadas (`-fno-exceptions`) e porque, em uma chamada de rede, falhar não é excepcional — é rotina. É a mesma análise de *frequência* que decide entre exceção e valor em código próprio.

Armadilha — `Outcome` não é `[[nodiscard]]`

Como `std::expected` (Seção II.8), ignorar o retorno compila em silêncio — e o erro desaparece por completo. A diferença é que aqui não há como marcar a função: ela é do SDK. Resta a disciplina de nunca chamar uma operação da AWS sem guardar e testar o `Outcome`.

VII.6.4 A medição que contrariou a expectativa

Executar o binário sem credenciais válidas era o teste esperado: uma mensagem clara de autenticação. O que saiu foi outra coisa:

```
1 erro: XML_ERROR_PARSING_ATTRIBUTE
```

Listagem 263: Saída medida de `./dist/bin/aws-s3` sem credenciais configuradas. Código de saída do processo: **0**.

Duas coisas erradas de uma vez, e as duas ensinam.

A primeira é a mensagem. `XML_ERROR_PARSING_ATTRIBUTE` descreve o sintoma no nível mais baixo possível — o *parser* de XML do SDK não conseguiu ler a resposta — e esconde a causa, que é a requisição ter sido rejeitada e a resposta não ser o XML que o cliente esperava. É o inverso exato da regra enunciada no Seção VII.4: em vez de dizer o que fazer, a mensagem obriga o leitor a adivinhar em que camada procurar.

Armadilha — o programa reportou o erro e mesmo assim saiu com código zero

O `main` imprime a mensagem e cai no fim da função, e o fim de `main` devolve 0. Para qualquer *script*, CI ou orquestrador, esse processo *teve sucesso*. Compare com o produtor do Seção VII.5, que termina com `return handler.erro() ? 1 : 0` e sai com 1 na mesma situação. Reportar o erro na saída padrão não é reportar o erro: código de saída é a única coisa que o resto do sistema lê.

Regra — todo caminho de falha termina em código de saída diferente de zero

Vale para os cinco subprojetos deste capítulo, e é a regra mais barata de seguir do livro inteiro: um `return 1` no ramo de erro. Um programa que reporta falha em texto e sucesso em código de saída é pior do que um que não reporta nada, porque parece confiável.

O SDK autentica sozinho, nesta ordem: variáveis de ambiente (`AWS_ACCESS_KEY_ID`, `AWS_SECRET_ACCESS_KEY`), depois `~/.aws/credentials`, depois o perfil da instância EC2 ou o *task role* do ECS. É a cadeia que permite ao mesmo binário rodar na máquina do desenvolvedor e na nuvem sem nenhuma credencial no código — e é a razão de o `S3Client` ser construído sem argumento nenhum.

Regra — credencial não entra no binário, nem no repositório

A cadeia de provedores existe exatamente para tornar desnecessário escrever chave no código. Uma chave literal num `.cpp` sobrevive no histórico do Git mesmo depois de removida, e é o vazamento mais comum de todos. A alternativa custa uma linha de `export`.

VII.7 Síntese: cinco bibliotecas, cinco convenções de erro

O capítulo tem um achado que vale mais que qualquer das APIs individuais: as cinco bibliotecas resolveram o mesmo problema — como reportar que algo deu errado — de cinco maneiras diferentes (Tabela 94). Um programa que use três delas conviverá com três convenções ao mesmo tempo.

Tabela 94: Como cada biblioteca deste capítulo reporta falha, e onde o erro é observado.

Biblioteca	Convenção	Seção	Onde se observa
Asio	exceção *ou* `error_code&`	VII.2	na chamada, ou no *handler*
libcurl (envolvida)	exceção no transporte, campo no status	VII.3	no `catch` e no `if`
SOCI	exceção, com *rollback* pelo destrutor	VII.4	no `catch` do chamador
AMQP-CPP	*callback* `onError`	VII.5	depois, no laço de eventos
AWS SDK	`Outcome<R,E>`, sem exceções	VII.6	no retorno, se alguém olhar

A quarta coluna é a que dói. Em Asio síncrono e em SOCI, o erro chega no ponto da chamada e é impossível ignorá-lo por acidente. Em AMQP-CPP ele chega *noutro momento*, dentro do laço de eventos, e a única forma de não perdê-lo é ter registrado o *callback* antes. No AWS SDK ele chega no valor de retorno e some se ninguém o testar. Três modelos temporais diferentes de "quando eu fico sabendo", num só programa.

Regra — traduza na fronteira, não no meio do código

A saída não é adotar a convenção de nenhuma delas, e sim envolver cada biblioteca numa camada fina que traduza para a convenção *do seu programa* — exatamente o que `http::get` faz com libcurl e o que `db::Connection` faz com SOCI. É a mesma inversão de dependência da Seção V.1, aplicada ao vocabulário de erro: a política decide como erros são representados, e os adaptadores se ajustam.

E resta a lição que abriu o capítulo e que os cinco subprojetos comprovaram, um a um: o `#include` que faltava no AWS, o *literal* UTF-8 dentro de aspas simples no cliente HTTP, o demo de banco que quebrava na segunda execução, a API do AMQP que tinha mudado sem ninguém notar. Nenhum desses defeitos exigia perspicácia para ser encontrado — exigia apenas que alguém mandasse o compilador olhar. **Um alvo que não é construído é um alvo que apodrece**, e o custo de descobrir isso é sempre pago mais tarde, com juros.

VIII Gráficos, física e simulação

Este capítulo tem uma progressão deliberada, e ela é o conteúdo tanto quanto os subprojetos individuais: primeiro a física *sem* desenho (VIII.1), depois o desenho (VIII.2), depois as articulações e os corpos deformáveis (VIII.3), depois a interface gráfica em volta (VIII.4), e por fim a matemática (VIII.5) que sustenta o subprojeto mais integrado do repositório (VIII.6).

Entender a física antes de desenhá-la não é preciosismo pedagógico: quando algo está errado numa simulação com renderização, a primeira pergunta é sempre se o erro está no simulador ou no desenho — e quem nunca viu o simulador sozinho não tem como responder. Este capítulo leva essa pergunta a sério: em três dos seis subprojetos a resposta medida foi "o erro está no desenho", e num deles a discrepância entre onde a esfera *está* e onde ela é *pintada* é de 392 pixels (Seção VIII.2.4).

Regra — todos os alvos deste capítulo são construídos

Houve um tempo em que os subprojetos gráficos ficavam comentados em `src/meson.build` "porque as dependências são pesadas". Isso acabou. Hoje **os 57 subprojetos são construídos**, os 98 binários resultantes são instalados em `dist/bin`, e a compilação passa limpa sob `-Wall -Wextra -Wpedantic`. Bullet3, SFML, Qt 6, Eigen3 e JSBSim entram via Conan; o JSBSim em particular vem de uma receita própria em `src/flight-simulator/deps/jsbsim`, construída por `src/flight-simulator/deps/install.sh` — não é um *submodule* de Git, e não há `.gitmodules` no repositório.

Por que — um alvo que não é construído é um alvo que apodrece

A decisão de habilitar tudo não é estética. No minuto em que os alvos "pesados" voltaram a compilar, o repositório pagou a conta acumulada: quatro cabeçalhos com `#include` faltando (`<algorithm>`, `<array>`, `<cmath>`, `<fmt/core.h>`) e dois bugs de ABI em *assembly* que só apareceram ao *executar* os binários. Todos estavam latentes exatamente nos alvos que ninguém construía.
Um `#` na frente de um `subdir()` não desativa um subprojeto: ele apenas transfere o custo do defeito para o futuro, com juros. Se um alvo não pode ser construído, conserte-o ou apague-o.

VIII.1 Física sem renderização

O subprojeto `src/collision/` faz uma coisa só: uma esfera cai de dez metros, bate num plano e quica com coeficiente de restituição 0,6. Não há janela, não há desenho — apenas uma tabela de números impressa a cada dez passos. É exatamente essa austeridade que o torna útil: tudo o que aparece na tela é um número que veio do integrador, e nada mais.

Por que — antes de desenhar, é preciso ter o que desenhar

Um motor de física é uma função que recebe um estado (posições, velocidades, orientações de n corpos) e um intervalo de tempo, e devolve o estado seguinte. Toda a parte visual — janelas, texturas, câmeras — é uma *leitura* desse estado, e pode ser trocada inteira sem que o motor perceba. Se você começa pelo desenho, nunca sabe se um objeto atravessou o chão porque o *solver* falhou ou porque a sua conversão de coordenadas estava errada. Começando pelo simulador puro, você fixa uma referência: uma tabela de números que ou está certa ou está errada, independentemente de qualquer biblioteca gráfica.

VIII.1.1 Vocabulário: o que é cada peça

Quatro termos aparecem em toda documentação de motor de física, e é melhor defini-los antes de usá-los.

- **Corpo rígido** (`btRigidBody`): um objeto que não se deforma. Seu estado completo cabe em treze números — posição (3), orientação como quaternion (4), velocidade linear (3) e velocidade angular (3).
- **Forma de colisão** (`btCollisionShape`): a geometria usada *para detectar contato*. Não precisa ser a geometria desenhada, e quase nunca é: um personagem complexo colide como uma cápsula.
- **AABB** (*axis-aligned bounding box*): a menor caixa alinhada aos eixos do mundo que contém a forma. Testar sobreposição de duas AABBs custa seis comparações; é a razão de a *broadphase* ser barata.
- ***Motion state*** (`btMotionState`): a ponte entre o motor e o desenho. O motor escreve nele a transformação *interpolada* do corpo; o renderizador lê. Guarde este adjetivo — ele volta medido na Seção VIII.1.6.

VIII.1.2 O *pipeline* de um quadro

Toda chamada a `stepSimulation` percorre quatro etapas, e conhecer a divisão explica quase todo o comportamento e todo o custo do motor (Figura 7).

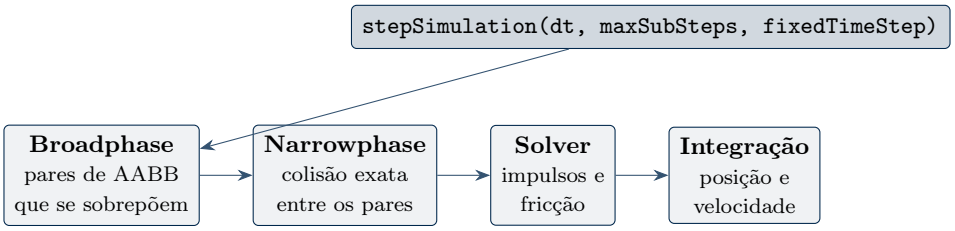


Figura 7: O *pipeline* do Bullet. A *broadphase* é barata e aproximada — descarta a maioria dos pares comparando caixas alinhadas aos eixos; a *narrowphase* é cara e exata, e só roda no que sobrou. A separação é a razão de uma cena com milhares de corpos ser viável.

A montagem do mundo reflete essa divisão: uma configuração de colisão, um despachante para a *narrowphase*, uma estrutura de aceleração para a *broadphase*, um *solver* e o mundo que une os quatro.

```
1  btDefaultCollisionConfiguration* config = new btDefaultCollisionConfiguration();
2  btCollisionDispatcher* dispatcher = new btCollisionDispatcher(config);
3
4  // btDbvtBroadphase = Dynamic AABB Tree → O(n log n), ótimo para cenas dinâmicas
5  // Alternativa: btAxisSweep3 para cenas com bounds fixos
6  btDbvtBroadphase* broadphase = new btDbvtBroadphase();
7
8  btSequentialImpulseConstraintSolver* solver = new btSequentialImpulseConstraintSolver();
9
10 btDiscreteDynamicsWorld* world = new btDiscreteDynamicsWorld(
11     dispatcher, broadphase, solver, config);
12 world->setGravity(btVector3(0, -9.81f, 0));
```

Listagem 264: As cinco peças, e a escolha da estrutura de *broadphase*.

O *solver* escolhido, `btSequentialImpulseConstraintSolver`, resolve os contatos por **Gauss-Seidel projetado**: em vez de montar e inverter um sistema linear com todos os contatos de uma vez, ele percorre os

contatos um a um aplicando impulsos, repete a varredura algumas vezes por passo e aceita a aproximação. É barato, converge razoavelmente e *não* é exato — o que explica, mais adiante, por que cadeias longas de restrições tremem (Seção VIII.3).

VIII.1.3 Estático e dinâmico: a massa é a chave

A distinção entre um corpo que se move e um que não se move não é um sinalizador — é o valor da massa. **Massa zero significa estático**, e o Bullet interpreta isso como massa infinita: o corpo participa das colisões, mas nenhum impulso o desloca. Corpos dinâmicos precisam ainda do tensor de inércia, que o próprio *shape* calcula.

```
1  btScalar mass = 1.0f;                          // 1 kg
2  btVector3 localInertia (0, 0, 0);
3  // para uma esfera maciça: I = (2/5) * m * r^2
4  sphereShape->calculateLocalInertia(mass, localInertia);
```

Listagem 265: O tensor de inércia sai da forma, não do usuário — `src/collision/main.cpp`.

Regra — a ordem de destruição é obrigação sua

O Bullet não tem *smart pointers* nem coletor de lixo, e a ordem importa: remover os corpos do mundo, depois deletar corpos, *motion states* e *shapes*, e só então o mundo e os quatro subsistemas. Deletar o mundo antes dos corpos é acesso a memória inválida. Esta é uma das poucas situações do repositório em que `new` e `delete` crus aparecem em código de aplicação — e a razão de os subprojetos seguintes envolverem tudo isso numa classe `PhysicsWorld` (Seção VIII.2). Guarde a ressalva: a Seção VIII.2.5 mostra, com *sanitizer* e tudo, que envolver em `std::unique_ptr` **não** resolve o problema sozinho — apenas o move para um lugar mais silencioso.

VIII.1.4 Por que o passo de tempo é fixo por dentro

```
1  // stepSimulation(timeStep, maxSubSteps, fixedTimeStep):
2  //   - fixedTimeStep: dt interno para estabilidade numérica (padrão 1/60)
3  //   - maxSubSteps: se timeStep > fixedTimeStep, subdivide (até maxSubSteps vezes)
4  //   - passando maxSubSteps=1 e dt=fixedTimeStep → determinístico
5  world->stepSimulation(dt, 1, dt);
```

Listagem 266: Os três argumentos, e o que cada um controla.

Por que — o integrador diverge se o passo varia

Motores de física de jogo usam integradores explícitos, cuja estabilidade depende do tamanho do passo. Um quadro que demore o dobro do esperado — porque o sistema operacional escalonou outra coisa, porque houve uma pausa do coletor da GPU — produziria um salto grande demais, e corpos atravessariam paredes. A solução do Bullet é aceitar um *dt* variável do mundo externo e subdividi-lo internamente em passos fixos. É por isso que uma simulação com `maxSubSteps` maior que 1 **não é determinística**: o número de subpassos depende do relógio.

VIII.1.5 O que a execução realmente imprime

Este livro mede em vez de afirmar, e a primeira medição já contraria a documentação do próprio subprojeto.

```
  step     time (s)          y (m)        vy (m/s)
-- -- -- -- -- -- -- -- -- -- -- -- -- -- -- -- -- --
   0        0.000        10.0000          −0.1635
  10        0.167         9.8501          −1.7985
  40        0.667         7.7655          −6.7035
  80        1.333         1.1710         −13.2435
  90        1.500         1.5773          +3.2962
 110        1.833         2.1582          +0.0262
 140        2.333         0.9859          +1.8662
 170        2.833         1.0000           0.0000
```

Listagem 267: Saída real de `./build/src/collision/main` (linhas selecionadas).

Cinco leituras que a tabela permite, e que nenhuma delas é óbvia:

1. **O passo 0 já não tem velocidade zero.** O laço chama `stepSimulation` *antes* de imprimir, então a linha rotulada "passo 0" é o estado *depois* do primeiro passo. E $-0{,}1635 = -9{,}81 \times \frac{1}{60}$, exatamente um passo de gravidade. O `README.md` do subprojeto prevê `0.0000` nessa célula — e está errado.

2. **O repouso é em $y = 1$, não em $y = 0$.** A posição de um corpo é a do seu centro de massa, e a esfera tem raio 1 m. Quem espera zero está confundindo o centro com a superfície.

3. **O quique perde 40% da velocidade.** A esfera chega ao contato a cerca de $-13{,}6$ m/s e sai a cerca de $+8{,}2$ m/s: a restituição 0,6 multiplica a velocidade normal, não a energia (a energia cai por $0{,}6^2 = 36\%$ do valor original).

4. **O terceiro quique some.** Entre os passos 140 e 170 a amplitude colapsa: com restituição 0,6 cada quique tem 36% da altura anterior, e depois de três quiques a altura já é menor que a tolerância de penetração do *solver*.

5. **A última linha é exatamente 1,0000 e 0,0000.** Não é coincidência nem arredondamento: o Bullet **adormece** corpos cuja velocidade fica abaixo de um limiar por tempo suficiente (*deactivation*), zerando as velocidades e parando de integrá-los. É uma otimização, e é também a explicação para o bug clássico "meu corpo não reage quando eu empurro": é preciso chamar `body->activate(true)` antes de aplicar um impulso — exatamente o que o *ragdoll* da Seção VIII.3 faz.

Armadilha — a saída "esperada" de um README não é uma medição

O `README.md` deste subprojeto traz uma tabela de saída com `vy = 0.0000` no passo 0 e o quique acontecendo por volta de `y = 0.9`. Nenhum dos dois números aparece na execução. Documentação escrita "de cabeça" envelhece pior que código, porque nada a compila. A regra prática deste livro: se um número aparece num texto, ele veio de uma execução, e a legenda diz de qual.

VIII.1.6 Duas leituras diferentes da mesma posição

Fica um enigma na tabela: no passo 0 a velocidade já é $-0{,}1635$ m/s, mas a posição ainda é `10.0000`. Se o corpo tem velocidade, ele deveria ter descido. A explicação é o adjetivo guardado lá atrás: `getMotionState()` devolve a transformação **interpolada**, e não a transformação do corpo.

Uma instrumentação mínima — o mesmo cenário, imprimindo as duas leituras lado a lado e comparando com a queda livre analítica $y(t) = 10 - \frac{1}{2}gt^2$ — separa os três valores:

```
step          t          ms.y       body.y     freefall
    0     0.0000      10.00000      9.99728      9.99864
   10     0.1667       9.85013      9.82015      9.83514
   20     0.3333       9.42775      9.37053      9.39914
   30     0.5000       8.73287      8.64840      8.69064
```

Listagem 268: Medição comparando `motionState`, `btRigidBody::getWorldTransform` e a solução analítica.

Duas diferenças, e ambas fecham na terceira casa decimal:

- **`ms.y` menos `body.y`** vale 0,08447 m no passo 30. A velocidade ali é −5,069 m/s, e 0,08447/5,069 = 0,01666 s — **exatamente um** *dt*. O *motion state* está sempre um passo atrás do corpo: ele é a posição que o renderizador deve desenhar, não a que o *solver* está usando.
- **`body.y` menos a queda livre** vale −0,04224 m no passo 30 — o corpo caiu *mais* que a fórmula. Isso é o viés conhecido do integrador de Euler semi-implícito (simplético), que atualiza a velocidade antes da posição e acumula um erro $\frac{1}{2} g\, dt\, t$. Substituindo: $0{,}5 \times 9{,}81 \times 0{,}016667 \times 0{,}51667 = 0{,}04224$. O erro previsto e o erro medido coincidem em cinco casas.

Regra — escolha a leitura conforme o consumidor

Use `getMotionState()->getWorldTransform()` para **desenhar**: é a posição suavizada, e é o que evita tremor quando a taxa de quadros não é múltipla do passo físico. Use `body->getWorldTransform()` para **decidir**: *raycast*, lógica de jogo, gravação em CSV, teste automatizado. Misturar as duas produz um sistema em que o que se vê e o que se testa discordam por um quadro — e um quadro é suficiente para uma bala passar por uma parede.

Armadilha — dois alvos com o mesmo nome disputam `dist/bin`

Uma medição colateral, e desconfortável. O alvo do `src/collision/` chama-se `main`, e o do `src/sandbox` também:

```
src/ collision /main: cpp_LINKER
src/sandbox/main:   cpp_LINKER
```

Listagem 269: `ninja -C build -t targets all | grep main`

O Meson aceita os dois — eles vivem em diretórios de *build* distintos — mas ambos declaram `install: true`, e a instalação copia os dois para `dist/bin/main`. Quem vencer a corrida fica. Rodando `./dist/bin/main` hoje sai o demo de JSON do `sandbox`, não a simulação de queda livre; para ver a queda é preciso executar `./build/src/collision/main`. **Dê ao alvo o nome do subprojeto**, não o nome do arquivo: `executable('collision', ...)`. Nomear alvos como arquivos é a maneira mais fácil de criar uma colisão de *namespace* num diretório de instalação plano.

VIII.2 Renderizando a física

O subprojeto `src/sfml_bullet/` acrescenta ao anterior uma janela SFML, interação por mouse e um *game loop*. E acrescenta também os quatro problemas que aparecem sempre que física e desenho se encontram — três deles resolvidos no código, e um deles ainda aberto e mensurável.

A separação de responsabilidades é a de sempre, e vale enunciá-la antes do código: `PhysicsWorld` (em `physics_world.hpp/cpp`) não conhece o SFML; `Simulation` (em `simulation.hpp/cpp`) não conhece o

Bullet a não ser através de `BodyHandle`. Tudo o que atravessa a fronteira são dois *aggregates*: `BodyDef` na ida e `BodyHandle` na volta.

```
enum class ShapeType { Sphere, Box };

struct BodyDef {          // o que a camada visual PEDE
    ShapeType type;
    float     x, y;       // posição inicial em METROS
    float     half_extent;// raio, ou meia-aresta da caixa
    float     mass;       // 0 = estático
    float     restitution ;// elasticidade [0, 1]
    float     friction ;
};

struct BodyHandle {       // o que a camada de física DEVOLVE
    btRigidBody*      body;
    btCollisionShape* shape;
    ShapeType         type;
    float             half_extent;
};
```

Listagem 270: Os dois tipos de vocabulário da fronteira — `src/sfml_bullet/physics_world.hpp`.

VIII.2.1 Problema 1: um motor 3D fazendo física 2D

Bullet não tem modo 2D. A solução é restringir os graus de liberdade por corpo, e são duas linhas:

```
body->setLinearFactor(btVector3(1, 1, 0));   // só se move em X e Y
body->setAngularFactor(btVector3(0, 0, 1));  // só rota em torno de Z
```

Listagem 271: O detalhe mais importante do subprojeto.

Os dois vetores são multiplicadores aplicados à velocidade a cada passo: um zero em Z significa "qualquer velocidade que apareça neste eixo é descartada". O resultado é comportamento perfeitamente bidimensional com a qualidade do *solver* 3D. As caixas continuam sendo `btBoxShape` tridimensionais — apenas nunca saem do plano.

Armadilha — o fator não é uma restrição de posição

`setLinearFactor` zera a *velocidade* em Z, não a *posição*. Um corpo criado com $z \neq 0$, ou empurrado para fora do plano por uma penetração resolvida pelo *solver*, fica lá para sempre — ele apenas para de acelerar naquele eixo. Crie tudo com $z = 0$ e não confie no fator para consertar geometria inicial errada.

VIII.2.2 Problema 2: dois sistemas de coordenadas incompatíveis

Bullet mede em metros, com Y crescendo para *cima* e a origem onde você quiser. SFML mede em pixels, com Y crescendo para *baixo* e a origem no canto superior esquerdo. A conversão precisa de uma escala e de uma inversão:

```
constexpr float SCALE    = 80.f;   // 1 metro Bullet = 80 pixels SFML
constexpr float OFFSET_X = 400.f;  // x = 0 (Bullet) cai no meio da janela
constexpr float OFFSET_Y = 550.f;  // y = 0 (Bullet) cai 50 px acima da borda

inline sf::Vector2f to_sfml(float bx, float by) {
    return { bx * SCALE + OFFSET_X, -by * SCALE + OFFSET_Y };
}
inline sf::Vector2f to_bullet(float sx, float sy) {
```

```
    return { (sx − OFFSET_X) / SCALE, −(sy − OFFSET_Y) / SCALE };
}
```

Listagem 272: As três constantes e as duas funções — `src/sfml_bullet/simulation.hpp`.

Com uma janela de 800×600 pixels e 80 pixels por metro, o mundo visível mede 10×7,5 metros: $x \in [-5, +5]$ e $y \in [-0{,}625,\ +6{,}875]$. Vale fazer essa conta uma vez e anotá-la: ela é a régua com que qualquer posição do código passa a ser conferida.

Regra — a escala é uma decisão de simulação, não de apresentação

O Bullet é ajustado para objetos de escala humana — entre alguns centímetros e algumas dezenas de metros. Modelar um personagem com um metro de altura como um corpo de *oitenta* unidades (isto é, trabalhar em pixels) degrada a estabilidade do *solver*: os limites de tolerância, a penetração permitida e o amortecimento padrão são todos calibrados para metros. Simule em metros e converta na hora de desenhar — nunca o contrário.

VIII.2.3 Problema 3: o quadro lento

```
void Simulation::update(float dt) {
    if (paused_) return;
    float clamped = std::min(dt, 0.05f);   // no máximo 50 ms por quadro
    physics_.step(clamped);
    sync_entities();
}

void PhysicsWorld::step(float dt) {
    world_->stepSimulation(dt, 10, 1.f / 60.f);
}
```

Listagem 273: O *clamp*, em `Simulation::update` — e o passo, em `PhysicsWorld::step`.

Sem o `clamp`, uma pausa de dois segundos — o usuário arrastou a janela, o sistema entrou em suspensão — pediria ao Bullet 120 subpassos de uma vez. Cada subpasso custa tempo real, e se ele custar mais do que o tempo que representa, o próximo quadro chega ainda mais atrasado e pede ainda mais subpassos: é a *spiral of death*. Limitar o *dt* significa que a simulação "perde tempo" em vez de tentar recuperá-lo: um compromisso quase sempre correto em aplicação interativa, e quase sempre errado em simulação determinística (Seção VIII.6).

Repare também no teto implícito: com `maxSubSteps = 10` e `fixedTimeStep = 1/60`, o Bullet consome no máximo 10/60 = 0,1667 s por chamada. Qualquer *dt* acima disso é **silenciosamente descartado** — o *clamp* de 50 ms apenas garante que nunca se chega perto desse limite.

VIII.2.4 Problema 4: a física e o desenho discordam em 392 pixels

Este é o problema que o subprojeto *não* resolve, e é o mais instrutivo dos quatro. `PhysicsWorld::add_body` constrói caixas assim:

```
shape = std::make_unique<btBoxShape>(
    btVector3(def.half_extent, def.half_extent, def.half_extent)
);
```

Listagem 274: Uma só medida para os três eixos — `src/sfml_bullet/physics_world.cpp`.

`BodyDef` tem um único campo `half_extent`, e ele é aplicado aos três eixos. Logo, **toda caixa deste subprojeto é um cubo**. Agora leia o chão que `Simulation::create_boundaries()` cria: centro em $(0, -0{,}5)$ e `half_extent` $= 5{,}1$ — "10,2 m de largura para cobrir a tela". A largura sai correta; a altura também sai 10,2 m. O topo do "chão" não está em $y \approx 0$: está em $-0{,}5 + 5{,}1 = 4{,}6$ m.

Uma reprodução mínima, sem SFML, com as três fronteiras exatamente como o código as cria e uma esfera de 0,3 m solta do alto da área visível:

```
esfera repousa em y = 4.9000 m  -> pixel 158.0 (chao desenhado em 550 px)
topo do 'chao' fisico : y = 4.60 m -> pixel 182.0
face interna da parede esq.: x = -4.60 m -> pixel 32.0 (parede desenhada em 0..12 px)
```

Listagem 275: Reprodução dirigida da geometria de `create_boundaries()`, 600 passos a 60 Hz.

A esfera para em $y = 4{,}90$ m, ou seja, no pixel 158. O retângulo cinza que representa o chão é desenhado em `ground.setPosition(-5.f, 550.f)`: pixel 550. **A diferença é de 392 pixels**, quase dois terços da altura da janela. As paredes têm o mesmo problema em menor escala: a face interna da parede esquerda fica no pixel 32, e o retângulo é pintado de 0 a 12.

Armadilha — o motor não sabe o que você desenhou

Não existe nada no Bullet, no SFML ou no compilador capaz de detectar essa divergência: são dois conjuntos de números que ninguém obriga a concordar. As três defesas, em ordem de eficácia:

1. **Desenhe a partir do corpo físico**, nunca de constantes paralelas. O retângulo do chão deveria sair da AABB do corpo estático, pela mesma função `to_sfml` que posiciona as esferas.
2. **Ative o *debug drawer*** do Bullet (`btIDebugDraw` + `world->debugDrawWorld()`): ele desenha a geometria de colisão *real*, e um chão 392 px fora do lugar aparece no primeiro quadro.
3. **Dê ao `BodyDef` três meias-extensões** em vez de uma. O campo único não é uma simplificação inofensiva: ele torna impossível expressar a forma que o subprojeto precisa.

VIII.2.5 RAII sobre uma API que não o oferece — e o que ele não conserta

O `PhysicsWorld` deste subprojeto resolve o problema de gestão de memória da Seção VIII.1 pela via idiomática: cada ponteiro criado vira um `std::unique_ptr` guardado num membro, e a destruição acontece sozinha.

```
class PhysicsWorld {
public:
    PhysicsWorld(PhysicsWorld&&)            = default;   // <<< guarde esta linha
    PhysicsWorld& operator=(PhysicsWorld&&) = default;   // <<< e esta
    ~PhysicsWorld();
private:
    std::unique_ptr<btDefaultCollisionConfiguration>     config_;
    std::unique_ptr<btCollisionDispatcher>               dispatcher_;
    std::unique_ptr<btDbvtBroadphase>                    broadphase_;
    std::unique_ptr<btSequentialImpulseConstraintSolver> solver_;
    std::unique_ptr<btDiscreteDynamicsWorld>             world_;

    std::vector<std::unique_ptr<btRigidBody>>            rigid_bodies_;
    std::vector<std::unique_ptr<btDefaultMotionState>>   motion_states_;
    std::vector<std::unique_ptr<btCollisionShape>>       shapes_;
};
```

Listagem 276: A ordem de *declaração* passa a ser a ordem de destruição — `src/sfml_bullet/physics_world.hpp`.

Por que — por que a ordem de declaração é uma decisão de projeto

Os membros de uma classe são destruídos na **ordem inversa da declaração**. Aqui isso funciona a favor: `world_` é declarado depois de `config_`, `dispatcher_`, `broadphase_` e `solver_`, e portanto é destruído *antes* deles — que é o que o Bullet exige, já que o destrutor do mundo ainda usa a *broadphase* e o despachante para limpar os *proxies*. Escrever os cinco `unique_ptr` na ordem errada não gera aviso nenhum e produz um *crash* no encerramento. A ordem de declaração de membros é, nesta classe, código executável.

E então vem o resto da história. A tecla `R` do subprojeto reinicia a cena assim:

```
if (ev.key.code == sf::Keyboard::R) {
    physics_  = PhysicsWorld{};   // move-assign: destrói o mundo antigo
    entities_.clear();
    circles_.clear();
    rects_.clear();
    create_boundaries();
}
```

Listagem 277: O reset, em `Simulation::handle_events` — `src/sfml_bullet/simulation.cpp`.

A atribuição por movimento gerada por `= default` é **membro a membro, na ordem de declaração**, e cada `unique_ptr::operator=` destrói o objeto antigo *naquele instante*. Ou seja: o `config_` antigo morre primeiro, depois o `dispatcher_`, depois a `broadphase_`, e o `world_` antigo morre por **último** — a ordem exatamente oposta à que o destrutor tanto se esforçou para garantir. O destrutor, aliás, nem chega a rodar: uma atribuição não invoca `~PhysicsWorld`, então os corpos nunca são removidos do mundo.

Uma reprodução dirigida sob *AddressSanitizer*, com a mesma ordem de membros e um único corpo no mundo, confirma:

```
antes do reset
AddressSanitizer:DEADLYSIGNAL
ERROR: AddressSanitizer: SEGV on unknown address 0x0000000000b4
The signal is caused by a READ memory access.
    #0 btCollisionWorld::~btCollisionWorld() btCollisionWorld.cpp:89
    #2 btDiscreteDynamicsWorld::~btDiscreteDynamicsWorld()
    #5 std::__uniq_ptr_impl<btDiscreteDynamicsWorld, ...>::reset(...)
    #9 PW::operator=(PW&&)
   #10 main mv.cpp:37
```

Listagem 278: `g++ -fsanitize=address`; a linha 37 é o `p = PW{}`.

A linha 89 de `btCollisionWorld.cpp` é `getBroadphase()->destroyProxy(bp, m_dispatcher1)` — a *broadphase* já foi liberada três membros antes.

Armadilha — `= default` num tipo com dependências entre membros

`std::unique_ptr` garante que *cada* recurso é liberado uma vez. Ele não sabe nada sobre a **ordem relativa** em que os recursos podem ser liberados, e não tem como saber: essa informação está na documentação do Bullet, não no sistema de tipos. Quando os membros de uma classe têm dependências de tempo de vida entre si, `= default` para mover é um erro por omissão:

- se o tipo não precisa ser movível — e este não precisa —, escreva `PhysicsWorld(PhysicsWorld&&) = delete;` e deixe o compilador recusar o `physics_ = PhysicsWorld{}` em tempo de compilação;
- se precisa mesmo ser movível, escreva as operações à mão, liberando na mesma ordem do destrutor;
- ou dê ao objeto um método `reset()` explícito, que é o que a tecla `R` realmente queria.

A pista estava à vista o tempo todo: uma classe que declara destrutor *e* pede `= default` para as

operações de movimento está anunciando que gerencia recursos e, ao mesmo tempo, delegando ao compilador a única parte que exigia julgamento.

VIII.2.6 Da orientação do Bullet ao ângulo do SFML

A sincronização física → visual tem uma sutileza na rotação. O Bullet guarda orientação como *quaternion*; o SFML quer um ângulo em graus, no sentido horário.

```
1  btScalar  angle = t.getRotation().getAngle();   // sempre >= 0
2  btVector3 axis  = t.getRotation().getAxis();
3
4  float angle_deg = static_cast<float>(angle) * (180.f / 3.14159f);
5  if (axis.getZ() < 0.f) angle_deg = -angle_deg;  // 1: eixo pode ser -Z
6  e.visual->setRotation(-angle_deg);              // 2: SFML gira ao contrário
```

Listagem 279: As duas inversões de sinal — `Simulation::sync_entities`.

Armadilha — `getAngle()` nunca devolve ângulo negativo

Um quaternion representa a rotação de θ em torno de $\hat{n}$ e a de $-\theta$ em torno de $-\hat{n}$ com o mesmo par de valores até o sinal. `btQuaternion::getAngle()` devolve sempre a magnitude, no intervalo $[0, \pi]$; o sinal está no eixo. Quem esquece a consulta a `getAxis().getZ()` vê o objeto girar sempre para o mesmo lado. E a segunda inversão é independente da primeira: no SFML o eixo Y aponta para baixo, o que troca o sentido positivo de rotação. Duas convenções, dois sinais, e nenhum aviso do compilador — a mesma classe de erro do Seção VIII.2.4, em outro grau de liberdade.

VIII.3 Constraints, *ragdoll* e corpos deformáveis

O subprojeto `src/sfml_bullet2/` acrescenta o que faltava para modelar algo articulado — as *constraints* — e, junto, algo que nenhum corpo rígido consegue ser: uma corda que se deforma.

Por que — um corpo rígido não dobra, e o mundo é cheio de coisas que dobram

Todo o Seção VIII.1 tratou de objetos cuja forma é constante. Uma porta, um braço, uma corrente e um cabo não são isso. Há duas maneiras de chegar lá, e o subprojeto usa as duas: (i) manter os corpos rígidos e **limitar o movimento relativo** entre eles com restrições — é o *ragdoll*; ou (ii) trocar o corpo rígido por uma malha de partículas ligadas por molas — é o *soft body*, e é a corda. A primeira é barata e controlável; a segunda é cara e expressiva.

VIII.3.1 Grau de liberdade: o conceito que organiza tudo

Dois corpos livres no espaço têm doze graus de liberdade — seis cada um (três de translação, três de rotação). Uma *constraint* é uma equação que **remove** alguns desses graus. O que sobra é o nome da junta.

O `btConeTwistConstraint` merece a explicação completa, porque é o único que o subprojeto de fato usa. Ele fixa a posição relativa (remove os três graus de translação) e limita os três de rotação por três ângulos:

- `swingSpan1` — abertura máxima do cone num plano;

Tabela 95: As três *constraints* do vocabulário do Bullet, e o que cada uma modela.

Constraint	DOF relativos	Modela
`btHingeConstraint`	1 (dobradiça)	joelho, cotovelo, porta
`btConeTwistConstraint`	3 (cone + torção)	ombro, quadril, pescoço
`btPoint2PointConstraint`	3 (livre)	elo de corrente, pêndulo

- `swingSpan2` — abertura máxima no plano perpendicular;
- `twistSpan` — torção em torno do próprio eixo do "osso".

A imagem correta é um cone: o eixo do membro pode apontar para qualquer direção *dentro* de um cone elíptico de aberturas `swingSpan1` e `swingSpan2`, e pode girar em torno de si mesmo até `twistSpan`.

VIII.3.2 O *ragdoll* medido, e o README que mente

O *ragdoll* é um grafo: corpos como nós, juntas como arestas. Vale ler o código antes de acreditar em qualquer descrição dele.

```
1  struct Ragdoll {
2      enum Segment { TORSO=0, HEAD, ARM_L, ARM_R, LEG_L, LEG_R, COUNT };
3      btRigidBody* bodies[COUNT] = {};
4      void build(PhysicsWorld& pw, float cx, float cy);
5      void apply_impulse(const btVector3& impulse, const btVector3& origin);
6  };
```

Listagem 280: A enumeração inteira — `src/sfml_bullet2/ragdoll.hpp`.

São **seis** segmentos, não sete: torso, cabeça, dois braços e duas pernas — `COUNT` vale 6. E as juntas construídas em `ragdoll.cpp` são cinco, **todas** `btConeTwistConstraint`: pescoço, dois ombros e dois quadris (Tabela 96). Não há um único `btHingeConstraint` no subprojeto — não há cotovelo nem joelho, porque não há antebraço nem canela para dobrar.

Tabela 96: Os limites angulares de fato programados em `ragdoll.cpp`, convertidos de radianos para graus.

Junta	swing1	swing2	twist	Observação
pescoço	27°	27°	18°	mais `setDamping(0.3f)`
ombros	90°	90°	72°	os mais livres
quadris	54°	54°	18°	torção quase travada

Os números contam a anatomia sozinhos: o ombro é a junta mais livre do corpo, o quadril tem quase a mesma abertura mas quase nenhuma torção, e o pescoço mal se mexe. Sintonizar um *ragdoll* é, em boa medida, mexer nesses nove números até que as poses impossíveis parem de acontecer.

Armadilha — o README descreve um cotovelo que não existe

O `README.md` do `src/sfml_bullet2/` e os comentários no topo de `ragdoll.hpp` descrevem `btHingeConstraint` com limite de $-45°$ a $+90°$ para cotovelo e joelho, e uma corda feita de cápsulas ligadas por `btPoint2PointConstraint`. Nenhuma das duas coisas está no código. Documentação e comentário de cabeçalho não são compilados; a única descrição de um programa que não pode divergir dele é o próprio programa. Ao estudar um subprojeto deste repositório — ou de qualquer outro — comece pelo `.cpp`, e use o README para descobrir a *intenção*, não o *estado*.

VIII.3.3 Duas linhas fáceis de esquecer

```
world->addConstraint(c, true);   // true = desativa colisão entre os dois corpos

// ... e em Ragdoll::apply_impulse:
bodies[i]->activate(true);       // sem isto, um corpo adormecido ignora o impulso
float scale = 1.f / (1.f + dist * dist * 0.5f);   // atenuação quadrática
bodies[i]->applyCentralImpulse(impulse * scale);
```

Listagem 281: O último argumento de `addConstraint`, e o despertar antes do impulso.

Armadilha — corpos ligados precisam ignorar-se mutuamente

O *disable collision between linked bodies* não é otimização: dois corpos unidos por uma junta se *sobrepõem por construção* — o braço encosta no torso. Sem o `true`, o *solver* de colisão empurra os dois para longe enquanto a *constraint* os puxa de volta, a cada iteração de Gauss-Seidel. O resultado é tremor; em cadeias longas, divergência numérica. E o `activate(true)` é o outro lado da moeda da *deactivation* medida na Seção VIII.1.5: um *ragdoll* parado há três segundos está dormindo, e uma explosão que não o acorda simplesmente não acontece.

VIII.3.4 A corda é um *soft body*, e o mundo muda por causa dela

Aqui está a diferença arquitetural mais importante entre o `src/sfml_bullet/` e o `src/sfml_bullet2/`, e ela não está na corda: está no *mundo*.

```
// btDefaultCollisionConfiguration              → só rigid-rigid
// btSoftBodyRigidBodyCollisionConfiguration → rigid-rigid, rigid-soft, soft-soft
config_ = std::make_unique<btSoftBodyRigidBodyCollisionConfiguration>();
...
// btDiscreteDynamicsWorld → btSoftRigidDynamicsWorld
world_ = std::make_unique<btSoftRigidDynamicsWorld>(
    dispatcher_.get(), broadphase_.get(), solver_.get(), config_.get());

// o solver de soft body não recebe o mundo: ele lê deste bloco compartilhado
soft_info_.m_broadphase = broadphase_.get();
soft_info_.m_dispatcher = dispatcher_.get();
soft_info_.m_gravity    = btVector3(0.f, -9.81f, 0.f);
soft_info_.m_sparsesdf.Initialize();
```

Listagem 282: Três trocas de tipo, e um bloco de parâmetros extra — `src/sfml_bullet2/physics_world.cpp`.

Armadilha — a gravidade precisa ser escrita duas vezes

`world_->setGravity(...)` governa os corpos rígidos; `soft_info_.m_gravity` governa os *soft bodies*. São dois campos distintos e o Bullet não os sincroniza. Esquecer o segundo produz uma corda que flutua enquanto tudo o mais cai — e o sintoma é tão estranho que ninguém suspeita da gravidade. O `m_sparsesdf.Initialize()` é igualmente obrigatório: é o campo de distância assinada usado na colisão soft-rigid, e sem ele a corda atravessa o chão.

Com o mundo certo, a corda cabe em poucas linhas — e cada linha é um parâmetro de material que vale conhecer:

```
body = btSoftBodyHelpers::CreateRope(
    pw.soft_info(),
    btVector3(x0, y0, 0.f), btVector3(x1, y1, 0.f),
    segments,
    1);                          // bitmask de âncoras: 1 = nó 0, 2 = último, 3 = ambos

btSoftBody::Material* mat = body->m_materials[0];
```

```
 8  mat->m_kLST = 0.8f;          // Linear Stiffness: resistência ao alongamento [0,1]
 9  mat->m_kAST = 0.5f;          // Angular Stiffness: resistência à curvatura
10
11  body->m_cfg.kDP  = 0.05f;    // damping do movimento dos nós
12  body->m_cfg.kDG  = 0.01f;    // drag: arrasto do ar
13  body->m_cfg.kSHR = 1.0f;     // shear stiffness
14  body->m_cfg.kCHR = 1.0f;     // rigid contact hardness
15
16  body->setTotalMass(1.0f, true);        // 1 kg distribuído pelos nós
17  body->generateBendingConstraints(2);   // links de 2ª ordem: resiste a dobrar
18
19  if (attach) {                          // amarra a ponta a um corpo rígido
20      int last = body->m_nodes.size() - 1;
21      body->appendAnchor(last, attach, false, 1.0f);
22  }
```

Listagem 283: A corda inteira — `src/sfml_bullet2/rope.cpp`.

O modelo por trás disso é simples de enunciar: um *soft body* é um conjunto de **nós** com massa, ligados por **links** que se comportam como molas rígidas. `kLST` próximo de 1 faz os links quase inextensíveis; `kLST` próximo de 0 faz uma tira de borracha. `generateBendingConstraints(2)` acrescenta links entre nós *não adjacentes* (a dois de distância), e é isso que dá à corda resistência a dobrar — sem eles ela se comportaria como uma corrente frouxa.

Armadilha — um *soft body* não obedece a `setLinearFactor`

`setLinearFactor` e `setAngularFactor` são métodos de `btRigidBody`. Um `btSoftBody` não os tem: seus nós se movem livremente nos três eixos. A corda deste subprojeto fica no plano apenas porque é *criada* com $z = 0$ nas duas pontas e a gravidade só age em Y — e sairá do plano no instante em que colidir de raspão com um corpo rígido. Num mundo "2D" construído sobre um motor 3D, cada tipo novo de objeto precisa da sua própria resposta para a pergunta "o que o mantém no plano?".

Armadilha — quem cria com `new` continua devendo o `delete`

`btSoftBodyHelpers::CreateRope` devolve um `btSoftBody*` alocado com `new`, e `PhysicsWorld::add_soft` apenas o registra no mundo — não guarda *ownership* nenhum. `removeSoftBody` desregistra, mas não destrói. O *ragdoll* tem o mesmo padrão nas *constraints*: os cinco `new btConeTwistConstraint` não têm `delete` correspondente. Repare no contraste com os corpos rígidos, que passam por `take_shape`, `take_body` e `take_state` e ficam sob `std::unique_ptr`: o subprojeto *sabe* como fazer certo, e faz certo em dois dos quatro tipos de recurso. Vazamento parcial é o estado natural de um envoltório RAII escrito aos poucos — audite cada `new` da API estrangeira, não só os mais visíveis.

VIII.4 Qt e SFML na mesma janela

Os subprojetos `src/qt_sfml/` e `src/qt_sfml2/` resolvem um problema diferente: como ter os *widgets* do Qt — *sliders*, botões, *layouts* — em volta de uma área desenhada em tempo real pelo SFML. As duas bibliotecas querem, cada uma, ser dona da janela e do laço de eventos.

Por que — duas bibliotecas, um só sistema de janelas

Tanto o Qt quanto o SFML são, por baixo, envoltórios sobre o mesmo recurso do sistema operacional: uma janela nativa (um *XID* no X11, um `HWND` no Windows) e uma fila de eventos ligada a ela. Nenhum dos dois foi escrito esperando dividir. A integração não consiste em "fazê-los cooperar": consiste em

escolher **um dono para cada recurso**. O Qt fica com a janela de nível superior, o *layout* e a fila de eventos; o SFML fica com os pixels de uma sub-região. E cada linha da solução abaixo é uma renúncia explícita de um dos dois a alguma coisa.

VIII.4.1 O truque: entregar o *handle* nativo ao SFML

```
1   // Renúncia 1: retornar nullptr diz ao Qt para não criar um QPainter.
2   // O widget deixa de ter um "paint engine" — o SFML assume esta área.
3   QPaintEngine* paintEngine() const override { return nullptr; }
4
5   // Renúncia 2: o SFML pede emprestado o handle nativo do widget.
6   void SFMLWidget::showEvent(QShowEvent*) {
7       if (! initialized_ ) {
8           window_.create(sf::WindowHandle(winId()));  // XID no Linux, HWND no Windows
9           timer_.start(1000 / 60);
10          initialized_  = true;
11      }
12  }
13
14  // E o paintEvent fica VAZIO: quem pinta é o tick() do timer.
15  void SFMLWidget::paintEvent(QPaintEvent*) { }
```

Listagem 284: As duas renúncias que fazem a integração funcionar — `src/qt_sfml/sfml_widget.cpp`.

E três atributos de *widget* completam o acordo, cada um desligando uma etapa do desenho do Qt: `WA_PaintOnScreen` (não pinta sobre este *widget*), `WA_OpaquePaintEvent` (não limpa o fundo antes) e `WA_NoSystemBackground` (não preenche com cor de fundo). Sem eles, o resultado é cintilação ou o conteúdo do SFML sendo apagado a cada quadro.

Armadilha — a criação da janela SFML tem de esperar o `showEvent`

`winId()` só devolve um *handle* válido depois que o sistema de janelas criou a janela nativa — o que acontece na exibição, não na construção do *widget*. Chamar `window_.create(winId())` no construtor produz um *handle* inválido, e o sintoma é uma área preta que nunca desenha. Daí a inicialização preguiçosa com a *flag* `initialized_`. O mesmo vale para `resizeEvent`, que precisa checar a mesma *flag* antes de chamar `window_.setSize()`.

VIII.4.2 `QTimer` como *game loop*

Regra — nunca escreva `while(true)` dentro de uma aplicação Qt

Um *game loop* tradicional monopoliza a *thread* e o laço de eventos do Qt nunca roda: os botões param de responder, a janela não redesenha, o sistema operacional marca a aplicação como travada. O `QTimer` inverte o controle — o laço de eventos continua sendo o dono, e chama `tick()` sessenta vezes por segundo. É a mesma inversão que o `io_context` do Asio impõe (Seção VII.2), e a mesma que qualquer *framework* de UI impõe.

A comunicação entre os *widgets* de controle e a área de desenho é por *signals and slots* — o mecanismo do Qt que é, na prática, o padrão *Observer* da Seção V.2 com suporte de pré-processador. O `moc` (*meta-object compiler*) lê os cabeçalhos marcados com `Q_OBJECT` e gera, para cada classe, uma tabela de despacho que permite conectar um sinal a um *slot* pelo nome, em tempo de execução. O `meson.build` do `src/qt_sfml/` torna essas etapas visíveis, o que é raro: `qt6.compile_ui` roda o `uic` sobre o arquivo `.ui` do Designer,

`qt6.compile_moc` roda o `moc` sobre os dois cabeçalhos com `Q_OBJECT`, e os arquivos gerados entram na lista de fontes do executável como qualquer `.cpp`.

Armadilha — esquecer o `Q_OBJECT` ou o `moc`

Uma classe que herda de `QObject` sem a macro `Q_OBJECT` compila e liga sem reclamação — e simplesmente não emite sinais. Um cabeçalho com a macro que não é listado em `compile_moc` produz o erro de ligação `undefined reference to vtable for SFMLWidget`, que não menciona o `moc` em lugar nenhum. Os dois sintomas são desproporcionais à causa, e os dois são a primeira coisa a conferir num projeto Qt novo.

Há um segundo laço de eventos escondido nessa arquitetura, e o `src/qt_sfml2/` não o esquece:

```
1 void SFMLWidget::tick() {
2     // Obrigatório: sem drenar a fila, o X11 marca a janela como "not responding"
3     sf::Event evt;
4     while (window_.pollEvent(evt)) {}
5     ...
6 }
```

Listagem 285: Drenar a fila do SFML mesmo sem usá-la — `src/qt_sfml2/sfml_widget.cpp`.

O `sf::RenderWindow` criado sobre o *handle* emprestado recebe eventos do servidor de janelas mesmo que o Qt seja quem os trate. Uma fila que nunca é esvaziada é uma janela que o gerenciador considera morta.

VIII.4.3 Uma medição incômoda: 60 quadros por segundo não são 60

```
1 timer_.start(1000 / 60);          // int / int = 16  → 16 ms → 62,5 Hz
2 constexpr float dt = 1.f / 60.f;  // 16,667 ms       → 60,0 Hz
```

Listagem 286: Duas constantes, dois arredondamentos diferentes — ambos os subprojetos.

`1000 / 60` é divisão *inteira* e vale 16, não 16,667. O `QTimer` dispara então a cada 16 ms — 62,5 vezes por segundo — enquanto o `tick()` avança a simulação em 1/60 de segundo a cada disparo. A simulação corre 62,5/60 = 1,0417, ou seja **4,2% mais rápido que o relógio de parede**. Em dez minutos de execução, são vinte e cinco segundos de tempo simulado a mais.

Armadilha — o passo fixo só é correto se o relógio concordar com ele

O erro tem três causas somadas, e todas são comuns:

1. **Divisão inteira** onde se queria um período fracionário. Escreva `std::chrono::milliseconds(16)` e o número fica visível, ou prefira `16667` microssegundos.
2. **Um período nominal tratado como período real.** Um `QTimer` garante "não antes de 16 ms", nunca "exatamente 16 ms": sob carga o intervalo cresce, e a simulação passa a correr *devagar*, não rápido.
3. ***dt* constante desacoplado da medição.** O `src/sfml_bullet/` acerta aqui: mede o intervalo real com `sf::Clock`, aplica o *clamp* e passa o valor medido. Os dois subprojetos Qt assumem o valor nominal.

Nenhum aviso do compilador, nenhum teste que falhe — e o único sintoma é que a gravação em vídeo não bate com o cronômetro. Meça o intervalo, ou acumule o tempo real e consuma-o em passos fixos (*fixed timestep accumulator*).

VIII.4.4 `qt_sfml`: a física escrita à mão, e o seu defeito clássico

A bola do `src/qt_sfml/` não usa motor nenhum — e é justamente por isso que ela ilustra o defeito que todo motor precisa evitar:

```
1  if (pos.x − r < 0.f || pos.x + r > w) velocity_.x *= −1.f;
2  if (pos.y − r < 0.f || pos.y + r > h) velocity_.y *= −1.f;
```

Listagem 287: Rebatimento por inversão de sinal — `SFMLWidget::tick`.

Armadilha — inverter o sinal não é o mesmo que rebater

O teste olha a *posição*, mas a correção mexe na *velocidade*, e nada recoloca a bola dentro da área. Se num quadro qualquer a bola ficar além da borda — e isso acontece a cada vez que o *slider* de raio a engorda, ou que a janela é encolhida — a condição continua verdadeira no quadro seguinte, o sinal inverte de novo, e a bola fica vibrando presa na parede. A correção canônica tem duas partes: reposicionar (`pos.x = r`) **e** só inverter se a velocidade apontar para fora (`if (pos.x - r < 0 && velocity_.x < 0)`). É exatamente o que um *solver* de contato faz — separar penetração de velocidade — e por isso esse subprojeto é o melhor argumento a favor de usar um.

O ajuste de velocidade, em contrapartida, está bem-feito e vale citar: ele preserva a direção e altera só a magnitude, normalizando com `std::hypot` — que evita o *overflow* intermediário de $\sqrt{x^2+y^2}$ calculado ingenuamente.

```
1  void SFMLWidget::set_speed(float speed) {
2      speed_ = speed;
3      const float len = std::hypot(velocity_.x, velocity_.y);
4      if (len > 0.f) velocity_ = (velocity_ / len) * speed_;
5  }
```

Listagem 288: Mudar o módulo sem mudar a direção.

VIII.4.5 `qt_sfml2`: um renderizador 3D escrito à mão sobre um canvas 2D

O `src/qt_sfml2/` deixou de ser "o `src/qt_sfml/` com Bullet". Hoje ele é uma cena **tridimensional** — uma sala cúbica de $10 \times 10 \times 10$ m com esferas, caixas e cilindros quicando — projetada à mão em duas dimensões e desenhada com as primitivas 2D do SFML (Tabela 97). Não há OpenGL escrito pelo subprojeto, não há biblioteca de matemática gráfica: a câmera, a projeção e a ordenação por profundidade são cerca de setenta linhas de código próprio, e é isso que o torna o melhor lugar do repositório para aprender como uma imagem 3D é feita.

Tabela 97: Os dois subprojetos de integração Qt + SFML, como estão hoje.

Aspecto	`qt_sfml`	`qt_sfml2`
dimensão	2D puro	**3D projetado à mão**
física	rebatimento à mão	Bullet3, seis planos estáticos
gravidade	não	$-9{,}82$ m/s^2
restrição 2D	—	**nenhuma**: é 3D de verdade
colisões	bordas da área	entre todos os corpos e as paredes
câmera	—	orbital, mouse arrasta e roda
interação	*slider*	*spawn*, explosão, gravidade

A sala. Seis `btStaticPlaneShape` — planos infinitos, massa zero — fecham um cubo de aresta 10 m, com restituição 0,85 e fricção 0,4. Plano infinito é a forma de colisão mais barata que existe: não tem AABB limitada, não tem vértices, e o teste ponto-plano é um produto escalar.

```
const float h = half_room_;                  // 5.0 → sala 10x10x10
add_static_plane({  0.f,   1.f,   0.f }, -h);  // chão
add_static_plane({  0.f,  -1.f,   0.f }, -h);  // teto
add_static_plane({  1.f,   0.f,   0.f }, -h);  // parede esquerda
add_static_plane({ -1.f,   0.f,   0.f }, -h);  // parede direita
add_static_plane({  0.f,   0.f,   1.f }, -h);  // parede frontal
add_static_plane({  0.f,   0.f,  -1.f }, -h);  // parede traseira
```

Listagem 289: A sala inteira em seis linhas — `src/qt_sfml2/physics_world.cpp`.

A câmera orbital. A posição do observador é dada em coordenadas esféricas por três números: azimute θ, elevação φ e raio r. O mouse arrastado muda θ e φ; a roda muda r.

$$\mathbf{p}_{\text{cam}} = r\,\big(\sin\varphi\,\sin\theta,\ \ \cos\varphi,\ \ \sin\varphi\,\cos\theta\big)$$

A partir dela monta-se uma base ortonormal: `forward` aponta para o centro da sala, `right` é o produto vetorial de `forward` com o "para cima" do mundo, e `up` fecha o triedro.

```
cam_forward_ = normalize(-cam_pos_);
Vec3f r = cross(cam_forward_, {0.f, 1.f, 0.f});   // world up
float rlen = length(r);
if (rlen < 1e-4f) { r = {0.f, 0.f, 1.f}; rlen = 1.f; }  // fallback no polo
cam_right_ = r / rlen;
cam_up_    = cross(cam_right_, cam_forward_);
```

Listagem 290: A base da câmera, com o cuidado que salva os polos — `update_camera_matrix()`.

Armadilha — o produto vetorial degenera exatamente onde o usuário quer ir

Quando a câmera está sobre o polo, `forward` fica paralelo ao "para cima" do mundo e o produto vetorial devolve o vetor nulo: a base explode e a imagem some. O subprojeto se defende **duas** vezes, e as duas são necessárias:

1. o arrasto do mouse limita φ a $[0{,}1,\ \pi - 0{,}1]$ radianos, mantendo a câmera a uns 6° de cada polo;
2. mesmo assim, o cálculo testa `rlen < 1e-4f` e cai num eixo alternativo.

Defender-se apenas com o *clamp* deixa o programa à mercê de qualquer outro caminho que escreva φ — um botão "vista de topo", a leitura de um arquivo de cena. Trate a degenerescência no lugar onde ela é matemática, não apenas onde ela é entrada do usuário.

A projeção perspectiva. Três passos, e nenhuma matriz 4×4:

```
float dx = wp.x() - cam_pos_.x, dy = ..., dz = ...;     // 1. vetor olho→ponto

float vx =  dot(d, cam_right_);                          // 2. base da câmera
float vy = -dot(d, cam_up_);                             //    Y invertido p/ tela
float vz =  dot(d, cam_forward_);                        //    profundidade

if (vz < near_plane_) vz = near_plane_;                  // 3. evita divisão por ~0

float scale = (H / 2.f) / (fov_scale_ * vz);             //    divisão perspectiva
return { W/2.f + vx * scale, H/2.f + vy * scale };
```

Listagem 291: De ponto do mundo a pixel — `SFMLWidget::project3D`.

A linha que faz a perspectiva é a divisão por `vz`: quanto mais longe o ponto, menor o fator de escala, e é só isso que separa uma projeção perspectiva de uma ortográfica. O `fov_scale_` é $\tan(\text{FOV}/2)$ — menor valor, campo de visão mais estreito, mais "zoom". E o `near_plane_` existe porque um ponto atrás da câmera tem `vz` negativo e produziria uma imagem espelhada no infinito.

A ordenação por profundidade. Sem *z-buffer*, a única maneira de um objeto distante não aparecer na frente de um próximo é desenhar de trás para frente — o *algoritmo do pintor*:

```
1  float depth = dot(pos - cam_pos_, cam_forward_);
2  ...
3  std::sort(entries.begin(), entries.end(),
4            [](const Entry& a, const Entry& b){ return a.depth > b.depth; });
```

Listagem 292: Ordenar pelo centro de cada objeto — `SFMLWidget::draw_objects`.

Armadilha — o algoritmo do pintor erra, e erra em silêncio

Ordenar por profundidade do *centro* pressupõe que um objeto está inteiramente à frente ou inteiramente atrás de outro. Isso é falso para objetos que se interpenetram, para objetos longos vistos de viés, e para o caso clássico de três objetos em ciclo de oclusão — em que nenhuma ordem é correta. O sintoma é uma esfera que "pisca" à frente de outra quando as duas passam perto. As saídas são o *z-buffer* por pixel (o que uma GPU faz) ou subdividir a geometria; nenhuma das duas cabe num canvas de `sf::CircleShape`. Aceitar o erro e saber onde ele aparece é uma decisão legítima — *não* saber é que não é.

O toque final é o *depth cue*: `depth_scale()` mapeia a profundidade de câmera do intervalo $[2, 27]$ m para um fator de 1,0 a 0,3, aplicado ao tamanho e ao brilho do objeto. É a pista visual mais barata que existe para dar sensação de distância — e a mais antiga: pintores a usam há seis séculos com o nome de perspectiva aérea.

Regra — o `tick()` condensa a arquitetura inteira do capítulo

Avançar a física, drenar eventos, limpar, ler a transformação de cada corpo e desenhar. É a **sincronização física → visual**, e ela é sempre nessa direção: o motor de física é a fonte da verdade, e o desenho é uma leitura dela. Repare que `PhysicsWorld` do `src/qt_sfml2/` não inclui um só cabeçalho do Qt, e que `MainWindow` não inclui um só cabeçalho do Bullet. Essa separação é o que permite ao mesmo mundo físico ser desenhado por um projetor caseiro hoje e por OpenGL amanhã.

VIII.5 Álgebra linear numérica

O subprojeto `src/calculus/` usa Eigen3 [9] para percorrer as quatro decomposições matriciais que sustentam praticamente toda computação numérica aplicada (Tabela 98). É o pré-requisito matemático da Seção VIII.6, e o vocabulário de qualquer trabalho com *machine learning*, controle ou processamento de sinais. O binário instalado chama-se `eigen_demo`, não `calculus`.

Por que — por que decompor em vez de calcular direto

Quase todo problema numérico interessante se reduz a "resolva $Ax = b$" ou a "descubra a estrutura de A". Atacar isso diretamente — eliminação a cada novo b, inversa explícita, determinante pela definição — é caro e numericamente frágil. Uma **decomposição** fatora A uma única vez num produto de matrizes com estrutura simples (triangulares, ortogonais, diagonais), e a partir daí cada pergunta vira uma operação barata sobre os fatores. Trocar $O(n^3)$ repetido por $O(n^3)$ uma vez mais $O(n^2)$ por pergunta é o argumento óbvio; o argumento importante é o outro: as matrizes ortogonais preservam norma, e por isso não amplificam erro de arredondamento.

Tabela 98: As quatro decomposições, e quando cada uma é a ferramenta certa.

Decomposição	Quando usar	Custo
LU (`PartialPivLU`)	resolver $Ax = b$, determinante	$O(n^3)$
QR (`HouseholderQR`)	mínimos quadrados, sistema sobredeterminado	$O(mn^2)$
SVD (`JacobiSVD`)	compressão, PCA, pseudo-inversa, posto	$O(mn^2)$
autovalores (`SelfAdjointEigenSolver`)	PCA, estabilidade, vibrações	$O(n^3)$

VIII.5.1 LU: fatore uma vez, resolva muitas

A fatoração $A = P^{-1}LU$ escreve A como o produto de uma permutação de linhas P, uma triangular inferior L com uns na diagonal e uma triangular superior U. Resolver $Ax = b$ passa a ser resolver dois sistemas triangulares, cada um por substituição direta em $O(n^2)$.

A permutação não é detalhe de implementação: é o **pivoteamento parcial**. A cada coluna, o algoritmo troca linhas para colocar na diagonal o maior elemento em módulo. Sem isso, um pivô minúsculo dividiria a coluna inteira e explodiria o erro relativo — e um pivô exatamente zero pararia o algoritmo numa matriz perfeitamente invertível.

```
1  A:                                L (triangular  inferior ):
2      2.0000    1.0000   −1.0000      1.0000   0.0000   0.0000
3     −3.0000   −1.0000    2.0000      0.6667   1.0000   0.0000
4     −2.0000    1.0000    2.0000     −0.6667   0.2000   1.0000
5
6  U (triangular superior):      x (solução de Ax = b):
7     −3.0000   −1.0000    2.0000      2.0000   3.0000  −1.0000
8      0.0000    1.6667    0.6667
9      0.0000    0.0000    0.2000  norma do residuo ||Ax − b||: 8.88e−16
10                                   det(A): −1.0000
```

Listagem 293: Saída real de `./dist/bin/eigen_demo`, demo 1 (trechos).

Três coisas para ler nesses números:

- **A primeira linha de U é a segunda linha de A.** O pivô escolhido foi -3, e não o 2 que estava na posição $(0, 0)$: ali está o pivoteamento acontecendo, visível a olho nu.
- **O resíduo é** $8{,}9 \times 10^{-16}$, cerca de quatro vezes o épsilon de `double` ($2{,}2 \times 10^{-16}$). Não é zero, e não deveria ser: é o acúmulo esperado de algumas dezenas de operações em ponto flutuante. Um resíduo *exatamente* zero num sistema arbitrário seria motivo de desconfiança, não de comemoração.
- **O determinante é o produto da diagonal de U**, com o sinal da permutação: $(-3) \times 1{,}6667 \times 0{,}2 = -1{,}0$, e o sinal confere. Sai de graça, e nunca — nunca — se calcula determinante pela expansão de Laplace, cujo custo é $O(n!)$.

VIII.5.2 QR: regressão linear é mínimos quadrados

Quando há mais equações que incógnitas ($m > n$), o sistema é **sobredeterminado** e em geral não tem solução exata. O que se pede então é o x que minimiza $\|Ax - b\|^2$ — a soma dos quadrados dos erros. A fatoração $A = QR$, com Q ortogonal e R triangular superior, entrega isso diretamente, porque multiplicar por $Q^\top$ não muda a norma do resíduo.

O exemplo do subprojeto é literalmente uma regressão linear: quatro pontos (1; 2,1), (2; 3,9), (3; 6,2), (4; 7,8), duas colunas (intercepto e inclinação).

```
x (minimos quadrados: [intercepto, inclinacao]):
    0.1500    1.9400

reta ajustada: y = 0.1500 + 1.9400 * x
residuo ||Ax - b||: 0.2864
```

Listagem 294: Saída real de `./dist/bin/eigen_demo`, demo 2.

Armadilha — o README afirma outra reta, e a conta decide quem está certo

O `README.md` do `src/calculus/` promete $y \approx 0{,}08 + 1{,}93x$. A execução dá $y = 0{,}1500 + 1{,}9400x$. Quem está certo? Aqui a pergunta tem resposta fechada, porque a regressão simples tem fórmula:

$$\hat{\beta} = \frac{\sum (x_i - \bar{x})(y_i - \bar{y})}{\sum (x_i - \bar{x})^2} = \frac{9{,}7}{5{,}0} = 1{,}94, \qquad \hat{\alpha} = \bar{y} - \hat{\beta}\bar{x} = 5{,}0 - 1{,}94 \times 2{,}5 = 0{,}15.$$

E o resíduo: os valores ajustados são 2,09; 4,03; 5,97; 7,91, os erros são $+0{,}01$; $-0{,}13$; $+0{,}23$; $-0{,}11$, e a norma é $\sqrt{0{,}082} = 0{,}2864$ — os quatro dígitos impressos. O programa está certo, o README está errado, e a única razão de sabermos disso é que alguém refez a conta. **Quando um número da documentação diverge do número medido, a documentação perdeu** — mas confirme com uma terceira fonte antes de tirar conclusões sobre o código.

Por que — por que ninguém usa a equação normal

Resolver $A^\top Ax = A^\top b$ diretamente é matematicamente equivalente e numericamente pior: o número de condição de $A^\top A$ é o *quadrado* do de A. Como cada fator dez no número de condição custa aproximadamente um dígito decimal de precisão, elevar ao quadrado **dobra o número de dígitos perdidos**. Uma matriz com $\kappa = 10^8$ — perfeitamente comum em ajuste polinomial — ainda é tratável em `double`, mas $\kappa^2 = 10^{16}$ consome toda a precisão disponível e a resposta vira ruído. A QR chega ao mesmo mínimo sem nunca formar $A^\top A$.

VIII.5.3 SVD: a mais cara e a mais informativa

A decomposição em valores singulares escreve qualquer matriz — retangular, singular, o que for — como $A = U\Sigma V^\top$, com U e V ortogonais e Σ diagonal com entradas $\sigma_1 \geq \sigma_2 \geq \cdots \geq 0$. A leitura geométrica é a que vale a pena guardar: toda transformação linear é uma **rotação, seguida de esticamentos independentes ao longo dos eixos, seguida de outra rotação**. Os σ_i são esses fatores de esticamento.

O exemplo é a matriz 3×3 com os números de 1 a 9 — cujas linhas estão em progressão aritmética e, portanto, são linearmente dependentes.

```
valores singulares:  16.8481    1.0684    0.0000
numero de condicao: 7.84e+16
rank numerico: 2
```

Tabela 99: Cinco perguntas respondidas pela mesma decomposição.

Pergunta	Resposta pela SVD
qual o posto?	quantos σ_i acima do limiar numérico
o problema é bem condicionado?	$\sigma_{\max}/\sigma_{\min}$
como comprimir?	manter só os k maiores σ_i
quais as componentes principais?	SVD dos dados centralizados
como inverter o não inversível?	pseudo-inversa $A^{+} = V\Sigma^{+}U^{\top}$

```
4 erro de aproximacao rank-1: 1.0684
```

Listagem 295: Saída real de `./dist/bin/eigen_demo`, demo 3.

Quatro leituras, e uma delas é uma contradição aparente que ensina método:

1. **O terceiro valor singular é zero**, como tem de ser: a terceira linha é o dobro da segunda menos a primeira. O posto verdadeiro é 2.

2. **O posto numérico impresso é 2**, e o Eigen chegou a ele *contando* quantos σ_i superam um limiar relativo (aproximadamente $\varepsilon \cdot \sigma_{\max} \cdot \max(m, n)$). Posto não é uma propriedade decidível em ponto flutuante — é uma pergunta sobre uma tolerância.

3. **O número de condição impresso é** $7{,}8 \times 10^{16}$, o que *parece* contradizer o item anterior. Não contradiz: o programa calcula $\sigma_{\max}/\sigma_{\min}$ com um $\sigma_{\min}$ que é lixo de arredondamento. Para uma matriz de posto deficiente, esse quociente não mede nada — mede o inverso do erro de arredondamento da máquina. A grandeza útil nesse caso é o *número de condição efetivo*, σ_1/σ_r com r igual ao posto numérico: aqui $16{,}8481/1{,}0684 = 15{,}8$, um problema perfeitamente bem condicionado *no subespaço em que ele existe*.

4. **O erro da aproximação de posto 1 é** 1,0684 — **exatamente** σ_2**.** Isso não é coincidência: é o teorema de Eckart-Young, que garante que truncar a SVD nos k maiores valores singulares dá a melhor aproximação de posto k possível, e que o erro em norma de Frobenius é $\sqrt{\sigma_{k+1}^2 + \cdots + \sigma_n^2}$. Com $\sigma_3 \approx 0$, a raiz vale σ_2. Um teorema conferido em quatro casas decimais por um programa de dez linhas.

Armadilha — $\sigma_{\max}/\sigma_{\min}$ só é o número de condição se a matriz tiver posto cheio

A fórmula é ensinada sem a ressalva, e o resultado é código que classifica toda matriz singular como "mal condicionada" e toda matriz de posto deficiente como "instável", quando o problema real pode ser trivial depois de projetado no subespaço certo. Antes de dividir, conte o posto; e ao contar o posto, decida e documente o limiar. Se a decisão de negócio depende dele — e depende — ela não pode ficar escondida num padrão de biblioteca.

Armadilha — `JacobiSVD` não calcula U e V a menos que você peça

`Eigen::JacobiSVD<Matrix> svd(A)` calcula só os valores singulares; `svd.matrixU()` nesse caso dispara uma asserção. Os sinalizadores `ComputeThinU | ComputeThinV` pedem apenas as colunas necessárias — para uma matriz $m \times n$ com $m \gg n$, isso é a diferença entre guardar $m \times m$ e $m \times n$ valores. `ComputeFullU` existe e quase nunca é o que se quer.

VIII.5.4 Autovalores, PCA e a escolha do *solver*

Um autovetor de A é uma direção que A não gira, apenas escala; o fator é o autovalor. Numa **matriz de covariância** — que é sempre simétrica e semidefinida positiva — os autovetores são os eixos ao longo dos quais os dados variam de forma independente, e os autovalores são as variâncias nessas direções. Isso é a análise de componentes principais inteira, em uma frase.

```
C (matriz de covariancia):
    4.0000   2.0000   0.6000
    2.0000   2.0000   0.5000
    0.6000   0.5000   1.0000

variancia explicada:
  PC1: 76.8%   PC2: 13.3%
  PC3:  9.9%

autovalores (ordem CRESCENTE):
    0.6945   0.9326   5.3729

verificacao  C.v = lambda.v:
  lambda0 =  0.6945  residuo = 1.14e-15
  lambda1 =  0.9326  residuo = 4.90e-16
  lambda2 =  5.3729  residuo = 2.22e-15
```

Listagem 296: Saída real de `./dist/bin/eigen_demo`, demo 4.

Duas verificações que qualquer leitor pode refazer no papel:

- **A soma dos autovalores é o traço.** $0{,}6945 + 0{,}9326 + 5{,}3729 = 7{,}0000$, e $\mathrm{tr}(C) = 4 + 2 + 1 = 7$. É uma identidade exata da álgebra linear, e serve como teste de sanidade barato para qualquer rotina de autovalores que você escreva ou use.
- **A "variância explicada" é só uma divisão.** $5{,}3729/7 = 76{,}8\%$. A primeira componente principal responde por pouco mais de três quartos da variação; as três juntas, por definição, respondem por 100%.

Armadilha — o Eigen devolve autovalores em ordem *crescente*

`SelfAdjointEigenSolver` ordena do menor para o maior — o oposto da convenção da SVD, que ordena do maior para o menor. A primeira componente principal é a **última** coluna de `eigenvectors()`. É por isso que o laço da variância explicada no subprojeto é escrito ao contrário:

```
for (int i = 2; i >= 0; --i) {                  // do maior autovalor para o menor
    double pct = 100.0 * solver.eigenvalues()(i) / total;
    fmt::println("  PC{}: {:.1f}%", 3 - i, pct);
}
```

Uma PCA que reporta como "principal" a componente de *menor* variância é o resultado silencioso de ignorar isso, e o gráfico produzido parece plausível.

Para matrizes simétricas — e toda matriz de covariância é simétrica — `SelfAdjointEigenSolver` é a escolha obrigatória: explora a simetria, é mais rápido, é numericamente mais estável e **garante autovalores reais**. Usar o `EigenSolver` geral numa matriz simétrica devolve autovalores complexos com parte imaginária da ordem do erro de arredondamento — e o código que os consome passa a lidar com números complexos sem motivo.

Regra — nunca chame `.inverse()` para resolver um sistema

`x = A.inverse() * b` é mais lento, menos preciso e mais frágil que `x = A.lu().solve(b)`. Inverter calcula n soluções para obter uma; e a inversa explícita amplifica o erro de arredondamento de forma que a solução direta não amplifica. A inversa tem usos legítimos — expressões simbólicas, propagação de covariância — mas resolver sistema não é um deles. A mesma regra vale para a pseudo-inversa: prefira `svd.solve(b)` a `pinv(A) * b`.

Regra — escolha a decomposição pelo problema, não pelo hábito

Sistema quadrado e bem comportado: LU. Sistema sobredeterminado ou colunas quase dependentes: QR (e `ColPivHouseholderQR` se houver risco de posto deficiente). Posto, condicionamento, compressão ou PCA: SVD, e pague o preço. Matriz simétrica: sempre o *solver* auto-adjunto. Usar SVD para tudo funciona e custa de cinco a dez vezes mais que LU; usar LU para tudo é rápido até o dia em que a matriz é singular.

VIII.6 Simulação de voo com autopiloto

O subprojeto `src/flight-simulator/` é o mais integrado do repositório, e por isso a trilha o deixa por último: JSBSim [10] como modelo de dinâmica de voo, três controladores PID como autopiloto, um laço de controle a 120 Hz e registro em CSV para análise posterior. É o único subprojeto em que o C++ não é o assunto — ele é o veículo para um sistema de controle real.

VIII.6.1 A arquitetura, e o padrão que ela repete

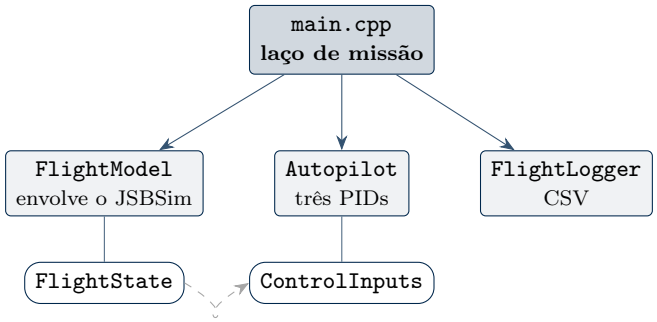


Figura 8: O laço de controle. `FlightState` e `ControlInputs` são *aggregates* simples — os tipos de vocabulário desta aplicação — e é por eles, e não por chamadas à API do JSBSim, que as três peças conversam.

O `FlightModel` é um invólucro sobre uma API C complexa — exatamente o mesmo padrão de `http::get` sobre libcurl (Seção VII.3) e de `db::Connection` sobre SOCI (Seção VII.4). A recorrência não é coincidência: é a inversão de dependência da Seção V.1 aplicada toda vez que uma biblioteca externa entra no programa.

VIII.6.2 PID: três termos, três papéis

Um controlador PID calcula o sinal de controle a partir do erro entre o valor desejado e o atual:

$$u(t) \;=\; K_p\, e(t) \;+\; K_i \int_0^t e(\tau)\, d\tau \;+\; K_d\, \frac{de(t)}{dt}$$

O termo proporcional reage ao erro presente; o integral elimina o erro acumulado — o desvio residual que o proporcional sozinho nunca fecha; e o derivativo amortece a oscilação, antecipando a tendência.

O autopiloto usa três laços independentes, cada um com um ganho próprio: a altitude comanda o profundor, a proa comanda o aileron, e a velocidade comanda o acelerador.

Armadilha — o erro de proa precisa de aritmética circular

Subtrair proas em graus dá o resultado errado na descontinuidade: indo de 350° para 10°, a subtração ingênua devolve −340°, e o autopiloto comanda uma curva de quase uma volta completa em vez de 20°. É por isso que o subprojeto tem uma função `heading_error()` separada, que normaliza a diferença ao intervalo $(-180°, +180°]$. Todo controle sobre grandeza angular tem esse problema, e ele não aparece em teste que não cruze a descontinuidade.

Armadilha — *integral windup*

Enquanto o atuador está saturado — profundor no batente, acelerador no máximo — o erro persiste e o termo integral continua acumulando. Quando o sistema finalmente alcança o alvo, o integral acumulado empurra muito além dele, e o resultado é um *overshoot* grande seguido de oscilação. As correções usuais são limitar o acumulador (*clamping*) ou parar de integrar enquanto a saída estiver saturada. É a primeira coisa a investigar quando um PID sintonizado "em bancada" se comporta mal no sistema real.

VIII.6.3 O laço de missão

```
1  constexpr double DT      = 1.0 / 120.0; // 120 Hz (padrão JSBSim)
2
3  while (sim_time <= SIM_END) {
4      if (sim_time >= 30.0) ap.set_target_altitude_ft(5000.0);
5      if (sim_time >= 90.0) ap.set_target_heading_deg(180.0);
6
7      auto state = fdm.state();              // lê o estado atual
8      auto ctrl  = ap.update(state, DT);  // autopiloto calcula controles
9      fdm.set_controls(ctrl);                // aplica controles
10     fdm.step();                            // avança a física por DT
11
12     logger.record(sim_time, state);
13     sim_time += DT;
14 }
```

Listagem 297: Ler, decidir, aplicar, avançar — e registrar.

Por que — por que o passo é fixo aqui, e variável no `sfml_bullet`

São objetivos opostos. Uma aplicação interativa precisa acompanhar o relógio de parede: se um quadro atrasa, o mundo tem de continuar coerente com o que o usuário vê, e daí o dt medido e limitado da Seção VIII.2. Uma simulação precisa de **reprodutibilidade**: o mesmo binário com a mesma entrada tem de produzir o mesmo CSV, e para isso o passo é constante e o tempo é contado em passos, não em segundos de relógio. Trocar um pelo outro produz ou uma interface que engasga, ou um experimento que não se repete.

O registro em CSV a taxa configurável fecha o ciclo: o produto do subprojeto não é o executável, é o arquivo de dados que permite verificar depois se o autopiloto de fato estabilizou — e comparar sintonias de ganho sem olhar para a tela.

VIII.7 Síntese

O capítulo tem um padrão que atravessa os sete subprojetos: **a simulação é a fonte da verdade, e tudo o mais é uma leitura dela**. O Bullet produz transformações que o SFML lê para desenhar; o JSBSim

produz um estado que o autopiloto lê para decidir e que o *logger* lê para registrar. Em nenhum caso o desenho ou o registro escrevem de volta no simulador — e é essa direção única que torna possível trocar a renderização, acrescentar um segundo consumidor de dados, ou rodar a simulação sem interface nenhuma.

A segunda recorrência é a do **invólucro sobre API estrangeira**: `PhysicsWorld` sobre Bullet, `SFMLWidget` sobre a integração Qt/SFML, `FlightModel` sobre JSBSim. Os três existem pelo mesmo motivo — transformar gestão manual de recursos em RAII, e uma API C em tipos do domínio — e os três são a mesma resposta que a Seção VII deu para libcurl e SOCI. E os três merecem a ressalva que este capítulo mediu: um invólucro RAII resolve *quem* libera o recurso, nunca *em que ordem* — essa segunda pergunta continua sendo sua, e o compilador não a fará por você (Seção VIII.2.5).

A terceira recorrência é a mais desconfortável, e é o motivo de este capítulo medir tanto: **em quatro dos seis subprojetos, a descrição escrita divergia do código**. O README do `src/collision/` prevê uma saída que o binário não produz; o do `src/sfml_bullet/` descreve uma gravidade e um lugar para as fronteiras que não são os do código; o do `src/sfml_bullet2/` documenta uma corda de cápsulas onde há um *soft body* e um cotovelo que não existe; o do `src/calculus/` publica uma reta de regressão que a aritmética contradiz. Nenhuma dessas divergências é sinal de descuido excepcional — todas são o comportamento normal de texto que nada verifica. A defesa é a que este livro pratica: **cada número tem uma legenda dizendo de qual execução ele veio**.

Tabela 100: Os números medidos deste capítulo, e o que cada um ensina.

Medida	Valor	Lição
atraso do *motion state*	$1 \times dt$	desenho e decisão leem fontes diferentes
viés do Euler semi-implícito	$\frac{1}{2} g\, dt\, t$	o integrador tem erro sistemático, não aleatório
chão físico contra chão pintado	392 px	nada obriga geometria e desenho a concordar
período do `QTimer`	16 ms (não 16,667)	divisão inteira num período é 4,2% de deriva
resíduo de $Ax = b$ por LU	$8{,}9 \cdot 10^{-16}$	quatro épsilons é o resultado *certo*
erro da aproximação posto 1	$1{,}0684 = \sigma_2$	Eckart-Young, conferido em quatro casas
soma dos autovalores	$7{,}0 = \mathrm{tr}(C)$	teste de sanidade barato para PCA

Regra — unidades e sistemas de coordenadas são parte do tipo, ou serão bug

Metros contra pixels, Y para cima contra Y para baixo, pés contra metros, graus contra radianos, proa circular contra número real, ângulo de quaternion sempre positivo contra ângulo com sinal — este capítulo tem um erro de conversão em cada seção. A defesa estrutural existe e está na Seção II.4: tipos fortes com literais de unidade, em que somar metros com segundos não compila. Onde isso for demais, ao menos concentre as conversões em duas funções nomeadas, como o `to_sfml` e o `to_bullet` deste capítulo — e nunca as escreva em linha, no meio do código de desenho.

IX Baixo nível e hardware

Este capítulo desce abaixo do C++. Não porque o C++ seja insuficiente — em 2026, o compilador vence o assembly escrito à mão em quase todo caso — mas porque *ler* o que o compilador gera é a única forma de entender por que um código é rápido e outro não, e porque existe um punhado de instruções sem equivalente na linguagem.

Três subprojetos, em ordem de escopo crescente: `src/inline_asm/` para trechos curtos embutidos no C++; `src/outline_asm/` para rotinas inteiras em arquivo separado, o que traz a ABI para o primeiro plano; e `src/asm_course/`, quinze lições numeradas que vão dos registradores ao *disassembly* de código de produção.

Os três são alvos construídos e instalados: além de `inline_asm` e `outline_asm`, o curso produz quinze executáveis próprios — de `asm_01_registers` a `asm_15_capstone` — entre os 98 binários que o repositório instala em `dist/bin`. Isso não é detalhe de empacotamento: é a condição sob a qual dois defeitos de ABI deste capítulo puderam ser encontrados, e o assunto da Seção XI.3. Todas as saídas de terminal reproduzidas adiante foram medidas nesses binários, compilados com `-O2 -msse4.2 -mavx2 -mfma`.

Regra — antes de escrever assembly, leia o que o compilador escreveu

A pergunta correta quase nunca é "como escrevo isto em assembly?", e sim "o que o compilador gerou, e por quê?". As ferramentas são `g++ -O2 -S -masm=intel`, `objdump -d -M intel` passado por `c++filt`, e o Compiler Explorer. O módulo 14 do `asm_course` é inteiramente sobre isso — e é, na prática, o mais útil dos quinze.

IX.1 Assembly inline

O subprojeto `src/inline_asm/` percorre doze demonstrações progressivas, do `nop` sem operandos até laços completos, CPUID e RDTSC. Tudo em sintaxe AT&T, que é o padrão do GCC e do Clang no Linux — e cuja principal diferença para a sintaxe Intel é a ordem dos operandos, invertida (Tabela 101).

Tabela 101: As duas sintaxes, lado a lado.

Aspecto	AT&T (GCC/Linux)	Intel (NASM/Windows)
ordem dos operandos	`src, dst`	`dst, src`
registradores	`%rax`	`rax`
imediatos	`$42`	`42`
sufixo de tamanho	`movl`, `movq`	no operando
desreferência	`(%rax)`	`[rax]`

IX.1.1 A forma de um `asm` estendido

```
asm [volatile] (
    "instruções"          // template de código assembly
    : outputs             // [opcional] operandos de saída
    : inputs              // [opcional] operandos de entrada
    : clobbers            // [opcional] registradores que o asm modifica
);
```

Listagem 298: As quatro seções, três delas opcionais.

O que faz o mecanismo funcionar é o *contrato* declarado nessas três listas: o compilador não lê o assembly, ele apenas confia no que você declarou. As *constraints* são a linguagem desse contrato (Tabela 102), e cada símbolo tem consequência direta no código que o compilador gera em volta.

Tabela 102: As *constraints* usadas no subprojeto.

Constraint	Significado
`"r"`	qualquer registrador de propósito geral
`"a"`, `"b"`, `"c"`, `"d"`	especificamente `rax`, `rbx`, `rcx`, `rdx`
`"m"`	operando em memória (o compilador gera o endereçamento)
`"i"`	imediato inteiro
`"0"`	o mesmo registrador que o operando 0
`"="`	*write-only* — é saída
`"+"`	*read-write* — entrada e saída

Armadilha — o compilador acredita no que você declarou, não no que você escreveu

Se o bloco modifica `%eax` e `%eax` não está nos *outputs* nem nos *clobbers*, o compilador continua supondo que o valor que ele havia posto lá permanece — e usa esse valor depois. O bug resultante aparece só com otimização ligada, muda com a versão do compilador, e não tem nada de errado no assembly em si. As duas declarações mais esquecidas são `"cc"` (o bloco altera as *flags*) e `"memory"` (o bloco pode ler ou escrever memória arbitrária, e serve como barreira).

```
asm(
    "movl %2, %%eax\n\t"
    "movl %3, %%ebx\n\t"
    "xchgl %%eax, %%ebx\n\t"
    "movl %%eax, %0\n\t"
    "movl %%ebx, %1"
    : "=r"(troca_a), "=r"(troca_b)
    : "r"(x), "r"(y)
    : "%eax", "%ebx"  // ← clobbers: esses regs foram usados internamente
);
```

Listagem 299: Os registradores usados internamente declarados como *clobbers*.

IX.1.2 `volatile`: quando o compilador apagaria o bloco

Um bloco `asm` sem *outputs* não tem, do ponto de vista do compilador, efeito nenhum — e portanto pode ser removido, movido ou duplicado. `volatile` é o que impede isso, e é o que torna possível a barreira de memória mais usada em código de baixo nível:

```
asm volatile ("" ::: "memory");
```

Listagem 300: A barreira que não gera instrução alguma.

Ela não emite nenhum *opcode*: o efeito inteiro está na declaração `"memory"`, que impede o compilador de reordenar leituras e escritas através daquele ponto. Note que isso é uma barreira de *compilação*, não de *hardware* — ela não substitui `std::atomic` nem as ordens de memória da Seção VI.1, que também controlam a reordenação feita pelo processador.

IX.1.3 Instruções sem equivalente em C++

Duas justificativas do subprojeto são incontestáveis, porque a linguagem não oferece alternativa.

CPUID identifica o processador e as extensões disponíveis, e é a base de qualquer despacho por capacidade em tempo de execução — escolher entre uma rotina AVX2 e uma SSE conforme o que a máquina tem. O padrão é distribuir os quatro registradores de saída para quatro variáveis por meio de *constraints* específicas:

```
asm volatile (
    "cpuid"
    : "=a"(eax), "=b"(ebx), "=c"(ecx), "=d"(edx)
    : "a"(0)  // leaf 0 = vendor info
);
```

Listagem 301: Cada *constraint* nomeia um registrador exato.

RDTSC lê o contador de ciclos do processador, e é a medição de menor granularidade disponível.

Armadilha — RDTSC não mede tempo, e nem sempre mede ciclos

Em processadores modernos, o TSC é *invariante* — avança a uma frequência fixa, independente do estado de energia — de modo que ele mede tempo, não ciclos executados. Pior: a execução fora de ordem permite que instruções ao redor do `rdtsc` sejam reordenadas, o que borra a medição de trechos curtos (a correção usual é `rdtscp` ou uma serialização com `cpuid`). E o valor não é comparável entre núcleos sem garantia de sincronização. Para medir desempenho, `std::chrono::steady_clock` está certo em quase todos os casos; RDTSC é para quando a granularidade dele não basta, e com essas ressalvas em mente.

IX.1.4 Legibilidade: operandos nomeados

O último demo do subprojeto é o mais imediatamente aplicável: em vez de `%0`, `%1`, `%2`, dar nomes aos operandos.

```
asm (
    "movl %[w], %[out]\n\t"
    "imull %[h], %[out]"
    : [out] "=r"(area)
    : [w]   "r"(largura),
      [h]   "r"(altura)
);
```

Listagem 302: A mesma multiplicação, legível.

Com numeração posicional, acrescentar um *input* no meio renumera todos os seguintes e quebra o bloco em silêncio. Com nomes, não.

```
== 8. CPUID -- lendo informacoes da CPU ==
  CPUID leaf 0:
    eax (max leaf) = 13
    vendor string  = "AuthenticAMD"

== 9. RDTSC -- contando ciclos de CPU ==
  TSC antes : 61820684363285
  TSC depois: 61820684365690
  Ciclos para loop de 1000 somas: 2405
```

Listagem 303: Saída medida de `dist/bin/inline_asm` (recortada): CPUID e RDTSC lendo a máquina em que este relatório foi compilado.

Os dois números dizem coisas diferentes e igualmente úteis. A *vendor string* é a única forma portátil de descobrir em que máquina o código está rodando, e é o alicerce de qualquer despacho por capacidade. Já os 2405 ciclos para mil somas — pouco mais de dois por soma — só fazem sentido com as ressalvas da armadilha acima: são ciclos de um TSC invariante, num laço que o compilador não pôde eliminar porque o bloco é `volatile`.

Regra — assembly inline é a última opção, não a primeira

Antes dele, na ordem: uma função *intrinsic* (`_mm_add_ps`, `__builtin_popcount`), que o compilador entende e otimiza em volta; um atributo ou *pragma* que diga ao compilador o que fazer; e uma reformulação do C++ que exponha a propriedade que faltava. Assembly inline bloqueia otimizações — o compilador não pode enxergar dentro do bloco — e amarra o código a um compilador, a uma arquitetura e a uma sintaxe.

IX.2 Assembly em arquivo separado

O subprojeto `src/outline_asm/` move o assembly para um arquivo `.s`, montado pelo GAS e ligado ao objeto do C++. A troca é clara (Tabela 103), e o divisor de águas é o tamanho da rotina.

Tabela 103: Assembly embutido contra assembly em arquivo próprio.

Aspecto	Inline	Arquivo `.s`
tamanho ideal	1–5 instruções	rotinas longas
ferramenta	*extended asm* do GCC	GAS, NASM, qualquer *assembler*
otimizador	pode interferir	o código é exatamente o escrito
portabilidade	só GCC/Clang	qualquer *toolchain*
manutenção	difícil de ler	separado e comentável

IX.2.1 A fronteira: `extern "C"` e a ABI

Do lado do C++, a declaração precisa suprimir a decoração de nomes [5] — o *assembler* conhece `asm_add`, não `_Z7asm_addii`:

```
1  extern "C" {
2
3  // 3. Valor absoluto sem branch
4  int      asm_valor_absoluto(int n);
5
6  // 5. Máximo de dois inteiros (branchless via CMOV)
7  int      asm_maximo(int a, int b);
8
9  // 8. Memcpy de N blocos de 64 bits com REP MOVSQ
10 void     asm_memcpy_64(void* dst, const void* src, int64_t n);
11
12 } // extern "C"
```

Listagem 304: A interface, em `src/outline_asm/asm_funcs.hpp` (recortada).

Do lado do assembly, a implementação precisa concordar com a ABI [11] sobre onde os argumentos chegam e onde o retorno é deixado (Tabela 104). Não há verificação: uma discordância compila, liga e produz lixo.

Tabela 104: O essencial da ABI System V AMD64 (Linux).

Papel	Registradores
argumentos inteiros e ponteiros	`rdi`, `rsi`, `rdx`, `rcx`, `r8`, `r9`
argumentos em ponto flutuante	`xmm0`–`xmm7`
retorno inteiro/ponteiro	`rax` (e `rdx` para 128 bits)
retorno em ponto flutuante	`xmm0`
preservados pelo chamado	`rbx`, `rbp`, `r12`–`r15`
livres para o chamado	`rax`, `rcx`, `rdx`, `rsi`, `rdi`, `r8`–`r11`, `xmm*`

Armadilha — usar um registrador *callee-saved* sem salvá-lo

Se a rotina em assembly escrever em `rbx`, `rbp` ou `r12`–`r15` sem empilhar e restaurar o valor original, ela corrompe estado que o chamador presumia intacto. O sintoma não aparece na função em si: aparece depois, no C++, como uma variável que mudou de valor sem que nada a tenha atribuído. É a razão de as rotinas do subprojeto usarem apenas registradores livres.

IX.2.2 Duas instruções que valem a leitura

O arquivo `.s` do subprojeto tem dez rotinas, e três delas ensinam mais do que o tamanho sugere.

`cmov` — comparação sem desvio.

```
    .globl asm_maximo
    .type  asm_maximo, @function
asm_maximo:
    movl   %edi, %eax      # eax = a  (resultado inicial)
    cmpl   %esi, %edi      # compara a com b (afeta flags, não escreve)
    cmovll %esi, %eax      # se a < b (less): eax = b
                           # cmov não cria branch → pipeline não quebra
    ret
    .size asm_maximo, .-asm_maximo
```

Listagem 305: `asm_maximo` sem *branch*, em `src/outline_asm/asm_funcs.s`.

Um desvio condicional sobre dado imprevisível é predito errado em torno de metade das vezes, e cada erro custa o esvaziamento do *pipeline* — dezenas de ciclos. O `cmov` executa sempre, sem desviar, e troca essa loteria por um custo fixo. A lição é geral: para *dados imprevisíveis*, sem desvio ganha; para dados com padrão claro, o preditor acerta e o desvio é mais barato.

`rep movsq` — a instrução que o hardware reescreveu. A rotina de cópia do subprojeto cabe em três linhas, e duas delas são a convenção:

```
    .globl asm_memcpy_64
    .type  asm_memcpy_64, @function
asm_memcpy_64:
    movq   %rdx, %rcx      # rcx = n  (REP usa rcx como contador)
    rep    movsq           # copia rcx qwords: [rsi] → [rdi],  rsi++, rdi++
    ret
    .size asm_memcpy_64, .-asm_memcpy_64
```

Listagem 306: `asm_memcpy_64`, em `src/outline_asm/asm_funcs.s`.

O `rep` usa `rcx` como contador e `rsi`/`rdi` como origem e destino — que são, por sorte da ABI, exatamente onde os dois ponteiros já chegaram. Semanticamente é um laço; em processadores modernos com ERMSB

(*Enhanced REP MOVSB/STOSB*), o hardware reconhece o padrão e move blocos de 16, 32 ou 64 bytes internamente, tornando a instrução competitiva com — ou superior a — um laço manual com SSE ou AVX. É o exemplo mais claro de por que "contar instruções" deixou de ser uma métrica de desempenho.

Máscara de sinal — valor absoluto sem desvio. A terceira rotina do arquivo faz `abs()` com um deslocamento aritmético, um XOR e uma subtração: `n » 31` produz `0` para não negativos e `0xFFFFFFFF` para negativos, e `(n ^ mask) - mask` é a identidade que devolve `n` num caso e `-n` no outro. Não é curiosidade: é o mesmo truque que o compilador emite para `std::abs` em `-O2`, e vê-lo escrito à mão explica por que a versão com `if` gera o mesmo código.

IX.2.3 O *build* não precisa de nada especial

O Meson reconhece `.s` pela extensão e chama o *assembler* correspondente. Basta listar o arquivo entre as fontes do alvo, junto com os `.cpp` — o que é, em si, uma pequena demonstração de por que o Seção XI.2 vale a leitura.

IX.2.4 O que a execução revela — e que a compilação não revelaria

O `main.cpp` do subprojeto não apenas chama as dez rotinas: compara cada resultado com uma referência em C++ ou da libc. Nove passam. Uma não.

```
-- 5. asm_maximo (branchless CMOV) --
  max(3, 7) = 7 ... max(-1, 1) = 1          [ok] todos corretos
-- 6. asm_popcount (instrucao POPCNT) --
  popcount(0xFFFFFFFF) = 32                 [ok] todos corretos
-- 7. asm_strlen (REPNE SCASB) --
  strlen("")      = -1   [FALHOU] igual a strlen() da libc
  strlen("hello") =  4   [FALHOU] igual a strlen() da libc
-- 10. asm_eh_primo (preserva %rbx) --
  7919 -> primo                             [ok] igual a referencia C++
```

Listagem 307: Saída medida de `dist/bin/outline_asm`, recortada.

O defeito é um *off-by-one* no idioma de `strlen` com `repne scasb`, e reaparece — copiado literalmente — em dois módulos do curso. Ele é dissecado na Seção IX.3.8 e seguintes, junto com a razão de ter sobrevivido tanto tempo: durante boa parte da história deste repositório, esses alvos não eram construídos.

IX.3 O curso de assembly

O diretório `src/asm_course/` é diferente de todos os outros do repositório: não é um subprojeto com um `README` e um executável, e sim uma sequência de **quinze lições numeradas**, cada uma um par `.cpp` mais `.s` (a partir da sexta), para serem estudadas na ordem dos arquivos. O material didático está nos comentários de cabeçalho de cada arquivo — e cada lição é um executável independente, instalado como `asm_NN_nome`, de modo que o curso inteiro se estuda *rodando*, e não lendo.

O restante desta seção percorre os quinze módulos na ordem, ensinando cada conceito do zero e citando as saídas medidas de cada binário. Quem já conhece assembly pode ir direto aos módulos 7, 10, 13 e 15: são os que contêm os defeitos reais encontrados no repositório, e é neles que este capítulo tem algo a dizer que um manual de arquitetura não diria.

Tabela 105: As quinze lições, e o assunto de cada uma.

#	Módulo	Assunto
1	`registers`	os 16 registradores e o mapa de *aliases*
2	`memory`	modos de endereçamento
3	`arithmetic`	aritmética e o registrador RFLAGS
4	`bitwise`	operações bit a bit
5	`control_flow`	saltos condicionais, `cmov`, `setcc`
6	`stack_frames`	`push`, `pop`, `call`, `ret`
7	`calling_conv`	a ABI System V completa
8	`simd_sse`	registradores XMM, 128 bits
9	`simd_avx`	registradores YMM, 256 bits
10	`string_ops`	`rep movs`, `stos`, `scas`, `cmps`
11	`syscalls`	a fronteira com o *kernel*, sem libc
12	`fp_x87`	a FPU legada e a pilha de 80 bits
13	`optimization`	*unrolling*, *prefetch*, previsão de desvio
14	`disassembly`	ler o que o compilador gerou
15	`capstone`	`memcpy`, `strlen`, `memset` de produção

IX.3.1 Três ideias que atravessam o curso

O registrador é uma hierarquia de memória em miniatura. A primeira lição enquadra tudo o que vem depois: um registrador tem latência zero, o cache L1 custa cerca de quatro ciclos, e a RAM cerca de duzentos. São *duas ordens de grandeza* entre estar dentro da CPU e estar fora dela — e é essa razão que explica por que a `std::list` perde para o `std::vector` na Seção III.1, por que o *pool allocator* da Seção III.5 rende, e por que o ECS da Seção V.3 organiza componentes por tipo.

Um *syscall* é uma fronteira, não uma chamada. A lição 11 mostra `write` e `exit` feitos direto pela instrução `syscall`, sem libc: número em `rax`, argumentos em `rdi`, `rsi`, `rdx`, `r10`, `r8`, `r9`, retorno em `rax` com valor negativo indicando `errno`. Repare que `r10` ocupa o lugar de `rcx`: a convenção de *syscall* não é a mesma da ABI de funções, porque `rcx` é destruído pela própria instrução.

SIMD é sobre *layout*, não sobre instruções. As lições 8 e 9 apresentam XMM (128 bits) e YMM (256 bits), com um registrador comportando quatro `float` ou dois `double`. O que o curso deixa claro é que a dificuldade real não são os *opcodes* — é organizar os dados para que quatro valores adjacentes façam sentido juntos. É a diferença entre *array of structs* e *struct of arrays*, e é uma decisão de tipo, tomada muito antes de qualquer instrução vetorial.

Armadilha — medir com o relógio errado, e medir código que não existe

A lição 13 depende de medição, e medir assembly tem duas armadilhas próprias. A primeira: o compilador remove código cujo resultado não é usado — um *benchmark* que descarta o resultado mede um laço vazio. A segunda: a frequência do processador varia (*turbo*, térmica, escalonamento), de modo que a mesma rotina medida duas vezes dá números diferentes. Some a isso o aquecimento de cache, e fica claro por que *microbenchmark* é uma disciplina própria — e por que a resposta certa quase sempre é medir o programa inteiro com `perf`, não a função isolada.

IX.3.2 Módulo 1 — o que é, afinal, um registrador

Por que — por que a CPU precisa de "variáveis" próprias

O processador não sabe somar coisas que estão na memória: ele sabe somar coisas que estão *dentro dele*. Um **registrador** é exatamente isso — uma célula de armazenamento fixa, dentro do núcleo, com nome próprio e tamanho fixo. Não tem endereço, não pode crescer, não se aloca: existem dezesseis de propósito geral no x86-64, e é com esses dezesseis que todo programa do mundo é executado.
O motivo de eles existirem cabe numa lista de latências, que o módulo 1 usa como abertura do curso: registrador, 0 ciclos; cache L1, ≈ 4; L2, ≈ 12; L3, ≈ 40; RAM, ≈ 200. São *duas ordens de grandeza* entre estar dentro da CPU e estar fora dela, e é essa razão que explica por que a `std::list` perde para o `std::vector` na Seção III.1, por que o *pool allocator* da Seção III.5 rende, e por que o ECS da Seção V.3 agrupa componentes por tipo.

Os dezesseis são `rax`, `rbx`, `rcx`, `rdx`, `rsi`, `rdi`, `rbp`, `rsp` e `r8`–`r15`. Cada um tem *nomes menores*, que não são outros registradores e sim *janelas* sobre o mesmo silício: `rax` de 64 bits contém `eax` de 32, que contém `ax` de 16, que se divide em `ah` e `al` de 8. Os registradores novos seguem o mesmo padrão com sufixos: `r8`, `r8d`, `r8w`, `r8b`.

Armadilha — escrever em `eax` zera a metade alta de `rax`

Esta é a regra mais contraintuitiva do x86-64, e o módulo 1 a mede. Escrever num registrador de 32 bits **zera** os 32 bits superiores do de 64; escrever num de 16 ou de 8 bits **não** zera nada. A decisão é do projeto do processador — zerar a metade alta quebra a dependência com o valor anterior e libera o renomeador de registradores — mas significa que `movl` e `movw` têm efeitos colaterais opostos.

```
1  -- 1.1 Aliases -- mesmo registrador, visoes diferentes --
2    rax = 0xDEADBEEFCAFEBABE
3    eax = 0xCAFEBABE  (32 bits baixos de rax)
4    ax  = 0xBABE      | al = 0xBE | ah = 0xBA
5
6  -- 1.2 Armadilha: escrever em eax ZERA os 32 bits altos --
7    antes:  rax = 0xDEADBEEF00000000
8    movl $0x12345678, %eax
9    depois: rax = 0x0000000012345678
10   apos movw $0x1234, %ax:  rax = 0xDEADBEEFCAFE1234
```

Listagem 308: Saída medida de `dist/bin/asm_01_registers` (cores removidas).

Com `movl`, `0xDEADBEEF` sumiu; com `movw`, sobreviveu. Nada no código C++ mudou entre os dois casos — mudou o sufixo de tamanho da instrução.

Três registradores especiais aparecem o tempo todo daqui em diante: `rip` (o endereço da instrução atual, que o hardware avança sozinho e que nenhum `mov` escreve), `rsp` (o topo da pilha, módulo 6) e `rflags` (o estado da última operação, módulo 3). E duas instruções existem porque um *byte* precisa virar um `int64`: `movzx` preenche com zeros (é o `unsigned char` do C++) e `movsx` replica o bit de sinal (é o `signed char`). O módulo mede: `0xFF` vira `255` sob a primeira e `-1` sob a segunda.

IX.3.3 Módulo 2 — endereçar memória

Por que — por que um modo de endereçamento tão elaborado

`arr[i]` é uma multiplicação, uma soma e uma leitura. Se cada etapa fosse uma instrução, todo acesso a vetor custaria três *slots* de *pipeline*. O x86 põe a conta inteira *dentro* do operando de memória: há uma unidade dedicada de cálculo de endereço, e ela computa base, índice, escala e deslocamento de graça, em paralelo com o resto.

A forma geral, em AT&T, é `disp(base, índice, escala)`, com endereço efetivo `base + índice×escala + disp`. A escala só pode ser 1, 2, 4 ou 8 — exatamente os tamanhos de `char`, `short`, `int` e `int64_t`, o que não é coincidência.

Tabela 106: Modos de endereçamento e a construção C++ correspondente (de `src/asm_course/02_memory.cpp`).

AT&T	Intel	Em C++
`$42`	`42`	literal
`(%rax)`	`[rax]`	`*p`
`8(%rax)`	`[rax+8]`	`p->campo`
`(%rax,%rbx,4)`	`[rax+rbx*4]`	`arr[i]`
`-8(%rbp)`	`[rbp-8]`	variável local
`x(%rip)`	`[rip+x]`	global ou `static`

Duas linhas merecem comentário. `-8(%rbp)` é onde moram as variáveis locais: uma variável local não é uma entidade do hardware, é um deslocamento negativo a partir da base do quadro de pilha. E `x(%rip)` é o endereçamento relativo ao ponteiro de instrução — o mecanismo que torna o código independente de posição e que toda biblioteca compartilhada usa para alcançar suas globais sem saber onde foi carregada.

`lea` — **calcular sem ler.** A instrução `lea` (*load effective address*) executa a conta de endereçamento e entrega o *resultado da conta*, sem tocar na memória. Isso a torna o canivete aritmético do x86 — e, ao contrário de `add` e `imul`, ela **não altera as *flags***, o que permite intercalá-la entre um `cmp` e o salto que depende dele.

```
1  -- 2.4 LEA -- calcula endereco SEM acessar memoria --
2    leaq (,x,4)   -> x*4 = 28  (x=7)
3    leaq (x,x,4)  -> x*5 = 35  (x=7)
4    leaq (x,x,8)  -> x*9 = 63
5    leaq 10(x,y)  -> x+y+10 = 20  (x=7, y=3)
6    LEA nao altera RFLAGS -- vantagem sobre ADD/IMUL!
```

Listagem 309: Saída medida de `dist/bin/asm_02_memory`.

Ver `leaq (x,x,4)` produzir `x*5` explica de uma vez por que o compilador quase nunca emite `imul` para constantes pequenas: ele decompõe a constante em somas de `lea` e deslocamentos.

IX.3.4 Módulo 3 — aritmética e o registrador RFLAGS

Por que — como o processador se lembra do resultado de uma comparação

Não existe `if` no hardware. O que existe é um registrador de estado, **RFLAGS**, cujos bits são atualizados como efeito colateral de quase toda operação aritmética ou lógica, e saltos que *leem* esses bits. Um `if (a < b)` do C++ vira, invariavelmente, duas instruções: uma que escreve nas *flags* (`cmp`) e uma que as consulta (`jl`).

Tabela 107: Os bits de RFLAGS que importam, do cabeçalho de `src/asm_course/03_arithmetic.cpp`.

Bit	Sigla	Nome	Significa
0	CF	*carry*	estouro **sem** sinal
6	ZF	*zero*	resultado igual a zero
7	SF	*sign*	bit mais alto do resultado é 1
10	DF	*direction*	direção das operações de *string*
11	OF	*overflow*	estouro **com** sinal

A distinção que o módulo existe para ensinar é a de **CF contra OF**. Os mesmos bits somados dão o mesmo resultado — o processador não sabe se você considera o operando com ou sem sinal. Ele calcula as duas condições de estouro e guarda *as duas*: CF é o estouro da leitura sem sinal, OF o da leitura com sinal. Quem escolhe qual consultar é a instrução de salto, e é aí que a assinatura do tipo em C++ vira uma decisão de *opcode*.

```
-- 3.1 ADD -- flags em diferentes cenarios --
      -1 +          1 =          0 | resultado zero
  CF=1 PF=1 ZF=1 SF=0 OF=0
2147483647 +          1 = -2147483648 | overflow com sinal
  CF=0 PF=1 ZF=0 SF=1 OF=1
```

Listagem 310: Saída medida de `dist/bin/asm_03_arithmetic`: cada operação é seguida pelas *flags* capturadas dentro do mesmo bloco `asm`.

As duas linhas são a demonstração inteira. Somar -1 com 1 levanta CF (houve vai-um na leitura sem sinal) mas não OF. Somar `INT32_MAX` com 1 levanta OF mas não CF. O mesmo circuito, dois veredictos — e note que a segunda dessas somas é UB (comportamento indefinido) em C++ e aqui aparece como o que ela é no hardware: previsível, e completamente inútil como garantia, porque o compilador tem o direito de supor que ela nunca acontece.

Armadilha — ler RFLAGS de fora do bloco não mede nada

O módulo traz uma função `read_flags()` feita com `pushfq; popq` — a única maneira de ler RFLAGS, já que não existe `mov` a partir dele. E o próprio comentário do arquivo avisa que ela é quase inútil: entre a instrução que se quer observar e o `pushfq`, o compilador pode inserir qualquer coisa, e *quase toda* instrução mexe nas *flags*. Por isso todos os demos capturam as *flags* dentro do mesmo bloco `asm` da operação — e a saída ainda imprime, antes, o estado "ambiente", justamente para o leitor ver que ele não corresponde a operação nenhuma.

Três notas fecham o módulo. `sub` e `cmp` são a mesma subtração com as mesmas *flags*; `cmp` apenas descarta o resultado. `mul` e `imul` produzem um resultado do *dobro* da largura, em `rdx:rax` — é como se implementa `__int128` e como uma multiplicação de 64 bits detecta o próprio estouro. E `inc`/`dec`, ao contrário de `add $1`/`sub $1`, *preservam* CF, para não destruir o vai-um de uma aritmética de precisão múltipla em andamento (`adc` e `sbb`, na seção 3.6, são o outro lado dessa moeda).

IX.3.5 Módulo 4 — operações bit a bit

O módulo 4 percorre `and`, `or`, `xor`, `not`, os três deslocamentos e as rotações, além do quarteto de busca de bits (`bsf`, `bsr`, `lzcnt`, `tzcnt`). A distinção que mais rende está na Tabela 108.

Tabela 108: As três direitas do x86, com os valores medidos em `dist/bin/asm_04_bitwise`.

Instrução	Preenche com	Equivale a	Medido
`shr`	zeros	divisão sem sinal	`-8 » 3 = 536870911`
`sar`	cópia do sinal	divisão com sinal	`-8 » 3 = -1`
`ror`	o bit que saiu	rotação (nada se perde)	`0x80000001 -> 0xC0000000`

A primeira linha é a razão pela qual `unsigned` e `signed` geram *instruções diferentes* para o mesmo operador `»` do C++: o tipo escolhe entre `shr` e `sar`. A terceira é a razão pela qual rotação não existia como operador da linguagem até o C++20.

Regra — o `<bit>` do C++20 cobre quase tudo deste módulo

`std::popcount`, `std::rotl`, `std::rotr`, `std::countl_zero`, `std::countr_zero`, `std::bit_width` e `std::has_single_bit` compilam exatamente para `popcnt`, `rol`, `ror`, `lzcnt` e `tzcnt` — e são `constexpr`, portáteis e sem UB (comportamento indefinido) quando o deslocamento é grande demais. Estude o módulo 4 para entender o que a instrução faz; escreva o `<bit>` no código de produção. O subprojeto `src/bits/` cobre esse lado da fronteira.

IX.3.6 Módulo 5 — controle de fluxo

Por que — não existe `if`, existe `cmp` seguido de salto

Todo controle de fluxo do x86 se reduz a uma instrução que escreve nas *flags* e outra que salta (ou não) conforme elas. O que muda de um `if` para um `while`, um `switch` ou um `?:` é apenas *qual* salto e *para onde*.

A tabela de saltos tem uma coluna a mais do que o esperado: o x86 oferece *dois conjuntos completos* de comparações, um com sinal (`jg`, `jge`, `jl`, `jle`) e outro sem (`ja`, `jae`, `jb`, `jbe`); só a igualdade (`je`/`jne`) é comum aos dois, porque bits iguais são bits iguais em qualquer interpretação.

```
1  -- 5.2 Jumps condicionais: JL vs JB (signed vs unsigned) --
2    -1 (0xFFFFFFFF) vs 5:
3      JL  (signed   <): SIM -> -1 < 5 = verdadeiro
4      JB  (unsigned<): NAO -> 4294967295 < 5 = falso
5    Mesmos bits, interpretacoes OPOSTAS!
```

Listagem 311: Saída medida de `dist/bin/asm_05_control_flow`: os mesmos 32 bits, dois veredictos opostos.

Essa é, medida no metal, a origem de toda a classe de bugs de comparação mista entre `int` e `size_t` — inclusive a que o `src/http-client/` carregava, onde `40 - title.size()` dá a volta por ser aritmética sem sinal. O compilador escolheu `jb` porque o tipo mandou; o autor esperava `jl`.

Duas alternativas ao salto fecham o módulo: `setcc`, que escreve `0` ou `1` num *byte* conforme a condição — é assim que o compilador materializa um `bool` — e `cmov`, já visto na Seção IX.2.

Armadilha — a instrução `loop` é mais lenta que escrever o laço à mão

O x86 tem uma instrução `loop`, que decrementa `rcx` e salta se ele não zerou: exatamente o que um laço faz, numa instrução só. E ela é *mais lenta* que o par `dec` + `jnz` em qualquer processador moderno, por ser microcodificada. É o exemplo canônico de que "menos instruções" deixou de significar "mais rápido" ainda nos anos noventa.

IX.3.7 Módulo 6 — a pilha, e onde moram as variáveis locais

Por que — por que uma função sabe para onde voltar

Chamar uma função exige guardar duas coisas em algum lugar: o endereço de retorno e as variáveis locais de quem chamou. O "algum lugar" é a **pilha** — uma região de memória que cresce para baixo, e cujo topo `rsp` sempre aponta. Não há nada de mágico nela: é memória comum, e o único motivo de funcionar é que todos concordam em usá-la da mesma forma.

```
1  PUSH val   -> RSP -= 8; [RSP] = val
2  POP  dst   -> dst = [RSP]; RSP += 8
3  CALL alvo  -> PUSH(endereco da proxima instrucao); JMP alvo
4  RET        -> POP RIP
5
6  Enderecos altos
7  +---------------------+  <- RSP antes do CALL
8  |  endereco de ret.   |  <- o CALL empurrou isto
9  +---------------------+  <- RSP ao entrar na funcao
10 |  RBP anterior       |  <- pushq %rbp   (prologo)
11 +---------------------+  <- RBP = RSP    (base do quadro)
12 |  variavel local 1   |     -8(%rbp)
13 |  variavel local 2   |     -16(%rbp)
14 +---------------------+  <- RSP atual
15 Enderecos baixos
```

Listagem 312: As quatro instruções da pilha e o quadro que elas montam (resumo de (de `src/asm_course/06_stack_frames.cpp`).

O par `pushq %rbp` / `movq %rsp, %rbp` é o **prólogo**; `leave` (que é `mov rsp,rbp` seguido de `pop rbp`) e `ret` formam o **epílogo**. Com o quadro montado, `8(%rbp)` é sempre o endereço de retorno e `-8(%rbp)` a primeira variável local — e o módulo comprova isso lendo os dois em tempo de execução.

```
-- 6.1 PUSH / POP --
  RSP antes:        0x00007FFF646AD0A0
  RSP apos 2 PUSH: 0x00007FFF646AD090  (decrementou -16 bytes)
  RSP apos 2 POP:  0x00007FFF646AD0A0  (restaurado)
  Primeiro pop: 0xBBBBBBBB (era val2)  -> LIFO

-- 6.2 Prologo/Epilogo -- frame convencional --
     RSP = 0x00007FFF646AD0C0  (topo da stack)
     RBP = 0x00007FFF646AD0C0  (base do frame)
     [RBP+8] = endereco de retorno = 0x0000561EC97A6815
```

Listagem 313: Saída medida de `dist/bin/asm_06_stack_frames`.

Dois `push` moveram `rsp` em exatamente 16 *bytes*; dois `pop` o devolveram ao valor original. É trivial, e é a única coisa que separa um programa que funciona de um que não volta: se as contas de `rsp` não fecharem, o `ret` salta para o que quer que esteja no topo da pilha.

A função `asm_fibonacci` do módulo é o exemplo completo: é recursiva, precisa de `n` vivo *depois* da primeira chamada recursiva e do resultado dela vivo *durante* a segunda — e por isso usa `rbx` e `r12`, que são *callee-saved*, empilhando-os no prólogo e restaurando-os na ordem inversa no epílogo.

Armadilha — a *red zone* são 128 bytes que só uma função-folha tem

A ABI reserva 128 *bytes abaixo* de `rsp` que uma função pode usar sem mover o ponteiro de pilha — é o que permite a uma função sem chamadas dispensar o prólogo inteiro. A pegadinha está em "sem chamadas": se a função chamar qualquer outra, o `call` escreve exatamente ali. E código de *kernel*, onde interrupções não respeitam a convenção, é compilado com `-mno-red-zone` por essa razão.

IX.3.8 Módulo 7 — a ABI, e o bug que explica por que ela existe

Por que — por que duas linguagens conseguem se chamar

Nada no formato binário obriga uma função a receber o primeiro argumento em `rdi`. Essa é uma *convenção*: a ABI System V AMD64 [11], que todo compilador de Linux x86-64 implementa. Ela é o motivo de o C++ conseguir chamar Rust, Fortran, uma rotina `.s` escrita à mão e a libc do sistema sem que nenhum deles conheça o outro. E é uma convenção sem fiscal: quando um dos lados a desobedece, ninguém avisa — o programa compila, liga, executa e produz lixo, ou estoura em outro lugar.

A tabela de registradores já apareceu na Tabela 104. O módulo 7 acrescenta três regras que ela não cabe e que respondem por quase todo bug de fronteira:

1. Do sétimo argumento em diante, os valores vão pela **pilha**, na ordem inversa — e a função os lê em `16(%rbp)`, `24(%rbp)`, e assim por diante, *depois* do prólogo.
2. Numa função **variádica**, o registrador `al` deve conter *quantos* registradores XMM carregam argumentos em ponto flutuante. Chamar `printf` de assembly sem zerar `al` é uma das formas mais rápidas de derrubar um programa.
3. No instante em que a instrução `call` é **executada**, `rsp` tem de ser múltiplo de 16.

A terceira é a que interessa aqui, e a aritmética dela é curta o bastante para caber num parágrafo. Como o `call` empurra 8 bytes de endereço de retorno, ao *entrar* numa função `rsp mod 16` vale 8. O `pushq %rbp` do prólogo empurra mais 8 e devolve o valor a 0. A partir daí, a função já está pronta para chamar outra: qualquer ajuste extra tem de ser múltiplo de 16.

```
entrada de asm_apply ..... RSP mod 16 = 8
pushq %rbp    ..... RSP mod 16 = 0   <- JA ALINHADO
subq $8, %rsp ..... RSP mod 16 = 8   <- e aqui QUEBRAMOS
call *%rax    ..... o destino recebe RSP mod 16 = 0  ERRADO
```

Listagem 314: A conta, linha a linha — resumo de `src/asm_course/07_calling_asm.s`.

O primeiro bug de ABI. A versão anterior de `asm_apply` — a rotina que chama um ponteiro de função vindo do C++ — trazia um `subq $8, %rsp` antes do `call`, com o comentário "após push rbp: rsp está *misaligned* por 8". A conta estava invertida: o `push` já havia alinhado, e o `sub` desalinhou. A rotina funcionava — até o destino executar uma instrução SSE alinhada.

```
#0  fmt::v10::detail::vformat_to<char>
#4  cpp_callback  (07_calling_conv.cpp:52)
#5  asm_apply     (07_calling_asm.s:124)   <- o culpado
```

Listagem 315: A pilha do *crash*, do relato em `src/asm_course/07_calling_asm.s`.

Armadilha — o bug de alinhamento estoura em quem *usa* SSE, nunca em quem o quebrou

Esta é a assinatura do defeito, e a razão de ele ser tão caro de encontrar. Quebrar o alinhamento de 16 bytes não causa falha nenhuma: `sub`, `mov` e `call` funcionam com `rsp` em qualquer valor. A falha aparece quando alguém, quatro quadros acima, emite um `movaps` ou `movdqa` — os *loads* SSE **alinhados**, que o compilador usa à vontade em `-O2` para copiar 16 bytes de uma vez — sobre um endereço que herdou o deslocamento. O resultado é um `SIGSEGV` dentro de código de terceiros, perfeitamente correto, com a pilha apontando para a biblioteca de formatação. O culpado está no quadro de baixo, e não fez nada errado no instante em que estourou.

A correção foi remover uma linha. O que ela ensina não é "cuidado com o `sub`": é que a ABI não é burocracia, e sim um conjunto de invariantes das quais o código *gerado* depende sem nunca verificar.

```
-- 7.5 Ponteiro de funcao como argumento --
  asm_apply(cpp_callback, 7):
    [cpp_callback chamado com 7]
  resultado = 49 (esperado: 49 = 7 ao quadrado)

-- 7.7 Alinhamento de 16 bytes da stack no CALL --
  RSP atual: 0x00007FFD80B04AA0
  RSP mod 16 = 0
  Alinhado a 16 bytes? SIM (apos push rbp)
```

Listagem 316: Saída medida de `dist/bin/asm_07_calling_conv` depois da correção: o *callback* em C++ agora retorna.

IX.3.9 Módulos 8 e 9 — SIMD: XMM, YMM e o custo de misturá-los

Por que — por que somar quatro números custa o mesmo que somar um

SIMD é *Single Instruction, Multiple Data*: um registrador largo dividido em faixas independentes, e uma instrução que opera em todas ao mesmo tempo. Um XMM tem 128 bits e cabe quatro `float`; um YMM tem 256 e cabe oito. `addps` soma quatro pares de `float` no mesmo tempo em que `addss` soma um. O ganho não vem de mais silício por operação — vem de não pagar decodificação, despacho e controle de laço quatro vezes.

Tabela 109: O que cabe num registrador vetorial, e o sufixo que o diz (de `08_simd_sse.cpp` e `09_simd_avx.cpp`).

Sufixo	Significa	XMM (128 bits)	YMM (256 bits)
`ps`	*packed single*	4 `float`	8 `float`
`pd`	*packed double*	2 `double`	4 `double`
`ss`	*scalar single*	1 `float`	—
`sd`	*scalar double*	1 `double`	—
`d`	*packed dword*	4 `int32`	8 `int32` (AVX2)
`b`	*packed byte*	16 `int8`	32 `int8` (AVX2)

Ler o sufixo é ler a instrução: `mulps` multiplica quatro `float`, `mulsd` multiplica um `double`, `paddd` soma oito `int32` num YMM. O `p` inicial das instruções de inteiro é de *packed*, e o `v` das versões AVX (`vaddps`) é a codificação VEX de três operandos, que preserva os dois fontes em vez de destruir um deles.

Armadilha — `movaps` exige alinhamento; `movups` não

As instruções de carga vêm em dois sabores: `movaps` (*aligned*) falha com `SIGSEGV` se o endereço não for múltiplo de 16, e `movups` (*unaligned*) aceita qualquer um. Em processadores atuais a diferença de custo

entre as duas, quando o dado *está* alinhado, é praticamente nula — de modo que a versão alinhada compra pouco desempenho e vende muita fragilidade. É a mesma instrução do bug de ABI do módulo 7, agora vista de frente: escrever `alignas(16)` no dado (ou `alignas(32)` para AVX) é o que torna `movaps` seguro, e é o assunto do subprojeto `src/layout/`.

A redução horizontal é o gargalo, não a soma. O produto interno de quatro `float` do módulo 8 mostra a assimetria fundamental do SIMD: a parte vetorial é uma instrução (`mulps`), e reduzir os quatro resultados a um único número exige duas rodadas de `shufps` + `addps`. As operações *verticais* (faixa com faixa) são baratas; as *horizontais* (entre faixas do mesmo registrador) são caras. É por isso que o conselho prático de SIMD é sempre o mesmo: mantenha o acumulador vetorial, reduza uma vez só, no fim.

```
-- 8.2 Aritmetica Packed -- 4 operacoes em 1 instrucao --
  ADDPS [1,2,3,4] + [10,20,30,40] = [11,22,33,44]
  SQRTPS raiz de [4,9,16,25] = [2,3,4,5]

-- 8.3 Produto interno 4D via SSE --
  a.b = 1x5 + 2x6 + 3x7 + 4x8 = 70  (esperado: 70)

-- 9.1 Registradores YMM -- 8 floats em paralelo --
  VADDPS (8 floats): [11,22,33,44,55,66,77,88]

-- 9.2 FMA -- Fused Multiply-Add (a*b + c em uma instrucao) --
  FMA: a[i]*2 + 10:  [12,14,16,18,20,22,24,26]
  FMA e mais preciso que MUL+ADD separados (1 arredondamento vs 2)
```

Listagem 317: Saída medida de `dist/bin/asm_08_simd_sse` e `dist/bin/asm_09_simd_avx`.

A observação sobre FMA merece destaque porque é frequentemente lida ao contrário: `vfmadd` não é apenas mais rápido, é *mais preciso* — calcula $a \times b + c$ com um único arredondamento no fim, em vez de dois. Essa é exatamente a razão pela qual o compilador **não** funde `a*b + c` em FMA por conta própria sem `-ffast-math`: o resultado seria diferente do que o IEEE 754 manda, ainda que melhor.

Armadilha — misturar AVX e SSE sem `vzeroupper` custa dezenas de ciclos

Quando código AVX deixa a metade alta de um registrador YMM "suja" e em seguida se executa uma instrução SSE legada, o processador precisa salvar e restaurar esse estado — uma penalidade de transição que chegou a 70 ciclos no Sandy Bridge. A instrução `vzeroupper` zera as metades altas e elimina o problema; o compilador a emite sozinho ao fim de funções AVX, mas em assembly escrito à mão ela é responsabilidade de quem escreve, e o momento certo é antes de retornar para um chamador SSE ou de chamar qualquer função da libc.

IX.3.10 Módulo 10 — operações de *string*, e o segundo bug de ABI

As instruções de *string* do x86 são as mais antigas do conjunto e as únicas que usam registradores *implícitos*: elas não recebem operandos, e sim leem `rsi`, `rdi` e `rcx` por decreto.

Os prefixos que as tornam laços são três: `rep` repete enquanto `rcx` não zera; `repe` repete enquanto `rcx` não zera *e* ZF=1; `repne`, enquanto `rcx` não zera *e* ZF=0. E a direção da caminhada é ditada pela *flag* DF: `cld` a zera (endereços crescentes, o normal) e `std` a liga (decrescentes).

Tabela 110: As quatro operações de *string* e seus registradores implícitos (de `src/asm_course/10_string_ops.cpp`).

Instrução	Faz	Serve para
`movs`	`[rdi] <- [rsi]`, avança ambos	`memcpy`
`stos`	`[rdi] <- al/eax/rax`, avança	`memset`
`scas`	compara `[rdi]` com `al`	`strlen`, `memchr`
`cmps`	compara `[rdi]` com `[rsi]`	`memcmp`

Regra — a ABI garante DF=0 na entrada de toda função — e cobra a volta

Se a sua rotina emite `std`, ela **deve** emitir `cld` antes de retornar. Uma função chamada depois, que suponha o padrão, copiaria memória para trás. É uma das poucas cláusulas da ABI sobre *flags*, e é silenciosa: a violação não gera nem *warning* nem falha imediata.

O segundo bug de ABI. O demo 10.5 usa `std` + `rep movsb` num bloco de `asm` inline. A versão anterior declarava assim:

```
asm volatile (
    "std\n\t"
    "rep movsb\n\t"
    "cld"
    :                                          // sem saidas
    : "D"(&rev[7]), "S"(&arr[7]), "c"((size_t)8)
    : "memory"
);
```

Listagem 318: A declaração errada — os três registradores como *entrada pura*.

Declarar um operando como *input* é uma promessa ao compilador: "este bloco lê esse registrador e não o altera". Mas `rep movsb` altera os três — move `rdi` e `rsi` a cada byte e zera `rcx`. O GCC acreditou na promessa, manteve outros valores vivos nesses registradores e os reutilizou depois do bloco. O sintoma foi um ponteiro corrompido, e o *crash* apareceu na linha *seguinte* ao `asm`, no laço de verificação — não no assembly.

```
void*       d = &rev[7];
const void* o = &arr[7];
std::size_t n = 8;

asm volatile (
    "std\n\t"
    "rep movsb\n\t"
    "cld"
    : "+D"(d), "+S"(o), "+c"(n)   // read-write: o bloco os ALTERA
    :
    : "memory"
);
```

Listagem 319: A correção, em `src/asm_course/10_string_ops.cpp`: `"+"` = *read-write*.

Armadilha — não se pode "consertar" isso com *clobber*

O reflexo natural é listar `"rdi"`, `"rsi"` e `"rcx"` nos *clobbers*. O GCC recusa: um registrador não pode ser simultaneamente operando e *clobber*. É exatamente o vazio que a *constraint* `"+"` preenche — e ela exige que o operando seja uma *variável*, mesmo que o valor final não interesse a ninguém. A alternativa equivalente é declarar saídas `"=D"`, `"=S"`, `"=c"` e amarrar as entradas a elas com `"0"`, `"1"`, `"2"`.

Regra — os dois bugs contam a mesma história

O do módulo 7 quebrou um contrato entre *duas funções* (o alinhamento de 16 bytes no `call`); o do módulo 10 quebrou um contrato entre *o programador e o compilador* (a lista de operandos do `asm`). Nos dois casos o compilador não verificou nada, o código montou e ligou sem uma única mensagem, e a falha se manifestou longe do erro — quatro quadros acima num, uma linha abaixo no outro. É a definição operacional do que uma ABI é: uma promessa que ninguém confere e todo mundo usa.

IX.3.11 Módulo `11` — *syscalls*: a fronteira com o *kernel*

Um *syscall* não é uma chamada de função: é uma troca de nível de privilégio. A instrução `syscall` salva o estado, entra no *kernel*, executa e volta — e por isso tem uma convenção *própria*, parecida com a da ABI mas não igual.

Tabela 111: Convenção de *syscall* contra convenção de função (de `src/asm_course/11_syscalls.cpp`).

Papel	Função (System V)	*Syscall*
seletor	—	`rax` = número
argumentos 1–3	`rdi rsi rdx`	`rdi rsi rdx`
argumento 4	`rcx`	`r10`
argumentos 5–6	`r8 r9`	`r8 r9`
retorno	`rax`	`rax`, negativo = `-errno`
destruídos	`rax rcx rdx ...`	`rcx`, `r11`

A troca de `rcx` por `r10` não é capricho: a própria instrução `syscall` guarda o `rip` de retorno em `rcx` e o RFLAGS em `r11`, para que o `sysret` saiba voltar. Os dois registradores são, portanto, *destruídos pelo hardware* — e omiti-los do *clobber* de um `asm` inline é a mesma classe de erro do módulo 10.

```
-- 11.3 Clobbers importantes do SYSCALL --
  rcx antes do syscall: 0x0000000000000000
  rcx apos  do syscall: 0x00005566635B78A4  (modificado pelo kernel!)

-- 11.4 Verificacao de erro -- retorno negativo = -errno --
  open("/arquivo_que_nao_existe") -> -2
  -ret = 2 = ENOENT (arquivo nao encontrado)
```

Listagem 320: Saída medida de `dist/bin/asm_11_syscalls`: a prova de que `rcx` é destruído, e o retorno negativo como `errno`.

Note também o que a libc faz e o *syscall* cru não faz: traduzir `-2` em "retorne `-1` e ponha `2` em `errno`". A variável `errno` é uma invenção de biblioteca, não do *kernel*.

IX.3.12 Módulo `12` — a FPU x87, e por que ela não deve ser usada

Antes do SSE2, ponto flutuante em x86 era feito numa unidade separada, a x87, cujos oito registradores formam uma *pilha* — `st(0)` é o topo, e as instruções empurram (`fld`), leem (`fst`) e retiram (`fstp`). O módulo 12 existe por três razões honestas: o formato interno de 80 bits ainda é a maior precisão disponível no hardware x86; instruções como `fsin`, `fpatan` e `fyl2x` não têm equivalente em SSE; e código legado (e o modo 32 bits) ainda a produzem, de modo que ela aparece em *disassembly* de terceiros.

```
-- 12.1 A pilha x87 -- FLD, FADD, FSTP --
  3.0 + 4.0 via x87 = 7.0
  3.0 x 4.0 via x87 = 12.0
```

```
4
5  -- 12.2 Funcoes trigonometricas x87 --
6    sin(pi/6) = 0.500000  (esperado: 0.500000)
7    atan2(1,1) = 0.785398  (esperado: pi/4 = 0.785398)
```

Listagem 321: Saída medida de `dist/bin/asm_12_fp_x87`.

Armadilha — 80 bits internos são a causa, não a cura, de resultados irreproduzíveis

O x87 calcula em 80 bits e arredonda ao guardar em memória. Isso significa que o mesmo `double` pode valer uma coisa enquanto está no registrador e outra depois de passar pela pilha — e que o resultado muda conforme o compilador decidiu ou não derramar um valor intermediário. É a origem do folclore de que "ponto flutuante é imprevisível". O SSE2 encerrou o assunto calculando em exatamente 64 bits, e é por isso que ele, e não a maior precisão, é o padrão em x86-64: reprodutibilidade vale mais do que dígitos extras.

IX.3.13 Módulo 13 — otimização, e a medição que contrariou a expectativa

O módulo 13 compara três versões da mesma soma de um vetor de 1024 `int32`: um laço escrito em assembly inline, uma versão com *unrolling* de fator 4 (`asm_sum_unrolled`) e uma versão SIMD com SSE2 (`asm_sum_simd`). As três produzem o mesmo número — o que já é metade do valor do exercício — e a medição é feita com RDTSC sobre dez mil repetições.

```
1  -- 13.1 Loop Unrolling -- reduz overhead do loop --
2    N=1024, soma esperada: 523776
3    loop simples:   523776
4    loop unrolled:  523776
5    loop SIMD:      523776
6    simples         :   8222658 ciclos/10k iteracoes
7    unrolled(4x)    :   7702105 ciclos/10k iteracoes
8    SIMD(SSE2)      :   4103781 ciclos/10k iteracoes
```

Listagem 322: Saída medida de `dist/bin/asm_13_optimization`, seção 13.1.

Os números dizem o que se esperava, e com proporções instrutivas. O *unrolling* de fator 4 rendeu 1,07× — pouco, porque o que ele elimina (incremento, comparação e salto) já era quase de graça num *pipeline* com predição de desvio quase perfeita. O SIMD rendeu 2,00× — muito, e ainda assim menos do que os 4× que a aritmética ingênua prometeria, porque quatro somas por instrução não compram nada quando o gargalo passa a ser trazer os dados até a ALU.

```
1  -- 13.3 PREFETCHNTA -- pre-carrega dados no cache --
2    Copia simples:      1887 ciclos
3    Copia c/ prefetch:  3441 ciclos
```

Listagem 323: Mesma execução, seção 13.3: a medição que contrariou a expectativa.

Armadilha — o *prefetch* saiu quase duas vezes mais lento — e o resultado está certo

A rotina `asm_prefetch_copy` emite `prefetchnta 256(%rdi)` a cada iteração, pedindo à CPU que traga a linha de cache quatro linhas à frente. A expectativa era ganhar; o medido foi perder 1,8×. Três razões, todas verificáveis no código do módulo:

(1) O dado já estava no cache. O vetor tem 4096 `int64`, ou 32 KiB — cabe no L1/L2 — e o laço *simples* roda *primeiro*, deixando tudo quente. A "cópia com *prefetch*" foi medida sobre dados que o *prefetch* não precisava buscar.

(2) `nta` significa *non-temporal*. Essa variante diz ao processador "vou usar isto uma vez, não polua

o cache com ele" — e portanto trabalha contra a reutilização que o *benchmark* acabara de criar.
(3) A instrução não é de graça. Uma leitura extra por iteração, num laço em que o dado já está a quatro ciclos de distância, é custo puro.
A lição de método é maior que a de *prefetch*: um número que contraria a hipótese quase nunca é "ruído". Aqui ele diz que o *benchmark* media outra coisa — ordem de execução, estado de cache e semântica da instrução — e não a pergunta que se queria fazer. Compare com a advertência já feita neste capítulo sobre medir o relógio errado: as duas apontam para a mesma conclusão, a de que *microbenchmark* é uma disciplina própria e que a resposta certa quase sempre é medir o programa inteiro com `perf`.

O módulo fecha com `.p2align` — a diretiva que alinha o início de um laço a 16 bytes (SSE) ou 32 (AVX), para que a primeira iteração não desperdice ciclos esperando a busca de instruções completar — e com a versão sem desvio de `clamp`, dois `cmov` encadeados que substituem dois `if`.

IX.3.14 Módulo 14 — ler o que o compilador escreveu

Este é, na prática, o módulo mais útil dos quinze, e o único cujo conteúdo se aplica a todo dia de trabalho. Ele não ensina a escrever assembly: ensina a *ler* o que já existe.

Tabela 112: As ferramentas, do cabeçalho de `src/asm_course/14_disassembly.cpp`.

Comando	Serve para
`objdump -d -M intel bin`	desmontar o binário inteiro
`objdump -d -M intel bin | c++filt`	o mesmo, com nomes C++ legíveis
`g++ -O2 -S -masm=intel a.cpp`	o assembly do compilador, sem montar
`gdb: disassemble /m f`	fonte e assembly intercalados
`godbolt.org`	comparar compiladores e níveis de otimização

O formato da saída do `objdump` é sempre o mesmo — endereço, *bytes* em hexadecimal, mnemônico, operandos — e a barreira de entrada não é a sintaxe, é o vocabulário de expressões idiomáticas (Tabela 113). Reconhecer meia dúzia delas transforma uma parede de texto em prosa.

Tabela 113: Expressões idiomáticas que o compilador emite o tempo todo.

O que aparece	O que significa
`xor eax, eax`	`eax = 0` (dois *bytes* em vez de cinco)
`test eax, eax`	`eax == 0?` (sem imediato)
`lea eax, [rdi+rdi]`	multiplicação, sem tocar nas *flags*
`movsx` / `movzx`	conversão com / sem sinal
`mov rax,[rdi]` + `call [rax]`	despacho virtual pela vtable / vptr
`jmp outra_funcao`	*tail call* — `call`+`ret` fundidos
`endbr64`	alvo válido de salto indireto (CET)

```
1  Primeiros bytes: F3 0F 1E FA 41 55 BA 0C 00 00 00 BE 23 00 00 00
2  (prologo tipico: 55=push rbp, 48 89 E5=mov rbp,rsp)
```

Listagem 324: Saída medida de `dist/bin/asm_14_disassembly`: os primeiros *bytes* de uma função real deste repositório.

Armadilha — o prólogo do livro-texto não é o prólogo do seu binário

O próprio módulo imprime a legenda "prólogo típico: `55` = push rbp" e os *bytes* medidos começam com `F3 0F 1E FA` — que é `endbr64`, a marca de alvo legítimo de salto indireto que o GCC insere

por causa de `-fcf-protection` (Intel CET), ligada por padrão nas distribuições atuais. Só depois vem `41 55`, que é `push r13` e não `push rbp`: com `-O2` o compilador dispensou o ponteiro de quadro (`-fomit-frame-pointer`) e usa `rsp` diretamente. Dois desencontros entre o material didático e o binário real, na mesma linha de saída — e nenhum deles é erro: o hardware e as políticas de segurança mudaram debaixo do texto. Ler o *seu* binário é a única leitura que vale.

IX.3.15 Módulo 15 — o *capstone*, e o defeito que só aparece rodando

O módulo final reimplementa cinco funções da libc em assembly — `memcpy` com blocos de 16 *bytes* via XMM, `memset` com `rep stosq` e o truque de replicar o *byte* multiplicando por `0x0101010101010101`, `memcmp`, `strlen` com `repne scasb` e um *insertion sort* — e, crucialmente, compara cada uma com a implementação do sistema.

```
--- 15.1 asm_memcpy -- copia com SIMD ---
  [ok] n=   0 ...  n=1024: conteudo correto   (12 tamanhos)
  [ok] retorno == dst
--- 15.2 asm_memset -- preenchimento rapido ---
  [ok] todos os bytes == 0xAB / zeragem completa / retorno == dst
--- 15.3 asm_memcmp -- comparacao de memoria ---
  [ok] iguais -> 0 / diferentes -> != 0 / sinal correto
--- 15.4 asm_strlen -- comprimento de string ---
  [FALHOU] strlen("") = 18446744073709551615 (esperado 0)
  [FALHOU] strlen("hello") = 4 (esperado 5)
  [FALHOU] strlen("cpp_academy") = 10 (esperado 11)
--- 15.5 asm_isort -- insertion sort ---
  [ok] array ordenado corretamente / igual ao std::sort
```

Listagem 325: Saída medida de `dist/bin/asm_15_capstone`, recortada.

Aviso

Esta é uma falha **aberta** no repositório no momento em que este capítulo foi escrito, e está registrada aqui deliberadamente: o valor didático de uma medição está em publicá-la como ela saiu.

A aritmética do defeito é curta. O idioma de `strlen` com `repne scasb` carrega `rcx` com -1 e deixa a instrução decrementá-lo uma vez por *byte* comparado. Para `"hello"` são seis comparações — cinco letras mais o `'\0'`, que é o que faz a busca parar — de modo que `rcx` termina em -7. O `notq %rcx` converte isso em 6, o número de *bytes* varridos. O comprimento é, portanto, $6 - 1 = 5$: subtrai-se *um* pelo terminador. A rotina subtrai dois.

```
asm_strlen:
    movq   %rdi, %rcx        # dead store: sobrescrito duas linhas abaixo
    xorb   %al, %al          # al = 0 -- o byte procurado
    movq   $-1, %rdx
    movq   %rdx, %rcx        # rcx = -1 (contador "infinito")
    repne  scasb             # rdi++, rcx-- ate [rdi] == al
    notq   %rcx              # rcx = numero de bytes varridos
    leaq   -2(%rcx), %rax    # <- deveria ser -1(%rcx)
    ret
```

Listagem 326: O trecho (resumo de `src/asm_course/15_capstone_asm.s`) — e, idêntico, em `10_string_asm.s` e em `src/outline_asm/asm_funcs.s`.

Para a cadeia vazia o efeito é espetacular: um *byte* varrido, menos dois, -1 — que num `size_t` é `18446744073709551615`. Alocar essa quantidade de memória é o tipo de bug que derruba um servidor.

Armadilha — o mesmo defeito, copiado em três arquivos

A mesma receita de sete linhas aparece em `15_capstone_asm.s`, em `10_string_asm.s` (onde o módulo 10 acusa `strlen("hello, assembly!") = 15`, um a menos que os 16 corretos) e em `src/outline_asm/asm_funcs.s` (onde a comparação com a libc falha nos quatro casos testados). Três alvos distintos, três verificações independentes, e todas as três dizem a mesma coisa — porque o trecho foi copiado antes de ser conferido. O que mudou não foi o código: foi o repositório passar a *construir e executar* os três alvos.

E uma segunda descoberta, no módulo 10. Depois que a declaração de operandos do `rep movsb` foi corrigida, o programa deixou de estourar e passou a imprimir o veredicto que a verificação sempre teve:

```
-- 10.5 Direction Flag -- CLD vs STD --
  [FALHOU] STD+REP MOVSB inverteu o array
  original :   [1,2,3,4,5,6,7,8]
  invertido  :[1,2,3,4,5,6,7,8]
```

Listagem 327: Saída medida de `dist/bin/asm_10_string_ops`, seção 10.5.

A verificação esperava `rev[i] == arr[7-i]`, e o que saiu foi uma cópia idêntica. O programa está certo e a expectativa é que estava errada: `std` inverte o sentido da *caminhada*, não a ordem dos *dados*. Com `rdi` e `rsi` apontando ambos para o último *byte* e ambos decrementando, copia-se `arr[7]` para `rev[7]`, `arr[6]` para `rev[6]`, e assim por diante — o resultado é o mesmo vetor. Inverter exigiria os dois ponteiros caminhando em *sentidos opostos*, e nenhum prefixo `rep` faz isso. O `std` tem exatamente um uso legítimo, e é o oposto do pretendido aqui: copiar regiões que se *sobrepõem*, quando o destino está acima da origem — que é como `memmove` se distingue de `memcpy`.

IX.4 Um alvo que não é construído é um alvo que apodrece

Os dois bugs de ABI deste capítulo não foram encontrados por revisão de código, nem por uma ferramenta de análise estática. Foram encontrados no dia em que o repositório passou a construir *todos* os alvos e a executá-los. Antes disso, o `asm_course` compilava — e não rodava.

Regra — um alvo desabilitado é dívida técnica com juros compostos

Enquanto um alvo não é construído, ele não é compilado, não é ligado, não é executado e não é verificado. Nada nele é falso ainda; simplesmente nada nele é *verdadeiro*. Ao serem habilitados, os alvos "pesados" do repositório entregaram, de uma vez: quatro `#include` faltando (entre eles o `<fmt/core.h>` de `src/aws/s3/main.cpp`, que compilava na cabeça de quem escreveu e em lugar nenhum além dela), uma API de terceiros que mudou de forma sem que ninguém notasse (os *callbacks* de ciclo de vida do AMQP-CPP migraram do `TcpConnection` para o *handler*), um `demo` de banco de dados que só funcionava na primeira execução, uma constante de caractere multibyte truncada em silêncio, uma subtração sem sinal que dava a volta — e os dois defeitos de ABI descritos aqui.

A regra prática que se extrai disso é curta, e vale para qualquer repositório: **comente um `subdir()` e você não desabilitou um alvo, criou um prazo**. Hoje o `src/meson.build` do repositório constrói os 57 subprojetos, sem nenhuma linha comentada, produzindo 98 binários instalados em `dist/bin` — dos quais 15 são os módulos deste curso, de `asm_01_registers` a `asm_15_capstone` — e tudo isso sob `-Wall -Wextra -Wpedantic` sem um único *warning*. Não é uma métrica de vaidade: é a condição sob a qual os bugs deste capítulo puderam ser vistos.

IX.5 Síntese

O capítulo tem uma conclusão que contraria a expectativa: **o principal produto de aprender assembly não é escrever assembly**. É passar a ler o que o compilador gera, e com isso entender de onde vem o desempenho — por que `final` permite devirtualizar (Seção II.7), por que o *pipeline* de *ranges* some em `-O2` e não em `-O0` (Seção III.2), por que um *functor* é inlinável e `std::function` não (Seção IV.4).

E tem uma segunda conclusão, que só apareceu quando o repositório passou a construir e executar tudo: os defeitos deste capítulo não estavam no assembly difícil. Estavam nos *contratos* — 16 bytes de alinhamento no `call`, três registradores declarados como entrada quando eram entrada-e-saída, uma subtração a mais num `lea`. Nenhum deles é uma questão de esperteza com *opcodes*; todos são questões de dizer a verdade a quem confia em você sem conferir. É essa, e não a habilidade de escrever `movaps`, a competência que descer de nível ensina.

Tabela 114: Quando descer de nível — e por qual porta.

Necessidade	Ferramenta
entender por que está lento	`objdump`, `-S`, `perf`
usar uma instrução vetorial	*intrinsic*, não assembly
contar bits, girar, trocar *bytes*	`<bit>` do C++20
identificar a CPU em execução	CPUID por `asm`
barreira só de compilação	`asm volatile ("" ::: "memory")`
rotina longa e crítica	arquivo `.s` com ABI respeitada
syscall sem libc	instrução `syscall`

Regra — o C++20 absorveu boa parte do que exigia assembly

`<bit>` trouxe `std::popcount`, `std::rotl`, `std::countl_zero` e `std::bit_cast` — todos com a mesma instrução gerada pelo compilador, e todos `constexpr`, portáteis e sem UB (comportamento indefinido). Antes de escrever um bloco `asm` para uma manipulação de bits, verifique se o cabeçalho já a oferece: o subprojeto `src/bits/` — hoje construído e instalado como `dist/bin/bits`, e não mais um item de `ROADMAP` — cobre justamente esse terreno.

X Interoperação e visão computacional

Os quatro subprojetos deste capítulo têm em comum o fato de o C++ não estar sozinho. Em dois deles ele convive com Python — uma vez como convidado (X.1) e outra como anfitrião (X.2); no terceiro, ele é o produto de um gerador de código (X.3); e no quarto, ele é a camada em volta de um modelo de rede neural treinado em outro lugar (X.4).

O assunto técnico recorrente é a **fronteira**: quem é dono do quê, quem libera o quê, e o que acontece com uma exceção que tenta atravessar. É a mesma pergunta da Seção V.4, agora com um interpretador do outro lado em vez de um `.so`.

X.1 Python importando C++

O subprojeto `src/python/` implementa o *mesmo* módulo de extensão duas vezes — uma com a API C do CPython, outra com pybind11 [12] — e a comparação é o conteúdo. As duas versões exportam as mesmas funções, e a diferença de volume de código é de cerca de cinco para um.

X.1.1 pybind11: a lógica separada do *binding*

O que pybind11 permite é escrever C++ que não sabe que Python existe, e depois declarar a exposição em um bloco separado:

```
int somar(int a, int b) { return a + b; }

PYBIND11_MODULE(academy_pb, m) {
    m.doc() = "Módulo academy com PyBind11 + OpenMP";
    m.def("somar",    &somar,    "Soma dois inteiros",      py::arg("a"), py::arg("b"));
    m.def("fatorial", &fatorial, "Calcula fatorial de n", py::arg("n"));
}
```

Listagem 328: A lógica não menciona Python; a declaração do módulo é puramente descritiva.

A conversão de tipos é automática — `std::vector<double>` vira lista Python e volta — e as exceções também: um `throw py::value_error(...)` chega ao Python como um `ValueError` legítimo, com *traceback*.

X.1.2 A API C, e o que ela obriga a escrever

A mesma função de fatorial, na API do CPython, mistura a lógica com a burocracia do *binding*:

```
static PyObject* py_fatorial(PyObject* self, PyObject* args) {
    int n;
    if (!PyArg_ParseTuple(args, "i:fatorial", &n))
        return nullptr;

    if (n < 0) {
        PyErr_SetString(PyExc_ValueError, "fatorial não definido para negativos");
        return nullptr;
    }

    long r = 1;
    for (int i = 2; i <= n; i++) r *= i;

    return PyLong_FromLong(r); // converte long C → Python int
```

```
15 }
```

Listagem 329: *Parsing* manual dos argumentos, sinalização manual de erro, conversão manual do retorno.

Armadilha — na API C, "lançar" é devolver `nullptr` depois de armar o erro

Não há exceção: o protocolo é `PyErr_SetString` seguido de `return nullptr`, e o chamador precisa verificar. Devolver `nullptr` sem armar o erro produz um `SystemError` no Python; armar o erro e devolver algo diferente de `nullptr` deixa o erro pendente, que estoura na próxima chamada não relacionada — num ponto arbitrário do programa. É a mesma classe de problema do `dlerror()` da Seção V.4: um protocolo de erro por convenção, sem apoio do tipo.

Tabela 115: As duas formas de escrever a mesma extensão.

Aspecto	API CPython	pybind11
volume de código	cerca de 5× maior	conciso
lógica e *binding*	misturados	separados
conversão de tipos	manual	automática
erros	`PyErr_SetString` + `nullptr`	`throw` normal
GIL	macro `Py_BEGIN_ALLOW_THREADS`	RAII
containers da STL	manual	um `#include`
overhead	zero	mínimo

X.1.3 O GIL, e por que ele obriga a pensar

O *Global Interpreter Lock* é um *mutex* que serializa a execução de *bytecode* Python: apenas uma *thread* interpreta por vez. Enquanto o C++ está executando, ele segura o GIL — e nenhuma outra *thread* Python avança. Uma rotina longa em C++ que não o libere transforma um programa Python multithread em um programa sequencial.

```
1  std::vector<double> multiplicar_vetor(const std::vector<double>& v, double escalar) {
2      std::vector<double> resultado(v.size());
3      {
4          py::gil_scoped_release release;   // libera GIL
5          #pragma omp parallel for
6          for (int i = 0; i < (int)v.size(); i++)
7              resultado[i] = v[i] * escalar;
8      } // destrutor → GIL readquirido automaticamente (RAII)
9      return resultado;
10 }
```

Listagem 330: Liberar o GIL durante o trabalho paralelo, e readquiri-lo pelo destrutor.

Regra — nenhuma API do Python pode ser tocada com o GIL liberado

Dentro do bloco de `gil_scoped_release`, o código só pode mexer em dados C++ puros — `std::vector`, ponteiros, aritmética. Tocar em qualquer `PyObject*` sem o GIL é corrupção de estado do interpretador, e o sintoma é um *crash* sem relação com o ponto do erro. Note que isso é exatamente o que o exemplo faz certo: o `vector` de entrada já foi convertido antes de liberar, e o de saída só é convertido depois de readquirir.

Por que — por que o C++ é uma saída legítima para o GIL

Não é apenas velocidade. Como uma extensão em C++ pode liberar o GIL durante o trabalho pesado, ela dá *paralelismo real* a um programa Python — o que *threads* Python puras não dão. É por isso que

NumPy, SciPy e todo o ecossistema numérico são, por baixo, C, C++ e Fortran: eles não estão ali só para serem rápidos, estão ali para poderem soltar o GIL.

X.2 C++ hospedando Python

O subprojeto `src/embed-python/` inverte a direção: o executável é C++, e o interpretador Python vive dentro dele. Os casos de uso são reconhecíveis — motor de *scripting* para o usuário final, recarga de lógica sem recompilar, configuração em Python em vez de YAML, *plugins* escritos por quem não compila C++.

X.2.1 O ciclo de vida, e o *guard* que o garante

```
1  struct PythonGuard {
2      PythonGuard()  { Py_Initialize(); }
3      ~PythonGuard() { Py_Finalize();   }
4
5      PythonGuard(const PythonGuard&)            = delete;
6      PythonGuard& operator=(const PythonGuard&) = delete;
7  };
```

Listagem 331: A mesma técnica do `ThreadGuard` e do `RedirectGuard`, aplicada ao interpretador.

A cópia é deletada porque o interpretador é único por processo — é o tipo *move-only* (aqui, nem isso) da Seção II.3 aplicado a um recurso genuinamente singular. E a regra que o *guard* não garante sozinho: **todo `PyObject*` precisa ser liberado antes de `Py_Finalize()`**, o que é o mesmo problema de ordem de destruição que a Seção VII.6 descreveu para o SDK da AWS.

X.2.2 A contagem de referências, e as duas espécies de ponteiro

O CPython gerencia memória por contagem de referências, e a API distingue dois tipos de `PyObject*` que *têm o mesmo tipo estático*:

Tabela 116: Duas semânticas de posse com a mesma assinatura.

Espécie	Quem libera	Exemplos
new reference	você, com `Py_DECREF`	`PyImport_ImportModule`, `PyLong_FromLong`
borrowed reference	o dono; **não** decremente	`PyImport_AddModule`, `PyList_GetItem`

Armadilha — a diferença entre vazar e derrubar o programa está na documentação, não no tipo

Decrementar uma referência emprestada libera um objeto que ainda tem dono, e o programa morre depois, num acesso a memória liberada. Esquecer de decrementar uma referência própria vaza. E *nada* no código distingue os dois casos — os dois são `PyObject*`. Pior ainda: algumas funções **roubam** a referência, como `PyList_SetItem`, e chamar `Py_DECREF` depois dela é um *double free*. É o exemplo mais didático do repositório sobre por que posse deve estar no tipo — e sobre o que `unique_ptr` e `shared_ptr` (Seção II.5) resolveram.

Regra — envolva `PyObject*` num tipo com destrutor

Um `unique_ptr<PyObject, decltype(&Py_DecRef)>` — ou o `py::object` do pybind11, que faz exatamente isso — transforma a disciplina manual em invariante do tipo. O subprojeto usa a API crua de propósito, para que a disciplina fique visível; em código de produção, ela deve ficar escondida atrás de um invólucro.

X.2.3 As três formas de conversar

O subprojeto percorre seis demonstrações que se reduzem a três padrões. `PyRun_SimpleString` executa código e não devolve nada — serve para efeito colateral, não para cálculo. Importar um módulo e chamar uma função (`PyImport_ImportModule` → `PyObject_GetAttrString` → `PyObject_CallOneArg`) é o caminho para obter um valor de volta. E injetar dados no *namespace* `__main__` é como se passa um `std::vector` para um trecho de análise escrito em Python.

O sexto demo fecha a fronteira no sentido inverso: capturar uma exceção Python e transformá-la em algo tratável em C++, com `PyErr_Fetch` e `PyErr_NormalizeException`. Sem isso, um `ZeroDivisionError` dentro de um *script* do usuário se manifesta apenas como um código de retorno -1.

X.3 Análise léxica e sintática

O subprojeto `src/parser/` usa Flex e Bison [13] para avaliar expressões aritméticas — a mesma gramática que o padrão *Interpreter* da Seção V.2 resolve à mão, o que torna os dois diretamente comparáveis.

X.3.1 Duas fases, duas ferramentas

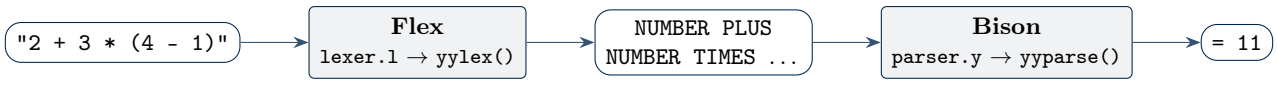


Figura 9: A separação clássica. O analisador léxico não conhece gramática — só reconhece padrões e emite *tokens*; o analisador sintático não conhece caracteres — só consome *tokens*. É a mesma divisão de responsabilidade que o SRP (Seção V.1) recomendaria.

X.3.2 Precedência declarada, não codificada

O ponto mais instrutivo do subprojeto é como a precedência é expressa. A gramática das expressões é ambígua por escrito — `expr : expr PLUS expr` não diz se `2 + 3 * 4` é 20 ou 14. Bison resolve por declaração:

```
%left PLUS MINUS      /* menor precedência */
%left TIMES DIVIDE    /* maior precedência */
%right UMINUS         /* unário, maior precedência */
```

Listagem 332: Declaradas de baixo para cima: quem vem depois tem precedência maior.

`%left` declara também a associatividade, que resolve `a - b - c`. E `%prec UMINUS` permite que a mesma *regra* — `MINUS expr` — tenha precedência diferente da do `MINUS` binário: é como o menos unário fica mais forte que a multiplicação sem que o *token* mude.

Por que — a precedência é uma propriedade do parser, não da gramática

No *Interpreter* escrito à mão (Seção V.2), a precedência é estrutural: `expr` chama `term`, que chama `factor`, e a ordem das chamadas *é* a precedência. Mudar a tabela de precedência exige reescrever a cadeia de funções. Com Bison, a gramática fica plana e ambígua de propósito, e a precedência é uma tabela declarativa — muito mais fácil de ajustar, e muito mais fácil de errar sem perceber, porque a ambiguidade resolvida por declaração não aparece na leitura da gramática.

X.3.3 A fronteira com o C++

Flex e Bison geram C, e o `main.cpp` precisa de `extern "C"` para chamar o que eles produziram — o mesmo mecanismo do Seção IX.2 e do Seção V.4. E a integração no *build* é por `custom_target` do Meson, que declara "deste arquivo, com este comando, sai aquele" e deixa o Ninja resolver a ordem (Seção XI.2).

Tabela 117: Gerador de *parser* contra *parser* escrito à mão.

	Flex/Bison	**Recursivo descendente**
dependências	`flex`, `bison` no sistema	nenhuma
gramáticas grandes	ideal	trabalhoso
recuperação de erro	robusta	manual
mensagens de erro	genéricas	sob medida
depuração	o código gerado é ilegível	é código seu
quando usar	linguagem completa	DSL com poucas regras

X.4 Inferência de rede neural em C++

O subprojeto `src/yolo/` carrega um modelo YOLOv10 [14] exportado para ONNX e o executa com ONNX Runtime [15], com OpenCV cuidando de imagem e câmera. Ele fecha o repositório porque é o único subprojeto que entrega um *pipeline* de produção completo: entrada, pré-processamento, inferência, pós-processamento, saída — e medição.

X.4.1 O trabalho está nas pontas



Figura 10: O *pipeline*. A caixa do meio é uma chamada de biblioteca; as duas caixas em volta são onde estão os erros — e onde está boa parte do tempo de execução.

Armadilha — quatro conversões, e cada uma tem uma convenção invisível

O pré-processamento faz quatro coisas, e errar qualquer uma produz detecções silenciosamente ruins em vez de erro:

- ***Letterbox*** — redimensionar preservando a proporção e preencher o resto. Um `resize` direto para 640×640 distorce a imagem, e as caixas saem deslocadas.
- **BGR → RGB** — o OpenCV lê em BGR; o modelo foi treinado em RGB. Trocar os canais não

gera erro, só degrada a confiança.

- **Normalização** — dividir por 255 para chegar a $[0, 1]$, que é a escala em que o modelo foi treinado.
- **HWC → CHW** — o OpenCV guarda os canais entrelaçados por pixel; o tensor espera cada canal em um plano contíguo.

E o pós-processamento tem a inversa da primeira: as coordenadas voltam do espaço de 640×640 com preenchimento para o espaço da imagem original.

X.4.2 Medir latência com percentis, não com média

O terceiro modo do subprojeto é um *benchmark*, e o formato dele é a lição:

```
média:   28.50ms
mediana: 27.80ms
p95:     35.20ms
p99:     42.10ms
min:     25.30ms
max:     55.10ms
FPS teórico (média): 35.1
```

Listagem 333: Saída de `src/yolo/`: *benchmark* do `yolov10n` em CPU.

Regra — a média esconde exatamente o que importa

Numa aplicação interativa, o usuário não percebe a média — percebe os quadros lentos. Entre a média de 28,5 ms e o p99 de 42,1 ms há quase 50% de diferença, e é o p99 que determina se a experiência parece fluida. Some a isso a prática de **descartar as primeiras medições** (o *warmup* de cinco inferências do subprojeto), que existe porque a primeira execução paga carregamento de página, alocação de *arenas* internas e aquecimento de cache. Uma medição sem *warmup* e sem percentis é, na prática, uma medição do carregamento do modelo.

X.4.3 A interface, e a decisão de projeto que ela revela

```cpp
struct Detection {
    cv::Rect2f bbox;     // bounding box (x, y, w, h) em pixels
    float      score;    // confiança (0.0 a 1.0)
    int        class_id; // índice da classe (0–79 para COCO)
    const char* label() const; // nome da classe
};
```

Listagem 334: O tipo de domínio que atravessa a fronteira.

Nada de `Ort::Value`, nada de tensores, nada de *strides*: o que sai da classe `YOLOv10` é um `std::vector<Detection>` com coordenadas já em pixels da imagem original. É o mesmo invólucro sobre API estrangeira que apareceu em `http::get`, `db::Connection`, `PhysicsWorld` e `FlightModel` — a quinta ocorrência do mesmo padrão no relatório, e a evidência de que ele não é preferência de estilo.

Armadilha — o modelo não vem no repositório

O `.onnx` precisa ser exportado antes (`export_yolo.py`, ou `yolo export` do *ultralytics*), o que faz o subprojeto depender de um *pipeline* de Python para produzir a entrada do *pipeline* de C++. É uma dependência realista — treinar em Python e servir em C++ é o arranjo usual — e vale registrar que

ela existe: o binário compila sem o modelo e falha em tempo de execução.

X.5 Síntese

O capítulo tem uma pergunta recorrente, e vale enunciá-la como tal: *quem é o dono, e o que acontece na fronteira?*

Na extensão em Python, o dono da memória é o interpretador, e o C++ precisa liberar o GIL para não travar o processo. No interpretador embarcado, o dono é compartilhado, e a contagem de referências — com suas referências próprias, emprestadas e roubadas — é uma disciplina que o tipo não impõe. No *parser*, o dono do código é o gerador, e a fronteira é `extern "C"`. No YOLO, o dono do tensor é o *runtime*, e a fronteira é a conversão de *layout*.

Regra — toda fronteira merece um tipo de vocabulário próprio

`Detection` não é um tensor; `FlightState` não é um objeto do JSBSim; `HttpResponse` não é um `CURL*`. O padrão que atravessa os capítulos VII a X é sempre o mesmo: definir tipos simples do domínio, converter uma vez na fronteira, e não deixar os tipos da biblioteca externa vazarem para o resto do programa. É a inversão de dependência da Seção V.1 — e o que a torna concreta é justamente esse punhado de *structs* sem métodos.

XI Ferramentas do repositório

Os subprojetos deste capítulo não estão em nenhuma etapa da trilha porque não são *sequência*: são *consulta*. O `src/meson/` descreve o sistema de *build* que compila todos os outros; o `src/cmake/` descreve o sistema de *build* que o resto do mundo usa, e é construído por si mesmo, fora do Meson; o `src/parser/` mostra o que acontece quando o fonte de entrada *não é* C++ e precisa virar C++ antes da compilação; o `src/doxygen/` descreve como documentar o que se escreve; e o `src/sandbox/` é o espaço livre para experimentar antes de comprometer um subprojeto novo.

Os dois primeiros merecem mais espaço que todos os outros somados — são os dois maiores `README` do repositório — porque um sistema de *build* mal entendido produz a categoria de problema mais frustrante que existe: aquela em que o código está certo.

XI.1 O estado do *build*, em números

Antes de qualquer explicação, os números medidos. Este capítulo, e o relatório inteiro, descrevem um repositório em que **tudo é construído**: não existe nenhum `subdir()` comentado em `src/meson.build`, e os alvos historicamente "pesados" — Qt, SFML, Bullet, ONNX Runtime, JSBSim, AWS SDK, RabbitMQ, SOCI — estão habilitados e compilando junto com o resto (Tabela 118).

Tabela 118: O *build* completo, medido com `make configure && make` numa árvore limpa.

Grandeza	Valor
subprojetos em `src/` construídos pelo Meson	57
subprojetos com sistema de *build* próprio (`cmake`, `modules`)	2
binários instalados em `dist/bin`	98
warnings sob `-Wall -Wextra -Wpedantic`	0
alvos registrados no `meson test`	6
casos de teste GoogleTest sob o Meson	41
casos de teste sob o CTest (em `src/cmake/`)	11

Aviso

Números como estes envelhecem. O que *não* envelhece é o método: todos eles são reproduzíveis com dois comandos — `ninja -C build` para os *warnings* e `meson test -C build` para os testes — e o critério de aceitação deste repositório é que os três primeiros continuem em "tudo" e o quarto continue em zero.

XI.2 Meson: o sistema de *build*

O subprojeto `src/meson/` é um guia ancorado no *build* real do repositório: cada conceito é explicado e apontado para o arquivo em que já está em uso. E vem com um exemplo vivo — o próprio `meson.build` da pasta exercita os recursos principais, e o executável `meson_demo` imprime, em tempo de execução, tudo o que o *build* injetou nele.

Tabela 119: Quem faz o quê, e quando.

Programa	Papel	Quando roda
Conan 2	baixa e compila as dependências, gera os `.pc`	`make configure`
Meson	lê os `meson.build` e **gera arquivos de *build***	`meson setup`
Ninja	executa o *build*, em paralelo e incrementalmente	`meson compile`

XI.2.1 Três programas, e a divisão que explica tudo

Regra — o Meson não compila nada

Ele é um *gerador*: transforma a descrição declarativa em `build.ninja`. Quem chama o `g++` é o Ninja. Entender essa divisão explica quase todo comportamento estranho — inclusive por que existem duas fases distintas, "configuração" e "*build*", e por que mudar um `meson.build` às vezes exige reconfigurar e às vezes não.

XI.2.2 Os dois tempos

Esta é a ideia que organiza todo o resto (Tabela 120). O que existe em tempo de configuração são *variáveis* do Meson; o que existe em tempo de *build* são *arquivos*.

Tabela 120: Configuração e *build* são universos distintos.

	Configuração	*Build*
quem roda	`meson setup`	`ninja`
frequência	uma vez, ou quando um `meson.build` muda	a cada compilação
o que ocorre	procura o compilador, resolve `dependency()`, avalia `if`	compila, liga, roda `custom_target`
o que existe	as *variáveis* (`fmt_dep`, `qtd_itens`)	os *arquivos* (`.o`, executáveis)

O exemplo da pasta põe os dois lado a lado sobre o *mesmo* arquivo: `fs.read('tabela.txt')` conta as linhas na *configuração*, e um `custom_target` gera um cabeçalho a partir dele no *build*. O programa imprime os dois números, e eles batem — o que é uma demonstração melhor da distinção do que qualquer explicação.

XI.2.3 Anatomia do `meson.build` da raiz

```
project('academy', 'cpp', 'c',
    version: '1.0.0',
    default_options: [
        'cpp_std=c++20',
        'warning_level=3',
    ],
)
```

Listagem 335: As primeiras linhas de `meson.build`, na raiz do repositório.

Três consequências práticas desse arranjo merecem ser sabidas de cor:

1. **`project()` é a primeira linha executável**, e só existe na raiz. Nenhum `src/*/meson.build` tem um — eles são *incluídos*, não são projetos.

2. **`subdir()` não cria escopo novo.** É inclusão textual: `fmt_dep`, declarado na raiz, está visível em `src/syntax/meson.build` sem nenhuma importação. É por isso que a ordem importa — `subdir('proto')` vem antes de `subdir('src')` porque `src/` consome o que `proto/` gera.

3. **`default_options` é padrão, não imposição.** Um alvo discorda com `override_options`, que é como `errors` e `operators` chegam a C++23 sem mexer na raiz (Seção I.6).

Por que — `warning_level=3` é uma decisão didática

Equivale a `-Wall -Wextra -Wpedantic`. Boa parte das armadilhas do Seção II só é apontada nesse nível — `-Wreorder`, `-Wcatch-value`, `-Wpessimizing-move` — e um repositório que existe para ensinar armadilhas não pode silenciá-las. O custo é que código de terceiros incluído por cabeçalho passa a gerar ruído; o benefício é que o código próprio não passa.

XI.2.4 `declare_dependency()`: o recurso que separa um *build* limpo

Uma biblioteca interna tem três coisas que os consumidores precisam saber: o que ligar, onde procurar cabeçalhos, e de que ela mesma depende. Sem `declare_dependency`, cada consumidor repete as três — e erra uma delas cedo ou tarde.

```
saudacao_dep = declare_dependency(
    link_with:           lib_saudacao,
    include_directories: include_directories('.'),
    dependencies:        [fmt_dep],
)
```

Listagem 336: O empacotamento, em `src/meson/meson.build`.

Consumir passa a ser uma linha, e — o detalhe que importa — com *exatamente* a mesma sintaxe de uma dependência externa: `dependencies: [saudacao_dep]`. É a inversão de dependência da Seção V.1 aplicada ao *build*: o consumidor não sabe se aquilo é uma biblioteca interna, um pacote do Conan ou um subprojeto baixado por *wrap*.

XI.2.5 Dependências, e o par que torna uma delas opcional

`dependency()` procura, nesta ordem: `pkg-config` (os `.pc` que o Conan gerou em `build/`, graças ao `-Dpkg_config_path`), os métodos nativos do sistema, e por fim o `fallback` — um subprojeto declarado em um `.wrap`.

Armadilha — sem `required: false`, um programa ausente aborta o projeto inteiro

```
doxygen_prog = find_program('doxygen', required: false)
if doxygen_prog.found()
    custom_target('docs', ...)
else
    warning('doxygen não encontrado — target ‘docs‘ desabilitado.')
endif
```

É exatamente o que `src/doxygen/meson.build` faz. Sem o `required: false`, a máquina que não tiver Doxygen instalado não configura *nada* — nem os 98 binários que não têm relação com documentação.

E vale a mesma lição em outra chave: o objeto do compilador responde a perguntas — `cpp.get_id()`, `cpp.has_header('sys/mman.h')`, `cpp.compiles(...)`. Escrever um *build* portátil é testar o ambiente, não adivinhar.

XI.2.6 Do *build* para dentro do C++

Duas funções atravessam a fronteira, uma em cada tempo (Tabela 121).

Tabela 121: As duas formas de gerar arquivo.

	`configure_file()`	`custom_target()`
quando roda	configuração	*build*
o que faz	substitui `@VAR@` num *template*	executa um comando
reexecuta	ao reconfigurar	quando a entrada muda
uso típico	versão, opções, *flags* no cabeçalho	`protoc`, gerador em Python

```
tabela_gerada_h = custom_target('tabela_gerada',
    input:   'tabela.txt',
    output:  'tabela_gerada.hpp',
    command: [python, files('gerar_tabela.py'), '@INPUT@', '@OUTPUT@'],
)
```

Listagem 337: Um gerador rodando no *build*, em `src/meson/meson.build`.

Regra — passe o `custom_target` na lista de *fontes* de quem o consome

```
demo_exe = executable('meson_demo',
    files('main.cpp'),
    build_info_h,
    tabela_gerada_h,
    dependencies: [saudacao_dep],
    ...
)
```

Os dois cabeçalhos gerados não estão ali para serem compilados: estão para o Ninja saber que precisa gerá-los *antes* do `main.cpp`. Sem isso o *build* funciona por acaso — e falha no primeiro *build* limpo, com "arquivo não encontrado". É o sintoma clássico de dependência não declarada, e o subprojeto o descreve como "erro só no primeiro *build*, some no segundo".

Armadilha — o `configure_file` não sabe o que é comentário

Um `@NOME@` escrito dentro de um comentário do *template* também é substituído — e, se `NOME` não existir na configuração, o *setup* emite `WARNING: The variable(s) 'NOME' ... are not present`. Por isso o *template* do subprojeto escreve `<at>NOME<at>` quando precisa documentar a própria sintaxe.

XI.2.7 Testes, e a dívida que foi quitada

`test()` só precisa de um comando que devolva zero em caso de sucesso — GoogleTest é conveniência, não requisito. O executável de teste não é instalado: existe apenas para o `meson test`, que aceita filtrar por nome, por suíte e repetir para caçar instabilidade (`--repeat 10`).

Isso importa porque durante muito tempo existiu na raiz do repositório um diretório `tests/` cujo `meson.build` estava **inteiramente comentado** — ele nunca construiu nada, e figurava como dívida no

`ROADMAP`. Esse diretório foi **removido**, e com ele a dívida: os testes passaram a morar nos subprojetos que os ensinam e os exercitam de verdade (Tabela 122).

Tabela 122: Onde os testes moram hoje. Fonte: o comentário de `meson.build` da raiz e `src/testing/README.md`.

Onde	O quê	Como rodar
`src/testing/`	5 alvos: fixtures, `TEST_P`, gmock, *death tests*	`meson test -C build`
`src/meson/`	1 alvo: o `test()` do próprio Meson	`meson test -C build`
`src/cmake/`	CTest, 11 casos, *build* próprio	`ctest --test-dir build`

O total sob o Meson é de **6 alvos e 41 casos**, todos passando.

Por que — por que apagar um diretório é melhor que deixá-lo comentado

Um `tests/` que não constrói nada é pior que a ausência de testes: ele *parece* cobertura. Quem chega ao repositório vê a pasta, conclui que existe uma suíte e não procura mais; quem já está nele vê a linha comentada e adia. E, pela lição da Seção XI.3, o conteúdo comentado apodreceu de qualquer forma — ele referenciava alvos que já não existiam. Apagar foi mais honesto e mais barato do que ressuscitar, e a cobertura real foi reconstruída onde ela também ensina alguma coisa.

XI.2.8 As armadilhas, e o diagnóstico de cada uma

Tabela 123: Sintoma, causa e correção — a tabela mais útil do subprojeto.

Sintoma	Causa	Solução
`Can't invoke target 'x': target not found`	alvo novo, *build dir* antigo	`meson setup --reconfigure build .`
`Dependency "fmt" not found`	`.pc` fora do caminho	`source build/conanbuild.sh`
`fatal error: gerado.hpp: No such file`	`custom_target` fora das fontes	passe o alvo na lista de fontes
`WARNING: The variable(s) 'X' ... not present`	`@X@` no *template*	defina `X` ou não escreva o literal
flag nova não fez efeito	`cpp_args` é por alvo	`default_options` na raiz
build quebra depois de `make clean`	dependência não declarada	declare a ordem

Regra — o *build dir* é descartável

Ele nunca vai para o Git, e ter vários ao mesmo tempo — `build`, `build-asan`, `build-release` — é o modo normal de trabalhar em Meson: eles não se atrapalham. Se algo ficou inexplicável, `rm -rf build && make configure` resolve, e não perde nada. Um sistema de *build* em que apagar o diretório de saída é arriscado tem um problema mais grave que o sintoma investigado.

Para inspecionar em vez de adivinhar, quatro comandos: `meson introspect build --targets` (todos os alvos, em JSON), `meson configure build` (todas as opções e valores atuais), `ninja -C build -t commands <alvo>` (a linha de compilação exata), e `build/meson-logs/meson-log.txt` (o log completo da configuração).

XI.3 Um alvo que não é construído é um alvo que apodrece

Por que — por que não basta comentar o `subdir()` do que "ainda não compila"

Comentar uma linha de *build* parece a decisão barata: o alvo sai do caminho, o *build* volta a ficar verde, e o problema fica "para depois". O que essa decisão realmente faz é **desligar o único verificador automático que aquele código tinha**. Um arquivo `.cpp` que ninguém compila não é código dormindo — é código que continua a ser lido, copiado e editado por humanos sem que nada verifique nenhuma das edições. Ele apodrece em silêncio, e o apodrecimento só é descoberto no dia em que alguém precisa dele, que é sempre o pior dia possível.

Este repositório pagou essa conta e resolveu documentá-la, porque o diagnóstico é bom material didático. Ao habilitar os `subdir()` que estavam comentados, seis defeitos apareceram no minuto seguinte — e nenhum deles era novo: todos estavam lá havia meses.

XI.3.1 Quatro `#include` que faltavam

O primeiro grupo é o mais banal, e por isso o mais instrutivo (Tabela 124). Cabeçalho faltando não é erro de lógica: é o tipo de defeito que *qualquer* compilação pega, e que *nenhuma* revisão humana pega de forma confiável.

Tabela 124: Os quatro cabeçalhos ausentes, todos em alvos que não eram compilados.

Arquivo	Faltava
`src/solid/srp/demo_pipeline.hpp`	`#include <algorithm>`
`src/concepts/demo_constraints.hpp`	`#include <array>`
`src/file_io/demo_sstream.hpp`	`#include <cmath>`
`src/aws/s3/main.cpp`	`#include <fmt/core.h>`

Armadilha — o *include* transitivo funciona até parar de funcionar

Nenhum desses quatro arquivos estava "errado" quando foi escrito: cada um funcionava porque *outro* cabeçalho, incluído antes, arrastava o que faltava. Isso é sorte, não contrato — a implementação da biblioteca padrão pode reorganizar seus *includes* entre duas versões do compilador, e o código que dependia do arraste quebra sem que nenhuma linha dele tenha mudado. A regra é a *Include What You Use*: cada arquivo inclui o que ele mesmo usa, mesmo que já esteja lá por acidente.

XI.3.2 Dois bugs de ABI, que só aparecem ao *executar*

O segundo grupo é de outra natureza. Esses não compilavam errado — compilavam perfeitamente, e quebravam na execução, em lugares que não têm relação nenhuma com a causa.

O alinhamento de pilha. Em `src/asm_course/07_calling_asm.s`, um `subq $8, %rsp` extra quebrava o alinhamento de 16 bytes que o System V ABI de x86-64 exige no momento do `CALL` (Seção IX.3). O sintoma foi um `SIGSEGV` *dentro do `fmt`*, quatro quadros de pilha acima do código culpado.

Regra — quem quebra o alinhamento não é quem estoura

Essa distância entre causa e sintoma é a assinatura do bug de alinhamento, e vale memorizá-la. A ABI exige que `%rsp` seja múltiplo de 16 na entrada de uma função porque as instruções SSE alinhadas (`movaps` e parentes) *faltam* num endereço desalinhado. Quem escreve assembly com aritmética inteira nunca usa SSE, e por isso nunca sente o problema; a falha estoura na primeira função da cadeia que use SSE — tipicamente uma biblioteca de terceiros, compilada com otimização, que vetorizou uma cópia de *string*. Ao ver um estouro dentro de uma biblioteca madura e testada por milhões de pessoas, a hipótese certa é "eu quebrei a ABI antes de chamá-la", não "achei um bug no `fmt`".

A promessa falsa ao compilador. Em `src/asm_course/10_string_ops.cpp`, os registradores `RDI`, `RSI` e `RCX` estavam declarados como *entrada pura* de um `asm` *inline* — e `rep movsb` modifica os três. O GCC acreditou na declaração, considerou que os valores continuavam válidos depois do bloco e reusou um registrador já corrompido.

Armadilha — o `asm` *inline* é um contrato, e o compilador confia nele

Uma restrição de entrada (`"D"(dst)`) promete que o bloco *lê* e não *escreve*. O compilador usa essa promessa para decidir o que precisa recarregar depois — e se a promessa for falsa ele não tem como descobrir. A correção é declarar o que de fato acontece: saída (`"+D"`), ou a lista `clobber`. E o sintoma da promessa falsa é o pior possível: o código funciona sem otimização, porque aí o GCC recarrega tudo de qualquer jeito, e quebra com `-O2`.

XI.3.3 Três bugs de biblioteca externa

O terceiro grupo é o que mais se parece com o dia a dia (Tabela 125). Nenhum deles é sobre C++: são sobre o mundo se mexer enquanto o alvo estava desligado.

Tabela 125: Defeitos revelados ao habilitar os alvos "pesados".

Onde	O quê
`src/http-client/main.cpp`	`std::string(n, '–')`: o traço U+2500 tem **três** *bytes*, e entre aspas simples vira uma *multi-character constant* truncada. E `40 - title.size()` é aritmética **sem sinal**: com um título de 45 caracteres o resultado dá a volta e vira um número astronômico (Seção VII.3).
`src/rabbitmq/`	a API do AMQP-CPP mudou: os *callbacks* de ciclo de vida passaram a ser métodos virtuais do **handler**, não do `TcpConnection` (Seção VII.5).
`src/database/main.cpp`	o demo não era idempotente — a segunda execução violava uma *constraint* `UNIQUE` e abortava (Seção VII.4).

Armadilha — o demo que só funciona uma vez

O caso do `src/database/` é o mais fácil de reproduzir em qualquer projeto: um exemplo que insere linhas numa base persistente funciona na primeira execução e falha em todas as demais. Enquanto o alvo não é construído nem executado por ninguém, o defeito é invisível; no primeiro CI que roda o binário duas vezes, ele vira "falha intermitente". Um demo com efeito colateral precisa ser idempotente por construção — `DROP TABLE IF EXISTS`, `INSERT OR REPLACE` ou uma base temporária por execução.

Regra — se um subprojeto não pode ser construído, o problema é dele

Conserte-o ou apague-o — não o esconda atrás de um `#`. É exatamente o que o cabeçalho de `src/meson.build` diz por escrito, e a lista acima é a justificativa empírica: seis defeitos reais, zero deles detectáveis sem compilar, todos concentrados nos alvos que estavam desligados. Uma *build farm* verde que não constrói metade do repositório mede a metade errada.

XI.4 CMake: alvos, escopos e o *build* que o ecossistema fala

O subprojeto `src/cmake/` é o par do `src/meson/`: ali o `meson.build` *é* o material didático; aqui quem ensina é o `CMakeLists.txt`, com 419 linhas comentadas. Ele é o único subprojeto, junto com `src/modules/`, que **não é construído pelo Meson da raiz** — e isso é de propósito: ele tem sistema de *build* próprio porque o sistema de *build* *é* o assunto.

Por que — por que um repositório que já usa Meson precisa ensinar CMake

Porque escolher a ferramenta e conhecer a ferramenta são decisões diferentes. O Meson é mais agradável de escrever, e este repositório o escolheu; o CMake é o que praticamente toda biblioteca C++ publicada nos últimos vinte anos fala. Quem consome uma biblioteca precisa entender o que o `find_package` dela espera; quem *publica* uma biblioteca precisa exportar um `<Nome>Config.cmake`, senão ninguém consegue usá-la com uma linha. Não saber CMake não é uma preferência de estilo — é uma limitação de alcance.

XI.4.1 Antes de tudo: o que o CMake é

CMake, como o Meson, é um **gerador** (Seção XI.2): ele lê a descrição declarativa em `CMakeLists.txt` e *escreve* os arquivos que outra ferramenta vai executar — `Makefile`s para o `make`, um `build.ninja` para o Ninja, uma *solution* para o Visual Studio. Ele não chama o compilador. Daí as duas fases, com nomes diferentes dos do Meson mas com o mesmo significado (Tabela 126).

Tabela 126: As três fases do CMake, e o que existe em cada uma.

Fase	Comando	O que acontece
configuração	`cmake -S . -B build`	executa os `CMakeLists.txt`, resolve `find_package`
geração	(a mesma chamada)	avalia as *generator expressions*, escreve os arquivos
build	`cmake --build build`	`make`/`ninja` compila e liga

A fase de *geração*, que o Meson não tem separada, é a razão de existirem as *generator expressions* — e a explicação está logo adiante.

XI.4.2 Alvos: a única ideia do "CMake moderno"

Por que — o problema que as variáveis globais criam

No CMake anterior à versão 3.0, tudo era variável global de diretório: para acrescentar um `-D` a *um* alvo, mexia-se numa variável que *todos* os alvos daquele diretório e dos subdiretórios liam. Num projeto de vinte bibliotecas isso vira uma teia impossível de auditar: ninguém consegue responder "com quais *flags* este arquivo foi compilado?" sem executar o *build* e ler a linha de comando. E, quando dois alvos acabam compilados com definições diferentes e ligados no mesmo binário, o resultado é violação de ODR — o defeito silencioso descrito em `src/linkage/`.

"CMake moderno" significa uma coisa só: **propriedades vivem em alvos, não em variáveis globais**. Um *alvo* é uma biblioteca ou um executável com nome próprio, e cada alvo carrega consigo os seus *includes*, suas *flags*, suas definições e suas dependências. Todo o resto decorre daí (Tabela 127).

Tabela 127: A tradução, comando a comando. Fonte: `src/cmake/CMakeLists.txt`, seção 2.

CMake antigo (global)	CMake moderno (por alvo)
`include_directories(...)`	`target_include_directories(alvo ...)`
`add_definitions(-DX)`	`target_compile_definitions(alvo ...)`
`set(CMAKE_CXX_FLAGS "-Wall")`	`target_compile_options(alvo ...)`
`link_libraries(...)`	`target_link_libraries(alvo ...)`

XI.4.3 Os três escopos, e a pergunta que decide entre eles

Cada comando `target_*` exige um escopo, e é aqui que está a densidade conceitual do CMake. O escopo responde a uma pergunta de *propagação*: esta propriedade vale só para compilar este alvo, ou também para quem o *usa*? (Tabela 128)

Tabela 128: Os três escopos de propagação.

Escopo	vale para o alvo	vale para quem o usa
`PRIVATE`	sim	não
`INTERFACE`	não	sim
`PUBLIC`	sim	sim

Regra — a pergunta única: isto aparece no meu cabeçalho público?

- o cabeçalho público faz `#include <x.hpp>` → `PUBLIC`;
- só o `.cpp` usa → `PRIVATE`;
- biblioteca só de cabeçalho, que não compila nada → `INTERFACE`.

```
1  add_library(nucleo STATIC nucleo.cpp)
2
3  # '.' é PUBLIC porque nucleo.hpp aparece na API: quem usa precisa achá-lo.
4  target_include_directories(nucleo PUBLIC ${CMAKE_CURRENT_SOURCE_DIR})
5
6  # Avisos são política de COMO compilar ESTE alvo — não se impõem a quem
7  # apenas o consome.
8  target_compile_options(nucleo PRIVATE
9      $<$<CXX_COMPILER_ID:GNU,Clang>:-Wall;-Wextra;-Wpedantic>
10     $<$<CXX_COMPILER_ID:MSVC>:/W4>
11 )
```

```
12
13 add_library(academy::nucleo ALIAS nucleo)
```

Listagem 338: O alvo `nucleo`, em `src/cmake/CMakeLists.txt`: o cabeçalho é público, os avisos são privados.

Armadilha — marcar tudo como `PUBLIC` "por segurança" custa caro

É o erro mais comum, e ele tem dois preços. O primeiro é tempo: o projeto inteiro recompila quando qualquer detalhe interno da biblioteca muda, porque o detalhe interno virou parte da interface de compilação de todo mundo. O segundo é pior e é de *design*: uma dependência que só o `.cpp` usava passa a ser vista pelos consumidores, e algum deles vai acabar usando-a — e a partir daí você não pode mais trocá-la sem quebrar clientes. A dependência virou parte da sua API sem que ninguém tenha decidido isso.

Armadilha — caminho relativo em `target_include_directories`

Escrever `'.'` em vez de `${CMAKE_CURRENT_SOURCE_DIR}` parece equivalente e não é: caminhos relativos são resolvidos contra o diretório de quem **consome** o alvo, não de quem o declarou. Enquanto todo mundo estiver na mesma pasta ninguém percebe; no dia em que o alvo for consumido de outro diretório — ou exportado com `install(EXPORT)` — o *include* aponta para o lugar errado.

Regra — dê nomes com `::` aos seus alvos

`add_library(academy::nucleo ALIAS nucleo)` custa uma linha e compra duas coisas. Primeiro, o uso interno fica idêntico ao uso via `find_package`, de modo que mover a biblioteca para fora do projeto não muda nenhuma linha de quem a consome. Segundo, e mais importante: um nome *com* `::` que não existe é **erro de configuração**, e o CMake diz qual nome não achou; um nome *sem* `::` que não existe vira silenciosamente `-lnome_que_nao_existe` e só falha no *linker*, com uma mensagem muito pior.

XI.4.4 *Generator expressions*: o `if` que roda tarde demais

Uma *generator expression* é uma expressão da forma `$<...>` que **não** é avaliada na configuração, e sim na *geração* — a fase intermediária da Tabela 126.

```
1 $<$<CXX_COMPILER_ID:GNU,Clang>:-Wall;-Wextra>   # depende do compilador
2 $<$<CONFIG:Debug>:-DDEPURACAO=1>                # depende do buildtype
3 $<BUILD_INTERFACE:${CMAKE_CURRENT_SOURCE_DIR}/include>   # caminho no build
4 $<INSTALL_INTERFACE:include>                    # caminho instalado
```

Listagem 339: As quatro formas que resolvem quase tudo. Fonte: `src/cmake/README.md`, demo 1.

Por que — por que não simplesmente um `if()`

Porque com geradores *multi-config* — Visual Studio, Xcode, Ninja Multi-Config — o *buildtype* não é escolhido na configuração: é escolhido na hora de compilar, com `cmake --build build --config Release`. Nesse momento o `if(CMAKE_BUILD_TYPE STREQUAL "Debug")` já rodou há muito tempo, e rodou uma vez só, para todos os *buildtypes* de uma vez. `$<CONFIG:Debug>` sobrevive até lá porque é escrito *no arquivo gerado*, não resolvido antes dele. A mesma diferença vale para o par `BUILD_INTERFACE`/`INSTALL_INTERFACE`: o mesmo alvo precisa anunciar um caminho de *include* enquanto está na árvore de *build* e outro depois de instalado, e um `if` não consegue dizer as duas coisas.

Armadilha — quatro erros de CMake que não parecem erros

- `file(GLOB)` é avaliado na **configuração**. Um arquivo novo não é notado até você reconfigurar à mão, e o *build* "funciona" sem ele — o que produz um *link* com símbolo faltando, ou pior, um binário sem o teste novo. **Liste os arquivos.**
- `target_link_libraries(a foo)` **sem escopo** ainda compila: é a forma antiga, mantida por compatibilidade, e ela mistura os dois modelos de propagação no mesmo projeto. Diga sempre `PRIVATE`, `PUBLIC` ou `INTERFACE`.
- **sem** `CMAKE_BUILD_TYPE`, o *build* sai *sem* otimização e *sem* `-g`: nem *debug*, nem *release*. É a pegadinha número 1 para quem vem do Meson, cujo padrão é `debug`. O `CMakeLists.txt` deste subprojeto assume `Release` quando nada é informado, e imprime que assumiu.
- o cache é **persistente**. Mudar uma opção que só é lida na primeira configuração — o *toolchain file*, à frente — não tem efeito nenhum enquanto o diretório de *build* existir.

XI.4.5 `find_package`: os dois modos

`find_package(Nome)` é a forma canônica de achar uma dependência, e ela tem **dois modos completamente distintos** que compartilham o mesmo nome — confusão frequente de quem lê uma mensagem de erro do CMake pela primeira vez.

Modo `CONFIG` (preferido) procura um arquivo `<Nome>Config.cmake` que a **própria biblioteca** instalou junto com os seus binários. A biblioteca descreve a si mesma: versão, alvos importados, dependências transitivas. Quem escreveu o arquivo sabe do que a biblioteca precisa.

Modo `MODULE` (legado) procura um `Find<Nome>.cmake`, um *script* que sai caçando a biblioteca com `find_library` e `find_path`. É necessário para bibliotecas antigas, que nunca instalaram um `Config`; o próprio CMake traz dezenas prontos — `FindThreads`, `FindMPI`, `FindOpenMP`. Quem escreveu o arquivo *adivinha* do que a biblioteca precisa.

XI.4.6 A detecção graciosa, e a máquina de quem não tem GPU

Por que — o projeto que não configura na máquina do outro

`find_package(MPI)` sem mais nada **aborta a configuração inteira** quando não acha o MPI. O efeito prático: quem clonar o projeto numa máquina sem `libopenmpi-dev` não consegue construir *nada* — nem os alvos que não têm relação alguma com MPI. Isso não é uma limitação do ambiente; é um bug de *build*, e é a razão de `src/cmake/CMakeLists.txt` usar o mesmo idioma em todas as dependências opcionais.

```
1 find_package(Threads REQUIRED)          # REQUIRED: aborta se faltar
2 find_package(MPI QUIET COMPONENTS CXX)  # QUIET + if: opcional
3
4 if(MPI_CXX_FOUND)
5     add_executable(mpi_anel mpi_anel.cpp)
6     target_link_libraries(mpi_anel PRIVATE MPI::MPI_CXX)
7 endif()
```

Listagem 340: O idioma da dependência opcional, em `src/cmake/CMakeLists.txt`, seção 3.

CUDA é um caso à parte, e vale saber por quê: para o CMake, CUDA não é um *pacote* — é uma **linguagem**, como C ou C++. Perguntar por ela tem um comando próprio.

```
include(CheckLanguage)
check_language(CUDA)                    # pergunta, não exige
if(CMAKE_CUDA_COMPILER)
    enable_language(CUDA)
    set(CMAKE_CUDA_ARCHITECTURES native)    # detecta a GPU local (3.24+)
    add_executable(cuda_saxpy cuda_saxpy.cu)
endif()
```

Listagem 341: `check_language` pergunta sem abortar. Resumo de `src/cmake/CMakeLists.txt`, seção 3.

Armadilha — `enable_language(CUDA)` direto aborta a configuração

Sem o `check_language` antes, qualquer máquina sem `nvcc` deixa de configurar o projeto inteiro. E `CMAKE_CUDA_ARCHITECTURES` tem uma armadilha própria: sem defini-lo, o `nvcc` compila para a arquitetura padrão *dele*, que pode não ser a da GPU presente — o binário compila, liga, roda e falha na hora de lançar o *kernel*.

O resultado dessa disciplina é que a configuração termina com um resumo impresso que **diz o que achou e o que não achou** — e, quando não achou, diz como resolver:

```
--   Protobuf          : TRUE
--   GoogleTest        : TRUE
--   OpenMP            : TRUE
--   MPI               : FALSE
--   CUDA              : FALSE
--       para MPI : sudo apt install libopenmpi-dev
--       para CUDA : sudo apt install nvidia-cuda-toolkit
```

Listagem 342: Saída real de `cmake -S . -B build` nesta máquina, copiada do `README` de `src/cmake/`.

Aviso

O `README` do subprojeto separa, numa tabela, o que foi **verificado** do que é **referência**: CMake 3.24 com Unix Makefiles, Protobuf, GoogleTest e OpenMP foram compilados e executados aqui; `mpi_anel.cpp` e `cuda_saxpy.cu` estão completos e comentados, mas **não foram compilados** — não há MPI nesta máquina, e há GPU e *driver* mas não há `nvcc`. Um livro que mede é um livro que também declara o que não mediu.

XI.4.7 `FetchContent` contra `find_package`

`FetchContent` é a terceira forma de obter uma dependência: em vez de procurá-la instalada, o CMake **baixa o código-fonte** dela e a compila junto com o seu projeto, como se fosse um subdiretório seu. O trade-off é real nos dois sentidos (Tabela 129).

Tabela 129: As duas formas de obter uma dependência. Fonte: `src/cmake/README.md`, demo 2.

Aspecto	`FetchContent`	`find_package`
funciona sem instalar nada	**sim**	não
build reprodutível	**sim**	depende da máquina
precisa de rede	sim	**não**
tempo do primeiro *build*	alto	**baixo**
compila com os seus *flags*	sim	não

A última linha corta nos dois sentidos, e é o ponto que costuma passar despercebido: compilar a dependência com os *seus flags* garante compatibilidade de ABI — o assunto de `src/linkage/` — e ao mesmo tempo triplica o tempo de *build*, porque agora você compila também tudo o que antes vinha pronto.

Armadilha — `GIT_TAG main` faz o seu *build* depender do dia

Apontar `FetchContent` para um *branch* significa que dois *builds* da mesma revisão do *seu* código podem produzir binários diferentes, porque outra pessoa fez *commit* no meio. **Tag ou *hash*, sempre.** E prefira tentar `find_package` primeiro: o CMake 3.24 tem `FIND_PACKAGE_ARGS` exatamente para isso — usa o instalado se existir, baixa se não existir.

XI.4.8 Conan e vcpkg: eles não substituem o CMake, alimentam-no

Conan e vcpkg são **gerenciadores de pacotes**: baixam, compilam e armazenam dependências. É comum imaginá-los como alternativas ao CMake, e não são — os dois terminam o trabalho **gerando os arquivos que o `find_package` consome**. O CMake continua sendo quem procura; o gerenciador só garante que haja o que achar. É a mesma relação que o Conan tem com o Meson neste repositório, em que ele gera os `.pc` do `pkg-config` (Seção XI.2).

Armadilha — o *toolchain file* só vale na primeira configuração

`-DCMAKE_TOOLCHAIN_FILE=...` é lido uma única vez, quando o cache é criado. Passá-lo de novo numa árvore de *build* já configurada não tem efeito *nenhum* — e o CMake não avisa. O sintoma é "mudei o *toolchain* e nada aconteceu". A solução é apagar o diretório de *build*; a prevenção é tratá-lo como descartável, exatamente como o *build dir* do Meson.

A conexão com este repositório é literal: o `README` de `src/cmake/` configura apontando `CMAKE_PREFIX_PATH` para os pacotes que o Conan da raiz já baixou, extraindo o prefixo dos próprios `.pc` com `sed`. Um gerenciador, dois sistemas de *build*, o mesmo cache de dependências.

XI.4.9 Código gerado durante o *build*

Por que — o arquivo que precisa virar C++ antes de o compilador rodar

`medida.proto` não é C++. Para que o programa o use, ele precisa virar `medida.pb.cc` e `medida.pb.h` *antes* da compilação; precisa ser regerado *se* o `.proto` mudar; e precisa **não** ser regerado se ele não mudar, senão todo *build* incremental vira um *build* completo. Escrever isso como um *script* que roda "antes de compilar" resolve o primeiro requisito e falha nos outros dois. O assunto aqui não é Protobuf: é o padrão, e ele vale igual para Qt (`moc`/`uic`), FlatBuffers, gRPC, Flex/Bison e geradores caseiros.

```
add_custom_command(
    OUTPUT ${PROTO_CC} ${PROTO_H}
    COMMAND protobuf::protoc --cpp_out=${PROTO_GERADO_DIR}
            -I${CMAKE_CURRENT_SOURCE_DIR} ${PROTO_FONTE}
    DEPENDS ${PROTO_FONTE}          # <- a dependencia declarada
    COMMENT "protoc: medida.proto -> medida.pb.{cc,h}"
    VERBATIM)

add_library(medida_proto STATIC ${PROTO_CC})
target_include_directories(medida_proto PUBLIC ${PROTO_GERADO_DIR})
target_link_libraries(medida_proto PUBLIC protobuf::libprotobuf)
target_compile_options(medida_proto PRIVATE $<$<CXX_COMPILER_ID:GNU,Clang>:-w>)
```

Listagem 343: A regra, em `src/cmake/CMakeLists.txt`, seção 4.

O que essas linhas declaram é uma frase: *"estas saídas vêm destas entradas, por este comando"*. A partir daí o `make` ou o Ninja trata os `.pb.cc` como qualquer outro arquivo com dependência — tocou no `.proto`, regera; não tocou, não regera. É a mesma ideia do `custom_target` do Meson (Tabela 121), com outro nome.

Armadilha — a confusão clássica: `OUTPUT` contra `TARGET/POST_BUILD`

`add_custom_command` tem **duas formas**, com o mesmo nome e propósitos opostos:

- `add_custom_command(OUTPUT ...)` declara uma **regra**: arquivos de saída, arquivos de entrada, comando. O grafo de dependências passa a conhecê-la.
- `add_custom_command(TARGET x POST_BUILD ...)` pendura um comando **depois** da construção do alvo `x`. Ele roda *sempre* que `x` é reconstruído e *nunca* quando não é — não rastreia dependência nenhuma. Serve para copiar um arquivo para o lado do executável ou assinar um binário.

Usar a segunda para gerar código produz um *build* que regenera tudo toda vez, ou que não regenera nunca — dependendo da ordem em que os alvos forem construídos. E, como os dois casos "funcionam" num *build* limpo, o defeito só aparece depois, como "preciso apagar o `build/` de vez em quando".

Armadilha — três detalhes que não são opcionais

- `VERBATIM`: sem ele, um caminho com espaço vira dois argumentos. O erro só aparece na máquina de quem tem espaço no caminho do projeto — historicamente, todo mundo no Windows.
- **a saída vai para o diretório de *build***, nunca para o de fonte. Gerar dentro do fonte suja o `git status`, quebra *builds* paralelos e cria conflito entre *Debug* e *Release*, que passam a disputar o mesmo arquivo.
- **código gerado não é código seu**: ele dispara avisos que você não pode corrigir. Suprima-os *no alvo dele* (`target_compile_options(medida_proto PRIVATE -w)`), nunca globalmente — senão você silencia também os seus.

O Protobuf em si, medido. O mesmo objeto, nas duas representações que a biblioteca oferece:

```
serializado   :   66 bytes
texto (debug): 210 bytes
```

Listagem 344: Saída de `./build/cmake_guia` nesta máquina.

O binário é compacto porque cada campo é identificado por um **número** — o `= 1`, `= 2` do `.proto` — e não pelo nome, e porque campos com valor padrão simplesmente não são escritos. É essa numeração que dá a compatibilidade entre versões: um campo novo com número novo não quebra quem lê com o *schema* antigo, porque o leitor antigo ignora números que não conhece.

Armadilha — reutilizar um número já usado é o pecado capital do Protobuf

Se o campo 3 era um `int32 id` e virou uma `string nome`, todos os dados gravados antes passam a ser lidos como o campo errado — sem erro, sem aviso, com valores absurdos. A defesa é declarar `reserved 3, 7;` ao remover um campo, e o compilador passa a recusar quem tentar reusar o número. Segunda armadilha da mesma família: uma versão do `protoc` diferente da `libprotobuf` produz um erro críptico (*"This file was generated by a newer version of protoc"*); a defesa é usar o alvo importado `protobuf::protoc` em vez de um `protoc` qualquer do `PATH` — que é o que o `CMakeLists.txt` faz.

XI.4.10 OpenMP, MPI e CUDA: a mesma conta, quatro caminhos

O subprojeto calcula a mesma operação — `saxpy`, ou $y \leftarrow a \cdot x + y$, o nível 1 do BLAS — em serial, com OpenMP, com MPI e com CUDA, e cada versão compara o resultado contra a referência serial. É o que torna a comparação honesta (Tabela 130).

Tabela 130: Os três modelos de paralelismo, lado a lado. Fonte: `src/cmake/README.md`, demo 4.

	OpenMP	**MPI**	**CUDA**
unidade	*thread*	**processo**	*thread* de GPU
quantidade	dezenas	dezenas a milhares	dezenas de milhares
memória	compartilhada	privada	separada (VRAM)
comunicação	variáveis	mensagens	cópias *host*/*device*
escala até	1 máquina	um *cluster*	1 GPU (ou N)
como se lança	`./prog`	`mpirun -np 4 ./prog`	`./prog`
risco típico	*data race*	**deadlock**	erro silencioso
no CMake	`OpenMP::OpenMP_CXX`	`MPI::MPI_CXX`	linguagem `CUDA`

Eles se **combinam**, e é o padrão em computação de alto desempenho: MPI entre os nós, OpenMP dentro de cada nó, CUDA nas GPUs de cada nó — o modelo conhecido como "MPI+X".

O resultado medido, que contraria a expectativa.

```
 elementos |        MB |   serial |    OpenMP | speedup
-----------|-----------|----------|-----------|--------
    250000 |         3 |     0.06 |      0.02 |   2.60x
   2000000 |        23 |     0.52 |      0.17 |   3.09x
  20000000 |       229 |     7.01 |      5.50 |   1.27x
  60000000 |       687 |    22.10 |     18.49 |   1.20x
```

Listagem 345: Saída de `./build/omp_saxpy`, mediana de 7 execuções, 24 *threads* disponíveis. Tempos em milissegundos.

A expectativa ingênua é que o *speedup cresça* com o tamanho do problema, porque o custo fixo de criar *threads* se dilui. Aconteceu o contrário: o ganho cai de 3,09× para 1,20× com o *mesmo* número de *threads* o tempo todo.

Por que — o SAXPY não é limitado por CPU, é limitado por banda de memória

A conta faz uma multiplicação-soma para cada 12 *bytes* movidos — lê `x[i]`, lê `y[i]`, escreve `y[i]`. Enquanto os vetores cabem no cache, as *threads* somam a banda dos *seus* caches, e o ganho é real. Quando os dados passam a vir da RAM, todas disputam os **mesmos** canais de memória — e acrescentar *thread* não acrescenta banda. O limite deixou de ser quantas contas por segundo o processador faz e passou a ser quantos *bytes* por segundo a memória entrega. É a mesma lição que `src/performance/` desenvolve com *cache misses*.

Aviso

Nota de método, e ela vale mais que o número. A primeira versão deste *benchmark* reportou *speedup* de **0,32**×: a versão "paralela" saiu três vezes *mais lenta*. Causa: sem aquecimento, a primeira região paralela paga a criação do *pool* de *threads*. A segunda versão reportou **1,03**× — dessa vez porque o *reset* do vetor, uma cópia de 80 MB, estava *dentro* da região cronometrada e dominava igualmente as duas medições. Só a terceira versão, com preparação fora do relógio e mediana de sete execuções, produziu a tabela acima. Três medições, três números diferentes, o mesmo código sendo medido. Quem publica a primeira conclui que OpenMP piora o programa.

As quatro diretivas que resolvem quase tudo.

```
#pragma omp parallel for                    // divide o laço entre threads
#pragma omp parallel for reduction(+ : soma)  // cada thread soma a sua parte
#pragma omp critical                        // seção exclusiva (cara)
#pragma omp atomic                          // uma operação só, mais barato
```

Listagem 346: O repertório mínimo de OpenMP.

Armadilha — a armadilha número 1: o código compila sem `-fopenmp` e roda serial

Sem a *flag*, os `#pragma` viram comentários e o programa continua compilando, ligando e produzindo o resultado certo — só que em uma *thread*. Um *build* mal configurado não falha: fica lento, em silêncio, e a conclusão que se tira é "OpenMP não ajudou neste caso". Por isso o `CMakeLists.txt` imprime se achou OpenMP e o `omp_saxpy.cpp` testa `#ifdef _OPENMP` em tempo de compilação.

Armadilha — `reduction` não é otimização, é correção

Sem ele, a soma paralela é um *data race* puro: todas as *threads* escrevendo na mesma variável. Compila, roda e dá um resultado *diferente a cada execução* — e isso é comportamento indefinido, não "erro de arredondamento". E há um efeito legítimo que se parece com bug: a soma paralela agrupa os termos em outra **ordem**, e a adição de ponto flutuante **não é associativa**. Um resultado bit a bit idêntico ao serial é coincidência, não contrato: ele muda com o número de *threads* e com o `schedule`. Testes que exigem igualdade exata falham por isso, e não por defeito.

Sobre `schedule`: use `static` quando toda iteração custa o mesmo — é o padrão, e é o certo no SAXPY — e `dynamic` quando o custo é irregular, como no conjunto de Mandelbrot (Seção VI.4), em que um pixel pode custar 2 iterações e o vizinho 1000. `dynamic` tem *overhead* por fatia distribuída; usá-lo num laço uniforme costuma **piorar** o tempo.

MPI, em cinco chamadas. O modelo é oposto ao do OpenMP: em vez de *threads* numa memória compartilhada, são **processos** com memórias privadas que só trocam informação por mensagem — e por isso o mesmo programa escala para além de uma máquina.

```
MPI_Init(&argc, &argv);                  // SEMPRE primeiro
MPI_Comm_rank(MPI_COMM_WORLD, &rank);    // quem sou eu (0..N-1)
MPI_Comm_size(MPI_COMM_WORLD, &n);       // quantos somos
// ... MPI_Send / MPI_Recv / MPI_Bcast / MPI_Reduce ...
MPI_Finalize();                          // SEMPRE último
```

Listagem 347: O esqueleto obrigatório de todo programa MPI.

Armadilha — as três formas de travar um programa MPI

- **coletiva parcial**: *todos* os processos precisam chamar `MPI_Bcast`, `MPI_Reduce` ou `MPI_Barrier`, com os mesmos argumentos de tamanho. Um `if (rank == 0)` em volta de uma coletiva trava o programa inteiro.
- **o *deadlock* ponto a ponto**: `MPI_Send` em todos, depois `MPI_Recv` em todos. Se a mensagem couber no *buffer* interno da implementação, funciona; quando ela cresce, todos bloqueiam. O bug aparece *só com dados grandes* — ou seja, em produção. Use `MPI_Sendrecv`, ou o par `MPI_Isend`/`MPI_Irecv` com `MPI_Wait`.
- **medir escalabilidade na própria máquina**: rodar `-np 1` e `-np 4` no mesmo computador não mede escalabilidade nenhuma — os quatro processos disputam os mesmos núcleos e a mesma banda de memória.

CUDA, e a guarda que não é opcional.

```
__global__ void saxpy(int n, float a, const float* x, float* y) {
    int i = blockIdx.x * blockDim.x + threadIdx.x;
    if (i < n) y[i] = a * x[i] + y[i];      // <- a GUARDA e obrigatoria
}
saxpy<<<blocos, 256>>>(n, 2.5f, x_dev, y_dev);
```

Listagem 348: O *kernel* de `src/cmake/cuda_saxpy.cu` (não compilado nesta máquina: não há `nvcc`).

Armadilha — três erros de CUDA que não dão erro

- **sem a guarda `if (i < n)`**: o número de *threads* é arredondado para cima até um múltiplo do tamanho do bloco, então as últimas têm índice *além* do vetor. Quem acusa é o `compute-sanitizer` — o ASan não vê memória de GPU.
- ***kernels* são assíncronos**: um erro dentro do *kernel* só se manifesta na próxima chamada sincronizante. Daí o par `cudaGetLastError()` (erro de lançamento) mais `cudaDeviceSynchronize()` (erro de execução). Ignorar o código de retorno é o erro nº 1 em CUDA: o *kernel* falha em silêncio e o resultado sai zerado.
- **a mentira comum do *benchmark* de GPU**: reportar só o tempo do *kernel*. Com as cópias *host*/*device* incluídas, o ganho encolhe ou desaparece. O SAXPY é o caso clássico em que a GPU **não** compensa, porque o gargalo é o PCIe; ela compensa quando os dados *ficam* nela entre vários *kernels*, ou quando a intensidade aritmética é alta — multiplicação de matrizes, convolução, FFT.

XI.4.11 CTest, e a diferença entre `1` teste e `11`

CTest é o executor de testes que vem com o CMake. Como o `test()` do Meson, ele não sabe nada sobre *frameworks*: um teste é um comando que devolve zero em caso de sucesso.

```
include(CTest)
enable_testing()                    # ANTES de qualquer add_test

add_executable(test_nucleo test_nucleo.cpp)
target_link_libraries(test_nucleo PRIVATE
    academy::nucleo GTest::gtest GTest::gtest_main)

include(GoogleTest)
gtest_discover_tests(test_nucleo)   # registra CADA CASO, um a um
```

Listagem 349: O registro dos testes, resumo de `src/cmake/CMakeLists.txt`, seção 8.

A diferença entre as duas formas de registrar é maior do que parece (Tabela 131).

Tabela 131: Registrar o binário contra registrar os casos.

Forma	Resultado
`add_test(NAME nucleo COMMAND test_nucleo)`	**1** teste no CTest; a falha diz apenas "nucleo falhou"
`gtest_discover_tests(test_nucleo)`	**N** testes; a falha nomeia o caso, e `ctest -R Saxpy` filtra

Verificado nesta máquina: os casos aparecem individualmente — `Saxpy.VetorVazioNaoQuebra`, `Gerar.EhDeterministico` e os demais —, somando **11 de 11 passando**.

Armadilha — `gtest_discover_tests` *executa* o binário durante o *build*

É assim que ele descobre os nomes dos casos — e é por isso que ele quebra em *cross-compilation*: o binário construído para a placa não roda na máquina que compila. Nesse cenário use `gtest_add_tests`, que lê o **código-fonte** em vez de executar, ou um `add_test` simples.

```
set_tests_properties(meu_teste PROPERTIES
    TIMEOUT     60          # o padrão é 1500 s: um teste travado prende o CI
    LABELS      "rapido"    # habilita 'ctest -L rapido'
    RUN_SERIAL  TRUE)       # para quem disputa porta, arquivo ou GPU
```

Listagem 350: As três propriedades de teste que mais importam.

`RUN_SERIAL` merece atenção porque a sua ausência não se parece com a sua ausência: sem ele, `ctest -j8` produz falhas intermitentes que *parecem* instabilidade do teste — e o tempo gasto caçando o "*flaky test*" é gasto no lugar errado. O assunto é o mesmo de `src/testing/`.

Armadilha — sem `--output-on-failure`, o CTest esconde a saída do teste que falhou

Ele mostra apenas "Failed" e o nome. É a *flag* mais importante da ferramenta e não é o padrão — ao contrário do `meson test`, que imprime a saída da falha sem que se peça (Seção XI.2). Vale decorar as cinco formas de invocação:

```
ctest --test-dir build --output-on-failure   # use SEMPRE
ctest --test-dir build -j8                   # em paralelo
ctest --test-dir build -R Saxpy              # filtra por nome
ctest --test-dir build --rerun-failed        # so os que falharam
ctest --test-dir build --repeat until-fail:100   # o cacador de flaky
```

Armadilha — o CMake não tem `-Db_sanitize`

No Meson, ligar o *AddressSanitizer* é uma opção de configuração (Seção I.9). No CMake é manual, e por alvo:

```
target_compile_options(alvo PRIVATE -fsanitize=address,undefined)
target_link_options(alvo    PRIVATE -fsanitize=address,undefined)
```

A *flag* precisa estar nas **duas** linhas. Esquecer a segunda produz `undefined reference to __asan_*` — uma mensagem de *linker* que não sugere em nada a causa real.

XI.4.12 CMake ou Meson? A comparação honesta

Este repositório tem os dois em produção, o que torna a comparação verificável em vez de opinativa (Tabela 132).

Regra — Meson é mais agradável; CMake é o que o ecossistema fala

As duas metades importam. A primeira justifica a escolha deste repositório para o seu próprio *build*: menos armadilhas por linha escrita, padrões sensatos, mensagens melhores. A segunda é a razão de o `src/cmake/` existir mesmo assim — se você publica uma biblioteca C++, exportar um `<Nome>Config.cmake` não é opcional: é o que permite que os outros a usem com uma linha em vez de com uma página de instruções.

Tabela 132: Fonte: `src/cmake/README.md`, seção final.

	CMake	**Meson**
linguagem	DSL própria, imperativa, sem tipos	DSL própria, mais restrita
curva de aprendizado	íngreme; 20 anos de camadas	suave
buildtype padrão	**nenhum** (sem `-O`, sem `-g`)	`debug`
sanitizers	manual, por alvo	`-Db_sanitize=address`
saída do teste que falha	exige `--output-on-failure`	**padrão**
resumo da configuração	manual, com `message()`	`summary()` embutido
adoção	**o padrão de facto**	crescente
suporte de IDE	universal	bom
CUDA, *cross*, *toolchains*	maduro	menos maduro

XI.5 Flex e Bison: quando o fonte de entrada não é C++

O `src/parser/` avalia expressões aritméticas com Flex e Bison, e a Seção X.3 já tratou do que ele ensina sobre linguagens — tokens, gramática, precedência declarada. Ele reaparece aqui por outro motivo: é o exemplo mais direto do repositório do padrão que a Seção XI.4 descreveu com o Protobuf. **Dois dos três fontes do subprojeto não são C++**, e precisam virar C++ antes de o compilador ser chamado.

Por que — por que isto é um problema de *build*, e não de código

Um *script* que roda `flex` e `bison` "antes de compilar" funciona — uma vez. Depois começam as perguntas que só o sistema de *build* consegue responder: se ninguém tocou no `lexer.l`, é preciso rodar o `flex` de novo? Se o `parser.y` mudou, quais objetos precisam recompilar? Como o compilador acha o `parser.tab.h`, que não existe no diretório de fonte e sim no de *build*? E como fazer tudo isso sem serializar o *build* inteiro atrás do gerador? Nenhuma dessas perguntas é sobre *parsing*.

XI.5.1 Duas ferramentas, três saídas, um executável

```
flex  = find_program('flex',  required: true)
bison = find_program('bison', required: true)

# Bison: gera parser.tab.c E parser.tab.h — duas saidas, um comando
bison_out = custom_target('parser_bison',
    input  : 'parser.y',
    output : ['parser.tab.c', 'parser.tab.h'],
    command : [bison, '-d', '-o', '@OUTPUT0@', '@INPUT@'],
)

# Flex: gera lex.yy.c
flex_out = custom_target('parser_flex',
    input  : 'lexer.l',
    output : 'lex.yy.c',
    command : [flex, '-o', '@OUTPUT@', '@INPUT@'],
)

executable('parser',
    sources : ['main.cpp', bison_out, flex_out],
    include_directories : include_directories('.'),
    implicit_include_directories : true,   # parser.tab.h vive no build dir
    install : true,
)
```

Listagem 351: `src/parser/meson.build`, na íntegra — o *pipeline* de geração completo em vinte linhas.

Três detalhes desse arquivo carregam quase toda a lição:

1. **`output` é uma lista.** O `bison -d` produz *dois* arquivos de uma vez — o `.c` com o *parser* e o `.h` com os *tokens* — e é preciso declarar os dois. Declarar só o `.c` produz um *build* que funciona por acidente: o `.h` existe no disco, mas o Ninja não sabe de onde ele veio, e portanto não o regenera quando a gramática muda. O `@OUTPUT0@` no comando é a forma de dizer "a primeira saída" — o `bison` deriva o nome do cabeçalho a partir dela.

2. **os alvos gerados entram na lista de `sources`.** É a mesma regra do Seção XI.2: eles não estão ali para serem compilados como fontes soltos, e sim para que o Ninja saiba que precisa gerá-los *antes* do `main.cpp`.

3. `implicit_include_directories: true` acrescenta o diretório de *build* correspondente ao caminho de busca de cabeçalhos. Sem ele, o `#include "parser.tab.h"` do `lexer.l` não acha nada — porque o arquivo não está, e nunca estará, ao lado do fonte.

Armadilha — o ciclo de dependência entre o *lexer* e o *parser*

O `lexer.l` inclui `parser.tab.h`, que é gerado pelo Bison; e o Bison, para gerar, não precisa de nada do Flex. A ordem certa é portanto *bison, depois flex, depois o compilador* — e ela não é óbvia lendo os dois arquivos, porque no fonte a dependência aparece invertida (o `parser.y` é quem chama `yylex()`). Declarar as saídas corretamente deixa o Ninja descobrir a ordem sozinho; tentar impô-la à mão é o caminho para o "erro só no primeiro *build*, some no segundo".

Armadilha — `required: true` num programa que nem todos têm

`find_program('flex', required: true)` aborta a configuração inteira em qualquer máquina sem `flex` instalado — exatamente o comportamento que o `src/doxygen/` evita com `required: false` (Seção XI.6) e que o `src/cmake/` evita com `QUIET` mais `if` (Seção XI.4). Aqui a escolha é deliberada, porque o subprojeto *não existe* sem as duas ferramentas: um alvo que não pode ser construído deve falhar dizendo o quê e como resolver (`sudo apt install flex bison`), e não ser silenciosamente omitido. A distinção entre as duas políticas é a pergunta: *este alvo é opcional, ou o projeto todo depende dele?*

Regra — o padrão vale para tudo que gera código

`protoc`, `moc` e `uic` do Qt, `flatc`, `grpc_cpp_plugin`, o gerador em Python de `src/meson/` e o `bison` desta seção fazem todos a mesma coisa: *destas entradas, por este comando, saem aquelas saídas.* A ferramenta muda, o formato de entrada muda, e a declaração é sempre a mesma — `custom_target` no Meson, `add_custom_command(OUTPUT ...)` no CMake. Quem entende uma, entende todas.

XI.6 Doxygen: documentação a partir do código

O subprojeto `src/doxygen/` cobre as convenções de comentário que geram documentação navegável [16]. O que ele ensina de mais durável, porém, não é a lista de comandos — é *o que* vale a pena documentar.

XI.6.1 Os comandos que descrevem o contrato

A maioria dos comandos é descritiva e prescindível: o nome da função costuma dizer o que `@brief` diria. Os que *não* são prescindíveis são os que descrevem o **contrato** — exatamente aquele que a Seção V.1 discutiu ao tratar de LSP:

```
/**
 * @brief Normaliza o vetor.
 * @pre   `length() > 0`
 * @post  `length() == 1.0`
 * @throws std::domain_error Se o vetor for nulo.
 * @note  Use `length_sq()` para comparações — evita `sqrt`.
 */
Vector2D normalized() const;
```

Listagem 352: Pré-condição, pós-condição e exceção: o que a assinatura não diz.

Regra — documente o que o tipo não consegue dizer

`@param[in]` num parâmetro `const T&` é redundante — o tipo já disse. `@pre` não é: a linguagem não tem como exigir que `length() > 0` antes da chamada. A regra prática: se o compilador já garante, não documente; se ele não garante e o chamador precisa saber, documente — e considere transformar isso num `assert` ou num tipo forte (Seção II.4), que é documentação executável.

XI.6.2 A configuração que muda o resultado

Duas opções do `Doxyfile` decidem se a documentação é útil ou ruído:

```
EXTRACT_ALL        = NO    # documenta apenas o que tem comentário Doxygen
WARN_IF_UNDOCUMENTED = YES
```

Listagem 353: As duas linhas que separam documentação de listagem.

Com `EXTRACT_ALL = YES`, o Doxygen lista *tudo*, documentado ou não — e o resultado é uma referência em que 90% das páginas não têm uma frase sequer, o que treina o leitor a não consultá-la. Com `NO`, o que aparece foi escrito de propósito.

A integração com o Meson é o `custom_target` da Seção XI.2, com `build_by_default: false` — documentação não precisa ser regerada a cada compilação — e o `find_program(required: false)` que a torna opcional.

XI.7 Sandbox

Houve, por algum tempo, *dois* lugares para código descartável: um `sandbox/` na raiz e o `src/sandbox/`. O da raiz foi **removido** — pela mesma razão do `tests/`: dois lugares para a mesma coisa significam que ninguém sabe qual usar, e o menos usado apodrece (Seção XI.3). Restou um, e ele é construído com todos os outros.

O `src/sandbox/` é o único subprojeto do repositório sem um assunto fixo: é espaço para testar uma *feature*, experimentar uma biblioteca nova, verificar um comportamento antes de comprometê-lo em outro lugar, ou rodar uma medição rápida. Atualmente ele demonstra `nlohmann/json` — a mesma biblioteca do Seção VII.3, aqui sem a camada de rede em volta.

Por que — por que ter um lugar oficial para código descartável

Sem ele, o experimento acontece dentro de outro subprojeto — e às vezes fica lá. O *sandbox* dá ao repositório uma resposta clara para a pergunta "onde eu ponho isto enquanto não sei se vai virar alguma coisa?", e uma regra implícita: o que estiver ali não tem compromisso de estabilidade nem de qualidade

didática. Se o experimento vingar, ele vira um subprojeto próprio — com `README`, entrada no `ROADMAP` e linha no `README` da raiz.

XI.8 Síntese: a receita completa para um subprojeto novo

Os subprojetos deste capítulo convergem para um procedimento, e vale registrá-lo por extenso porque é o passo que todo mundo erra — o terceiro:

1. Crie `src/novo_topico/` com `main.cpp` e `meson.build`. O `meson.build` é um `executable()` com `dependencies`, `install` e `install_dir` — e `override_options` se precisar de C++23.
2. Acrescente uma linha `subdir('novo_topico')` em `src/meson.build`, em ordem alfabética. **Não** no `meson.build` da raiz — e **não** comentada "até funcionar": se ainda não compila, ele não está pronto para entrar (Seção XI.3). A exceção é o subprojeto cujo assunto *é* outro sistema de *build*, como `src/cmake/` e `src/modules/`, que documentam a escolha nos próprios `README`.
3. **Reconfigure**: `source build/conanbuild.sh` seguido de `meson setup --reconfigure build ..` Sem este passo, o `meson compile` responde `target not found`, e a causa não é óbvia.
4. Escreva o `README.md` do subprojeto, e acrescente a linha correspondente no `src/ROADMAP.md` e no `README.md` da raiz.

O quarto passo é o que mantém este relatório possível: cada capítulo aqui foi derivado dos `README` dos subprojetos, e um subprojeto sem `README` é um subprojeto que ninguém encontra — inclusive quem o escreveu.

XII Tipos de vocabulário e computação em tempo de compilação

Este capítulo trata de duas ideias que parecem não ter relação: um punhado de tipos da biblioteca padrão — `optional`, `variant`, `any`, `tuple`, `span` — e um conjunto de palavras-chave que mandam o compilador calcular coisas antes de o programa existir — `constexpr`, `consteval`, `constinit`, `static_assert`.

A relação é esta: as duas movem informação para dentro do **tipo**, e tudo o que está no tipo é verificado por alguém que nunca se distrai. Um `optional<int>` diz, na assinatura, que pode não haver valor — e quem esquecer de testar recebe um erro em vez de ler lixo. Uma `Matriz<2,3>` diz, no tipo, quantas linhas e colunas tem — e quem multiplicar por uma `Matriz<2,2>` não compila. É a mesma estratégia em escalas diferentes: **fazer o compilador trabalhar no seu lugar**.

O capítulo percorre quatro subprojetos. `src/vocabulary_types/` apresenta os tipos que atravessam fronteiras de biblioteca (Seção XII.1); `src/compile_time/` mostra o que o compilador calcula sozinho (Seção XII.2); `src/cpp23/` *mede* o que o compilador de referência realmente implementa, em vez de supor (Seção XII.3); e `src/modules/` enfrenta a substituição do `#include` e documenta dois defeitos do GCC 13 encontrados na prática (Seção XII.4). Os dois primeiros assuntos apareceram resumidos no Seção III.4 e no Seção IV.3; aqui são desenvolvidos do zero.

XII.1 O vocabulário compartilhado

Dois termos, antes de tudo. Uma **unidade de tradução** (TU) é o que o compilador enxerga de cada vez: um `.cpp` depois de o pré-processador ter colado dentro dele, textualmente, todos os `#include`. É a unidade real de compilação — um arquivo-objeto por TU, e nenhuma sabe das outras. Uma **fronteira de biblioteca** é o ponto em que código escrito por uma pessoa passa um valor a código escrito por outra, sem que as duas possam combinar convenções.

Por que — o problema: convenções que ninguém verifica

Antes desses tipos, cada biblioteca inventava a sua forma de dizer as mesmas quatro ou cinco coisas, e todas tinham o mesmo defeito: nada no tipo obrigava o outro lado a respeitar a convenção. Para "pode não haver valor" usava-se um ponteiro nulo, um índice -1 ou um `bool` de saída — e nada exigia a verificação. Para "é um destes", uma `union` com etiqueta ao lado — e nada impedia ler o membro errado. Para "é qualquer coisa", `void*` — que perde o tipo por completo. Para "é uma sequência contígua", o par (ponteiro, tamanho) — dois parâmetros que precisam concordar e que nada obriga a concordar. O ganho dos tipos de vocabulário não é elegância: é que o erro passa a ser detectável.

Cada um dos cinco responde a uma pergunta diferente sobre "que valor está aqui?" (Tabela 133). A coluna da direita organiza a seção: ela diz *quando* o conjunto de tipos possíveis é conhecido.

Tabela 133: As cinco perguntas do vocabulário compartilhado.

Tipo	Responde	Tipos conhecidos em
`optional<T>`	*pode não haver valor*	compilação, 1 tipo
`variant<Ts...>`	*é um destes*	compilação, N tipos
`any`	*é qualquer coisa*	**execução**
`tuple<Ts...>`	*são todos estes*	compilação, N tipos
`span<T>`	*é uma janela sobre aqueles*	compilação, 1 tipo

O subprojeto compila com C++23, e não com o C++20 da raiz, porque as operações monádicas de `optional` só chegaram nesse padrão (Seção I.6).

XII.1.1 `std::optional`: a ausência dentro do tipo

Uma função que procura pode não achar, e a questão é como ela comunica isso. A solução tradicional escolhe um valor de retorno para significar "não achei": `nullptr`, -1, *string* vazia. Todas funcionam e todas têm o mesmo problema — o valor especial *cabe* no tipo de retorno, então quem chama pode usá-lo como resultado legítimo, e o compilador não tem o que reclamar.

`std::optional` põe a ausência **no tipo**. Um `std::optional<int>` não é um `int`: chegar ao `int` exige um passo explícito, e é nesse passo que a decisão sobre a ausência é tomada.

```
1 std::optional<int> buscar_idade(const std::string& nome) {
2     static const std::unordered_map<std::string, int> pessoas{
3         {"Ada", 36}, {"Grace", 45}, {"Alan", 41},
4     };
5     auto it = pessoas.find(nome);
6     if (it == pessoas.end()) return std::nullopt;    // ausência explícita
7     return it->second;                 // conversão implícita para optional
8 }
```

Listagem 354: A ausência declarada na assinatura, em `src/vocabulary_types/demo_optional.hpp`.

O `std::nullopt` é um objeto de tipo próprio que significa "vazio"; o `return it->second;` funciona porque há construtor implícito a partir de `T`, de modo que escrever o caso de sucesso não custa nada a mais.

As três formas de acessar diferem no que fazem quando não há valor (Tabela 134), e a primeira é a perigosa justamente porque não falha: falha em silêncio.

Tabela 134: Extraindo o valor de um `optional`.

Forma	Se estiver vazio	Quando usar
`*opt`	**UB (comportamento indefinido)**	já testou com `if`
`opt.value()`	lança `bad_optional_access`	quer rede de segurança
`opt.value_or(x)`	devolve `x`	tem um padrão óbvio

Armadilha — `*opt` vazio não é exceção nem *crash*

É UB (comportamento indefinido): o padrão não diz o que acontece, e o que acontece na prática é ler os bytes do *buffer* interno como se fossem um `T`. Isso costuma produzir um valor plausível, e o programa segue com um número inventado. Sem aviso, sem exceção, sem *segmentation fault* garantido. É **lixo silencioso**, a pior categoria de erro que existe.

O nome sugere ponteiro, e a sugestão é falsa: não há alocação. O `T` vive dentro do próprio `optional`, num *buffer* alinhado, ao lado de um `bool`. O subprojeto prova imprimindo os dois endereços:

```
1 sizeof(int)                 = 4
2 sizeof(optional<int>)       = 8      ← int + flag + padding
3 &optional  = 0x7ffe9c642e00
4 &*optional = 0x7ffe9c642e00          ← o mesmo endereço
```

Listagem 355: Saída de `src/vocabulary_types/`: o `T` vive dentro do `optional`.

Padding, nessa saída, são bytes de enchimento que o compilador insere para que cada membro caia num endereço múltiplo do seu alinhamento: 4 mais 1 dariam 5, mas o objeto precisa ter tamanho múltiplo de 4 para que arranjos dele fiquem alinhados.

As operações monádicas (C++23). Encadear operações que podem falhar gera, na forma ingênua, um `if` de propagação por passo. As três operações monádicas eliminam esses `if` — se o `optional` estiver vazio, a função nem é chamada, e a ausência propaga sozinha:

```
1  nome_do_id(id)                                  // optional<string>
2      .and_then(comprimento_se_curto)             // T → optional<U>
3      .transform([](int n) { return n * 100; })   // T → U
4      .or_else([] { return std::optional<int>{-1}; });
```

Listagem 356: O encadeamento monádico, em `src/vocabulary_types/demo_optional.hpp`.

A distinção entre `and_then` e `transform` é o retorno da função passada: `transform` devolve `U` e a biblioteca embrulha; `and_then` já devolve `optional<U>` e a biblioteca não embrulha de novo — é o que evita o `optional<optional<U>>`.

Por que — por que `optional<T&>` não existe

A pergunta que decide é: o que `opt = outro` deveria fazer com uma referência dentro? *Religar* a referência para outro objeto, ou *atribuir* ao objeto referido? As duas respostas são defensáveis, nenhuma é obviamente correta, e o comitê preferiu proibir a escolher. Na prática o C++ já tinha o "*optional reference*": é o ponteiro cru — um `const Config*` devolvido por uma busca significa exatamente "opcional e não-proprietário", com a desvantagem de não dizer isso no nome. Quando a assinatura precisa ser inequívoca, `optional<std::reference_wrapper<const T>>` compila, ao custo de um `.get()` no uso.

Armadilha — `optional<bool>` tem três estados e o `if` lê o errado

O `operator bool` responde "*tem valor?*", não "*o valor é verdadeiro?*". Com um `bool` dentro, as duas perguntas se confundem:

```
1  vazio    if(o) = false   value_or(false) = false
2  false    if(o) = true    value_or(false) = false   ← a armadilha
3  true     if(o) = true    value_or(false) = true
```

A linha do meio pega todo mundo: `if (o)` é verdadeiro porque *há* valor, ainda que o valor seja `false`. E `value_or(false)` responde `false` tanto para o vazio quanto para o `false` armazenado — apagando exatamente a distinção que motivou usar `optional`. A saída é modelar com três estados nomeados: `enum class Resposta { Sim, Nao, NaoInformado };`. Se o `optional<bool>` for inevitável, escreva sempre `if (o.has_value() && *o)`.

Regra — `optional` diz "pode não haver"; não diz "por quê"

Use como retorno de busca, *parse* ou consulta; como membro genuinamente opcional; e para inicialização tardia de membro sem construtor padrão. Não o use quando o **motivo** da falha importa — aí a ferramenta é `std::expected` (Seção II.8), que carrega o erro. E não use `optional<unique_ptr<T>>`: o ponteiro já tem estado nulo.

XII.1.2 `std::variant`: a soma de tipos que faltava

Uma `struct` é um *produto* de tipos: contém um `int` **e** um `double` **e** uma `string`. A operação complementar — **ou** um **ou** outro — chama-se soma de tipos, e o C++ só tinha uma versão insegura dela: a `union`, que guarda vários membros no mesmo espaço mas não guarda *qual* está ativo.

```
union: escrevi u.i = 42
  ler u.i → 42 ✓
  ler u.d → seria UNDEFINED BEHAVIOR (membro inativo)
  e NADA no código impede

variant: v = 42
  v.index()                    = 0  (0 = int)
  holds_alternative<double> = false
  std::get<double>(v)          → bad_variant_access
```

Listagem 357: Saída de `src/vocabulary_types/`: a `union` não sabe quem está ativo; o `variant` sabe.

`std::variant` guarda o mesmo espaço **mais** o índice do tipo ativo, e verifica todo acesso: paga um inteiro pequeno e devolve segurança de tipo. O que ele não paga é alocação — reserva inline o espaço do maior membro.

Das quatro formas de acessar (Tabela 135), uma se destaca. **Exaustividade** aqui significa que o compilador garante que todos os tipos possíveis foram tratados: `std::visit` exige um invocável que aceite *todos* os tipos, e se faltar um a chamada não compila. É a mesma propriedade que faz valer a pena um `switch` sobre `enum class` sem `default` (Seção II.1) — acrescentar um caso quebra o *build* nos lugares que precisam ser atualizados.

Tabela 135: Acessando um `variant`. Só `visit` verifica exaustividade.

Forma	Se o tipo estiver errado	Exaustividade
`get_if<T>(&v)`	devolve `nullptr`	não verifica
`get<T>(v)`	`bad_variant_access`	não verifica
`get<I>(v)`	`bad_variant_access`	não verifica
`visit(f, v)`	não acontece	**garantida**

O *overload idiom*. O obstáculo prático do `visit` é que ele quer *um* objeto invocável que aceite todos os tipos. Um *lambda* genérico `[](const auto& x)` atende, mas o corpo passa a ser o mesmo para todos — e o motivo de usar `variant` era tratar cada um de um jeito. O *overload idiom* são duas linhas de biblioteca que resolvem isso:

```
// Herda de todos os lambdas e traz o operator() de cada um para o escopo.
template <typename... Ts>
struct Overload : Ts... { using Ts::operator()...; };

// Deduction guide — em C++20 é opcional, mas explícito não custa.
template <typename... Ts> Overload(Ts...) -> Overload<Ts...>;
```

Listagem 358: O *overload idiom*, em `src/vocabulary_types/demo_variant.hpp`.

Cada pedaço é um recurso distinto. `typename... Ts` é um *parameter pack*: uma lista de tipos de tamanho arbitrário. `: Ts...` expande essa lista na cláusula de herança — a estrutura herda de *cada* tipo, e como cada *lambda* é uma classe com um `operator()`, herdar de N *lambdas* traz N funções de chamada. `using Ts::operator()...;` expande uma declaração `using` por tipo, o que é necessário porque, sem ela, os `operator()` herdados de bases diferentes não formam um conjunto de sobrecargas — ficam escondidos uns pelos outros. Feito isso, a resolução de sobrecarga normal faz o despacho.

```cpp
using Valor = std::variant<int, double, std::string, bool>;

std::string descrever(const Valor& v) {
    return std::visit(Overload{
        [](int i)                { return fmt::format("inteiro {}", i); },
        [](double d)             { return fmt::format("real {:.2f}", d); },
        [](const std::string& s) { return fmt::format("texto '{}'", s); },
        [](bool b)               { return fmt::format("booleano {}", b); },
    }, v);
}
```

Listagem 359: Um tratamento por tipo, com exaustividade verificada.

Acrescentar um tipo a `Valor` sem acrescentar o *lambda* correspondente faz `descrever` deixar de compilar — é a exaustividade em ação. E `visit` aceita *vários* `variant` de uma vez, com o compilador gerando a matriz de combinações: o subprojeto usa pedra-papel-tesoura, com seis *lambdas* de dois parâmetros mais um `[](auto, auto)` que absorve os empates. Fazer o mesmo com herança é o padrão *Visitor* do GoF (Seção V.2) — duas hierarquias, um `accept()` por classe e despacho duplo escrito à mão. Aqui não há hierarquia, `accept()` nem vtable / vptr.

Armadilha — `valueless_by_exception`: o único estado inválido

Se a construção do novo valor lançar *durante* uma atribuição, o antigo já foi destruído e o novo não chegou a existir. O `variant` fica sem tipo ativo, e `index()` devolve `variant_npos`:

```
antes:  index=0 valueless=false
exceção durante a atribuição: falhei na cópia
depois: index=-1 valueless=true
```

Um `visit` sobre esse estado lança `bad_variant_access`. Evita-se com três hábitos: pôr um tipo trivial — ou `std::monostate` — como primeira alternativa; preferir `emplace` à atribuição; e marcar o construtor de movimento como `noexcept`. É a Rule of 5 do Seção II.3 cobrando de novo.

`std::monostate` é um tipo vazio cuja única função é servir de primeira alternativa, e ele resolve um problema concreto: um `variant` só tem construtor padrão se o *primeiro* tipo da lista tiver. `variant<SemDefault, int> v;` não compila; `variant<std::monostate, SemDefault, int> v;` compila e nasce no estado "nada ainda".

Um `variant` recursivo. Um `variant` não pode conter a si mesmo diretamente — o tamanho seria infinito, já que reserva espaço inline para o maior tipo. A saída é a indireção:

```cpp
using Expressao = std::variant<double,
                               std::unique_ptr<Soma>,
                               std::unique_ptr<Multiplicacao>>;

struct Soma          { Expressao esq, dir; };
struct Multiplicacao { Expressao esq, dir; };

double avaliar(const Expressao& e) {
    return std::visit(Overload{
        [](double d) { return d; },
        [](const std::unique_ptr<Soma>& s) {
            return avaliar(s->esq) + avaliar(s->dir);
        },
        [](const std::unique_ptr<Multiplicacao>& m) {
            return avaliar(m->esq) * avaliar(m->dir);
        },
    }, e);
}
```

Listagem 360: Uma árvore de expressão sem uma única função virtual, em `src/vocabulary_types/demo_variant.hpp`.

Nenhuma classe base, nenhum método virtual, nenhum `accept()`. A hierarquia é **fechada** — e é por ser fechada que o compilador consegue verificar a exaustividade.

O *expression problem*. A comparação com herança não tem vencedor absoluto, e isso tem nome: os dois eixos de extensão de um programa — acrescentar *tipos* e acrescentar *operações* — puxam em direções opostas, e a estrutura que facilita um dificulta o outro (Tabela 136). Com `variant`, uma operação nova é uma função nova e nenhum tipo existente muda; mas um tipo novo obriga a revisitar todos os `visit`. Com herança é o inverso: um tipo novo pode nascer em outra biblioteca, e uma operação nova obriga a mexer na base e em todas as derivadas. Escolha pelo eixo que vai variar mais.

Tabela 136: `variant` favorece operações novas; herança favorece tipos novos.

Critério	`variant`	**herança virtual**
tipos novos *externos*	não	**sim**
operações novas	**sim**	recompila tudo
exaustividade verificada	**sim**	não
alocação por nó	opcional	sempre

XII.1.3 `std::any` e o apagamento de tipo

`optional<T>` sabe o tipo; `variant<Ts...>` sabe o *conjunto* de tipos; `std::any` não sabe nada em compilação: aceita qualquer tipo copiável e só descobre qual é em execução.

O mecanismo chama-se *type erasure*: guardar, junto com o objeto, ponteiros para as funções que sabem operar sobre ele — copiar, destruir, informar o `typeid` — de modo que o código que manipula o `any` faça o necessário sem conhecer o tipo. É o princípio do vtable / vptr, com a diferença de que a tabela não vive dentro do objeto e o tipo armazenado não precisa herdar de nada. A recuperação é feita por `any_cast`, em duas formas: a que recebe o objeto e lança `bad_any_cast`, e a que recebe um *ponteiro* e devolve `nullptr`. O casamento de tipo é **exato** — um `any` com `double` não responde a `any_cast<int>`.

O custo, contado. Em vez de afirmar que `any` aloca, o subprojeto *substitui o `operator new` global* por uma versão que conta chamadas e bytes, e mede:

```
1  sizeof(std::any)                          = 16
2  any<Pequeno>  ( 4B) → 0 alocação(ões)
3  any<Medio>    (16B) → 1 alocação(ões), 16 bytes
4  any<Grande>   (64B) → 1 alocação(ões), 64 bytes
5  variant<...Grande>  → 0 alocação(ões)      ← nunca aloca
```

Listagem 361: Saída de `src/vocabulary_types/`: alocações contadas ao construir um objeto.

A primeira linha é a **otimização de objeto pequeno** (SBO): a implementação reserva alguns bytes dentro do próprio objeto e, se o valor couber, guarda-o ali sem *heap*. Na libstdc++ o limite do SBO do `any` é `sizeof(void*)` — oito bytes numa máquina de 64 bits. Acima disso, *cada* `any` construído é um `malloc` mais um `free`. A última linha é o contraste que importa: o `variant` **nunca** aloca. Essa diferença estrutural

pesa muito mais que a velocidade bruta de uma chamada isolada — num laço quente, o que se mede é a pressão sobre o alocador.

Regra — a pergunta que elimina o `any` quase sempre

Pergunte: *"eu consigo listar os tipos possíveis?"* **Sim** → `variant`, que ganha exaustividade e zero alocação. **Não, mas todos fazem a mesma coisa** → interface virtual: os tipos são abertos e o que os une é o comportamento. **Não, e cada um faz coisa diferente** → aí sim, `any`. O sintoma de `any` mal usado é uma cascata de `any_cast` encadeados — e essa cascata *é* um `variant`, escrito à mão e sem exaustividade.

Onde `any` é a resposta certa: quando o conjunto de tipos é aberto **por projeto**, definido por quem *usa* a biblioteca. O subprojeto mostra o caso canônico — uma classe `Propriedades` sobre um `std::map<std::string, std::any>` cujo método de leitura devolve `std::optional<T>`, combinando "pode não existir" e "pode ser de qualquer tipo". É assim que funcionam os "*user data*" de bibliotecas de interface gráfica e os *callbacks* de APIs em C.

XII.1.4 `tuple`, `pair` e os *structured bindings*

Uma `struct` nomeia os campos; uma `std::tuple` não. É a única diferença relevante, e ela decide quando usar cada uma.

O problema histórico da `tuple` era o acesso: `std::get<0>(r)` não diz nada sobre o que é o campo zero. Os *structured bindings* do C++17 mudaram essa conta ao dar nomes no ponto de uso — e funcionam sobre `tuple`, `pair`, arranjo e `struct`:

```
auto [nome, idade, salario] = obter_registro();    // tuple
auto [x, y] = ponto;                               // struct, sem métodos

for (const auto& [fruta, qtd] : estoque)           // map: chave e valor
    fmt::print("{} → {}\n", fruta, qtd);

if (auto [it, inserido] = m.try_emplace("nova", 1); inserido)
    fmt::print("inseriu\n");                       // pair<iterator, bool>
```

Listagem 362: *Structured bindings* nos quatro contextos mais comuns.

Armadilha — os nomes do *binding* são novos, não *aliases*

`auto [a,b]` **copia** o objeto inteiro antes de dar nome às partes; `auto& [a,b]` referencia; `const auto& [a,b]` é o padrão seguro. Escrever `auto` por reflexo sobre um objeto grande copia o objeto sem avisar — e alterar `a` não altera o original, o que costuma ser descoberto tarde.

`std::tie`. Constrói uma `tuple` de *referências* aos argumentos, sem copiar. Como `tuple` já tem `operator<` lexicográfico, isso resolve comparação por múltiplos critérios numa linha:

```
return std::tie(a.idade, a.nome) < std::tie(b.idade, b.nome);  // desempate
std::tie(nome, idade, salario) = obter_registro();      // variáveis existentes
std::tie(std::ignore, so_a_idade, std::ignore) = obter_registro();
```

Listagem 363: Três usos de `std::tie`, em `src/vocabulary_types/demo_tuple.hpp`.

A alternativa escrita à mão são duas linhas com um `if`; com quatro critérios, viram sete linhas e um defeito esperando para acontecer.

Armadilha — `tie` guarda referências: nunca o devolva

O resultado referencia os objetos passados. Devolvê-lo de uma função deixa referências penduradas para locais que já morreram. Para devolver, `std::make_tuple`, que copia.

`std::apply` desempacota uma `tuple` nos parâmetros de uma função e `std::make_from_tuple` faz o mesmo para um construtor: os dois são a base de qualquer mecanismo que guarde argumentos agora e chame depois — `std::bind`, filas de *thread pool*, sistemas de tarefas (Seção VI.1).

Percorrer uma `tuple` não é possível com *range-for*, porque cada elemento tem um **tipo** diferente e não há `operator++` que faça sentido. Exige desdobrar em compilação, o que uma *fold expression* resolve em uma linha: `std::apply([&f](const auto&... e) { (f(e), ...); }, t);`. Uma *fold expression* (C++17) aplica um operador binário a todos os elementos de um *pack*; aqui o operador é a vírgula, e `(f(e), ...)` expande em `f(e1), f(e2), f(e3)` — o corpo é instanciado uma vez por elemento, com o tipo certo.

Quanto a escolher (Tabela 137): o `sizeof` é **idêntico** — o subprojeto verifica —, então a decisão é inteiramente de legibilidade. E desde o C++20 a inicialização designada (`return R{.encontrou = true, .indice = 3};`) tirou o último argumento prático a favor da `tuple` em código de aplicação.

Tabela 137: A escolha é 100% de legibilidade: o custo é o mesmo.

Use `tuple`/`pair` quando...	Use `struct` quando...
os campos não têm nome natural	o agregado é do **domínio**
é interno e efêmero	atravessa uma API pública
precisa de `tie`/`apply`	tem invariantes a manter
é código genérico sobre N tipos	tem métodos próprios
são 2 valores óbvios (`pair`)	tem 3 ou mais campos

XII.1.5 `std::span`: a janela sobre memória alheia

A assinatura mais comum do C legado é o par (ponteiro, tamanho):

```
1 int soma_antiga(const int* dados, std::size_t n);  // soma_antiga(bruto, 500) compila
2 int soma_span(std::span<const int> dados);         // o tamanho vem do objeto
```

Listagem 364: O parâmetro que não pode discordar de si mesmo, em `src/vocabulary_types/demo_span.hpp`.

A primeira funciona e é rápida, mas separa duas informações que só fazem sentido juntas. `soma_antiga(v, 500)` sobre um arranjo de cinco elementos compila sem um único aviso e lê 495 inteiros de memória que não é dela. E o corpo da função não pode usar *range-for* nem os algoritmos da biblioteca, porque um ponteiro cru não é um *range*.

`std::span` é esse par unificado num tipo que *é* um *range*. Não possui os dados, aceita arranjo C, `std::array`, `std::vector` e qualquer sequência contígua, e o tamanho vem sempre do objeto de origem — é impossível errar.

O segundo parâmetro é o *extent*: o número de elementos, quando conhecido em compilação. A diferença aparece no `sizeof` medido:

```
1 sizeof(span<int>)    = 16    ponteiro + tamanho
2 sizeof(span<int, 3>) =  8    só o ponteiro — o 3 está no TIPO
3 sizeof(vector<int>)  = 24    ponteiro + size + capacity
```

Listagem 365: Saída de `src/vocabulary_types/`: o tamanho no tipo não ocupa espaço no objeto.

Um `span<double, 3>` pode declarar `static_assert(v.extent == 3);` dentro da própria função. A conversão de estático para dinâmico é implícita; a inversa exige `.first<3>()`, e quem escreve assume o risco. As janelas — `first(n)`, `last(n)`, `subspan(i, n)` — devolvem outro `span` sobre o mesmo *buffer*, sem copiar nada.

Armadilha — `subspan` fora dos limites é UB (comportamento indefinido), não exceção

As versões com *template* (`.first<4>()`) verificam em compilação *quando o extent é estático*; as versões com argumento de execução não verificam nada. Prefira as primeiras sempre que o tamanho for conhecido.

Há duas coisas diferentes que podem ser congeladas, e a sintaxe engana (Tabela 138). `span<const T>` protege os *elementos*: não dá para escrever pela janela. `const span<T>` protege a *janela*: não dá para reapontar o `span`, mas escrever nos elementos compila normalmente. Numa assinatura, `std::span<const T>` é o padrão para "só lê".

Tabela 138: `span<const T>` contra `const span<T>`.

Quer proteger	Escreva
os **elementos**	`span<const T>` — 95% dos casos
a **janela**	`const span<T>`
ambos	`const span<const T>`

Um *borrowed range* é um *range* cujos iteradores continuam válidos depois de o objeto *range* morrer. Vale para `span` e `std::string_view` porque eles não possuem nada:

```
1 span<int>          : true
2 string_view        : true
3 vector<int>        : false    ← o objeto POSSUI os dados
4 vector<int>&       : true     ← a referência não possui
```

Listagem 366: Saída de `src/vocabulary_types/`: `std::ranges::borrowed_range`.

A consequência prática é que os algoritmos de `<ranges>` aceitam um `span` temporário e devolvem iterador utilizável, mas recusam um `vector` temporário — devolvem `std::ranges::dangling` (Seção III.2).

Armadilha — um `span` é tão frágil quanto um par de iteradores

```
1 std::span<int> s{v};
2 v.push_back(4);      // realocou → s está pendurado
3 s[0] = 1;            // heap-use-after-free
```

Porque *é* isso: um ponteiro e um tamanho. Vale a mesma tabela de invalidação do Seção III.1. As três regras de uso: (1) `span` é **parâmetro** — o uso dominante e sempre seguro; (2) como retorno, só se o *buffer* sobreviver à chamada, o que na prática significa apontar para um membro do objeto; (3) como membro, exige documentar quem é o dono. O subprojeto recomenda `-Db_sanitize=address,undefined` justamente por isto.

Regra — o resumo do vocabulário

Função que pode não achar nada: `optional<T>`; e se o *motivo* importa, `expected<T,E>`. "Um destes N tipos", lista fechada: `variant` com `visit` e o *overload idiom*. Tipos definidos por quem **usa** a API: `any` — e, se aparecer cascata de `any_cast`, era `variant`. Dois valores sem nome natural: `pair`; três ou mais, ou API pública: `struct` nomeado. Parâmetro que recebe sequência contígua: `span<const T>`.

XII.2 Computação em tempo de compilação

A segunda metade do capítulo troca de assunto sem trocar de estratégia. Se pôr informação no tipo permite ao compilador verificar, pôr *cálculo* no tempo de compilação permite a ele **executar** — e, o que é mais interessante, *provar*.

Por que — o argumento não é desempenho: é a ausência de UB (comportamento indefinido)

O avaliador de expressões constantes é obrigado, pelo padrão, a recusar qualquer operação cujo comportamento não esteja definido. Isso transforma uma classe inteira de defeitos de execução em erros de compilação:

```
constexpr int x = 1 / 0;          // erro de compilação, não crash
constexpr int y = arr[10];        // erro se estiver fora dos limites
constexpr int z = *(int*)nullptr; // erro
```

Código que passa sob `static_assert` é código **provadamente** livre de UB (comportamento indefinido) naquele caminho. Nenhuma bateria de testes dá essa garantia: testes cobrem as entradas que alguém pensou em escrever; isto cobre o caminho inteiro. A vantagem de desempenho é real, mas é a segunda melhor razão.

XII.2.1 `constexpr` é permissão, não garantia

O erro conceitual mais comum é ler `constexpr` como "isto sempre roda na compilação". É uma **permissão**: "isto *pode* rodar na compilação, se os argumentos permitirem". A mesma função serve aos dois mundos, e quem decide é o contexto da chamada:

```
constexpr int fatorial(int n) {
    int r = 1;
    for (int i = 2; i <= n; ++i) r *= i;   // laço: legal desde C++14
    return r;
}

constexpr int em_compilacao = fatorial(10);   // avaliado AGORA
int n = 5;
const int em_runtime = fatorial(n);           // vira código de máquina
```

Listagem 367: A mesma função nos dois mundos, em `src/compile_time/demo_constexpr.hpp`.

Para **forçar** a avaliação existem três contextos, do mais fraco para o mais forte: `constexpr int a = f();` força naquela declaração; `static_assert(f() == x);` força e ainda verifica o valor; e `std::array<int, f()>` força, porque um parâmetro de *template* que é valor é o contexto mais exigente que a linguagem tem. Sem nenhum desses, o compilador *pode* avaliar em compilação — e com `-O2` normalmente avalia —, mas não é obrigado. Para *garantir* que uma função nunca rode em execução, a ferramenta é `consteval`.

O que cabe dentro de uma função `constexpr` cresceu a cada padrão (Tabela 139). Conhecer essa tabela evita duas frustrações: tentar escrever um laço em C++11 e concluir que `constexpr` é inútil, e tentar devolver um `vector` em C++20 e concluir que o compilador está com defeito.

O C++20 permitiu alocar durante uma avaliação em compilação, o que trouxe `std::vector` e `std::string` para dentro do `constexpr`. A permissão vem com uma condição — **toda alocação feita durante a avaliação precisa ser liberada antes de a avaliação terminar** —, a chamada alocação *transitória*. Isso permite usar um `vector` como memória de rascunho, e o subprojeto resolve o crivo de Eratóstenes inteiro antes de o programa existir:

Tabela 139: O que passou a caber numa função `constexpr`, por padrão.

Padrão	Passou a aceitar
C++11	praticamente só `return expr;` — uma única instrução
C++14	laços, variáveis locais, `if`/`switch`, vários `return`
C++17	`if constexpr`, *lambdas* `constexpr`
C++20	alocação transitória, `try`/`catch`, `virtual`, `vector`, `string`
C++23	`if consteval`, `goto`, variáveis `static` locais

```
constexpr int soma_dos_primos_ate(int limite) {
    std::vector<bool> composto(limite + 1, false);      // aloca
    int soma = 0;
    for (int i = 2; i <= limite; ++i) {
        if (composto[i]) continue;
        soma += i;
        for (int j = i * i; j <= limite; j += i) composto[j] = true;
    }
    return soma;                          // o vector é destruído aqui
}

constexpr int soma_primos = soma_dos_primos_ate(1000);
static_assert(soma_primos == 76127);
```

Listagem 368: O crivo de Eratóstenes rodando no compilador.

Armadilha — `constexpr std::vector<int> v = {1,2,3};` não compila

A regra transitória proíbe que a alocação sobreviva ao fim da avaliação — e uma variável `constexpr` *é* o resultado que sobrevive. A memória do compilador não existe mais quando o programa roda; não haveria para onde apontar. Logo, um `vector` pode aparecer no *meio* de um cálculo `constexpr` e nunca ser o *resultado* dele. Para devolver dados de uma função `constexpr`, o tipo é `std::array`, cujo armazenamento é o próprio objeto.

Um construtor `constexpr` permite que objetos existam em contexto de compilação, e desde o C++14 métodos `constexpr` não são implicitamente `const` — podem modificar o objeto durante a avaliação. O `Racional` do subprojeto normaliza `2/4` para `1/2` no construtor, e `static_assert(meio.numerador() == 1);` verifica isso sem executar nada. Onde a mutação é necessária, o idioma é um *lambda* chamado imediatamente, que cria um escopo mutável e devolve a constante pronta.

Uma avaliação `constexpr` não pode fazer entrada/saída — não há sistema operacional na compilação —, chamar função não-`constexpr`, ler variável não-`constexpr`, usar `reinterpret_cast` nem deixar alocação viva ao fim. E há um limite prático: o GCC impõe um teto de passos de avaliação (`-fconstexpr-ops-limit`, padrão 33.554.432); ultrapassá-lo produz *"constexpr evaluation operation count exceeded"*.

XII.2.2 `consteval` e `constinit`: as garantias

`constexpr` deixa duas perguntas sem resposta, e o C++20 acrescentou uma palavra-chave para cada uma (Tabela 140). Funções `consteval` chamam-se *immediate functions*: toda chamada precisa ser, ela própria, expressão constante. Chamar com uma variável não compila, com a mensagem *"call to consteval function is not a constant expression"*.

A regra que confunde é que `consteval` é **contagiosa para cima**: uma função `constexpr` *pode* ser chamada em execução; se ela repassasse o próprio parâmetro a uma `consteval`, essa chamada teria de acontecer em execução — proibido por definição. O compilador recusa já na definição:

Tabela 140: As três palavras e o que cada uma exige.

Palavra	Avaliação em compilação	Argumento de execução
(nenhuma)	não	aceita
`constexpr`	**permitida**	aceita
`consteval`	**obrigatória**	**erro**

```
consteval int dobro_imediato(int x) { return x * 2; }

constexpr int f(int x) { return dobro_imediato(x); }      // ✗ não compila
consteval int g(int x) { return dobro_imediato(x) + 1; }  // ✓ contexto imediato
constexpr int h(int x) { return x + dobro_imediato(10); } // ✓ argumento literal
```

Listagem 369: Quem repassa um parâmetro para uma `consteval` precisa ser `consteval`.

A aplicação mais útil é a **validação de argumento literal**. Como um `throw` que escapa de uma avaliação constante é erro de compilação, um construtor `consteval` recusa valores inválidos antes de o programa existir:

```
class Porta {
public:
    consteval explicit Porta(int valor) : valor_(valor) {
        if (valor < 1 || valor > 65535)
            throw "porta fora do intervalo 1..65535";
    }
    constexpr int valor() const { return valor_; }
private:
    int valor_;
};

constexpr Porta http{80};        // ✓
// constexpr Porta ruim{70000}; // ✗ NÃO COMPILA — a mensagem cita o throw
```

Listagem 370: Validação em compilação, em `src/compile_time/demo_consteval.hpp`.

É exatamente o mecanismo do `std::format` do C++20: a *string* de formato é validada por uma `consteval`, e por isso `std::format("{:d}", "texto")` falha na **compilação**, não com exceção em produção. O subprojeto reproduz a ideia com uma `contar_placeholders` que recusa uma chave sem fechamento.

`constinit` e o *static initialization order fiasco*. Variáveis globais de unidades de tradução diferentes são inicializadas em ordem **não especificada** — ordem que, na prática, depende de como o *linker* arrumou os objetos. Se a inicialização de uma global em `b.cpp` lê uma global de `a.cpp`, o programa pode ler um objeto ainda não construído. Esse é o fiasco: um defeito que aparece e desaparece conforme a ordem de link muda.

`constinit` exige inicialização **estática** — feita na compilação e gravada direto no binário. Se o valor já está lá quando o programa começa, não há ordem a respeitar; e o compilador recusa qualquer inicialização que não caiba na compilação, de modo que o fiasco vira erro de *build*. A terceira linha da Tabela 141 é a razão de a palavra existir: sem ela, a única forma de garantir inicialização estática era tornar a variável `constexpr`, o que a congela. A outra saída clássica é o *Meyers singleton* (Seção V.1).

`if consteval` (C++23). Às vezes o algoritmo ótimo difere entre os dois mundos: em execução convém o intrínseco do processador; em compilação, ele não existe e é preciso um laço.

Tabela 141: `constinit` não implica `const`.

Palavra	Inicialização	Mutável depois
`const`	qualquer	não
`constexpr`	compilação	não
`constinit`	**compilação**	**sim**

```
constexpr int contar_bits(unsigned x) {
    if consteval {
        int n = 0;                          // caminho de COMPILAÇÃO
        while (x) { n += static_cast<int>(x & 1u); x >>= 1; }
        return n;
    } else {
        return __builtin_popcount(x);       // caminho de EXECUÇÃO
    }
}
```

Listagem 371: Uma função, dois algoritmos, um resultado.

Armadilha — `std::is_constant_evaluated()` tem três pegadinhas

A função do C++20 que precedeu o `if consteval` erra de três maneiras. **Um:** dentro de um `if` comum, responde `true` sempre que a função *é* `constexpr`, mesmo quando aquela chamada está sendo compilada para execução. **Dois:** `if constexpr (std::is_constant_evaluated())` é **sempre** verdadeiro, porque a própria condição é avaliada como constante — o ramo `else` vira código morto, silenciosamente. **Três:** o ramo de um `if` comum **não é contexto imediato**, então chamar uma função `consteval` lá dentro é erro de compilação, mesmo que o ramo só fosse executado durante a avaliação constante. O `if consteval` *cria* esse contexto, e é essa a diferença que justificou uma palavra-chave nova.

XII.2.3 `static_assert` e `type_traits`

`static_assert` é a única forma de fazer o compilador falhar de propósito, com uma mensagem sua. Combinado com `<type_traits>`, transforma suposições implícitas sobre tipos em contratos verificados. A regra que orienta o uso: *se você escreveu o comentário "T precisa ser copiável", esse comentário deveria ser um `static_assert`.*

Regra — a mensagem é o produto

Um `static_assert` sem mensagem faz o compilador repetir a *expressão*, que raramente ajuda. A mensagem deve dizer **o que fazer**, não o que falhou — quem lê já sabe o que falhou.

```
static_assert(std::is_trivially_copyable_v<T>,
    "serializar() copia bytes crus e só aceita tipos trivialmente "
    "copiáveis. Tipos com ponteiros, std::string ou destrutor próprio "
    "precisam de uma sobrecarga específica.");
```

Compare com a alternativa: *"static assertion failed"*.

Um *type trait* é um *template* que responde uma pergunta sobre um tipo em compilação. Os que respondem sim ou não terminam em `_v`; os de transformação terminam em `_t` e produzem outro tipo (Tabela 142). Entre os de transformação, um explica um comportamento que todo mundo já encontrou: `decay_t<const int&>` é `int` e `decay_t<int[5]>` é `int*` — exatamente o que `auto` faz na dedução por valor.

A pergunta "este tipo tem o método X?" não tem *trait* pronto. A forma clássica usa `std::void_t` com especialização parcial e SFINAE; a moderna usa um *concept*:

Tabela 142: O catálogo de *traits* que aparece de verdade.

Pergunta	Trait
dá para `memcpy`?	`is_trivially_copyable_v`
tem vtable / vptr?	`is_polymorphic_v`
o *move* é barato **e** seguro?	`is_nothrow_move_constructible_v`
dá para inicializar com `{}`?	`is_aggregate_v`
herda de X? converte para X?	`is_base_of_v`, `is_convertible_v`
é chamável com estes argumentos?	`is_invocable_v`

```cpp
// C++17: void_t + especialização parcial (8 linhas)
template <typename, typename = void>
struct tem_serializar : std::false_type {};

template <typename T>
struct tem_serializar<T,
        std::void_t<decltype(std::declval<const T&>().serializar())>>
    : std::true_type {};

// C++20: um concept diz a mesma coisa em 3 linhas — e diz mais
template <typename T>
concept Serializavel = requires(const T& t) {
    { t.serializar() } -> std::convertible_to<std::string>;
};
```

Listagem 372: O mesmo teste, em C++17 e em C++20, em `src/compile_time/demo_static_assert.hpp`.

O *concept* ganha em três frentes, e a terceira surpreende: além de ser mais curto e de dar mensagem de erro na chamada em vez de no meio da instanciação, ele verifica também o **tipo de retorno**. Um tipo cujo `serializar()` devolve `int` passa no *detection idiom* — que só pergunta se a expressão é válida — e é corretamente rejeitado pelo *concept*.

Os dois não são substitutos: falham em **momentos diferentes**, e isso muda a resolução de sobrecarga (Tabela 143). `com_assert(3.14)` é erro — o *template* foi escolhido e só então o *assert* falhou, sem chance de outra sobrecarga assumir. `com_concept(3.14)` funciona: a restrição removeu o *template* do conjunto de candidatos, e `void com_concept(double)` venceu. Use *concept* para **restringir**; `static_assert` para **afirmar** fatos sem alternativa.

Tabela 143: O momento da falha decide se há alternativa.

Ferramenta	Quando falha	Afeta a sobrecarga?
`static_assert`	**após** a escolha	não
concept	**durante** a escolha	**sim**

Há um domínio em que *concepts* não substituem nada: *layout* e ABI. Estruturas que atravessam uma fronteira binária — protocolo de rede, formato de arquivo, chamada para outra linguagem — têm requisitos de tamanho e de deslocamento de campo que *concepts* não expressam, porque restringem *templates* e não dizem nada sobre disposição de bytes.

```cpp
static_assert(sizeof(CabecalhoDeRede) == 8,
    "O cabeçalho do protocolo tem 8 bytes por especificação. "
    "Se este assert falhou, o padding mudou e os pacotes gerados "
    "serão incompatíveis com a outra ponta.");
static_assert( offsetof (CabecalhoDeRede, comprimento) == 2);
static_assert(std::is_trivially_copyable_v<CabecalhoDeRede>);
```

Listagem 373: O contrato do protocolo, verificado no *build*.

O subprojeto mostra o mesmo `struct` com e sem `#pragma pack(1)`, e os tamanhos diferem — que é precisamente o acidente que os três *asserts* existem para transformar em erro de *build* em vez de pacote corrompido em produção.

XII.2.4 NTTP: quando o valor entra no tipo

Um *template* pode receber tipos (`template <typename T>`) e também **valores** (`template <int N>`). O segundo caso chama-se **NTTP**, de *non-type template parameter*, e o ponto é que o valor passa a fazer parte da identidade do tipo: `BufferFixo<int,4>` e `BufferFixo<int,16>` são tipos **diferentes**, não são atribuíveis um ao outro, e `capacidade()` não lê memória nenhuma.

A aplicação que mais rende é verificação dimensional, porque transforma uma classe inteira de defeitos de execução em erro de compilação:

```
template <std::size_t Linhas, std::size_t Colunas>
class Matriz {
    // Só compila se as dimensões internas casarem, e o resultado tem o
    // tipo certo, derivado das entradas.
    template <std::size_t OutrasColunas>
    constexpr Matriz<Linhas, OutrasColunas>
    operator*(const Matriz<Colunas, OutrasColunas>& o) const;
};

Matriz<2,3> * Matriz<3,4> → Matriz<2,4>     // compila
Matriz<2,3> * Matriz<2,2> → NÃO COMPILA     // dimensões incompatíveis
```

Listagem 374: Álgebra de matrizes verificada pelo compilador, em `src/compile_time/demo_nttp.hpp`.

Sem NTTP, as dimensões seriam campos e o erro só apareceria em execução — provavelmente como leitura fora dos limites, que é UB (comportamento indefinido) silencioso.

Até o C++17, NTTP só admitia inteiros, `enum`, ponteiros e referências. O C++20 ampliou para tipos de classe literais, o que permitiu parametrizar por texto. A condição é o tipo ser ***structural***: todos os membros de dados públicos, sem `private` ou `protected`, sem `virtual`, e os membros também *structural*. É por isso que `std::string` **não** pode ser NTTP — tem membros privados e aloca — e é por isso que se escreve um `StringFixa` à mão:

```
template <std::size_t N>
struct StringFixa {
    char dados[N]{};
    consteval StringFixa(const char (&lit)[N]) { std::copy_n(lit, N, dados); }
    constexpr std::string_view view() const { return {dados, N - 1}; }
    constexpr bool operator==(const StringFixa&) const = default;  // obrigatório
};
template <std::size_t N> StringFixa(const char (&)[N]) -> StringFixa<N>;

template <StringFixa Nome, typename T> class Campo { /* ... */ };

Campo<"largura", int> largura{1920};
static_assert(decltype(largura)::nome() == "largura");
```

Listagem 375: *Strings* como parâmetro de *template* (C++20).

O `operator==` padronizado não é decoração: a comparação estrutural é requisito para servir de NTTP, porque o compilador precisa decidir se duas instanciações são a mesma. O `N - 1` corta o `'\0'` do literal. E a consequência é que `Campo<"largura",int>` e `Campo<"altura",int>` são tipos **distintos**: trocá-los é erro de compilação, não erro de lógica. Vale mencionar ainda o `template <auto Valor>`, que aceita qualquer constante — `int`, `double`, enumerador — sem um *template* por tipo de valor.

Armadilha — cada valor distinto é uma instanciação nova

NTTP tem um custo que não aparece em *benchmark* nenhum de execução: `BufferFixo<int,4>`, `<int,8>` e `<int,16>` são três classes completas, com todos os métodos gerados três vezes. As consequências são binário maior, compilação mais lenta e pressão sobre o cache de instruções num laço que alterne entre instanciações. Use quando os valores forem **poucos, fixos e conhecidos** e participarem da **correção** do programa — dimensões, unidades. Não use quando o valor vier de configuração, arquivo ou entrada; e, se a intenção era apenas evitar uma alocação, o instrumento certo é `std::span`. A técnica clássica para conter a explosão é extrair para uma base não-*template* tudo o que não depende do NTTP.

XII.2.5 Tabelas geradas na compilação

Os recursos anteriores são mecanismo; este é o pagamento: substituir trabalho de inicialização por dados que já nascem prontos dentro do binário. O exemplo canônico é a tabela de 256 entradas do CRC-32, tradicionalmente gerada na inicialização do programa ou colada à mão a partir de um arquivo gerado.

```
1  constexpr std::array<std::uint32_t, 256> gerar_tabela_crc32() {
2      constexpr std::uint32_t polinomio = 0xEDB88320u;
3      std::array<std::uint32_t, 256> tabela{};
4      for (std::uint32_t i = 0; i < 256; ++i) {
5          std::uint32_t c = i;
6          for (int k = 0; k < 8; ++k)
7              c = (c & 1u) ? (polinomio ^ (c >> 1)) : (c >> 1);
8          tabela[i] = c;
9      }
10     return tabela;
11 }
12
13 // 'inline constexpr' numa variável de header: uma única instância no
14 // programa, sem violar a ODR. Antes do C++17 isso exigiria um .cpp.
15 inline constexpr auto TABELA_CRC32 = gerar_tabela_crc32();
16
17 static_assert(TABELA_CRC32[255] == 0x2D02EF8Du);
```

Listagem 376: A tabela do CRC-32 calculada pelo compilador, em `src/compile_time/demo_tabelas.hpp`.

A ODR — *One Definition Rule* — exige que cada entidade tenha exatamente uma definição no programa inteiro. Um *header* incluído por dez `.cpp` definiria a variável dez vezes; `inline` instrui o *linker* a fundir as definições numa só.

A tabela vai mesmo para o binário — verificado. Em vez de afirmar que a tabela está no binário, o subprojeto manda inspecionar o binário:

```
1 $ objdump -t dist/bin/compile_time | grep -i TABELA
2 .rodata          0x400  TABELA_CRC32
3 .data.rel.ro     0x0d8  TABELA_HTTP
```

Listagem 377: Medição em `src/compile_time/`: onde as tabelas foram parar no executável.

O resultado contraria a expectativa ingênua de que as duas estariam no mesmo lugar, e a razão é instrutiva. `TABELA_CRC32` contém apenas `uint32_t` — valores puros, gravados literalmente no arquivo e mapeados somente-leitura: vão para `.rodata`, ocupando $0x400 = 1024$ bytes, exatamente 256×4. `TABELA_HTTP` contém `std::string_view`, que são **ponteiros**: o endereço final só é conhecido quando o *loader* mapeia o binário, então ela precisa de correção em tempo de carga e vai para `.data.rel.ro` — somente-leitura

após a relocação. O que mais importa é o que *não* apareceu: nenhuma das duas está em `.bss`, a seção de dados zerados que seria preenchida por código de inicialização. Se estivessem lá, a promessa de "calculado na compilação" seria falsa.

Desde o C++20 `std::sort` é `constexpr`, o que permite escrever a tabela fora de ordem, deixar o compilador ordenar, e usar busca binária em execução:

```
constexpr auto gerar_tabela_http() {
    std::array<EntradaHttp, 9> t{{ /* escrita fora de ordem */ }};
    std::sort(t.begin(), t.end());          // constexpr desde C++20
    return t;
}
inline constexpr auto TABELA_HTTP = gerar_tabela_http();

static_assert(std::is_sorted(TABELA_HTTP.begin(), TABELA_HTTP.end()));
static_assert(descrever_http(404) == "Not Found");
static_assert(descrever_http(999) == "Unknown");
```

Listagem 378: A pré-condição da busca binária, provada em vez de testada.

Aplicar busca binária a uma sequência não ordenada é UB (comportamento indefinido) silencioso (Seção III.1). Aqui a pré-condição não é testada: é *provada*. Essa diferença é o argumento inteiro da seção.

`std::map` não é `constexpr`, porque aloca nós. Para um mapeamento pequeno e fixo, um arranjo ordenado com busca binária ganha nas três dimensões (Tabela 144) — inclusive em velocidade, porque é contíguo, com a mesma lógica de cache do Seção III.1.

Tabela 144: Para poucas dezenas de entradas fixas, o arranjo ordenado ganha em tudo.

Estrutura	`constexpr`	Aloca	Busca
`std::map`	não	sim	$O(\log n)$
`std::unordered_map`	não	sim	$O(1)$ médio
arranjo ordenado	**sim**	**não**	$O(\log n)$

Armadilha — uma API `constexpr` robusta usa índices, não ponteiros

Esta lição só aparece escrevendo o código. A forma natural de devolver "achou ou não" é um ponteiro, e depois `return p ? *p : padrao;`. Acontece que converter um ponteiro-para-membro-de-objeto-`constexpr` em `bool` **não é expressão constante** em todos os modos do GCC: o mesmo código compila normalmente e falha sob `-fsanitize`, com a avaliação constante recusada. O `MapaFixo` do subprojeto devolve, por isso, o **índice** — ou `N`, se não achou. Aritmética de índice é sempre expressão constante, e por isso é a base mais robusta. É um caso em que a medição — aqui, um *build* que falha — contrariou o que o projeto "natural" recomendava.

Regra — o custo é pago pelo compilador, e ele é lento

O interpretador de `constexpr` do GCC é ordens de grandeza mais lento que código nativo, e tem teto (`-fconstexpr-ops-limit`, 33,5 milhões de passos). **Não** vale a pena quando a tabela é grande e o *build* fica lento, quando os dados mudam com frequência — um arquivo lido em execução é mais fácil de manter — ou quando o cálculo já era trivial na inicialização. **Vale muito** quando a tabela é pequena ou média e imutável (CRC, seno, códigos de erro), quando se quer *provar* uma propriedade em vez de testá-la, em ambiente sem *heap* (embarcado, *kernel*), e para eliminar inicialização estática dinâmica. A forma de decidir é medir: `time g++ -std=c++23 -c src/compile_time/main.cpp`.

XII.3 O inventário medido do C++23

O subprojeto `src/cpp23/` é diferente de todos os outros: além de *ensinar* os recursos do C++23, ele **mede** quais estão disponíveis no compilador local e imprime o resultado. A tabela de suporte do `src/ROADMAP.md` é gerada a partir dessa saída.

Por que — "o C++23" não é algo que um compilador tem ou não tem

Cada recurso do padrão chega numa versão diferente do compilador, e a biblioteca costuma atrasar em relação à linguagem — são equipes e cronogramas distintos. Perguntar "este compilador suporta C++23?" é uma pergunta mal formulada: a pergunta útil é "suporta *este recurso*?", e ela tem resposta mecânica.

Uma ***feature-test macro*** é uma macro que o padrão obriga a implementação a definir quando um recurso está disponível, com valor igual à data de adoção no formato AAAAMM. Não são convenção de compilador — são parte do padrão, o que as torna a única forma portátil de escrever código que se adapta. As de **linguagem** começam com `__cpp_` e são predefinidas pelo compilador; as de **biblioteca** começam com `__cpp_lib_` e vivem em `<version>`, um *header* que não declara nada além delas. O valor numérico importa, porque recursos ganham revisões: `#if __cpp_lib_optional >= 202110` testa especificamente se as operações monádicas estão presentes.

XII.3.1 O inventário, e as seis entradas corrigidas

```
if consteval                  ✓ 202106
operator[](i, j, k)           ✓ 202211
static operator()             ✓ 202207
sufixo uz / z                 ✓ 202011
\N{NOME DO CARACTERE}         ✓ 202207
deducing this                 ✗ ausente      ← a mais sentida
```

Listagem 379: Saída de `src/cpp23/`: o que este GCC 13.3 tem de *linguagem*.

Na biblioteca estão **presentes** `std::expected`, `std::byteswap`, `std::to_underlying`, `std::unreachable`, `views::zip`, `string::contains` e `std::stacktrace` — este último com uma ressalva grave, tratada adiante. Estão **ausentes** `std::print`/`println`, `std::mdspan`, `std::generator`, `ranges::to<C>()`, `std::flat_map`, `views::enumerate`, `ranges::starts_with` e `std::out_ptr`.

E aqui está o resultado que ensina método. A tabela de suporte do repositório foi escrita, antes da medição, a partir de leitura de documentação. Quando o programa passou a medir, **seis entradas estavam erradas** (Tabela 145) — todas na mesma direção: a de **subestimar** o que já estava disponível. Ou seja, o repositório estava reimplementando à mão coisas que a biblioteca já oferecia. O viés tem explicação: documentação de terceiros e tabelas de compatibilidade envelhecem, e a memória de quando um recurso "ainda não existia" persiste anos depois de deixar de ser verdade. A macro não envelhece, porque é o próprio compilador respondendo.

Regra — não presuma: meça

Reproduzir o inventário em qualquer *toolchain* custa duas linhas:

```
echo '#include <version>' > /tmp/v.cpp
g++ -std=c++23 -dM -E /tmp/v.cpp | grep __cpp_lib | sort
```

Isso despeja todas as macros de biblioteca definidas, com os valores. É a resposta autoritativa, e é local.

Tabela 145: O que a documentação sugeria contra o que a medição encontrou no GCC 13.3.

Recurso	Presumido	Medido
`std::stacktrace`	ausente	**presente** (com `-lstdc++_libbacktrace`)
`views::zip`	ausente	**presente**
`ranges::fold_left`	ausente	**presente**
`views::chunk` / `slide`	ausente	**presente**
`std::move_only_function`	ausente	**presente**
`std::spanstream`	ausente	**presente**

Armadilha — a macro diz que a API existe, não que ela linka

Esta é a lição mais útil do subprojeto inteiro. O `__cpp_lib_stacktrace` está definido neste GCC, e o *header* existe. Mesmo assim:

```
$ g++ -std=c++23 usa_stacktrace.cpp
undefined reference to '__glibcxx_backtrace_simple'

$ g++ -std=c++23 usa_stacktrace.cpp -lstdc++_libbacktrace
frames=5  ✓
```

A *feature-test macro* responde sobre a **interface**, que é o que o compilador enxerga; ela não sabe nada sobre bibliotecas de link. Sempre há um passo de verificação além da macro, e ele é **compilar de verdade**. O subprojeto opta por não usar `std::stacktrace` justamente porque `libstdc++_libbacktrace` não está em toda distribuição, e exigi-la quebraria o *build* de quem clonar o repositório.

O padrão que permite usar um recurso novo sem quebrar em *toolchains* antigos é o `#if` sobre a macro, com um caminho alternativo equivalente:

```cpp
constexpr std::uint32_t trocar_bytes(std::uint32_t v) {
#if defined(__cpp_lib_byteswap) && __cpp_lib_byteswap >= 202110L
    return std::byteswap(v);
#elif defined(__GNUC__)
    return __builtin_bswap32(v);
#else
    return ((v & 0x000000FFu) << 24) | ((v & 0x0000FF00u) << 8) |
           ((v & 0x00FF0000u) >> 8)  | ((v & 0xFF000000u) >> 24);
#endif
}

static_assert(trocar_bytes(0x12345678u) == 0x78563412u);
```

Listagem 380: Três implementações escolhidas em compilação, em `src/cpp23/demo_inventario.hpp`.

Três regras governam esse padrão. **Um:** teste a *macro*, não a versão do compilador — `#if __GNUC__ >= 13` erra em Clang e em MSVC. **Dois:** compare com o *valor*, não apenas `#ifdef`, porque recursos ganham revisões. **Três:** o caminho alternativo precisa ser *equivalente*, não aproximado — e o `static_assert` da última linha é o que verifica isso.

XII.3.2 O que a linguagem ganhou

Além do `if consteval` já tratado, quatro adições merecem registro.

O **subscript multidimensional** — `operator[](i, j, k)` — finalmente existe, e funciona inclusive em `constexpr`. Mas vem com uma armadilha de compatibilidade rara: uma sintaxe reciclada.

Armadilha — `a[i, j]` mudou de significado entre C++20 e C++23

Em C++20, `a[i, j]` era o operador **vírgula** dentro dos colchetes: avalia `i`, **descarta** o resultado, e indexa por `j`. O C++20 depreciou esse uso e o C++23 reciclou a sintaxe. A consequência é que código antigo com `a[i, j]` **muda de comportamento** ao ser recompilado em C++23 — silenciosamente, se o tipo de `a` tiver o operador novo. Em arranjos crus a vírgula continua valendo, porque arranjos não podem ter `operator[]` próprio. Compile com `-Wcomma-subscript` para o GCC avisar.

`static operator()` e `static operator[]` atendem ao caso do *functor* sem estado, que não precisa do `this`: marcar a chamada como `static` remove o parâmetro implícito — em código genérico muito quente, é um argumento a menos por invocação. É por isso que os adaptadores de `<ranges>` do C++23 usam essa forma: são chamados milhões de vezes e nenhum tem estado. Note que `static` e `const` são mutuamente exclusivos aqui: não há `this` para ser `const`.

Os **sufixos `uz` e `z`** resolvem a ausência de sufixo de literal para `std::size_t`, que obrigava a `static_cast` decorativos ou gerava avisos de comparação com sinais diferentes. Agora `0uz` já é `size_t` e `0z` é `std::ptrdiff_t`: `for (auto i = 0uz; i < v.size(); ++i)` não gera `-Wsign-compare`.

`[[assume]]` é uma promessa ao otimizador: "esta condição é verdadeira, pode contar com isso". Com ela, o compilador pode eliminar verificações, escolher instruções mais estreitas e remover ramos.

Armadilha — `[[assume]]` é o oposto de `assert`

`assert(x > 0)` *verifica* e aborta se for falso; `[[assume(x > 0)]]` **não verifica** e é UB (comportamento indefinido) se for falso. E a expressão **não é avaliada**: `[[assume(++i > 0)]]` não incrementa `i`. Use com muita parcimônia, e sempre em par com um `assert` em *debug* — assim o *build* de depuração **prova** o que o de *release* **assume**.

Por fim, os escapes nomeados: `"\N{RIGHTWARDS ARROW}"` usa o nome oficial do caractere no Unicode em vez do número, e o compilador valida o nome — um nome errado é erro de compilação, não um caractere errado descoberto na tela.

XII.3.3 O que a biblioteca ganhou

`std::to_underlying` substitui `static_cast<std::underlying_type_t<Codigo»(c)`. O ganho não é digitar menos: é **não repetir o tipo**. Se `Codigo` mudar de `uint16_t` para `uint32_t`, o `static_cast` explícito continua compilando e **trunca**; o `to_underlying` acompanha sozinho. Ele não faz o caminho de volta — `int` para `enum class` continua exigindo `static_cast`, e continua sem validar que o valor corresponde a um enumerador existente.

`std::unreachable` resolve um aviso legítimo: um `switch` que cobre todos os enumeradores ainda faz o compilador dizer *"control reaches end of non-void function"*, porque um `enum class` *pode* guardar um valor fora da lista, obtido por `static_cast`. Das três saídas, só uma diz a verdade: `return {};` esconde o defeito atrás de um valor falso; `throw` custa e trata um caso impossível; `std::unreachable()` informa o compilador do que realmente se sabe. Ele é equivalente a `[[assume(false)]]`, com a mesma consequência: se a premissa estiver errada, é UB (comportamento indefinido).

`views::zip` preenche o buraco mais visível de `<ranges>` no C++20 — percorrer sequências em paralelo:

```
1 for (const auto& [nome, idade, salario] : std::views::zip(nomes, idades, salarios))
2     fmt::print("{:<7} {:>3} anos  R$ {:>8.2f}\n", nome, idade, salario);
```

Listagem 381: `zip` de três sequências, com *structured bindings*.

Ele para no **menor** *range*, o que elimina o `min` das formas antigas com índice — fácil de esquecer. Como `zip` referencia em vez de copiar, valem as mesmas regras de tempo de vida das *views* (Seção III.2). E o substituto para o `views::enumerate` ausente sai de graça: `views::zip(views::iota(0uz), v)` funciona porque `iota(0uz)` é infinito e o `zip` para no menor.

`std::function` exige que o chamável seja **copiável**, mesmo quando ninguém vai copiá-lo — é uma exigência do tipo, não do uso. Isso impede guardar um *lambda* que capturou um `std::unique_ptr`, uma `std::thread` ou um `std::promise`:

```
1 std::function<std::string()> f = [r = std::move(ptr)] { return *r; };              // ✗
2 std::move_only_function<std::string()> f = [r = std::move(ptr)] { return *r; }; // ✓
```

Listagem 382: O chamável que possui um recurso.

Onde isso morde na prática é a fila de tarefas de um *thread pool*: a tarefa captura um `promise`, que é *move-only* (Seção VI.1). E a API nova ainda qualifica a chamada — `void() const`, `void() &&`, `void() noexcept` — coisas que `std::function` nunca teve.

`std::spanstream` é o *stream* sobre um *buffer* que já existe. Um `stringstream` sempre aloca; para analisar um *buffer* pronto, isso é uma cópia inteira desnecessária. O antigo `strstream` resolvia e foi depreciado porque a API não deixava claro quem era dono do *buffer*; o `span` resolve isso por construção, porque **nunca** é dono.

Armadilha — o `- 1` do `span` sobre um `char[]` não é detalhe

```
1 char dados[] = "42 3.14 texto";
2 std::ispanstream entrada{std::span<char>{dados, std::size(dados) - 1}};
```

`std::span{dados}` sobre um `char[]` inclui o `'\0'` final — o arranjo tem 14 bytes, não 13. Sem o `- 1`, o terminador vira um caractere de dado e a última *string* lida sai com um byte a mais. O sintoma é um tamanho errado que *parece* certo quando impresso.

Por fim, duas *views* de janela que parecem iguais e não são:

```
1 chunk(3): [1 2 3] [4 5 6] [7 8 9] [10]      ← disjuntos, o último pode ser menor
2 slide(3): [1 2 3] [2 3 4] [3 4 5] ...        ← deslizante, todos do mesmo tamanho
```

Listagem 383: Saída de `src/cpp23/`: janelas disjuntas contra deslizantes.

`chunk` avança de N em N e serve para processar em lotes; `slide` avança de 1 em 1 e serve para média móvel — que fica em uma linha, combinando `slide(3)` com `ranges::fold_left`. Antes do C++23, os dois eram laços com índice e aritmética de fronteira fácil de errar no último bloco.

XII.3.4 O que falta, e o substituto de cada coisa

Toda ausência vem com uma alternativa que funciona hoje (Tabela 146). O objetivo não é lamentar: é escrever código que já resolve o problema e que migra sem dor quando o *toolchain* acompanhar. O caso do `std::print` merece nota: ele **nasceu** da biblioteca `fmt` — a proposta P2093 foi escrita pelo autor dela, e a API é idêntica de propósito. É por isso que este repositório usa `fmt` em todos os subprojetos: quando o `std::print` chegar, a migração é trocar o prefixo.

Tabela 146: O que o GCC 13.3 não tem, e o que usar no lugar.

Ausente	Substituto que funciona hoje
`std::print` / `println`	`fmt::print` / `fmt::println` — API idêntica
`std::mdspan`	`operator[](i,j)` próprio; ou `kokkos/mdspan`
`ranges::to<C>()`	`std::vector v(view.begin(), view.end())`
`views::enumerate`	`views::zip(views::iota(0uz), v)`
`std::generator`	`promise_type` à mão (Seção VI.3)
deducing this	*y-combinator* (Seção IV.4); CRTP
`std::flat_map`	arranjo ordenado (Seção XII.2)
`ranges::starts_with`	`std::equal`

Armadilha — o substituto de `ranges::to` exige um *common range*

Construir um `vector` a partir de `view.begin()` e `view.end()` só funciona se os dois forem do *mesmo tipo* — um *common range*. *Views* que terminam em sentinela (Seção III.1) precisam de `views::common` antes, ou de um auxiliar genérico de seis linhas que percorre o *range* e empurra num `vector`.

Regra — a régua para adotar um recurso novo

A pergunta não é "o C++23 tem?", é "o meu *toolchain* **mínimo** tem?". Na ordem: (1) qual é o compilador **mais antigo** que precisa compilar este código — a integração contínua, a distribuição LTS, o cliente? (2) a *feature-test macro* está definida nele? (3) ela **linka** sem *flags* extras? Se as três respostas forem sim, use direto. Se a segunda ou a terceira falhar, o padrão é o `#if` com caminho alternativo equivalente.

XII.4 Módulos nomeados

O último subprojeto enfrenta a maior mudança estrutural que o C++20 propôs: a substituição do `#include`.

Por que — o `#include` é uma cópia de texto, e todos os problemas vêm daí

O pré-processador não entende C++. Ele lê `#include <vector>` e substitui a linha pelo *conteúdo textual* do arquivo, recursivamente. Cinco consequências decorrem disso. **Compilação lenta:** o mesmo *header* é reprocessado do zero em *cada* `.cpp` — um projeto com 200 arquivos que incluem `<string>` analisa `<string>` 200 vezes. **Macros vazando:** um `#define` feito antes do `#include` afeta o texto incluído, e vice-versa, sem barreira. **A ordem passa a importar:** o *header* B só compila se A vier antes, e isso não está escrito em lugar nenhum. ***Include guards*:** necessários porque a inclusão é textual. **Tudo é público:** quem inclui A recebe também tudo que A incluiu. Módulos resolvem os cinco de uma vez — ao custo de uma **ordem de compilação obrigatória**, que é o que torna a adoção difícil.

Uma **interface de módulo** é um arquivo cujo conteúdo significativo começa com `export module NOME;`. A extensão `.cppm` é convenção — o Clang a reconhece, o GCC não.

```
1  module;                          // ← início do "global module fragment"
2  #include <cmath>                 //   o ÚNICO lugar para #include numa interface
3
4  export module geometria;         // ← a declaração que define o módulo
5
6  namespace detalhe {              // não exportado → invisível para quem importa
7      double quadrado(double x) { return x * x; }
8  }
9
```

```
10 export namespace geometria {   // exportado → a interface pública
11     struct Ponto { double x{}, y{}; };
12
13     double distancia(const Ponto& a, const Ponto& b) {
14         return std::sqrt(detalhe::quadrado(b.x − a.x) +
15                          detalhe::quadrado(b.y − a.y));
16     }
17
18     template <typename T>      // templates funcionam: o .gcm guarda a árvore
19     T maior(T a, T b) { return a > b ? a : b; }
20
21     inline constexpr double PI = 3.14159265358979323846;
22 }
```

Listagem 384: A estrutura completa, em `src/modules/geometria.cppm`.

O ***global module fragment*** é o trecho entre `module;` e `export module`: existe porque *headers* antigos ainda precisam ser incluídos, e é o único lugar de uma interface onde um `#include` pode aparecer. A diferença de fundo em relação a um *header* é que o módulo é compilado **uma vez**, e o consumidor lê a forma binária do resultado — no GCC, um arquivo `.gcm`. Templates funcionam normalmente, ao contrário do que a intuição sugere: o `.gcm` guarda a árvore sintática, não texto.

Tabela 147: O que muda entre *header* e módulo.

	Header	**Módulo**
compilação	reprocessado em cada TU	compilado **uma** vez
ordem	irrelevante	**obrigatória**
isolamento	tudo vaza	só o que tem `export`
macros	atravessam nos dois sentidos	**não** atravessam
include guards	necessários	desnecessários
templates	precisam estar no *header*	ficam no `.gcm`

O demo prova o isolamento com um erro provocado de propósito. `geometria.cppm` inclui `<cmath>` no *global module fragment* e usa `std::sqrt` internamente; o `main.cpp` **não** inclui `<cmath>`:

```
1 // em main.cpp:  double r = std::sqrt(16.0);
2 error: 'sqrt' is not a member of 'std'
```

Listagem 385: Medição em `src/modules/`: o `#include` do módulo não atravessa o `import`.

Com um *header*, `std::sqrt` estaria disponível ali sem ninguém ter pedido — e a poluição é **cumulativa**: um *header* que inclui outro que inclui outro acaba trazendo milhares de nomes. O mesmo vale para `detalhe::quadrado`, definida sem `export`: é invisível no consumidor, e o nome não colide com nada dele.

Um módulo pode ainda importar outro e repassá-lo: `export import geometria;` em `formatacao.cppm` faz com que `main.cpp`, que só escreve `import formatacao;`, enxergue `geometria` também. E a ordem passa a importar: o `geometria.gcm` precisa existir antes de `formatacao.cppm` compilar.

XII.4.1 Dois defeitos do GCC 13, encontrados escrevendo o demo

Estas duas armadilhas custaram tempo real e não aparecem nos tutoriais. Ambas foram verificadas no GCC 13.3 com `-fmodules-ts`.

Armadilha — `export` de variável global some com o *global module fragment*

Reduzido ao mínimo:

```
module;
#include <cmath>
export module m;
export constexpr double PI = 3.14;   // ← invisível no consumidor

// no consumidor:
import m;
int main() { return (int)PI; }       // error: 'PI' was not declared
```

Sem o *global module fragment* — removendo o `module;` e o `#include` do topo — a mesma declaração funciona. Dentro de um `namespace` exportado, funciona nos dois casos. É por isso que o `PI` do subprojeto está dentro de `export namespace geometria`: não é estilo, é necessidade.

Armadilha — `constexpr` em escopo de *namespace* tem *linkage* interna

```
export namespace geometria {
    constexpr double PI = 3.14;          // ✗ 'PI' is not a member of 'geometria'
    inline constexpr double PI = 3.14;   // ✓
}
```

Linkage é a propriedade que determina se um nome pode ser referenciado de outra unidade de tradução. Uma variável `constexpr` em escopo de *namespace* tem *linkage* **interna** por padrão, como um `const`: cada TU teria a sua cópia, e o módulo não a exporta de verdade. A regra é a mesma que vale para *headers* desde o C++17; o que muda é o sintoma. Num *header*, a consequência é uma violação silenciosa da ODR — cada TU com o seu endereço; num módulo, é erro de compilação no consumidor. O erro é preferível, mas só se souber lê-lo.

XII.4.2 As armadilhas do *toolchain*

Armadilha — `.cppm` precisa de `-x c++` no GCC

```
g++ -fmodules-ts -c geometria.cppm          # ✗ "linker input file unused"
g++ -fmodules-ts -x c++ -c geometria.cppm   # ✓
```

O GCC não reconhece a extensão `.cppm` e trata o arquivo como entrada de *linker*. A mensagem de erro não sugere a causa. O Clang reconhece.

Compilar uma interface produz **dois** artefatos: o `.o`, com o código, e o `.gcm`, com a interface, num diretório `gcm.cache/` criado automaticamente. O `.gcm` **não entra no link** — é consumido em tempo de compilação pelos consumidores. E o tamanho dele é revelador:

```
$ make mostrar-cache
-rw-r--r-- 3.3K  gcm.cache/geometria.gcm
-rw-r--r-- 2.1M  gcm.cache/formatacao.gcm      ← inclui <string> transitivamente
```

Listagem 386: Medição em `src/modules/`: os artefatos binários de módulo.

A diferença de três ordens de grandeza tem explicação direta: `formatacao` incluiu `<string>` no seu *global module fragment*, e o `.gcm` guarda a representação compilada de tudo aquilo. É o mesmo custo de sempre — só que agora pago uma vez e visível num arquivo, em vez de pago em toda TU e invisível.

Além dos módulos nomeados, o GCC 13 suporta *header units*: pré-compilar um *header* com `g++ -fmodules-ts -x c++-system-header vector` e depois escrever `import <vector>;` — em linha própria,

porque `import <vector>; int main()` na mesma linha não compila. O `import std;`, que traria a biblioteca padrão inteira como módulo, **não** está disponível no GCC 13.

XII.4.3 Por que este subprojeto fica fora do Meson

Esta é a única exceção do repositório: `src/modules/` tem um `Makefile` próprio e não participa do `make build`. A razão é conteúdo didático, não conveniência.

Por que — a ordem de compilação é o problema não resolvido de módulos

Um sistema de *build* precisa descobrir a ordem de compilação **antes** de compilar. Com `#include` isso é fácil: o `-MMD` do GCC emite a lista de arquivos incluídos e o Make ou o Ninja constroem o grafo. Com `import`, não: a dependência é sobre o `.gcm`, que *só existe depois de compilar* o módulo que o produz — uma circularidade entre o planejamento e a execução do *build*.
A saída é um *scanner* que lê os fontes e extrai as declarações `import` antes de qualquer compilação: o `-fdeps-format` padronizado, que chegou no GCC 14, ou o `clang-scan-deps`. O suporte do Meson a módulos com GCC 13 é experimental e frágil justamente porque o formato que o GCC 13 emite não é o que o Meson espera. **É exatamente esse problema de *tooling* que atrasou a adoção de módulos por cinco anos**, e não qualquer defeito da ideia.

O `Makefile` de 40 linhas do subprojeto demonstra melhor que a integração, porque a ordem fica explícita e escrita à mão:

```
# regra para interfaces de módulo: o -x c++ é obrigatório
%.o: %.cppm
	$(CXX) $(CXXFLAGS) -x c++ -c $< -o $@

# um #include o Make rastreia com -MMD; um import NÃO
formatacao.o: geometria.o
main.o:       formatacao.o geometria.o
```

Listagem 387: As dependências que o Make não descobre sozinho, em `src/modules/Makefile`.

Tabela 148: Estado da adoção de módulos, por *toolchain*.

Compilador	Módulos nomeados	*Header units*	`import std;`
GCC 13	sim (`-fmodules-ts`)	sim	não
GCC 14+	sim	sim	não
Clang 16+	sim	sim	parcial
MSVC 19.3x	sim	sim	sim

Regra — quando adotar módulos

Só se o compilador mínimo **e** o sistema de *build* suportarem — e o segundo é o gargalo real, não o primeiro. Hoje isso significa CMake 3.28 ou superior com Ninja 1.11 ou superior. Para migrar um projeto grande, comece pelas **folhas** do grafo de dependências: os módulos que não importam ninguém são os únicos que podem ser convertidos sem tocar em mais nada. Constantes exportadas: sempre `inline constexpr`, dentro de um *namespace* exportado. Para esconder um detalhe: simplesmente não o exporte.

XII.5 Síntese

As quatro seções defendem a mesma tese por caminhos diferentes: **informação que está no tipo é verificada; informação que está numa convenção, não**.

Os tipos de vocabulário movem para o tipo aquilo que antes era combinado por comentário — "pode ser nulo", "o segundo parâmetro é o tamanho do primeiro", "só leia o membro que a etiqueta indicar". O ganho é que o compilador passa a poder recusar o uso errado; o custo é aprender cinco tipos e as armadilhas de tempo de vida de dois deles.

A computação em tempo de compilação move para o compilador aquilo que antes era feito na inicialização — tabelas, validação de literais, ordenação. O ganho maior não é o tempo economizado na execução: é que o avaliador de expressões constantes recusa UB (comportamento indefinido), transformando uma classe inteira de defeitos em erro de *build*. O custo é tempo de compilação, que é real e mensurável.

O subprojeto de C++23 acrescenta uma lição de método que vale além do assunto: **meça em vez de supor**. Seis entradas de uma tabela escrita a partir de documentação estavam erradas, todas subestimando o que já existia. E a verificação não termina na *feature-test macro*: o `std::stacktrace` está declarado, tem *header*, e não linka.

Os módulos são o único dos quatro assuntos em que a recomendação prática é esperar. A ideia resolve os cinco problemas do `#include` de uma vez, e a implementação nos compiladores existe; o que ainda não existe, de forma madura, é o suporte dos sistemas de *build* ao problema de descobrir a ordem de compilação antes de compilar. Os dois defeitos do GCC 13 documentados aqui — o `export` que some com o *global module fragment* e o `constexpr` sem `inline` — são sintomas dessa juventude.

XIII Texto, tempo e bits

Três assuntos que parecem não ter nada em comum ocupam este capítulo: cadeias de caracteres, medição de tempo e manipulação de bits. Eles estão juntos por uma razão precisa — são os três lugares em que o programador acredita saber o que está acontecendo e **está errado**. Concatenar duas *strings* parece grátis e aloca memória quatro vezes; cronometrar um trecho parece trivial e mede o relógio em vez do código; inverter os bits de um byte parece produzir outro byte e produz um inteiro negativo de 32 bits.

O capítulo percorre os subprojetos `src/strings/`, `src/chrono/`, `src/bits/` e `src/file_io/`. O último é a saída natural dos três primeiros: texto formatado, instantes registrados e bytes serializados precisam, no fim, chegar a um arquivo.

Por que — por que estes três assuntos merecem medição, e não afirmação

Em quase todo o resto do C++ o erro é detectável: o compilador recusa, o *sanitizer* aborta, o teste falha. Nos três assuntos deste capítulo o erro é **quantitativo**. Um programa que aloca quatro vezes onde poderia alocar uma vez continua correto; um *benchmark* que mede o compilador em vez do algoritmo continua produzindo números; uma `regex` que custa cem vezes mais que um `find` continua devolvendo a resposta certa. A única forma de descobrir a diferença é instrumentar e medir — e é isso que os subprojetos fazem, contando alocações de verdade e cronometrando com o relógio certo.

Vale fixar de saída um vocabulário mínimo, porque ele reaparece em todas as seções.

Alocação. O pedido de memória ao sistema de tempo de execução, feito por `operator new` ou `malloc`. Ela custa muito mais que uma atribuição: envolve procurar um bloco livre, atualizar metadados e, eventualmente, pedir mais memória ao sistema operacional. Contar alocações é, por isso, uma métrica melhor que cronometrar — é determinística.

Heap. A região de memória de onde vêm as alocações dinâmicas, em oposição à *pilha*, que é a região onde vivem as variáveis locais e cujo espaço é reservado e devolvido pelo simples avanço e recuo de um registrador.

Linha de cache. A unidade mínima que o processador traz da memória principal para a memória rápida interna — tipicamente 64 bytes. Ler um byte custa o mesmo que ler os 64 da linha; por isso dados vizinhos são quase gratuitos e dados espalhados custam uma espera por elemento.

Unidade de tradução. Um arquivo `.cpp` depois de o pré-processador ter inserido todos os cabeçalhos que ele inclui. É o que o compilador vê de cada vez, e a razão de o subprojeto `src/strings/` precisar declarar o `operator new` instrumentado em uma unidade só.

XIII.1 Texto

O subprojeto `src/strings/` abre com três números que resumem o problema inteiro:

```
strings de até 15 chars            →  0 alocações (SSO)
200 chars, um a um, sem reserve    →  4 alocações
extrair um campo de log:
  find + from_chars    →   0 ms
  std::regex           → 133 ms      ← mesmo com a regex fora do laço
```

Listagem 388: Saída de `src/strings/`: o resumo do subprojeto.

As três linhas dizem respectivamente que texto curto é barato, que texto montado sem planejamento é caro, e que a ferramenta genérica de casamento de padrões custa duas ordens de grandeza mais que o código específico. As subseções seguintes explicam cada uma.

XIII.1.1 `std::string` por dentro: a SSO, medida

O primeiro problema que `std::string` resolve é a posse: quem libera a memória de uma sequência de caracteres de tamanho variável. A resposta é RAII — o construtor aloca, o destrutor libera — e ela custa uma alocação por *string*. Para nomes, chaves de dicionário e identificadores, que são curtos e existem aos milhares, essa alocação seria ruinosa.

A solução das implementações é a **SSO**, sigla de *Small String Optimization*: o objeto `string` reserva alguns bytes dentro de si mesmo e guarda ali as *strings* curtas, sem tocar na *heap*. O detalhe é que o padrão não obriga nem documenta o limite — ele é uma decisão de implementação. O subprojeto, então, mede:

```
1  sizeof(std::string) = 32 bytes
2   1 chars → 0 alocações  ← cabe no SSO
3  15 chars → 0 alocações  ← cabe no SSO
4  16 chars → 1 alocação
```

Listagem 389: Saída de `src/strings/`: o limite da SSO na libstdc++.

A medição é feita substituindo o `operator new` global por uma versão que incrementa um contador — a mesma técnica de diagnóstico usada em Seção III.4. E há uma segunda prova, estrutural: numa *string* curta, o endereço devolvido por `.data()` fica 16 bytes depois do endereço do próprio objeto, ou seja, *dentro* dele; numa *string* longa, fica a uma distância arbitrária, na *heap*.

Três consequências práticas decorrem disso, e a segunda contraria a intuição corrente:

- *strings* curtas são quase de graça — não evite `std::string` em nomes, chaves e identificadores por medo de alocação;
- mas o move de uma *string* curta **copia** os bytes: não há ponteiro a roubar, porque os dados estão dentro do objeto. O *move* de `string` só é $O(1)$ *acima* do limite da SSO;
- `sizeof(std::string)` é 32 bytes, então um `vector<string>` de um milhão de elementos gasta 32 MB apenas de cabeçalhos, antes de qualquer caractere.

Armadilha — o *move* barato de `string` não é sempre barato

A regra de bolso "mover é $O(1)$, copiar é $O(n)$" vale para `std::vector` sem exceção e para `std::string` apenas acima de 15 caracteres. Abaixo disso, mover e copiar custam exatamente o mesmo — uma cópia de 16 bytes. Isso não é motivo para deixar de mover; é motivo para não *esperar* ganho ao mover uma coleção de *strings* curtas, e para desconfiar de um perfil que mostra `std::string` caro sem que haja alocação envolvida.

XIII.1.2 `capacity`, crescimento e `reserve`

`size()` é quantos caracteres existem; `capacity()` é quantos cabem antes de a *string* precisar de um *buffer* maior. Quando a capacidade acaba, a implementação aloca um bloco maior, copia o conteúdo e libera o antigo — e a política de crescimento é a mesma do `vector` (Seção III.1): dobrar.

```
crescendo char a char: 15→30 30→60 60→120 120→240     ← dobra, igual ao vector
200 chars sem reserve → 4 alocações
200 chars com reserve → 1 alocação
```

Listagem 390: Saída de `src/strings/`: o mesmo laço, com e sem `reserve`.

Repare que a sequência começa em 15, e não em zero: a capacidade inicial de uma *string* vazia já é o *buffer* da SSO, que existe de qualquer maneira dentro do objeto.

Assim como no `vector`, `clear()` zera o tamanho e **mantém** a capacidade, e `shrink_to_fit()` é uma requisição não vinculante. O idioma que funciona sempre é o mesmo: `std::string{}.swap(s);`.

XIII.1.3 Concatenação: por que `+` custa mais

A expressão `a + b + c + d` não é uma operação com quatro argumentos: é `((a + b) + c) + d`, e cada `+` produz uma *string* **temporária** nova. Numa cadeia com N trechos, isso tende a $N - 1$ objetos intermediários.

Na prática o número não explode, e a razão é interessante: a libstdc++ sobrecarrega `operator+` para o caso em que o operando da esquerda é um categoria de valor *rvalue* — isto é, um temporário — e nesse caso reaproveita o *buffer* dele em vez de alocar outro. O subprojeto conta as alocações das quatro formas de montar o mesmo texto e mostra que a diferença existe, mas é menor do que a análise ingênua prevê.

Regra — para N trechos, `reserve` mais `append`

Quando o tamanho final é conhecido ou estimável, `reserve` seguido de `append` (ou de `+=`) faz exatamente uma alocação. Para texto formatado, `std::format` e `fmt::format` já fazem isso internamente, e `format_to` escreve direto num destino existente — ver a Seção XIII.1.6.

XIII.1.4 O que `std::string` não sabe

`std::string` é `basic_string<char>`: uma sequência de **bytes**. Ela não sabe o que é um caractere Unicode, e `size()` conta bytes:

```
"cafe".size() = 4     (4 letras, 4 bytes)
"café".size() = 5     (4 letras, 5 BYTES)
```

Listagem 391: Saída de `src/strings/`: bytes não são letras.

O acento em UTF-8 ocupa dois bytes (`C3 A9`), e a consequência atinge todas as operações posicionais: `s[i]` pode cair no meio de um caractere, `substr` pode cortá-lo ao meio, e `std::reverse` sobre UTF-8 produz lixo. Comparar sem normalização Unicode também dá resultados errados, porque a mesma letra acentuada tem mais de uma representação válida.

Armadilha — a biblioteca padrão não resolve Unicode, e finge que sim

Os tipos `u8string`, `u16string` e `u32string` apenas marcam a intenção: nenhum deles implementa normalização, segmentação de *grafemas* ou comparação sensível a idioma. O cabeçalho `<codecvt>`, que prometia conversão entre codificações, foi **depreciado no C++17** e nunca ganhou substituto. Para Unicode de verdade a resposta está fora da biblioteca padrão: ICU, utf8cpp ou Boost.Locale.

O ponto de extensão que existe é o `char_traits`, o segundo parâmetro de `basic_string`, que define comparação, cópia e busca. Trocá-lo permite, por exemplo, uma *string* insensível a maiúsculas sem alocar — mas o resultado é um **tipo diferente**, que não converte para `std::string`. Quase sempre é melhor normalizar os dados na entrada.

XIII.1.5 `std::string_view`: a janela que não possui

Por que — que problema o *view* resolve

Uma função que aceita `const std::string&` obriga quem tem um `const char*` — um literal, um *buffer* de rede, um trecho de outra *string* — a **construir uma `std::string` inteira** só para passar o argumento. É uma alocação e uma cópia por chamada, para uma função que só vai ler. `std::string_view` é o par (ponteiro, tamanho) que descreve caracteres pertencentes a outra pessoa: converte implicitamente de tudo o que já é contíguo, e não aloca nada.

O ganho é duplo. Na chamada, `contar_vogais(literal)` deixa de alocar. E nas operações de recorte: `str.substr(4,5)` constrói uma *string* nova, enquanto `view.substr(4,5)` apenas ajusta o ponteiro e o tamanho — é $O(1)$ e não toca na *heap*.

O preço é que o *view* não possui nada, e disso decorrem duas famílias de armadilhas.

A ausência do terminador nulo. Uma *C-string* é uma sequência de bytes terminada por um byte zero (`'\0'`); é assim que `printf`, `fopen` e toda API C descobrem onde o texto acaba. Um *view* não tem esse byte: ele é um recorte de outra coisa, e o byte seguinte ao último caractere pertence à origem.

```
parcial = "abc" (size=3)
o que printf("%s", parcial.data())  leria :  'abcdefghij'      ← foi até o fim da ORIGEM
```

Listagem 392: Saída de `src/strings/`: o que uma API C leria.

No exemplo do subprojeto a origem é uma `std::string`, que por acaso termina em nulo, e a leitura para em dez caracteres. Se a origem fosse um *buffer* binário, a leitura seguiria memória adentro até encontrar um zero qualquer — leitura fora de limites, que o AddressSanitizer (Seção I.9) detectaria.

Tabela 149: As três formas corretas de levar um *view* a uma API que espera C-string.

Forma	Como
(a) copiar	`std::string{v}.c_str()` — aloca, mas é correto
(b) passar o par	`printf("%.*s", (int)v.size(), v.data())`
(c) garantir na origem	assegurar que o *view* cobre até um `'\0'`

Regra — `.data()` de um *view* nunca vai direto para uma API C

A assinatura de `.data()` é a mesma de `std::string::data()` — `const char*` — e é justamente por serem indistinguíveis que o erro passa na revisão. Se a função de destino não recebe também o tamanho, o *view* precisa virar `std::string` antes.

Tempo de vida. O *view* é tão frágil quanto um ponteiro, porque *é* um ponteiro. Os três erros clássicos são: apontar para um temporário que morre no fim da expressão; devolver um *view* para uma variável local; e guardar um *view* como membro sem documentar quem é o dono do texto.

```cpp
class Documento {
public:
    explicit Documento(std::string conteudo) : conteudo_(std::move(conteudo)) {}

    // Seguro: o view aponta para o membro, cuja vida é a do Documento
    std::string_view titulo() const {
        const auto fim = conteudo_.find('\n');
        return std::string_view{conteudo_}.substr(0, fim);
    }

private:
    std::string conteudo_;
};
```

Listagem 393: O caso seguro em `src/strings/demo_string_view.hpp`: o *view* aponta para um membro.

Regra — as três regras de tempo de vida de um *view*

(1) *view* é **parâmetro** — o uso dominante, e sempre seguro, porque o argumento vive até o fim da expressão que contém a chamada. (2) *view* como **retorno** só se a origem sobrevive à chamada, como no `Documento` acima. (3) *view* como **membro** exige documentar quem é o dono — e, quase sempre, é melhor guardar uma `std::string`.

As conversões, e a que custa. `string` → `string_view` é implícita e barata. `string_view` → `string` é **explícita**, porque aloca: `std::string t = v;` não compila, e `std::string t{v};` compila. O desenho é intencional — a conversão cara precisa ser visível no código.

Há, porém, uma conversão implícita que custa: `const char*` → `string_view` chama `strlen`, que é $O(n)$. Num laço quente com literais, isso se paga a cada iteração. A saída é o literal `sv` do C++17, cujo tamanho é `constexpr` / `consteval`:

```cpp
    using namespace std::string_view_literals;
    constexpr auto literal_sv = "tamanho conhecido na compilação"sv;

    // ! 33, não 31: a frase tem 31 CARACTERES, mas o 'ç' e o 'ã' ocupam 2
    //   bytes cada em UTF-8. `size()` conta bytes — é a mesma lição de
    //   demo_string.hpp, seção 1.5, e aqui ela foi pega por um static_assert.
    static_assert(literal_sv.size() == 33);
```

Listagem 394: Transcrição de `src/strings/demo_string_view.hpp`: o literal cujo tamanho é conhecido em compilação.

O comentário registra um caso em que a medição contrariou a expectativa do próprio autor: o `static_assert` foi escrito com 31, o valor que a contagem de letras sugere, e o compilador recusou. São 33 bytes, porque `ç` e `ã` ocupam dois cada — exatamente a lição da subseção anterior, aplicada a quem já a tinha escrito.

Tabela 150: Escolhendo a assinatura de um parâmetro de texto.

Assinatura	Quando
`std::string_view`	só **lê**, e não guarda — o padrão
`const std::string&`	precisa de `.c_str()` garantido
`std::string` por valor	vai **guardar** uma cópia (mais `std::move`)
`std::string&`	vai **modificar** a do chamador

XIII.1.6 `std::format`: formatação verificada em compilação

Por que — por que uma terceira forma de formatar

O C++ tinha duas, e as duas eram ruins. O `printf` é conciso e rápido, e não tem segurança de tipo: `printf("%d", 1.5)` compila e imprime lixo, `printf("%s", std::string{})` compila e é UB (comportamento indefinido), e `printf("%d %d", 1)` lê a pilha. Os *iostreams* são seguros e verbosíssimos, e seus manipuladores são **estado global e pegajoso**: um `std::hex` esquecido contamina toda a saída seguinte. `std::format` (C++20) tem a sintaxe do primeiro, a segurança do segundo e uma propriedade que nenhum dos dois tem — a *string* de formato é validada na **compilação**.

O subprojeto usa `fmt::format`, a biblioteca que originou a proposta, porque é o que já está no `conanfile.py` do repositório (Seção I.6); a API é a mesma.

O que a validação pega. As três linhas abaixo não compilam:

```
1 fmt::format("{:d}", "texto")    // NÃO COMPILA — 'd' não se aplica a texto
2 fmt::format("{} {}", 1)         // NÃO COMPILA — dois campos, um argumento
3 fmt::format("{", 1)             // NÃO COMPILA — chave não fechada
```

Listagem 395: Erros de formato que viram erro de *build*.

O mecanismo é o `consteval` (Seção IV.3): a *string* de formato é analisada em tempo de compilação, e um `throw` durante uma avaliação constante é, por definição, erro de *build*. Não há custo em tempo de execução — a análise já aconteceu.

Armadilha — a validação pega o autor deste livro também

O demo original continha um `{:.1%}` para imprimir uma porcentagem. O `%` não é especificador válido para `double` nem na fmt 10 nem no `std::format` do C++20, e o **arquivo não compilou**. É o comportamento anunciado funcionando contra quem o anunciou: com `printf`, o mesmo erro teria produzido saída errada em silêncio. A porcentagem se faz multiplicando por 100 antes e usando `{:.1f}%`.

A mini-linguagem. A forma geral de um campo de substituição é `{índice:preenchimento alinhamento sinal # 0 largura .precisão tipo}`. Vale memorizar os grupos, não a gramática:

```
1 {:<10}  {:>10}  {:^10}  {:*^10}     alinhamento e preenchimento
2 {:08.3f}  {:+}  {:.2e}  {:g}        números
3 {:b}  {:#b}  {:o}  {:#x}  {:#X}     bases
4 {0} {1} {0}                         índices explícitos e reuso
5 {:{}.{}f}                           largura/precisão dinâmicas
```

Listagem 396: Saída de `src/strings/`: os grupos da especificação.

Duas capacidades merecem nota porque não têm equivalente em `printf`: o **reuso de argumento** por índice explícito (`{0} {1} {0}` imprime o primeiro argumento duas vezes, sem passá-lo duas vezes) e a formatação de tipos da biblioteca padrão — `vector`, `pair`, `map` — que na fmt vem de `<fmt/ranges.h>` e no padrão chegou no C++23.

Formatar sem alocar. `format` devolve uma `std::string` nova, portanto aloca. Quando isso importa, a família tem variantes que escrevem num destino existente:

Tabela 151: As quatro portas de saída da biblioteca de formatação.

Função	Destino	Aloca
`format`	`std::string` nova	sim
`format_to`	qualquer iterador de saída	não*
`format_to_n`	idem, com limite de bytes	não
`print`	*stdout* direto	não

* depende do destino: um `back_inserter` sobre um `vector` pode realocar.

`format_to_n` escrevendo num vetor de `char` na pilha é o equivalente seguro do `snprintf`; `fmt::print` escreve direto no *stdout* sem construir a *string* intermediária, e é por isso que ele aparece em todo o repositório em vez de `std::cout << fmt::format(...)`.

Um `formatter` para o seu tipo. Ensinar a biblioteca a formatar um tipo próprio é especializar `std::formatter<T>` (ou `fmt::formatter<T>`) com dois métodos: `parse`, que consome os especificadores entre `:` e `}`, e `format`, que escreve no iterador de saída.

```
template <>
struct fmt::formatter<demo_format::Temperatura> {
    char unidade = 'C';

    constexpr auto parse(format_parse_context& ctx) {
        auto it = ctx.begin();
        if (it != ctx.end() && (*it == 'C' || *it == 'F' || *it == 'K')) {
            unidade = *it++;
        }
        if (it != ctx.end() && *it != '}') {
            throw format_error("Temperatura aceita apenas C, F ou K");
        }
        return it;
    }

    auto format(const demo_format::Temperatura& t, format_context& ctx) const {
        switch (unidade) {
            case 'F': return fmt::format_to(ctx.out(), "{:.1f}°F", t.celsius * 9 / 5 + 32);
            case 'K': return fmt::format_to(ctx.out(), "{:.2f}K", t.celsius + 273.15);
            default: return fmt::format_to(ctx.out(), "{:.1f}°C", t.celsius);
        }
    }
};
```

Listagem 397: Transcrição de `src/strings/demo_format.hpp`: um *formatter* com especificadores próprios.

O detalhe decisivo é o `constexpr` no `parse`: como a análise acontece em tempo de compilação, o `throw` daquele terceiro `if` não é uma exceção em tempo de execução — é um **erro de compilação**. `fmt::print("{:F}", t)` imprime `77.0°F`; `"{:X}"` não compila. O seu tipo ganha a mesma verificação que os tipos nativos têm.

O atalho, quando o tipo é essencialmente um número, é herdar de um *formatter* existente: `struct formatter<Metros> : formatter<double>` reaproveita toda a mini-linguagem, e `{:8.2f}` passa a funcionar de graça.

O custo, e a lição que vale mais que ele. O subprojeto cronometra quatro formas de produzir a mesma linha, cem mil vezes — e o resultado **não** mostra a fmt ganhando:

```
−O0: snprintf 18ms · format_to_n 78ms · format 91ms
−O2: snprintf 18ms · format_to_n 53ms · format 57ms
```

Listagem 398: Saída de `src/strings/`: 100 000 formatações, nas duas configurações de compilação.

O demo explica por quê, em vez de esconder: este repositório linka `libfmtd`, a variante de **depuração** que o Conan compilou com `build_type=Debug`. Compilar o arquivo do demo com `-O2` quase não muda o quadro, porque o custo está dentro da biblioteca — que já foi compilada. O `snprintf`, por sua vez, vem da glibc, sempre otimizada.

Regra — um número sem as *flags* ao lado é um boato

Três exigências decorrem do caso acima, e nenhuma delas é sobre a fmt. (1) Saiba em que modo a **biblioteca** foi compilada, não só o seu código. (2) Compare pares que diferem em **uma** variável só — aqui, a única comparação controlada é `format_to_n` contra `format`, que diferem apenas no destino e isolam o custo de alocar uma *string* por iteração. (3) Desconfie de qualquer *benchmark* cujo autor não declara as *flags* — inclusive deste.

E há o argumento que não depende de medição alguma: `snprintf` não tem segurança de tipo, e o `ostringstream` carrega estado global pegajoso. `format` dá as duas coisas.

XIII.1.7 `<charconv>`: texto e número sem alocar e sem locale

Converter `"42"` em `42` parece trivial, e o C++ acumulou cinco formas de fazê-lo — todas com algum defeito.

Tabela 152: As cinco formas de converter texto em número, e o defeito de cada uma.

Forma	Defeito
`atoi`	sem detecção de erro — `"abc"` devolve 0, igual a `"0"`
`strtol`	detecta, via a variável global `errno`
`stoi`	**lança** — caro no caminho de erro
`stringstream`	aloca, e respeita o *locale*
`from_chars`	—

Locale é a configuração global de convenções culturais do processo: separador decimal, separador de milhar, nomes de meses. É definido por `setlocale` e afeta `printf`, os *iostreams* e as funções `sto*` — inclusive quando quem o mudou foi uma biblioteca de terceiros na inicialização.

`from_chars`. A assinatura recebe um intervalo `[primeiro, último)` de `char` e uma referência ao destino, e devolve uma *struct* com dois campos: `ptr`, onde a conversão parou, e `ec`, um `std::errc`. Os casos:

```
'42'              → 42 ✓
'3.14'            → 3 — mas SOBROU '.14'      ← o ptr permite detectar
'42abc'           → 42 — mas SOBROU 'abc'     ← stoi devolveria 42 calado
' 42'             → invalid_argument          ← não pula espaços
'99999999999999' → result_out_of_range
```

Listagem 399: Saída de `src/strings/`: `from_chars` sobre entradas típicas.

O `ptr` devolvido é a diferença prática mais importante: `stoi("42abc")` devolve 42 sem reclamar, e não há como o chamador descobrir que sobrou lixo. Com `from_chars`, comparar `ptr` com o fim do intervalo responde exatamente isso. E é o mesmo `ptr` que realimenta o laço quando se *parseia* uma sequência inteira — o padrão que sustenta os *parsers* rápidos de JSON e CSV.

A lista do que `from_chars` **não** faz é a lista de suas qualidades: não pula espaços, não aceita o prefixo `0x` (a base vai por parâmetro), não aceita `+` inicial, não aloca, não lança e não olha o *locale*. Cada "não" é uma política devolvida a quem chama, que passa a pagar por ela só quando precisa.

`to_chars` e o *round-trip* garantido. Na direção inversa há uma garantia que nenhuma alternativa oferece: chamado **sem** especificar precisão, `to_chars` produz a **menor** *string* que faz `from_chars` devolver exatamente o mesmo `double`.

```
0.1                 → '0.1'                 ✓ round-trip exato
0.3333333333333333  → '0.3333333333333333'  ✓
123456789.12345679  → '123456789.12345679'  ✓
```

Listagem 400: Saída de `src/strings/`: ida e volta exata.

O algoritmo por trás disso (Ryū / Grisu) resolve um problema real: `printf("%g")` usa seis dígitos significativos e perde precisão, e `printf("%.17g")` sempre imprime dezessete, produzindo `0.10000000000000001` para `0.1`. Onde o número precisa voltar idêntico depois de virar texto — serialização JSON, telemetria, formatos textuais — essa é a ferramenta.

Armadilha — o *locale* corrompe dados em silêncio

Com o *locale* global em `pt_BR` ou `de_DE`, o separador decimal é a vírgula:

```
ostringstream << 3.14  → "3,14"   ← VÍRGULA
stod("3.14")           → 3.0      ← para no ponto, e não reclama
```

O bug completo tem quatro passos: o programa grava um CSV numa máquina `en_US`, outra máquina em `pt_BR` lê o mesmo arquivo, `stod("3.14")` devolve 3.0 sem erro, e o dado fica errado para sempre. E basta uma biblioteca de terceiros chamar `setlocale(LC_ALL, "")` para o comportamento do *seu* código mudar. `<charconv>` é imune por projeto: não tem parâmetro de *locale*, e o separador é sempre o ponto.

Regra — *locale* é apresentação, não dado

Para **dados** — arquivos, rede, protocolos, formatos de intercâmbio — use `<charconv>`. Para **apresentação** ao usuário, use `std::format` com o especificador `{:L}`, que pede a formatação sensível a *locale* *explicitamente*, no ponto em que ela é desejada.

O subprojeto também cronometra as três formas convertendo a mesma `std::string` no mesmo `int`, e aqui a comparação *é* controlada. As diferenças são estruturais e não dependem de *build*: `stoi` constrói uma *string* temporária internamente e usa `strtol` com `errno`; `istringstream` constrói um objeto de *stream* inteiro por conversão, com *locale*, *sentry* e *flags*; `from_chars` é um laço sobre os dígitos. Há ainda um custo que o cronômetro não vê: `stoi` **lança** em entrada inválida, e num *parser* que encontra lixo com frequência cada erro custa um desempilhamento completo — medido em Seção II.8 como cerca de mil vezes mais caro que o caminho feliz.

XIII.1.8 `std::regex`: o custo real

Uma expressão regular é uma notação para descrever conjuntos de cadeias de caracteres. `std::regex` chegou no C++11 e é, com folga, a parte mais criticada da biblioteca padrão: a API é razoável, a implementação é notoriamente lenta.

`match` contra `search`. É o erro número um, e vem de quem conhece outras linguagens:

Tabela 153: As três formas de aplicar uma expressão regular.

Função	Casa
`regex_match`	a *string* **inteira**
`regex_search`	a primeira ocorrência, em qualquer posição
`sregex_iterator`	todas as ocorrências

Em Python, `re.match` casa no *início* da *string*; em C++, `regex_match` exige a *string* toda. Sobre `"pedido 42 entregue"` com o padrão `\d+`, `regex_match` devolve falso e `regex_search` devolve verdadeiro.

Capturas. Grupos entre parênteses são recuperados por um objeto `std::smatch`: `m[0]` é o casamento inteiro, `m[1]` em diante são os grupos, e `m.prefix()` e `m.suffix()` são o texto antes e depois. Em `regex_replace`, o formato de substituição usa `$1` a `$9` para os grupos, `$&` para o casamento inteiro e `$$` para um cifrão literal.

Armadilha — `smatch` guarda iteradores, não cópias

Um `std::smatch` não copia o texto: ele guarda **iteradores** para a *string* original. Se ela morrer, o `smatch` fica pendurado, e o caso mais comum é fatal:

```
1 std::smatch m;
2 std::regex_search(obterTexto(), m, re);   // ✗ argumento temporário
3 m[1].str();                               // use-after-free
```

O comitê considerou o caso grave o bastante para **deletar** a sobrecarga que recebe `const std::string&&`: a primeira linha nem compila. É um dos poucos lugares da biblioteca padrão em que um erro de tempo de vida virou erro de compilação por decreto.

Duas regras de escrita. Sempre use *raw string* para o padrão — `R"(\d+)"` em vez de `"\\d+"` —, porque sem ela o compilador processa os escapes antes de a expressão chegar à biblioteca. E nunca construa uma `std::regex` dentro de um laço: o construtor *compila* a expressão, e isso é caro.

O custo, medido. O subprojeto extrai o campo `codigo=(\d+)` de uma linha de *log* de três formas:

```
1 find + from_chars    :   0 ms   (20000 iterações)
2 regex fora do laço   : 133 ms   (20000 iterações)
3 regex DENTRO do laço : 176 ms   ( 2000 iterações — 10× MENOS trabalho!)
```

Listagem 401: Saída de `src/strings/`: o mesmo campo, três formas.

São duas lições distintas, e a segunda é a que surpreende. A primeira: construir a expressão dentro do laço domina tudo — a terceira linha fez **dez vezes menos trabalho** e ainda assim levou mais tempo que a segunda. A segunda: mesmo pré-construída, para um padrão **fixo e simples** a expressão regular não se paga; um `find` seguido de `from_chars` resolve o mesmo problema num tempo que o cronômetro em milissegundos nem registra.

Armadilha — ReDoS: entrada de terceiros com padrão de terceiros

`std::regex` usa *backtracking* — ao falhar, volta atrás e tenta outra combinação. Para padrões com quantificadores aninhados, como `(a+)+$`, o número de combinações cresce **exponencialmente** com o tamanho da entrada, e uma cadeia de algumas dezenas de caracteres trava o processo. É o ataque

Tabela 154: Quando a expressão regular é a ferramenta certa.

Use `std::regex` quando	Não use quando
o padrão vem do **usuário**	laço quente com padrão fixo
a expressão é complexa e roda uma vez	o formato é conhecido: escreva o *parser*
legibilidade importa mais que velocidade	precisa de desempenho: RE2, PCRE2, CTRE

conhecido como ReDoS. A regra é dupla: nunca aplique uma expressão vinda de terceiros a uma entrada vinda de terceiros; e, se o padrão é configurável pelo usuário, prefira RE2, que garante tempo linear por construção.

XIII.2 Tempo

Por que — por que `<chrono>` não é "pegar a hora"

A API C de tempo é um conjunto de inteiros sem unidade: `time_t` conta segundos, `clock_t` conta tiques de origem indefinida, e `sleep(1)` dorme um segundo enquanto `usleep(1)` dorme um microssegundo. Nada no tipo distingue os dois, e passar um pelo outro é um bug de fator um milhão que compila. `<chrono>` é a resposta: um **sistema de unidades com verificação em compilação**, aplicado ao tempo. Somar horas com milissegundos funciona e dá o resultado certo; somar horas com o número 5 não compila, porque 5 não é uma quantidade de tempo.

XIII.2.1 `duration`: a unidade vive no tipo

A declaração é:

```
1 template <class Rep, class Period = std::ratio<1>>
2 class duration;
```

Listagem 402: A forma geral de uma duração.

`Rep` é o tipo que guarda a contagem — `int64_t`, `double`; `Period` é a fração de **segundo** que cada tique representa, expressa como um `std::ratio`, que é um par de inteiros (numerador e denominador) codificado no tipo. Assim, `std::chrono::milliseconds` é `duration<int64_t, ratio<1,1000>>`: o "milli" está no **tipo**, não em uma variável.

```
1 nanoseconds    Rep=int64_t  Period=1/1000000000
2 milliseconds   Rep=int64_t  Period=1/1000
3 hours          Rep=int64_t  Period=3600/1
4 sizeof(milliseconds) = 8 bytes      ← é só o int64_t
```

Listagem 403: Saída de `src/chrono/`: as unidades e o que elas custam.

Toda a segurança dimensional custa **zero** em tempo de execução: o objeto é do tamanho do inteiro que ele guarda, e as conversões viram multiplicações por constantes que o compilador resolve.

O método `.count()` extrai o número cru e **descarta a unidade**. Ele é a saída do sistema de tipos, e deve aparecer só na fronteira — ao imprimir, ao gravar, ao chamar uma API C. Usá-lo no meio de um cálculo devolve o problema que a biblioteca resolve.

XIII.2.2 Conversões: implícitas quando são exatas

A regra é uma só, e é o coração do desenho: **a conversão é implícita quando não pode perder informação**.

```
2h → milliseconds      ✓ implícito (exato)
1500ms → seconds       ✗ NÃO COMPILA (truncaria)
duration_cast<seconds>(1500ms) = 1 s      ← trunca em direção a ZERO
```

Listagem 404: Saída de `src/chrono/`: o que o compilador aceita e o que recusa.

Converter para uma unidade *menor* multiplica por um inteiro e é sempre exato; converter para uma unidade *maior* pode truncar, e o compilador exige que a perda seja escrita à mão. Quando truncar não é a política desejada, existem `floor`, `ceil` e `round`, todos do C++17.

Armadilha — para negativos, truncar e arredondar para baixo são coisas diferentes

`duration_cast` trunca **em direção a zero**; `floor` arredonda **para baixo**. Para valores positivos os dois coincidem, e é por isso que a diferença passa despercebida em teste:

```
duration_cast<seconds>(−1500ms) = −1 s
floor<seconds>(−1500ms)         = −2 s
```

Em cálculo de fuso horário, de compensação de deriva ou de qualquer diferença que possa ser negativa, escolher o errado produz um segundo de erro que só aparece em metade dos casos.

XIII.2.3 Aritmética e unidades derivadas

Somar durações de unidades diferentes produz o *tipo comum*: a unidade mais fina que representa exatamente as duas.

```
1h + 30min + 45s = 5445 s      ← tipo comum: seconds
1s + 500ms       = 1500 ms     ← tipo comum: milliseconds
1h / 15min       = 4           ← escalar puro, sem unidade
1h * 1h          ✗ tempo² não é um duration
1h + 5           ✗ 5 do quê?
```

Listagem 405: Saída de `src/chrono/`: a álgebra das durações.

A terceira linha é a mais instrutiva: dividir tempo por tempo produz um número **sem unidade**, e o tipo do resultado é um inteiro, não uma `duration`. É a álgebra dimensional correta, obtida de graça pelo sistema de tipos.

`Rep` não precisa ser inteiro. Com `double`, converter para uma unidade maior deixa de perder informação e passa a ser implícito — `duration<double> s = 1500ms;` compila. A troca é a exatidão: para contabilidade de tempo (agendamento, *timeouts*), prefira inteiros; para *relatar* medições, `double` é mais legível.

Definir uma unidade nova é uma linha, e ela participa de toda a aritmética e de todas as verificações automaticamente:

```
    using Frame60 = ch::duration<int64_t, std::ratio<1, 60>>;
    using Frame24 = ch::duration<int64_t, std::ratio<1, 24>>;

    const Frame60 um_segundo_de_video = 1s;
```

Listagem 406: Transcrição de `src/chrono/demo_duration.hpp`: quadros de vídeo como unidade de tempo.

`1s` converte implicitamente para 60 quadros, porque a conversão é exata; converter de 60 fps para 24 fps exige `duration_cast`, porque trunca. O compilador aplica a mesma regra a uma unidade que ele nunca viu.

XIII.2.4 Os relógios, e por que escolher o errado é um bug

A diferença entre os relógios da biblioteca não é de precisão — é de **garantia**. Um relógio *monotônico* nunca anda para trás; um relógio de *parede* corresponde à hora do mundo e, por isso mesmo, pode ser ajustado — pelo NTP, pelo usuário, pelo horário de verão.

Tabela 155: Os três relógios clássicos e para que serve cada um.

Relógio	Monotônico	É hora real	Usar para
`system_clock`	**não**	sim	**datar**
`steady_clock`	**sim**	não	**medir**
`high_resolution_clock`	depende!	depende!	**nada**

O bug que a distinção previne é este: com `system_clock`, um ajuste do NTP entre as duas leituras faz a diferença ser **negativa**. Com `steady_clock`, isso é impossível por construção.

Armadilha — `high_resolution_clock` é um apelido, e neste ambiente é o errado

O padrão permite que `high_resolution_clock` seja um *alias* para outro relógio, e não diz qual. O subprojeto mede:

```
é o mesmo que system_clock? true        ← libstdc++
high_resolution_clock::is_steady = false
```

Ou seja: na libstdc++, o relógio cujo nome sugere "o melhor para medir" é justamente o **não monotônico**. Todo código que mede duração com ele neste compilador está sujeito a resultado negativo. E o mapeamento muda: libstdc++ aponta para `system_clock`, libc++ e MSVC apontam para `steady_clock`. O mesmo código muda de comportamento ao trocar de compilador, e o bug não aparece em teste — só quando o relógio do sistema é ajustado. A recomendação, que é também a do cppreference, é não usá-lo nunca.

Resolução do tipo contra resolução real. O `period` de um relógio diz a resolução que o **tipo** é capaz de representar. A resolução *efetiva* — o menor intervalo que duas leituras consecutivas conseguem distinguir — é outra coisa, e se mede:

```
system_clock   tipo: 1/1000000000 s  |  resolução medida: 10 ns
custo de steady_clock::now(): ~19 ns por chamada
```

Listagem 407: Saída de `src/chrono/`: o tipo promete nanossegundos; a realidade é outra.

Chamar `now()` custa cerca de dezenove nanossegundos. Medir algo que dura menos que isso é **medir o relógio** — e é a razão de existir a Seção XIII.2.7.

XIII.2.5 `time_point`: instantes, e a álgebra que faz sentido

A distinção que organiza a biblioteca inteira é entre *quanto tempo* (`duration`) e *quando* (`time_point`). Um `time_point` é, por dentro, apenas uma `duration` contada desde a ***epoch*** do seu relógio — o instante

de referência a partir do qual ele conta. Para `system_clock`, o C++20 garante que a *epoch* é 1970-01-01 00:00:00 UTC; para `steady_clock`, é um ponto arbitrário, tipicamente o momento em que a máquina ligou.

Tabela 156: A álgebra de instantes e durações.

Expressão	Resultado
`time_point - time_point`	`duration` — "quanto durou"
`time_point + duration`	`time_point` — "daqui a"
`duration ± duration`	`duration`
`time_point + time_point`	**erro de compilação**

A última linha é a que importa: somar dois instantes não tem significado, e o sistema de tipos impede. Pela mesma razão, o relógio faz parte do *tipo* do `time_point`, e `system_clock::now() - steady_clock::now()` não compila — as *epochs* são diferentes, e a subtração não teria sentido. A consequência prática é boa: não há como, por acidente, medir duração com o relógio de parede e datar um evento com o monotônico.

Instante em texto. A via C ainda aparece em código legado: `to_time_t`, `localtime_r`, `strftime`. Vale registrar que `localtime` **sem** o sufixo `_r` não é seguro em programas com várias *threads*: ele devolve um ponteiro para um *buffer* estático compartilhado. A via C++20 é formatar o `time_point` diretamente, com `{:%Y-%m-%d}` e afins; e `time_point_cast<seconds>` trunca a precisão quando se quer comparar carimbos sem os nanossegundos.

Regra — *deadline* em vez de *timeout* sempre que puder aninhar

Uma API que espera pode receber "quanto tempo esperar" (*timeout*) ou "até quando esperar" (*deadline*). A diferença aparece em cadeia:

```
conectar(timeout=100ms) → autenticar(timeout=100ms) → consultar(timeout=100ms)
Total possível: 300ms, embora o chamador tenha pedido 100.

auto limite = steady_clock::now() + 100ms;
conectar(limite) → autenticar(limite) → consultar(limite)
Total garantido: 100ms.
```

É por isso que a biblioteca oferece os dois: `wait_for` recebe duração, `wait_until` recebe instante (Seção VI.1). Use `_until` em qualquer função que possa ser chamada dentro de outra, e `_for` apenas na folha da cadeia.

XIII.2.6 O calendário do C++20

Até o C++17, trabalhar com datas significava `std::tm`: uma *struct* C de campos `int` em que o mês vai de 0 a 11, o ano é "anos desde 1900", e nada impede escrever `tm.tm_mon = 47`. O C++20 trouxe um sistema de tipos completo: `year`, `month`, `day`, `weekday`, e as combinações entre eles.

A sintaxe de composição usa `operator/`, escolhido por já lembrar data em quase toda notação do mundo, e aceita as três ordens culturais:

```
2026y / August / 25d
August / 25d / 2026y
25d / August / 2026y
```

Listagem 408: As três ordens, todas válidas e todas o mesmo dia.

Os tipos são minúsculos — a segurança não custa memória — e trazem validação que `std::tm` nunca teve: `ok()` responde se a data existe, e `year::is_leap()` implementa a regra bissexta completa (divisível por 4, exceto os divisíveis por 100 que não sejam por 400 — por isso 1900 e 2100 não são bissextos, e 2000 é).

Armadilha — `ok()` não é chamado automaticamente

Construir uma data inválida **compila e não lança**: `2026y/February/30d` é um objeto perfeitamente formado cujo `ok()` devolve falso. A biblioteca deixa a verificação para você, e o lugar de fazê-la é a fronteira de entrada — ao ler de um arquivo, de um formulário, de uma requisição.

`sys_days`: a ponte. `year_month_day` é o tipo do *calendário*: mês, ano, validação. `sys_days` é um `time_point<system_clock, days>`, e é o tipo da *aritmética*: somar dias, calcular diferenças, descobrir o dia da semana. Converter entre os dois é barato, e o padrão de uso é converter conforme a operação.

Armadilha — somar meses pode produzir uma data que não existe

Somar dias nunca produz data inválida; somar meses ou anos, sim:

```
2026−01−31 + 1 mês  = 2026−02−31 is not a valid date  → ok() = false
2026−01−31 + 31 dias = 2026−03−03                     → ok() = true
```

A biblioteca **não conserta**: ela devolve a data inválida e deixa a política com você. As três políticas possíveis são ir para o último dia do mês (28/02), transbordar — que é o que `sys_days{data_inválida}` faz, normalizando para 03/03 — ou rejeitar, porque `ok()` é falso. O mesmo vale para aniversários de quem nasceu em 29 de fevereiro: `2024-02-29 + 1 ano` é inválido.

Fusos horários. A API existe — `current_zone`, `zoned_time` — mas depende do banco `tzdata` e do suporte compilado na libstdc++; no GCC 13 ela pode lançar `std::runtime_error` ou falhar na ligação, e por isso o demo não a chama. A regra que não depende de suporte nenhum é outra, e é a mais importante da subseção:

Regra — armazene em UTC, converta para local só na apresentação

Guardar hora local em banco de dados é a origem de metade dos *bugs* de horário de verão que existem: uma hora do ano acontece duas vezes e outra não acontece, e não há como distinguir os dois carimbos depois de gravados. Armazene `system_clock` (que é UTC), e converta apenas ao exibir.

XIII.2.7 Como cronometrar sem se enganar

Esta é a subseção que sustenta todos os números do livro. São cinco armadilhas, em ordem de frequência.

1. O compilador elimina o que você quer medir. Chama-se *eliminação de código morto*: se o resultado de um cálculo não é usado, o compilador tem o direito de apagar o cálculo. Em `-O2`, um laço que soma dois milhões de números e descarta o resultado simplesmente **não existe** no binário, e o cronômetro mede zero.

A barreira cabe em três linhas e é o que o Google Benchmark chama de `DoNotOptimize`:

```
1 template <typename T>
2 inline void nao_otimize(T&& valor) {
```

Tabela 157: As três defesas contra a eliminação, em ordem de preferência.

Defesa	Qualidade
barreira de otimização (`asm volatile`)	**a correta**
`volatile` na variável de resultado	simples, mas impede otimizações legítimas em volta
imprimir ou somar o resultado	polui a medida

```
    // Diz ao compilador: "este valor pode ser lido e modificado por código
    // que você não vê". Ele então precisa materializá-lo de verdade.
    asm volatile("" : : "r,m"(valor) : "memory");
}
```

Listagem 409: Transcrição de `src/chrono/demo_medicao.hpp`: a barreira de otimização.

2. O trecho é curto demais. Como `now()` custa cerca de vinte nanossegundos, medir uma operação de poucos nanossegundos mede o relógio:

```
uma execução medida direto : ~20 ns    ← mediu o RELÓGIO
1000000 execuções / N      :  ~2 ns    ← mediu a operação
```

Listagem 410: Saída de `src/chrono/`: a mesma operação, medida de duas formas.

A regra prática: o trecho medido precisa durar pelo menos cem vezes a resolução do relógio. Com vinte nanossegundos de sobrecarga, isso significa alvos de dois microssegundos ou mais — ou repetir e dividir.

3. Aquecimento. A primeira execução paga o que as seguintes não pagam: *page faults* — a materialização, pelo núcleo do sistema operacional, de páginas de memória que foram reservadas mas nunca tocadas —, cache frio, preditor de saltos sem histórico, CPU ainda em frequência baixa.

O demo mede dois casos, e é honesto quando o efeito não aparece: no caso em que um `std::iota` preencheu o vetor antes da medição, as cinco execuções saem parecidas, porque o `iota` já pagou tudo. A conclusão é a mesma nos dois casos — *descarte as primeiras execuções.* O custo de descartar é zero; o de não descartar é uma conclusão errada.

4. Mediana, não média.

```
mínimo  : 221 µs   ← o mais próximo do custo REAL
mediana : 225 µs   ← robusta a outliers
média   : 228 µs   ← puxada por qualquer pico
p95     : 240 µs
máximo  : 261 µs   ← o escalonador interferiu
```

Listagem 411: Saída de `src/chrono/`: 21 execuções do mesmo trabalho.

A média é a pior estatística possível aqui, porque um único pico — uma troca de contexto, uma interrupção, outro processo acordando — a contamina inteira.

5. Um cronômetro reutilizável. O subprojeto fecha com um cronômetro RAII e uma função de medição que fazem o básico certo:

```
template <typename F>
ch::nanoseconds medir(F&& f, int repeticoes = 1) {
    // Aquecimento: descarta a primeira
    { auto r = f(); nao_otimize(r); }
```

Tabela 158: Qual estatística responde a qual pergunta.

Você quer	Use
o custo típico	mediana
o melhor caso real	mínimo
a latência de um contrato de serviço	p95 / p99
—	média (quase nunca)

```
5
6       const auto t0 = ch::steady_clock::now();
7       for (int i = 0; i < repeticoes; ++i) { auto r = f(); nao_otimize(r); }
8       const auto dt = ch::steady_clock::now() - t0;
9
10      return ch::duration_cast<ch::nanoseconds>(dt) / repeticoes;
11  }
```

Listagem 412: Transcrição de `src/chrono/demo_medicao.hpp`: aquecimento, repetição e barreira, em oito linhas.

E o subprojeto lista o que ele **não** faz, e que um *benchmark* sério faria: estatística (mediana, desvio, intervalo de confiança), ajuste automático de N até a medida estabilizar, fixação da *thread* numa CPU com *turbo boost* desligado, detecção de interferência do escalonador. Para isso existem Google Benchmark, nanobench e o suporte do Catch2.

XIII.3 Bits

Os operadores `&`, `|`, `^`, `~`, `«` e `»` parecem os mais simples da linguagem, e escondem três regras que produzem UB (comportamento indefinido). O subprojeto `src/bits/` abre com as três, resumidas em três linhas:

```
1  uint8_t x = 0x0F;
2  ~x                      // NÃO é 0xF0 — é −16, um int     (promoção inteira)
3  uint64_t m = 1 << 40; // ✗ UB: o literal '1' é int        (shift fora de faixa)
4  int f : 1; f = 1;       // f vale −1                        (bit de sinal)
```

Listagem 413: As três armadilhas dos operadores de bit.

XIII.3.1 As quatro operações com máscara

Uma *máscara* é um valor em que os bits de interesse estão em 1. Com ela, as quatro operações fundamentais se escrevem sempre da mesma forma:

Tabela 159: Os quatro idiomas com máscara.

Operação	Idioma
testar	`(x & M) != 0`
ligar	`x |= M`
desligar	`x &= ~M`
alternar	`x ^= M`

XIII.3.2 A promoção inteira, que morde

Por que — por que existe a promoção inteira

Herdada do C, a regra diz que todo operando de tipo *menor* que `int` — `char`, `short`, `uint8_t` — é convertido para `int` **antes** da operação. A motivação era histórica e razoável: as unidades aritméticas trabalham na largura da palavra, e uniformizar os operandos simplifica a linguagem. A consequência é que uma operação escrita sobre oito bits acontece sobre trinta e dois.

```
1  uint8_t x = 0x0F;
2  ~x como int        : -16 (0xFFFFFFF0)     ← não é 0xF0
3  uint8_t(0xFF) << 4 = 0xFF0 (12 bits!)     ← não trunca
```

Listagem 414: Saída de `src/bits/`: a promoção, medida.

O bug real que isso causa é uma comparação que nunca é verdadeira: `if (flags == ~MASCARA)` compara um `uint8_t` com um `int` **negativo**. O código compila, roda e nunca entra no `if`.

As defesas são três: use `unsigned` ou `uint32_t` para máscaras; faça o `static_cast` de volta ao tipo pequeno, explicitamente; e compile com `-Wconversion`, para que o compilador cobre a conversão em vez de fazê-la calado.

XIII.3.3 *Shift* fora de faixa

A regra é: `x << n` e `x >> n` são UB (comportamento indefinido) se `n` for negativo ou maior ou igual ao número de bits de `x` **após a promoção**. As consequências são pouco intuitivas:

```
1  uint32_t x = 1;  x << 32;    // ✗ UB (32 bits, deslocamento de 32)
2  uint8_t  b = 1;  b << 8;     // ✓ ok! b virou int (32 bits)
3  uint64_t m = 1 << 40;        // ✗ UB — o literal '1' é int
4  uint64_t m = 1ull << 40;     // ✓
```

Listagem 415: Quatro deslocamentos: dois corretos, dois UB (comportamento indefinido).

A segunda linha é o caso em que a medição contraria a expectativa: deslocar um `uint8_t` por 8 seria UB (comportamento indefinido) se a operação fosse de 8 bits, e é segura precisamente por causa da promoção inteira — a mesma regra que causa o problema da subseção anterior. A terceira é a armadilha mais frequente do mundo real: o literal `1` é `int`, e o tipo do *destino* não influencia o tipo da *expressão*. Máscaras de 64 bits exigem `1ull`.

Armadilha — o UBSan pega, o compilador não

Nenhum dos casos acima gera *warning* com `-Wall`. O UndefinedBehaviorSanitizer (Seção I.9) detecta em tempo de execução e imprime `runtime error: shift exponent 32 is too large`. É mais um argumento para rodar os testes sob *sanitizer* ao menos uma vez.

Vale registrar uma regra que **mudou**: `-1 << 1` era UB (comportamento indefinido) até o C++17 e passou a ser definido no C++20, porque esse padrão fixou a representação em complemento de dois como obrigatória.

XIII.3.4 `>>` em *signed* contra *unsigned*

Em tipos sem sinal, `>>` insere zeros à esquerda (deslocamento *lógico*). Em tipos com sinal e valor negativo, insere cópias do bit de sinal (deslocamento *aritmético*), preservando o valor:

```
int32_t(−16) >> 2 = −4          ← preencheu com o bit de SINAL
uint32_t     >> 2 = 1073741820  ← preencheu com zeros

−7 >> 1 = −4    (arredonda para −∞)
−7 /  2 = −3    (trunca para zero)
```

Listagem 416: Saída de `src/bits/`: os mesmos bits, dois resultados.

As duas últimas linhas desmentem a substituição que quase todo programador já fez: `x » 1` **não** é `x / 2` para negativos.

Regra — *unsigned* para bits, *signed* para aritmética

Para manipulação de bits, use sempre tipos sem sinal: eles não têm bit de sinal para propagar, não têm estouro com comportamento indefinido e não mudam de significado ao deslocar. Tipos com sinal são para aritmética, onde o sinal significa alguma coisa.

XIII.3.5 Extrair e inserir campos, de forma portável

O padrão que aparece em todo protocolo binário e em todo registrador de hardware é um inteiro que empacota vários campos. Fazê-lo à mão, com deslocamento e máscara, dá um tipo pequeno e `constexpr` / `consteval`:

```
struct CampoBits {
    unsigned deslocamento;
    unsigned largura;

    constexpr std::uint32_t mascara() const {
        return ((1u << largura) − 1u) << deslocamento;
    }
    constexpr std::uint32_t extrair(std::uint32_t reg) const {
        return (reg & mascara()) >> deslocamento;
    }
    constexpr std::uint32_t inserir(std::uint32_t reg, std::uint32_t valor) const {
        return (reg & ~mascara()) | ((valor << deslocamento) & mascara());
    }
};

constexpr CampoBits VERSAO {24, 8};
constexpr CampoBits TIPO   {16, 8};
constexpr CampoBits TAMANHO{ 4, 12};
constexpr CampoBits FLAGS  { 0, 4};
```

Listagem 417: Transcrição de `src/bits/demo_operadores.hpp`: campos com layout definido por você.

Como tudo é `constexpr`, o layout é verificável em compilação, e um deslocamento errado vira erro de *build*:

```
    static_assert(VERSAO.mascara()  == 0xFF000000u);
    static_assert(TAMANHO.mascara() == 0x0000FFF0u);
    static_assert(TAMANHO.extrair(TAMANHO.inserir(0, 4095)) == 4095);
```

Listagem 418: Transcrição de `src/bits/demo_operadores.hpp`: o layout conferido pelo compilador.

Guarde este exemplo: ele é a alternativa **portável** aos campos de bits nativos, e a diferença entre os dois é o assunto de duas subseções adiante.

XIII.3.6 `<bit>`: as operações que o hardware já sabia fazer

Todo processador moderno tem instruções para contar bits, achar o bit mais significativo e rotacionar. Até o C++20 não havia forma portável de chamá-las: o código usava `__builtin_popcount` no GCC, `_BitScanReverse` no MSVC, ou um laço que o compilador talvez reconhecesse. O `<bit>` padronizou tudo.

`bit_cast`: o fim do *type punning* informal. *Type punning* é reinterpretar os bits de um objeto como se fossem de outro tipo — operação legítima e comum em serialização e em algoritmos de ponto flutuante. Todas as formas antigas eram inválidas ou incômodas.

Tabela 160: As quatro formas de reinterpretar bits, e o veredito de cada uma.

Forma	Veredito
`*(int*)&f`	viola *strict aliasing* — UB (comportamento indefinido)
`union { float f; int i; }`	ler o membro inativo é UB (comportamento indefinido) em C++
`memcpy(&i, &f, 4)`	correto, mas verboso e **não** é `constexpr` / `consteval`
`std::bit_cast<int>(f)`	correto, conciso **e** `constexpr` / `consteval`

Strict aliasing é a regra que permite ao compilador supor que ponteiros de tipos diferentes não apontam para o mesmo objeto — suposição sem a qual quase nenhuma otimização de laço seria possível. Violá-la não produz erro: produz código que funciona em `-O0` e falha em `-O2`.

`bit_cast` exige que os dois tipos tenham o mesmo `sizeof` e sejam trivialmente copiáveis, e verifica isso em compilação. E, por ser `constexpr` / `consteval`, a representação de um `float` passa a ser conferível por `static_assert`:

```
1  constexpr auto bits_const = std::bit_cast<std::uint32_t>(2.0f);
2  static_assert(bits_const == 0x40000000u);
```

Listagem 419: Transcrição de `src/bits/demo_bit.hpp`: a representação conferida em compilação.

Contagem. Três funções, e um uso prático para cada uma:

```
1  v = 00001011011000000000000000000000
2    popcount    = 5    bits 1 no total
3    countl_zero = 4    zeros à esquerda
4    countr_zero = 21   zeros à direita
```

Listagem 420: Saída de `src/bits/`: contagem sobre um valor de 32 bits.

`popcount` dá a distância de Hamming entre dois valores (basta contar os bits de `a ^ b`) e a cardinalidade de um conjunto representado como máscara. `countl_zero` dá a posição do bit mais significativo, e portanto o logaritmo de base 2 inteiro. `countr_zero` dá o alinhamento de um endereço e a posição do próximo *slot* livre num alocador de bitmap.

Potências de dois. Quatro funções respondem a perguntas distintas: `has_single_bit` ("é potência de 2?"), `bit_width` (quantos bits são necessários), `bit_floor` (maior potência de 2 menor ou igual) e `bit_ceil` (menor potência de 2 maior ou igual).

O uso canônico é dimensionar um *buffer*: pedir mil elementos e alocar `bit_ceil(1000)` = 1024 permite trocar o resto de divisão `i % n` pelo mascaramento `i & (n-1)`, que é uma instrução em vez de dezenas.

Tabela 161: Saída de `src/bits/`: as quatro funções sobre dois valores.

Valor	has_single_bit	bit_width	bit_floor	bit_ceil
100	não	7	64	128
1000	não	10	512	1024

Antes do C++20, `has_single_bit(v)` se escrevia `v && !(v & (v - 1))` — correto, ilegível, e errado para `v == 0` se alguém esquecesse o primeiro termo.

Armadilha — `bit_ceil` tem um caso de UB (comportamento indefinido)

`bit_ceil(v)` é UB (comportamento indefinido) se o resultado não couber no tipo. Para `uint32_t`, qualquer `v` acima de 2^{31} pede 2^{32}, que não existe no tipo. Verifique antes, ou trabalhe num tipo mais largo.

Rotação. Rotacionar é diferente de deslocar: os bits que saem de um lado **entram** pelo outro. É a operação-base de funções de *hash* e de criptografia.

```
1  v            = 10010011
2  v << 3       = 10011000   (shift: entram zeros, perde bits)
3  rotl(v,  3)  = 10011100   (rotação: os bits VOLTAM)
4  rotl(v,  11) = 10011100   (11 % 8 = 3 — sem UB)
5  rotl(v, -3)  = 01110010   (negativo inverte o sentido)
```

Listagem 421: Saída de `src/bits/`: deslocamento contra rotação.

Duas propriedades merecem destaque, porque contrastam com o deslocamento: `rotl` reduz a contagem módulo a largura do tipo, e aceita contagem **negativa** invertendo o sentido. Não há UB (comportamento indefinido) para valor nenhum — enquanto o idioma manual pré-C++20, `(v << n) | (v >> (32 - n))`, é UB (comportamento indefinido) exatamente quando `n == 0`, que é o caso que ninguém testa.

Armadilha — a instrução única depende da *flag* de compilação

`popcount` mapeia para a instrução `POPCNT`, `countl_zero` para `LZCNT`, `rotl` para `ROL` — mas só se o compilador tiver permissão para usar a extensão correspondente. Com `g++ -O2` e nada mais, `POPCNT` não é garantido, e o compilador emite a sequência genérica de cerca de doze instruções. Com `-msse4.2` ou `-march=native`, vira uma. Este subprojeto compila *sem* a *flag* de propósito: o objetivo é o comportamento, não a instrução.

XIII.3.7 `std::bitset`: bits com nome

`bitset<N>` é um vetor de N bits empacotados com uma interface de alto nível: `b.test(3)` em vez de `(flags >> 3) & 1`, `b.set(3)` em vez de `flags |= (1u << 3)`, `b.count()` em vez de `__builtin_popcount(flags)`. É mais legível e igualmente rápido — as operações compilam para as mesmas instruções.

Ele suporta as operações de conjunto diretamente: `|` é união, `&` é interseção, `^` é diferença simétrica, `a & ~b` é diferença, e `(a & b).none()` responde se são disjuntos.

Armadilha — o índice 0 fica à direita da *string*

O índice i corresponde ao valor 2^i, portanto o índice 0 é o bit **menos** significativo — o mais à **direita** quando se imprime:

```
b.set(0) → to_string() = "00000001"
b.set(7) → to_string() = "10000000"
```

A regra completa é `string[j]` → índice `(N-1-j)`. A *string* lê da esquerda para a direita como um número binário; os índices contam da direita para a esquerda, como potências. Confundir os dois ao converter de e para texto é a fonte de *bug* mais comum com `bitset`.

Como no `vector`, há duas formas de acessar com contratos diferentes: `test(i)` verifica os limites e lança `std::out_of_range`; `b[i]` não verifica, e é UB (comportamento indefinido) fora da faixa. E `to_ulong()` lança `overflow_error` se houver bits significativos além do que cabe num `unsigned long`.

Tabela 162: `bitset` contra as alternativas.

Opção	Tamanho	Legível	Seguro	Dinâmico
`uint32_t` + máscara	fixo	não	não	não
`enum class` de *flags*	fixo	**sim**	**sim**	não
`std::bitset<N>`	fixo	**sim**	sim	não
`std::vector<bool>`	**dinâmico**	sim	não*	**sim**
`std::vector<char>`	dinâmico	sim	sim	sim

* `operator[]` devolve um *proxy*, não um `bool&` — ver Seção III.1.

A comparação com `std::vector` de `bool` merece ser explícita, porque os dois empacotam bits e a escolha entre eles não é de gosto. `vector<bool>` é a única opção quando o tamanho só é conhecido em tempo de execução, e paga por isso com o *proxy*: `auto x = vb[0]` não produz um `bool`, produz uma referência disfarçada, e atribuir a ela modifica o vetor. `bitset<N>` tem tamanho fixo em compilação e devolve `bool` de verdade em `test()`. Quando o tamanho é dinâmico e o *proxy* incomoda, a resposta é `vector<char>` ou `boost::dynamic_bitset`.

O tamanho do `bitset` é arredondado para múltiplos da palavra:

```
sizeof:  bitset<8>=8  bitset<64>=8  bitset<65>=16  bitset<100>=16
```

Listagem 422: Saída de `src/bits/`: o arredondamento para a palavra.

Um único bit a mais que 64 dobra o custo. Para conjuntos grandes e homogêneos — o crivo de Eratóstenes do subprojeto usa `bitset<1001>`, cerca de 126 bytes — o ganho é de oito vezes sobre um vetor de `bool` nativo, e o que importa não é a memória em si: é que a estrutura inteira cabe no cache L1.

XIII.3.8 Campos de bits nativos, e por que evitá-los

O C++ tem sintaxe própria para empacotar campos num inteiro: `unsigned versao : 4;`. Parece a solução ideal para protocolos binários. **Não é** — porque quase tudo sobre o layout é *definido pela implementação*: a ordem dos campos dentro da palavra, se um campo pode atravessar a fronteira da unidade de alocação, o alinhamento e o tamanho total.

A "unidade de alocação" aqui é a palavra de memória que o compilador escolhe para acomodar os campos — e a escolha é dele. O subprojeto *mede* o layout no GCC para x86-64 e encontra o primeiro campo nos bits menos significativos; nada garante que outro compilador faça igual.

Armadilha — um *struct* com campos de bits não é um formato

Escrever um cabeçalho de protocolo com campos de bits e enviá-lo pela rede é um *bug* esperando o dia em que a outra ponta usa outro compilador ou outra arquitetura. O formato precisa ser definido pelo **seu** código — é exatamente o que o `CampoBits` da subseção anterior faz, ao custo de mais verbosidade.

Armadilha — um campo com sinal de 1 bit vale −1, não 1

A faixa de `int x : 1` é $\{-1, 0\}$, não $\{0, 1\}$: o único bit *é* o bit de sinal. Atribuir 1 e ler −1 é o comportamento correto, e o GCC avisa (`overflow in conversion... changes value from '1' to '-1'`). O *bug* clássico:

```
1 struct Flags { int ativo : 1; };
2 f.ativo = 1;
3 if (f.ativo == 1) ...     // FALSO! f.ativo é −1
4 if (f.ativo)      ...     // ✓ verdadeiro (−1 é truthy)
```

Campos de bits são **sempre** `unsigned` ou `bool`. E `char x : 1` tem o mesmo problema, porque a existência de sinal em `char` também é definida pela implementação.

Armadilha — um campo de bits não tem endereço

Um campo de bits não ocupa bytes inteiros, logo não tem endereço próprio: `&x.campo` não compila, e nenhuma referência pode ser ligada a ele. Isso significa que passá-lo a qualquer função que receba por referência — `fmt::print`, por exemplo — falha com `error: cannot bind bit-field to 'unsigned char&'`. O contorno é o `+` unário, que promove o campo a um `int` temporário: `fmt::print("{}", +h.versao)`. O demo precisou do `+` em cada campo, e isso *é* a lição.

Há ainda dois custos que não são óbvios. Ler ou escrever um campo vira deslocamento mais máscara — mais instruções que acessar um byte inteiro. E dois campos na mesma palavra **não** podem ser acessados concorrentemente: escrever num deles faz leitura-modificação-escrita da palavra inteira, o que é uma *data race* pela definição que o C++11 introduziu (Seção VI.1).

Tabela 163: A ferramenta certa para cada objetivo com bits.

Objetivo	Ferramenta
economizar memória **interna** ao programa	campos de bits
protocolo binário ou arquivo	máscaras e deslocamento
registrador de hardware	máscaras e `volatile`
conjunto de *flags* nomeadas	`enum class` com operadores
muitos bits homogêneos	`std::bitset`

XIII.3.9 *Endianness*, `bit_cast` e serialização

Um `uint32_t` com o valor `0x12345678` ocupa quatro bytes. A **ordem** desses bytes na memória depende da arquitetura: em *little-endian* (x86, ARM na configuração usual), o byte menos significativo fica no menor endereço; em *big-endian* (PowerPC, e a convenção de rede), o mais significativo.

```
1 uint32_t 0x12345678 → 78 56 34 12      ← little−endian: menos significativo 1º
```

Listagem 423: Saída de `src/bits/`: os bytes na memória, nesta máquina.

Enquanto os dados vivem na memória do processo, isso é invisível. Assim que eles viram bytes — arquivo, *socket*, memória compartilhada — a ordem passa a fazer parte do formato. Duas observações fecham o

conceito: o *endianness* afeta a ordem dos **bytes**, nunca a dos bits dentro de cada byte; e, desde o C++20, `std::endian::native` responde a pergunta em tempo de *compilação*, permitindo `if constexpr` — antes disso o truque era ler o primeiro byte de um `uint16_t` por um ponteiro para `char`, o que tecnicamente violava *strict aliasing*.

Armadilha — serializar o *struct* inteiro está sempre errado

```
struct Pacote { uint16_t tipo; uint32_t tamanho; uint8_t versao; };
  sizeof = 12 bytes (os campos somam 7 — o resto é PADDING)

write(fd, &p, sizeof(p)) escreve:
  02 01 00 00 06 05 04 03 07 00 00 00
```

São três problemas, todos silenciosos. **Padding**: os bytes que o compilador insere entre campos para respeitar o alinhamento têm valor indefinido, e enviá-los vaza memória do processo — já foi CVE. ***Endianness***: a ordem gravada é a da arquitetura de quem escreveu, não a do formato. **Layout**: outro compilador pode ordenar ou alinhar diferente. Nenhum dos três aparece em teste, porque quem escreve e quem lê é o *mesmo binário*.

A forma correta é serializar campo a campo, numa ordem que o *seu* código define:

```cpp
    template <typename T>
    void escrever(T v) {
        static_assert(std::is_unsigned_v<T>, "serialize só tipos sem sinal");
        for (int i = sizeof(T) - 1; i >= 0; --i) {
            saida_.push_back(static_cast<std::uint8_t>(v >> (i * 8)));
        }
    }
```

Listagem 424: Transcrição de `src/bits/demo_endianness.hpp`: escrita em *big-endian*, independente da máquina.

```
serializado campo a campo, big-endian:
  01 02 03 04 05 06 07          ← 7 bytes, contra 12 do struct cru
round-trip: ✓ idêntico
```

Listagem 425: Saída de `src/bits/`: sete bytes contra doze, e ida e volta exata.

Por que — por que o deslocamento resolve o *endianness*

Porque `v >> (i*8)` é uma operação sobre o **valor numérico**, não sobre os bytes. `v >> 24` devolve o byte mais significativo em qualquer arquitetura, porque "mais significativo" é uma propriedade do número, não da memória. *Endianness* só existe quando alguém olha os bytes — e este código nunca olha. É o mecanismo por trás de Protobuf, MessagePack e Cap'n Proto.

Quando é mesmo necessário inverter a ordem de um valor inteiro — ao ler um formato de outra convenção, por exemplo — a ferramenta é `std::byteswap`, do C++23, que vira uma instrução `BSWAP`. As alternativas, em ordem decrescente de qualidade, são `__builtin_bswapNN` do GCC e do Clang, o par `htonl`/`ntohl` do POSIX (limitado a 16 e 32 bits) e os deslocamentos escritos à mão, que o compilador em geral reconhece e converte na mesma instrução.

XIII.4 O IO de arquivo com *streams*

A Seção III.3 apresentou a hierarquia dos *streams* e o `std::filesystem`; esta seção completa o assunto pelo lado prático, com o que o subprojeto `src/file_io/` demonstra em código.

Um *stream* é uma abstração de sequência de caracteres com direção. Por baixo dele há um `streambuf`, o objeto que efetivamente conversa com o destino — um arquivo, um terminal, uma região de memória — e que guarda o *buffer*: uma área intermediária onde os caracteres se acumulam antes de serem entregues, para que uma única chamada ao sistema operacional escreva muitos deles de uma vez.

XIII.4.1 Os quatro objetos globais, e a diferença que importa

`std::cin` lê da entrada padrão; `std::cout` escreve na saída padrão, com *buffer*; `std::cerr` e `std::clog` escrevem ambos no erro padrão, e a diferença entre eles é exatamente o *buffer*.

Regra — `cerr` para o que precisa sobreviver ao *crash*

`std::cerr` não tem *buffer*: cada escrita chega ao destino imediatamente. Isso custa caro — uma chamada ao sistema por operação — e é precisamente o que se quer da última mensagem antes de um aborto. `std::clog` escreve no mesmo lugar com *buffer*, e é a escolha certa para *log* volumoso, desde que se aceite perder as últimas linhas num encerramento anormal. Pela mesma razão, prefira `'\n'` a `std::endl`: o segundo é o primeiro *mais um flush*, e o flush desnecessário em laço é um dos custos de IO mais comuns.

XIII.4.2 Manipuladores: poderosos e pegajosos

Os manipuladores de `<iomanip>` controlam base (`dec`, `hex`, `oct`), notação de ponto flutuante (`fixed`, `scientific`), precisão, largura, preenchimento e alinhamento.

Armadilha — o manipulador muda o *stream*, não a expressão

`std::hex` não formata o próximo valor: ele altera o **estado** do *stream*, e o estado permanece. Um `std::hex` esquecido faz toda a saída seguinte sair em hexadecimal, inclusive a de outra função, de outro arquivo, escrita por outra pessoa. A exceção é `std::setw`, que vale *apenas* para a próxima inserção — o que torna o conjunto ainda menos previsível. É a razão de existir `std::format` (Seção XIII.1.6), cuja especificação é local à chamada e não deixa resíduo.

XIII.4.3 Arquivos: texto, binário e posicionamento

As três classes de `<fstream>` correspondem às três direções: `ofstream` escreve, `ifstream` lê, `fstream` faz as duas. Todas são RAII: o destrutor fecha o arquivo e esvazia o *buffer*.

Tabela 164: Os modos de abertura mais usados.

Declaração	Efeito
`ofstream f("a.txt");`	`out | trunc` — apaga o conteúdo
`ofstream f("a.txt", ios::app);`	acrescenta ao fim
`ifstream f("a.txt");`	`in`
`fstream f("a.txt", ios::in|ios::out);`	lê e escreve, sem truncar
`ofstream f("a.bin", ios::binary);`	binário, sem tradução de fim de linha

O posicionamento tem **dois** cursores independentes: `tellg` e `seekg` governam a leitura (*get*), `tellp` e `seekp` governam a escrita (*put*). As origens são `ios::beg`, `ios::cur` e `ios::end`. É com eles que se lê um arquivo

inteiro de uma vez — ir ao fim, medir a posição, voltar ao início, ler tudo — e que se faz acesso aleatório a registros de tamanho fixo, saltando `n * sizeof(Registro)` bytes.

Armadilha — o modo binário do demo é um exemplo do que não fazer em rede

O subprojeto grava *structs* com `ofs.write(reinterpret_cast<const char*>(&r), sizeof(Registro))`, e isso é legítimo enquanto o arquivo é privado do programa. Para qualquer formato que atravesse máquinas, compiladores ou versões, vale integralmente a advertência da subseção anterior: *padding*, *endianness* e layout tornam o resultado dependente de quem escreveu. Formato que sai do processo se serializa campo a campo.

Quanto a erros, um *stream* tem quatro estados consultáveis — `good`, `fail`, `bad`, `eof` — e, por padrão, **não lança**: uma leitura que falha apenas liga um bit, e é o `if (!ifs)` que precisa existir. Quem preferir exceções pode pedi-las com `ifs.exceptions(std::ios::failbit | std::ios::badbit)`, e aí um `open` malsucedido lança `std::ios_base::failure`.

XIII.4.4 *String streams*: a mesma interface, em memória

`ostringstream` monta texto, `istringstream` *parseia*, `stringstream` faz as duas sobre o mesmo *buffer*. O uso mais frequente do segundo é a tokenização: `std::getline(iss, token, ',')` separa um CSV, e `std::getline(iss, chave, '=')` seguido de `std::getline(iss, valor)` *parseia* uma linha de configuração.

Duas notas de desempenho. Primeiro, `str()` devolve uma **cópia** da *string* interna; desde o C++20 existe `view()`, que devolve um `std::string_view` sem alocar — e que, como todo *view*, é invalidado assim que o *stream* é modificado. Segundo, para converter texto em número, *string streams* são a opção mais cara das cinco da Tabela 152: cada conversão constrói um objeto de *stream* inteiro, com *locale* e *flags*. Para dados, `<charconv>`.

XIII.4.5 `rdbuf`: o ponto de extensão que torna tudo componível

O método `rdbuf()` devolve — e, com argumento, **troca** — o `streambuf` de um *stream*. É isso que permite redirecionar `std::cout` para um arquivo sem tocar em nenhuma das chamadas que escrevem nele, e capturar a saída de uma função em uma *string* para testá-la.

Restaurar o *buffer* à mão é frágil — uma exceção no meio deixa `cout` apontando para um arquivo já destruído — e é exatamente o problema que RAII resolve:

```
class RedirectGuard {
public:
    // Redireciona 'stream' para 'destino'
    explicit RedirectGuard(std::ostream& stream, std::ostream& destino)
        : stream_(stream), buf_original_(stream.rdbuf(destino.rdbuf())) {}

    ~RedirectGuard() {
        stream_.rdbuf(buf_original_);   // restaura automaticamente
    }

    RedirectGuard(const RedirectGuard&)            = delete;
    RedirectGuard& operator=(const RedirectGuard&) = delete;

private:
    std::ostream&   stream_;
    std::ofstream   arquivo_;        // vazio se redirecionou para outro stream
    std::streambuf* buf_original_;
```

```
18 };
```

Listagem 426: Transcrição de `src/file_io/demo_logging.hpp`: o redirecionamento que se desfaz sozinho.

E, uma vez que o `streambuf` é um ponto de extensão, escrever um novo dá capacidades que nenhuma *flag* oferece. O `tee` do Unix, que envia a mesma saída ao terminal e a um arquivo, cabe em duas funções virtuais:

```
1  class TeeBuf : public std::streambuf {
2  public:
3      TeeBuf(std::streambuf* a, std::streambuf* b) : a_(a), b_(b) {}
4
5  protected:
6      int overflow(int c) override {
7          if (c == EOF) return !EOF;
8          if (a_->sputc(static_cast<char>(c)) == EOF) return EOF;
9          if (b_->sputc(static_cast<char>(c)) == EOF) return EOF;
10         return c;
11     }
12
13     std::streamsize xsputn(const char* s, std::streamsize n) override {
14         a_->sputn(s, n);
15         b_->sputn(s, n);
16         return n;
17     }
18
19 private:
20     std::streambuf* a_;
21     std::streambuf* b_;
22 };
```

Listagem 427: Transcrição de `src/file_io/demo_logging.hpp`: um `streambuf` que escreve em dois destinos.

`overflow` é chamado quando o *buffer* enche ou quando um caractere solto precisa ser entregue; `xsputn` é o caminho rápido para blocos. Sobrescrever os dois é o contrato mínimo de um `streambuf` de saída.

XIII.4.6 Um *logger*, e o que ele reúne

O `Logger` do subprojeto junta quase tudo o que o capítulo tratou: carimbo de tempo do `<chrono>`, formatação, escrita simultânea em terminal e arquivo, e um `std::mutex` para que várias *threads* possam registrar sem entrelaçar linhas (Seção VI.1).

```
1  [08:42:01.123] [TRACE] inicializando...
2  [08:42:01.124] [DEBUG] variável x = 42
3  [08:42:01.124] [INFO ] conexão estabelecida
4  [08:42:01.125] [WARN ] buffer 80% cheio
5  [08:42:01.125] [ERROR] falha ao abrir arquivo   ← vai para stderr
6  [08:42:01.125] [FATAL] estado inválido          ← vai para stderr
```

Listagem 428: Saída de `src/file_io/`: níveis, carimbo e destino.

Duas decisões de projeto merecem nota. A primeira é o roteamento por nível: `ERROR` e `FATAL` vão para `stderr`, o resto para `stdout`, de modo que `./prog > saida.txt` deixa os erros visíveis no terminal. A segunda é o `flush` explícito após cada linha gravada no arquivo: ele custa caro e é a única forma de garantir que a última mensagem antes de um *crash* chegue ao disco — a mesma troca de `cerr` contra `clog`, agora sob controle do programa.

Tabela 165: Redirecionamento no *shell* — a referência que acompanha o subprojeto.

Comando	Efeito
`./prog > out.txt`	*stdout* para arquivo (trunca)
`./prog >> out.txt`	*stdout* para arquivo (acrescenta)
`./prog 2> err.txt`	*stderr* para arquivo
`./prog > out.txt 2>&1`	*stderr* segue *stdout*
`./prog 2>&1 | tee log.txt`	terminal e arquivo ao mesmo tempo
`./prog > /dev/null 2>&1`	descarta tudo

Armadilha — a ordem do redirecionamento importa

`./prog > out.txt 2>&1` e `./prog 2>&1 > out.txt` não são a mesma coisa. O `2>&1` copia o destino *atual* do descritor 1; no segundo comando ele é executado **antes** de o descritor 1 ser redirecionado, então *stderr* continua no terminal e só *stdout* vai para o arquivo. Os descritores são 0 para *stdin*, 1 para *stdout* e 2 para *stderr*.

XIII.4.7 `std::filesystem`, em uma linha por conceito

Para completar: `fs::path` compõe caminhos com `operator/` e os decompõe com `stem()`, `extension()`, `parent_path()` e `filename()`, além de normalizar com `lexically_normal()`; `create_directories` cria a hierarquia inteira; `directory_iterator` e `recursive_directory_iterator` percorrem diretórios como *ranges* (Seção III.2); `file_size`, `last_write_time`, `status().permissions()` e `space` dão os metadados.

E, como registrado na Seção III.3, cada função tem duas assinaturas: uma que lança `fs::filesystem_error` e outra que recebe um `std::error_code&` e nunca lança. O critério de escolha é a *frequência esperada* do erro — um arquivo de configuração que deveria existir e não existe é excepcional; um arquivo que some no meio de uma varredura de diretório não é.

XIII.5 Síntese

Há um padrão atravessando os quatro subprojetos deste capítulo, e vale nomeá-lo: **o custo está sempre onde a abstração é mais confortável**. `std::string` é confortável porque possui os dados, e é por possuí-los que ela aloca; `std::string_view` custa zero porque não possui nada, e é por não possuir que ela pende. `std::regex` aceita qualquer padrão, e é por aceitar qualquer padrão que ela não pode ser rápida para um padrão fixo. Os campos de bits nativos escondem os deslocamentos, e é por escondê-los que deixam o layout com o compilador. Os *iostreams* guardam estado de formatação, e é por guardá-lo que o `std::hex` esquecido contamina a saída inteira.

Em todos os casos, o preço da propriedade desejada é a armadilha que vem junto.

O segundo padrão é metodológico, e é o que este capítulo tem de mais transferível. Em quatro ocasiões a medição contrariou quem a fez, e nas quatro a lição estava no desacordo, não no número:

- o *benchmark* de formatação **não** mostrou a `fmt` ganhando — porque o repositório linka a variante de depuração da biblioteca, e nenhuma *flag* no código do usuário conserta isso;
- a `regex` construída dentro do laço levou mais tempo fazendo **dez vezes menos trabalho**, o que separa o custo de construir do custo de casar;

- o `static_assert` sobre o tamanho de um literal recusou 31 e exigiu 33, aplicando ao autor a lição de que `size()` conta bytes;
- `high_resolution_clock`, cujo nome promete o melhor relógio para medir, revelou-se o **não monotônico** nesta implementação.

Regra — o que sobra quando o número some

Quatro hábitos resumem todo o capítulo. Conte alocações em vez de cronometrar quando puder — é determinístico. Cronometre com `steady_clock`, descartando a primeira execução, repetindo e tomando a mediana. Declare as *flags* de compilação ao lado de qualquer número, incluindo as da biblioteca de terceiros. E, quando o resultado contrariar a expectativa, suspeite primeiro da medição — e, se ela resistir, publique a surpresa: é ela que ensina.

XIV O modelo de objeto por dentro

Os capítulos anteriores trataram do que a linguagem *diz*. Este trata do que ela *faz* — byte a byte, símbolo a símbolo. São três perguntas, cada uma com um subprojeto para respondê-la com números em vez de afirmações:

1. **Como os bytes de um objeto são arrumados?** O `src/layout/` responde: alinhamento, *padding*, as categorias de tipo que autorizam `memcpy`, e o que `virtual` acrescenta ao objeto.
2. **Como se guarda um objeto sem mencionar o tipo dele?** O `src/type_erasure/` constrói a `std::function` do zero, mede a otimização de *buffer* pequeno com um `operator new` instrumentado e compara quatro formas de despacho no cronômetro.
3. **Como o compilador e o *linker* combinam sobre nomes, tamanhos e endereços?** O `src/linkage/` — único subprojeto do repositório com *várias* unidades de tradução — mede a One Definition Rule com dois `.cpp` reais e chama código C compilado por um compilador C de verdade.

A ordem não é arbitrária: o apagamento de tipo só faz sentido depois de se saber o que um ponteiro de vtable / vptr custa em bytes, e a ABI da última seção é, em boa medida, a soma das duas primeiras — *layout* de objeto mais nomes de símbolo.

Por que — por que o modelo de objeto é assunto de C++, e não de Java

O C++ é a única linguagem de uso corrente em que o programador *escolhe* o modelo de objeto e paga por ele. Em Java ou Python todo objeto tem cabeçalho, toda chamada de método é virtual e todo valor mora no *heap*: não há decisão a tomar. Em C++, um destrutor virtual que ninguém usa custa 8 bytes por objeto — 76 MB num vetor de dez milhões de partículas, medidos na Seção XIV.1.9 — e trocar a ordem de quatro membros corta um terço do tamanho da `struct`. Quem não conhece o modelo não faz nenhuma dessas escolhas: só sofre as consequências.

XIV.1 O layout de um objeto

XIV.1.1 Alinhamento: a exigência do hardware

Uma definição primeiro. **Alinhamento** é a exigência de que o endereço de um objeto seja múltiplo de um certo número. Todo tipo `T` tem um `alignof(T)`, e o objeto só pode começar em endereço divisível por ele: um `double` com `alignof` 8 mora em $0, 8, 16, \ldots$, nunca em 3.

A exigência não vem da linguagem, vem do processador. A memória é lida em blocos de tamanho fixo — a **linha de cache**, tipicamente 64 bytes em x86-64 — e as instruções de carga esperam operandos que não atravessem fronteiras. Ler um `double` desalinhado custa duas leituras em vez de uma no x86, e em ARM antigo aborta o processo com `SIGBUS`.

Regra — as três regras do layout

(1) Todo tipo tem `alignof(T)`: o endereço precisa ser múltiplo dele. **(2)** O alinhamento de uma `struct` é o **maior** entre os de seus membros. **(3)** `sizeof` é sempre múltiplo de `alignof`, para que *arrays* funcionem. Todo o resto — inclusive por que `sizeof` quase nunca é a soma dos membros —

decorre daí.

O nome do "resto" é ***padding***: bytes que o compilador insere entre os membros, e depois do último, para que cada um caia em endereço válido. O *padding* não guarda nada, não faz parte do valor do objeto e — como se verá — pode conter qualquer coisa.

XIV.1.2 A ordem dos membros muda o tamanho

Por que — por que declarar em ordem decrescente de alinhamento

O compilador **não pode** reordenar os membros de dados: a ordem de declaração é garantida pelo padrão (para membros com o mesmo especificador de acesso), justamente para que o *layout* seja previsível e interoperável com C. A única pessoa que pode arrumar o *padding* da sua `struct` é você.

```
1  struct Ruim {          // 1 + 8 + 1 + 4 = 14 bytes de dado...
2      char       a;
3      double     b;
4      char       c;
5      int        d;
6  };
7
8  struct Bom {           // os MESMOS membros, decrescendo por alinhamento
9      double     b;
10     int        d;
11     char       a;
12     char       c;
13 };
```

Listagem 429: Os mesmos membros, duas ordens, em `src/layout/demo_tamanho.hpp`.

```
1  struct Ruim { char a; double b; char c; int d; };
2    sizeof  = 24   alignof = 8
3    offsetof: a=0  b=8  c=16  d=20
4    mapa: [a][×7 pad][b b b b b b b b][c][×3 pad][d d d d]
5
6  struct Bom  { double b; int d; char a; char c; };
7    sizeof  = 16   alignof = 8
8    mapa: [b b b b b b b b][d d d d][a][c][×2 pad]
9
10 MESMOS dados, 8 bytes de diferença (33% menor).
11 Num vetor de 1 milhão: 22.9 MB contra 15.3 MB.
```

Listagem 430: Saída de `src/layout/`: 14 bytes de dado útil, 8 de diferença.

O `char a` de `Ruim` ocupa o *offset* 0 e o `double b` seguinte só pode começar no 8: sete bytes perdidos. Depois o `char c` no 16 e o `int d` no 20: mais três. Em `Bom` nada disso acontece, porque os membros grandes vêm primeiro e os pequenos preenchem o rabo. Trinta e três por cento de memória de graça — e, o que costuma importar mais, um terço a menos de linhas de cache tocadas em cada passada.

XIV.1.3 Por que existe *padding* no fim

`struct { int a; char b; }` soma cinco bytes e ocupa oito. A razão é a regra (3), e a razão da regra (3) são os *arrays*: em `ComRabo v[3]` cada elemento precisa começar em múltiplo de `alignof`. Se `sizeof` fosse 5, `v[1]` cairia no *offset* 5 — e o `int` de dentro dele, desalinhado.

XIV.1.4 `memcmp` e o lixo do *padding*

O subprojeto constrói duas instâncias com os **mesmos valores de membro**, uma delas com o *padding* marcado com `0xAA`:

```
x.a == y.a && x.b == y.b     → true
memcmp(&x, &y, sizeof)  == 0 → false
  x: 07 00 00 00 7A AA AA AA
  y: 07 00 00 00 7A 00 00 00
                  ^^ ^^ ^^   ← padding
```

Listagem 431: Saída de `src/layout/`: membros iguais, bytes diferentes.

Armadilha — `memcmp` compara bytes; `==` compara valores

O conteúdo do *padding* é **indeterminado**: ele não faz parte do valor do objeto e nada obriga o compilador a inicializá-lo. Daí três proibições: nunca comparar `struct`s com `memcmp` (use `operator==`, escrito ou `= default`); nunca transmitir nem persistir uma `struct` com *padding* sem zerá-la antes — o *padding* **vaza memória do processo** para o disco ou para a rede, que é a família de bug do Heartbleed em outra forma; e, se precisa de bytes, serializar campo a campo.

XIV.1.5 `struct` vazia: por que `1` e não `0`

`sizeof(struct{})` é **1**. Zero seria mais honesto e quebraria a linguagem: dois objetos distintos precisam de endereços distintos, e num `Vazia v[3]` os três ocupariam o mesmo lugar. Métodos não ocupam espaço: uma função membro é uma função comum com um `this` implícito, guardada uma vez no binário — só `virtual` acrescenta algo ao objeto (Seção XIV.1.9). E há a exceção que salva o dia: `struct HerdaVazia : Vazia { int x; }` tem `sizeof` **4**, não 8, porque a base vazia foi otimizada para zero bytes — a EBO da Seção XIV.1.8.

Tabela 166: `sizeof` e `alignof` medidos por `src/layout/` (GCC 13, x86-64 System V).

Tipo	`sizeof`	`alignof`
`int`	4	4
`double`	8	8
`long double`	**16**	16
`void*`	8	8
`std::string`	32	8
`std::vector<int>`	24	8
`std::max_align_t`	16	16

Duas linhas merecem nota. O `long double` do x86-64 System V tem 80 *bits* de dado útil e `sizeof` 16 — quase metade é *padding* —, e no MSVC ele é um `double` de 8 bytes, o que faz dele o tipo menos portável do C++. E `std::max_align_t` é o alinhamento que `malloc` garante: 16 bytes. Pedir mais é o assunto seguinte.

XIV.1.6 `alignas`: mandando no alinhamento

Por que — três situações em que o alinhamento natural não basta

SIMD: uma instrução AVX carrega 32 bytes de uma vez e exige o operando alinhado a 32. **Cache**: dois campos escritos por *threads* diferentes na mesma linha de cache provocam *false sharing* — cada escrita invalida a linha do outro núcleo, e o desempenho despenca sem que haja *data race* nenhum. **Hardware**: um descritor de DMA que o dispositivo exige em endereço específico.

```
1  struct alignas(32) Vetor8f { float v[8]; };      // pronto para AVX
2
3  struct Espacada {
4      alignas(64) int produtor;                    // cada um na sua linha de cache
5      alignas(64) int consumidor;
6  };
```

Listagem 432: `alignas` em tipo e em membro, em `src/layout/demo_alignas.hpp`.

Medido pelo subprojeto: `Compacta` (os dois `int` juntos) ocupa 8 bytes; `Espacada` ocupa **128**, com o consumidor no *offset* 64. Dezesseis vezes mais memória para separar dois inteiros — se compensa é medição, não princípio, mas a alternativa pode custar uma ordem de grandeza quando duas *threads* os escrevem em laço.

Armadilha — `alignas` só aumenta; empacotar de verdade é fora do padrão

`struct alignas(1) Tentativa { double d; };` não compila — `alignment of 'Tentativa' is less than the minimum alignment (8) of its members`. A recusa é proposital: ler um `double` desalinhado é UB (comportamento indefinido). Para empacotar mesmo há extensões **não padronizadas** (`#pragma pack`, `__attribute__((packed))`), e elas criam um perigo silencioso: um ponteiro para membro de `struct` empacotada é *desalinhado*, e passá-lo adiante é UB (comportamento indefinido) — o GCC avisa com `-Waddress-of-packed-member`. Para protocolo de rede ou formato de arquivo, serialize campo a campo.

Alocação super-alinhada. `malloc` garante 16 bytes; um tipo que peça mais é *over-aligned*. Antes do C++17 o `new` **ignorava** o `alignas(32)` e devolvia memória alinhada a 16: o código compilava, rodava e falhava na primeira instrução AVX. O C++17 (P0035) acrescentou as sobrecargas com `std::align_val_t`, e o compilador as chama sozinho — o subprojeto verifica que **8 de 8** alocações de `Vetor8f` vieram alinhadas a 32. À mão há três formas: `std::aligned_alloc(32, n)`, `::operator new(n, std::align_val_t{32})` e `std::align`, que ajusta um bloco existente.

Armadilha — cada alocador tem o seu liberador, e `assume_aligned` é promessa

Memória de `aligned_alloc` se libera com `free`; a de `operator new(std::align_val_t)`, com o `operator delete` correspondente — misturar é UB (comportamento indefinido). Já `std::assume_aligned<32>(p)` (C++20) *promete* ao compilador que o ponteiro cru está alinhado, dispensando o prólogo escalar antes do laço vetorizado. É promessa, não verificação: mentir funciona no x86 e falha no ARM. O UBSan pega com `-fsanitize=alignment` (Seção I.9).

XIV.1.7 As quatro categorias de tipo

Quatro palavras parecidas, quatro autorizações diferentes. Não são curiosidade acadêmica: cada uma **autoriza uma operação concreta**, e usar a operação sem a autorização é UB (comportamento indefinido).

Tabela 167: O que cada categoria de tipo autoriza.

Categoria	Autoriza
trivially copyable	`memcpy` do objeto inteiro — é o que deixa `vector<T>` usar `memmove` ao realocar
standard-layout	`offsetof` legal; interoperação com C; `&s == &s.primeiro_membro`
agregado	inicialização por chaves e *designated initializers*
trivially destructible	`vector<T>::clear()` só zera o tamanho, sem percorrer

```
| tipo           | triv.cp | triv.ctor | std.layout | agregado |
|----------------|---------|-----------|------------|----------|
| int            |   sim   |    sim    |    sim     |   não    |
| Simples        |   sim   |    sim    |    sim     |   sim    |
| ComCtor        |   sim   |    não    |    sim     |   não    |
| ComVirtual     |   não   |    não    |    não     |   não    |
| Privado        |   sim   |    sim    |    não     |   não    |
| HerdaDados     |   sim   |    sim    |    não     |   sim    |
| ComString      |   não   |    não    |    sim     |   sim    |
| std::string    |   não   |    não    |    sim     |   não    |
```

Listagem 433: Saída de `src/layout/`: os *traits* instanciados sobre oito cobaias.

Cada linha ensina uma regra. `ComCtor` (construtor escrito à mão) perdeu *trivially default constructible* e continua *trivially copyable* — `memcpy` sobre ele é legal. `ComVirtual` perdeu tudo: o vptr é preenchido pelo construtor, e um `memcpy` o corromperia. `Privado`, com membros de dados em `public` *e* `private`, perdeu *standard-layout*, porque aí o compilador pode reordenar; `HerdaDados` perdeu pelo mesmo motivo, tendo dados em dois níveis da hierarquia. E `ComString` não é trivial em nada e **ainda é agregado**: `ComString{1, "x"}` compila.

Armadilha — `memcpy` sobre o tipo errado

Sobre um `std::string`, `memcpy` copia o **ponteiro interno**: os dois objetos apontam para o mesmo *buffer* e os dois destrutores o liberam — *double free*. Com SSO e string curta pode até "funcionar", e é isso que torna o bug traiçoeiro: some nos testes. Sobre um tipo com `virtual`, copia o **vptr**, e o destino passa a despachar para os métodos da classe errada. A defesa cabe em duas linhas:

```
template <typename T>
void copia_bruta(T& d, const T& o) {
    static_assert(std::is_trivially_copyable_v<T>, "memcpy exige trivial");
    std::memcpy(&d, &o, sizeof(T));
}
```

Em C++20, `std::bit_cast` faz o mesmo com verificação em compilação e sem UB (comportamento indefinido) de *aliasing* (Seção II.2).

POD foi aposentado. `std::is_pod` está **depreciado desde o C++20** (P0767). "POD" significava trivial *e* *standard-layout*, e juntar as duas propriedades numa palavra só escondia qual delas o programador precisava:

```
quero memcpy?          → is_trivially_copyable
quero offsetof?        → is_standard_layout
quero 'T x;' de graça? → is_trivially_default_constructible
quero chaves?          → is_aggregate
```

Listagem 434: Saída de `src/layout/`: peça a autorização exata.

As quatro perguntas são independentes, e `is_pod` respondia "sim" só quando *todas* eram sim — recusando tipos perfeitamente `memcpy`-áveis, como o `ComCtor` da tabela.

XIV.1.8 EBO e `[[no_unique_address]]`

Por que — o problema: o 1 byte que vira 8

Um objeto vazio ocupa 1 byte. Como *membro*, esse byte custa muito mais: se o membro seguinte é um `double`, ele só pode começar no *offset* 8, e os sete bytes intermediários viram *padding*. Uma política vazia — comparador, alocador, *deleter* — custa oito bytes por objeto sem guardar coisa nenhuma.

```
1  struct PorMembro {
2      Vazio  politica;        // 1 byte ...
3      double dado;            // ... mas 'dado' precisa de offset múltiplo de 8
4  };
5
6  struct PorHeranca : Vazio {
7      double dado;
8  };
9
10 struct ComAtributo {
11     [[no_unique_address]] Vazio politica;
12     double                      dado;
13 };
```

Listagem 435: As três formas, em `src/layout/demo_no_unique_address.hpp`.

```
1  struct { Vazio p; double d; }          → sizeof 16  ← 8 a mais!
2  struct : Vazio { double d; }           → sizeof  8  (EBO)
3  struct { [[nua]] Vazio p; double d; } → sizeof  8  (C++20)
4
5  Num std::vector de 10 milhões: 152.6 MB contra 76.3 MB
```

Listagem 436: Saída de `src/layout/`: metade da memória, num atributo.

A saída antiga chama-se **EBO** — *Empty Base Optimization*: uma classe base vazia pode ocupar zero bytes no derivado, porque a linguagem não exige endereço próprio para subobjetos de base. É por isso que a libstdc++ faz o `std::vector` **herdar** do alocador:

```
1  sizeof(std::allocator<int>)          =  1 (é VAZIO)
2  sizeof(std::vector<int>)             = 24 (3 ponteiros, nada mais)
3  → se o alocador fosse membro, o vector teria 32.
4
5  unique_ptr<int>                      =  8 bytes
6  unique_ptr<int, lambda SEM captura>  =  8 bytes (EBO)
7  unique_ptr<int, void(*)(int*)>       = 16 bytes (ponteiro!)
```

Listagem 437: Saída de `src/layout/`: a EBO na biblioteca padrão.

Armadilha — passar `&funcao` como *deleter* dobra o `unique_ptr`

Um *lambda* sem captura é um tipo vazio e some pela EBO; um ponteiro de função é um objeto de 8 bytes que precisa ser guardado. O `std::unique_ptr` sai de 8 para 16 bytes, deixa de caber num registrador e dobra o custo de guardá-lo em container (Seção II.5).

Herdar de algo que não é interface incomoda, e era a única saída antes do C++20 — o `boost::compressed_pair` existia para embrulhar essa feiura. O `[[no_unique_address]]` diz o mesmo sem herança: "este membro pode compartilhar endereço com outro".

Armadilha — dois membros vazios do mesmo tipo não colapsam

Dois subobjetos do **mesmo tipo** não podem ter o mesmo endereço, e a regra vale mesmo com o atributo. Mas isso não significa que o custo apareça na `struct` que os contém: basta haver outro membro para absorvê-los.

```
struct { [[nua]] Vazio a; [[nua]] Vazio b;       } → sizeof 2
struct { [[nua]] Vazio a; [[nua]] Comparador b; } → sizeof 1
struct { [[nua]] Vazio a, b; int dado;           } → sizeof 4
  offset de a = 0   offset de b = 1   offset de dado = 0
```

Os dois vazios têm endereços diferentes entre si, como manda a regra, e ao mesmo tempo moram *dentro* do `int` — que é exatamente o que o atributo autoriza.

Armadilha — o atributo é autorização, não garantia

O compilador pode ignorá-lo, e o MSVC de fato o ignora por compatibilidade de ABI, oferecendo `[[msvc::no_unique_address]]` no lugar. Se o *layout* importa, trave o contrato com `static_assert(sizeof(T) == N)` em vez de confiar.

Regra — toda política passada por *template* merece o atributo

Comparador, alocador, *deleter*, *hasher*, política de *mutex*: quase sempre vazios, e todos custam 8 bytes por objeto se forem membros comuns. O subprojeto mede o caso num `ConjuntoOrdenado<T, Comp>`: o `std::less<int>` custou **0 bytes**; um comparador com um `int` de estado custou 8 — e aí é justo pagar.

XIV.1.9 O vptr, e o que `virtual` custa em bytes

A Seção II.7 tratou do *comportamento* do despacho dinâmico; aqui o assunto é o preço em bytes e em endereços. Quando uma classe tem ao menos uma função `virtual`, o compilador cria para ela uma **vtable** — um vetor de ponteiros de função, guardado *no binário*, um por classe — e acrescenta a cada *objeto* um **vptr**, ponteiro para a vtable da sua classe real. É o vptr que permite a `forma->area()` chamar a implementação certa sem saber o tipo.

```
struct { int a; int b; }                  → sizeof  8
struct { int a; int b; virtual ~T(); }    → sizeof 16

os 16 bytes de um ComVirtual{a=0x11111111, b=0x22222222}:
  80 E7 47 87 CA 55 00 00 | 11 11 11 11 22 22 22 22
  '—— vptr (8 bytes) ——   '—— a ——   '—— b ——
offsetof(ComVirtual, a) = 8 — o vptr vem ANTES.
```

Listagem 438: Saída de `src/layout/`: o custo de um único `virtual`.

Um *único* método virtual custa 8 bytes por objeto; dez métodos custam os *mesmos* 8, porque o vptr é um só e a vtable, que cresce, fica no binário. A posição no início é convenção do **Itanium C++ ABI** (GCC e Clang em Linux e macOS), não exigência do padrão — mas é o que permite despachar sem conhecer o tipo, já que o vptr está sempre no *offset* 0.

Regra — destrutor virtual só onde há destruição por ponteiro-para-base

Num `vector<Particula>` de dez milhões, um `virtual ~Particula()` que ninguém usa custa **76 MB**. Se a classe não é destruída através de ponteiro para a base, não declare o destrutor virtual: torne-o `protected` e não virtual, o que impede o uso errado sem cobrar o vptr.

Herança simples, e uma surpresa. Com herança simples o subobjeto da base fica no início, o derivado acrescenta depois e o *upcast* é gratuito. Mas a medição contraria a expectativa:

```
sizeof(Base1) = 16   sizeof(Deriv1) = 16
Base1:  [vptr 8][x 4][padding 4]
Deriv1: [vptr 8][x 4][   y   4]      ← y foi para DENTRO do padding de cauda
offsetof: x=8, y=12
```

Listagem 439: Saída de `src/layout/`: o derivado tem um `int` a mais e o mesmo `sizeof`.

O compilador só pode reaproveitar o *padding* de cauda da base porque `Base1` **não é** *standard-layout* (ela tem `virtual`); se fosse, o padrão proibiria. Aqui a Seção XIV.1.7 encontra a prática: é por isso que `memcpy` de um objeto para o *subobjeto-base* de outro pode corromper o derivado — os bytes de "*padding*" da base, no destino, são o `y` de alguém. E é esse mesmo *layout* que torna o *object slicing* possível: copiar `*pb` para um `Base1` leva só a parte-base, e o resultado é um `Base1` legítimo, sem aviso nenhum.

Herança múltipla: o `this` que se desloca. Com duas bases polimórficas o objeto tem **dois** vptr — a parte-B precisa da sua própria vtable, e um `B*` tem de apontar para o começo dela.

```
struct A { int a = 1; virtual ~A() = default; virtual void fa() {} };
struct B { int b = 2; virtual ~B() = default; virtual void fb() {} };
struct AB : A, B { int c = 3; };
```

Listagem 440: As três classes de `src/layout/demo_polimorfico.hpp`.

```
&obj        offset  0
(A*)&obj    offset  0   ← igual ao início
(B*)&obj    offset 16   ← DESLOCADO!
Layout: [vptr-A][a][pad][vptr-B][b][pad][c][pad]

(void*)pa == (void*)pb ? false
pa == &obj && pb == &obj ? true
dynamic_cast<void*>(pa) == dynamic_cast<void*>(pb) ? true
```

Listagem 441: Saída de `src/layout/`: o mesmo objeto, dois endereços.

Armadilha — três consequências da herança múltipla

(1) `static_cast<B*>(&obj)` **não é um no-op**: o compilador soma 16. Um `reinterpret_cast` no lugar produziria um ponteiro errado que compila sem aviso — é a diferença entre os dois *casts* da Seção II.2, tornada visível. **(2)** Comparar ponteiros engana: `(void*)pa == (void*)pb` é *false* embora os dois apontem para o mesmo objeto; para comparar identidade, use `dynamic_cast<void*>`, que devolve o início do objeto mais derivado. **(3)** `delete pb` precisa *ajustar* o ponteiro antes de liberar, e quem faz isso é um *thunk* na vtable de B — sem destrutor virtual em B, seria UB (comportamento indefinido).

Herança virtual: o diamante que custa mais tendo menos. Quando duas classes herdam da mesma base e uma terceira herda das duas, ou a base aparece duas vezes, ou é compartilhada; `virtual` escolhe a segunda opção. O número contraria a expectativa: `sizeof(NaoDiamante)` = **40**, com a base `V` duplicada, contra `sizeof(Diamante)` = **48**, com ela compartilhada. O virtual custou *mais* bytes tendo *menos* dados.

O mecanismo explica o paradoxo. `E1` e `E2` não sabem, em compilação, *onde* `V` vai estar — quem decide é a classe mais derivada, que pode nem existir ainda. Cada uma ganha então um ponteiro extra, o *virtual base offset*, consultado em tempo de execução, e o `V` compartilhado vai para o fim do objeto. Acessar `d.v` passa

a custar uma indireção a mais que acessar `d.d`, e construir `V` vira responsabilidade da classe mais derivada, pulando os construtores de `E1` e `E2`.

Regra — herança virtual só para interfaces sem dados

Com bases puramente abstratas o custo é pequeno e o diamante é inevitável. Com *dados* na base virtual, reconsidere o desenho: composição costuma ser mais simples, mais rápida e mais fácil de explicar.

XIV.1.10 Não adivinhe: inspecione

```
clang++ -Xclang -fdump-record-layouts    # layout completo, clang
g++     -fdump-lang-class                # gera arquivos .class
pahole -C NomeDaStruct ./binario         # mostra os BURACOS
static_assert(sizeof(T) == 16);          # trava o contrato
```

Listagem 442: As quatro ferramentas de inspeção de *layout*.

O `static_assert` é o mais valioso e o único que sobrevive ao tempo: transforma uma suposição sobre *layout* numa falha de compilação no dia em que alguém acrescentar um membro. O `pahole`, do pacote `dwarves`, lê a informação de depuração do binário e imprime literalmente os buracos de *padding* de cada `struct`.

Vale fechar dizendo quando nada disso importa: com dezenas ou centenas de objetos, com um objeto atrás de um ponteiro seguido uma vez por quadro, ou quando a clareza vale mais que 8 bytes. O *layout* importa quando há milhões de objetos, quando os dados atravessam uma fronteira de ABI ou vão para disco, quando há interoperação com C ou GPU, e quando o laço quente percorre um *array* e o gargalo é o cache.

XIV.2 Apagamento de tipo

XIV.2.1 O problema, e o que a herança cobra por resolvê-lo

Por que — o buraco no sistema de tipos estático

Três `struct`s que não se conhecem e não herdam de nada — `Circulo`, `Quadrado`, `Triangulo`, cada uma com `area()`. Como guardar as três num único `std::vector`? Não há resposta boa: `vector<Circulo>` só guarda círculos; um *template* é resolvido em compilação e não dá heterogeneidade; e `void*` perde toda a informação, inclusive *como destruir*.

Type erasure — apagamento de tipo — é guardar um objeto de tipo `T` num lugar que **não menciona** `T`, preservando as operações que interessam. O tipo some da assinatura; o comportamento fica. Você já usa isso sem chamar pelo nome:

```
std::function<int(int)> f;   // aceita lambda, functor, ponteiro de função...
std::any                a;   // aceita qualquer coisa copiável
std::shared_ptr<void>   p;   // guarda o deleter de QUALQUER tipo
std::unique_ptr<Base>   b;   // a herança é a forma mais antiga
```

Listagem 443: Quatro apagadores de tipo da biblioteca padrão.

A última linha é a que quase ninguém percebe. `std::unique_ptr<Forma>` não menciona `Circulo` nem `Quadrado`: o tipo foi apagado, e a vtable / vptr é o mecanismo. Herança polimórfica *é* apagamento de tipo

— só que **intrusivo**: para entrar no vetor, o tipo precisa *herdar*. Isso exclui `int`, `std::string` e qualquer tipo de terceiros que você não pode alterar; acrescenta um vptr ao seu tipo, que deixa de ser trivial e de caber num registrador (Seção XIV.1.9); obriga a alocar no *heap* e usar ponteiro, sob pena de *object slicing*; e acopla o seu tipo a uma hierarquia que ele não escolheu.

Tabela 168: As três formas de guardar coisas heterogêneas.

Forma	Conjunto de tipos	Despacho
herança + `virtual`	**aberto**, intrusivo	vtable
`std::variant` + `visit`	**fechado**	índice / *jump table*
apagamento de tipo manual	**aberto**, não intrusivo	vtable manual

Aberto significa que qualquer um pode acrescentar um tipo novo depois, sem tocar no seu código; *fechado*, que a lista está fixa na declaração. A terceira linha é o assunto desta seção.

XIV.2.2 `std::function` do zero: Conceito, Modelo, Wrapper

A receita tem sempre as mesmas três peças (Figura 11): um **Conceito**, interface abstrata que descreve as operações mas não o tipo; um **Modelo**, *template* sobre `T`, que implementa a interface delegando para o `T`; e um **Wrapper** com semântica de valor, que guarda um ponteiro para o Conceito.

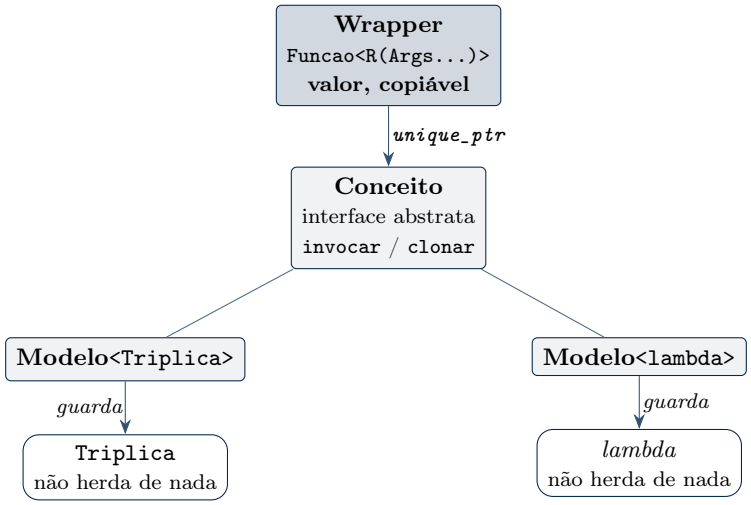


Figura 11: As três peças do apagamento de tipo. Só o **Modelo** conhece o `T`: ele herda do Conceito e some atrás do ponteiro. Quem escreveu o `T` nunca ouviu falar de nenhuma das três.

```
template <typename R, typename... Args>
class Funcao<R(Args...)> {
    // PEÇA 1: o Conceito -- o que sabemos fazer, sem saber com o quê
    struct Conceito {
        virtual ~Conceito() = default;
        virtual R                          invocar(Args...)  const = 0;
        virtual std::unique_ptr<Conceito>  clonar()          const = 0;
    };

    // PEÇA 2: o Modelo -- conhece T, implementa o Conceito
    template <typename T>
    struct Modelo final : Conceito {
        T alvo;
        explicit Modelo(T t) : alvo{std::move(t)} {}

        R invocar(Args... a) const override {
            return const_cast<T&>(alvo)(std::forward<Args>(a)...);
```

```
        }
        std::unique_ptr<Conceito> clonar() const override {
            return std::make_unique<Modelo>(alvo);   // exige T copiável
        }
    };

    // PEÇA 3: o Wrapper -- valor semântico
    std::unique_ptr<Conceito> pimpl_;

public:
    template <typename T>
        requires (!std::is_same_v<std::decay_t<T>, Funcao>)
    Funcao(T&& t)
        : pimpl_{std::make_unique<Modelo<std::decay_t<T>>>(std::forward<T>(t))} {}
};
```

Listagem 444: A receita completa (resumo de `src/type_erasure/demo_implementacao.hpp`).

Com isso, um *functor*, um *lambda* com captura e um ponteiro de função — três tipos sem base comum, que não sabem que `Funcao` existe — entram no mesmo objeto. O acoplamento é zero: o único contrato é ser invocável com `(Args...)` devolvendo algo conversível para `R`.

Armadilha — sem o `requires`, o construtor *template* sequestra a cópia

O construtor *template* aceita qualquer coisa — inclusive uma `Funcao`. Sem a restrição, `Funcao f2{f1}` escolheria o *template* em vez do construtor de cópia e construiria uma `Funcao<Funcao<...>>`: um objeto que embrulha o outro em vez de copiá-lo. Compila, roda e faz outra coisa. Toda classe com construtor *template* irrestrito tem esse bug latente.

Por que `clonar()`. `unique_ptr<Conceito>` não é copiável — logo, sem o `clonar()` virtual, a `Funcao` deixaria de ser um *valor*. Há exatamente três saídas, e cada biblioteca escolheu uma: `clonar()` virtual dá cópia profunda (é o padrão *Prototype*, e o que a `std::function` faz); `shared_ptr<const Conceito>` dá cópia rasa e barata, com o alvo compartilhado; proibir a cópia é o `std::move_only_function` do C++23. A `std::function` escolheu a primeira e paga o preço: ela **exige** *callable* copiável, e um *lambda* que captura um `unique_ptr` por move não cabe nela — `error: static assertion failed: std::function target must be copy-constructible`.

XIV.2.3 SBO: medido, não presumido

Por que — o problema que o *buffer* pequeno resolve

A `Funcao` acima aloca no *heap* **sempre**, mesmo para um *lambda* sem captura — que não guarda nada. Como o *callback* pequeno é o caso dominante, as implementações reais reservam alguns bytes *dentro* do próprio objeto e constroem o alvo ali quando ele cabe: é a **SBO**, *small buffer optimization*. Cabe até quanto? O padrão não diz, então só resta medir.

O subprojeto substitui o `operator new` global por uma versão que conta alocações e bytes, e constrói *lambdas* com capturas de tamanho conhecido:

```
sizeof(std::function<void()>)  = 32 bytes
sizeof(v1::Funcao<void()>)     =  8 bytes  (só o unique_ptr)

std::function, sem captura → 0 alocação(ões), INLINE no buffer
std::function, captura 8B  → 0 alocação(ões), INLINE no buffer
std::function, captura 16B → 0 alocação(ões), INLINE no buffer
std::function, captura 24B → 1 alocação(ões), 24 bytes
```

```
std::function, captura 64B → 1 alocação(ões), 64 bytes
v1::Funcao, sem captura    → 1 alocação(ões), SEMPRE
```

Listagem 445: Saída de `src/type_erasure/`: onde exatamente o `new` aparece.

O corte está entre 16 e 24 bytes, e o `sizeof` de 32 explica por quê: ponteiro de vtable, ponteiro de invocação e 16 bytes de *buffer*. A regra é da **implementação**, não do padrão — e a libstdc++ acrescenta uma condição própria: o *callable* precisa ser *nothrow-move-constructible* para usar o *buffer*; se o move puder lançar, ela aloca, porque mover a `function` teria de mover o *buffer*. É o mesmo raciocínio que faz o `std::vector` copiar em vez de mover ao realocar (Seção II.3).

Regra — o que cabe de graça numa `std::function`

Um *lambda* que captura `[this]`, um `int` ou dois ponteiros é gratuito. Um que captura três `std::string` por valor aloca — na construção e em cada cópia. Em código quente, prefira passar o *callable* por *template* e reserve a `std::function` para as fronteiras em que o tipo precisa mesmo ser apagado.

Armadilha — substituir o `operator new` global tem três regras não óbvias

(1) A substituição precisa estar no **escopo global**: declará-la dentro de um *namespace* é erro de compilação, porque ali ela não seria substituição, seria outra função com o mesmo nome. **(2)** Precisa ser **única no programa** — é uma função com *linkage* externa reservada —, e por isso não pode ser `inline`: o cabeçalho que a contém só pode ser incluído por *uma* unidade de tradução (Seção XIV.3). **(3)** Os `operator delete` pareados também precisam ser substituídos: misturar o `new` substituído com o `delete` padrão é UB (comportamento indefinido). De quebra, o GCC emite um `-Wmismatched-new-delete` falso-positivo ao *inlinar* os corpos, suprimido no subprojeto com *pragma* local.

XIV.2.4 Vtable manual, sem herança nenhuma

A versão acima usa `virtual`, o que é legível e caro: o objeto guardado ganha um vptr, e vptr no *buffer* complica o move. Dá para apagar o tipo **sem herança**: um `struct` de ponteiros de função, `static constexpr`, uma instância por tipo `T`.

```cpp
template <typename R, typename... Args>
class Funcao {
    // A "vtable": um struct de ponteiros de função, UMA instância estática
    // por tipo  T. Sem 'virtual',  sem vptr no objeto.
    struct VTable {
        R     (*invocar)(void*, Args...) ;
        void (*destruir)(void*);
        void*(*clonar)(void*);
    };

    template <typename T>
    static const VTable* tabela_de() {
        static constexpr VTable t{
            +[](void* p, Args... a)  -> R {
                return (*static_cast<T*>(p))(std::forward<Args>(a)...);
            },
            +[](void* p) { delete static_cast<T*>(p); },
            +[](void* p) -> void* { return new T{*static_cast<T*>(p)}; },
        };
        return &t;
    }

    void*          alvo_ = nullptr;
    const VTable* vt_   = nullptr;
};
```

Listagem 446: A vtable escrita à mão (resumo de `src/type_erasure/demo_implementacao.hpp`).

É literalmente o que o compilador faz com `virtual`, escrito à mão — e essa diferença aparentemente estética muda o que é possível (Tabela 169). Sem vptr, o objeto guardado pode ir direto para o *buffer* interno: é por isso que a libstdc++ escolhe esta forma. O `virtual` da primeira versão é mais claro e serve para aprender.

Tabela 169: `virtual` contra vtable manual.

Aspecto	`virtual`	vtable manual
ponteiro no **objeto**	sim (vptr)	**não**
permite SBO	difícil	**sim**
escolher as operações	fixo	por chamada
legibilidade	melhor	pior

XIV.2.5 O apagamento que já vem no padrão

Cada apagador da biblioteca padrão preserva um conjunto diferente de operações, e é essa escolha que os distingue: `std::function` preserva **chamar**; `std::any`, destruir e copiar (nem imprimir dá); `std::shared_ptr<T>`, **destruir certo**; `std::move_only_function`, chamar sem exigir cópia; `std::type_index`, identidade de tipo em execução.

`std::any` exige o tipo **exato** na recuperação: `any_cast<long>` sobre um `int` lança `bad_any_cast`, porque a comparação é `typeid` exato e não há conversão nenhuma — a forma de ponteiro devolve `nullptr` em vez de lançar. Quando usar: quase nunca. Antes, veja se um `variant` serve (Seção III.4) ou se um apagamento sob medida preserva exatamente as operações necessárias. `any` é para guardar algo e devolvê-lo intacto **sem nunca olhar dentro**.

Armadilha — o *deleter* apagado do `shared_ptr`

Em `std::unique_ptr<T, D>` o *deleter* está **no tipo**; em `std::shared_ptr<T>`, está no **bloco de controle**, com o tipo apagado — e isso tem três consequências. **(1)** Um `shared_ptr<Base>` destrói corretamente mesmo *sem* destrutor virtual, porque o *deleter* foi capturado quando o tipo ainda era o derivado (o subprojeto verifica que `~Contada` roda). Isso **não** é licença para omitir o destrutor virtual: só funciona se o `shared_ptr` foi criado a partir do tipo derivado. **(2)** Recursos de C saem de graça: `std::shared_ptr<FILE> f{fopen(...), &fclose}`. **(3)** O *aliasing constructor* guarda um ponteiro e mantém vivo *outro* objeto. O preço é o bloco de controle — uma alocação, evitável com `make_shared` — e uma indireção a mais na destruição.

O `std::move_only_function` do C++23 aceita o *lambda* que captura por move e, de quebra, acertou os qualificadores da assinatura: `move_only_function<int() const>` tem `operator()` constante de verdade, `<int() &&>` só é invocável em *rvalue*, e `<int() noexcept>` propaga o `noexcept`. A `std::function` faz um `const_cast` interno — exatamente como a versão didática acima — e portanto *mente sobre const-correção*.

Já o `std::type_index` é o embrulho que torna `std::type_info` — não copiável e sem `operator<` — utilizável como chave de `map`. Com ele se constrói o despacho dinâmico **aberto**: um `map<type_index, handler>` em que qualquer módulo registra um tipo novo sem tocar em nada, base do *plugin registry* (Seção V.4) e do ECS (Seção V.3). O custo é uma busca em `std::map` por chamada, contra uma indireção só na vtable.

Armadilha — `typeid(T).name()` é manglado e não é portável

O nome devolvido é o símbolo decorado pelo compilador e muda entre compiladores e entre versões do mesmo compilador. Para exibir, use `abi::__cxa_demangle` do GCC e do Clang. **Nunca serialize esse nome**: um arquivo gravado hoje pode não ser lido pelo binário de amanhã.

XIV.2.6 A medição que decide

Tabelas comparando `virtual`, CRTP, `variant` e apagamento de tipo existem em todo lugar; o que falta é o número. O subprojeto soma a área de 200 000 formas, alternando círculos e quadrados para que a predição de desvio não favoreça ninguém, com média de 20 rodadas e `-O2`:

```
virtual  (unique_ptr<Base>)          0.514 ms/rodada   3.9×
CRTP (vetores homogêneos)            0.130 ms/rodada   1.0×
variant + visit                      0.131 ms/rodada   1.0×
type erasure (unique_ptr interno)    0.517 ms/rodada   4.0×

Memória por elemento:
  virtual       :  8 B (ponteiro) + heap: vptr + dado
  CRTP          :  8 B, contíguo, SEM vptr
  variant       : 16 B, contíguo (maior alternativa + tag)
  type erasure  :  8 B (ponteiro) + heap
```

Listagem 447: Saída de `src/type_erasure/`: 200 000 elementos, 20 rodadas, `-O2`.

Por que — o que domina o resultado não é o despacho

A leitura ingênua atribuiria o 4× ao custo da chamada indireta, e os próprios números a desmentem: `variant + visit` *também* é despacho dinâmico — um salto por tabela — e empatou com o CRTP, que não tem despacho nenhum. O que separa os dois grupos é a **localidade**: `variant` e CRTP percorrem memória contígua, enquanto `virtual` e apagamento de tipo percorrem um vetor de *ponteiros* para objetos espalhados no *heap*. Uma falta de cache custa cerca de 200 ciclos; uma chamada indireta bem predita, cerca de 2. O 4× é o cache falando, não a vtable — e a consequência prática é que eliminar o `virtual` sem mudar o *layout* não resolveria nada.

XIV.2.7 O problema da expressão

Você tem **tipos** (círculo, quadrado, ...) e **operações** (área, perímetro, desenhar, ...), e só dá para deixar *um* dos dois eixos barato de estender.

Tabela 170: O problema da expressão: cada abordagem escolhe um eixo.

Abordagem	Tipo novo	Operação nova
`virtual`	fácil: 1 classe	**caro**: mexe na base e em todo derivado
`variant + visit`	**caro**: recompila todos os *visitors*	fácil: outro *visitor*
apagamento de tipo	fácil, e sem tocar no tipo	**caro**: mexe no `Conceito`
CRTP	fácil	fácil — *mas sem heterogeneidade em execução*

A última linha é uma armadilha de leitura: o CRTP parece ganhar dos dois lados porque **não resolve o mesmo problema** — ele não dá container heterogêneo. Se você precisa de um `vector<Forma>` com tipos decididos em execução, o CRTP não é candidato; foi por isso que a medição da Seção XIV.2.6 precisou dar a ele *dois vetores homogêneos*.

Regra — como decidir, na ordem

1. Tipos conhecidos, lista curta e estável? → `variant`: contíguo, sem alocação, exaustividade verificada em compilação. **2.** Tipos abertos e você controla as classes? → herança + `virtual`, o mais legível. **3.** Tipos abertos e você **não** controla as classes (`int`, `std::string`, terceiros)? → apagamento de tipo. **4.** Sem heterogeneidade em execução, só reuso de código? → *template*/CRTP, e nem pense em vtable.

XIV.2.8 Polimorfismo com semântica de valor

A crítica de Sean Parent (*Inheritance Is The Base Class of Evil*, GoingNative 2013) cabe em uma frase: herdar para ser polimórfico contamina o seu tipo com ponteiro, *heap* e *aliasing* — problemas que ele não tinha. A alternativa é uma inversão simples de enunciar e difícil de perceber sozinho: **o polimorfismo mora no container, não no tipo guardado**.

```
// O contrato, explícito. Sem ele, um tipo inadequado gera erro dentro de
// Modelo<T> -- dez níveis de template abaixo da linha do usuário.
template <typename T>
concept SabeDesenhar = requires(const T& t, int nivel) {
    { t.desenhar(nivel) } -> std::same_as<void>;
    { t.nome() }          -> std::convertible_to<std::string>;
};

class Desenhavel {
    struct Conceito {
        virtual ~Conceito() = default;
        virtual void                       desenhar(int) const = 0;
        virtual std::unique_ptr<Conceito> clonar()       const = 0;
    };
    template <typename T>
    struct Modelo final : Conceito {
        T v;
        explicit Modelo(T x) : v{std::move(x)} {}
        void desenhar(int n) const override { v.desenhar(n); }
        std::unique_ptr<Conceito> clonar() const override {
            return std::make_unique<Modelo>(v);
        }
    };
    std::unique_ptr<Conceito> p_;

public:
    template <SabeDesenhar T>
    Desenhavel(T x) : p_{std::make_unique<Modelo<T>>(std::move(x))} {}

    // Rule of 5 -- é isto que dá semântica de valor
    Desenhavel(const Desenhavel& o) : p_{o.p_->clonar()} {}
    Desenhavel& operator=(const Desenhavel& o) {
        if (this != &o) p_ = o.p_->clonar();
        return *this;
    }
    Desenhavel(Desenhavel&&) noexcept            = default;
    Desenhavel& operator=(Desenhavel&&) noexcept = default;
    ~Desenhavel()                                = default;

    void desenhar(int n = 0) const { p_->desenhar(n); }
};
```

Listagem 448: O tipo-valor polimórfico (resumo de `src/type_erasure/demo_pratica.hpp`).

O que isso compra é uma cena copiável:

```
std::vector<Desenhavel> cena;          // por VALOR, não unique_ptr
cena.emplace_back(Ponto{1, 2});
cena.emplace_back(Texto{"olá"});
cena.emplace_back(Grupo{"sub", {Ponto{9,9}, Texto{"aninhado"}}});
```

```
5
6 std::vector<Desenhavel> copia = cena;  // PROFUNDA -- nem compilaria
7                                        // com unique_ptr<Base>
```

Listagem 449: Uma cena por valor, com cópia profunda.

`Ponto`, `Texto` e `Grupo` são `struct`s comuns: sem vptr, copiáveis, usáveis fora de qualquer cena. E o `Grupo`, que guarda `std::vector<Desenhavel>`, é o padrão *Composite* construído sobre valores em vez de ponteiros (Seção V.2).

Tabela 171: `unique_ptr<Base>` contra o tipo-valor polimórfico.

Aspecto	`unique_ptr<Base>`	`Desenhavel`
copiável	**não**	sim
o tipo precisa herdar	sim	**não**
funciona com `int`/`string`	não	se atender ao *concept*
pode ser `nullptr`	sim	**não**
`==` e `<`	compara **ponteiro**	pode ser por valor
aloca	sim	sim

A linha "pode ser `nullptr`" merece atenção: `Desenhavel` não tem construtor padrão, logo não existe "desenhável vazio" — o mesmo argumento de `optional<T>` contra `T*` da Seção III.4. E a última linha é a honestidade necessária: os dois alocam e pagam indireção. O ganho aqui é de **projeto**, não de velocidade: a medição pôs os dois na mesma faixa, cerca de 4× o CRTP.

Por que — por que o *concept* não é enfeite

Sem o `SabeDesenhar`, um tipo inadequado produz o erro *dentro* de `Modelo<T>`, com cinco linhas de "*required from*". Com ele, o erro volta para a linha do usuário e diz qual expressão faltou: `note: the required expression 't.desenhar(nivel)' is invalid`. E o *concept* vira documentação executável — "o que preciso implementar para caber aqui?" passa a ter resposta em quatro linhas de código, e não num comentário que envelhece (Seção IV.2).

Armadilha — quando NÃO fazer apagamento de tipo

(a) Você controla todos os tipos e a lista é curta: `variant` é 4× mais rápido, não aloca e verifica exaustividade — o apagamento de tipo não tem exaustividade nenhuma. **(b)** O conjunto de operações é grande ou instável: cada operação nova entra em *três* lugares (Conceito, Modelo e *wrapper*), e uma interface de vinte métodos vira sessenta linhas de repasse mecânico. **(c)** Não há heterogeneidade em execução: um *template* resolve em compilação e some. **(d)** `virtual` já resolve e é mais legível: são cerca de 40 linhas de *boilerplate* para o que `virtual` faz em 3.

Existem bibliotecas que geram esse *boilerplate* — `dyno`, de Louis Dionne; `Boost.TypeErasure`; `folly::Poly` — e todas custam macro ou muita metaprogramação, porque a linguagem ainda não tem reflexão. O C++26 deve mudar esse quadro.

XIV.3 Da unidade de tradução ao binário

XIV.3.1 O que é uma unidade de tradução

O C++ não compila "o projeto". Ele compila **uma unidade de tradução por vez**, em total ignorância das outras. Uma **unidade de tradução** (TU, de *translation unit*) é *um `.cpp` mais tudo o que os `#include`*

dele trouxeram, recursivamente. Cada TU vira um arquivo-objeto com duas listas — os símbolos que ela **define** e os que **precisa** — e o *linker* cola tudo no fim (Figura 12). Praticamente todo erro confuso de *build* é consequência direta desse modelo.

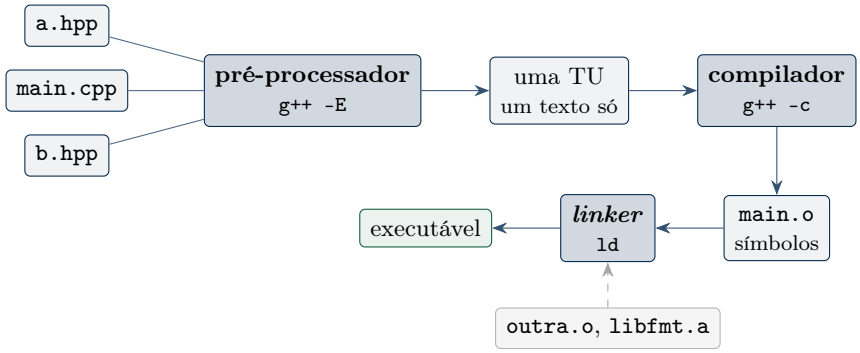


Figura 12: O *pipeline* de `src/linkage/`. O compilador nunca vê duas TUs ao mesmo tempo; quem descobre que faltou ou sobrou uma definição é o *linker* — e por isso os erros dele são de outra natureza.

Ver cada etapa é uma linha de *shell*: `g++ -E` para o pré-processador, `g++ -c` para o objeto, `nm -C arq.o` para os símbolos desmanglados e `nm -C -undefined-only arq.o` para os que a TU *precisa*.

XIV.3.2 Quanto custa um `#include`

```
1  arquivo vazio      →      6 linhas
2  <cstddef>          →    253
3  <vector>           →  26.258
4  <algorithm>        →  28.270
5  <string>           →  31.825
6  <iostream>         →  43.165
7  <ranges>           →  44.309
```

Listagem 450: Saída de `src/linkage/`: linhas depois do pré-processador, GCC 13.3 com `-std=c++20`.

Cada TU paga esse custo **inteiro**, de novo. Um projeto com cem `.cpp` que incluem `<string>` compila `<string>` cem vezes: é por isso que *build* de C++ é lento.

Regra — reproduza os números que você cita

A primeira versão desta tabela, no repositório, era **chutada**: cinco dos seis números estavam errados, e o `<cstddef>` errado por um fator de quatro. Reproduzir custou quatro linhas de *shell* — `echo "#include <$h>" > /tmp/t.cpp` seguido de `g++ -std=c++20 -E /tmp/t.cpp | wc -l` para cada cabeçalho. Os valores mudam com a versão da libstdc++, o que é mais um motivo para medir no seu *toolchain* em vez de copiar os de alguém.

As saídas, em ordem de eficácia: **módulos** (C++20 — a interface é compilada uma vez e importada em forma binária); **declaração adiantada** (`class Widget;` em vez de `#include`, quando só se usa ponteiro ou referência); e **pimpl**, assunto da Seção XIV.3.7. *Precompiled headers* e *unity builds* atacam o sintoma.

Regra — reconhecer a etapa resolve metade do problema

Erro do **compilador** → falta uma **declaração**. Erro do **linker** → falta (ou sobra) uma **definição**. Nenhuma quantidade de `#include` conserta um `undefined reference`.

A última linha da tabela é a que ninguém adivinha. O GCC emite a vtable / vptr de uma classe na TU que define a **primeira função virtual não-`inline` e não-pura** — a *key function*. Se você declarou `virtual`

Tabela 172: Cada mensagem, a etapa em que nasce e a causa típica.

Mensagem	Etapa	Causa típica
`No such file or directory`	pré-processador	`-I` faltando
`'X' was not declared`	compilador	`#include` faltando
`invalid use of incomplete type`	compilador	só a *forward declaration*
`undefined reference to 'f()'`	**linker**	declarou, não definiu
`multiple definition of 'f()'`	**linker**	função não-`inline` em cabeçalho
`undefined reference to vtable`	**linker**	a *key function* não tem corpo

`void f();` e nunca a definiu, a vtable não é emitida em lugar nenhum, e o *linker* reclama de um símbolo que você nunca escreveu.

XIV.3.3 A ODR, medida com duas TUs reais

A **One Definition Rule** tem três partes, e só a terceira surpreende: **(1)** em *uma* TU, cada entidade pode ser declarada muitas vezes e definida no máximo uma; **(2)** no *programa*, cada função ou variável não-`inline` pode ser definida no máximo uma vez; **(3)** classes, funções `inline` e *templates* **podem** ser definidos em várias TUs — mas as definições precisam ser **token a token idênticas**. Se não forem, é UB (comportamento indefinido), e o *linker* **não é obrigado a avisar**.

Este é o único subprojeto do repositório com múltiplas unidades de tradução, e não por capricho. `unidade_a.cpp` e `unidade_b.cpp` são idênticos a menos do nome, e cada um devolve os *seus* endereços para as mesmas entidades de `comum.hpp`:

```
inline int funcao_inline() { return 1; }   // UMA definição no programa
static int funcao_static() { return 2; }   // uma cópia POR TU
namespace { int funcao_anonima() { return 3; } }   // idem, forma moderna

inline int contador_inline = 0;   // C++17: UM objeto no programa
static int contador_static = 0;   // uma cópia POR TU

const int  kConstante = 42;        // const em namespace: linkage INTERNA
constexpr  int kConstexpr = 43;   // constexpr implica const, logo interna
inline constexpr int kConstexprInline = 44;          // externa

template <typename T>
T identidade(T v) { return v; }   // instanciado em cada TU, e fundido
```

Listagem 451: O palco do experimento (resumo de `src/linkage/comum.hpp`).

```
inline int f()          MESMO endereço        → 1 no programa inteiro
static int f()          endereços DIFERENTES  → 1 por TU
namespace{} int f()     endereços DIFERENTES  → 1 por TU
inline int var          MESMO endereço        → 1 no programa inteiro
static int var          endereços DIFERENTES  → 1 por TU
const int k = 42        endereços DIFERENTES  → 1 por TU
inline constexpr int    MESMO endereço        → 1 no programa inteiro
template<T> T f(T)      MESMO endereço        → 1 no programa inteiro
```

Listagem 452: Saída de `src/linkage/`: os endereços vêm do *linker*, não de simulação.

Quatro regras saem daí de uma vez: `inline` faz o *linker* **fundir** as cópias das TUs; `static` e o *namespace* anônimo dão *linkage* interna, e cada TU fica com a sua; `const` em escopo de *namespace* é implicitamente `static` em C++ (em C seria externa!); e um *template* é instanciado em cada TU, com as cópias fundidas como se fossem `inline`.

Armadilha — o `static` em cabeçalho: N variáveis, não uma

As duas TUs incrementam *os dois* contadores dez vezes cada:

```
| variável          | vista de A | vista de B |
| contador_inline   |         20 |         20 |   ← um objeto
| contador_static   |         10 |         10 |   ← DUAS variáveis
```

O bug é traiçoeiro por três motivos: linka sem erro nenhum; funciona perfeitamente enquanto houver *uma* TU incluindo o cabeçalho; e quebra no dia em que alguém o inclui num segundo `.cpp` — quando o sintoma ("o *cache* não persiste") não aponta para o cabeçalho. Antes do C++17 a saída era `extern` no cabeçalho mais uma definição num `.cpp`; desde o C++17, `inline` resolve no próprio cabeçalho.

Por que — `inline` não é sobre *inlining*

A palavra engana desde 1998. `inline` significa *"esta entidade pode ser definida em várias TUs; o* linker *deve escolher uma e descartar as outras"*. Ela não obriga o compilador a expandir a chamada — essa decisão é do otimizador, que a toma sozinho mesmo em funções não marcadas e a ignora em funções marcadas que sejam grandes demais. Para **forçar** a expansão existe `[[gnu::always_inline]]`; para **impedir**, `[[gnu::noinline]]`. São implicitamente `inline`: funções definidas no corpo da classe, funções `constexpr` e `consteval`, e membros `static constexpr` (C++17).

Armadilha — a violação de ODR que linka, roda e corrompe

```cpp
struct Widget {
    int a;
#ifndef NDEBUG
    int contador_de_depuracao;   // só em debug
#endif
};
// a.cpp SEM -DNDEBUG → sizeof(Widget) == 8
// b.cpp COM -DNDEBUG → sizeof(Widget) == 4
```

As duas TUs discordam sobre o *layout* da mesma classe. O *linker* não compara definições de classe — ele só vê nomes de símbolo. O programa linka, roda, e escreve fora dos limites do objeto quando cruza a fronteira. Outras formas da mesma armadilha: `-D` diferente entre alvos; `#pragma pack` ativo numa TU e não na outra; `-std` diferente; linkar *Debug* com *Release* da mesma biblioteca (`_GLIBCXX_ASSERTIONS` muda o *layout* dos containers); duas versões de uma dependência *header-only* na mesma árvore. Defesas: **flags uniformes** — é para isso que serve o *build system* (Seção XI.2) —, `-flto` com `-Wodr`, `-detect-odr-violations` no `gold`/`lld`, o ASan (Seção I.9), e **nunca** `#if` em torno de membro de classe pública.

XIV.3.4 Os três tipos de *linkage*

Linkage responde a uma pergunta só: **este nome pode se referir à mesma entidade que um nome igual em outro lugar? Externa**: sim, inclusive em outra TU. **Interna**: sim, mas só dentro desta TU. **Nenhuma**: não; o nome só existe onde foi declarado.

Regra — não confunda *linkage*, escopo e tempo de vida

São três eixos independentes: *linkage* diz quem pode se referir ao nome, **escopo** diz onde o nome é visível, **tempo de vida** diz quando o objeto existe. Uma variável `static` dentro de uma função tem *linkage* **nenhuma**, escopo de **bloco** e tempo de vida do **programa inteiro** — três respostas diferentes, na mesma declaração.

Tabela 173: *Linkage* por forma de declaração.

Declaração em escopo de *namespace*	***Linkage***
`int x;` `void f(){}` `inline void f(){}`	**externa**
`static int x;` `static void f(){}`	interna
`const int x = 1;`	interna — *a surpresa*
`constexpr int x = 1;`	interna (implica `const`)
`inline constexpr int x = 1;`	**externa**
`namespace { int x; }`	interna
`extern const int x = 1;`	**externa**
`typedef` / `using` / *enum* sem nome	nenhuma
Declaração em escopo de bloco	
`int x;` (local comum)	nenhuma
`static int x;` (local estático)	nenhuma — *mas dura o programa todo*
`extern int x;`	**externa**

A linha que mais surpreende é `const int x = 1;`. Em C ela teria *linkage* externa; em C++, interna. A razão é prática: permite pôr constantes em cabeçalhos sem `multiple definition` — cada TU ganha a sua cópia e, como são constantes iguais, ninguém percebe. O preço é o que a medição mostrou: endereços diferentes em cada TU. Se você precisa de *um* objeto, escreva `inline constexpr`.

XIV.3.5 `static`: uma palavra, quatro significados

Tabela 174: Os quatro `static`, que não têm nada em comum entre si.

Onde aparece	**O que significa**
escopo de *namespace*	*linkage* **interna**
membro de classe	**um por classe**, não por objeto (fora do `sizeof`)
método de classe	**sem `this`**, chamável sem objeto
local de função	**tempo de vida do programa**, *linkage* nenhuma

Quatro conceitos ortogonais na mesma palavra — herança do C, que o C++ não pôde consertar sem quebrar tudo. Para o primeiro significado, prefira o ***namespace* anônimo**: mesmo efeito, sem ambiguidade, e funciona também para **tipos**, coisa que `static` não faz. O compilador dá ao *namespace* um nome único por TU e insere um `using namespace` implícito; os nomes ficam inacessíveis de fora, e o otimizador ganha uma informação valiosa — ninguém de fora chama, então ele pode *inlinar* ou eliminar à vontade.

Armadilha — *namespace* anônimo nunca em cabeçalho

Cada TU que incluir ganha a sua cópia: é o mesmo bug do `static` em cabeçalho, agora com **tipos** — duas classes com o mesmo nome e identidades diferentes, o que faz `dynamic_cast` e `typeid` discordarem entre TUs.

Armadilha — o `static` local é *thread-safe*, mas não de graça

Desde o C++11 a **inicialização** de um `static` local é garantidamente *thread-safe*: se duas *threads* chegarem juntas, uma inicializa e a outra espera. O subprojeto mede — 8 *threads* chamando ao mesmo tempo, **1 inicialização** —, e é isso que torna o *Meyers singleton* correto sem *mutex*. O custo é uma *guard variable* e uma checagem atômica em *cada* chamada (`__cxa_guard_acquire`); depois da primeira vez é uma leitura atômica relaxada, barata mas não gratuita. E a garantia é sobre a *inicialização*, não sobre o uso: se várias *threads* mutam o objeto depois, ainda é preciso sincronizar (Seção VI).

O `static` local resolve, de quebra, o *static initialization order fiasco*: globais de TUs diferentes têm ordem de inicialização **indefinida** entre si, ao passo que um `static` local é inicializado na primeira chamada — quando tudo de que ele depende já existe.

Quanto a `extern`, ele diz "isto é uma **declaração**, não uma definição". Era o único jeito de ter uma global compartilhada antes do C++17; hoje `inline int nivel_de_log = 3;` faz o mesmo em uma linha. Continua necessário em três casos: `extern const int k;` (forçar *linkage* externa numa constante), `extern "C"` (*linkage* de *linguagem*, coisa diferente) e `extern template` (suprimir a instanciação numa TU para acelerar o *build*).

XIV.3.6 *Name mangling* e `extern "C"`

Em C, o símbolo de `int soma(int,int)` é `soma`. Em C++ é `_Z4somaii` — porque o C++ precisa distinguir sobrecargas, *namespaces* e instanciações de *template*, e o formato de arquivo objeto só tem *strings* para trabalhar. Essa codificação é o **name mangling**.

```
int  f(int)            → _ZN13demo_extern_c8exemplos1fEi
int  f(double)         → _ZN13demo_extern_c8exemplos1fEd
int  f(int, int)       → _ZN13demo_extern_c8exemplos1fEii
extern "C" f_c(int) → f_c                          ← nome CRU

_Z                 prefixo de nome manglado
N ... E            nome aninhado (namespaces)
13demo_extern_c    comprimento + nome
8exemplos          comprimento + nome
1f                 comprimento + nome da função
ii                 os PARÂMETROS: int, int
```

Listagem 453: Saída de `src/linkage/`: os símbolos reais do binário, copiados de `nm`.

O *mangling* codifica *namespace*, nome e **tipos dos parâmetros**. Ele **não** codifica o tipo de *retorno* — e é exatamente por isso que sobrecarregar apenas pelo retorno é proibido na linguagem: os dois símbolos seriam o mesmo. Para ler à mão, `nm -C` já desmangla, e `echo '_Z4somaii' | c++filt` resolve casos avulsos.

Armadilha — o esquema de *mangling* não é padronizado

GCC e Clang usam o Itanium C++ ABI; o MSVC usa outro, completamente diferente. É por isso que um `.lib` do MSVC não linka com o MinGW — e é o motivo mais profundo de toda API binária estável do mundo ser exposta em C.

`extern "C"` diz: "para esta declaração, use a convenção do C". Não muda a linguagem em que o código é escrito — muda como o **símbolo** é nomeado. O subprojeto tem uma API em C *de verdade*, compilada pelo `cc` e não pelo `c++` (o Meson escolhe o compilador pela extensão do arquivo), e o cabeçalho que os dois lados compartilham traz o idioma canônico:

```c
#ifdef __cplusplus
extern "C" {
#endif

int          soma_c(int a, int b);
const char* nome_da_linguagem_c(void);

typedef int (*transformador_t)(int);
int aplica_c(transformador_t f, int v);

#ifdef __cplusplus
}  /* extern "C" */
#endif
```

Listagem 454: O idioma que todo cabeçalho de biblioteca C traz no topo (resumo de `src/linkage/c_api.h`).

Sem ele, o C++ geraria uma chamada para `_Z6soma_cii` e o *linker* responderia `undefined reference to 'soma_c(int, int)'` — enquanto `nm c_api.o` mostraria, ali, um `T soma_c`. É o erro de integração C/C++ mais comum que existe, e o `__cplusplus` é o que permite ao *mesmo* arquivo servir aos dois compiladores.

Armadilha — o que `extern "C"` proíbe — e o que ele não protege

Não permite: sobrecarga (há um único símbolo `soma`, não dois), *templates* com *linkage* C, funções membro não estáticas. **Permite**: usar C++ à vontade *dentro* do corpo — classes, exceções, STL. E não protege contra a pior das travessias: **exceções não atravessam**. Uma exceção que escapa de uma função `extern "C"` chamada a partir de C é UB (comportamento indefinido) — as *frames* de C não têm informação de *unwinding*, e os destrutores intermediários não rodam:

```
extern "C" int minha_api(int x) noexcept {
    try { return impl(x); }
    catch (const std::exception& e) { guarda_erro(e.what()); return -1; }
    catch (...) { return -2; }
}
```

Erros viram **códigos de retorno** na fronteira — é o motivo de toda API de C ser assim (Seção II.8). E a outra fronteira que vaza: um objeto alocado com `new` no C++ **não** pode ser liberado com `free` no C. Exporte sempre um `minha_api_free(ptr)` junto do que aloca.

Sobre *callbacks*: um *lambda* sem captura converte para ponteiro de função, mas o resultado tem *linkage* C++ — e, a rigor, o tipo de um ponteiro para função com *linkage* C é diferente do de uma função com *linkage* C++ (GCC e Clang aceitam os dois; o padrão não garante). Um *lambda* **com** captura não converte de jeito nenhum, porque não há onde guardar o estado; o idioma é o parâmetro `void* user_data` que quase toda API de C oferece, com uma função `extern "C"` de trampolim.

XIV.3.7 ABI: o contrato binário

Por que — por que a ABI é pior de quebrar do que a API

API é o contrato de *código-fonte*: quais funções existem, com que assinatura. Quebrá-la é desagradável e **seguro** — o compilador reclama, alguém conserta, recompila. **ABI** é o contrato *binário*. Quebrá-la sem quebrar a API é o pior dos mundos: recompila, linka, e **corrompe memória em tempo de execução** — sem uma linha de aviso.

Fazem parte da ABI mais coisas do que se imagina: os nomes manglados; o `sizeof` e o *layout* de cada classe exposta; a **ordem das entradas na vtable**; a convenção de chamada; quem destrói cada argumento; o formato das exceções — e o *layout* de **todo tipo da biblioteca padrão que apareça na sua interface**. Esse último é o que pega: se a sua biblioteca expõe `void configura(const std::string&)`, o *layout* de `std::string` faz parte da **sua** ABI. Foi exatamente isso o rompimento do GCC 5 (`_GLIBCXX_USE_CXX11_ABI`): `std::string` deixou de ser *copy-on-write* e todo o ecossistema teve de recompilar — o símbolo passou a carregar `__cxx11` no nome justamente para que a incompatibilidade virasse **erro de *linker*** em vez de corrupção silenciosa.

Duas linhas surpreendem sistematicamente. "Acrescentar parâmetro com valor padrão": o valor padrão é gravado no lado do **chamador**, de modo que um cliente antigo, não recompilado, continua chamando um símbolo que não existe mais. E "acrescentar método virtual": a vtable muda de tamanho e os índices

Tabela 175: Mudanças que quebram a ABI sem quebrar a API.

Mudança	API	ABI
acrescentar ou reordenar membro de dado	ok	✗
acrescentar ou reordenar método **virtual**	ok	✗
tornar virtual um método comum	ok	✗
acrescentar parâmetro com valor padrão	ok	✗
mudar um *enum* de `int` para `char`	ok	✗
tornar `inline` uma função	ok	✗
acrescentar método **não** virtual	ok	ok
acrescentar sobrecarga nova	ok	ok
acrescentar membro `static`	ok	ok
mudar o **corpo** de uma função	ok	ok

seguintes se deslocam, então um cliente antigo chamando `->f()` pega o *slot* errado e executa outra função, com os argumentos errados. É a Seção XIV.1.9 cobrando a conta.

XIV.3.8 Pimpl: o *firewall* de ABI

O idioma *pointer to implementation* responde às duas pressões de uma vez: a classe pública tem um único membro — um ponteiro — e tudo o mais vive numa `struct` que o cabeçalho nunca vê.

```
1  class Widget {
2      struct Impl;                                  // só DECLARADA aqui
3      std::unique_ptr<Impl> pimpl_;
4
5  public:
6      explicit Widget(std::string nome);
7
8      // Declarados aqui e definidos em widget.cpp: `unique_ptr<Impl>` precisa
9      // do tipo COMPLETO para instanciar o seu deleter.
10     ~Widget();
11     Widget(Widget&&) noexcept;
12     Widget& operator=(Widget&&) noexcept;
13
14     std::string descricao() const;
15     void        renomear(std::string novo);
16 };
```

Listagem 455: A metade pública (resumo de `src/linkage/widget.hpp`).

`sizeof(Widget)` é **8 bytes**, e continuará 8 para sempre, independentemente do que a `Impl` ganhe ou perca. Os ganhos são ABI estável e *build* rápido — o cabeçalho não inclui nada do que a implementação usa, o que ataca diretamente a medição do `#include`. Os custos são uma alocação por objeto, uma indireção por acesso e a impossibilidade de *inlinar* através da fronteira.

Armadilha — o destrutor do pimpl não pode ser `= default` no cabeçalho

`~Widget()` precisa do tipo **completo** de `Impl` para instanciar o *deleter* do `std::unique_ptr`, e no cabeçalho `Impl` é incompleta. O erro é críptico — `static assertion failed: can't delete an incomplete type` — e a solução é fixa: **declare** no `.hpp` e escreva `= default` no `.cpp`, depois da definição de `Impl`. O mesmo vale para o construtor e o operador de move.

Armadilha — a segunda armadilha do pimpl custou um erro de *linker* real

Na primeira versão do subprojeto, `Widget` estava declarada em `demo_abi.hpp` e `widget.cpp` incluía esse cabeçalho para alcançá-la. Como as funções `demo_*()` daquele arquivo não são `inline`, elas passaram a ser definidas em **duas** TUs: `multiple definition of 'demo_abi::demo_o_que_e()'`. É o erro da Tabela 172, ao vivo, cometido por quem estava escrevendo sobre ele. A correção é a que todo projeto acaba adotando: um cabeçalho pequeno, com só o que precisa ser compartilhado entre TUs, e **nada definido**.

Inline namespaces. Declarar `inline namespace v2` dentro de `namespace biblioteca` faz os nomes de dentro aparecerem como se estivessem em `biblioteca` — o usuário escreve `biblioteca::versao()` sem versão nenhuma —, mas o símbolo emitido carrega o `v2` no *mangling*. Um binário antigo, que pedia `v1`, continua encontrando o símbolo de `v1` se ele ainda existir: duas versões incompatíveis **coexistem** no mesmo processo. É como a libstdc++ faz — `std::__cxx11::string` é exatamente isto — e também é o mecanismo de `std::literals::chrono_literals`.

Regra — decida cedo o que é ABI no seu código

Todo mundo sabe que `std::unordered_map` (um nó alocado por elemento) é lento comparado a uma tabela de endereçamento aberto, que `std::regex` é ordens de grandeza mais lento que RE2, e que `std::vector<bool>` é a especialização que todos lamentam (Seção III.1). Consertar qualquer um muda o *layout*, quebra a ABI e derruba toda biblioteca binária do planeta que os exponha na interface — por isso nenhum foi consertado. A lição para o seu código: se você publica uma biblioteca binária, escolha **antes** da primeira versão o que é ABI — pimpl nas classes que vão crescer, tipos da biblioteca padrão fora das assinaturas públicas, nenhuma função virtual em classe que os clientes instanciam. Depois de publicado, a escolha é irreversível: pergunte à `std`.

XIV.4 Síntese

As três seções parecem tratar de assuntos distantes — *padding*, `std::function`, *linker* — e são três vistas do mesmo objeto: o **modelo de objeto** que o compilador implementa por baixo da sintaxe. A prova é que cada uma explica as armadilhas das outras. O vptr da Seção XIV.1 é o que torna o apagamento de tipo por herança *intrusivo*, e é por isso que a `std::function` real usa vtable manual: sem vptr no objeto guardado, ele cabe no *buffer*. O *layout* de classe *é* metade da ABI da Seção XIV.3 — acrescentar um membro de dado e acrescentar um método virtual quebram o binário pela mesma razão. A medição da Seção XIV.2.6, em que a localidade dominou o despacho por um fator de quatro, é a Seção XIV.1 explicando por que reordenar membros vale mais que trocar o mecanismo de chamada. E a substituição do `operator new` global, ferramenta de medição do `src/type_erasure/`, só é correta por causa das regras de ODR do `src/linkage/`.

Há também um padrão nas medições, e ele é o argumento central deste livro: **três das principais surpresas do capítulo contrariaram a expectativa razoável**. O diamante com herança virtual ficou *maior* tendo *menos* dados. O derivado com um `int` a mais teve o *mesmo* `sizeof` da base. E o 4× do apagamento de tipo não veio da chamada indireta, mas do cache — de modo que a "otimização" óbvia, eliminar o `virtual`, não teria resolvido nada. Nos três casos, quem raciocinasse sem medir chegaria à conclusão errada com toda a convicção.

Regra — as três ferramentas deste capítulo

`pahole -C Struct ./binario` mostra os buracos de *padding* que o `sizeof` esconde. Um `operator new` global instrumentado — quinze linhas — responde "isto aloca?" para qualquer biblioteca, inclusive a padrão. E `nm -C` com `c++filt` transformam `undefined reference` de mistério em pergunta respondível. Nenhuma das três exige mudar o código de produção, e todas substituem uma discussão por um número.

XV Corretude e desempenho

Um programa em C++ pode errar de três maneiras distintas, e elas exigem ferramentas distintas. Pode dar a **resposta errada** — e o instrumento é o teste automatizado. Pode não ter resposta *nenhuma* definida pelo padrão — é o UB (comportamento indefinido), e o instrumento é o *sanitizer*, o aviso do compilador e, sobretudo, o estilo de código que não dá chance ao erro. E pode dar a resposta certa **tarde demais** — e o instrumento é a medição. Este capítulo percorre os três: `src/testing/` (Seção XV.1), `src/undefined_behavior/` (Seção XV.2) e `src/performance/` (Seção XV.3).

O que amarra os três é um método, não um assunto. Eles **medem** em vez de afirmar: a matriz de detecção dos *sanitizers* não foi copiada de documentação, saiu de rodar código com UB de propósito neste *toolchain*; os fatores de 35× e 11,7× saíram de um cronômetro. E, em vários pontos, **o número contrariou a expectativa de quem escreveu o texto**. Esses casos estão todos registrados aqui, porque um resultado que desmente a hipótese ensina método; um que a confirma só ensina o assunto.

Por que — por que corretude e desempenho estão no mesmo capítulo

Porque as três disciplinas se cobrem mutuamente e nenhuma fecha sozinha. O teste encontra a lógica errada e é cego para o UB (comportamento indefinido) que ainda não se manifestou. O *sanitizer* encontra o UB, mas só no caminho que *de fato* executou — por isso depende dos testes para ter o que executar. A cobertura diz qual caminho **não** rodou, que é exatamente o ponto cego do *sanitizer*. E a medição de desempenho só faz sentido sobre um programa já correto: acelerar código errado é produzir a resposta errada mais depressa.

XV.1 Testes automatizados

O subprojeto `src/testing/` é o único do repositório com **dois tipos de alvo**: um executável didático, que narra o assunto, e arquivos `test_*.cpp` que são testes de verdade, rodados por `meson test`. Estado atual: 6 alvos, 41 casos, todos passando.

Por que — que problema o teste automatizado resolve

Não é "garantir que o programa funciona" — nenhuma suíte prova isso. É **tornar barata a mudança**. Sem testes, cada alteração exige reler mentalmente tudo o que ela pode ter quebrado, e o custo dessa releitura cresce com o sistema até o ponto em que ninguém mais mexe no que funciona. Com testes, a pergunta "eu quebrei alguma coisa?" vira um comando de dez segundos. A suíte não é prova de corretude; é **rede de segurança para a refatoração**.

A pergunta útil, portanto, não é "devo testar?". É **"que teste vale a pena escrever?"** — porque uma suíte grande de testes ruins é *pior* que nenhuma: custa manutenção, falha por motivos errados e ensina a equipe a ignorar a luz vermelha.

XV.1.1 A pirâmide, e por que ela é uma pirâmide

Três definições primeiro, porque os termos são usados de forma descuidada. Um **teste unitário** exercita uma unidade — tipicamente uma classe ou função — isolada de tudo o que ela depende: sem banco, sem rede, sem arquivo, sem relógio; roda em microssegundos. Um **teste de integração** exercita duas ou mais

peças juntas, com as dependências reais. Um **teste de ponta a ponta** (E2E) exercita o sistema inteiro pela mesma porta que o usuário usa.



Figura 13: A pirâmide de testes. A forma é consequência do custo por teste em três eixos: **tempo** (microssegundos contra minutos), **estabilidade** (mais peças, mais motivos de falhar sem haver bug) e **diagnóstico** (um unitário aponta a linha; um E2E aponta "algo, em algum lugar").

Armadilha — a casquinha de sorvete

A pirâmide invertida acontece quando a equipe testa só pela interface do usuário, porque é ali que "dá para ver". O sintoma é conhecido: a suíte demora 40 minutos e falha uma vez em cinco sem motivo. A partir daí ninguém confia no vermelho, e a suíte inteira vira decoração de *pipeline*.

XV.1.2 F.I.R.S.T. e o que testar primeiro

Tabela 176: F.I.R.S.T.: o que cada letra exige, e o que a viola.

Letra	Exigência	Inimigo típico
Fast	a suíte inteira roda em segundos	IO, *sleep*, banco real
Isolated	independe dos outros e da **ordem**	estado global, *singleton*
Repeatable	mesmo resultado em qualquer máquina	relógio, rede, sorteio sem semente
Self-validating	passa ou falha, sem ninguém ler a saída	teste que pede conferência humana
Timely	escrito junto com o código	"a gente testa depois"

A lista de inimigos do **R** é maior do que parece: além do relógio e da rede, entram a ordem de iteração de um `std::unordered_map` — que não é especificada e muda entre versões da biblioteca —, caminhos absolutos e fuso horário. E o **T** carrega a lição mais valiosa do subprojeto: **código difícil de testar é código mal desenhado**, e o teste escrito cedo devolve esse recado a tempo de agir.

Em ordem de retorno sobre o esforço, teste: (1) **as fronteiras** — a regra de desconto do sistema sob teste tem faixas em 10, 50 e 100 peças, e os casos que importam são 9/10, 49/50, 99/100, porque é ali que mora o `>` que deveria ser `>=`; (2) **o caminho de erro**, que é o menos exercitado manualmente e por isso o mais quebrado; (3) **o bug que você acabou de consertar** — um teste por bug corrigido é a regra de melhor custo-benefício que existe, porque ele nunca volta; (4) **a regra de negócio**, aquilo que, errado, cobra do cliente o valor errado; (5) o caminho feliz, por último. E não teste *getters* triviais, a biblioteca padrão, detalhes privados de implementação nem código gerado.

Armadilha — os cinco testes que dão prejuízo

(a) O frágil — falha em toda refatoração *correta*; sintoma: cheio de `EXPECT_CALL`. Testa *como*, não *o quê*. **(b) O instável** (*flaky*) — passa às vezes. É o pior de todos, porque destrói a confiança na suíte inteira: depois de dois falsos alarmes, todo mundo reroda em vez de investigar. Deve ser **consertado ou removido**, nunca deixado piscando. **(c) O tautológico** — repete a implementação:

`EXPECT_EQ(c.total(), c.subtotal() * (1 - c.desconto()))` passa mesmo com a fórmula errada; o valor esperado precisa vir **de fora**, como literal. **(d) O sem asserção** — só verifica que não explodiu. **(e) O gigante** — quando falha a asserção nº 4, você não descobre se as outras 26 falhariam.

XV.1.3 O sistema sob teste

Tudo o que vem a seguir gira em torno de uma classe pequena, em `src/testing/carrinho.hpp`, escolhida por ter de propósito os cinco problemas que testes de verdade encontram: uma regra de negócio com faixas, uma pré-condição que aborta, uma exceção, uma dependência externa e aritmética de ponto flutuante.

```
class RepositorioDePrecos {                    // a DEPENDÊNCIA, como interface
    virtual std::optional<double> preco_de(const std::string& sku) const = 0;
};

class Carrinho {
    // A dependência entra pelo CONSTRUTOR. É esta linha que torna a classe
    // testável em microssegundos, sem banco de dados.
    explicit Carrinho(const RepositorioDePrecos& precos) : precos_{precos} {}

    void adicionar(const std::string& sku, int quantidade) {
        // PRÉ-CONDIÇÃO: quantidade <= 0 é BUG DO CHAMADOR, não erro de entrada.
        assert(quantidade > 0 && "quantidade precisa ser positiva");
        const auto preco = precos_.preco_de(sku);
        if (!preco) throw ProdutoDesconhecido{sku};   // ← erro de ENTRADA
    }
    double percentual_de_desconto() const noexcept;  // faixas: 10, 50, 100
};
```

Listagem 456: O sistema sob teste, resumo de `src/testing/carrinho.hpp`.

A distinção entre as duas espécies de coisa ruim — erro de entrada, que vira exceção, e bug do chamador, que vira aborto — é o assunto do Seção II.8, e aqui tem consequência direta sobre *como* se testa cada uma.

XV.1.4 Anatomia de um teste

Um caso tem a forma `TEST(Suite, Caso) { ... }`, e o par `(Suite, Caso)` precisa ser único no binário inteiro. Dentro do corpo, a organização que funciona tem três blocos separados por uma linha em branco: **Arrange** (preparar), **Act** (a ação testada) e **Assert**.

```
TEST(Carrinho, MesmoSkuDuasVezesSomaQuantidade) {
    PrecosFixos precos;                        // ARRANGE
    loja::Carrinho c{precos};

    c.adicionar("caneta", 2);                  // ACT
    c.adicionar("caneta", 5);

    ASSERT_EQ(c.tamanho(), 1u) << "deveria continuar sendo UM item";
    EXPECT_EQ(c.itens()[0].quantidade, 7);  // ASSERT
}
```

Listagem 457: Arrange / Act / Assert, em `src/testing/test_basico.cpp`.

Se o *Act* tem mais de duas ou três linhas, ou se há *dois* blocos act-assert no mesmo caso, o teste está testando duas coisas — separe.

Regra — num teste, a repetição é preferível à abstração

É o único código em que isso vale. `EXPECT_DOUBLE_EQ(total, 77.9)` com o número literal é **melhor** que uma função `calcula_esperado()` — que repetiria a implementação e passaria a esconder exatamente o bug que o teste deveria pegar. O valor esperado precisa entrar *de fora*, calculado por outro meio que não o código sob teste.

O nome do caso é lido por alguém que não o escreveu, no relatório de uma falha. A fórmula que funciona é `<situação><ação><resultado esperado>`: `TEST(Desconto, ExatamenteDezPecasDaCincoPorCento)` contra `TEST(CarrinhoTest, Test1)` ou `FuncionaCorretamente`. Os nomes viram **identificadores C++**: nada de espaço, acento ou hífen — e nada de sublinhado, que o próprio GoogleTest desaconselha, porque ele usa `_` internamente para colar suíte e caso.

XV.1.5 `EXPECT` contra `ASSERT`, e o catálogo

A diferença é de uma palavra e muda o fluxo: `EXPECT_*` registra a falha e **continua**; `ASSERT_*` registra e faz `return`.

Regra — `EXPECT` por padrão, `ASSERT` quando o resto não faz sentido

`EXPECT` dá mais informação por execução. Use `ASSERT` quando continuar seria inútil ou perigoso — o caso clássico é verificar o tamanho antes de indexar: `ASSERT_EQ(c.tamanho(), 1u);` e só então `EXPECT_EQ(c.itens()[0].sku, "caneta");`. Sem o `ASSERT`, a linha seguinte indexaria um vetor vazio, o que é UB (comportamento indefinido).

Armadilha — `ASSERT_*` faz `return`, e isso tem três consequências

(1) Não funciona em função que devolva valor — o `return` é sem argumento. (2) Dentro de uma **função auxiliar** ele retorna da *auxiliar*, e o teste continua como se nada tivesse acontecido; para propagar, envolva a chamada em `ASSERT_NO_FATAL_FAILURE(minha_verificacao())`. (3) Não pode ser usado em construtor nem em destrutor — é para isso que existem `SetUp()` e `TearDown()`.

Tabela 177: As famílias de asserções. Todas têm a variante `ASSERT_`.

Família	Macros	Observação
booleanas	`EXPECT_TRUE` / `FALSE`	imprime só "false" na falha
comparação	`EXPECT_EQ NE LT LE GT GE`	imprimem **os dois** valores
strings C	`EXPECT_STREQ STRNE STRCASEEQ`	para `std::string`, use `EXPECT_EQ`
ponto flut.	`EXPECT_FLOAT_EQ DOUBLE_EQ NEAR`	4 ULPs, ou tolerância explícita
exceções	`EXPECT_THROW NO_THROW ANY_THROW`	o tipo é o segundo argumento
morte	`EXPECT_DEATH DEBUG_DEATH`	roda em processo filho
controle	`FAIL() ADD_FAILURE() SUCCEED() GTEST_SKIP()`	—

A razão de preferir `EXPECT_EQ(a, b)` a `EXPECT_TRUE(a == b)` está na terceira coluna: o primeiro imprime os dois valores quando falha, o segundo imprime "false". A mesma lógica leva aos *matchers* do gmock, usáveis em testes comuns via `EXPECT_THAT` — um `ElementsAre` que falha imprime os dois vetores inteiros e marca o índice divergente:

```
1 EXPECT_THAT(v, ElementsAre(1, 2, 3));      EXPECT_THAT(v, SizeIs(Ge(3)));
2 EXPECT_THAT(v, UnorderedElementsAre(3, 1, 2));  EXPECT_THAT(s, HasSubstr("erro"));
3 EXPECT_THAT(x, AllOf(Gt(0), Lt(100)));     EXPECT_THAT(o, Optional(Eq(42)));
```

Listagem 458: *Matchers*: valem sobretudo pela mensagem de falha.

XV.1.6 Ponto flutuante

Um `double` representa números em base 2, e `0.1` não tem representação exata em base 2 — como 1/3 não tem representação decimal finita. O subprojeto imprime a consequência:

```
0.1 + 0.2 = 0.30000000000000004441
0.3       = 0.29999999999999998890
diferença = 5.551e-17
```

Listagem 459: Saída de `src/testing/`: por que `==` não serve.

A unidade natural de tolerância é o **ULP** (*unit in the last place*): a distância entre dois `double` consecutivos representáveis em torno daquele valor. Ela é minúscula perto de zero e grande perto de 10^{15}, que é exatamente o que se quer — por isso `EXPECT_DOUBLE_EQ`, que aceita 4 ULPs, *escala com a magnitude*, enquanto `EXPECT_NEAR(a, b, 1e-9)` usa tolerância absoluta.

Armadilha — nenhuma comparação de *float* funciona bem em torno de zero

4 ULPs de zero é um número absurdamente pequeno — praticamente uma exigência de igualdade exata. Perto de zero, use `EXPECT_NEAR` com uma tolerância que faça sentido para o *domínio*. E `NaN` não é igual a nada, nem a si mesmo: verifique com `EXPECT_TRUE(std::isnan(x))`.

XV.1.7 Fixtures e o ciclo de vida

Uma **fixture** é uma classe que carrega a preparação comum a vários casos. Deriva de `::testing::Test`, e os casos passam a usar `TEST_F`, com o nome da *classe* como primeiro argumento. A regra que mais surpreende é a que garante o "I" de isolado: o GoogleTest constrói uma instância **nova** da fixture para **cada** caso.

```
SetUpTestSuite()          ← UMA vez, antes de todos os casos
   construtor → SetUp() → caso 1 → TearDown() → destrutor
   construtor → SetUp() → caso 2 → TearDown() → destrutor
TearDownTestSuite()       ← UMA vez, depois de todos
```

Listagem 460: O ciclo de vida, resumo de `src/testing/demo_organizacao.hpp`.

Há, portanto, dois pares de ganchos, e a escolha entre eles não é estilística: use o **construtor** por padrão — ele permite membros `const` e sem construtor padrão; use `SetUp()` quando a preparação puder falhar com `ASSERT_*` ou quando a fixture for herdada; use o **destrutor** por padrão, e `TearDown()` quando a limpeza usar `ASSERT_*` ou puder lançar. O arquivo `src/testing/test_fixture.cpp` tem um caso cuja única função é *provar* o isolamento — ele falharia se a fixture fosse compartilhada:

```
TEST_F(CarrinhoTest, SegundoTesteVeCarrinhoLimpo) {
    EXPECT_TRUE(carrinho.vazio())
        << "se este teste falhar, a fixture está sendo compartilhada";
}
```

Listagem 461: O teste que prova que a fixture é reconstruída.

Armadilha — `SetUpTestSuite` compartilha estado entre casos

Os ganchos de *suíte* rodam uma única vez e são estáticos, portanto o que eles preparam é comum a todos os casos. É a porta de entrada para o teste que só falha numa ordem específica — e que passa quando você o roda sozinho para investigar. Use apenas para recursos **caros e imutáveis**.

XV.1.8 As quatro variantes, e as FRONTEIRAS

Tabela 178: O que cada macro de teste faz variar.

Macro	Varia	Quando usar
`TEST`	nada	caso isolado, sem preparação
`TEST_F`	fixture	preparação repetida
`TEST_P`	**dados**	mesma lógica, muitas entradas
`TYPED_TEST`	**tipos**	*template*, ou N implementações da mesma interface

`TEST_P` é o que mais rende, e a razão não é economia de linhas. Um laço `for` dentro de um `TEST_F` produz **um** teste: ele para na primeira falha e a mensagem diz "algo, em algum caso". Um `INSTANTIATE_TEST_SUITE_P` produz **N** testes independentes, e a falha diz `Faixas/DescontoTest.AplicaAFaixaCorreta/exatamente_50`.

```
1  struct CasoDesconto {          // um struct com NOMES, não um std::tuple:
2      int          pecas;         // `caso.pecas` se lê melhor que get<0>(caso)
3      double       desconto_esperado;
4      const char* rotulo;         // vira parte do NOME do teste
5  };
6  class DescontoTest : public ::testing::TestWithParam<CasoDesconto> { /* ... */ };
7
8  TEST_P(DescontoTest, AplicaAFaixaCorreta) {
9      const auto& caso = GetParam();
10     loja::Carrinho c{precos};
11     c.adicionar("caneta", caso.pecas);
12     EXPECT_DOUBLE_EQ(c.percentual_de_desconto(), caso.desconto_esperado)
13         << "com " << caso.pecas << " peças";
14 }
15
16 // As FRONTEIRAS são o que importa: 9/10, 49/50, 99/100. Um teste que só
17 // verifica 5, 30 e 200 passaria com um `>` no lugar de um `>=`.
18 INSTANTIATE_TEST_SUITE_P(
19     Faixas, DescontoTest,
20     ::testing::Values(CasoDesconto{  9, 0.00, "logo_abaixo_de_10"},
21                       CasoDesconto{ 10, 0.05, "exatamente_10"},
22                       CasoDesconto{ 49, 0.05, "logo_abaixo_de_50"},
23                       CasoDesconto{ 50, 0.10, "exatamente_50"},
24                       CasoDesconto{100, 0.15, "exatamente_100"}),
25     [](const ::testing::TestParamInfo<CasoDesconto>& info) {
26         return std::string{info.param.rotulo};       // o gerador de nomes
27     });
```

Listagem 462: `TEST_P` sobre as fronteiras, em `src/testing/test_parametrizado.cpp`.

Os geradores prontos são `Values(...)`, `Range(0, 10)` (com passo opcional), `Bool()`, `ValuesIn(container)` — a partir de um vetor montado em execução — e `Combine(a, b)`, o **produto cartesiano**, que explode rápido.

Armadilha — sem o gerador de nomes, "o /3 falhou" não diz nada

O último argumento de `INSTANTIATE_TEST_SUITE_P` é opcional e, na prática, quase obrigatório: sem ele os casos se chamam `.../0`, `.../1`, e o relatório do CI vira um enigma. O *lambda* que devolve o rótulo custa três linhas.

Os **testes tipados** resolvem o problema simétrico: a mesma lógica sobre vários *tipos* — é como se testa um *template*, ou N implementações da mesma interface.

```
1  template <typename T> class ContainerTest : public ::testing::Test {
2  protected: T container;
3  };
4  using TiposTestados = ::testing::Types<std::vector<int>, std::deque<int>>;
```

```
TYPED_TEST_SUITE(ContainerTest, TiposTestados);

TYPED_TEST(ContainerTest, ComecaVazioEDepoisNao) {
    EXPECT_TRUE(this->container.empty());     // ← o 'this->' é obrigatório
    this->container.push_back(42);
    EXPECT_EQ(this->container.size(), 1u);
}
```

Listagem 463: `TYPED_TEST`: um corpo, vários tipos.

Armadilha — em `TYPED_TEST`, os membros da fixture exigem `this->`

A fixture é um *template*, e `container` é um **nome dependente** — depende do parâmetro `T`. Pela regra de busca de nomes do C++, o compilador não procura em classes-base dependentes na primeira fase da compilação do *template* (Seção IV.1). Sem o `this->`, ele nem olha em `ContainerTest<T>` e reporta "não declarado neste escopo".

```
-- gtest_filter =*Desconto*:-*Lento*   # incluir e excluir por padrão de nome
--gtest_repeat=200 --gtest_shuffle    # a combinação que caça flakiness
--gtest_break_on_failure              # parar no depurador
--gtest_output=xml:res.xml            # para o CI
meson test -C build testing_mock -v  # um alvo, com a saída na tela
meson test -C build --repeat 50 --gdb --wrapper=valgrind
```

Listagem 464: As opções de linha de comando que mais rendem.

Um teste temporariamente quebrado se marca renomeando-o para `DISABLED_...`: o GoogleTest o pula e **avisa** quantos estão desabilitados — melhor que comentar o teste e esquecer que ele existe.

Armadilha — dois erros de infraestrutura que deixam o CI verde à toa

(1) O Conan expõe quatro pacotes separados: `gtest` (asserções e o *runner*, **sem** `main`), `gtest_main`, `gmock` e `gmock_main`. Linkar só com `gtest` produz `undefined reference to 'main'` — foi o que aconteceu na primeira versão do `meson.build` deste subprojeto. **(2)** `RUN_ALL_TESTS()` precisa ter o retorno *retornado*: ignorá-lo faz o binário sair com código 0 mesmo com testes falhando, e o CI fica verde com a suíte quebrada. O GoogleTest marca a função `[[nodiscard]]` justamente por isso.

XV.1.9 Os cinco dublês e a injeção de dependência

Um **dublê de teste** é qualquer objeto que substitui uma dependência real durante o teste. A taxonomia de Meszaros distingue cinco, e a distinção importa porque cada um responde a uma pergunta diferente.

Tabela 179: Os cinco dublês de teste, e o que cada um verifica.

Dublê	O que faz	Verifica
dummy	só preenche um parâmetro; nunca é usado	nada
stub	devolve respostas prontas	**estado**
spy	um *stub* que anota o que recebeu	os dois
mock	tem expectativas programadas	**interação**
fake	implementação real, simplificada	**estado**

Regra — a escolha certa quase sempre é *fake* ou *stub*

O *mock* existe para uma pergunta específica — "o código chamou X?" — e essa pergunta só importa quando a chamada **é** o efeito que o código produz: enviou o e-mail, gravou o *log*, publicou a mensagem. Se o efeito é observável no estado, verifique o estado.

E tudo isso só é possível por causa de uma única linha de projeto — `explicit Carrinho(const RepositorioDePrecos& precos);`. Compare com o que quase todo mundo escreve primeiro:

```
class Carrinho {
    BancoDeDados db_{"localhost:5432"};   // ✗ só testável com Postgres rodando
};
class Carrinho {                          // ✓ a dependência entra de fora
public:
    explicit Carrinho(const RepositorioDePrecos& precos);
};
```

Listagem 465: A dependência construída por dentro contra a dependência injetada.

Isso é **injeção de dependência**, e não é sobre "usar interfaces em tudo": é sobre inverter *quem decide* a dependência concreta — não a classe, e sim quem a constrói. É o D de SOLID (Seção V.1). Em ordem de preferência: parâmetro de **função** (menor acoplamento possível), de **construtor** (quando vários métodos a usam), de **template** (despacho estático, sem vtable / vptr e sem alocação, Seção IV.1).

Armadilha — os cinco padrões que impedem o teste

(a) ***Singleton*** — amarra o código a uma configuração global compartilhada entre testes, e dois testes com configurações diferentes passam a depender da ordem. **(b) `new` dentro da classe** — se o objeto constrói a própria dependência, não há como substituí-la. **(c) O relógio** — `std::chrono::system_clock::now()` dentro da lógica torna impossível testar a virada do mês; injete o relógio. **(d) Funções estáticas livres que fazem IO** — não existe ponto de extensão. **(e) Métodos privados que você "precisa" testar** — é sintoma: ou o comportamento é alcançável pela interface pública, ou aquilo é outra classe querendo nascer. E `#define private public` antes do `#include` "funciona" e é UB (comportamento indefinido): muda o *layout* do objeto e viola a ODR — a Regra da Definição Única, que exige que todo tipo tenha *a mesma* definição em todas as unidades de tradução.

XV.1.10 gmock: comportamento e interação

Declarar um *mock* custa uma linha por método virtual. Os parênteses em volta dos parâmetros são obrigatórios, e um tipo com vírgula — `std::map<int,int>` — precisa dos seus próprios parênteses ou de um *alias*, senão o pré-processador quebra a lista de argumentos na vírgula errada.

```
class MockPrecos : public loja::RepositorioDePrecos {
public:
    // MOCK_METHOD(retorno, nome, (parâmetros), (qualificadores));
    MOCK_METHOD(std::optional<double>, preco_de, (const std::string& sku),
                (const, override));
};

TEST(MockPrecos, ConsultaUmaVezPorAdicionar) {
    MockPrecos precos;
    // Esta é a asserção do teste, e ela é verificada no DESTRUTOR do mock:
    // não há EXPECT_ nenhum no corpo, e ainda assim o teste pode falhar.
    EXPECT_CALL(precos, preco_de(Eq("caneta")))
        .Times(2)
        .WillRepeatedly(Return(std::optional<double>{2.50}));

    loja::Carrinho c{precos};
    c.adicionar("caneta", 1);
    c.adicionar("caneta", 1);
}
```

Listagem 466: `MOCK_METHOD` e `EXPECT_CALL`, em `src/testing/test_mock.cpp`.

São duas coisas distintas: `ON_CALL(...).WillByDefault(...)` **define comportamento** e não é asserção; `EXPECT_CALL(...)` **é a asserção**.

Armadilha — a ordem das regras de `ON_CALL` é a inversa da intuição

Descoberta na primeira execução de `test_mock.cpp`:

```
1 ON_CALL(precos, preco_de(Eq("caneta"))).WillByDefault(Return(2.50));
2 ON_CALL(precos, preco_de(_)).WillByDefault(Return(std::nullopt));   // !
```

Isto falha com `C++ exception with description "produto desconhecido: caneta"`. O gmock tenta as regras da **mais recente para a mais antiga** — e o `_`, declarado por último, captura também a caneta. **A regra genérica precisa vir primeiro**, para que as específicas, declaradas depois, ganhem.

Um *mock* tem ainda três níveis de rigor diante de uma chamada que nenhuma expectativa previu: `NaggyMock` (o padrão) imprime aviso no *log* e o teste *passa*; `NiceMock` ignora em silêncio; `StrictMock` **falha** o teste. Use `NiceMock` por padrão — o *naggy* enche o *log* de ruído e o *strict* quebra o teste toda vez que a implementação ganha uma chamada legítima a mais.

Regra — mais de dois ou três `EXPECT_CALL` é sinal de teste frágil

Um teste assim verifica a *implementação*, não o comportamento; ele passará a falhar em toda refatoração correta. **Mocke** o que é lento, não determinístico, irreversível, difícil de levar a um estado de erro, ou o que ainda não existe. **Não mocke** tipos de valor, a biblioteca padrão, código seu que é rápido e determinístico, nem tipos que você não possui — para esses, mocke a *sua* camada de adaptação em volta deles.

XV.1.11 *Death tests*: verificar que o programa morre

A pré-condição `assert(quantidade > 0)` aborta o processo quando violada. Como testar isso, se abortar mataria o processo de teste? O GoogleTest resolve com `fork()`: a expressão roda num processo **filho**, e o pai verifica o código de saída e a saída em *stderr*.

```
1 // Uma suíte terminada em 'DeathTest' é executada ANTES de todas as outras,
2 // para reduzir a chance de o fork acontecer com threads já criadas.
3 TEST_F(CarrinhoDeathTest, QuantidadeZeroAborta) {
4     loja::Carrinho c{precos};
5     // O segundo argumento casa contra a stderr do filho: é o que distingue
6     // "morreu pelo motivo certo" de "morreu por qualquer motivo".
7     EXPECT_DEATH(c.adicionar("caneta", 0), "quantidade precisa ser positiva");
8 }
```

Listagem 467: `EXPECT_DEATH`, em `src/testing/test_death.cpp`.

Armadilha — os quatro perigos dos *death tests*

(1) Threads. `fork()` num processo com *threads* clona apenas a *thread* que chamou; se outra segurava um *mutex*, o filho trava para sempre — daí a convenção do sufixo `DeathTest`. **(2) Efeitos colaterais.** O que a instrução faz acontece no *filho* e é descartado: depois de `EXPECT_DEATH({ x = 1; morre(); }, "")`, o `x` do pai continua zero. **(3) O regex.** É POSIX estendido no Linux e um subconjunto simples no Windows; um regex vazio casa qualquer coisa — e aí o teste só verifica que morreu, não por quê. **(4) Velocidade.** Cada caso é um *fork* mais uma execução.

Como `assert` vira um *no-op* sob `NDEBUG`, os *death tests* só existem em compilação de depuração; em *release* eles se auto-pulam com `GTEST_SKIP()` — testar um `assert` desativado seria testar o compilador. A macro

`EXPECT_DEBUG_DEATH` faz isso sozinha, dispensando o `#ifndef` escrito à mão. E uma exceção **não** é morte: é comportamento normal e previsto, para o qual existe `EXPECT_THROW`.

XV.1.12 Cobertura, e o que 100% não garante

Cobertura é a fração do código *executada* durante a suíte. Vem em quatro sabores, e a diferença entre eles é a diferença entre uma métrica confortável e uma métrica útil.

Tabela 180: Os quatro tipos de cobertura, para `if (a && b) faz_algo();`.

Tipo	O que mede	Testes necessários
linha	a linha executou?	1
ramo	os dois lados do `if`?	2
condição	cada subcondição foi `true` e `false`?	`a` e `b` variando
caminho	todas as combinações de todos os ramos	exponencial

O curto-circuito do `&&` é o que separa *linha* de *condição*: com `a` falso, `b` nunca é avaliado — a linha conta como coberta e metade da lógica nunca rodou. A maioria das ferramentas reporta *linha* por padrão, a métrica mais fácil e menos informativa; peça ramo com `gcov -b` ou `lcov -rc branch_coverage=1`.

```
g++ --coverage -O0 -g fonte.cpp teste.cpp -lgtest_main -lgtest
./a.out && gcov -b fonte.cpp        # relatório por arquivo, com ramos
lcov --capture -d . -o cov.info && genhtml cov.info -o html/

meson setup build -Db_coverage=true && meson test -C build
ninja -C build coverage-html
```

Listagem 468: Medindo cobertura com GCC e com o Meson.

Compile com `-O0`: com otimização, *inlining* e eliminação de código morto embaralham a numeração e o relatório passa a mentir nas duas direções.

Armadilha — o que 100% de cobertura NÃO garante

(a) Que as asserções existem. `TEST(X, Cobre) { minha_funcao(1, 2); }` dá 100% da função e não verifica **nada**. **(b) Que os valores certos foram testados.** Cobrir `if (n >= 10)` com `n=5` e `n=50` é 100% de ramo — e não descobre que o certo era `> 10`. As **fronteiras** encontram esse bug, e cobertura não pede por elas. **(c) Que o código que falta existe.** Um `else` esquecido não aparece em relatório nenhum: cobertura mede o que está escrito, não o que deveria estar. **(d) Ausência de UB (comportamento indefinido).** Um `int` que estoura executa a linha e conta como coberta.

Regra — cobertura é detector de buracos, não meta numérica

Lida como relatório — "este `catch` nunca rodou", "esta função inteira está em zero" — é uma das ferramentas mais úteis que existem. Como *gate* numérico do CI, produz exatamente o tipo de teste que o item (a) descreve. A métrica melhor é o **teste de mutação**: a ferramenta altera o código (troca `>=` por `>`, inverte um `if`) e verifica se *algum* teste falha — um mutante que sobrevive é um buraco real. Em C++: `mull`, `Dextool`.

Além dos testes de exemplo, que verificam *um* caso que você imaginou, há duas técnicas para os casos que você não imaginou. O **teste baseado em propriedades** troca "`f(2,3) == 5`" por uma propriedade que vale *sempre*, e deixa a ferramenta gerar centenas de entradas: idempotência (`ordena(ordena(v)) == ordena(v)`), ida e volta (`desserializa(serializa(x)) == x`), invariante ou oráculo. No carrinho: *"o total*

nunca é maior que o subtotal". Biblioteca: RapidCheck. O **fuzzing** alimenta a função com entradas geradas e guiadas por cobertura, procurando *crash* e UB — vale para tudo que recebe entrada **não confiável**: analisadores sintáticos, decodificadores, protocolos.

```
extern "C" int LLVMFuzzerTestOneInput(const uint8_t* d, size_t n) {
    meu_parser(std::string_view{reinterpret_cast<const char*>(d), n});
    return 0;
}   // clang++ -fsanitize=fuzzer,address fuzz.cpp
```

Listagem 469: O ponto de entrada do libFuzzer.

E a combinação de melhor retorno em C++ é rodar a suíte que já existe sob *sanitizer*: os testes já exercitam o código, e sob instrumentação cada um vira um detector de *use-after-free*, *overflow*, vazamento e UB — **sem escrever um teste a mais**. É a ponte para a próxima seção.

XV.2 Comportamento indefinido

Por que — que problema esta seção resolve

Uma crença: a de que um programa com UB (comportamento indefinido) "dá um valor errado". Não dá. Ele autoriza o compilador a **remover código que você escreveu** — inclusive a verificação que você pôs justamente para se proteger. A diferença entre as duas visões é a diferença entre um bug e uma classe inteira de vulnerabilidades.

A definição do padrão é curta e implacável: *"behavior for which this document imposes no requirements"*. Nenhuma exigência. E — o ponto que quase todo mundo perde — o efeito **não precisa se limitar ao trecho errado**: pode aparecer *antes* da linha do erro, porque o compilador reorganizou o código supondo que o UB não aconteceria. O exemplo canônico não é acadêmico:

```
struct sock* sk = tun->sk;      // ← desreferencia tun
if (!tun) return POLLERR;       // ← checa DEPOIS
```

Listagem 470: O `tun_chr_poll` do kernel Linux — CVE-2009-1897.

O GCC raciocinou: `tun->sk` só é legal se `tun != nullptr`, logo `!tun` é sempre falso, logo o `if` inteiro pode sumir. E sumiu. Virou escalação de privilégio — um atacante conseguia mapear a página zero.

XV.2.1 As quatro categorias, e por que UB existe

O padrão distingue quatro graus de compromisso, e confundi-los é a origem de metade das discussões improdutivas sobre portabilidade.

Tabela 181: As quatro categorias. As três primeiras são seguras; a quarta não tem limite.

Categoria	O compilador...	Exemplo
bem definido	faz o que o padrão manda	`2 + 2`
implementation-defined	escolhe **e documenta**	`sizeof(int)`
unspecified	escolhe, sem documentar	ordem de avaliação
UNDEFINED	**não tem obrigação nenhuma**	*overflow* de `int`

Neste *toolchain*, `sizeof(long)` é 8 — e é **4 no Windows**; `char` é `signed` — e é `unsigned` em ARM por padrão. Nos dois casos, o código que supõe o contrário compila e roda até ser portado. A ordem de avaliação

de argumentos é *unspecified*: numa chamada `print("{} {} {}", f(), f(), f())` o GCC imprime `3 2 1`. É legal, e é armadilha.

Por que — a troca que o comitê fez de propósito

(a) Desempenho. Se `i + 1 > i` pudesse ser falso, o compilador não poderia simplificar `for (int i = 0; i <= n; ++i)` nem promover `i` a um registrador de 64 bits. Como *overflow* com sinal é UB, ele *pode* supor que não acontece. **(b) Portabilidade.** O padrão precisa valer para máquinas com complemento de um, palavras de 9 bits e sem unidade de gerenciamento de memória. **(c) Custo zero para quem não usa.** Definir "acesso fora do vetor lança exceção" exigiria verificação em **todo** acesso — e o C++ não cobra por segurança que você não pediu.

O C++20 converteu um caso famoso: inteiros com sinal passaram a ser obrigatoriamente complemento de dois (P0907). O *overflow* deles, no entanto, **continua sendo UB** — a representação foi fixada, o *overflow* não.

Tabela 182: O catálogo de UB, agrupado por quem tem chance de pegá-lo.

Grupo	Casos
memória (ASan)	fora dos limites; *use-after-free*; *use-after-scope*; *double free*; `delete[]` em ponteiro de `new`; `nullptr`; referência a obj
aritmética (UBSan)	*overflow* **com sinal** (sem sinal é definido!); divisão por zero; `INT_MIN / -1`; *shift* $\geq$ largura do tipo; `double` $\rightarrow$ `int`
tipos	*strict aliasing*; ler membro inativo de `union`; desalinhamento; usar objeto antes do construtor ou depois do destruto
controle	função não-`void` sem `return`; *data race*; laço infinito sem efeito observável; exceção escapando de `noexcept`
biblioteca	`v[i]` com `i >= size()`; desreferenciar `end()` ou iterador inválido; `.front()` em container vazio; `memcpy` com `nullpt`

Dois itens surpreendem sempre: *overflow* de inteiro **sem** sinal é **bem definido** — envolve módulo 2^N, por exigência do padrão; é só o com sinal que é UB. E ler um membro inativo de uma `union` é o idioma padrão de *type punning* em C desde o C99, e é **UB em C++**.

XV.2.2 Como o UB licencia o otimizador

A cadeia de raciocínio do compilador é impecável e sempre a mesma: *"este programa não tem UB — o padrão diz que programas corretos não têm. Logo, se `p` foi desreferenciado, `p != nullptr`. Logo, a verificação `if (p)` três linhas abaixo é sempre verdadeira. Logo, posso apagá-la."*

Armadilha — a verificação de *overflow* que não verifica

```
1  if (a + b < a) trata_overflow();      // ✗ some com -O2
```

Se `a + b` estoura, o programa já tem UB antes da comparação; logo o compilador pode supor que não estoura; logo `a + b < a` é sempre falso; logo a chamada é código morto. As formas corretas verificam **antes** de estourar:

```
1  if (__builtin_add_overflow(a, b, &r)) trata_overflow();  // usa o flag da CPU
2  if (b > 0 && a > INT_MAX - b)         trata_overflow();  // sem extensão
```

O arquivo `src/undefined_behavior/ub_exemplos.cpp` — o único do repositório que contém UB de propósito, não instalado e com cada caso exigindo um argumento explícito — tem doze casos. Um deles produziu o resultado que mais ensina de todo o subprojeto:

```
1  int caso_uaf(int n) {
2      int* p = new int[4];
3      p[0] = n;
4      delete[] p;
```

```
    return p[0];              // ← use-after-free
}
```

Listagem 471: O caso `uaf`: leitura de memória já liberada.

```
-O0 → ERROR: AddressSanitizer: heap-use-after-free
-O1 → imprime 42, sem diagnóstico nenhum
```

Listagem 472: Medido em `src/undefined_behavior/` com `-fsanitize=address`: o MESMO código, dois níveis.

Com `-O1`, o compilador viu `p[0] = n; ... return p[0]` e propagou a constante: `return n`. **A leitura desapareceu antes de o *sanitizer* poder instrumentá-la.** Não há acesso, logo não há o que reportar.

Regra — duas conclusões que valem para qualquer projeto

(1) Rode a suíte sob *sanitizer* em `-O0` **e** em `-O1`/`-O2`: cada nível esconde e revela coisas diferentes. (2) **"O ASan não acusou" não é prova de que não há UB.** A recomendação oficial do ASan é `-O1`, pelo equilíbrio entre velocidade e fidelidade — e este exemplo mostra que ela também tem custo.

O padrão não diz que a execução fica indefinida *a partir* do UB: diz que a execução **inteira** fica indefinida. É o chamado *time travel*:

```
void f(int* p, bool cond) {
    printf("antes\n");        // ← pode NÃO ser impresso
    if (cond) *p = 1;         // ← o UB está aqui
}
```

Listagem 473: O efeito que aparece antes da causa.

Se o compilador provar que `cond` é sempre verdadeiro e `p` é sempre nulo, o corpo inteiro vira "inalcançável" e ele pode remover tudo — inclusive o `printf` anterior. É o que torna a depuração de UB tão desagradável: **a última mensagem que apareceu não delimita onde o problema está**. O corolário é que `std::unreachable()` (C++23) e `[[assume(x)]]` são **UB deliberado**: você promete que aquilo não acontece, e o compilador otimiza em cima da promessa.

Armadilha — três casos famosos, e o aviso que pega cada um

`memset` **para apagar senha** — não é UB, é eliminação de escrita morta perfeitamente legítima: ninguém lê o *buffer* depois, logo a escrita é removida, logo a senha fica na memória. Use `explicit_bzero` ou um ponteiro `volatile`. `if (this == nullptr)` — chamar um método já pressupõe `this != nullptr`, então a verificação some (`-Wnonnull-compare`). **Função sem** `return` — cair fora de uma função não-`void` é UB, e na prática o compilador emite código que "continua" para a função seguinte no binário. Ligue `-Werror=return-type`.

XV.2.3 Os *sanitizers*, medidos

Um ***sanitizer*** é instrumentação inserida em tempo de compilação: o compilador põe verificações em torno de cada acesso, e uma biblioteca de tempo de execução mantém o mapa do que é memória válida. Custa desempenho e memória, e por isso não vai para produção — vai para os **testes**.

O ASan traz junto o LeakSanitizer, ligado por padrão no Linux: ele reporta, ao fim do programa, o que não foi liberado.

Tabela 183: Os quatro *sanitizers* e o custo de cada um.

Nome	Flag	Pega	Custo
Address	`-fsanitize=address`	memória (UAF, *overflow*, vazamento)	≈2×
Undefined	`-fsanitize=undefined`	UB aritmético e de conversão	≈1,2×
Thread	`-fsanitize=thread`	*data race*	5–15×
Memory	`-fsanitize=memory`	leitura de não inicializado	≈3×

Armadilha — ASan e TSan são incompatíveis; MSan é quase inviável

Os dois primeiros instrumentam os mesmos acessos e não podem ser ligados juntos — exigem **dois jobs separados no CI**. Já o MSan exige que **toda** a pilha esteja instrumentada, inclusive a libstdc++: sem uma libc++ compilada com MSan, ele produz falsos positivos ou — como medido aqui — **simplesmente não reporta**. É o menos usável dos quatro, e o único que o GCC não implementa. A combinação padrão, barata e compatível, é `address,undefined`.

Nenhuma linha da Tabela 197 foi copiada de documentação: cada uma saiu de rodar `ub_exemplos.cpp` com `-fsanitize=address,undefined` neste *toolchain* (GCC 13.3 e clang 18.1.3).

Tabela 184: Matriz de detecção medida. Três resultados contrariam o senso comum.

Caso	`-O0`	`-O1`
heap	ASan: *buffer-overflow*	ASan: *buffer-overflow*
pilha	UBSan: *index out of bounds*	idem
uaf	**ASan: *heap-use-after-free***	**NADA**
dangling	UBSan: *null reference binding*	idem
overflow	UBSan: *signed integer overflow*	idem
shift	UBSan: *shift exponent too large*	idem
divisao	UBSan: *division by zero*	idem
nulo	UBSan: *load of null pointer*	idem
vazamento	LSan: *detected memory leaks*	idem
aliasing	**NADA**	**NADA**
naoinit	**NADA**	**NADA**
alinhamento	**NADA (GCC)**	**NADA (GCC)**

Os três destacados: **(1)** `uaf` some com `-O1`, pelo motivo já explicado; **(2)** `alinhamento` não é pego pelo GCC, nem com `-fsanitize=alignment` isolado — o clang pega (`runtime error: load of misaligned address ... which requires 4 byte alignment`), de modo que, se alinhamento importa no seu código, esta verificação exige clang; **(3)** `aliasing` não é pego por nenhum dos dois.

Armadilha — o que nenhum *sanitizer* pega

(a) *Strict aliasing* — não há instrumentação para isso; a violação simplesmente produz o valor que o otimizador decidiu. **(b) Leitura de não inicializado**, na prática — o MSan existe e é inviável sem toda a stdlib instrumentada; a defesa real é `-Wmaybe-uninitialized` e a regra de sempre inicializar. **(c) Violação de ODR entre unidades de tradução** — só `-flto` com `-Wodr` tem chance. **(d) A lógica errada** — nenhum *sanitizer* sabe que a sua fórmula de desconto está trocada; é para isso que existem os testes. **(e) O caminho que não foi executado** — *sanitizer* é análise **dinâmica**: só vê o que rodou. Um `catch` que os testes nunca exercitam pode ter UB e passar limpo. É por isso que *sanitizer* e cobertura formam um par: um mede o que rodou, o outro diz o que não rodou.

```
g++ -std=c++20 -O1 -g -fsanitize=address,undefined \
    -fno-omit-frame-pointer -fno-sanitize-recover=all arq.cpp -o prog

meson setup build-asan -Db_sanitize=address,undefined -Db_lundef=false
```

```
meson test -C build-asan

ASAN_OPTIONS=detect_leaks=1:abort_on_error=1
ASAN_OPTIONS=detect_stack_use_after_return=1
UBSAN_OPTIONS=print_stacktrace=1:halt_on_error=1
```

Listagem 474: Como rodar — e a flag que decide se o CI serve para alguma coisa.

Regra — `-fno-sanitize-recover=all` é essencial no CI

Sem ela, o UBSan apenas **imprime** o diagnóstico e continua a execução — e o código de saída do processo fica 0. A suíte passa *com o UB detectado*, e o relatório verde esconde a mensagem no meio do *log*.

XV.2.4 *Strict aliasing* e `bit_cast`

A regra, em uma frase: acessar o valor de um objeto por um *lvalue* de tipo **incompatível** é UB. As exceções são poucas e vale decorá-las: o tipo exato (com ou sem `const`/`volatile`); a versão com ou sem sinal do mesmo tipo; uma classe **base**; e `char`, `unsigned char` e `std::byte` — os três únicos universais. `signed char` **não** está na lista, e é uma assimetria fácil de esquecer.

Por que — por que a regra existe

Para o otimizador. Considere `void f(int* pi, float* pf)` que escreve em `*pi`, escreve em `*pf` e depois lê `*pi`. Com *strict aliasing*, o compilador **sabe** que `pf` não pode apontar para o mesmo objeto que `pi` e pode reusar o valor que já está no registrador. Sem a regra, toda escrita por ponteiro invalidaria todos os valores em registrador, e código com ponteiros ficaria sistematicamente mais lento.

```
int i = *reinterpret_cast<const int*>(&f);      // ✗ UB — e NENHUM sanitizer avisa
std::memcpy(&i, &f, sizeof f);                  // ✓ legal desde sempre
auto i = std::bit_cast<std::uint32_t>(f);       // ✓ C++20

static_assert(std::bit_cast<std::uint32_t>(1.0f) == 0x3F800000u);  // constexpr!
```

Listagem 475: *Type punning*: o errado e os dois certos.

Medido: os dois caminhos legais dão `0x4048F5C3` para `3.14f`. E `memcpy` **não é lento**: com `-O2` o compilador reconhece o padrão e emite um único `mov`. `std::bit_cast` ainda é melhor em quatro pontos: exige `sizeof` igual (erro de **compilação** se não for); exige os dois tipos *trivially copyable*; não menciona ponteiro, então não há como errar o `sizeof`; e é `constexpr`.

Armadilha — `union` para *type punning*: legal em C, UB em C++

Ler o membro **inativo** de uma `union` é o idioma padrão em C desde o C99. Em C++ é UB — só um membro *existe* por vez, e ler outro viola a regra de tempo de vida. GCC e Clang aceitam como extensão; o MSVC não dá a mesma garantia. **Use `bit_cast`.**

Quando a violação já está no código antigo e auditar é inviável, existe a saída de emergência `-fno-strict-aliasing`, que faz o compilador supor que qualquer ponteiro pode apontar para qualquer coisa — o kernel Linux compila com ela desde 2003, não porque a regra seja ruim, mas porque auditar milhões de linhas de C antigo não é possível. A ordem de preferência é: `bit_cast`/`memcpy`; depois `char`/`std::byte` quando você precisa mesmo ver bytes; depois `__attribute__((may_alias))` cirúrgico; e só então a flag no projeto inteiro.

Armadilha — a flag precisa valer para TODAS as unidades de tradução

Uma **unidade de tradução** é um arquivo `.cpp` depois do pré-processador — a menor coisa que o compilador compila de uma vez. Se uma delas for compilada *sem* `-fno-strict-aliasing`, ela otimizará em cima de uma suposição que outra viola: nenhum erro de *link*, nenhum aviso, corrupção em execução.

XV.2.5 As cinco camadas de defesa

Da mais barata para a mais cara — e a mais barata é também a mais eficaz, o que costuma surpreender. **Camada 1: escrever código que não tem como ter UB** (custo zero), pela substituição sistemática da Tabela 185.

Tabela 185: Substituições que eliminam categorias inteiras de UB.

Em vez de...	Use...
`new` / `delete`	`std::unique_ptr`, `std::vector`
`T*` mais tamanho	`std::span`
array de C	`std::array`
`v[i]` em código duvidoso	`v.at(i)`
`reinterpret_cast`	`std::bit_cast`
`int` para tamanhos	`std::size_t`
`strcpy` / `sprintf`	`std::string`, `std::format`
`T*` que pode ser nulo	`std::optional`, `T&`
`union` para *punning*	`std::variant` / `bit_cast`
laço com índice	`for (auto& x : c)`

O exemplo que resume a tabela é o `span`: `soma_c(v.data(), 100)` compila sem reclamar; `soma_span(v)` não tem *onde* errar o tamanho. Sobre `at()` contra `[]`: o custo é uma comparação por acesso, irrelevante fora do laço mais quente.

Camada 2: os avisos do compilador (custo zero). Além de `-Wall -Wextra -Wpedantic`, que não são negociáveis: `-Wshadow`, `-Wconversion`, `-Wsign-conversion`, `-Wold-style-cast`, `-Wnull-dereference` (o caso do CVE do kernel), `-Wnon-virtual-dtor`, `-Wduplicated-cond`, `-Wlogical-op`. E três que devem ser **erro**: `-Werror=return-type`, `-Werror=uninitialized`, `-Werror=implicit-fallthrough`.

Camada 3: análise estática — `g++ -fanalyzer`, `clang-tidy -checks='bugprone-*,cert-*'`, `cppcheck -enable=all`. O valor específico dela é encontrar o caminho que os testes **não** executaram, exatamente o ponto cego do *sanitizer*; falso positivo é a norma, então adote incrementalmente, ligando em código novo. **Camada 4: *sanitizers* nos testes** — a Seção XV.2.3. **Camada 5: *hardening* em produção** — `-D_GLIBCXX_ASSERTIONS` (verifica `v[i]`, `.front()`), `-D_FORTIFY_SOURCE=3`, `-fstack-protector-strong`, `-fcf-protection`. O primeiro transforma `v[i]` fora de faixa de UB em aborto com mensagem, ao custo de uma comparação; Red Hat e Google já o ligam por padrão.

Armadilha — `_GLIBCXX_ASSERTIONS` muda a ABI

Ele altera o *layout* de alguns tipos da biblioteca padrão. Precisa valer para **todas** as unidades de tradução e **todas** as bibliotecas linkadas: misturar produz corrupção silenciosa, não erro de *link*. Vale a mesma disciplina de `-fno-strict-aliasing`.

Aviso

A parte honesta: mesmo com as cinco camadas, C++ não é uma linguagem segura por construção. Elas reduzem a probabilidade; não a zeram. Isso não é argumento para abandonar C++ — é argumento para levar as camadas a sério, e para reconhecer que **"eu tomo cuidado" nunca foi uma delas**.

XV.3 Desempenho: medir, e não supor

Por que — que problema o subprojeto `performance` resolve

A intuição de desempenho que a maioria de nós tem foi formada contando *operações* — e a máquina moderna não cobra por operação, cobra por **acesso à memória que não estava no cache**. A consequência é que programadores experientes erram por uma ordem de grandeza ao prever qual de duas versões é mais rápida. A única saída é medir — e medir sem se enganar, que é mais difícil do que parece.

Este é o subprojeto que mais mede: nada nele é afirmado sem um número obtido na própria execução, e **três das medições contrariaram o texto escrito antes de medir**. Nos três casos o texto estava errado, e a explicação era mais interessante que a expectativa.

XV.3.1 A hierarquia de memória e a linha de cache

Um **cache** é uma memória pequena e rápida, junto ao processador, que guarda cópias do que foi lido recentemente da RAM. Há três níveis (L1, L2, L3), cada um maior e mais lento que o anterior.

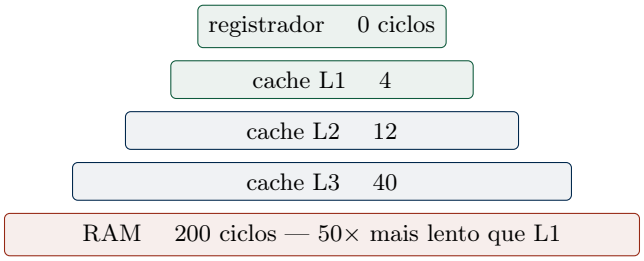


Figura 14: A maior assimetria do hardware moderno. Em 200 ciclos o processador executaria centenas de instruções — por isso, em código que percorre dados, "quantas operações" quase nunca é a pergunta certa, e "quantos *cache miss*" é.

O processador nunca lê 4 bytes da RAM. Ele lê uma **linha de cache** — um bloco alinhado de 64 bytes no x86-64. Ler `v[0]` traz `v[0..15]` junto, de graça: se você usar os quinze vizinhos, eles saíram grátis; se não usar, você pagou por eles do mesmo jeito. O experimento soma 1 milhão de elementos de um vetor de 64 MB variando o *passo* entre acessos — mesmo número de somas em todos os casos:

```
passo | bytes entre acessos | tempo
------|---------------------|-----------------
    1 |                   4 | 0.38 ms  (1.0×)
    4 |                  16 | 0.36 ms  (0.9×)
    8 |                  32 | 0.61 ms  (1.6×)
   16 |                  64 | 1.39 ms  (3.7×)   ← o joelho
   32 |                 128 | 2.48 ms  (6.5×)
```

Listagem 476: Saída de `src/performance/`: o joelho está exatamente em 64/`sizeof(int)`.

Até o passo 16, vários acessos compartilham a mesma linha e o custo mal se move; a partir dali, cada acesso paga uma linha inteira. O passo 32 é ainda pior por causa do **prefetcher** — o circuito que detecta padrões de acesso e busca as linhas *antes* de serem pedidas: ele traz linhas adjacentes, e com 128 bytes de passo metade do que ele traz nunca é usada.

Armadilha — nota de método: um benchmark que produz um número não produziu um número certo

A primeira versão deste experimento ia até o passo 1024 e afirmava que o tempo "estabiliza" depois de 16. A medição desmentiu as duas coisas:

```
passo  256 → 1.75 ms   ← CAIU
passo  512 → 1.56 ms   ← caiu mais
passo 1024 → 4.05 ms   ← disparou
```

A causa é um defeito do **experimento**: o `if (i >= kN) i = 0` faz o índice dar a volta. Com passo 256, só 65 536 posições distintas são tocadas — 4 MB, que **cabem no L3**. O benchmark deixou de medir latência de RAM e passou a medir L3. (O salto em 1024 é outro efeito: 4096 bytes é o tamanho da página de memória, e ali começa a pressão sobre o TLB, a cache de tradução de endereços.) A tabela para no passo 32 porque é a faixa em que o experimento mede o que diz medir.

XV.3.2 Linha contra coluna, `vector` contra `list`, e o prefetcher

Uma matriz 2048×2048 de `int` ocupa 16 MB e é armazenada por linhas. Somar *todos* os elementos nas duas ordens possíveis:

```
por linha   (i, depois j) :  0.44 ms
por coluna (j, depois i) : 15.54 ms   ← 35× mais lento
```

Listagem 477: Saída de `src/performance/`: mesmo $O(n^2)$, mesmo número de somas, 35 vezes de diferença.

A única diferença é a **ordem** dos acessos: por coluna, cada acesso salta 8192 bytes — uma linha de cache nova por elemento, e 15 dos 16 `int` trazidos são descartados antes de serem usados. É por isso que BLAS e Eigen fazem *tiling*: quebram a matriz em blocos que cabem em L1.

```
std::vector<int> : 0.43 ms      4 bytes/elemento, contíguo
std::list <int>  : 2.80 ms     20 bytes/elemento, uma alocação por nó
```

Listagem 478: Saída de `src/performance/`: somando 2 milhões de `int`.

A `std::list` paga por elemento: 4 bytes de dado mais dois ponteiros mais o custo do alocador, cada nó numa alocação separada e potencialmente distante das outras. O detalhe que fecha o argumento é que ela tem inserção $O(1)$ no meio — e ainda assim o `vector` costuma ganhar *até nesse caso*: o `memmove` de N elementos contíguos é mais rápido que *percorrer* a lista até a posição de inserção. É a palestra de Bjarne Stroustrup em Going Native 2012, e continua valendo (Seção III.1).

```
crescente    :  0.97 ms  (1.0×)
decrescente  :  1.04 ms  (1.1×)   ← o prefetcher também prevê para trás
aleatória    : 25.69 ms  (26.4×)
```

Listagem 479: Saída de `src/performance/`: os mesmos 8 milhões de elementos, em três ordens.

O prefetcher detecta passo constante em qualquer direção. Acesso aleatório não tem padrão: cada *miss* é pago integralmente. É por isso que uma tabela *hash* com sondagem linear ganha de uma com encadeamento — não por fazer menos operações, mas por o prefetcher conseguir ajudar.

Aviso

O caso aleatório percorre um *segundo* vetor, o de índices, então parte da diferença é esse tráfego a mais. O subprojeto registra a ressalva no próprio código, e ela é o tipo de honestidade que separa uma medição de uma propaganda — ainda que o fator seja grande demais para ser só isso.

XV.3.3 AoS contra SoA

Array of Structs (AoS) é como quase todo mundo modela, e é a forma mais legível: um `vector` de estruturas. *Struct of Arrays* (SoA) inverte: uma estrutura de vetores, um por campo.

```
1 // AoS — legível, e o padrão
2 struct Particula { float x, y, z, vx, vy, vz, massa, carga; int id; bool ativa; };
3 std::vector<Particula> mundo;
4
5 // SoA — o layout que a máquina prefere para alguns laços
6 struct Mundo { std::vector<float> x, y, z, vx, vy, vz, massa, carga; };
```

Listagem 480: Os dois layouts, de `src/performance/demo_aos_soa.hpp`.

```
1 AoS : 3.72 ms      trouxe 152 MB da RAM (a struct inteira, 40 B por partícula)
2 SoA : 2.47 ms      trouxe  15 MB        (só o vetor de massa)
```

Listagem 481: Saída de `src/performance/`: somando APENAS `massa` de 4 milhões de partículas.

Aqui está a primeira lição de método: a razão de **bytes** é 10×, e a de **tempo** é só 1,5×. A diferença entre esses dois números é o prefetcher — o acesso do AoS continua sequencial, e o hardware busca as linhas antes de serem pedidas. Ele não elimina o tráfego; elimina a **espera** por ele, até saturar a banda de memória.

Regra — contar bytes prevê o teto, não o tempo

O cálculo "quantos bytes este laço precisa trazer" dá o limite inferior imposto pela banda de memória. Quanto do teto se realiza depende do prefetcher, da vetorização e do número de fluxos simultâneos. Use a conta de bytes para saber *se vale a pena tentar*; use o cronômetro para saber *o que aconteceu*.

O demo foi escrito supondo que, num laço que usa *quase todos* os campos, o AoS venceria — afinal a *struct* inteira é útil e nada é desperdiçado. Não venceu:

```
1 AoS : 8.62 ms
2 SoA : 5.39 ms      ← o SoA ainda ganhou (1.60×)
```

Listagem 482: Saída de `src/performance/`: o laço que usa todos os campos — o resultado que contrariou a hipótese.

Duas razões prováveis nesta máquina. Primeira: `sizeof(Particula)` é **40**, e $64/40 = 1$ — cada partícula já ocupa mais de uma linha, então o AoS não colhe o benefício de "tudo na mesma linha". Segunda: o laço SoA **vetoriza** (o compilador emite uma instrução SIMD que soma oito `float` de uma vez) e o AoS, com passo de 40 bytes, não — carregar oito massas espalhadas exigiria um *gather*, que é lento e nem sempre é escolhido.

Regra — SoA não é "o layout rápido"

Ele troca **desperdício de linha** por **mais fluxos simultâneos**. O prefetcher acompanha um número limitado de *streams* — tipicamente 8 a 16 — e oito vetores percorridos em paralelo já disputam esse recurso. Para o AoS ganhar, a *struct* precisaria caber confortavelmente numa linha, o que torna

reordenar campos e cortar *padding* um pré-requisito, e não um detalhe.

O custo do SoA é legibilidade: some a entidade como objeto — não existe mais um `Particula&` para passar adiante, e funções que operavam sobre uma partícula passam a receber um índice e uma referência ao mundo inteiro. Os meios-termos, em ordem de esforço: **(1) separar por temperatura** — campos quentes numa *struct*, frios em outra, o que mantém a legibilidade e resolve a maior parte; **(2) AoSoA** — blocos de N em SoA dentro de um array (`struct Bloco { float x[8], y[8], z[8]; }`), que vetoriza bem e mantém localidade, e é o layout de *engines* de física e de código de alto desempenho; **(3) uma visão** sobre o SoA, um *proxy* que se parece com `Particula&`. E a regra que vale antes de todas: **só reorganize o layout depois que o *profiler* apontar aquele laço**.

XV.3.4 *False sharing*: 11,7× em código correto

Duas *threads*, duas variáveis **diferentes**, nenhum *mutex*, nenhum *data race*. E o programa fica quase doze vezes mais lento.

Por que — por que variáveis independentes brigam

Porque a unidade de coerência de cache não é a variável — é a **linha**. Os núcleos mantêm consistência entre seus caches por um protocolo (MESI) que opera em linhas inteiras. Se `a` e `b` caem na mesma linha de 64 bytes e o núcleo 0 escreve em `a`, o protocolo **invalida a linha** no núcleo 1 — que precisa buscá-la de novo para ler `b`. Os dois passam a trocar a mesma linha entre os caches, ida e volta, milhões de vezes.

```
1 contadores na MESMA linha : 336.32 ms
2 contadores SEPARADOS      :  28.72 ms   ← 11.7× mais rápido
```

Listagem 483: Saída de `src/performance/`: 4 threads × 20 milhões de incrementos, cada uma no SEU contador.

O código está correto nos dois casos. A única diferença entre eles é um `alignas(64)`:

```
1 struct Grudados {                          // 4 × 8 B = 32 B: tudo na mesma linha
2     std::atomic<long> c[kThreads]{};
3 };
4 struct Separados {
5     struct alignas(kLinha) Isolado {       // kLinha = 64
6         std::atomic<long> valor{0};
7     };
8     Isolado c[kThreads]{};
9 };
```

Listagem 484: As duas versões, de `src/performance/demo_false_sharing.hpp`.

As correções, em ordem: **(1) a melhor é não compartilhar** — acumule numa variável local da *thread* e escreva uma vez no fim; o `++local` vira um registrador, sem tocar em memória nenhuma; **(2)** `alignas(std::hardware_destructive_interference_size)`, quando o compartilhamento é inevitável — o custo aqui foi 32 para 256 bytes, irrelevante para quatro contadores e proibitivo para quatro milhões; **(3) *padding* manual**, se for preciso portabilidade estrita.

Armadilha — onde o *false sharing* se esconde

(a) Contadores por *thread* num array: `std::atomic<long> acessos[kThreads];`. **(b)** Uma *struct* com campos "independentes": `{ atomic<size_t> cabeca; atomic<size_t> cauda; }` — é exatamente por isso que a fila SPSC de Seção VI separa os dois índices com `alignas`. **(c)** Um `std::vector<bool>`

escrito por várias *threads* — aqui é *pior*: os bits compartilham bytes, então é *data race* de verdade, UB e não só lentidão. **(d)** Objetos pequenos alocados em sequência e distribuídos entre *threads*: o alocador os põe lado a lado. Para diagnosticar em vez de adivinhar: `perf c2c record` e `perf c2c report` mostram *quais* linhas estão pingando entre núcleos.

XV.3.5 Predição de desvio, e o experimento que não reproduz mais

Um processador moderno tem um *pipeline* de 15 a 20 estágios — começa a executar uma instrução muitos ciclos antes de terminar a anterior. Ao chegar num `if`, não pode esperar a condição: **aposta** num caminho e continua especulativamente. Se acerta, custo zero; se erra, o *pipeline* inteiro é descartado e recomeçado — 15 a 20 ciclos perdidos.

O experimento clássico soma condicionalmente 8 milhões de valores aleatórios, ordenados contra embaralhados: no vetor ordenado a condição muda uma única vez e o preditor acerta quase sempre; no embaralhado ela é aleatória, com ≈50% de erro.

```
nível  | ordenado | embaralhado | razão | cmov no binário
-- -- -- |-- -- -- -- -- |-- -- -- -- -- -- -- -- -|-- -- -- -|-- -- -- -- -- -- -- --
−O0   | 41.50 ms |    69.31 ms  | 1.7×  |   18
−O1   |  3.40 ms |     3.41 ms  | 1.0×  |   45
−O2   |  3.39 ms |     3.42 ms  | 1.0×  |   36
−O3   |  1.38 ms |     1.38 ms  | 1.0×  |   52
```

Listagem 485: Medido em `src/performance/`: `for (int x :  v) if (x >= 128) s += x;` em quatro níveis de otimização.

O efeito aparece em `-O0` e **desaparece a partir de** `-O1`. A causa está na última coluna: com otimização, o GCC 13 converte o `if` em `cmov` — *conditional move*, uma instrução que escolhe entre dois valores **sem desviar**. Não há aposta, logo não há como errar a aposta. Esta é a resposta mais votada de todo o Stack Overflow (*"Why is processing a sorted array faster than an unsorted array?"*, 2012) — e, no compilador de hoje com otimização ligada, **ela não reproduz mais**.

Regra — um benchmark medido em `-O0` não diz nada sobre o binário que você entrega

Duas lições, e a segunda vale para sempre: (1) o compilador moderno já elimina boa parte dos desvios imprevisíveis por conta própria; (2) medir no nível de otimização de **produção** não é detalhe de configuração, é a diferença entre um resultado e um artefato. O efeito de predição continua real — ele só não está mais *neste* laço; um desvio que o compilador não consegue converter, com chamada de função no corpo, continua pagando os ≈15 ciclos por erro.

A mesma medição resolve a tentação de reescrever `if` como aritmética:

```
(a) if (x >= 128) s += x;      3.42 ms
(b) s += x & −(x >= 128);      3.40 ms  (0.99×)
(c) s += x * (x >= 128);       3.35 ms  (0.98×)
```

Listagem 486: Saída de `src/performance/`: as três formas são a mesma coisa, porque o compilador já fazia.

Antes de trocar legibilidade por aritmética, confirme em vez de supor: `g++ -O2 -S -masm=intel arq.cpp -o - | grep -c cmov`. As dicas explícitas — `[[likely]]`, `[[unlikely]]`, `__builtin_expect` — ajudam quando você **sabe** a proporção; chutar errado piora, e o **PGO** (`-fprofile-generate` seguido de `-fprofile-use`) *mede* em vez de chutar.

Armadilha — quando não se preocupar com predição de desvio

Ela só importa quando **três** coisas coincidem: o desvio está num laço muito quente; a condição é genuinamente imprevisível; **e** o corpo é curto — se ele faz trabalho de verdade, os ≈15 ciclos desaparecem no meio. Desvios que o preditor acerta quase sempre: a condição de parada de um `for` (erra *uma* vez, no fim), a verificação de erro que quase nunca dispara, e qualquer desvio fora de um laço quente. Para medir: `perf stat -e branches,branch-misses ./prog` — abaixo de 1% é normal, acima de 5% num laço quente investigue, acima de 20% você achou.

XV.3.6 Como medir sem se enganar

Esta é a parte mais importante do subprojeto: as anteriores dão números, esta ensina a desconfiar deles. São cinco jeitos de produzir um número falso.

(1) O código foi eliminado. Se o resultado de um cálculo nunca é lido, o compilador tem licença para apagar o cálculo inteiro:

```
somando 10 M ints, resultado DESCARTADO :  0.000 ms
o mesmo, com ‘nao_otimize(s)‘            :  1.317 ms
→ o primeiro foi ELIMINADO: ~20000× "mais rápido" porque não aconteceu
```

Listagem 487: Saída de `src/performance/`: o número falso mais comum de todos.

A defesa é um **sumidouro** — uma construção que impede o compilador de provar que o valor é inútil. O bloco de *assembly* abaixo está vazio e não emite instrução nenhuma; o que ele faz é *informar* ao compilador que uma função leu `valor` e pode ter mexido em qualquer memória, e isso basta para que o cálculo não possa ser removido nem reordenado por cima dele. É o mesmo truque de `benchmark::DoNotOptimize` do Google Benchmark.

```
template <typename T>
inline void nao_otimize(T const& valor) {
    asm volatile("" : : "r,m"(valor) : "memory");
}

template <typename F>
double mediana_ms(F&& corpo, int repeticoes = 7) {
    std::vector<double> amostras;
    corpo();                                   // aquecimento, fora da conta
    for (int i = 0; i < repeticoes; ++i) {
        const auto t0 = std::chrono::steady_clock::now();
        corpo();
        const auto t1 = std::chrono::steady_clock::now();
        amostras.push_back(
            std::chrono::duration<double, std::milli>(t1 - t0).count());
    }
    std::sort(amostras.begin(), amostras.end());
    return amostras[amostras.size() / 2];      // MEDIANA, não média
}
```

Listagem 488: O sumidouro e o cronômetro, de `src/performance/bench.hpp`.

(2) A primeira execução paga a mais. Falhas de página — a primeira escrita numa página recém-alocada exige que o sistema operacional a mapeie de fato —, cache frio, TLB frio, carregamento de bibliotecas. Sempre **aqueça** antes de cronometrar.

(3) Uma amostra é ruído. Uma interrupção do sistema operacional no meio da medição distorce a **média** e não move a **mediana**. Use a mediana de 5 a 11 execuções — e o `std::steady_clock`, não o `system_clock`, que pode andar para trás quando o NTP ajusta a hora.

(4) O nível de otimização errado — é o caso da predição de desvio, 1,7× em `-O0` e 1,0× em `-O2`. **(5) O dado é amigável demais** — é o caso do passo 256 que cabia no L3.

Regra — *benchmark* ou *profiler*? Os dois, nesta ordem

O *benchmark* responde "a versão B é mais rápida que a A?"; o *profiler* responde "**onde** o programa gasta o tempo?". *Profiler* primeiro, para achar o trecho que importa; *benchmark* depois, para comparar alternativas naquele trecho. Otimizar sem *profiler* é o erro mais caro que existe — não porque a otimização não funcione, mas porque ela se aplica ao lugar errado: acelerar em 10× uma função que consome 2% do tempo melhora o total em **1,8%**.

```
perf stat ./prog                    # IPC, branch-misses, cache-misses
perf record -g ./prog               # -g = pilha de chamadas
perf report                         # navegável
perf annotate                       # linha a linha, com o assembly
perf stat -e LLC-load-misses ./prog
perf c2c record ./prog              # false sharing
```

Listagem 489: `perf`, o mínimo que vale saber.

O **IPC** (*instructions per cycle*) é o indicador mais informativo de uma linha só: abaixo de 1,0 significa que o processador está esperando memória; acima de 2,0 é bom.

Armadilha — compile com `-g` mesmo em *release*

Sem informação de depuração e sem `-fno-omit-frame-pointer`, o `perf report` mostra endereços em vez de nomes de função. `-g` não deixa o binário mais lento: só maior. É o que o `meson.build` deste subprojeto faz — ele é o único do repositório que **fixa** a otimização, porque o padrão do Meson vem do `buildtype` (*debug* implica `-O0`) e um *benchmark* sem otimização não diz nada sobre o binário que se entrega. Em WSL e em contêineres, o `perf` costuma exigir `sudo sysctl kernel.perf_event_paranoid=1`, e alguns eventos de hardware simplesmente não existem em máquina virtual.

Tabela 186: A ordem certa de otimizar: do que mais rende para o que menos rende.

Passo	O que é	Ganho típico
1. o algoritmo	$O(n \log n)$ no lugar de $O(n^2)$	ilimitado
2. fazer menos	cache de resultado, saída antecipada, lote	alto
3. o layout dos dados	é onde estão os 10× e 35× deste capítulo	alto
4. paralelizar	limitado pelos núcleos; nova classe de bugs	médio
5. micro-otimização	SIMD manual, código sem desvio	10–30%

E as *flags*, que são de graça: `-O2` (o padrão sensato); `-O3` (mais vetorização — **meça**, às vezes piora); `-march=native` (libera AVX2/AVX-512 da máquina local, e produz binário que **não roda** em CPU mais antiga); `-flto`; e o **PGO**, que é o ganho mais consistente (5 a 15%) e quase nunca usado.

Regra — meça, mude, meça de novo

Sem o primeiro passo você não sabe onde mexer; sem o terceiro, não sabe se melhorou. Todo o resto desta seção é consequência dessas duas frases.

XV.4 Síntese

As três seções tratam de assuntos diferentes e chegam à mesma conclusão, que vale nomear: **em C++, a ausência de sintoma não é evidência de correção**.

O teste que passa não prova que o código está certo — prova que aquele caminho, com aqueles valores, produziu aquele resultado; e 100% de cobertura não garante sequer que existam asserções. O *sanitizer* que não acusa não prova que não há UB — o *use-after-free* medido aqui **desaparece** em `-O1`, e *strict aliasing* não é pego por instrumentação nenhuma. E o *benchmark* que produz um número não produziu, com isso, um número certo — o experimento do passo de cache mediu L3 achando que media RAM, e o experimento mais citado do Stack Overflow não reproduz mais no compilador de hoje.

A resposta comum aos três é a mesma disciplina: **várias camadas independentes, e nenhuma confiança em uma só**. Testes mais cobertura mais *sanitizer* mais análise estática, porque cada um vê o que os outros não veem. Medição em vários níveis de otimização, porque cada nível esconde coisas diferentes. Mediana em vez de média, aquecimento antes do cronômetro, sumidouro em volta do resultado.

Regra — o que levar deste capítulo

Três frases, uma por seção. **Código difícil de testar é código mal desenhado** — o teste é o primeiro cliente a reclamar do seu projeto, e a injeção de dependência é o que separa uma classe testável em microssegundos de uma que exige um banco de dados. **UB não é um valor errado, é a remoção do código que você escreveu** — e a defesa mais barata, escrever código que não tem como ter UB, é também a mais eficaz. **O layout dos dados vem antes da instrução** — os fatores de 35× e 11,7× medidos aqui não vieram de trocar operação por operação, vieram de mudar a ordem em que a memória é tocada.

XVI Apêndice: trilhas, cobertura e o que falta

Os onze capítulos anteriores seguem uma ordem única, a da trilha completa. Este apêndice serve a quem não vai percorrê-la inteira: reúne as trilhas curtas por objetivo, o mapa de pré-requisitos que permite montar a sua, e o estado de cobertura da linguagem — o que já existe, o que ficou de fora e por quê.

Ele termina, porém, com a parte que mais rendeu conhecimento e que não estava prevista quando o relatório começou: o que aconteceu quando todos os alvos do repositório passaram a ser construídos ao mesmo tempo. A resposta ocupa a Seção XVI.5, e é, de longe, o material mais didático deste apêndice.

Aviso

Os números deste apêndice são os do estado atual do repositório, e mudaram desde a primeira redação: são **57 subprojetos** declarados em `src/meson.build` (mais dois com sistema de *build* próprio), **98 binários** instalados em `dist/bin`, **6 alvos de teste** registrados no `meson test` e **zero avisos** sob `-Wall -Wextra -Wpedantic`. Onde este texto diverge de uma edição anterior, vale este.

XVI.1 Atalhos por objetivo

Quem já sabe C++ e quer algo específico não precisa passar por tudo. As trilhas da Tabela 187 são as do `README.md` da raiz, traduzidas para as seções deste relatório e ampliadas com os subprojetos acrescentados depois da primeira redação.

Tabela 187: Sete trilhas mínimas, por objetivo. Entre parênteses, a seção correspondente deste relatório; os subprojetos marcados com † entraram no repositório depois dos capítulos e ainda não têm seção própria — o material deles está no `README.md` de cada um e, resumido, na Seção XVI.6.

Objetivo	Trilha mínima
passar em entrevista de C++	`casts` → `value_semantics` → `smart-pointers` → `oop` → `errors` → `stl` → `templates` → `concurrency` → `layout`† → `linkage`† (II.2, II.3, II.5, II.7, II.8, III.1, IV.1, VI.1)
escrever *backend* ou serviço	`casts` → `smart-pointers` → `errors` → `stl` → `solid` → `concurrency` → `coroutines` → `network` → `http-client` → `database` → `testing`† (II.2, II.5, II.8, III.1, V.1, VI.1, VI.3, VII.2, VII.3, VII.4)
jogos e simulação	`value_semantics` → `stl` → `memory` → `layout`† → `performance`† → `ecs` → `collision` → `sfml_bullet` → `sfml_bullet2` (II.3, III.1, III.5, V.3, VIII.1, VIII.2, VIII.3)
desempenho e otimização	`value_semantics` → `stl` → `memory` → `performance`† → `layout`† → `type_erasure`† → `concurrency` → `concurrency2`† → `inline_asm` → `outline_asm` → `asm_course` (II.3, III.1, III.5, VI.1, IX.1, IX.2, IX.3)
arquitetura e *code review*	`oop` → `solid` → `design-of-pattern` → `type_erasure`† → `ecs` → `plugin_registry` (II.7, V.1, V.2, V.3, V.4)
só o C++20/23 que falta	`ranges` → `concepts` → `coroutines` → `vocabulary_types` → `compile_time` → `operators` → `lambdas` → `strings`† → `chrono`† → `bits`† → `cpp23`† → `modules`† (III.2, IV.2, VI.3, III.4, IV.3, II.4, IV.4)
build, testes e qualidade	`meson` → `cmake`† → `testing`† → `undefined_behavior`† → `linkage`† → `doxygen` (XI.2, XI.6)

Regra — nenhuma trilha curta dispensa `value_semantics`

Quatro das sete trilhas começam por ele, e as demais o supõem. A razão é a da Figura 1: saber quando o compilador copia e quando move é pré-requisito para entender o custo de um container, o

comportamento de um *sink parameter*, a diferença entre `emplace` e `push`, e por que um destrutor vazio deixa o programa mais lento. É o único subprojeto que não dá para pular.

XVI.2 Onde cada conceito aparece

Vários conceitos reaparecem em capítulos distantes uns dos outros, e o índice remissivo os localiza por nome. A Tabela 188 faz o inverso: agrupa os que reaparecem *com papéis diferentes*, porque é aí que a leitura salteada perde o fio.

Tabela 188: Sete ideias que atravessam o relatório.

Ideia	Onde aparece, e em que papel
RAII	§II.5 como idioma; §II.8 como o que torna exceções seguras; §III.3 no `RedirectGuard`; §IV.4 como *scope guard*; §VI.1 no `ThreadGuard`; §V.4 no *deleter* do `.so`; §VII.6 para ordenar destruições; §X.2 no `PythonGuard`; e, fora do relatório, no `jthread` de `concurrency2`
type erasure	§III.4 em `std::any`; §IV.4 em `std::function`; §V.3 no `ComponentStore`; e reconstruído do zero, com o *small buffer* medido, em `type_erasure`
invólucro sobre API estrangeira	§VII.3 (libcurl); §VII.4 (SOCI); §VIII.2 (Bullet); §VIII.6 (JSBSim); §X.4 (ONNX Runtime); §VII.5 (AMQP-CPP)
erro como valor	§II.8 em `expected` e `error_code`; §III.3 na API dupla do `<filesystem>`; §VII.2 na de Asio; §VII.6 em `Outcome`
fold expression	§IV.1 como forma; §III.4 para percorrer `tuple`; §V.3 na *query*
CRTP	§II.7 como *mixin* sem *vtable*; §IV.1 por extenso; §III.2 em `view_interface`; e como linha de base do *benchmark* de despacho em `type_erasure`
inversão de dependência	§V.1 como princípio; §VII.4 em persistência; §V.4 até o `dlopen`; §XI.2 em `declare_dependency`

XVI.3 Estado de cobertura da linguagem

O `src/ROADMAP.md` rastreia o que a linguagem oferece contra o que o repositório já ensina. O repositório nasceu forte em ecossistema — SFML, Bullet, Qt, ONNX, JSBSim, AWS, Asio — e fraco em linguagem, e o `ROADMAP` existiu para fechar essa assimetria. **Ele está fechado**: os cinco *tiers* foram concluídos, e as tabelas a seguir registram o estado final.

Tabela 189: Tier 1 — fundamentos da linguagem. Todos concluídos.

Subprojeto	Seção	Estado
`syntax`	II.1	pronto
`casts`	II.2	pronto
`smart-pointers`	II.5	pronto
`templates`	IV.1	pronto
`concepts`	IV.2	pronto
`memory`	III.5	pronto
`value_semantics`	II.3	pronto
`oop`	II.7	pronto
`errors`	II.8	pronto
`stl`	III.1	pronto
`ranges`	III.2	pronto

Tabela 190: Tier 2 — C++ moderno. Os cinco subprojetos que a edição anterior listava como "planejados" foram concluídos, e um sexto (`file_io`) migrou para cá.

Subprojeto	Tópicos	Estado
`coroutines`	*generator*, *task*, internos, Asio	pronto (VI.3)
`vocabulary_types`	`optional`, `variant`, `any`, `tuple`, `span`	pronto (III.4)
`compile_time`	`constexpr`, `consteval`, NTTP	pronto (IV.3)
`operators`	sobrecarga, `<=>`, conversão, UDL	pronto (II.4)
`lambdas`	capturas, genéricas, `std::function`	pronto (IV.4)
`file_io`	*streams*, `sstream`, `<filesystem>`	pronto (III.3)
`strings`	SSO medido, `string_view`, `format`, `<charconv>`, `regex`	pronto
`chrono`	durações, relógios, calendário, como cronometrar	pronto
`bits`	operadores e UB, `<bit>`, `bitset`, *endianness*	pronto
`cpp23`	inventário *medido* por *feature-test macros*	pronto
`modules`	módulos nomeados e 2 *bugs* do GCC 13	pronto (fora do Meson)

Tabela 191: Tiers 3 a 5 — concorrência, modelo de objeto e arquitetura. Nenhuma linha continua "planejada".

Subprojeto	Tópicos	Estado
`concurrency`	*threads*, *mutex*, *future*, atômicos	pronto (VI.1)
`mandelbrot` / `_openmp`	paralelismo por *tiles*, OpenMP	pronto (VI.4)
`concurrency2`	`jthread`, `latch`, `barrier`, modelo de memória, *lock-free*	pronto
`type_erasure`	`std::function` por dentro, virtual *vs* CRTP *vs* `variant`	pronto
`layout`	alinhamento, *padding*, categorias de tipo, `[[no_unique_address]]`	pronto
`linkage`	ODR medida com 2 TUs, `extern "C"`, ABI, pimpl	pronto
`testing`	GoogleTest, *fixtures*, `TEST_P`, *gmock*, *death tests*	pronto
`undefined_behavior`	ASan/UBSan/TSan medidos, *strict aliasing*, *dangling*	pronto
`performance`	localidade de cache, *false sharing*, AoS *vs* SoA	pronto
`meson`	o sistema de *build* do repositório	pronto (XI.2)
`cmake`	CMake moderno, Protobuf, OpenMP, CTest	pronto (*build* próprio)
`solid`	os cinco princípios	pronto (V.1)
`design-of-pattern`	os 23 padrões GoF mais *null object*	pronto (V.2)
`ecs`, `plugin_registry`	composição e *plugins*	pronto (V.3, V.4)

XVI.3.1 Os padrões GoF: o catálogo fechou

A edição anterior deste apêndice registrava onze padrões implementados e doze pendentes. Os doze foram escritos. O catálogo do *Gang of Four* está completo — 23 padrões — mais o *null object*, que não é do livro original mas resolve o mesmo tipo de problema.

Tabela 192: Os 24 padrões implementados em `src/design-of-pattern/`, um diretório e um binário cada. Os treze acrescentados na última fase estão em **negrito**.

Categoria	Padrões
criacionais	*factory*, *singleton*, ***abstract factory***, ***builder***, ***prototype***
estruturais	*decorator*, ***adapter***, ***bridge***, ***composite***, ***facade***, ***flyweight***, ***proxy***
comportamentais	*chain*, *command*, *interpreter*, *iterator*, *observer*, *state*, *strategy*, ***mediator***, ***memento***, ***template method***, ***visitor***
fora do GoF	*null object*

Regra — todo padrão novo traz uma seção "versão moderna"

É exigência do `ROADMAP`, e a razão está na Tabela 72: vários padrões do GoF descrevem soluções para limitações de linguagens dos anos 1990, e em C++20 têm realização mais simples. O *visitor* — que era o mais evidente dos pendentes — foi escrito exatamente assim: a hierarquia clássica com `accept()` de um lado, e do outro `std::visit` sobre `std::variant` (Seção III.4), sem hierarquia, sem `accept()` e com exaustividade verificada pelo compilador. A seção também diz *quando não substitui*: `variant` fecha o conjunto de tipos, e um sistema de *plugins* precisa que ele fique aberto.

XVI.4 Os limites do *toolchain*

Parte do que não foi escrito não é falta de tempo: é falta de compilador. O GCC 13.3 implementa quase todo o C++20 e boa parte do C++23, e a Tabela 5 do Seção I.6 registra o que falta. Dois casos têm consequência direta sobre o conteúdo deste relatório:

- ***Deducing this*** (P0847, GCC 14) — é a forma limpa de escrever *lambda* recursiva e de deduplicar *overloads* `const`/não-`const`. Na sua ausência, o Seção IV.4 recomenda o *y-combinator*.
- **Módulos** — o subprojeto `modules` foi escrito, e o objetivo declarado incluía documentar *as limitações*: o suporte do GCC 13 a `-fmodules-ts` é parcial, `import std;` não existe, e a ordem de compilação passa a ser obrigatória. É por causa dessa última que `modules` tem `Makefile` próprio e não entra no Meson — e isso é conteúdo, não preguiça.

XVI.4.1 Dívida estrutural: as quatro pendências foram quitadas

O `ROADMAP` mantinha uma lista de dívidas do próprio repositório. A edição anterior deste apêndice registrava duas ainda abertas. Nenhuma continua.

A última linha merece explicação, porque a decisão é contraintuitiva: a dívida "os testes não rodam" foi resolvida *apagando* o diretório de testes.

Tabela 193: As quatro dívidas estruturais e como cada uma foi encerrada. Duas delas mudaram de estado *depois* da primeira redação deste relatório.

Dívida	Desfecho
`src/meson.build` com quase tudo comentado	**quitada por inteiro.** Não há mais nenhum `subdir()` comentado. Os 57 subprojetos são construídos, inclusive os "pesados" (Qt, SFML, Bullet, ONNX, JSBSim, AWS, RabbitMQ, SOCI)
`solid/{dip,isp,lsp,ocp}/meson.build` com 0 bytes	quitada: os quatro princípios foram implementados e habilitados
`srp/demo_pipeline.hpp` não compilava	quitada: faltava `#include <algorithm>`; ver Seção XVI.5
`tests/meson.build` 100% comentado	**quitada por remoção.** O `tests/` da raiz nunca construiu nada e foi apagado, junto com o `sandbox/` da raiz. Os testes passaram a morar onde são *ensinados*

Por que — por que apagar o `tests/` foi melhor do que consertá-lo

Um diretório `tests/` na raiz, inteiramente comentado, não é uma dívida de código: é uma *mentira de estrutura*. Ele anuncia ao leitor que existe uma suíte de testes, e não existe nenhuma. Pior: ele cria um lugar plausível para o teste seguinte, longe do código que o teste exercita.
Reativá-lo custaria escrever testes genéricos sobre subprojetos didáticos que não têm invariantes interessantes. Apagá-lo custou nada, e liberou a pergunta certa — *onde* deveriam morar os testes. A resposta é: junto do assunto. Os testes de verdade estão hoje em `src/testing/` (5 alvos, 41 casos), em `src/meson/` (1 alvo, para ensinar a função `test()` do Meson) e em `src/cmake/` (CTest, 11 casos, para ensinar o equivalente do outro lado). São 6 alvos no `meson test`, todos passando.

```
1 academy / testing_basico
2 academy / testing_fixture
3 academy / testing_parametrizado
4 academy / testing_death
5 academy / testing_mock
6 academy:guia / saudacao
```

Listagem 490: Os seis alvos registrados, obtidos com `meson test -C build -list`. Os cinco primeiros vêm de `src/testing/`; o sexto é o exemplo mínimo de `src/meson/`, que existe para ensinar a própria função `test()`.

XVI.5 Um alvo que não é construído é um alvo que apodrece

Esta é a seção mais importante do apêndice, e ela existe porque o repositório mudou de política no meio do caminho.

Por que — o problema, antes da solução

Um **alvo** (*target*), no vocabulário de um sistema de *build*, é uma unidade que o sistema sabe produzir: um executável, uma biblioteca, um arquivo gerado. Em Meson, cada `executable()` é um alvo, e cada linha `subdir('x')` manda o Meson ler o `meson.build` de `x/` e registrar os alvos que ele declara.
Comentar uma linha `subdir()` é, portanto, uma operação inocente na aparência: nada é apagado, o código continua no repositório, e o *build* fica mais rápido. Era exatamente esse o raciocínio da edição anterior deste relatório, que defendia manter comentados os alvos com dependências pesadas — um `make configure` que baixe e compile o AWS SDK inteiro afastaria quem só quer estudar semântica de valor.
O raciocínio parece sólido. O problema é que ele ignora uma assimetria: o custo de manter o alvo comentado é pago mais tarde, por outra pessoa, e cresce com o tempo. Enquanto isso, **o compilador**

é a única coisa que lê o código inteiro, todo dia, sem se cansar. Retirar um arquivo do alcance dele é retirar a única verificação automática que aquele arquivo tinha.

A política mudou para a oposta: *todos* os 57 subprojetos são construídos, sem exceção, e não há nenhum `subdir()` comentado em `src/meson.build`. Não é uma questão de gosto. Habilitar os alvos que estavam desligados revelou **nove defeitos reais**, e nenhum deles foi encontrado por leitura — todos apareceram no minuto seguinte a ligar o alvo.

Tabela 194: Os nove defeitos que estavam latentes em alvos desabilitados. Seis apareceram na compilação; três só na execução — e são os três mais instrutivos.

Onde	Quando apareceu	O que era
`solid/srp/demo_pipeline.hpp`	compilação	faltava `#include <algorithm>`
`concepts/demo_constraints.hpp`	compilação	faltava `#include <array>`
`file_io/demo_sstream.hpp`	compilação	faltava `#include <cmath>`
`aws/s3/main.cpp`	compilação	faltava `#include <fmt/core.h>`
`asm_course/07_calling_asm.s`	**execução**	ABI: alinhamento de 16 bytes quebrado antes do `call`
`asm_course/10_string_ops.cpp`	**execução**	*asm inline*: operandos de entrada que a instrução modifica
`http-client/main.cpp`	compilação	*multi-character constant* de 3 bytes, e subtração sem sinal
`rabbitmq/` `database/main.cpp`	compilação 2ª execução	a API do AMQP-CPP mudou de forma o demo não era idempotente

XVI.5.1 Os quatro cabeçalhos que não se bastavam

Quatro dos nove problemas são o mesmo problema, e é o mais barato de evitar: um cabeçalho que usa `std::sort` sem incluir `<algorithm>`.

A pergunta natural é: como isso compilava antes? Compilava porque *outro* cabeçalho, incluído antes, arrastava `<algorithm>` junto. A inclusão em C++ é textual — o pré-processador cola o conteúdo do arquivo no ponto do `#include` — e por isso um cabeçalho herda tudo o que os anteriores trouxeram. O `src/linkage/` mede esse arrasto (Tabela 196): um `#include <ranges>` traz 44 mil linhas.

Armadilha — um cabeçalho que compila hoje pode não compilar amanhã

A regra é a da **autossuficiência**: todo cabeçalho deve compilar sozinho, sem depender da ordem em que foi incluído. Se ele usa `std::sort`, inclui `<algorithm>`; se usa `std::array`, inclui `<array>`. Não importa que "já venha junto": a próxima versão da libstdc++ pode reorganizar os cabeçalhos internos e cortar o arrasto, e o código quebra sem que ninguém tenha tocado nele.
A verificação é mecânica: crie um `.cpp` que só faz `#include "seu_cabecalho.hpp"` e compile. É o que o alvo desabilitado deixava de fazer.

O caso do `aws/s3/main.cpp` é o mais eloquente, porque não é um cabeçalho interno: é o `main` de um programa. Ele chamava `fmt::print` e não incluía `<fmt/core.h>`. Esse arquivo não compilava desde que foi escrito, e ninguém sabia — porque nunca foi compilado.

XVI.5.2 O alinhamento de 16 bytes, e o crash que aparece no lugar errado

Este é o mais didático de todos, e precisa de contexto.

Por que — o que é uma ABI, e por que ela exige alinhamento

Uma ABI é o contrato binário entre funções compiladas separadamente: em que registrador vai cada argumento, onde volta o resultado, quem preserva o quê. Em Linux x86-64 esse contrato é a *System V AMD64 ABI* (Tabela 104).
Uma das cláusulas dela parece arbitrária e não é: **no instante em que a instrução `call` é executada, o registrador `RSP` (o topo da pilha) precisa ser múltiplo de 16**. A razão é o SSE. As instruções de carga alinhada — `movaps`, `movdqa` — exigem endereço múltiplo de 16 e disparam falha de proteção se receberem outro. O compilador emite essas instruções à vontade em `-O2`, colocando variáveis locais em posições calculadas *a partir de* `RSP`. Se `RSP` entrou torto, todo esse cálculo sai torto.
Como o `call` empilha 8 bytes (o endereço de retorno), a consequência aritmética é a que confunde todo mundo: ao *entrar* numa função, `RSP` $\equiv 8 \pmod{16}$, não 0.

A função `asm_apply` do `asm_course` fazia a conta ao contrário. O comentário original dizia "após `push rbp`, `rsp` está desalinhado por 8" — e ele estava, na verdade, alinhado.

```
1 entrada de asm_apply ..... RSP = 8  (mod 16)
2 pushq %rbp  ..............  RSP = 0  <- JA ALINHADO
3 subq  $8, %rsp  ..........  RSP = 8  <- e aqui QUEBRAMOS
4 call  *%rax  .............  o destino recebe RSP = 0  x ERRADO
```

Listagem 491: A aritmética do alinhamento, resumo de `src/asm_course/07_calling_asm.s`. Cada linha mostra o resto de `RSP` módulo 16 depois da instrução.

A correção foi apagar uma linha: o `pushq %rbp` já tinha alinhado, e o `subq $8, %rsp` desalinhava de novo.

O que torna o caso valioso não é o erro — é o *sintoma*. O programa não estourava em `asm_apply`. Estourava dentro do `fmt`, quatro quadros de pilha acima, numa função de formatação que estava perfeita.

```
1 #0  fmt::v10::detail::vformat_to<char>
2 #4  cpp_callback  (07_calling_conv.cpp:52)
3 #5  asm_apply     (07_calling_asm.s:124)   <- o culpado
```

Listagem 492: A pilha do `SIGSEGV`, como o depurador a mostrava. O culpado é o quadro `#5`; o crash acontece no `#0`.

Armadilha — o bug de alinhamento estoura em quem *usa* SSE, nunca em quem o quebrou

Quem desalinha a pilha não sofre nada: ele não faz cargas alinhadas. Quem sofre é a primeira função, na cadeia abaixo dele, que o compilador tenha otimizado com SSE — tipicamente uma biblioteca de terceiros, madura e testada, que parece a última suspeita possível.
O reflexo correto ao ver um `SIGSEGV` dentro de biblioteca conhecida, num programa que tem *assembly* escrito à mão ou `alignas` customizado, é subir a pilha até o primeiro quadro que *você* escreveu e conferir o `RSP`. É a marca registrada desta classe de erro.

XVI.5.3 A promessa que o `rep movsb` não cumpre

O segundo *bug* de execução está em `10_string_ops.cpp`, e ensina a parte do *asm inline* do GCC que mais gera erro silencioso.

Por que — por que *asm inline* tem uma lista de operandos

O GCC não entende o *assembly* que você escreve entre aspas. Ele o trata como texto opaco e o cola na saída. Para que o resto da função continue correto, você precisa *declarar* o que aquele texto faz: quais registradores lê (entradas), quais escreve (saídas) e quais destrói sem que o resultado interesse (*clobbers*).
Essa declaração é uma promessa. Um operando listado como *entrada* promete que o bloco **não altera** aquele registrador — e o GCC acredita: ele pode manter outro valor vivo ali e reutilizá-lo depois do bloco.

A instrução `rep movsb` copia `RCX` bytes de `[RSI]` para `[RDI]`. Ao fazê-lo, ela *modifica os três*: incrementa `RDI` e `RSI` a cada byte e zera `RCX`. A versão antiga declarava os três como entrada pura.

```
// ERRADO: promete que o bloco nao altera RDI/RSI/RCX -- e ele altera.
asm volatile (
    "std\n\t"
    "rep movsb\n\t"
    "cld"
    :                                                   // sem saidas
    : "D"(&rev[7]), "S"(&arr[7]), "c"((size_t)8)
    : "memory"
);

// CORRETO: "+" declara leitura-e-escrita, e exige variaveis de apoio.
void*       d = &rev[7];
const void* o = &arr[7];
std::size_t n = 8;

asm volatile (
    "std\n\t"                   // DF=1: decrementar apos cada byte
    "rep movsb\n\t"
    "cld"                       // OBRIGATORIO: restaura DF=0 antes de retornar
    : "+D"(d), "+S"(o), "+c"(n)   // <- o bloco os ALTERA
    :
    : "memory"
);
```

Listagem 493: A versão errada e a correta, de `src/asm_course/10_string_ops.cpp`. A diferença é o `+` — que promove os operandos a leitura-e-escrita.

O sintoma, de novo, apareceu no lugar errado: o `SIGSEGV` caía no laço de verificação *seguinte* ao bloco de *assembly*, porque era ali que o ponteiro corrompido era finalmente desreferenciado.

Armadilha — não se pode "resolver" listando o registrador no *clobber*

A tentação é declarar `RDI`, `RSI` e `RCX` na lista de *clobbers*. O GCC recusa: um registrador não pode ser simultaneamente operando e *clobber*. A saída é a do exemplo (`"+D"`, `"+S"`, `"+c"`) ou a forma equivalente com operandos amarrados, `"=D"(d), "=S"(s), "=c"(c)` nas saídas e `"0"`, `"1"`, `"2"` nas entradas.
Repare também no `cld` final: o `std` anterior ligou o sinalizador de direção, e a ABI exige que ele volte desligado antes do `ret`. Deixar `DF=1` corrompe a próxima função da biblioteca padrão que usar `rep`.

XVI.5.4 Três bytes que não cabem num `char`

O `http-client` guardava dois erros na mesma função de dez linhas, e os dois são erros de C++ puro — nada a ver com *assembly* ou com libcurl.

```
// ERRADO, duas vezes:
std::string linha(40 - title.size(), '—');    // com U+2500 no lugar do '—'

```

```
4  //  (a) '–' ocupa TRES bytes em UTF-8. Entre aspas SIMPLES vira uma
5  //      "multi-character constant": um int de valor definido pela
6  //      implementacao, truncado para char. O GCC avisa com -Wmultichar e
7  //      -Woverflow: "changes value from 14849152 to -128".
8  //  (b)  title.size() e std::size_t, SEM sinal. Com um titulo de 41 caracteres,
9  //       40 - 41 nao e -1: e 18446744073709551615.
10
11 // CORRETO:
12 const std::size_t largura = title.size() < 40 ? 40 - title.size() : 0;
13 std::string linha;
14 linha.reserve(largura * 3);                    // 3 bytes por caractere
15 for (std::size_t i = 0; i < largura; ++i) linha += "–";  // string, nao char
```

Listagem 494: Os dois *bugs* de `print_separator`, em `src/http-client/main.cpp`, e a correção.

Vale destrinchar os dois, porque cada um é uma família inteira de erros.

(a) Um `char` guarda um byte. A codificação UTF-8 representa caracteres fora do ASCII em dois a quatro bytes. O traço horizontal U+2500 usa três. Entre aspas simples, o compilador não vê "um caractere": vê uma sequência de três bytes numa constante de caractere, o que o padrão chama de *multi-character constant* — de tipo `int` e valor definido pela implementação. Convertida para `char`, ela é truncada. O construtor `std::string` recebia então um byte inválido, repetido 40 vezes.

(b) `size()` não tem sinal. `std::size_t` é um inteiro sem sinal, e a subtração de inteiros sem sinal não produz negativo: produz o resultado módulo 2^{64}. Com um título de 41 caracteres, `40 - 41` vale $1{,}8 \times 10^{19}$, e o `std::string` tentaria alocar 16 exabytes. O `src/bits/` desenvolve a família toda — promoção inteira, conversão e as comparações mistas.

Armadilha — a aritmética sem sinal dá a volta em silêncio

Nem `-Wall` nem `-Wextra` avisam sobre `a - b` quando os dois são `unsigned`: a operação é perfeitamente definida, e é justamente por isso que é perigosa. O compilador só avisa quando há *mistura* de sinal numa comparação (`-Wsign-compare`).
A defesa é estrutural: teste antes de subtrair, como na correção acima, ou converta explicitamente para um tipo com sinal largo o bastante. Nunca escreva `tamanho - 1` sem ter certeza de que `tamanho > 0`.

XVI.5.5 A API que mudou de forma embaixo do código

O `rabbitmq` não compilava porque a biblioteca AMQP-CPP mudou onde moram os eventos de ciclo de vida da conexão. A versão antiga do código fazia:

```
1  AMQP::TcpConnection connection(&handler, address);
2  connection.onConnected([]{ /* ... */ });          // nao existe
3  connection.onError([](const char* m){ /* ... */ });
4
5  // error: cannot convert 'main()::<lambda()>' to 'AMQP::TcpState*'
```

Listagem 495: A forma que não existe mais, e a mensagem de erro que ela produz — resumo de `src/rabbitmq/handler.hpp`.

A mensagem é enganosa porque `TcpConnection` *tem* métodos com esses nomes — só que são outra coisa, parte da máquina de estados interna. Os eventos do ciclo de vida pertencem ao **handler**, e a forma de reagir a eles é herdar e sobrescrever métodos virtuais.

Por que — dois estilos de *callback*, e como escolher

A confusão vale por si, porque a escolha entre os dois estilos aparece em toda API assíncrona:

- **Lambda por evento** — um *callback* independente por chamada. É o certo quando cada operação tem o seu próprio retorno: `declareQueue()` devolve um objeto ao qual se encadeia `.onSuccess(...)` e `.onError(...)`, e duas chamadas simultâneas têm dois retornos distintos.
- **Herança com métodos virtuais** — um conjunto *fixo e conhecido* de eventos, todos pertencentes ao mesmo objeto e compartilhando estado. É o certo para o ciclo de vida da conexão: os seis estágios são sempre os mesmos, e o *handler* precisa lembrar entre eles se já houve erro.

O Seção II.7 descreve o mecanismo da vtable / vptr; o `src/type_erasure/` descreve quando `std::function` substitui a herança — e quando não substitui. Este é um caso de "não substitui".

O conserto trouxe de brinde a documentação dos seis estágios, que é a origem da maioria dos relatos de "publiquei e a mensagem sumiu":

```
onAttached    objeto construido -- ainda nao ha socket
onConnected   TCP estabelecido -- ainda NAO ha login AMQP
onReady       handshake AMQP concluido   <- so AQUI da para publicar
onError       erro fatal ; a conexao vai fechar
onClosed      fechamento limpo
onDetached    pode destruir o objeto
```

Listagem 496: Os seis estágios do ciclo de vida de uma conexão AMQP, resumo de `src/rabbitmq/handler.hpp`. Publicar antes do `onReady` é cedo demais.

XVI.5.6 O demo que só funcionava na primeira execução

O último deles não é um erro de linguagem: é um erro de exemplo. O `database` criava a tabela com `CREATE TABLE IF NOT EXISTS` e inseria produtos. A coluna `name` tem restrição `UNIQUE`. Na segunda execução, as linhas da primeira ainda estavam lá, e o primeiro `INSERT` batia na restrição:

```
DbError: UNIQUE constraint failed: products.name
```

Listagem 497: O erro da segunda execução de `./dist/bin/database`, antes da correção. O programa saía com código 1.

A correção foi acrescentar `DROP TABLE IF EXISTS products` antes do `CREATE`, tornando o demo **idempotente** — rodável quantas vezes se queira, sempre com o mesmo resultado.

Armadilha — o `DROP` é correto no demo e desastroso em produção

Apagar a tabela apaga os dados. Em um sistema real, o instrumento equivalente é a **migração**: uma sequência versionada de scripts `ALTER TABLE`, cada um aplicado uma única vez e registrado numa tabela de controle (*Flyway*, *Liquibase*, *Alembic* fazem isso).
A distinção é a mesma de um teste: um demo, como um teste, precisa criar o próprio estado inicial e não depender do que a execução anterior deixou. Um exemplo didático que só funciona na primeira execução é um exemplo quebrado.

XVI.5.7 A lição, enunciada

Regra — um alvo que não é construído é um alvo que apodrece

Quatro `#include` ausentes, dois *bugs* de ABI, dois erros de C++ puro na mesma função de dez linhas, uma API obsoleta e um demo não idempotente — nove defeitos, todos residindo exatamente nos alvos que ninguém compilava. Nenhum deles foi encontrado por revisão de código; todos apareceram no minuto seguinte a habilitar o alvo.

O critério para desabilitar um alvo, portanto, nunca pode ser "ele dá trabalho". Se um subprojeto não pode ser construído, o problema é dele: conserte-o ou apague-o. Não o esconda atrás de um `#`.

E se o custo do *build* completo é mesmo alto, a solução é **opcional, não comentada**: uma `option()` do Meson com valor padrão, um *job* de CI que constrói tudo, uma etapa noturna. O que não pode existir é código que *nenhuma* configuração compila.

Aviso

Dois subprojetos continuam fora do `src/meson.build`, e por razão oposta: `src/modules/` usa `Makefile` próprio porque módulos do C++20 no GCC 13 exigem `-fmodules-ts` e ordem manual de compilação, e `src/cmake/` usa CMake porque o assunto *é* o CMake. Os dois documentam a escolha no próprio `README.md`, e os dois *são* construídos — por outro sistema. A regra não é "tudo no Meson"; é "tudo em algum *build* que alguém roda".

XVI.6 O que os subprojetos novos mediram

Os treze subprojetos acrescentados depois dos capítulos deste relatório compartilham um método: **toda alegação de custo ou de comportamento sai de um contador ou de um `<chrono>` no próprio demo**. Quando a medição contradiz o texto, o texto é reescrito — não a medição. Esta seção resume o que eles mediram, porque vários resultados mudam o que se deve escrever.

Tabela 195: Medições dos subprojetos novos, todas obtidas na máquina descrita na Tabela 4 (GCC 13.3). Os valores absolutos variam com o processador; a ordem de grandeza entre os casos comparados, não.

Subprojeto	Experimento	Resultado
`performance`	matriz 2048^2, por linha *vs* por coluna	35×
`performance`	soma sequencial *vs* aleatória (8 M `int`)	26×
`performance`	*false sharing*: mesma linha *vs* `alignas(64)`	11,7×
`performance`	`std::vector` *vs* `std::list` (2 M)	6,6×
`layout`	reordenar 4 membros da mesma *struct*	24 → 16 B
`layout`	`[[no_unique_address]]` num membro vazio	16 → 8 B
`layout`	um `virtual ~T()` que ninguém usa	+8 B/objeto
`linkage`	duas TUs incrementam `inline int` 10× cada	20 / 20
`linkage`	duas TUs incrementam `static int` 10× cada	10 / 10
`type_erasure`	*small buffer* da `std::function`	16 B *inline*
`type_erasure`	despacho sobre 200 000 elementos (base: CRTP)	`variant` 1,0×
`strings`	*string* de até 15 caracteres	0 alocações
`strings`	extrair um campo: `from_chars` *vs* `std::regex`	0 *vs* 133 ms
`concurrency2`	`seq_cst` *vs* `relaxed` sem contenção	1,01×
`concurrency2`	pilha *lock-free* *vs* `vector` sob *mutex*	2,7× *pior*

XVI.6.1 O custo de um `#include`, medido

O `src/linkage/` é o único subprojeto do repositório com múltiplas unidades de tradução, e não por capricho: o assunto é exatamente o que acontece quando o mesmo cabeçalho é compilado mais de uma vez.

Por que — o que é uma unidade de tradução

O C++ não compila "o projeto". Ele compila **uma unidade de tradução por vez**, em total ignorância das outras. Uma unidade de tradução (TU) é um `.cpp` mais tudo o que os `#include` dele trouxeram, recursivamente. Cada TU vira um `.o` com uma lista de símbolos *definidos* e outra de símbolos *necessários*; o *linker* cola tudo no fim.
Praticamente todo erro confuso de *build* — `undefined reference`, `multiple definition`, `undefined reference to vtable` — é consequência direta desse modelo.

Tabela 196: Linhas produzidas pelo pré-processador, GCC 13.3 com `-std=c++20`. Medido, não estimado: a primeira versão desta tabela era chutada e cinco dos seis números estavam errados — `<cstddef>` por um fator de 4.

Inclui	Linhas
arquivo vazio	6
`<cstddef>`	253
`<vector>`	26.258
`<algorithm>`	28.270
`<string>`	31.825
`<iostream>`	43.165
`<ranges>`	44.309

Cada TU paga esse custo *inteiro*, de novo. Um projeto com 100 arquivos `.cpp` que incluem `<string>` compila `<string>` cem vezes. É por isso que *build* de C++ é lento — e a medição é reprodutível em quatro linhas de *shell*:

```
for h in cstddef vector string ranges; do
  echo "#include <$h>" > /tmp/t.cpp
  printf '%-12s %s\n' "$h" "$(g++ -std=c++20 -E /tmp/t.cpp | wc -l)"
done
```

Listagem 498: O reprodutor da Tabela 196, do `README.md` de `src/linkage/`. Rode no seu *toolchain*: os números serão outros.

As saídas, em ordem de eficácia: **módulos** (C++20, `src/modules/`), **declaração adiantada** no cabeçalho e **pimpl**. Cabeçalhos pré-compilados e *unity builds* atacam o sintoma, não a causa.

XVI.6.2 O `static` em cabeçalho, medido com duas TUs

As duas linhas de `linkage` na Tabela 195 são o experimento central do subprojeto, e o resultado é o que separa entender *linkage* de decorar palavras-chave.

Duas unidades de tradução idênticas incluem o mesmo cabeçalho e incrementam a mesma variável dez vezes cada. Se a variável é `inline int`, as duas somam no *mesmo* objeto e o contador chega a 20 nas duas leituras. Se é `static int`, cada TU ganhou a *sua* variável, e cada uma lê 10.

Armadilha — o *bug* do `static` em cabeçalho não dá erro

Ele compila, linka e funciona — enquanto houver uma única TU incluindo o cabeçalho. Quebra no dia em que alguém acrescenta o segundo `.cpp`, e o sintoma ("o cache não persiste entre os módulos") não aponta para o cabeçalho.
É a classe de *bug* mais cara desta seção, porque a distância entre a causa e o efeito é máxima. A palavra-chave certa em cabeçalho é `inline`; `static` em escopo de arquivo significa *exatamente* "uma cópia por TU".

XVI.6.3 Os sanitizers, medidos em vez de citados

O `src/undefined_behavior/` tem dois alvos, e a distinção importa: o binário instalado *narra* o assunto e não executa UB (comportamento indefinido) nenhum; um segundo alvo, não instalado, contém doze casos de UB de propósito, um por argumento de linha de comando, para rodar sob *sanitizer*.

Nenhuma afirmação sobre o que os *sanitizers* pegam foi copiada de documentação. A Tabela 197 saiu de rodar esses doze casos, e três resultados contrariam o senso comum.

Tabela 197: Matriz de detecção, medida com `-fsanitize=address,undefined` no GCC 13.3. Os três casos em que o resultado surpreendeu estão em **negrito**.

Caso	`-O0`	`-O1`
heap overflow	ASan: *buffer-overflow*	idem
índice de pilha	UBSan: *index out of bounds*	idem
use-after-free	ASan: *heap-use-after-free*	**nada**
referência pendurada	UBSan: *null reference binding*	idem
overflow com sinal	UBSan: *signed integer overflow*	idem
deslocamento excessivo	UBSan: *shift exponent too large*	idem
divisão por zero	UBSan: *division by zero*	idem
vazamento	LSan: *detected memory leaks*	idem
strict aliasing	**nada**	**nada**
leitura de não inicializado	**nada**	**nada**
desalinhamento	**nada (GCC)**	**nada (GCC)**

As três surpresas:

1. **O *use-after-free* some com `-O1`.** O otimizador elimina a leitura que o ASan instrumentaria. Rodar o *sanitizer* só na configuração otimizada dá falsa segurança; rodar só em `-O0` dá falsa segurança sobre o binário que se entrega. A resposta é rodar nas duas.
2. **Desalinhamento não é pego pelo GCC**, nem com `-fsanitize=alignment` isolado. O *clang* pega. Se alinhamento importa no seu código — e a Seção XVI.5 mostra o que acontece quando importa —, esta checagem exige o outro compilador.
3. ***Strict aliasing* não é pego por nenhum dos dois.** É a categoria de UB sem rede de proteção; a defesa é `std::bit_cast` e `-fno-strict-aliasing` onde não houver alternativa.

Armadilha — *sanitizer* é análise dinâmica: só vê o que rodou

Um `catch` que os testes nunca exercitam pode conter UB e passar limpo por qualquer *sanitizer*. Por isso *sanitizer* e cobertura formam um par: um mede o que rodou, o outro diz o que *não* rodou.

E no CI, `-fno-sanitize-recover=all` é obrigatório: sem ele o UBSan apenas imprime e continua, o processo termina com código 0, e o *job* passa com o diagnóstico enterrado no log.

XVI.6.4 Quando a medição contrariou a expectativa

Esta é a parte que ensina método, e por isso está registrada em vez de silenciada. Doze vezes, ao longo dos subprojetos novos, o número contradisse o texto que já estava escrito. Nas doze, o texto foi reescrito.

Tabela 198: As contradições registradas no `src/ROADMAP.md`. Em todas, a explicação do resultado inesperado era mais interessante que a expectativa.

Onde	O que estava escrito	O que a medição mostrou
`performance`	o vetor ordenado é ∼3× mais rápido	**1,0**× em `-O1` ou acima — o GCC gera `cmov` e não há desvio para prever; o efeito só existe em `-O0`
`performance`	usando todos os campos, o AoS ganha	**o SoA ganhou 1,6**× — `sizeof` da partícula é 40, e 64/40 = 1: a linha de cache carrega uma partícula só
`performance`	a razão de tempo acompanha a de bytes lidos	bytes 10×, tempo **1,5**× — o *prefetcher* esconde a diferença
`concurrency2`	`seq_cst` é bem mais caro que `relaxed`	**1,01**× no x86-64 sem contenção — a otimização "que todo mundo faz" não pagou nada aqui, e pagaria num ARM
`linkage`	uma tabela de custo de `#include`	cinco dos seis números estavam errados; `<cstddef>` por um fator de 4
`cpp23`	seis *features* do C++23 ausentes no GCC 13	**as seis estão presentes** — `stacktrace`, `views::zip`, `fold_left`, `chunk`/`slide`, `move_only_function`, `spanstream`
`cmake`	OpenMP dá *speedup* no SAXPY	**0,32**× na primeira versão — a paralela saiu 3× mais LENTA; faltava aquecer o *pool* de *threads*

Duas dessas linhas merecem destaque porque a lição não é sobre o resultado, e sim sobre o *instrumento*.

O caso do `cpp23` é o mais confortável de todos: as seis correções foram na direção de **subestimar** o que já estava disponível. Havia uma tabela de suporte escrita de memória, e ela era pessimista. A substituição foi mecânica: um demo que consulta as *feature-test macros* (`__cpp_*` para a linguagem, `__cpp_lib_*` vindas de `<version>` para a biblioteca) e imprime o inventário. A tabela do `ROADMAP` passou a ser gerada do que o programa imprime.

O caso do `cmake` é o mais desconfortável, e passou por três versões antes de valer alguma coisa:

```
1a versao: speedup 0.32x  <- a paralela ficou 3x MAIS LENTA
           causa: o pool de threads do OpenMP e criado na 1a regiao paralela;
                  o custo da criacao entrou na medicao

2a versao: speedup 1.03x  <- praticamente nenhum ganho
           causa: o reset do vetor (80 MB) estava DENTRO da regiao cronometrada

3a versao: 3.09x -> 1.20x  <- conforme os dados saem do cache
           a QUEDA e a licao: o problema deixa de ser limitado por CPU e passa
           a ser limitado por largura de banda de memoria
```

Listagem 499: As três versões do *benchmark* de OpenMP em `src/cmake/`, e o que cada erro escondia. A terceira versão não tem "um" número — e é isso que a torna honesta.

Armadilha — as cinco maneiras de mentir para si mesmo num *benchmark*

O `src/chrono/` e o `src/performance/` catalogam as armadilhas que produziram os erros acima:

1. **Medir o que o compilador apagou.** Um laço cujo resultado é descartado é eliminado inteiro. Em `performance`, a razão medida foi ∞. A defesa é um sumidouro (*sink*) que force o uso do valor.
2. **Medir a primeira execução.** Cache frio, *pool* de *threads* inexistente, páginas ainda não mapeadas. Aqueça antes.
3. **Medir a preparação junto.** O `reset` de 80 MB dentro da região cronometrada, como acima.
4. **Medir uma vez.** Use a mediana de várias repetições; a média é sensível ao pico que o escalonador do sistema operacional causou.
5. **Medir em `-O0`.** Um *benchmark* sem otimização não diz nada sobre o binário que se entrega. O `performance` é o único subprojeto que *fixa* `-O2 -g` no próprio `meson.build`, exatamente por isso.

Armadilha — `high_resolution_clock` não é o relógio de medir

Medição do `src/chrono/` nesta `libstdc++`:

```
high_resolution_clock::is_steady = false
```

Ele *é* o `system_clock`. Um relógio não monotônico pode andar para trás quando o NTP ajusta a hora, e todo código que mede duração com ele está sujeito a resultado **negativo**. O relógio de medir intervalos é `std::chrono::steady_clock`, sempre. O `system_clock` serve para saber *que horas são*, não *quanto tempo passou*.

XVI.6.5 Três números de texto que mudam código

Por fim, o `src/strings/` mede três coisas que aparecem em quase todo programa e que quase ninguém mediu.

```
sizeof(std::string) = 32 bytes
 1 chars -> 0 alocacoes   <- cabe no SSO
15 chars -> 0 alocacoes   <- cabe no SSO
16 chars -> 1 alocacao

200 chars, um a um, sem reserve  ->  4 alocacoes

extrair um campo de log:
  find + from_chars    ->    0 ms
  std::regex           ->  133 ms      <- mesmo com a regex fora do laco
```

Listagem 500: Saída dos demos de `src/strings/`: o limite do *small string optimization* e o custo real de `std::regex`, ambos instrumentando `operator new`.

As três consequências práticas:

- **Strings curtas são quase de graça.** Até 15 caracteres nesta implementação, os bytes moram dentro do próprio objeto — não evite `std::string` em nomes e chaves por medo de alocação.

- **Mas o move de uma string curta *copia*.** Só é $O(1)$ acima do limite do SSO. É um dos poucos lugares em que "mover é barato" tem exceção.
- **`std::regex` é a última opção, não a primeira.** Duas ordens de grandeza contra `find` mais `from_chars`, *mesmo* com o objeto de expressão regular construído fora do laço. Para extrair um campo de posição conhecida, expressão regular é a ferramenta errada.

XVI.7 Encerramento

O relatório reuniu, numa ordem única, a teoria que cinquenta e sete subprojetos desenvolvem em separado. Vale terminar registrando o que essa reunião tornou visível — e que nenhum `README` isolado deixava ver.

O primeiro é a **recorrência dos mesmos poucos mecanismos**. RAII aparece em oito seções com oito nomes diferentes; *type erasure* sustenta `std::function`, `std::any` e o ECS; o invólucro sobre API estrangeira reaparece seis vezes, com seis bibliotecas que não se conhecem. Um repertório pequeno explica um domínio grande.

O segundo é que **quase toda decisão de projeto do C++ é uma troca explícita**, e o relatório é, em boa medida, um catálogo delas: *views* custam fragilidade e pagam em passadas evitadas; `span` custa tempo de vida e paga em zero *overhead*; CRTP custa a base comum e paga com *inline*; `variant` custa extensão por tipo e paga com exaustividade. Não há escolha gratuita, e conhecer o preço é o que separa usar a linguagem de brigar com ela.

O terceiro é o mais prático: **o compilador cala em quase tudo o que importa**. A Tabela 32 lista vinte erros que compilam sem aviso, e cada capítulo posterior acrescenta os seus. A defesa não é atenção — é ferramenta: `warning_level=3`, *sanitizers*, `-D_GLIBCXX_DEBUG`, `static_assert`, *concepts*, tipos fortes. Tudo o que transforma um erro de execução em erro de compilação paga por si mesmo na primeira vez que evita uma madrugada.

O quarto é o motivo de o repositório existir na forma em que existe: as tabelas mais convincentes deste relatório não são argumentos, são *saídas de programa*. O destrutor vazio que suprime os *moves*, os 140× entre `vector` e `list`, os mil entre lançar e não lançar, os 3,5 entre *template* e `std::function`, os 11,7 do *false sharing*, os 133 ms de `std::regex` — nenhum deles precisa ser acreditado. Basta compilar e rodar.

E o quinto foi aprendido depois dos outros quatro, ao ligar os 57 alvos de uma vez: **a ferramenta mais barata de garantia de qualidade que existe é compilar o código**. Não estática, não formal, não paga — apenas construir tudo, sempre, sem exceções escondidas atrás de um `#`. Foram nove defeitos encontrados em uma tarde, três deles capazes de derrubar o processo, todos residindo pacientemente nos únicos arquivos que o compilador não estava lendo. Um alvo que não é construído é um alvo que apodrece — e o único jeito de saber quanto ele já apodreceu é construí-lo.

Referências

[1] C++ Academy, "cpp_academy: subprojetos didáticos em C++ moderno," Repositório do projeto, `src/` e `src/ROADMAP.md`, 2026. Compilado com GCC 13.3, Meson e Conan 2.

[2] ISO/IEC, "ISO/IEC 14882:2020 — Programming languages — C++," `https://www.iso.org/standard/79358.html`, 2020. O padrão da linguagem; os rascunhos públicos estão em `https://eel.is/c++draft/`.

[3] cppreference.com, "C++ reference," `https://en.cppreference.com/w/cpp`, 2026. A referência de consulta diária citada ao longo do relatório.

[4] Stroustrup, B., and Sutter, H., "C++ Core Guidelines," `https://isocpp.github.io/CppCoreGuidelines/CppCoreGuidelines`, 2025. Origem das regras R., C. e F. referidas nas caixas "Regra".

[5] Itanium C++ ABI Committee, "Itanium C++ ABI," Tech. rep., `https://itanium-cxx-abi.github.io/cxx-abi/abi.html`, 2023. Define a decoração de nomes e o *layout* de vtable observados no subprojeto `src/oop/`.

[6] Martin, R. C., *Clean Architecture: A Craftsman's Guide to Software Structure and Design*, Prentice Hall, 2017. Formulação dos cinco princípios SOLID usada no subprojeto `src/solid/`.

[7] Gamma, E., Helm, R., Johnson, R., and Vlissides, J., *Design Patterns: Elements of Reusable Object-Oriented Software*, Addison-Wesley, 1994.

[8] Kohlhoff, C. M., "Asio C++ Library," `https://think-async.com/Asio/`, 2023.

[9] Guennebaud, G., Jacob, B., et al., "Eigen v3," `https://eigen.tuxfamily.org/`, 2021.

[10] Berndt, J. S., and JSBSim Development Team, "JSBSim: An Open Source Flight Dynamics Model," `https://jsbsim.sourceforge.net/`, 2023.

[11] Matz, M., Hubička, J., Jaeger, A., and Mitchell, M., "System V Application Binary Interface — AMD64 Architecture Processor Supplement," Tech. rep., `https://gitlab.com/x86-psABIs/x86-64-ABI`, 2024. A convenção de chamada usada no capítulo de baixo nível.

[12] Jakob, W., Rhinelander, J., and Moldovan, D., "pybind11: Seamless operability between C++11 and Python," `https://github.com/pybind/pybind11`, 2024.

[13] Levine, J., *flex & bison: Text Processing Tools*, O'Reilly Media, 2009.

[14] Wang, A., Chen, H., Liu, L., et al., "YOLOv10: Real-Time End-to-End Object Detection," `https://arxiv.org/abs/2405.14458`, 2024.

[15] ONNX Runtime developers, "ONNX Runtime," `https://onnxruntime.ai/`, 2024.

[16] van Heesch, D., "Doxygen: Generate documentation from source code," `https://www.doxygen.nl/`, 2025.

Glossário

ABI

Application Binary Interface: o contrato em nível de binário — convenção de chamada, *layout* de objetos, decoração de nomes. É o que decide se dois `.o` compilados separadamente podem ser ligados, e o que `extern "C"` e o ABI System V x86-64 fixam

categoria de valor

A classificação de uma *expressão* segundo duas perguntas independentes — tem identidade? pode ser movida? — que produzem *lvalue*, *xvalue* e *prvalue*. É do que depende a escolha entre copiar e mover, e a fonte da regra de que tudo o que tem nome é *lvalue*

Conan 2

O gerenciador de dependências do repositório. Lê o `conanfile.py`, resolve e compila as bibliotecas de terceiros e gera os arquivos que fazem o `dependency()` do Meson encontrá-las

concept

Predicado sobre tipos, avaliado em compilação, que restringe quais argumentos um *template* aceita. Move o erro do interior da instanciação para o ponto da chamada e participa da resolução de sobrecarga por *subsumption*

`constexpr` / `consteval`

`constexpr` marca o que *pode* ser avaliado em compilação; `consteval` marca o que *tem* de ser. A diferença entre "o compilador talvez calcule" e "o compilador calcula ou recusa o programa"

co-rotina (C++20)

Função que pode suspender a própria execução e ser retomada depois, preservando o estado local num *frame* alocado à parte. A linguagem fornece só o mecanismo (`co_await`, `co_yield`, `co_return`); os tipos `generator` e `task` são escritos pelo usuário ou pela biblioteca

CRTP

Curiously Recurring Template Pattern: a classe derivada passa a si mesma como argumento de *template* da base (`struct D : Base<D>`). Dá polimorfismo resolvido em compilação — sem *vtable*, sem indireção e com *inline* — ao preço de perder a base comum em tempo de execução

data race

Dois acessos concorrentes ao mesmo endereço, ao menos um deles de escrita, sem sincronização entre eles. É UB (comportamento indefinido) — não "valor errado ocasional". O detector prático é o ThreadSanitizer

elisão de cópia

Omissão da construção de um temporário: o objeto é construído diretamente no destino. Desde o C++17 é *obrigatória* para *prvalues* (RVO) e opcional para variáveis locais retornadas (NRVO). Escrever `return std::move(local)` desliga a segunda

invalidação de iteradores

A perda de validade de iteradores, ponteiros e referências após uma modificação do container. As regras variam por container e são a origem mais comum de UB (comportamento indefinido) em código que "funcionava"

iterador

A abstração que separa algoritmos de containers: um objeto que aponta para um elemento e sabe avançar. As categorias (*input*, *forward*, *bidirectional*, *random access*) definem que algoritmos podem operar sobre qual container e a que custo

Meson

O sistema de *build* deste repositório. Descreve o projeto numa DSL declarativa e gera arquivos do Ninja; a compilação em si é do Ninja. Todo subprojeto tem um `meson.build` próprio, registrado em `src/meson.build`

move

Transferência do conteúdo de um objeto para outro, deixando a origem num estado válido porém não especificado. `std::move` não move nada: é um `static_cast` para *rvalue reference* que apenas *autoriza* a operação, executada pelo construtor ou pela atribuição de *move* do tipo

object slicing

Cópia de um objeto derivado para uma variável do tipo base: o que não cabe em `sizeof(Base)` é amputado, e com ele o polimorfismo. Nenhum *warning* é emitido. Passar hierarquia por valor e usar `std::vector<Base>` são as duas formas usuais de cometê-lo

operação atômica

Operação indivisível sobre uma variável compartilhada, com uma ordem de memória associada. Resolve a corrida sobre *aquela* variável; não estabelece invariantes entre duas variáveis — para isso ainda é preciso um *mutex*

perfect forwarding

Repasse de um argumento preservando tipo, constância e categoria de valor. Exige as duas peças juntas: o parâmetro `T&&` deduzido (referência universal) e o `std::forward<T>` na hora de repassar

`std::pmr`

Polymorphic Memory Resources (C++17). Permite trocar a estratégia de alocação de um container em tempo de execução, sem que a escolha entre no tipo — ao contrário do parâmetro `Allocator` clássico, que faz `vector<int,A>` e `vector<int,B>` serem tipos distintos

RAII

Resource Acquisition Is Initialization. O idioma central do C++: o tempo de vida de um recurso (memória, arquivo, *lock*, *socket*) é amarrado ao tempo de vida de um objeto. Adquirir é construir; liberar é destruir. Como o destrutor roda também durante o desenrolamento da pilha, o idioma é o que torna exceções seguras

RTTI

Run-Time Type Information. O mecanismo que sustenta `typeid` e `dynamic_cast`. Exige pelo menos uma função virtual na base e pode ser desligado com `-fno-rtti`

SFINAE

Substitution Failure Is Not An Error. Se a substituição dos parâmetros de *template* produz um tipo inválido na assinatura, o candidato é apenas descartado da resolução de sobrecarga em vez de gerar erro. Era a forma pré-C++20 de restringir *templates*; *concepts* a substitui com mensagens legíveis

type erasure

Guardar objetos de tipos não relacionados atrás de uma interface uniforme, apagando o tipo estático mas preservando o comportamento. `std::function`, `std::any` e o `std::type_index` do ECS deste repositório são três realizações diferentes da mesma ideia

UB (comportamento indefinido)

Situação em que o padrão não impõe nenhuma exigência ao programa. Não é "dá erro": é o compilador autorizado a supor que aquilo nunca acontece, e a otimizar a partir dessa suposição. Ler variável não inicializada, estourar um índice, violar *strict aliasing* e escrever num objeto genuinamente `const` são casos deste relatório

***view* (C++20)**

Um *range* que não possui os elementos e cujas operações de cópia e destruição são O(1). *Views* são preguiçosas: compor `filter` com `transform` não percorre nada até que alguém itere o resultado

vtable / vptr

A implementação usual do despacho dinâmico: cada classe polimórfica tem uma tabela de ponteiros de função (*vtable*) e cada objeto guarda um ponteiro para a tabela da sua classe real (*vptr*). O custo é um ponteiro por objeto e uma indireção por chamada — mas o preço maior costuma ser a impossibilidade de *inline*

Índice remissivo

cabeçalhos
    <algorithm>, 65, 71
    <array>, 72
    <bit>, 213, 224, 299
    <charconv>, 287, 288, 305
    <chrono>, 290, 306
    <cmath>, 71, 276
    <codecvt>, 282
    <coroutine>, 62, 78, 79
    <cstddef>, 325
    <filesystem>, 47, 51, 71
    <fmt/core.h>, 223
    <fmt/ranges.h>, 285
    <fstream>, 304
    <iomanip>, 304
    <ranges>, 67, 262, 273
    <string>, 275, 277, 325
    <type_traits>, 266
    <version>, 271
const_cast, 23, 321

dynamic_cast, 28

explicit operator bool, 36

operator<=>, 35

reinterpret_cast, 28, 316

static_cast, 25
std::align, 312
std::align_val_t, 312
std::any, 5, 74, 259, 321
std::array, 66, 261, 264, 349
std::async, 131
std::atomic, 149
std::atomic<bool>, 137
std::atomic<int>, 139
std::atomic<T>, 140
std::bad_cast, 28
std::barrier, 149
std::bind_front, 100
std::bit_cast, 28, 313, 348, 349
std::bitset, 300, 302
std::byte, 348
std::call_once, 140
std::chrono::steady_clock, 5
std::complex, 146
std::complex<double>, 146
std::condition_variable, 135
std::deque, 115
std::error_code, 58
std::expected, 175, 256
std::filesystem, 5, 70, 169, 303, 307
std::format, 265, 349
std::forward, 29
std::function, 7, 9, 88, 108, 115, 118, 119, 274, 309, 319, 321, 367, 368
std::future, 84, 145
std::invoke, 100
std::jthread, 130, 134
std::latch, 149
std::list, 62, 210, 351, 368
std::lock_guard, 130
std::logic_error, 58
std::map, 64, 161, 270
std::max_align_t, 311
std::monostate, 74, 258
std::move, 29
std::move_only_function, 319, 321
std::mutex, 141
std::optional, 5, 72, 120, 255, 349
std::packaged_task, 131
std::promise, 131
std::regex, 5, 288, 289, 332, 368, 373
std::runtime_error, 58
std::scoped_lock, 130, 137
std::shared_mutex, 130
std::shared_ptr, 39, 109, 115, 119, 138
std::size_t, 19
std::span, 5, 63, 85, 156, 261, 349
std::stacktrace, 6, 13
std::steady_clock, 356
std::string, 5, 19, 115, 123, 163, 263, 268, 281–286, 288, 307, 311, 313, 318, 320, 330, 349, 366, 372
std::string_view, 68, 85, 262, 269, 283, 305, 307
std::terminate, 57
std::thread, 129, 133
std::to_address, 69
std::tuple, 5, 260
std::type_index, 121, 321
std::unique_lock, 130
std::unique_ptr, 39, 112, 113, 180, 185, 186, 190, 274, 314, 331, 349
std::unordered_map, 161, 332, 335
std::variant, 5, 73, 110, 257, 318, 349
std::vector, 39, 115, 167, 210, 261, 263, 281, 301,

349, 368
std::visit, 118
std::weak_ptr, 39
subprojetos
asm_course, 203, 208
aws, 174
bits, 213, 224, 280, 296, 366
calculus, 195, 197, 202
casts, 27
chrono, 280, 372
cmake, 232, 239, 244, 249, 251, 253, 362, 368
collision, 178, 182, 202
compile_time, 88, 94, 254
concepts, 88, 91
concurrency, 129
concurrency2, 129, 130, 133, 139, 149
coroutines, 78, 129, 143
cpp23, 79, 254, 271
database, 164, 238
design-of-pattern, 52, 102, 108
doxygen, 232, 251
ecs, 52, 102, 120
embed-python, 227
errors, 56, 60
file_io, 23, 45, 52, 70, 280, 303
flight-simulator, 200
http-client, 160, 213
inline_asm, 203
lambdas, 88, 97
layout, 217, 309
linkage, 309, 332, 363, 369
mandelbrot, 129, 147
mandelbrot_openmp, 129, 148
memory, 76
meson, 232, 239, 362
modules, 254, 278, 368, 369
network, 152
oop, 52, 60
operators, 23, 33
outline_asm, 203, 206
parser, 120, 228, 232, 250
performance, 142, 334, 372
plugin_registry, 52, 102, 123
python, 225
qt_sfml, 190, 191, 193
qt_sfml2, 190, 192, 193, 195
rabbitmq, 169
ranges, 67
sandbox, 232, 252
sfml_bullet, 182, 189, 192, 202
sfml_bullet2, 187–189, 202
smart-pointers, 23, 38
solid, 102
stl, 62
strings, 115, 280, 372
syntax, 23, 27
templates, 88
testing, 334, 362
type_erasure, 309, 332, 367
undefined_behavior, 334, 370
value_semantics, 23, 29, 60
vocabulary_types, 72, 81, 254
yolo, 229